

George Marsicano

# Requisitos de Software



Comunicação é tudo!

Versão: 2.1, Draft.

28/09/2026

# Sumário

|                                                                                                             |           |
|-------------------------------------------------------------------------------------------------------------|-----------|
| <b>Introdução.....</b>                                                                                      | <b>8</b>  |
| <b>Capítulo 1.....</b>                                                                                      | <b>11</b> |
| <b>1 Breve histórico da Engenharia de Software (ESW) e Engenharia de Requisitos (ER) .....</b>              | <b>11</b> |
| 1.1 Década de 1970: O Surgimento da Engenharia de Software .....                                            | 12        |
| 1.2 Década de 1980: Reconhecimento da Engenharia de Requisitos .....                                        | 13        |
| 1.3 Década de 1990: Consolidação e Primeira Conferência da ER.....                                          | 15        |
| 1.4 Década de 2000: Consolidação do Ágil e Novas Ferramentas de Engenharia de Requisitos .....              | 17        |
| 1.5 Década de 2010: Integração Profunda com Práticas Ágeis e Foco na Experiência do Usuário .....           | 18        |
| 1.6 Década de 2020: Avanços Tecnológicos e Enfrentamento de Novos Desafios .....                            | 20        |
| <b>CAPÍTULO 1: PONTOS-CHAVE.....</b>                                                                        | <b>26</b> |
| <b>Capítulo 2.....</b>                                                                                      | <b>27</b> |
| <b>2 Conceitos .....</b>                                                                                    | <b>27</b> |
| 2.1 Abordagens de Desenvolvimento de Software .....                                                         | 27        |
| 2.2 Ciclos de Vida de Desenvolvimento de Software .....                                                     | 28        |
| 2.3 Processos de Desenvolvimento de Software .....                                                          | 28        |
| 2.4 Frameworks de Gerenciamento.....                                                                        | 28        |
| 2.5 Métodos e Ferramentas .....                                                                             | 29        |
| 2.6 Relacionamento entre os Conceitos .....                                                                 | 29        |
| 2.7 Elementos Contextuais de Adaptabilidade no Desenvolvimento de Software.....                             | 31        |
| 2.7.1 Particularidades de Cada Projeto .....                                                                | 31        |
| 2.7.2 Domínio de Aplicação.....                                                                             | 32        |
| 2.7.3 Cultura Organizacional .....                                                                          | 32        |
| <b>CAPÍTULO 2: PONTOS-CHAVE.....</b>                                                                        | <b>33</b> |
| <b>Capítulo 3.....</b>                                                                                      | <b>35</b> |
| <b>3 Abordagens, Ciclos de Vida e Processos de Desenvolvimento de Software .....</b>                        | <b>35</b> |
| 3.1 Abordagens de Desenvolvimento de Software .....                                                         | 35        |
| 3.1.1 Dirigidas por Plano.....                                                                              | 35        |
| 3.1.2 Ágeis .....                                                                                           | 37        |
| 3.1.3 Dirigido por Plano e Ágeis .....                                                                      | 38        |
| 3.1.4 Abordagens Híbridas: Combinando o Melhor de Dois Mundos.....                                          | 40        |
| 3.1.5 Como lidam com requisitos de software: Dirigido por Plano, Ágeis e Híbridos.....                      | 41        |
| 3.2 Ciclos de Vida .....                                                                                    | 45        |
| 3.2.1 Ciclo de vida de Software.....                                                                        | 46        |
| 3.3 Processos de Desenvolvimento de Software e Frameworks .....                                             | 51        |
| 3.3.1 Processos Abrangentes .....                                                                           | 52        |
| 3.3.2 Processos com Foco em Práticas Técnicas Específicas .....                                             | 57        |
| 3.3.3 Processos Híbridos/Adaptativos .....                                                                  | 59        |
| 3.3.4 Frameworks de Gerenciamento Ágil .....                                                                | 62        |
| 3.3.5 Frameworks de Escalabilidade.....                                                                     | 64        |
| 3.4 Relação entre Abordagens, Ciclos de Vida, Processos e Frameworks .....                                  | 67        |
| <b>CAPÍTULO 3: PONTOS-CHAVE.....</b>                                                                        | <b>68</b> |
| <b>Capítulo 4.....</b>                                                                                      | <b>69</b> |
| <b>4 Como Processos de Desenvolvimento de Software e Frameworks Lidam com Requisitos de Software? .....</b> | <b>69</b> |
| 4.1 Cascata.....                                                                                            | 70        |

|       |                                                                                                         |            |
|-------|---------------------------------------------------------------------------------------------------------|------------|
| 4.2   | Modelo V.....                                                                                           | 72         |
| 4.3   | Unified Process (UP).....                                                                               | 74         |
| 4.4   | OpenUP.....                                                                                             | 75         |
| 4.5   | Agile Unified Process (AUP).....                                                                        | 77         |
| 4.6   | Feature-Driven Development (FDD).....                                                                   | 81         |
| 4.7   | Espiral.....                                                                                            | 82         |
| 4.8   | Dynamic Systems Development Method (DSDM).....                                                          | 84         |
| 4.9   | Rapid Application Development (RAD).....                                                                | 86         |
| 4.10  | Scrum.....                                                                                              | 87         |
| 4.11  | Kanban.....                                                                                             | 89         |
| 4.12  | Scaled Agile Framework (SAFe).....                                                                      | 91         |
| 4.13  | Large-Scale Scrum (LeSS).....                                                                           | 93         |
| 4.14  | Quadro Resumo: Como os Processos de Desenvolvimento de Software e Frameworks Lidam com Requisitos?..... | 95         |
|       | <b>CAPÍTULO 4: PONTOS-CHAVE.....</b>                                                                    | <b>100</b> |
|       | <b>Capítulo 5.....</b>                                                                                  | <b>102</b> |
| 5     | <b>A Engenharia de Requisitos.....</b>                                                                  | <b>102</b> |
| 5.1   | Considerações Iniciais sobre as Atividades da ER.....                                                   | 105        |
| 5.2   | Escolhas Terminológicas.....                                                                            | 106        |
| 5.3   | Atividades da Engenharia de Requisitos.....                                                             | 107        |
| 5.3.1 | Elicitação e Descoberta.....                                                                            | 107        |
| 5.3.2 | Análise e Consenso.....                                                                                 | 107        |
| 5.3.3 | Declaração.....                                                                                         | 108        |
| 5.3.4 | Representação.....                                                                                      | 108        |
| 5.3.5 | Verificação e Validação.....                                                                            | 109        |
| 5.3.6 | Organização e Atualização.....                                                                          | 109        |
| 5.3.7 | Síntese das atividades.....                                                                             | 109        |
| 5.4   | Valores e Princípios da Engenharia de Requisitos.....                                                   | 112        |
| 5.4.1 | Os 7 Valores da ER.....                                                                                 | 112        |
| 5.4.2 | Os 11 Princípios da ER.....                                                                             | 114        |
| 5.5   | A dimensão humana, social e sociotécnica da Engenharia de Requisitos.....                               | 117        |
| 5.5.1 | ER como prática sociotécnica.....                                                                       | 118        |
| 5.5.2 | Comunicação, mediação e construção de entendimento.....                                                 | 119        |
| 5.5.3 | Stakeholders, contextos de uso, poder e negociação.....                                                 | 120        |
| 5.5.4 | Software como intervenção social: impactos pretendidos e efeitos emergentes.....                        | 121        |
| 5.5.5 | Implicações para a prática da Engenharia de Requisitos.....                                             | 123        |
|       | <b>CAPÍTULO 5: PONTOS-CHAVE.....</b>                                                                    | <b>125</b> |
|       | <b>Capítulo 6.....</b>                                                                                  | <b>128</b> |
| 6     | <b>Introdução aos Requisitos de Software.....</b>                                                       | <b>128</b> |
| 6.1   | Conceituação de Requisitos.....                                                                         | 128        |
| 6.2   | Tipos de Requisitos: Funcionais e Não Funcionais.....                                                   | 130        |
| 6.2.1 | Requisitos Funcionais (RFs).....                                                                        | 130        |
| 6.2.2 | Requisitos Não Funcionais (RNFs).....                                                                   | 131        |
| 6.3   | Regras de Negócio (RNs).....                                                                            | 135        |
| 6.3.1 | Complementaridade e Relacionamento entre Regras de Negócio, Requisitos Funcionais e Não Funcionais..... | 136        |
|       | <b>CAPÍTULO 6: PONTOS-CHAVE.....</b>                                                                    | <b>138</b> |
|       | <b>Capítulo 7.....</b>                                                                                  | <b>139</b> |
| 7     | <b>Elicitação e Descoberta de Requisitos.....</b>                                                       | <b>139</b> |
| 7.1   | Introdução.....                                                                                         | 139        |
| 7.2   | A importância da atividade de elicitação e descoberta.....                                              | 140        |

|                                      |                                                                                                       |            |
|--------------------------------------|-------------------------------------------------------------------------------------------------------|------------|
| 7.3                                  | Fontes de informação .....                                                                            | 140        |
| 7.4                                  | Elementos fundamentais da elicitação e descoberta.....                                                | 143        |
| 7.5                                  | Necessidades, desejos e frustrações.....                                                              | 145        |
| 7.6                                  | Descoberta de requisitos funcionais e não funcionais .....                                            | 147        |
| 7.7                                  | Papéis e competências na elicitação e descoberta .....                                                | 149        |
| 7.8                                  | Dificuldades comuns na elicitação e descoberta .....                                                  | 151        |
| 7.8.1                                | Barreiras de comunicação, resistência à mudança e excesso ou falta de requisitos .....                | 151        |
| 7.8.2                                | Distorções sociais e políticas no processo de elicitação .....                                        | 152        |
| 7.8.3                                | Reflexões.....                                                                                        | 152        |
| <b>CAPÍTULO 7: PONTOS-CHAVE.....</b> |                                                                                                       | <b>153</b> |
| <b>Capítulo 8.....</b>               |                                                                                                       | <b>155</b> |
| 8                                    | Análise e Consenso de Requisitos.....                                                                 | 155        |
| 8.1                                  | Introdução .....                                                                                      | 155        |
| 8.2                                  | Natureza dos Requisitos Brutos.....                                                                   | 156        |
| 8.3                                  | Objetivos da Análise e Consenso.....                                                                  | 159        |
| 8.3.1                                | Remover Ambiguidades e Preencher Lacunas .....                                                        | 159        |
| 8.3.2                                | Resolver Conflitos e Inconsistências .....                                                            | 160        |
| 8.3.3                                | Estabelecer Entendimento Compartilhado .....                                                          | 160        |
| 8.3.4                                | Definir Prioridades e Escopo Acordado.....                                                            | 161        |
| 8.3.5                                | Analisar Viabilidade, Dependências, Riscos e Implicações .....                                        | 161        |
| 8.4                                  | Técnicas de Apoio à Análise e ao Consenso.....                                                        | 162        |
| 8.4.1                                | Workshops de Requisitos.....                                                                          | 162        |
| 8.4.2                                | Técnicas de Priorização (MoSCoW e Matriz Avaliação Técnica × Valor de Negócio) .....                  | 163        |
| 8.4.3                                | Negociação.....                                                                                       | 163        |
| 8.4.4                                | Resolução de Conflitos .....                                                                          | 164        |
| 8.4.5                                | Prompt IA .....                                                                                       | 164        |
| 8.5                                  | Colaboração e Dinâmica Humana no Consenso.....                                                        | 165        |
| 8.6                                  | Riscos e Armadilhas na Análise e Consenso.....                                                        | 167        |
| <b>CAPÍTULO 8: PONTOS-CHAVE.....</b> |                                                                                                       | <b>169</b> |
| <b>Capítulo 9.....</b>               |                                                                                                       | <b>170</b> |
| 9                                    | Declaração de Requisitos .....                                                                        | 170        |
| 9.1                                  | Introdução .....                                                                                      | 170        |
| 9.2                                  | Objetivos da Declaração de um Requisito.....                                                          | 171        |
| 9.3                                  | Níveis de Abstração de um Requisito .....                                                             | 172        |
| 9.3.1                                | Requisitos de Negócio.....                                                                            | 174        |
| 9.3.2                                | Requisitos de Usuário.....                                                                            | 174        |
| 9.3.3                                | Requisitos de Produto .....                                                                           | 175        |
| 9.3.4                                | Síntese dos Níveis de Abstração.....                                                                  | 176        |
| 9.4                                  | Tipos de declarações .....                                                                            | 176        |
| 9.4.1                                | Narrativas Descritivas.....                                                                           | 177        |
| 9.4.2                                | Declarações Estruturadas .....                                                                        | 180        |
| 9.4.3                                | Declarações orientadas a valor .....                                                                  | 185        |
| 9.4.4                                | Catálogos e artefatos técnicos .....                                                                  | 187        |
| 9.5                                  | Relação entre níveis de abstração de requisitos e tipos de técnicas de declaração de requisitos ..... | 191        |
| 9.6                                  | Fatores que influenciam a forma de declaração de requisitos.....                                      | 193        |
| 9.6.1                                | Abordagem de Desenvolvimento .....                                                                    | 193        |
| 9.6.2                                | Estrutura de organização em que o requisito será inserido .....                                       | 194        |
| 9.6.3                                | Tipo de produto a ser construído .....                                                                | 196        |
| 9.6.4                                | Foco do requisito.....                                                                                | 197        |
| 9.6.5                                | Proximidade com o cliente .....                                                                       | 198        |
| 9.6.6                                | Perfil do cliente .....                                                                               | 199        |
| 9.6.7                                | Características da equipe técnica .....                                                               | 201        |



|                                |                                                                                   |     |
|--------------------------------|-----------------------------------------------------------------------------------|-----|
| 9.6.8                          | Momento do produto e necessidade de uso.....                                      | 202 |
| 9.6.9                          | Resumo dos fatores de influência na declaração de requisitos.....                 | 203 |
| 9.7                            | Escolher é equilibrar: adequação e qualidade das declarações.....                 | 204 |
| 9.7.1                          | A escolha é um ato de mediação .....                                              | 205 |
| 9.7.2                          | Adequação depende de propósito, não de formato .....                              | 205 |
| 9.7.3                          | Combinar é mais realista do que escolher .....                                    | 205 |
| 9.7.4                          | A qualidade está na função comunicacional.....                                    | 206 |
| 9.7.5                          | O papel reflexivo do engenheiro de software.....                                  | 206 |
| 9.8                            | Integração e evolução das declarações .....                                       | 207 |
| 9.8.1                          | Complementaridade entre técnicas.....                                             | 207 |
| 9.8.2                          | Evolução temporal e revisões sucessivas das declarações.....                      | 208 |
| 9.8.3                          | Mudanças e continuidade de sentido nas declarações .....                          | 211 |
| 9.8.4                          | Meios Linguísticos e Multimodais de Declaração .....                              | 214 |
| CAPÍTULO 9: PONTOS-CHAVE.....  |                                                                                   | 217 |
| Capítulo 10.....               |                                                                                   | 218 |
| 10                             | Representação de Requisitos .....                                                 | 218 |
| 10.1                           | Introdução .....                                                                  | 218 |
| 10.2                           | Por que representar? .....                                                        | 220 |
| 10.3                           | Fronteiras da representação de requisitos segundo o SWEBOK v4.0 .....             | 221 |
| 10.4                           | Focos da representação.....                                                       | 225 |
| 10.5                           | Representações informais.....                                                     | 227 |
| 10.5.1                         | Representações Sistemáticas (contextuais) .....                                   | 228 |
| 10.5.2                         | Representações Livres (intuitivas).....                                           | 234 |
| 10.5.3                         | Representações Semiestruturadas (organizacionais).....                            | 238 |
| 10.5.4                         | Representações Cognitivas (conceituais e colaborativas).....                      | 241 |
| 10.6                           | Representações Semiformais .....                                                  | 245 |
| 10.6.1                         | Introdução e natureza das representações semiformais .....                        | 245 |
| 10.6.2                         | Modelos e notações .....                                                          | 246 |
| 10.6.3                         | Síntese do uso e integração .....                                                 | 256 |
| 10.7                           | Representações Formais .....                                                      | 257 |
| 10.7.1                         | Introdução e natureza das representações formais.....                             | 257 |
| 10.7.2                         | Quando formalizar: critérios de decisão e delimitação de escopo .....             | 258 |
| 10.7.3                         | Famílias de técnicas formais de representação .....                               | 260 |
| 10.7.4                         | Como integrar representações formais ao repertório do capítulo .....              | 263 |
| 10.7.5                         | Recomendações práticas para aplicação em projetos reais.....                      | 264 |
| 10.7.6                         | Limitações, riscos de uso inadequado e condições mínimas de sucesso.....          | 266 |
| 10.7.7                         | Síntese do uso e integração .....                                                 | 273 |
| 10.8                           | Síntese comparativa: entregas e riscos por tipo de representação .....            | 276 |
| CAPÍTULO 10: PONTOS-CHAVE..... |                                                                                   | 279 |
| Capítulo 11 .....              |                                                                                   | 280 |
| 11                             | Verificação e Validação de Requisitos.....                                        | 280 |
| 11.1                           | Introdução .....                                                                  | 280 |
| 11.2                           | Conceitos e distinções essenciais .....                                           | 281 |
| 11.3                           | Qualidade de requisitos como base de V&V .....                                    | 282 |
| 11.3.1                         | Confiança qualificada .....                                                       | 282 |
| 11.3.2                         | Critérios operacionais: de “qualidade desejável” a “perguntas verificáveis” ..... | 283 |
| 11.3.3                         | Defeitos típicos de requisitos e efeitos práticos.....                            | 285 |
| 11.3.4                         | Técnicas de V&V e sua utilidade na identificação de defeitos .....                | 286 |
| 11.3.5                         | Síntese: qualidade operacional como fundamento da prática de V&V.....             | 288 |
| 11.4                           | Estratégia proporcional de V&V .....                                              | 289 |
| 11.5                           | Técnicas de verificação .....                                                     | 291 |
| 11.5.1                         | Inspeções .....                                                                   | 292 |

|                                                    |                                                                                 |            |
|----------------------------------------------------|---------------------------------------------------------------------------------|------------|
| 11.5.2                                             | Revisões técnicas.....                                                          | 292        |
| 11.5.3                                             | Walkthroughs.....                                                               | 293        |
| 11.5.4                                             | Checklists .....                                                                | 294        |
| 11.5.5                                             | Uso de critérios de qualidade de requisitos como apoio à checagem.....          | 295        |
| 11.5.6                                             | Análise de consistência entre declarações e representações .....                | 296        |
| 11.5.7                                             | Análise de rastreabilidade e cobertura .....                                    | 297        |
| 11.5.8                                             | Definição de Preparado (Definition of Ready - DoR) .....                        | 297        |
| 11.5.9                                             | DEEP e INVEST como checklists de verificação em contextos ágeis e híbridos..... | 298        |
| 11.5.10                                            | IA generativa para a verificação .....                                          | 300        |
| 11.6                                               | Verificação e validação por tipo de requisito e forma de declaração .....       | 301        |
| 11.6.1                                             | Verificação e validação de RFs, RNFs e RNs.....                                 | 301        |
| 11.6.2                                             | Verificação e validação conforme a forma de declaração.....                     | 302        |
| 11.7                                               | Técnicas de validação.....                                                      | 303        |
| 11.7.1                                             | Feedback do cliente .....                                                       | 303        |
| 11.7.2                                             | Feedback da equipe .....                                                        | 304        |
| 11.7.3                                             | Uso de critérios de aceite.....                                                 | 304        |
| 11.7.4                                             | Validação colaborativa com stakeholders: oficinas e sessões de validação .....  | 305        |
| 11.7.5                                             | Cenários, histórias e casos de uso como instrumentos de validação .....         | 305        |
| 11.7.6                                             | Protótipos, simulações e experimentos como validação.....                       | 305        |
| 11.7.7                                             | IA Generativa para validação.....                                               | 306        |
| 11.7.8                                             | Síntese comparativa das técnicas de validação.....                              | 306        |
| 11.8                                               | Aplicação integrada das técnicas ao caso condutor do livro.....                 | 309        |
| 11.9                                               | Riscos, armadilhas e dificuldades comuns em V&V de requisitos .....             | 310        |
| 11.10                                              | Tratamento de achados, aprendizagem e retroalimentação da ER.....               | 312        |
| 11.11                                              | Síntese do capítulo e transição .....                                           | 313        |
| <b>CAPÍTULO 11: PONTOS-CHAVE DO CAPÍTULO .....</b> |                                                                                 | <b>314</b> |
| <b>Capítulo 12.....</b>                            |                                                                                 | <b>316</b> |
| 12                                                 | <b>Organização e Atualização de Requisitos .....</b>                            | <b>316</b> |
| 12.1                                               | Introdução .....                                                                | 316        |
| 12.2                                               | Organização e atualização na dinâmica da Engenharia de Requisitos.....          | 318        |
| 12.3                                               | Escopo de requisitos do produto .....                                           | 320        |
| 12.4                                               | Estruturas para organizar o escopo e o trabalho sobre o produto .....           | 322        |
| 12.5                                               | Organizar requisitos: dar forma ao entendimento .....                           | 325        |
| 12.6                                               | Atualizar requisitos: manter vivo o entendimento .....                          | 329        |
| 12.7                                               | Movimentos recorrentes de cuidado com o escopo.....                             | 332        |
| 12.8                                               | Rastreabilidade como memória suficiente .....                                   | 334        |
| 12.9                                               | Organização e atualização em diferentes contextos .....                         | 338        |
| 12.10                                              | Exemplo aplicado ao sistema de agendamento .....                                | 340        |
| 12.11                                              | Riscos e armadilhas na organização e atualização .....                          | 344        |
| 12.12                                              | Síntese do capítulo.....                                                        | 346        |
| <b>CAPÍTULO 12: PONTOS-CHAVE .....</b>             |                                                                                 | <b>348</b> |
| 13                                                 | <b>BIBLIOGRAFIA .....</b>                                                       | <b>350</b> |

## Sobre este material

Este documento corresponde a uma versão preliminar da obra **Requisitos de Software – Comunicação é tudo!** atualmente *em processo de consolidação, revisão e validação*. O material resulta da articulação entre pesquisa acadêmica, reflexão conceitual e experiência prática no campo da Engenharia de Requisitos, reunidas com o propósito de oferecer uma visão ao mesmo tempo fundamentada, crítica e aplicada da disciplina.

Embora este material já apresente uma estrutura conceitual e didática madura, ele ainda está em fase de aperfeiçoamento. Isso significa que redação, organização interna, exemplos, figuras, referências e escolhas pedagógicas poderão ser refinados em versões posteriores, à medida que a obra avance em seu processo de maturação editorial e acadêmica.

**IMPORTANTE:** Este documento é disponibilizado **exclusivamente como material complementar para a disciplina de Requisitos de Software** dos semestres 2025.1, 2025.2, 2026.1 e 2026.2 do curso de Engenharia de Software, na Universidade de Brasília. **Não deve ser compartilhado, reproduzido, distribuído ou publicado em qualquer formato ou meio sem autorização prévia do autor.**

Comentários, sugestões e observações críticas são bem-vindos, pois podem contribuir para o aprimoramento da versão definitiva da obra. Esse retorno é particularmente valioso em um livro que assume, como um de seus princípios centrais, que entendimento de qualidade se constrói por diálogo, revisão e aprendizagem contínua.

Agradeço seu compromisso com o uso responsável deste material, respeitando sua natureza preliminar e não pública.

Prof. Dr. George Marsicano.

# Introdução

Software não é apenas código, tecnologia ou automação. Ele atravessa práticas humanas concretas: organiza trabalho, media decisões, apoia cuidados, transforma experiências de aprendizagem, reconfigura serviços, conecta pessoas, impõe regras, amplia possibilidades e, por vezes, também produz novas restrições. Em razão disso, desenvolver software nunca é apenas construir uma solução técnica. Antes de qualquer arquitetura, implementação ou teste, há uma tarefa mais fundamental e, ao mesmo tempo, mais delicada: compreender o problema, interpretar necessidades, explicitar expectativas, reconhecer restrições e construir entendimento sobre aquilo que realmente precisa ser realizado. É nesse espaço que se situa a Engenharia de Requisitos.

Este livro parte de uma convicção central: a Engenharia de Requisitos é a mais humana das disciplinas da Engenharia de Software. Isso não significa reduzir sua natureza técnica, nem enfraquecer o rigor necessário ao trabalho com requisitos. Significa reconhecer, com mais precisão, aquilo que a prática mostra reiteradamente: requisitos não surgem prontos, objetivos e transparentes, esperando apenas ser “coletados”. Eles emergem de contextos de uso, de interesses nem sempre convergentes, de linguagens diferentes, de interpretações parciais, de negociações, de conflitos, de descobertas progressivas e de decisões construídas coletivamente. Requisitos são, em grande medida, resultado de comunicação, mediação e aprendizagem.

Por isso, compreender a Engenharia de Requisitos exige ir além de uma visão restrita que a trate somente como atividade de documentação ou especificação. Requisitos não se resumem a listas de funcionalidades, tampouco a catálogos de restrições ou atributos de qualidade. Eles expressam relações entre problema, contexto, valor e solução possível. Eles conectam objetivos de negócio, necessidades de usuários, condições operacionais, exigências regulatórias, limitações técnicas, riscos, critérios de qualidade e expectativas sociais mais amplas. Em consequência, a Engenharia de Requisitos deve ser entendida como uma prática simultaneamente técnica, humana, social, organizacional e sociotécnica.

Nessa perspectiva, o papel do engenheiro de software não é apenas registrar o que foi dito por stakeholders. Seu trabalho envolve escutar, interpretar, questionar, organizar, representar, verificar, validar, negociar e atualizar entendimentos. Envolve distinguir problema de solução, necessidade de preferência, objetivo de implementação, desejo imediato de valor duradouro. Envolve também construir pontes entre mundos distintos: o mundo do negócio, o mundo do uso, o mundo técnico, o mundo normativo e, cada vez mais, o mundo dos impactos sociais, éticos e organizacionais produzidos pelos sistemas de software.

É por isso que, nesta obra, a comunicação é tratada como princípio estruturante da disciplina. Antes de declarar, é preciso compreender. Antes de formalizar, é preciso construir sentido compartilhado. Antes de decidir como a solução será implementada, é preciso estabelecer, com clareza suficiente, o que deve ser atendido, por que isso importa, para quem gera valor, sob quais condições e com quais consequências possíveis. A qualidade dos requisitos depende, em larga medida, da qualidade dessa construção comunicativa. Requisitos mal compreendidos, ambíguos, contraditórios, incompletos ou descontextualizados tendem a comprometer decisões posteriores,

ampliando retrabalho, custos, frustrações e riscos. Em contrapartida, requisitos bem trabalhados fortalecem alinhamento, coerência, rastreabilidade, testabilidade e capacidade de adaptação.

Ao longo das últimas décadas, a Engenharia de Requisitos ampliou significativamente seu escopo e sua sofisticação. De uma ênfase inicial mais documental e orientada à estabilização prévia de especificações, a área evoluiu para incorporar abordagens iterativas, colaborativas e sensíveis ao contexto. Passou a dialogar mais intensamente com perspectivas centradas no usuário, com métodos ágeis e híbridos, com práticas visuais e multimodais de declaração e representação, com mecanismos contínuos de verificação e validação e com a compreensão de que mudança não é exceção, mas parte constitutiva do desenvolvimento de software. Ao mesmo tempo, questões como privacidade, segurança, acessibilidade, sustentabilidade, responsabilidade algorítmica e impactos sociais tornaram ainda mais evidente que requisitos ajudam a moldar modos de agir, perceber, decidir e interagir no mundo.

Este livro foi concebido para apresentar essa disciplina em sua amplitude e em sua profundidade. Nos capítulos iniciais, o leitor encontrará a evolução histórica da Engenharia de Software e da Engenharia de Requisitos, bem como os conceitos de abordagens, ciclos de vida, processos e frameworks, necessários para situar os requisitos no ecossistema mais amplo do desenvolvimento de software. Em seguida, a obra explicita os fundamentos da Engenharia de Requisitos, seus valores, princípios e sua natureza humana, social e sociotécnica. A partir daí, desenvolve progressivamente as atividades que estruturam a disciplina: elicitación e descoberta, análise e consenso, declaração, representação, verificação e validação, e organização e atualização. Em vez de tratá-las como etapas rigidamente sequenciais, o livro as apresenta como atividades articuladas, interdependentes e continuamente retroalimentadas.

A proposta desta obra não é defender um único método, um único processo ou uma única forma “correta” de fazer Engenharia de Requisitos. Ao contrário, seu argumento é que a disciplina exige sensibilidade contextual, capacidade de adaptação e consciência reflexiva. Projetos, produtos, domínios e organizações são diferentes; logo, as práticas de requisitos também precisam ser calibradas de modo coerente com criticidade, volatilidade, escala, cultura, maturidade da equipe e perfil dos stakeholders. O que não muda é a centralidade do entendimento compartilhado, da orientação a valor e da responsabilidade envolvida em transformar necessidades humanas em software.

Em síntese, este livro convida o leitor a compreender a Engenharia de Requisitos não como uma burocracia preliminar do desenvolvimento, mas como um trabalho essencial de construção de sentido, de mediação entre perspectivas e de qualificação das decisões que orientarão todo o ciclo de vida do software. Se o software interfere cada vez mais intensamente na vida social, organizacional e cotidiana, então trabalhar com requisitos é, também, participar das escolhas sobre que experiências, possibilidades, limites e impactos os sistemas irão materializar. É nesse sentido que a Engenharia de Requisitos permanece central: porque, antes de construir software, precisamos compreender — com rigor, responsabilidade e diálogo — o que vale a pena construir.



# Capítulo 1

O desenvolvimento de software tem passado por profundas transformações ao longo das décadas, refletindo não apenas a evolução tecnológica, mas também mudanças nos paradigmas de gestão, nas expectativas dos usuários e nas demandas do mercado. Neste primeiro capítulo, embarcamos em uma jornada histórica que traça a evolução da Engenharia de Software e da Engenharia de Requisitos desde a década de 1970 até os desafios dos anos 2020.

Compreender esta trajetória é fundamental para contextualizar as práticas atuais e vislumbrar tendências futuras (SOMMERVILLE, 2019; PRESSMAN; MAXIM, 2021). Partindo da “crise do software” que catalisou o surgimento da Engenharia de Software como disciplina formal, acompanharemos como a indústria gradualmente reconheceu a criticidade dos requisitos para o sucesso dos projetos, transformando a Engenharia de Requisitos de um processo documental rígido em uma prática colaborativa, adaptativa e centrada no usuário.

Cada década trouxe suas próprias inovações e aprendizados: dos modelos lineares e documentação exaustiva dos anos 70 e 80, passando pelo surgimento das metodologias iterativas e casos de uso nos anos 90, até a revolução ágil dos anos 2000 e a integração com práticas de design e experiência do usuário na década de 2010. Chegando aos anos 2020, exploraremos como a inteligência artificial, a privacidade de dados e as preocupações éticas estão remodelando a maneira como identificamos, analisamos e gerenciamos requisitos.

Mais do que um simples relato histórico, este capítulo oferece uma reflexão sobre como a Engenharia de Requisitos evoluiu para responder aos crescentes desafios de complexidade, volatilidade e escala, mantendo-se fiel à sua essência fundamental: **traduzir necessidades humanas em sistemas de software que geram valor e atendem às expectativas dos stakeholders** (LAPLANTE; KASSAB, 2022).

## 1 Breve histórico da Engenharia de Software (ESW) e Engenharia de Requisitos (ER)

A engenharia de requisitos, ao longo das últimas décadas, transformou-se profundamente, refletindo as mudanças nas metodologias de desenvolvimento de software, nas demandas do mercado e nos avanços tecnológicos (SOMMERVILLE, 2019; LAPLANTE; KASSAB, 2022). Levantamentos globais sobre o estado atual da engenharia de requisitos mostram uma diversidade crescente de práticas sendo adotadas em diferentes contextos organizacionais, com tendência clara de personalização e adaptação contextual (WAGNER et al., 2019). Vamos mergulhar em uma análise dessa evolução.



## 1.1 Década de 1970: O Surgimento da Engenharia de Software

A década de 1970 (Figura 1) é frequentemente citada como o nascimento da engenharia de software, um termo popularizado pela Conferência OTAN<sup>1</sup> sobre Engenharia de Software em 1968 (NAUR; RANDELL, 1969). Este período marcou o reconhecimento da “crise do software”, caracterizada por projetos de software que frequentemente falhavam em atender aos prazos, orçamentos e expectativas de qualidade.

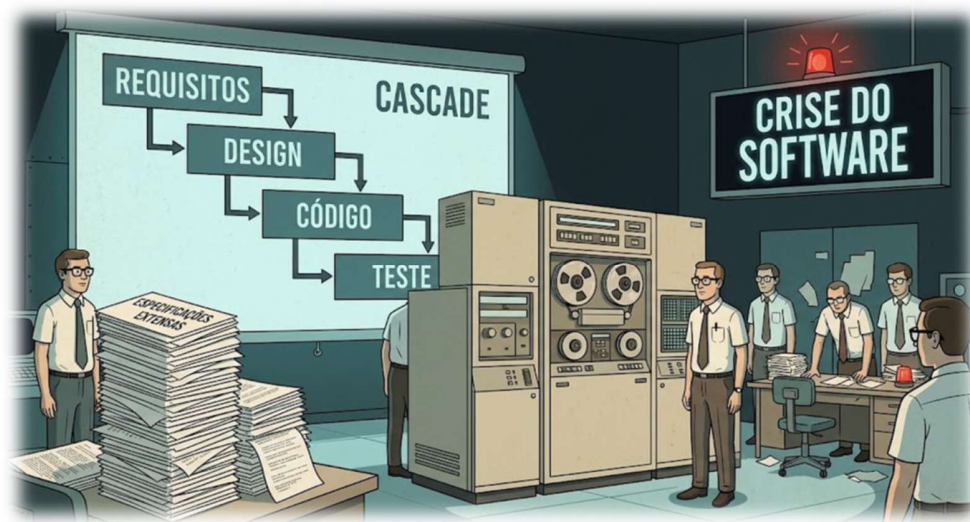


Figura 1 – Representação da década de 1970.  
Fonte: Autor (2026).

O Modelo em Cascata, formalmente descrito por Winston W. Royce em 1970, emergiu como uma das primeiras tentativas de introduzir uma abordagem sistemática e disciplinada ao processo de desenvolvimento de software (ROYCE, 1970). Embora Royce tenha destacado a necessidade de iterações dentro de algumas fases, o modelo foi amplamente interpretado e adotado como uma abordagem linear e sequencial (PRESSMAN; MAXIM, 2021; SOMMERVILLE, 2019).

A década de 1970 viu a engenharia de requisitos sendo praticada dentro de um quadro bastante restrito, onde a ênfase estava na documentação detalhada dos requisitos antes de qualquer esforço de design ou codificação. A captura de requisitos era frequentemente realizada por meio de especificações extensas e detalhadas, com o objetivo de definir completamente o sistema antes da implementação.

A engenharia de requisitos, neste período, enfrentou vários desafios, principalmente devido à rigidez do modelo em cascata e à dificuldade de prever todos os requisitos do sistema, antecipadamente. A falta de flexibilidade para adaptar-se a mudanças nos requisitos à medida

<sup>1</sup> A Conferência OTAN sobre Engenharia de Software de 1968, realizada em Garmisch, Alemanha Ocidental, representa um marco histórico que formalizou o termo "engenharia de software" e abordou a chamada "crise do software" - problemas enfrentados no desenvolvimento de sistemas grandes e complexos. Este evento reuniu especialistas internacionais que discutiram a necessidade de metodologias mais rigorosas, práticas de qualidade, testes e gestão adequada de projetos. A conferência estabeleceu as bases para a engenharia de software moderna, promovendo uma abordagem mais disciplinada e estruturada, e reconhecendo o desenvolvimento de software, cujas metodologias, técnicas e ferramentas continuam evoluindo até hoje.



que mais informações se tornavam disponíveis ou as necessidades dos usuários evoluíam era uma limitação significativa.

### **Impacto e Legado**

A década de 1970 estabeleceu a fundação sobre a qual as práticas de engenharia de requisitos e o desenvolvimento de software seriam construídos nas décadas seguintes (BOEHM; TURNER, 2003). A identificação da “crise do software” e a subsequente busca por métodos mais eficazes para o desenvolvimento de software levaram a importantes inovações nas abordagens de ER.

- **Reconhecimento da Importância da Engenharia de Requisitos:** a compreensão de que muitos problemas de software possuem origem em requisitos mal definidos ou incompletos levou à valorização crescente da engenharia de requisitos como uma disciplina crítica (POHL, 2010).
- **Evolução Contínua:** as limitações observadas com o modelo em cascata e a abordagem dirigida por plano à engenharia de requisitos durante a década de 1970 estimularam a busca por métodos mais iterativos, flexíveis e colaborativos, pavimentando o caminho para as inovações das décadas subsequentes.

Em retrospecto, a década de 1970 foi um ponto de partida crítico que destacou a necessidade de abordagens mais adaptáveis e centradas no usuário na engenharia de requisitos, um tema que continuaria a evoluir e se adaptar em resposta às mudanças tecnológicas e às demandas do mercado nas décadas seguintes.

## **1.2 Década de 1980: Reconhecimento da Engenharia de Requisitos**

Após a identificação da “crise do software” na década de 1970 e os primeiros passos em direção a uma abordagem mais disciplinada para o desenvolvimento de software, a década de 1980 (Figura 2) testemunhou um esforço concentrado para consolidar a engenharia de software como uma disciplina formal. A engenharia de requisitos, como um componente crítico desse esforço, começou a se estabelecer mais firmemente, com o reconhecimento de que o sucesso do desenvolvimento de software dependia crucialmente da correta identificação, documentação e gestão de requisitos.

Durante esta década, a abordagem dirigida por plano, exemplificada pelo Modelo Cascata, continuou a ser dominante, mas começou a ser questionada e complementada por métodos que incorporavam feedback e iteração mais explicitamente, dentre eles, o desenvolvimento Espiral proposto por Barry Boehm, em 1986 (LARMAN, 2004; SOMMERVILLE, 2019). Embora o Modelo Cascata oferecesse uma estrutura clara para o desenvolvimento, sua rigidez começou a ser vista como uma limitação, especialmente em projetos de software mais complexos e dinâmicos (LARMAN, 2004).

A engenharia de requisitos na década de 1980 focou na melhoria dos processos de identificação, documentação e verificação de requisitos (LAUESEN, 2002). A indústria começou a adotar outras técnicas para a especificação de requisitos, incluindo a modelagem de dados e o uso de linguagens formais (WOODCOCK; DAVIES, 1996). Além disso, a validação de

requisitos, por meio de revisões e inspeções, tornou-se uma prática mais comum para assegurar que os requisitos capturados refletissem fielmente as necessidades dos stakeholders (FREEDMAN; WEINBERG, 1990).

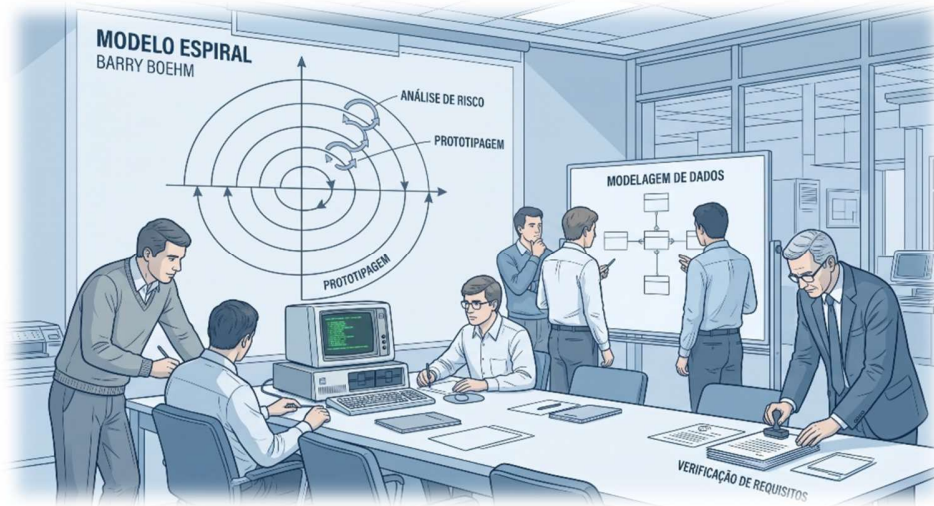


Figura 2 – Representação da década de 1980.  
Fonte: Autor (2026).

Apesar desses avanços, a década de 1980 também destacou os desafios persistentes na engenharia de requisitos, particularmente em relação à dificuldade de lidar com mudanças nos requisitos após o início do desenvolvimento. A comunicação eficaz entre desenvolvedores e stakeholders continuou a ser um desafio, exacerbado pela complexidade crescente dos sistemas de software e pela natureza frequentemente ambígua ou incompleta dos requisitos iniciais.

### Impacto e Legado

- **Reconhecimento da Engenharia de Requisitos:** a década de 1980 solidificou a compreensão de que a ER era uma etapa crítica no ciclo de vida do desenvolvimento de software, fundamental para o sucesso do projeto e para a satisfação do cliente. A comunidade começou a dedicar mais atenção à melhoria das práticas de engenharia de requisitos e ao desenvolvimento de ferramentas de suporte.
- **Evolução das Metodologias:** as limitações percebidas nas abordagens estritamente lineares e planificadas levaram à busca por metodologias que permitissem maior flexibilidade e adaptabilidade. Isso preparou o terreno para a emergência e popularização das metodologias ágeis no final dos anos 1990, que promoveriam uma revolução nas práticas da engenharia de requisitos.
- **Ferramentas e Tecnologia:** o avanço da tecnologia de computadores e o desenvolvimento de software, também influenciaram a ER, permitindo o registro e gerenciamento de requisitos em softwares, por meio do uso de processadores de texto, sistemas de controle de versão, e-mails e sistemas de compartilhamento

de documentos, facilitando a comunicação entre as equipes de desenvolvimento e os stakeholders.

A década de 1980 foi um período de crescimento e reflexão para a ER, marcando um movimento em direção a práticas mais estruturadas e formalizadas, ao mesmo tempo em que semeava as sementes para as inovações metodológicas que se seguiriam.

### 1.3 Década de 1990: Consolidação e Primeira Conferência da ER

Os anos 90 (Figura 3) marcaram a consolidação da engenharia de requisitos como uma disciplina da engenharia de software (VAN LAMSWEERDE, 2009). A academia e a indústria reconheceram e formalizaram sua importância, resultando na organização da primeira conferência internacional dedicada exclusivamente ao assunto, a International Conference on Requirements Engineering (RE), que ocorreu em 1993 em San Diego, Califórnia, EUA. Esse evento simbolizou o reconhecimento global da engenharia de requisitos como uma área de estudo e prática essencial dentro do desenvolvimento de software.

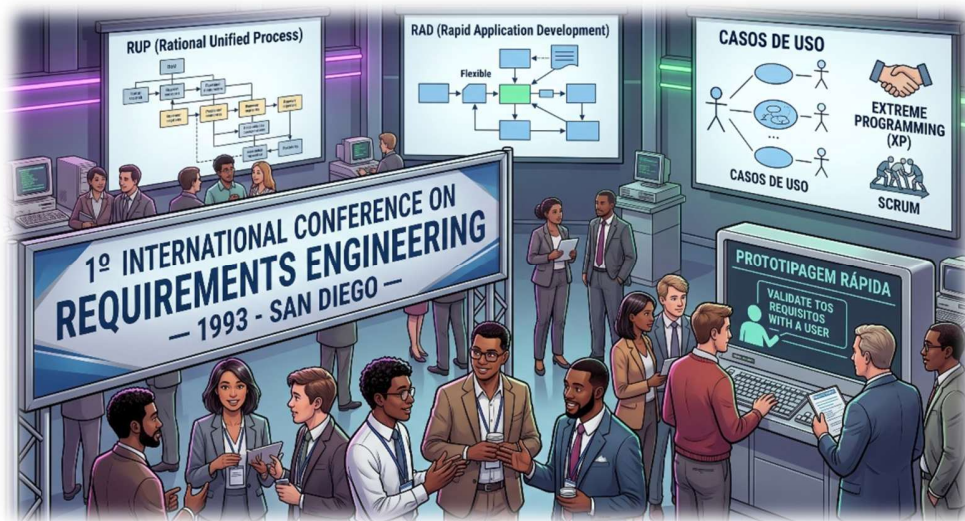


Figura 3 – Representação da década de 1990.  
Fonte: Autor (2026).

A década de 1990, também foi marcada por uma rápida evolução tecnológica, incluindo a expansão da internet, que transformou as práticas de desenvolvimento de software. As organizações enfrentavam desafios crescentes relacionados à complexidade dos sistemas, à aceleração do tempo de mercado e à necessidade de maior flexibilidade no desenvolvimento (EVANS, 2003). Foi neste contexto que as metodologias iterativas e incrementais, incluindo as ágeis começaram a surgir e ganhar destaque, como resposta às limitações, principalmente, do modelo cascata.

Neste sentido, a década de 1990 testemunhou a diversificação das abordagens de desenvolvimento de software. Além do contínuo uso de modelos como o Cascata e o Modelo V, emergiram abordagens que enfatizavam a iteração e a adaptabilidade, incluindo o Rapid Application Development (RAD), o Rational Unified Process (RUP), o Dynamic Systems

Development Method (DSDM) e o Feature-Driven Development (FDD) (KRUCHTEN, 2004; LARMAN, 2004).

Notavelmente, o final dos anos 90 foi um período fértil para o desenvolvimento das ideias que fundamentariam o ágil. Metodologias como Extreme Programming (XP), criada por Kent Beck, e Scrum, desenvolvido por Ken Schwaber e Jeff Sutherland, começaram a ser praticadas e a ganhar popularidade durante essa época (SCHWABER, 2004; COHN, 2004). Essas metodologias enfatizavam princípios como desenvolvimento iterativo, envolvimento do cliente, e adaptação a mudanças, que se tornariam centrais para o ágil.

Neste cenário, a ER na década de 1990 começou a refletir a necessidade de maior flexibilidade e envolvimento do cliente. As práticas ajustaram-se para acomodar a natureza iterativa e centrada no usuário das novas metodologias, por exemplo:

- **Casos de Uso:** popularizados por Ivar Jacobson, os casos de uso tornaram-se uma ferramenta fundamental para a captura de requisitos em um formato que descreve como os usuários interagem com o sistema (COCKBURN, 2001).
- **Prototipagem Rápida:** a prototipagem foi amplamente adotada para validar requisitos e conceitos de design com stakeholders, permitindo ajustes rápidos antes da implementação final (COOPER; REIMANN; CRONIN, 2014).
- **Feedback Contínuo:** a integração do feedback do cliente tornou-se uma prática essencial, com as metodologias ágeis promovendo a revisão e adaptação constantes dos requisitos ao longo do ciclo de desenvolvimento (COHN, 2004).
- **User Story:** tornaram-se uma prática central em metodologias ágeis, particularmente com a popularização do Extreme Programming (XP) e do Scrum. Uma user story descreve uma funcionalidade do software do ponto de vista do usuário final, focando no valor que a funcionalidade traz para o cliente (COHN, 2004).

A década de 1990 destacou a tensão entre a necessidade de processos de desenvolvimento rigorosos e a demanda por maior adaptabilidade e resposta rápida às mudanças do mercado. A engenharia de requisitos teve que equilibrar a precisão e a completude na captura de requisitos com a necessidade de adaptar-se às descobertas e mudanças emergentes.

### Impacto e Legado

- **Consolidação da Engenharia de Requisitos:** a década de 1990 consolidou a ER como um componente crítico do processo de desenvolvimento de software, com uma compreensão mais profunda de sua importância para o sucesso do projeto.
- **Evolução Metodológica:** as inovações metodológicas da década prepararam o terreno para a ampla adoção das práticas ágeis no século 21, marcando uma transição significativa nas abordagens de desenvolvimento de software.

Por fim, pode-se dizer que a década de 1990 foi um período de transição e experimentação no desenvolvimento de software, bem como na engenharia de requisitos. As práticas e metodologias desenvolvidas durante esta década continuam a influenciar a engenharia de software, evidenciando a natureza dinâmica e adaptável do campo.

## 1.4 Década de 2000: Consolidação do Ágil e Novas Ferramentas de Engenharia de Requisitos

A década de 2000 (Figura 4) foi um período de consolidação para as metodologias ágeis no desenvolvimento de software. Com o Manifesto Ágil formalizado em 2001 por Beck et al. (BECK et al., 2001), a indústria começou a adotar mais amplamente práticas ágeis, reconhecendo-as como soluções eficazes para os desafios de rapidez, adaptabilidade e cliente-centrismo que muitos projetos enfrentavam (SCHWABER; SUTHERLAND, 2020; COHN, 2010). Além disso, o aumento da complexidade dos sistemas de software e a globalização das equipes de desenvolvimento impulsionaram a busca por ferramentas e técnicas que facilitassem a colaboração e a gestão de requisitos.

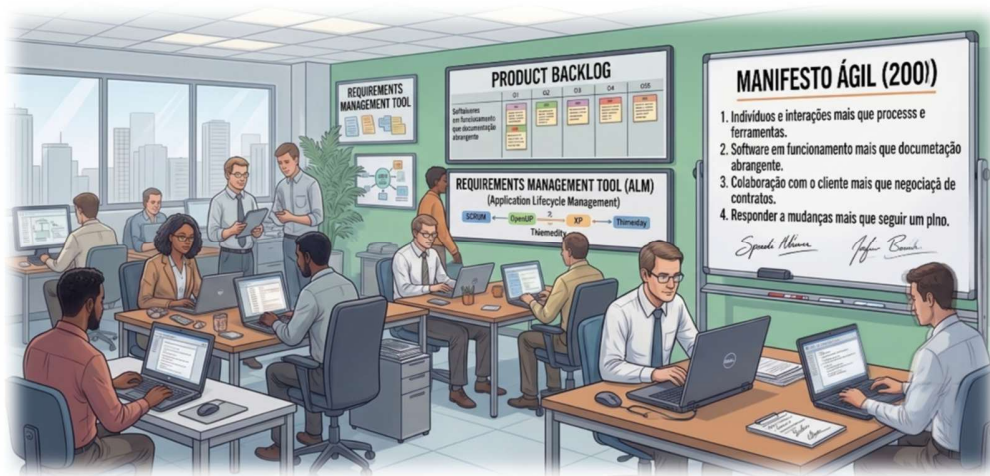


Figura 4 – Representação da década de 2000.  
Fonte: Autor (2026).

Além do contínuo uso e refinamento das metodologias ágeis, como Scrum e XP, a década de 2000 viu o desenvolvimento e a adoção de frameworks ágeis mais estruturados, a partir da adaptação do Rational Unified Process (RUP), dando origem ao Open Unified Process (OpenUP) e o Agile Unified Process (AUP), e o surgimento do Scrum of Scrums para coordenar múltiplos times ágeis.

As práticas da engenharia de requisitos evoluíram significativamente durante a década de 2000 para se alinhar com a filosofia ágil. Houve uma mudança em direção a técnicas que promoviam a colaboração direta com os stakeholders e a flexibilidade na gestão de mudanças nos requisitos (LEFFINGWELL, 2011).

A adoção de user stories se tornou generalizada como meio de capturar requisitos de maneira concisa e centrada no valor para o usuário. Além disso, o Behavior-Driven Development (BDD)<sup>2</sup> ganhou força como uma técnica para definir requisitos e testes em uma linguagem natural, promovendo um entendimento compartilhado entre desenvolvedores,

<sup>2</sup> O BDD é uma prática que utiliza linguagem natural estruturada para descrever o comportamento esperado do software a partir da perspectiva do usuário. As especificações são escritas em formato "Dado-Quando-Então" (Given-When-Then), facilitando o entendimento para todos os envolvidos (Smart, 2014).

testadores e clientes (SMART, 2014). E, o backlog de requisitos, frequentemente referido no contexto de metodologias ágeis como “product backlog” ou simplesmente “backlog”, também ganhou destaque fundamental para a ER nas metodologias ágeis, proporcionando uma visão abrangente e adaptável do trabalho (requisitos) necessário para atingir os objetivos do projeto (RUBIN, 2022; PICHLER, 2010; SUTHERLAND, 2020).

A década também testemunhou um crescimento no desenvolvimento e na utilização de ferramentas de software para a gestão de requisitos, que facilitaram a colaboração entre equipes distribuídas geograficamente e a rastreabilidade de requisitos ao longo do ciclo de vida do desenvolvimento (LAPLANTE, 2007).

A engenharia de requisitos na década de 2000 enfrentou o desafio de equilibrar a necessidade de agilidade e flexibilidade com a garantia de que os sistemas complexos atendessem aos padrões de qualidade e segurança. A crescente adoção de práticas ágeis levantou questões sobre como melhor integrar a engenharia de requisitos em ambientes que valorizavam a documentação mínima e a resposta rápida a mudanças.

### **Impacto e Legado**

- **Maturidade do Ágil:** a década de 2000 estabeleceu as metodologias ágeis como abordagens maduras e confiáveis para o desenvolvimento de software, com a engenharia de requisitos adaptando-se para apoiar uma colaboração mais estreita com os clientes e uma reação mais ágil às mudanças.
- **Avanços Tecnológicos:** as ferramentas de gestão de requisitos tornaram-se mais sofisticadas, permitindo melhor rastreabilidade, comunicação e colaboração, fundamentais para o sucesso dos projetos de software em um mundo cada vez mais conectado e rápido.

Com isso, a década de 2000 marcou um ponto de virada na forma como a indústria de software aborda a engenharia de requisitos, com um movimento claro em direção a práticas que enfatizam a flexibilidade, a colaboração e a entrega contínua de valor, preparando o caminho para inovações futuras na disciplina.

## **1.5 Década de 2010: Integração Profunda com Práticas Ágeis e Foco na Experiência do Usuário**

A década de 2010 (Figura 5) foi caracterizada pela adoção massiva e global das práticas ágeis em diversos setores da indústria de software (COHN, 2010; SCHWABER; SUTHERLAND, 2020). As organizações enfrentaram desafios decorrentes da necessidade de inovação rápida, da complexidade crescente dos sistemas de software e da demanda por uma experiência do usuário (UX) excepcional. A engenharia de requisitos, nesse contexto, teve que se adaptar não apenas para manter a agilidade e flexibilidade, mas também para assegurar que o desenvolvimento de software fosse orientado pelo valor entregue ao usuário final.

Esta década viu a consolidação do Scrum, Kanban e outros frameworks ágeis como práticas padrão no desenvolvimento de software. Frameworks ágeis em escala, como o SAgile (Scaled Agile Framework) e o LeSS (Large Scale Scrum), foram desenvolvidos para facilitar



a aplicação de práticas ágeis em grandes organizações e projetos complexos (LEFFINGWELL, 2011; SCALED AGILE, 2023). A engenharia de requisitos teve que se alinhar com essas abordagens, focando na colaboração contínua com os stakeholders e na adaptabilidade.



Figura 5 – Representação da década de 2010.  
Fonte: Autor (2026).

O uso de user stories e epics para capturar e organizar requisitos tornou-se ainda mais central, permitindo que equipes ágeis mantivessem o foco no valor do usuário enquanto gerenciavam requisitos em diferentes níveis de granularidade (PATTON, 2014; CAROLI, 2021).

A integração de práticas e metodologias, como, Design Thinking, Design Sprint, Lean Inception, PBB (Product Backlog Building) e a ênfase na UX no processo de desenvolvimento, reforçam a importância de uma abordagem centrada no usuário, pela engenharia de requisitos (BROWN, 2009; KNAPP; ZERATSKY; KOWITZ, 2016; PATTON, 2014; AGUIAR e CAROLI, 2021). Elas incentivam a colaboração, a experimentação e a iteração rápida, o que ajuda as equipes a identificar os requisitos mais relevantes e a adaptar-se às mudanças nas necessidades dos usuários e nas condições de mercado. Ao focar nas pessoas para quem o software foi desenvolvido, essas abordagens contribuem para a criação de produtos mais eficazes, desejáveis e bem-sucedidos.

O Behavior-Driven Development (BDD) e o Test-Driven Development (TDD)<sup>3</sup> ganharam ainda mais destaque como práticas que reforçam a definição de requisitos por meio de comportamentos esperados e testes, garantindo que o desenvolvimento esteja alinhado com as expectativas dos stakeholders (SMART, 2014; KEOGH, 2019).

A crescente complexidade dos projetos, juntamente com a expectativa de entregas rápidas e de alta qualidade, colocou pressão sobre a engenharia de requisitos para ser mais

<sup>3</sup> É uma prática de programação onde o desenvolvedor primeiro escreve um teste automatizado que define uma melhoria ou nova funcionalidade, depois implementa o código mínimo necessário para passar no teste, e finalmente refatora o código para atender aos padrões de qualidade (Beck, 2003).

eficiente, precisa e alinhada com os objetivos de negócios e as necessidades dos usuários. A necessidade de equilibrar detalhamento técnico com visão de negócios tornou-se um aspecto crítico da disciplina.

### **Impacto e Legado**

- **Adaptação e Inovação:** a década de 2010 solidificou a engenharia de requisitos como uma disciplina adaptável e inovadora, capaz de integrar novas práticas e tecnologias para melhor atender às demandas do desenvolvimento de software moderno.
- **Foco no Usuário:** a orientação para a experiência do usuário emergiu como um princípio central, refletindo uma mudança fundamental na forma como os requisitos são entendidos e priorizados.
- **Colaboração e Comunicação:** a ênfase em práticas ágeis reforçou a importância da comunicação eficaz e da colaboração entre as equipes de desenvolvimento e os stakeholders, incluindo usuários finais, como elementos vitais para o sucesso do projeto.

A década de 2010 marcou um período de maturação para a engenharia de requisitos, onde a disciplina não só se alinhou com as práticas ágeis, mas também adotou uma perspectiva mais holística e centrada no usuário. Essas mudanças refletem um entendimento mais profundo da importância de conectar o desenvolvimento de software aos objetivos estratégicos de negócios e às necessidades reais dos usuários.

## **1.6 Década de 2020: Avanços Tecnológicos e Enfrentamento de Novos Desafios**

A década de 2020 (Figura 6) tem sido marcada por avanços tecnológicos rápidos e pela transformação digital em escala global (LAPLANTE; KASSAB, 2022). As organizações enfrentam o desafio de desenvolver softwares em um ambiente cada vez mais volátil, incerto, complexo e ambíguo (VUCA), exigindo uma engenharia de requisitos que não apenas acompanhe o ritmo da inovação tecnológica, mas também aborde questões emergentes como privacidade de dados, segurança cibernética e sustentabilidade (DICK; HULL; JACKSON, 2017). Além disso, a pandemia de COVID-19 acelerou a adoção do trabalho remoto, trazendo novas dinâmicas para a colaboração em projetos de software.

As práticas ágeis e os frameworks em escala, como SAFe e LeSS, continuam a evoluir para atender às necessidades de organizações grandes e complexas. Simultaneamente, a engenharia de requisitos começa a incorporar princípios de DevOps e CI/CD (Continuous Integration/Continuous Deployment) para facilitar entregas mais rápidas e confiáveis, sendo, colaboração e comunicação melhoradas, feedback contínuo e iterações rápidas, automatização e eficiência, rastreabilidade e transparência, e adaptação contínua (AMBLER; LINES, 2012).

A inteligência artificial (IA) e a aprendizagem de máquina (ML) também começam a desempenhar um papel significativo, tanto como ferramentas para o desenvolvimento de software quanto como áreas de foco para a identificação, análise, declaração, organização e atualização de requisitos (LAPLANTE; KASSAB, 2022). Estudos recentes documentam avanços significativos na classificação automática de requisitos (MAALEJ et al., 2021) e na



extração e agrupamento de terminologia de requisitos (ARORA et al., 2017), demonstrando como técnicas de IA podem transformar as práticas tradicionais de engenharia de requisitos. Além da automação de tarefas repetitivas na declaração dos requisitos, ferramentas baseadas em IA (IBM Watson Requirements Quality Assistant (RQA)<sup>4</sup>, Visure Requirements ALM Platform<sup>5</sup>, ReqSuite RM<sup>6</sup>, Automated Quality User Story Assessment – AQUASA<sup>7</sup>, entre outras) estão sendo empregadas para análise preditiva de requisitos, identificando potenciais conflitos, ambiguidades e inconsistências antes mesmo que se tornem problemas.

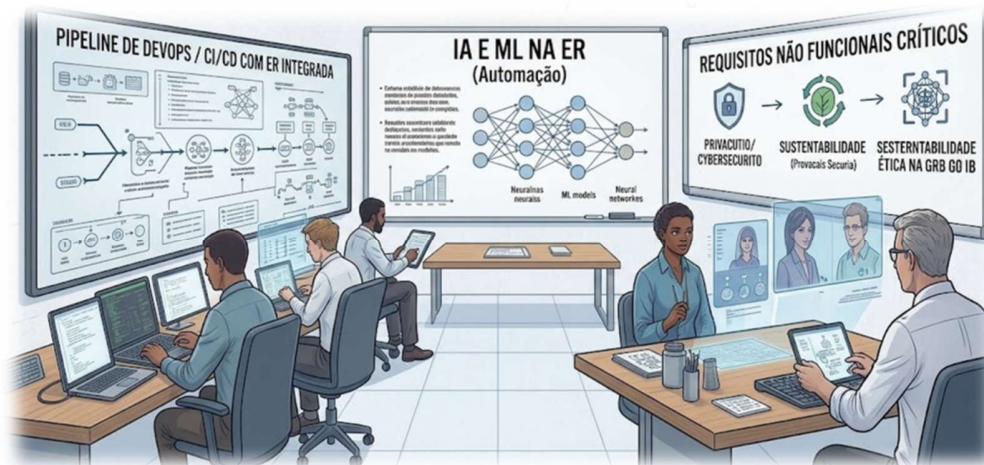


Figura 6 – Representação da década de 2020.  
Fonte: Autor (2026).

Um aspecto particularmente promissor é o uso de algoritmos de aprendizado de máquina para analisar históricos de projetos anteriores, permitindo recomendações baseadas em evidências sobre priorização de requisitos e estimativas de esforço mais precisas (JONES, 2007; MATHEW G., 2017). Estas ferramentas não substituem a expertise humana, mas amplificam sua eficácia ao permitir que engenheiros de software foquem em atividades de maior valor que exigem criatividade e pensamento crítico, como negociação com stakeholders e tomada de decisões estratégicas.

Além disso, a segurança e a privacidade tornaram-se aspectos cada vez mais críticos, exigindo que requisitos relacionados sejam considerados desde as fases iniciais do desenvolvimento (LAPLANTE; KASSAB, 2022). Isso reflete uma mudança em direção a uma abordagem mais holística e proativa na engenharia de requisitos, onde a conformidade, a

<sup>4</sup> Utiliza aprendizado de máquina para analisar documentos de requisitos, identificando ambiguidades, inconsistências e incompletudes. O sistema avalia a qualidade dos requisitos e sugere melhorias baseadas em padrões históricos e melhores práticas da indústria (<https://www.ibm.com/docs/en/erqa?topic=assistant-overview>). Acesso em 15 de janeiro de 2025.

<sup>5</sup> Integra funcionalidades de IA para prever inconsistências entre requisitos e identificar padrões que podem levar a problemas durante o desenvolvimento, como duplicações não óbvias ou requisitos conflitantes (<https://visuresolutions.com/tool-suite/requirements-alm-platform>). Acesso em 15 de janeiro de 2025.

<sup>6</sup> Utiliza mecanismos de aprendizado de máquina para analisar requisitos históricos de projetos anteriores e sugerir requisitos complementares que possam ter sido esquecidos, baseando-se em padrões de correlação identificados em projetos semelhantes (<https://www.reqsuite.io/en>). Acesso em 18 de janeiro de 2025.

<sup>7</sup> Ferramenta de código aberto que emprega técnicas de PLN para analisar histórias de usuário, verificando se elas aderem ao formato padrão e identificando problemas como falta de atores, ações não claras, ou benefícios não especificados (<https://github.com/gglucass/AQUASA>). Acesso em 10 de janeiro de 2025.

segurança e a satisfação do usuário são integradas ao ciclo de vida do desenvolvimento desde o início.

Também, começam a ser considerados, requisitos relacionados à sustentabilidade e ética no uso de tecnologias, especialmente IA refletindo uma maior conscientização sobre o impacto social e ambiental do software (LAPLANTE; KASSAB, 2022). A necessidade de desenvolver softwares que não apenas atendam às necessidades funcionais, mas também abordem questões éticas, de privacidade e sustentabilidade coloca novos desafios para a engenharia de requisitos (LAPLANTE; KASSAB, 2022). Revisões sistemáticas da literatura identificaram múltiplos desafios enfrentados pelas organizações que adotam engenharia de requisitos ágil, incluindo documentação insuficiente, falta de ênfase em requisitos não-funcionais e dificuldades com stakeholders distribuídos geograficamente (SCHÖN; THOMAS-CHEWSKI; ESCALONA, 2017). Além disso, a colaboração eficaz em equipes distribuídas globalmente e a integração de novas tecnologias e práticas em processos de desenvolvimento estabelecidos requerem adaptações contínuas nas abordagens identificação e gerenciamento de requisitos.

### Impacto e Legado

- **Resiliência e Adaptabilidade:** a década de 2020 está demonstrando a importância da resiliência e adaptabilidade na engenharia de requisitos, à medida que as equipes buscam maneiras de responder rapidamente às mudanças do mercado e às exigências tecnológicas.
- **Foco no Humano e Responsabilidade:** há um movimento crescente em direção ao desenvolvimento de software que prioriza as necessidades humanas e aborda questões de responsabilidade ética e sustentabilidade, refletindo uma abordagem mais holística e consciente na engenharia de requisitos.
- **Integração Tecnológica:** a integração de avanços tecnológicos na prática da engenharia de requisitos, especialmente o uso de IA e ML, está moldando o futuro da disciplina, oferecendo novas possibilidades para a análise e gestão de requisitos de forma mais eficiente e eficaz.

A década de 2020 está redefinindo a engenharia de requisitos com um foco renovado na adaptabilidade, responsabilidade e inovação tecnológica (DICK; HULL; JACKSON, 2017). Essas tendências estão preparando o terreno para futuros avanços na forma como os requisitos são elicitados, analisados e organizados, garantindo que o desenvolvimento de software continue a evoluir para atender às demandas de um mundo em rápida mudança.

Para fechar esse breve histórico sobre a engenharia de software (ESW) e a engenharia de requisitos (ER), o Quadro 1 apresenta um resumo quanto às suas características ao longo das décadas, assim como de suas transições entre os períodos e, a Figura 7 representa, temporalmente, os marcos de evolução da ESW e ER.

No próximo capítulo serão apresentadas as abordagens, ciclos de vida e processos de desenvolvimento de software e frameworks, os quais representam a evolução e aprendizado

ao longo do tempo da engenharia de software. Com isso, será possível entender como os requisitos de software são tratados diante das diferentes perspectivas ao longo do tempo.

| Década      | Engenharia de Software                                                                                                 | Engenharia de Requisitos                                                                                        | Transição entre as Décadas                                                                                                                                                                                                                                                                                                                                                   |
|-------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1970</b> | Surgimento da engenharia de software; foco no Modelo Cascata.                                                          | Prática restrita com ênfase na documentação detalhada dos requisitos.                                           | A década de <b>1970</b> estabeleceu as bases da engenharia de software, com foco em metodologias estruturadas como o Modelo Cascata. A transição para os anos <b>1980</b> viu um questionamento dessas abordagens rígidas, com o desenvolvimento espiral introduzindo conceitos de iteração e feedback mais cedo no ciclo de vida do desenvolvimento.                        |
| <b>1980</b> | Consolidação da engenharia de software; questionamentos ao Modelo Cascata; surgimento do desenvolvimento espiral.      | Melhoria nos processos de identificação, documentação e validação de requisitos; maior colaboração.             | Os anos <b>80</b> experimentaram uma busca por flexibilidade, preparando o terreno para a década de <b>1990</b> , que marcou o início da adoção das metodologias ágeis. Esta transição reflete um movimento em direção à adaptabilidade, colaboração e entrega rápida, caracterizado pela formalização do Scrum e Extreme Programming (XP).                                  |
| <b>1990</b> | Diversificação de abordagens; surgimento das metodologias ágeis (XP, Scrum); primeira conferência internacional de ER. | Adaptação para maior flexibilidade e envolvimento do cliente; utilização de casos de uso e prototipagem rápida. | A consolidação das práticas ágeis na década de <b>1990</b> abriu caminho para uma maior adoção e refinamento nos anos <b>2000</b> . A engenharia de requisitos começou a alinhar-se mais estreitamente com princípios ágeis, integrando técnicas como user stories e BDD para facilitar a colaboração e o feedback.                                                          |
| <b>2000</b> | Consolidação do ágil; avanço de ferramentas de gestão de requisitos.                                                   | Alinhamento com a filosofia ágil; adoção de user stories e BDD; ênfase na colaboração direta com stakeholders.  | A primeira década dos anos <b>2000</b> viu a maturação das práticas ágeis e a expansão de suas aplicações. A transição para os anos <b>2010</b> enfatizou a importância da experiência do usuário (UX), com a engenharia de requisitos adotando abordagens como Design Thinking para melhor capturar e atender às necessidades dos usuários.                                 |
| <b>2010</b> | Adoção global do ágil; foco em UX; desenvolvimento de frameworks ágeis em escala.                                      | Integração profunda com práticas ágeis; ênfase na experiência do usuário; inovação em práticas e tecnologias.   | Os anos <b>2010</b> solidificaram a integração da UX na engenharia de software e trouxeram novos desafios como privacidade e segurança. Avançando para <b>2020</b> , a engenharia de requisitos enfrenta a incorporação de IA e considerações éticas, refletindo uma evolução para abordagens mais responsáveis e tecnologicamente avançadas no desenvolvimento de software. |
| <b>2020</b> | Avanços tecnológicos rápidos; enfrentamento de novos desafios como privacidade e sustentabilidade.                     | Adaptação à inovação tecnológica e questões emergentes; ênfase em IA para gestão de requisitos.                 |                                                                                                                                                                                                                                                                                                                                                                              |

Quadro 1 – Características, por década, da engenharia de software e da engenharia de requisitos.  
Fonte: Autor, baseado e adaptado de Sommerville (2019), Pressman e Maxim (2021), Laplante e Kassab (2022).

## Evolução da Engenharia de Software e da Engenharia de Requisitos

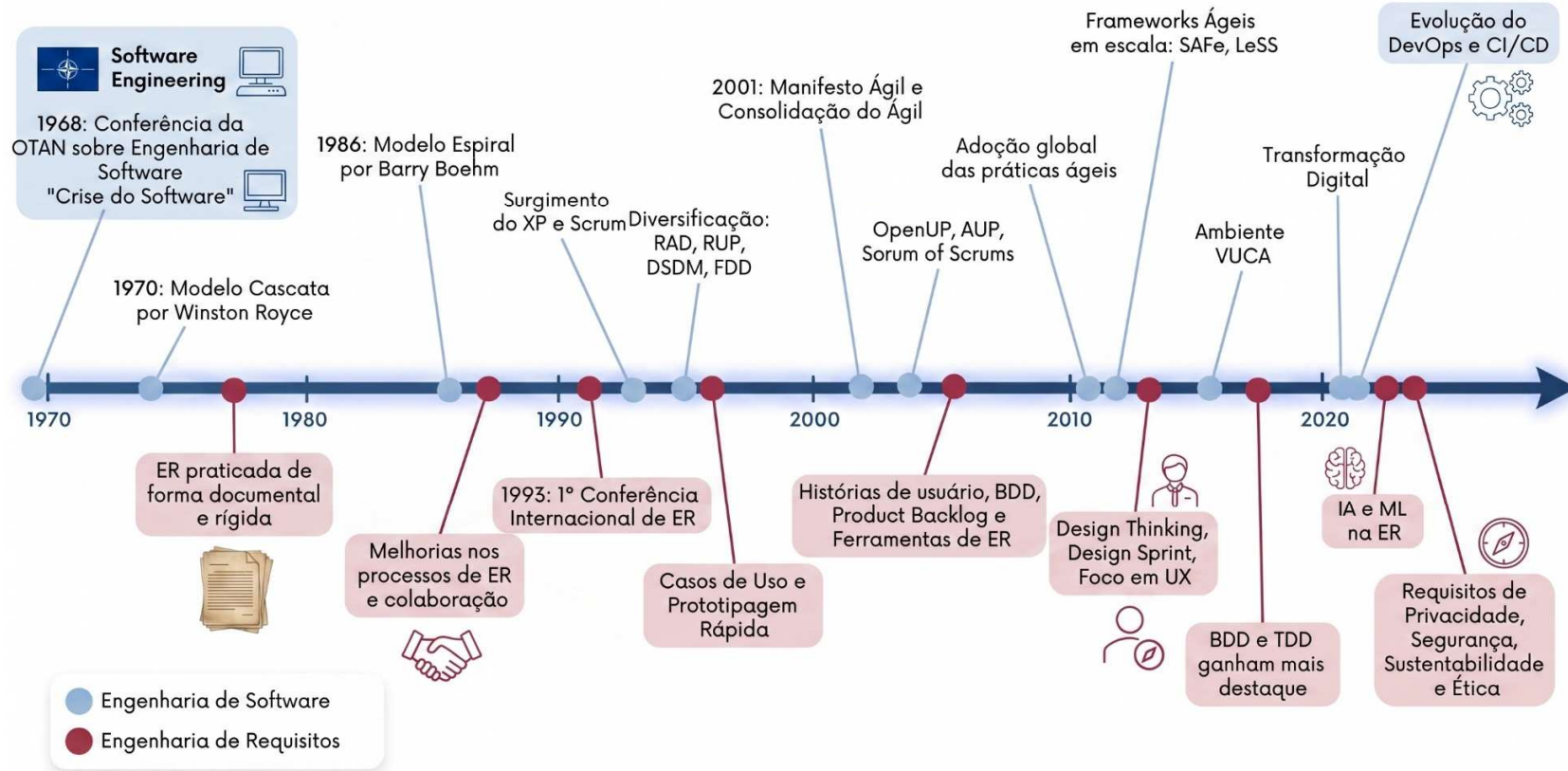


Figura 7 – Evolução temporal da Engenharia de Software e Engenharia de Requisitos.  
Fonte: Autor, baseado e adaptado de Larman (2004) e Leffingwell (2011).

## CAPÍTULO 1: PONTOS-CHAVE

- O capítulo apresenta uma **linha histórica da Engenharia de Software (ESW)** e da **Engenharia de Requisitos (ER)**, situando-as em suas transformações conceituais, metodológicas e tecnológicas desde a década de 1970 até os desafios contemporâneos.
- A **década de 1970** marcou o nascimento da ESW como disciplina formal, em resposta à *crise do software*. Nesse contexto, surgiram os **modelos sequenciais**, como o Modelo em Cascata, que introduziram sistematização e controle, mas impuseram rigidez e dificuldade de adaptação a mudanças.
- Nos **anos 1980**, a ER começou a ser reconhecida como área autônoma, motivada pela percepção de que **falhas nos requisitos eram a principal causa de insucesso dos projetos**. A atividade passou a buscar formas estruturadas de elicitação, análise e documentação.
- A **década de 1990** consolidou a ER como campo de pesquisa e prática profissional, com a **primeira conferência internacional de Engenharia de Requisitos** e a difusão de métodos baseados em **casos de uso e prototipação rápida**, aproximando técnicos e usuários.
- Nos **anos 2000**, com a consolidação do **ágil**, a ER incorporou novas práticas de comunicação e validação incremental, como **histórias de usuário e BDD (Behavior-Driven Development)**, reforçando o foco em colaboração e feedback contínuo.
- A partir de **2010**, a disciplina evoluiu para integrar **design centrado no usuário, UX e design thinking**, unindo engenharia e experiência, o que ampliou o entendimento dos requisitos como representações de valor e experiência, não apenas de funcionalidade.
- A **década de 2020** introduz novos desafios, como **inteligência artificial, privacidade, sustentabilidade e ética digital**, exigindo da ER uma postura adaptativa, crítica e responsável diante de sistemas complexos e sociotécnicos.
- O capítulo conclui que a trajetória da ER reflete uma **mudança de paradigma**: de uma prática documental e técnica para uma **disciplina colaborativa, interpretativa e orientada à criação de valor social e organizacional**.

# Capítulo 2

No universo do desenvolvimento de software, a compreensão clara dos conceitos fundamentais é essencial para navegar pela multiplicidade de abordagens, ciclos de vida, processos e frameworks disponíveis. O Capítulo 2 estabelece as bases conceituais que alicerçam as práticas de Engenharia de Software modernas, definindo os termos e estruturas que serão empregados ao longo deste livro.

Iniciaremos esclarecendo as distinções críticas entre abordagens de desenvolvimento (como filosofias que guiam o desenvolvimento), ciclos de vida (as estruturas temporais que organizam o trabalho), processos (as sequências detalhadas de atividades técnicas) e frameworks (as estruturas para coordenação e gerenciamento). Esta clareza terminológica é fundamental para evitar confusões comuns na literatura e na prática profissional, onde frequentemente estes termos são utilizados de forma intercambiável.

Ao explorar o relacionamento entre estes conceitos, examinaremos como cada um influencia e é influenciado pelos demais, formando um ecossistema metodológico interdependente. Destacaremos o papel central da adaptabilidade contextual – a capacidade de selecionar e combinar elementos metodológicos conforme as necessidades específicas de cada projeto, domínio de aplicação e cultura organizacional.

Este capítulo também introduz os elementos contextuais que moldam a aplicação prática dessas estruturas conceituais: as particularidades de cada projeto, as características do domínio de aplicação e os aspectos da cultura organizacional. Compreender estes elementos contextuais permite que equipes e organizações desenvolvam capacidade de adaptação metodológica, superando a adoção dogmática de metodologias únicas em favor de abordagens customizadas e eficazes.

Munidos deste arcabouço conceitual, estaremos preparados para examinar nos capítulos subsequentes como diferentes abordagens, ciclos de vida, processos e frameworks lidam com os requisitos de software, avaliando suas forças, limitações e contextos de aplicação mais adequados.

## 2 Conceitos

### 2.1 Abordagens de Desenvolvimento de Software

Uma abordagem de desenvolvimento de software estabelece a **filosofia** geral do desenvolvimento. Ela engloba os **valores e princípios** que orientam como o software é desenvolvido. A abordagem molda a cultura e mentalidade da equipe, determina como as equipes interagem e trabalham juntas, e influenciam todas as práticas, decisões e métodos utilizados no desenvolvimento do software. Por exemplo: a abordagem ágil é uma filosofia de desenvolvimento que valoriza a colaboração, a resposta a mudanças e a entrega contínua de valor.



## 2.2 Ciclos de Vida de Desenvolvimento de Software

O ciclo de vida de desenvolvimento de software (Software Development Life Cycle - SDLC) refere-se às **fases ou etapas** pelas quais um produto de software passa, desde a concepção até a sua retirada de operação. O ciclo de vida fornece uma estrutura de alto nível para a organização do processo de desenvolvimento. A abordagem escolhida direciona o tipo de ciclo de vida que será adotado. Exemplos: sequencial, iterativo e incremental, espiral, ágil.

## 2.3 Processos de Desenvolvimento de Software

Um processo de desenvolvimento de software é uma **sequência detalhada de passos, atividades e práticas** específicas realizadas durante o desenvolvimento de software. Cada processo é projetado para se alinhar com a abordagem e o ciclo de vida adotados, detalhando como os requisitos são elicitados, como o software é projetado, implementado, testado e mantido. Os processos concretizam a abordagem e são executados dentro da estrutura do ciclo de vida escolhido. Os processos de desenvolvimento podem ser categorizados em:

- **Processos abrangentes:** Definem detalhadamente atividades para todas as disciplinas de engenharia de software. Exemplos: Cascata, Modelo V, Rational Unified Process (RUP).
- **Processos com foco em práticas técnicas específicas:** Concentram-se em práticas específicas de engenharia, sem necessariamente cobrir todo o ciclo de vida. Exemplos: eXtreme Programming (XP), Feature-Driven Development (FDD).
- **Processos híbridos/adaptativos:** Combinam elementos de diferentes abordagens para equilibrar estrutura e flexibilidade. Exemplos: DSDM, RAD, Espiral.

## 2.4 Frameworks de Gerenciamento

Um framework de gerenciamento fornece uma **estrutura para organizar, planejar e coordenar** o trabalho em projetos de desenvolvimento, sem necessariamente prescrever práticas técnicas específicas de engenharia. Estes frameworks definem papéis, cerimônias, artefatos e regras que facilitam a organização do trabalho e a colaboração da equipe, mas geralmente não especificam como realizar atividades técnicas como design, codificação ou testes. Na prática, são frequentemente combinados com processos ou práticas técnicas específicas para formar uma abordagem completa de desenvolvimento. Os frameworks de gerenciamento podem ser categorizados em:

- **Frameworks de gerenciamento ágil:** Focados na organização e coordenação do trabalho seguindo princípios ágeis para equipes individuais. Exemplos: Scrum, Kanban.
- **Frameworks de escalabilidade:** Fornecem estruturas abrangentes para coordenar múltiplas equipes ágeis em grandes organizações, combinando elementos de frameworks de gerenciamento com orientações sobre processos técnicos e práticas organizacionais. Exemplos: SAFe e LeSS.

## 2.5 Métodos e Ferramentas

Métodos são **técnicas** específicas utilizadas para realizar uma tarefa dentro de um processo, como programação em pares ou desenvolvimento orientado a testes (TDD). E, ferramentas são **recursos de software ou hardware** que apoiam os métodos e processos, como sistemas de controle de versão, frameworks de teste ou plataformas de desenvolvimento integrado (IDEs).

## 2.6 Relacionamento entre os Conceitos

**Abordagem Informa o Ciclo de Vida:** A abordagem de desenvolvimento de software escolhida (dirigida por plano, ágil ou híbrida) influencia diretamente o tipo de ciclo de vida de desenvolvimento de software que será adotado. Por exemplo, uma abordagem ágil geralmente leva à adoção de um ciclo de vida iterativo e incremental, enquanto uma abordagem dirigida por plano tende a favorecer ciclos de vida mais preditivos.

**Ciclo de Vida Estrutura Processos e Frameworks:** O ciclo de vida de desenvolvimento de software fornece uma estrutura temporal geral dentro da qual tanto os processos de desenvolvimento quanto os frameworks de gerenciamento operam. Por exemplo, um ciclo de vida preditivo estabelece fases sequenciais que estruturam os processos como o modelo Cascata, enquanto um ciclo de vida ágil define iterações curtas que estruturam a aplicação de frameworks como o Scrum.

**Processos Implementam Aspectos Técnicos:** Os processos de desenvolvimento implementam as atividades técnicas específicas necessárias para criar o software, detalhando como os requisitos são elicitados, como o design é realizado, como o código é escrito e testado, e como o sistema é implantado e mantido. Dependendo de sua categoria (abrangentes, com foco em práticas específicas, ou híbridos/adaptativos), eles podem fornecer orientações detalhadas para todas ou apenas algumas dessas atividades.

**Frameworks Organizam e Coordenam o Trabalho:** Os frameworks de gerenciamento fornecem estruturas para organizar equipes, planejar o trabalho, e coordenar a execução das atividades de desenvolvimento. Eles definem papéis, cerimônias, artefatos e regras que facilitam a colaboração e o progresso do projeto, sem necessariamente prescrever as práticas técnicas específicas.

**Complementaridade entre Processos e Frameworks:** Processos e frameworks são complementares e frequentemente usados em conjunto. Um framework como o Scrum pode ser combinado com práticas técnicas de um processo como o XP (programação em pares, TDD) para formar uma abordagem completa ao desenvolvimento. Esta complementaridade permite que as equipes aproveitem as forças de ambos: a orientação no gerenciamento do trabalho proporcionada pelos frameworks e as práticas técnicas específicas fornecidas pelos processos.

**Processos e Frameworks Utilizam Métodos e Ferramentas:** Tanto os processos quanto os frameworks utilizam métodos (técnicas específicas) e ferramentas (recursos de software ou hardware) para realizar suas atividades. Os métodos, como programação em pares,

TDD, ou revisões de código, são aplicados dentro do contexto de processos ou frameworks. Da mesma forma, ferramentas como IDEs, sistemas de controle de versão, ou ferramentas de integração contínua, apoiam a execução de atividades em ambos. A escolha dos métodos e ferramentas é influenciada pelo processo ou framework adotado e deve ser alinhada com a abordagem geral.

**Retroalimentação e Evolução Contínua:** Existe uma retroalimentação contínua entre todos os níveis conceituais. A experiência prática com processos e frameworks, apoiada por métodos e ferramentas específicos, influencia a percepção sobre os ciclos de vida e, eventualmente, pode levar a ajustes nas abordagens adotadas. Esta retroalimentação é um mecanismo importante para a evolução e refinamento das práticas de desenvolvimento de software ao longo do tempo.

**Adaptabilidade Contextual:** Permeando todos estes relacionamentos, a adaptabilidade contextual representa a capacidade de selecionar e combinar elementos de diferentes abordagens, ciclos de vida, processos, frameworks, métodos e ferramentas conforme as necessidades específicas de cada projeto, considerando fatores como domínio de aplicação, tamanho e distribuição da equipe, criticidade do sistema, e cultura organizacional.

A Figura 8 ilustra o relacionamento entre conceitos descritos, anteriormente.

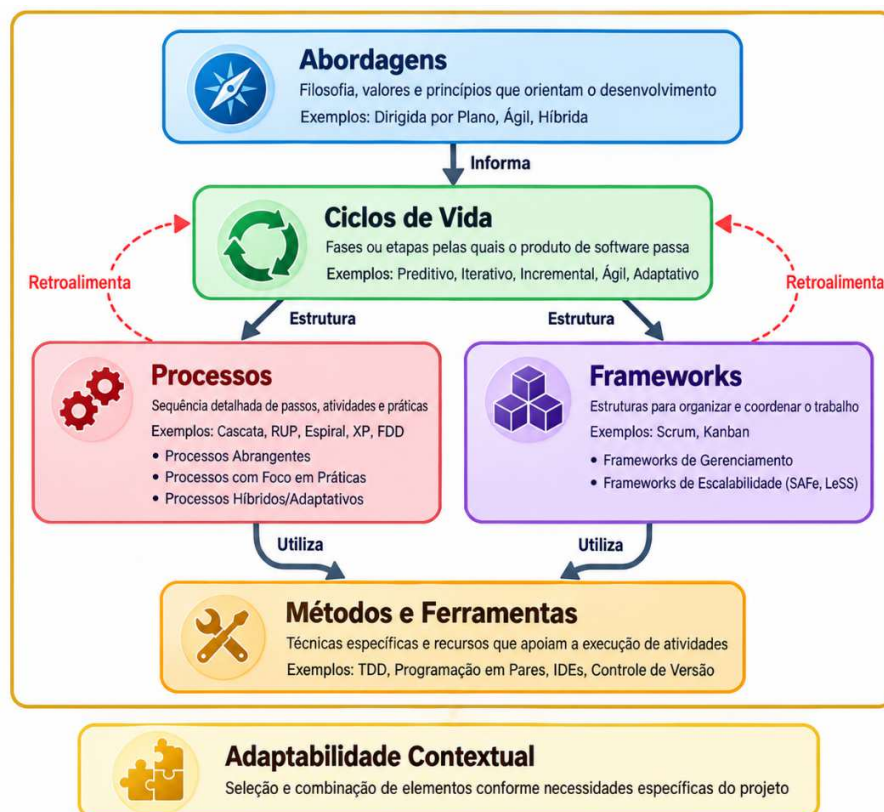


Figura 8 – Relacionamento entre abordagens, ciclos de vida e processos de desenvolvimento de software.  
Fonte: elaborado pelo autor (2025).

Em resumo, as abordagens, os ciclos de vida, os processos de desenvolvimento, os frameworks de gerenciamento e os métodos e ferramentas estão intrinsecamente ligados e se influenciam mutuamente, formando a espinha dorsal da maneira como o software será desenvolvido, entregue e mantido. A escolha consciente e a adaptação desses elementos são cruciais para o sucesso de produtos de software.

Um aspecto fundamental no relacionamento entre todos estes componentes é a **adaptabilidade contextual**. Pesquisas indicam que organizações bem-sucedidas raramente adotam uma única metodologia de forma dogmática, mas desenvolvem capacidade de adaptação metodológica, selecionando e combinando elementos de diferentes abordagens conforme as necessidades específicas de cada projeto (Boehm e Turner, 2003; Kuhrmann et al., 2017; Dingsøyr et al., 2018). A adaptabilidade contextual não é um componente isolado, mas um princípio que permeia todas as decisões relacionadas ao desenvolvimento de software. Ela influencia como as abordagens informam os ciclos de vida, como os ciclos de vida estruturam os processos e frameworks, como processos e frameworks se complementam, e como os métodos e ferramentas são selecionados para apoiar estas atividades. Esta capacidade de ajuste contextual permite que organizações mantenham a coerência filosófica enquanto adaptam suas práticas às particularidades de cada projeto, domínio de aplicação e cultura organizacional.

## 2.7 Elementos Contextuais de Adaptabilidade no Desenvolvimento de Software

### 2.7.1 Particularidades de Cada Projeto

As particularidades de cada projeto referem-se às características e condições específicas que tornam um projeto de software único e que influenciam diretamente as decisões metodológicas (BOEHM; TURNER, 2003). Estas incluem:

- **Escala e complexidade:** O tamanho do sistema, número de usuários esperados, volume de dados, e a complexidade das interações entre os componentes do sistema (SOMMERVILLE, 2019).
- **Criticidade e riscos:** O impacto potencial de falhas (riscos à segurança, financeiros, reputacionais), requisitos de conformidade regulatória, e níveis de segurança necessários (KRUCHTEN, 2013).
- **Estabilidade dos requisitos:** O grau em que os requisitos são conhecidos antecipadamente e a probabilidade de mudanças significativas durante o desenvolvimento (SOMMERVILLE, 2019).
- **Restrições de projeto:** Limitações de recursos (orçamento, tempo, pessoal), tecnologias específicas que devem ser utilizadas, e interdependências com outros sistemas (PROJECT MANAGEMENT INSTITUTE, 2017).
- **Distribuição da equipe:** Localização geográfica dos membros da equipe, fusos horários, e consequências para a comunicação e colaboração (KRUCHTEN, 2013).

- **Perfil dos stakeholders:** Nível de envolvimento dos stakeholders, sua disponibilidade para feedback contínuo, e familiaridade com processos de desenvolvimento de software (BOEHM; TURNER, 2003).

### 2.7.2 Domínio de Aplicação

O domínio de aplicação refere-se ao setor ou área específica para a qual o software está sendo desenvolvido, com seus requisitos e características distintos (EVANS, 2003). Aspectos relevantes incluem:

- **Natureza do domínio:** Características específicas de setores como finanças, saúde, aeroespacial, telecomunicações, jogos, ou comércio eletrônico (JACKSON, 2000).
- **Maturidade tecnológica:** O nível de estabelecimento e estabilidade das tecnologias e padrões no domínio específico (PRESSMAN; MAXIM, 2021).
- **Requisitos regulatórios:** Padrões, certificações e conformidades específicas do setor (como HIPAA na saúde, SOX em finanças, ou DO-178C em software aeronáutico) (LAPLANTE, 2017).
- **Expectativas de qualidade:** Atributos de qualidade prioritários como desempenho, segurança, usabilidade, ou confiabilidade, que variam conforme o domínio (SOMMERVILLE, 2019).
- **Ciclo de evolução:** Velocidade com que tecnologias e expectativas evoluem no domínio, influenciando a frequência necessária de atualizações e adaptações (EVANS, 2003).
- **Terminologia e conceitos:** Vocabulário especializado e modelos conceituais específicos do domínio que impactam a comunicação e a modelagem do sistema (JACKSON, 2000).

### 2.7.3 Cultura Organizacional

A cultura organizacional engloba os valores, crenças, comportamentos e práticas compartilhadas que caracterizam uma organização e influenciam como o trabalho é realizado (SCHEIN, 2016). Elementos relevantes incluem:

- **Estrutura hierárquica:** O grau de hierarquia presente na organização e como as decisões são tomadas (centralizada vs. descentralizada) (IIVARI; IIVARI, 2011).
- **Tolerância a riscos:** A disposição da organização para aceitar incertezas e experimentar novas abordagens ou tecnologias (DIKERT; PAASIVAARA; LASSENUS, 2016).
- **Autonomia das equipes:** O nível de liberdade que as equipes têm para tomar decisões sobre como realizar seu trabalho (BASS; WEBER; ZHU, 2015).
- **Maturidade em processos:** A experiência da organização com diferentes metodologias de desenvolvimento e sua capacidade de adaptação (KUHRMANN et al., 2019).

- **Comunicação:** Padrões e canais de comunicação estabelecidos, transparência de informações e abertura ao feedback (SCHEIN, 2016).
- **Orientação a longo prazo:** O equilíbrio entre resultados rápidos e investimentos em qualidade e sustentabilidade a longo prazo (DINGSØYR et al., 2018).
- **Multidisciplinaridade:** A capacidade e disposição para integrar diferentes funções e especialidades em equipes colaborativas (CONBOY; FITZGERALD, 2010).

A adaptabilidade contextual requer uma compreensão destes três aspectos e a capacidade de calibrar as abordagens, ciclos de vida e processos para criar um alinhamento otimizado entre a metodologia de desenvolvimento e as realidades específicas do projeto, domínio e organização (BOEHM; TURNER, 2003; DIKERT; PAASIVAARA; LASSENIUS, 2016).

## CAPÍTULO 2: PONTOS-CHAVE

- O capítulo estabelece as **bases conceituais fundamentais** para compreender o ecossistema do desenvolvimento de software, distinguindo e conectando quatro conceitos centrais: **abordagens, ciclos de vida, processos e frameworks**.
- As **abordagens de desenvolvimento de software** expressam **filosofias e valores** que orientam como os times concebem e organizam o trabalho. Elas moldam a cultura, a colaboração e as decisões técnicas que sustentam o desenvolvimento.
- Os **ciclos de vida** são estruturas temporais que organizam o projeto em fases ou iterações, definindo o fluxo de atividades — desde a concepção até a entrega e evolução —, podendo assumir naturezas **sequenciais, incrementais ou iterativas**.
- Os **processos de desenvolvimento** descrevem **as atividades, papéis, artefatos e métodos** que operacionalizam as abordagens e ciclos de vida, convertendo princípios abstratos em práticas concretas de engenharia.
- Os **frameworks de gerenciamento** fornecem **estruturas de coordenação e controle**, definindo papéis, cerimônias e artefatos que apoiam a execução e o monitoramento das atividades, sem impor práticas técnicas específicas.
- O texto apresenta também o conceito de **métodos e ferramentas**, que atuam como mecanismos de apoio prático ao processo, contribuindo para a padronização, rastreabilidade e automação de tarefas.
- A **relação entre os conceitos** é enfatizada: abordagens fornecem os princípios orientadores; ciclos de vida estruturam a temporalidade; processos detalham as atividades; e frameworks conectam gestão e execução — formando um **sistema metodológico integrado e interdependente**.
- O capítulo encerra com a discussão sobre **adaptabilidade contextual**, apresentando três dimensões que influenciam a escolha e combinação de práticas:
  - **Particularidades de cada projeto**, como escopo, criticidade e restrições;
  - **Domínio de aplicação**, que define linguagem, padrões e expectativas de qualidade;
  - **Cultura organizacional**, que envolve valores, estrutura, autonomia e maturidade da equipe.

- Conclui-se que **não há uma única forma correta de desenvolver software**, mas sim a necessidade de **ajustar e combinar abordagens, processos e ferramentas conforme o contexto**, promovendo coerência entre método e realidade organizacional;

# Capítulo 3

## 3 Abordagens, Ciclos de Vida e Processos de Desenvolvimento de Software

### 3.1 Abordagens de Desenvolvimento de Software

As abordagens de desenvolvimento de software, conforme representado no topo de nossa estrutura conceitual, estabelecem a filosofia e os princípios que orientam todo o processo de desenvolvimento (SOMMERVILLE, 2019; NERUR; MAHAPATRA; MANGALARAJ, 2005). Elas não existem de forma isolada, mas influenciam diretamente a escolha e implementação dos ciclos de vida e processos. A adaptabilidade contextual permite que organizações selecionem a abordagem mais adequada para um determinado projeto ou componente, ou mesmo combinem elementos de diferentes abordagens.

#### 3.1.1 Dirigidas por Plano

Nas abordagens de desenvolvimento de software dirigidas por plano, a ênfase está no **planejamento e na previsibilidade**. Elas seguem, em geral, uma sequência linear e faseado de atividades, como exemplificado pelo modelo em cascata (ROYCE, 1970; DAVIS; BERSOFF; COMER, 1988). O projeto começa com foco nas fases de planejamento e requisitos, seguidas por design, implementação, teste e manutenção, sendo que cada fase possui objetivos específicos e deve ser concluída antes que a próxima seja iniciada.

Para que as abordagens dirigidas por plano sejam eficazes, é necessário que os requisitos de software sejam identificados de forma abrangente ou quase completa no início do projeto. Ressalta-se que é imprescindível que os **requisitos do produto** a ser construído sejam **estáveis e conhecidos**, caso contrário a utilização de uma abordagem dirigida por plano pode se tornar inadequada. A partir disso, o conjunto de requisitos gera insumo para o **planejamento detalhado**, incluindo o dimensionamento da equipe, a identificação de pré-requisitos técnicos, riscos, restrições, estimativas de custos, prazos e planos de entregas (LYCETT et al., 2003; PRESSMAN; MAXIM, 2021).

Neste contexto, as abordagens dirigidas por plano tendem a ser menos adequadas para o desenvolvimento rápido de software, pois envolvem **mais planejamento, requisitos estáveis e formalismos** em comparação com as abordagens ágeis. Por exemplo, mudanças nos requisitos devem ser avaliadas e aprovadas formalmente, podendo afetar custos e prazos. Nesse caso, os ajustes nos requisitos devem ser refletidos, posteriormente, nas disciplinas de análise/design, construção e teste, podendo gerar retrabalho e sobrecarga para a equipe.

Embora as abordagens dirigidas por plano possam gerar maior esforço de planejamento, desenvolvimento e documentação do software, esse esforço é justificado, por exemplo, quando vários times de desenvolvimento precisam ser coordenados, quando o produto de software é crítico, quando muitas pessoas estarão envolvidas na manutenção do software ao



longo de sua vida útil, ou quando não é viável uma comunicação constante entre os envolvidos.

Além disso, para alguns tipos de produtos, como sistemas de controle críticos em segurança, sistemas aeronáuticos e sistemas de UTI, onde uma análise completa do sistema é essencial e os requisitos são bem compreendidos, estáveis e formais, a abordagem dirigida por plano pode ser a mais adequada. O controle de mudanças é formal e estruturado, tornando o processo menos flexível, o que pode ser benéfico em ambientes onde a **estabilidade** e a **conformidade com regulamentações** são prioritárias.

As abordagens dirigidas por plano, baseiam-se em uma filosofia que enfatiza a previsibilidade, a estrutura e a documentação detalhada em todas as fases do desenvolvimento de software. Essas abordagens são caracterizadas por uma série de valores e princípios fundamentais que orientam o processo de desenvolvimento, destacando-se os seguintes:

### Valores

- **Estabilidade:** valoriza-se a estabilidade do escopo do projeto e a minimização de mudanças, uma vez que as alterações são vistas como fontes potenciais de risco e incerteza.
- **Segurança:** acredita-se que um planejamento detalhado e uma documentação abrangente proporcionam uma base segura para o desenvolvimento e a manutenção futura do software.
- **Previsão:** valoriza-se a capacidade de prever resultados, custos e cronogramas com precisão, baseando-se na ideia de que uma compreensão completa do projeto desde o início pode levar a uma execução mais suave.

### Princípios

- **Previsibilidade e Planejamento:** acredita-se que o sucesso do projeto de software depende de um planejamento detalhado e abrangente. As tarefas, recursos e cronogramas são definidos antecipadamente para minimizar riscos e incertezas.
- **Estrutura Faseada:** o desenvolvimento é estruturado em fases sequenciais, onde cada fase deve ser completada antes de se avançar para a próxima. Exemplos típicos de fases incluem requisitos, design, implementação, teste e manutenção.
- **Documentação Abrangente:** grande ênfase é colocada na criação de documentação detalhada em cada fase do desenvolvimento para garantir que todos os aspectos do sistema sejam completamente especificados e compreendidos.
- **Controle Rigoroso:** mantém-se um controle rigoroso sobre o processo de desenvolvimento, com revisões e aprovações em cada etapa, para assegurar que o projeto permaneça alinhado com o plano inicial.
- **Gerenciamento de Mudanças Formal:** mudanças nos requisitos ou no projeto são gerenciadas de forma formal e criteriosa, muitas vezes exigindo revisão e aprovação através de um processo de controle de mudanças.

Embora as abordagens dirigidas por plano tenham sido amplamente utilizadas e sejam adequadas para projetos com requisitos bem definidos e estáveis, elas enfrentam críticas por sua rigidez e dificuldade em acomodar mudanças.

### 3.1.2 Ágeis

As abordagens ágeis, em contraste com as abordagens dirigidas por plano, baseiam-se em princípios de **flexibilidade, colaboração e resposta rápida a mudanças** (COCKBURN; HIGHSMITH, 2001; DYBÅ; DINGSØYR, 2008). O framework Scrum e o processo XP (Extreme Programming), por exemplo, destacam o desenvolvimento iterativo e incremental, em que o projeto é dividido em pequenas iterações, cada uma resultando em um incremento do software.

A ênfase na **comunicação contínua e colaboração** com o cliente é fundamental nas abordagens ágeis, garantindo que o produto final atenda às necessidades e expectativas do cliente, mesmo que essas necessidades mudem ao longo do projeto. A documentação é mantida enxuta, concentrando-se no essencial e priorizando o software funcional.

Neste sentido, as abordagens ágeis são adequadas, por exemplo, em produtos de software voltados para negócios, em **ambientes instáveis e sujeitos a mudanças constantes**, valorizando processos de desenvolvimento e **entregas rápidas**. Esses processos intercalam a especificação, projeto e implementação, evitando especificações detalhadas do produto e minimizando a documentação do projeto. Os requisitos do usuário são definidos de forma resumida, contendo apenas as características mais importantes do produto de software.

Ao falarmos de abordagens ágeis é mandatório observar e materializar os valores e princípios registrados no Manifesto Ágil, o qual foi formulado em 2001 por 17 profissionais de software que buscavam uma maneira mais eficiente e flexível de desenvolver softwares (BECK et al., 2001; FOWLER; HIGHSMITH, 2001). O Manifesto estabelece quatro valores fundamentais e doze princípios que guiam as abordagens ágeis de desenvolvimento de software.

#### Valores Ágeis

- **Indivíduos e interações** mais que processos e ferramentas: valoriza as pessoas que fazem o trabalho e como elas trabalham juntas mais do que os processos ou ferramentas usadas.
- **Software funcionando** mais que documentação abrangente: dá mais valor ao desenvolvimento de software funcional do que à criação de documentação extensa.
- **Colaboração com o cliente** mais que negociação de contratos: prioriza a colaboração contínua com o cliente ao longo do projeto, em vez de depender de negociações contratuais rígidas.
- **Responder a mudanças** mais que seguir um plano: valoriza a capacidade de adaptar-se a mudanças ao longo do desenvolvimento do projeto, em vez de seguir rigidamente um plano estabelecido.

### Princípios Ágeis

1. Priorizar a satisfação do cliente através da entrega contínua de software valioso.
2. Aceitar mudanças nos requisitos, mesmo em fases tardias do desenvolvimento.
3. Entregar software funcional frequentemente, com preferência para períodos mais curtos.
4. Promover o trabalho conjunto diário entre profissionais de negócios e desenvolvedores.
5. Construir projetos em torno de indivíduos motivados, fornecendo o ambiente e suporte necessários.
6. A comunicação face a face é o método mais eficiente e eficaz de transmitir informações.
7. O software funcional é a principal medida de progresso.
8. Manter um ritmo sustentável de trabalho para a equipe.
9. Dar atenção contínua à excelência técnica e ao bom design.
10. Simplicidade — a arte de maximizar a quantidade de trabalho não realizado — é essencial.
11. As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizáveis.
12. Em intervalos regulares, a equipe reflete sobre como se tornar mais eficaz, ajustando-se conforme necessário.

Os valores e princípios ágeis destacam a importância da flexibilidade, colaboração, foco no cliente e entrega contínua de valor. Eles formam a base para diversos processos e frameworks ágeis, como Extreme Programming (XP), Scrum e Kanban, cada um aplicando esses valores e princípios de maneira que se adaptem ao contexto e aos objetivos específicos do projeto de software.

#### 3.1.3 Dirigido por Plano e Ágeis

Enquanto as abordagens ágeis são ideais para projetos em ambientes dinâmicos, onde os requisitos podem evoluir ou não ser completamente conhecidos desde o início, as abordagens de desenvolvimento dirigidas por plano priorizam processos mais formais e uma abordagem mais planejada (GLASS, 2001; SOMMERVILLE, 2019).

A seguir, a Quadro 2, oferece uma visão geral de como abordagens dirigidas por plano e ágeis, lidam com algumas características do desenvolvimento de software.

| Característica                 | Abordagens Dirigidas por Plano                                                                                                                | Abordagens Ágeis                                                 |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Planejamento</b>            | Detalhado no início do projeto, com ênfase na previsibilidade. Modelos iterativos e incrementais adaptam o planejamento ao longo do projeto.  | Adaptativo e contínuo, com planos evoluindo ao longo do projeto. |
| <b>Mudanças nos Requisitos</b> | Geralmente resistente a mudanças após a fase inicial. Processo formal para solicitação e implementação de mudanças em modelos mais flexíveis. | Acolhe mudanças a qualquer momento do projeto.                   |

| Característica                   | Abordagens Dirigidas por Plano                                                                                                                        | Abordagens Ágeis                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Entrega do Produto</b>        | Tradicionalmente entrega única, mas também podem permitir entregas incrementais.                                                                      | Entregas incrementais e iterativas ao longo do projeto.                    |
| <b>Foco</b>                      | Na previsibilidade, controle e documentação detalhada.                                                                                                | Na colaboração com o cliente, adaptabilidade e resposta rápida a mudanças. |
| <b>Comunicação</b>               | Formal, com documentação extensiva.                                                                                                                   | Direta e frequente, com menos ênfase na documentação.                      |
| <b>Testes</b>                    | Realizados após a conclusão do desenvolvimento.                                                                                                       | Contínuos, desde o início do projeto.                                      |
| <b>Gestão de Riscos</b>          | Avaliada inicialmente, com tentativa de mitigação antecipada. Quando iterativos e incrementais permitem gestão contínua e adaptativa de riscos.       | Gerenciamento contínuo e adaptativo de riscos.                             |
| <b>Flexibilidade</b>             | Limitada nos modelos sequenciais; maior em abordagens iterativas e incrementais.                                                                      | Alta, permitindo rápida adaptação a novas informações.                     |
| <b>Colaboração com o Cliente</b> | Geralmente limitada a etapas específicas do projeto.                                                                                                  | Constante e fundamental para o desenvolvimento.                            |
| <b>Eficiência e Velocidade</b>   | Varia; pode ser lenta quando sequencial; modelos iterativos e incrementais podem melhorar a eficiência e a velocidade.                                | Geralmente mais alta devido à natureza iterativa e ao feedback contínuo.   |
| <b>Gerenciamento de Projetos</b> | Baseado em tarefas e marcos predeterminados.                                                                                                          | Baseado em prioridades dinâmicas e resultados.                             |
| <b>Adaptação a Mudanças</b>      | Processo formal para solicitação e implementação de mudanças em modelos mais rígidos; modelos mais flexíveis permitem adaptações mais ágeis.          | Mudanças são parte integrante do processo.                                 |
| <b>Visibilidade do Progresso</b> | Por meio de marcos e entregas de documentação em abordagens mais sequenciais; modelos iterativos e incrementais podem oferecer visibilidade contínua. | Através de demonstrações regulares de software funcional.                  |
| <b>Cultura e Mentalidade</b>     | Enfatiza a conformidade e seguimento de processos.                                                                                                    | Valoriza a inovação, colaboração e adaptabilidade.                         |
| <b>Estrutura do Processo</b>     | Linear e faseado nos modelos cascata; iterativo e incremental nos modelos adaptativos.                                                                | Iterativo e incremental.                                                   |
| <b>Documentação</b>              | Abrangente e detalhada.                                                                                                                               | Enxuta, focada no essencial.                                               |
| <b>Envolvimento do Cliente</b>   | Principalmente nas fases iniciais e finais.                                                                                                           | Contínuo e colaborativo.                                                   |

Quadro 2 – Características de abordagens dirigidas por plano e ágeis.  
Fonte: elaborado pelo autor (2025).

O Quadro 2, consolidada oferece uma visão abrangente das diferenças e semelhanças entre abordagens dirigidas por plano e ágeis, destacando como cada uma lida com várias

características do desenvolvimento de software. A inclusão de modelos iterativos e incrementais nas abordagens dirigidas por plano reconhece a diversidade dentro dessa categoria, mostrando que a flexibilidade e adaptação não são exclusivas das abordagens ágeis.

### 3.1.4 Abordagens Híbridas: Combinando o Melhor de Dois Mundos

Em resposta às limitações tanto das abordagens puramente dirigidas por plano quanto das puramente ágeis, as organizações estão cada vez mais adotando abordagens híbridas que combinam elementos de ambas as filosofias (BOEHM; TURNER, 2003). Estudos empíricos recentes confirmam esta tendência, revelando que a maioria das organizações na prática implementa abordagens híbridas personalizadas, combinando elementos metodológicos de diferentes fontes (KUHRMANN et al., 2019; THEOCHARIS et al., 2015). Estas abordagens representam uma evolução no pensamento sobre desenvolvimento de software, reconhecendo que a dicotomia simplista entre “tradicional” e “ágil” não serve adequadamente às complexidades dos projetos modernos.

As abordagens híbridas operam primordialmente no nível de princípios e filosofia organizacional. Elas estabelecem um conjunto de valores e orientações que reconhecem a necessidade de equilíbrio entre:

- Planejamento e adaptabilidade
- Documentação formal e comunicação informal
- Processos estruturados e flexibilidade situacional
- Controle de governança e autonomia das equipes

As abordagens híbridas são caracterizadas por uma adaptação contextual, onde o nível de planejamento antecipado e a flexibilidade são calibrados de acordo com as especificidades de cada componente do projeto (CONBOY; FITZGERALD, 2010; BATRA; XIA; ZHANG, 2017). Por exemplo, em um sistema de software crítico, os componentes essenciais de segurança podem seguir uma abordagem mais dirigida por plano, com documentação detalhada e verificação rigorosa, enquanto a interface do usuário pode ser desenvolvida usando métodos ágeis para permitir iteração rápida com base no feedback dos usuários (WEST; GRANT, 2010).

#### Valores e Princípios Híbridos:

- Adaptação Contextual: Reconhecimento de que diferentes partes do projeto podem exigir diferentes níveis de formalidade e flexibilidade (CONBOY; FITZGERALD, 2010).
- Equilíbrio Pragmático: Busca do equilíbrio entre planejamento antecipado e adaptabilidade, baseando-se em avaliação contínua de riscos e incertezas (BOEHM; TURNER, 2003).
- Comunicação Multifacetada: Combinação de comunicação formal (documentação, processos estruturados) e informal (colaboração direta, feedback contínuo) conforme apropriado (BATRA; XIA; ZHANG, 2017).

- Governança Escalonada: Adaptação dos mecanismos de governança e controle de acordo com a criticidade e o nível de risco de cada componente (DIKERT; PAASIVAARA; LASSENIUS, 2016).

Esta filosofia híbrida manifesta-se em diferentes aspectos do desenvolvimento de software:

- Na estruturação de ciclos de vida (como veremos na seção sobre Ciclos de Vida Adaptativos)
- Na seleção e customização de processos
- Nas práticas de gestão de projetos
- Na organização das equipes e estruturas organizacionais

Frameworks como SAFe (Scaled Agile Framework) (SCALED AGILE, 2023), Disciplined Agile Delivery (DAD) (AMBLER; LINES, 2012) e Bimodal IT (GARTNER, 2014) representam tentativas estruturadas de implementar abordagens híbridas, oferecendo orientações sobre como combinar elementos dirigidos por plano e ágeis em projetos de grande escala ou em organizações com requisitos regulatórios rigorosos.

As abordagens híbridas, como filosofias e conjuntos de princípios, se manifestam concretamente através dos ciclos de vida adaptativos (seção 4.1.6), que implementam esta flexibilidade contextual no planejamento temporal dos projetos. Se as abordagens híbridas representam 'o quê' e 'por quê', os ciclos adaptativos mostram 'como' e 'quando' essa hibridização ocorre no desenvolvimento de software (KUHRMANN, M. et al., 2019).

### 3.1.5 Como lidar com requisitos de software: Dirigido por Plano, Ágeis e Híbridos

Ao analisar as abordagens dirigidas por plano, as ágeis e as híbridas no desenvolvimento de software, observamos diferenças fundamentais na maneira como cada uma delas lida com requisitos, recursos, prazos e o triângulo de ferro (escopo, tempo e custo). A compreensão dessas diferenças é crucial para selecionar a metodologia mais adequada para um projeto específico, considerando as necessidades dos stakeholders e o ambiente de negócios (SOMMERVILLE, 2019). Pesquisas empíricas recentes demonstram que estas diferenças fundamentais levam a desafios distintos na prática, especialmente em termos de documentação, validação e gerenciamento de mudanças de requisitos (RAMESH; CAO; BASKERVILLE, 2010; INAYAT et al., 2015).

As **abordagens dirigidas** por plano são caracterizadas por um planejamento detalhado e abrangente no início do projeto. Nesse modelo, os requisitos são elicitados, analisados e fixados antecipadamente, servindo como base para estimativas de recursos e prazos (PRESMAN; MAXIM, 2021). Essa abordagem busca minimizar mudanças posteriores, enfatizando a previsibilidade e o controle. Os requisitos estabilizados mais cedo, e qualquer alteração após a fase inicial é gerenciada por meio de um processo formal de controle de mudanças (POHL, 2010). No contexto do triângulo de ferro, o escopo é definido claramente e bloqueado no início, enquanto tempo e custo são estimados com base nesse escopo fixo (LEFFINGWELL, 2011).

Por outro lado, as **abordagens ágeis** tratam os requisitos de maneira flexível e adaptativa. Nesse modelo, os requisitos são capturados como histórias de usuário e são continuamente refinados e priorizados (COHN, 2004). O planejamento é adaptativo, com planos evoluindo ao longo do projeto com base no feedback dos usuários e nas necessidades emergentes. Os recursos e prazos são estabelecidos de forma fixa em sprints ou iterações, e os requisitos são priorizados dentro dessas limitações (RUBIN, 2022). Isso promove a entrega contínua de valor, permitindo ajustes rápidos e eficientes às mudanças. No triângulo de ferro, a abordagem ágil prioriza a flexibilidade no escopo para maximizar o valor entregue ao cliente, enquanto gerencia tempo e custo de maneira a suportar essa flexibilidade (COHN, 2006).

Ao observarmos as características das abordagens dirigidas por plano e ágeis, é possível verificar que a maneira como cada uma delas lida com os requisitos de software é distinta (Figura 9). Nas abordagens dirigidas por plano (Figura 9, lado esquerdo), os requisitos guiam o planejamento de recursos e prazos, ou seja, a partir do conjunto de requisitos identificados são estimados os recursos (pessoas, ferramentas, custos) necessários para o desenvolvimento do produto, assim como os prazos de entrega. Por outro lado, nas abordagens ágeis (Figura 9, lado direito) quem guia o desenvolvimento são os recursos e prazos. Desta forma, são priorizados o conjunto de requisitos passíveis de desenvolvimento e entrega dentro do prazo e com os recursos (humanos, ferramentas, custos), inicialmente, estabelecidos (LEFFINGWELL, 2011).

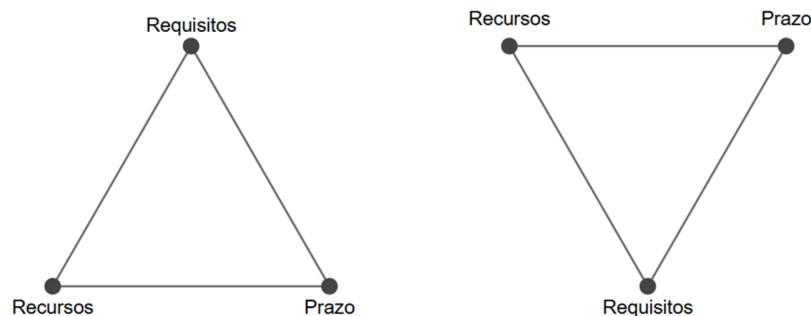


Figura 9 – Perspectiva de Requisitos, dirigido a plano e ágil.  
Fonte: LEFFINGWELL (2011), adaptado.

Conforme representado na Figura 10, nas abordagens dirigidas a plano, os requisitos de software são fixos (estáveis), e nas abordagens ágeis, são variáveis (instáveis). Quanto aos recursos e prazos, são variáveis (estimados) nas abordagens dirigidas a plano, e fixos, nas abordagens ágeis (LEFFINGWELL, 2011). Nas abordagens híbridas, tanto requisitos quanto recursos e prazos podem variar em um espectro de fixo a variável, dependendo do contexto específico, da criticidade do componente e da fase do projeto (SCHWABER; SUTHERLAND, 2020; KNAPP; ZERATSKY; KOWITZ, 2016).

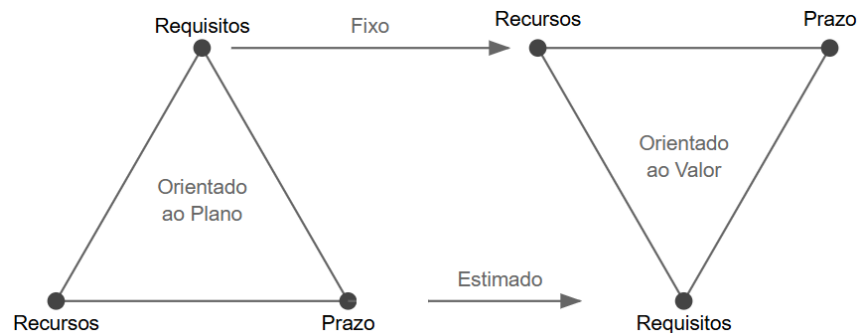


Figura 10 - Perspectiva de Requisitos, fixo e estimado.  
Fonte: LEFFINGWELL (2011), adaptado.

As **abordagens híbridas**, por sua vez, adotam uma postura intermediária e contextual no tratamento dos requisitos (DICK; HULL; JACKSON, 2017). Nestas abordagens, os requisitos são categorizados e gerenciados de acordo com sua natureza, criticidade e estabilidade. Reconhecendo que nem todos os requisitos têm o mesmo nível de incerteza ou impacto, as abordagens híbridas permitem que diferentes tipos de requisitos sejam tratados com níveis variáveis de rigor e formalidade (LAPLANTE; KASSAB, 2022).

Em um modelo híbrido, os requisitos fundamentais e arquiteturais que afetam múltiplos componentes do sistema podem ser documentados e gerenciados com práticas mais formais e dirigidas por plano, enquanto requisitos de menor impacto ou maior volatilidade podem ser tratados com práticas mais ágeis (VAN LAMSWEERDE, 2009). Esta estratificação permite balancear a estabilidade necessária para componentes críticos com a flexibilidade para adaptação em áreas onde a descoberta e o feedback frequente são mais valiosos (BOEHM; TURNER, 2003).

O tratamento de requisitos em abordagens híbridas frequentemente envolve:

1. **Documentação Multi-nível:** Uso de diferentes níveis de formalidade na documentação, desde especificações detalhadas para componentes críticos até user stories mais flexíveis para funcionalidades experimentais (POHL, 2010; LAUESEN, 2002).
2. **Rastreabilidade Seletiva:** Implementação de rastreabilidade robusta para requisitos arquiteturais e de maior risco, enquanto mantém processos mais leves para requisitos de menor impacto (SOMMERVILLE, 2019; LAUESEN, 2002).
3. **Priorização Contextual:** Utilização de diferentes técnicas de priorização dependendo da natureza do requisito - análise custo-benefício formal para requisitos estruturais versus técnicas mais colaborativas como votação para funcionalidades de experiência do usuário (MATHEW J., 2017; WIEGERS; BEATTY, 2013).
4. **Gestão de Mudanças Escalonada:** Processos de controle de mudanças que variam em formalidade de acordo com o impacto da mudança proposta na arquitetura, segurança ou domínios regulatórios (SOMMERVILLE, 2019; PRESSMAN; MAXIM, 2021).



No contexto de frameworks híbridos como o SAgile (Scaled Agile Framework), os requisitos são organizados em múltiplos níveis, desde estratégicos (Epics) até táticos (Features) e operacionais (Stories). Esta estruturação permite que diferentes horizontes de planejamento coexistam, com requisitos de nível superior fornecendo estabilidade e direção, enquanto requisitos de nível inferior permanecem adaptáveis e responsivos ao feedback (SCALED AGILE, 2023).

Ao integrar essas observações, fica evidente que a escolha entre abordagens dirigidas por plano, ágeis e híbridas depende da natureza do projeto, do domínio de aplicação e da cultura organizacional (DINGSØYR et al., 2018). Projetos em ambientes estáveis, com requisitos bem definidos e pouco sujeitos a mudanças, podem se beneficiar da previsibilidade e controle oferecidos pelas abordagens dirigidas por plano. Projetos em ambientes dinâmicos, onde os requisitos podem mudar frequentemente, são melhor atendidos pela adaptabilidade e pela capacidade de resposta rápida das abordagens ágeis. Enquanto isso, projetos complexos com componentes de diferentes níveis de incerteza e criticidade podem se beneficiar de abordagens híbridas que equilibram estabilidade e flexibilidade de acordo com o contexto específico (PRESSMAN; MAXIM, 2021; BOEHM; TURNER, 2003).

Cada abordagem tem suas vantagens e desafios, e a decisão deve considerar os objetivos do projeto, as expectativas dos stakeholders e as características do ambiente de desenvolvimento (SOMMERVILLE, 2019). A compreensão clara dos princípios, valores e práticas subjacentes a cada abordagem permite às equipes de desenvolvimento escolher a metodologia que melhor alinha a estratégia de desenvolvimento com as necessidades do projeto, maximizando as chances de sucesso (PRESSMAN; MAXIM, 2021).

As abordagens híbridas, por sua vez, buscam um equilíbrio dinâmico entre essas duas perspectivas (RUBIN, 2022; KRUCHTEN, 2023). Em componentes críticos do sistema ou em domínios altamente regulamentados, os requisitos podem ser tratados de maneira mais próxima ao modelo dirigido por plano, com documentação detalhada e processos formais de gestão de mudanças. Paralelamente, em componentes voltados à experiência do usuário ou em áreas de maior inovação, os requisitos podem ser gerenciados de forma mais ágil, permitindo emergir e evoluir durante o desenvolvimento (COHN, 2006; BROWN, 2009).

Esta flexibilidade contextual é particularmente valiosa em grandes projetos ou produtos que abrangem múltiplos domínios, onde diferentes componentes podem beneficiar-se de diferentes abordagens ao gerenciamento de requisitos (DICK; HULL; JACKSON, 2017; SCALED AGILE, 2023). Por exemplo, em um sistema médico, os requisitos relacionados à segurança do paciente podem seguir uma abordagem mais dirigida por plano, enquanto a interface do usuário pode ser desenvolvida seguindo práticas mais ágeis (LAPLANTE; KASSAB, 2022).

Em resumo, as abordagens híbridas oferecem um modelo adaptativo para o gerenciamento de requisitos, permitindo que as organizações calibrem o nível de formalidade e flexibilidade de acordo com as necessidades específicas de cada contexto (BOEHM; TURNER, 2003). Esta capacidade de adaptar práticas de engenharia de requisitos às características únicas de cada componente ou fase do projeto representa uma evolução natural na disciplina,

reconhecendo que a complexidade dos sistemas modernos frequentemente exige uma combinação de diferentes filosofias e práticas (LAPLANTE; KASSAB, 2022; SOMMERVILLE, 2019).

A Figura 11 ilustra como cada uma das abordagens lida com os requisitos de software.

Estas diferentes perspectivas (dirigida por plano, ágeis e híbridas) sobre o tratamento de requisitos refletem diretamente como as abordagens, filosofias e conjuntos de valores, informam as práticas concretas de engenharia de requisitos. A adaptabilidade contextual permite que as organizações selecionem e combinem elementos dessas diferentes perspectivas com base na natureza do projeto, na maturidade da equipe, nas expectativas dos stakeholders e nos riscos específicos envolvidos. Esta flexibilidade metodológica, quando aplicada com discernimento, potencializa a eficácia da engenharia de requisitos em diversos contextos.



Figura 11 – Tratamento dos Requisitos de Software, por cada abordagem.  
Fonte: elaborado pelo autor (2025).

### 3.2 Ciclos de Vida

Os seres humanos passam por 5 fases sequenciais na vida (Figura 12), que são: bebê, criança, adolescente, adulto e idoso. Elas ocorrem dentro do ciclo da vida que possui dois grandes eventos: o nascimento e a morte.

### Evolução Humana: Da Infância à Velhice

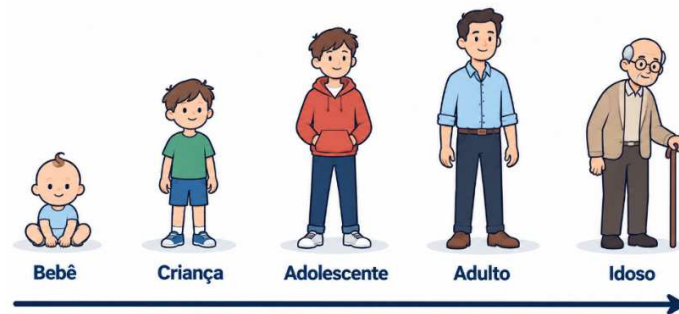


Figura 12 – Evolução Humana.  
Fonte: elaborado pelo autor (2025).

#### 3.2.1 Ciclo de vida de Software

Assim como os seres humanos um software também possui um ciclo de vida, passando desde a fase de definição dos seus requisitos, codificação, manutenção, até a sua descontinuidade. Porém diferente dos seres humanos, na engenharia de software é possível termos ciclos de vida com fases sendo executadas em paralelo e de forma cíclica.

Os ciclos de vida ocupam a posição intermediária em nossa estrutura conceitual, sendo informados pelas abordagens e, por sua vez, estruturando os processos específicos. A adaptabilidade contextual permite que as organizações avaliem qual ciclo de vida é mais apropriado para um projeto específico, considerando fatores como estabilidade dos requisitos, tolerância a riscos, e necessidades de entrega. Em alguns casos, diferentes ciclos de vida podem ser aplicados a diferentes componentes de um mesmo sistema, refletindo a natureza adaptativa do desenvolvimento moderno.

A seguir, são apresentados alguns ciclos de vida para o desenvolvimento de um produto de software, sendo: preditivo, iterativo, incremental, iterativo e incremental, ágil e híbrido. A Figura 13 ilustra de forma geral a perspectiva dos ciclos de vida, a partir da frequência de entrega e grau de mudança.

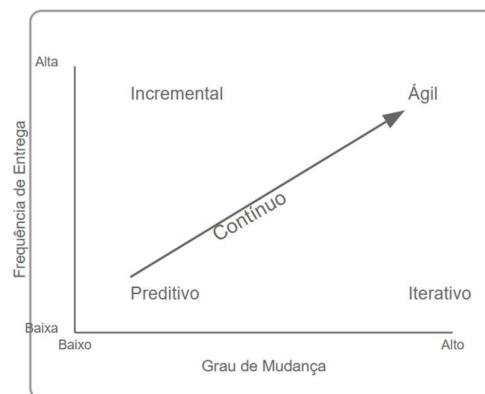


Figura 13 – Ciclos de vida versus Frequência de Entrega e Grau de Mudança  
Fonte: Project Management Institute (2017), adaptado.

### 3.2.1.1 Ciclo de vida preditivo

Este é um dos ciclos de vida mais antigos e simples, onde o processo de desenvolvimento segue uma **progressão linear e sequencial por meio de fases** como requisitos, design, construção, teste, entrega e manutenção (Figura 14). Esse ciclo de vida, não permite sem sobreposição, paralelismo ou ciclos, entre as fases. Assim, uma vez que uma fase é concluída, o desenvolvimento passa para a próxima fase sem retornar à fase anterior. Esse é um ciclo de vida adequado para projetos com **requisitos** bem definidos e pouco propensos a mudanças, ou seja, **conhecidos e estáveis**, os quais ajudam a diminuir as incertezas e complexidade permitindo que as equipes segmentem o trabalho em uma sequência de agrupamentos previsíveis. Neste ciclo de vida a entrega do produto de software é realizada apenas ao final da realização de todas as fases.

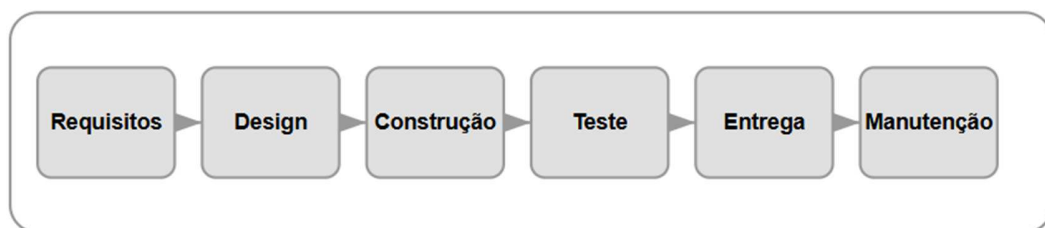


Figura 14 – Ciclo de vida preditivo.

Fonte: Project Management Institute (2017), adaptado.

### 3.2.1.2 Ciclo de vida iterativo

O ciclo de vida iterativo (Figura 15) refere-se à **repetição de atividades de desenvolvimento em ciclos**, permitindo refinamento e ajustes baseados no aprendizado e feedback sobre o trabalho parcialmente concluído ou inacabado para melhorar e modificar esse trabalho. Esse ciclo pode ser combinado com diferentes focos de execução, como prototipação, evolução, incrementos.

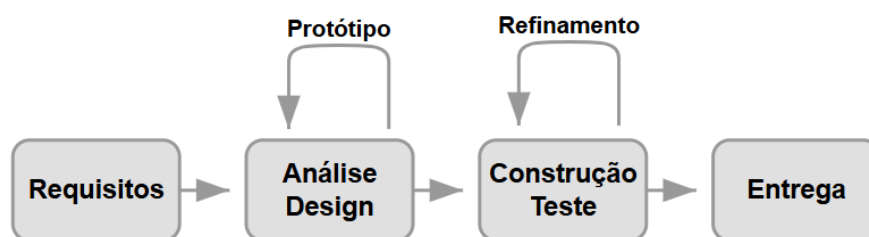


Figura 15 – Ciclo de vida iterativo.

Fonte: Project Management Institute (2017), adaptado.

### 3.2.1.3 Ciclo de vida incremental

Um ciclo de vida incremental **entrega vários incrementos** (partes funcionais) (Figura 16) de um produto de software, ao longo do seu desenvolvimento, onde cada uma delas já pode ser passível de utilização, em produção, por seus usuários finais. Desta forma, o produto de software vai sendo construído progressivamente até a sua conclusão.

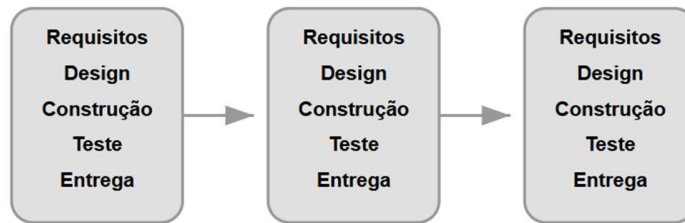


Figura 16 – Ciclo de vida incremental.  
Fonte: Project Management Institute (2017), adaptado.

#### 3.2.1.4 Ciclo de vida iterativo e incremental

Um ciclo de vida iterativo e incremental desenvolvimento de software enfatiza a realização do **trabalho em ciclos sucessivos** (iterações). Cada iteração produz **uma versão incrementalizada do software**, adicionando novas funcionalidades de forma gradual até que o produto final esteja completo, englobando as características de dois ciclos de vida:

- Iterativo: repetição de atividades de desenvolvimento em ciclos, permitindo refinamento e ajustes baseados no aprendizado e feedback.
- Incremental: cada iteração entrega uma parte do produto, construindo progressivamente até a conclusão do produto.

Como exemplo, de um ciclo de vida iterativo e incremental, a Figura 17 representa a organização das fases e iterações do processo unificado.

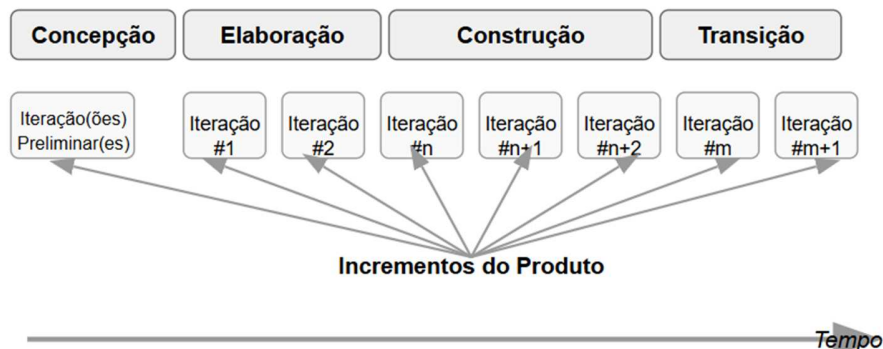


Figura 17 – Fases do Processo Unificado.  
Fonte: KROLL e MACISAAC (2006), adaptado.

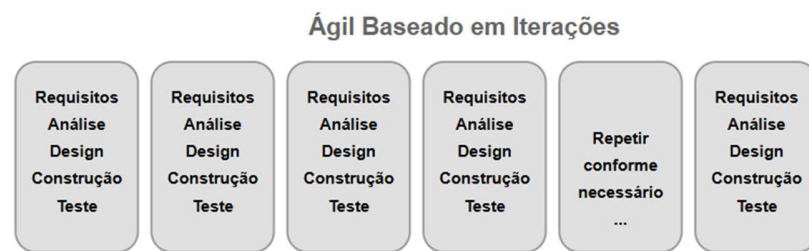
#### 3.2.1.5 Ciclo de vida ágil

O ciclo de vida ágil inclui e supõe ciclos de vida **iterativos e incrementais** como parte de sua execução, enfatizando a comunicação e colaboração constantes entre desenvolvedores e stakeholders, e valorizando a entrega contínua de partes funcionais do software ao cliente para feedback rápido. Além de ser iterativo e incremental, o ágil incorpora um conjunto de práticas e valores específicos destinados a melhorar a eficiência do desenvolvimento e a satisfação do cliente. Diante de um ciclo de vida ágil, a equipe obtém feedback antecipado e fornece visibilidade, confiança e controle do produto ao cliente.

É importante destacar que enquanto todo desenvolvimento ágil é iterativo e incremental, nem todo desenvolvimento iterativo e incremental adota os valores, princípios ou práticas

específicas associadas às abordagens ágeis. O ágil é um superset que inclui um ciclo de vida iterativo e incremental como um de seus componentes essenciais, juntamente com uma ênfase em princípios e práticas adicionais que promovem uma maior adaptabilidade e colaboração. A Figura 18 ilustra os ciclos (sprints) ágeis, nos quais as disciplinas da engenharia de software são repetidamente realizadas, gerando uma entrega de software funcional ao final de cada um deles.

Por fim, a Figura 19, apresenta de maneira resumida as características de cada um dos ciclos de vida.



NOTA: Cada timebox tem o mesmo tamanho. Cada timebox resulta em funcionalidades testadas e funcionais.

Figura 18 – Ciclo de vida ágil.

Fonte: Project Management Institute (2017), adaptado.

| Características |            |                                        |                              |                                                     |
|-----------------|------------|----------------------------------------|------------------------------|-----------------------------------------------------|
| Abordagem       | Requisitos | Atividades                             | Entrega                      | Objetivo                                            |
| Preditivo       | Fixos      | Executadas uma vez para todo o projeto | Entrega única                | Gerenciar custo                                     |
| Iterativo       | Dinâmicos  | Repetidas até estarem corretas         | Entrega única                | Correção da solução                                 |
| Incremental     | Dinâmicos  | Executadas uma vez por incremento      | Entregas menores frequentes  | Velocidade                                          |
| Ágil            | Dinâmicos  | Repetidas até estarem corretas         | Entregas pequenas frequentes | Valor ao cliente via entregas frequentes e feedback |

Figura 19 – Características dos ciclos de vida.

Fonte: Project Management Institute (2017), adaptado.

### 3.2.1.6 Ciclo de vida Adaptativo

Enquanto as abordagens híbridas fornecem princípios e filosofia, os ciclos de vida adaptativos representam sua implementação prática no planejamento temporal de projetos. Os ciclos adaptativos são a manifestação temporal das abordagens híbridas, permitindo a variação deliberada de práticas ao longo do tempo do projeto. Eles implementam o princípio de adaptação contextual no domínio do planejamento e execução de projetos.

Um ciclo de vida adaptativo é uma estrutura de desenvolvimento que deliberadamente varia as práticas, cerimônias e nível de formalidade ao longo do tempo, baseando-se na natureza da fase atual do projeto e no crescente entendimento dos requisitos (HIGHSMITH, 2013).

A característica definidora de um ciclo adaptativo é a capacidade de transitar entre diferentes modos de operação com base em critérios explícitos. Por exemplo, um projeto pode iniciar com uma fase altamente exploratória e experimental usando práticas ágeis, transitando para práticas mais estruturadas à medida que os requisitos se estabilizam e o sistema cresce em criticidade (HIGHSMITH, 2013).

As características dos ciclos de vida adaptativos incluem:

1. **Fases com propósitos distintos:** Reconhecimento explícito de que diferentes fases do projeto têm objetivos fundamentalmente diferentes e, portanto, exigem abordagens diferentes.
2. **Critérios de transição:** Pontos de decisão claramente definidos para avaliar quando mudar de uma abordagem para outra, baseados em fatores como redução de incerteza, estabilização de requisitos ou aumento da complexidade do sistema.
3. **Aplicação seletiva de práticas:** Uso deliberado de diferentes práticas em diferentes momentos - por exemplo, design thinking e prototipação rápida nas fases iniciais, processos mais estruturados nas fases de escala.
4. **Documentação progressiva:** Documentação que evolui em formalidade e completude à medida que o entendimento do sistema aumenta, evitando tanto a documentação excessiva prematura quanto a falta de documentação em sistemas críticos.

Frameworks como o Disciplined Agile Delivery (AMBLER; LINES, 2012) oferecem orientações específicas sobre como implementar ciclos de vida adaptativos, proporcionando um menu de opções para diferentes contextos do projeto.

### 3.2.1.7 Ciclo de vida iterativo e incremental: dirigido por plano e ágil

Conforme reportado, anteriormente, um ciclo de vida iterativo e incremental pode ser realizado, a partir de uma abordagem dirigida por plano, ou ágil. Aqui, vamos apresentar algumas semelhanças e diferenças entre essas possibilidades.

#### Semelhanças:

- **Iterativo e Incremental:** ambas as abordagens utilizam ciclos iterativos e incrementais para desenvolver o software, permitindo revisões e melhorias contínuas.
- **Melhoria Contínua:** tanto as abordagens dirigidas por plano quanto as ágeis buscam a melhoria contínua do produto e do processo ao longo do tempo.
- **Feedback:** feedback é um componente chave, embora a frequência e o modo como é integrado possam variar.

#### Diferenças:

- **Flexibilidade vs. Planejamento:** o ciclo de vida ágil é mais flexível e adaptável às mudanças, enquanto o dirigido por plano enfatiza um planejamento mais detalhado e a aderência a este plano.

- **Colaboração com o Cliente:** a colaboração com o cliente é mais intensiva e contínua no ágil, em comparação com abordagens mais formais e estruturadas no dirigido por plano.
- **Enfoque:** a abordagem ágil foca na entrega rápida de valor ao cliente e na capacidade de responder a mudanças, enquanto a dirigido por plano procura equilibrar entre seguir um plano detalhado e adaptar-se quando necessário.

Cabe ressaltar que, a escolha entre um ciclo de vida iterativo e incremental dirigido por plano e um ciclo de vida ágil depende, por exemplo, das necessidades do projeto, da cultura da organização e das preferências da equipe. Ambas as abordagens têm seus méritos e podem ser mais adequadas a diferentes tipos de projetos e contextos organizacionais.

### 3.3 Processos de Desenvolvimento de Software e Frameworks

Os processos de desenvolvimento e os frameworks de gerenciamento representam a camada mais concreta e operacional de nossa estrutura conceitual. Enquanto os processos implementam as atividades técnicas específicas, os frameworks organizam e coordenam o trabalho. É importante ressaltar que os exemplos selecionados para esta seção representam apenas uma amostra significativa do vasto universo de processos e frameworks existentes, escolhidos por sua representatividade e capacidade de ilustrar efetivamente as características de cada categoria.

Os processos discutidos nesta seção representam abordagens distintas que evoluíram ao longo das décadas em resposta a desafios, lições aprendidas e mudanças no ambiente de negócios (LEFFINGWELL, 2011), sendo:

- **Processos abrangentes:** Cascata, Modelo V, Unified Process (UP), OpenUP, AUP.
- **Processos com foco em práticas técnicas específicas:** XP, FDD.
- **Processos híbridos/adaptativos:** Espiral, DSDM, RAD.

No que se refere aos frameworks de gerenciamento, serão abordaremos os seguintes:

- **Frameworks de gerenciamento ágil:** Scrum e Kanban.
- **Frameworks de escalabilidade:** SAFe e LeSS.

Na prática, processos e frameworks são frequentemente combinados para formar uma abordagem completa ao desenvolvimento de software (RUBIN, 2022). Por exemplo, uma equipe pode adotar o Scrum para organizar seu trabalho, enquanto implementa práticas técnicas do XP para desenvolvimento e testes. A Figura 20 ilustra o surgimento temporal dos processos e frameworks abordados neste livro.



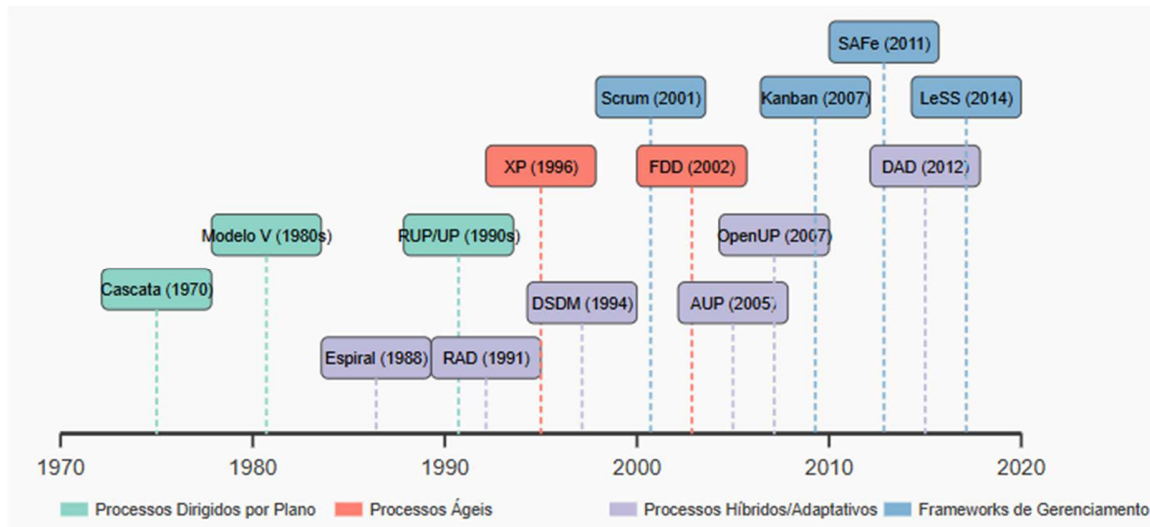


Figura 20 – Surgimento Temporal dos Processos e Frameworks.  
Fonte: elaborado pelo autor (2025).

### 3.3.1 Processos Abrangentes

#### 3.3.1.1 Cascata

O Cascata (Figura 21), um dos primeiros processos formalizados para desenvolvimento de software, caracteriza-se por sua natureza sequencial e linear (SOMMERVILLE, 2019). Originalmente descrito por Winston W. Royce em 1970, este modelo divide o desenvolvimento em fases distintas e bem definidas que devem ser completadas em sequência: levantamento de requisitos, projeto, implementação, verificação (teste) e manutenção (BALAJI; MURUGAIYAN, 2012).



Figura 21 –Cascata.  
Fonte: Sommerville (2019), adaptado.

Cada fase produz artefatos específicos que servem como entrada para a fase seguinte. Por exemplo, a fase de requisitos gera documentos de especificação que orientam o design do sistema; este, por sua vez, produz documentos de arquitetura e design detalhado que guiam a implementação (PETERSEN; WOHLIN; BACA, 2009).

Uma característica do modelo cascata é que o avanço para a próxima fase só ocorre após a conclusão total e aprovação formal da fase anterior, com pouca ou nenhuma sobreposição entre as fases (RUPARELIA, 2010). Isso proporciona uma estrutura de desenvolvimento clara e facilmente gerenciável, com pontos de verificação bem definidos.

O modelo cascata é frequentemente criticado por sua falta de flexibilidade em relação a mudanças nos requisitos e sua implementação tardia de código funcional, o que pode resultar

em problemas sendo descobertos apenas em estágios avançados do projeto (DINGSØYR et al., 2012). No entanto, ele ainda encontra aplicação em projetos com requisitos estáveis e bem compreendidos, particularmente em domínios altamente regulamentados onde a documentação extensiva e processos rigorosos são necessários (KHAN; KHAN, 2019).

### 3.3.1.2 Modelo V

O Modelo V (Figura 22), também conhecido como Modelo de Verificação e Validação, (KHAN; KHAN, 2019). Representado graficamente em formato de 'V', este modelo ilustra como cada fase de desenvolvimento (lado esquerdo da V) tem uma fase de teste associada (lado direito da V) (MATHUR; MALIK, 2016).

No Modelo V, cada entregável da fase de desenvolvimento deve ser verificado e validado durante a fase de teste correspondente. Por exemplo, os requisitos do sistema são testados através de testes de aceitação, o design de alto nível é validado por testes de sistema, e o design detalhado é verificado por testes de integração (GAROUSI et al., 2018).

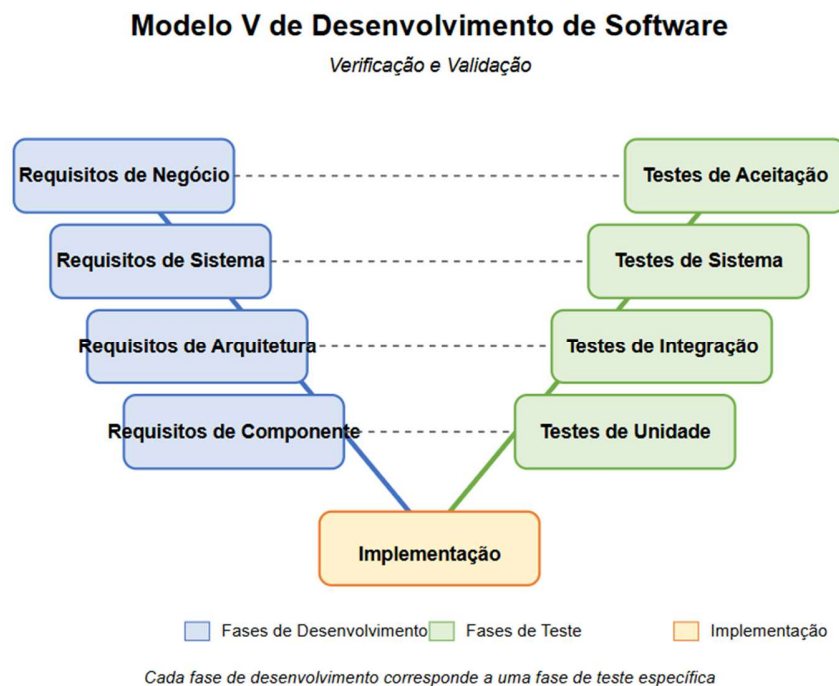


Figura 22 – Modelo V.  
Fonte: MATHUR e MALIK (2016), adaptado.

Esta abordagem sistemática à verificação e validação visa identificar defeitos o mais cedo possível no ciclo de vida do desenvolvimento, reduzindo o custo das correções (BLACK; VEENENDAAL; GRAHAM, 2019). Além disso, o planejamento dos testes começa muito antes da codificação, permitindo que as atividades de teste sejam mais bem preparadas e executadas (KASURINEN; TAIPALE; SMOLANDER, 2016).

O Modelo V é particularmente adequado para projetos onde a qualidade e a conformidade são críticas, como em sistemas médicos, aeroespaciais ou financeiros (WALID; AMRI, 2020). No entanto, assim como o modelo cascata, ele enfrenta desafios com mudanças de requisitos tardias e pode resultar em longos ciclos de desenvolvimento (LEMBERGER; MOREL, 2022).

### 3.3.1.3 Unified Process - UP

O Processo Unificado (UP) (Figura 23) é um framework de desenvolvimento de software iterativo e incremental, inicialmente desenvolvido por Ivar Jacobson, Grady Booch e James Rumbaugh quando atuavam na empresa Rational Software, e posteriormente refinado como Rational Unified Process (RUP) (KRUCHTEN, 2004). Ele organiza o desenvolvimento em quatro fases sequenciais: concepção, elaboração, construção e transição, cada uma contendo uma ou mais iterações que envolvem atividades de modelagem de negócios, requisitos, análise, design, implementação, teste, implantação, gerenciamento de configuração e mudança, gerenciamento de projetos e, ambiente (SOMMERVILLE, 2019).

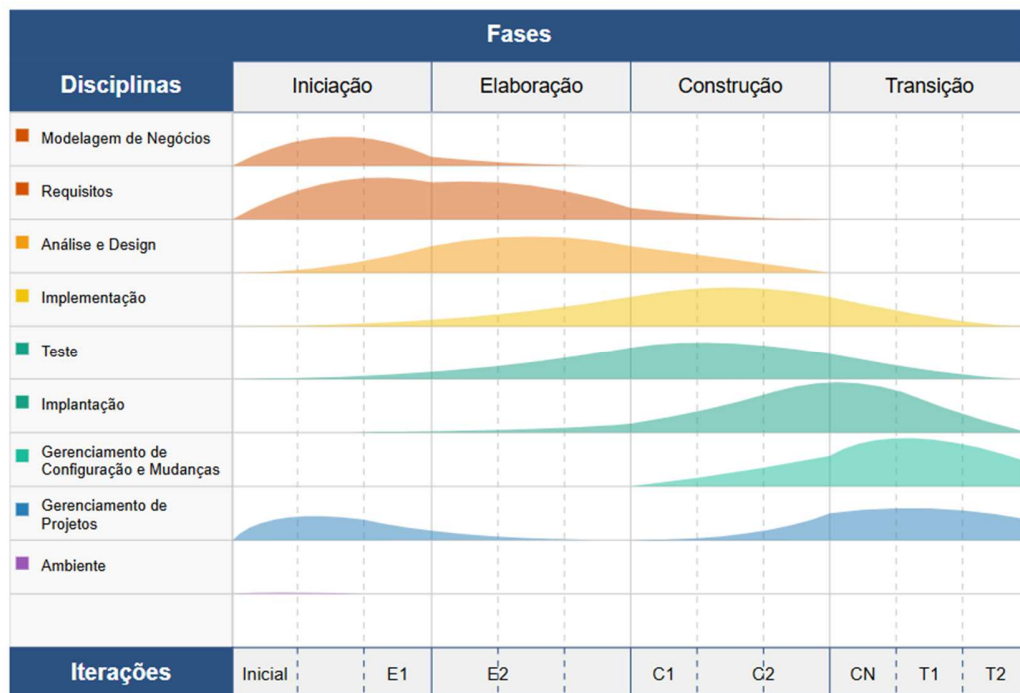


Figura 23 – Processo Unificado.  
Fonte: Jacobson, Booch, Rumbaugh (1999), adaptado.

O UP caracteriza-se por ser orientado a casos de uso, centrado na arquitetura, iterativo e incremental. Os casos de uso capturam os requisitos funcionais do sistema e orientam todo o processo de desenvolvimento, enquanto a arquitetura oferece uma visão estrutural que guia a construção do sistema (JACOBSON; BOOCH; RUMBAUGH, 1999).

Uma contribuição significativa do UP é sua natureza altamente adaptável, permitindo a personalização para diferentes contextos de projeto (KROLL; KRUCHTEN, 2003). Ele

incorpora práticas comprovadas como desenvolvimento iterativo, gerenciamento de requisitos, arquitetura baseada em componentes e verificação contínua da qualidade.

Embora o UP original tenha sido inicialmente percebido como um processo pesado com extensa documentação, versões mais leves e ágeis surgiram, como o OpenUP e o Agile Unified Process, abordando essa preocupação (AMBLER, 2016). O UP continua relevante, especialmente em organizações que necessitam de um processo abrangente que possa ser adaptado para diferentes tipos e tamanhos de projetos (KROLL; MACISAAC, 2006).

#### **3.3.1.4 Open Unified Process - OpenUP**

O OpenUP (Open Unified Process) (Figura 24) é um processo de desenvolvimento de software iterativo, incremental e ágil que representa uma versão simplificada e leve do Rational Unified Process (RUP) (BALDUINO, 2007). Desenvolvido como parte do Eclipse Process Framework, o OpenUP mantém as características essenciais do RUP - desenvolvimento iterativo, casos de uso, gerenciamento de riscos e abordagem centrada na arquitetura - enquanto reduz a complexidade e a documentação (KROLL; MACISAAC, 2006).

O OpenUP organiza-se em quatro fases: concepção, elaboração, construção e transição, cada uma consistindo em uma ou mais iterações. Cada iteração aborda um conjunto equilibrado de requisitos priorizados com base em riscos e valor para os stakeholders, resultando em uma versão testada e integrada do sistema (AMBLER, 2016).

Uma característica do OpenUP é sua escalabilidade e adaptabilidade, oferecendo um framework mínimo e completo que pode ser estendido para abordar necessidades específicas. O processo é estruturado em três camadas: micro-incrementos (horas ou dias), ciclo de vida da iteração (semanas) e ciclo de vida do projeto (meses) (SHUJA; KREBS, 2008).

O OpenUP promove colaboração entre stakeholders, transparência, e balanceamento de prioridades concorrentes. É adequado para pequenas equipes co-localizadas, trabalhando em projetos que não exigem o formalismo e a documentação extensiva do RUP completo (AMBLER, 2016). A natureza aberta e flexível do OpenUP permite sua adaptação a diferentes contextos organizacionais e de projeto, tornando-o uma opção atraente para organizações que desejam os benefícios do Processo Unificado com menor sobrecarga (KROLL; MACISAAC, 2006).

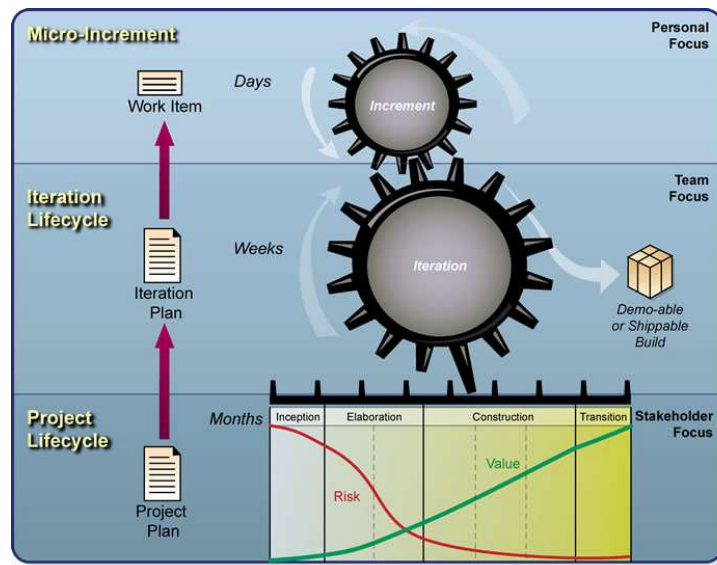


Figura 24 – Processo OpenUP.  
Fonte: IBM CORPORATION et al. (2007).

### 3.3.1.5 Agile Unified Process (AUP)

O Agile Unified Process (AUP) (Figura 25) é uma versão simplificada do Rational Unified Process (RUP) que incorpora técnicas ágeis, desenvolvida por Scott Ambler para oferecer uma abordagem mais compreensível e pragmática ao desenvolvimento de software (AMBLER, 2005). O AUP aplica técnicas ágeis como Desenvolvimento Orientado a Testes (TDD), Modelagem Ágil, Gerenciamento Ágil de Mudanças e Refatoração de Banco de Dados, mantendo a estrutura básica do RUP (QUMER; HENDERSON-SELLERS, 2008).

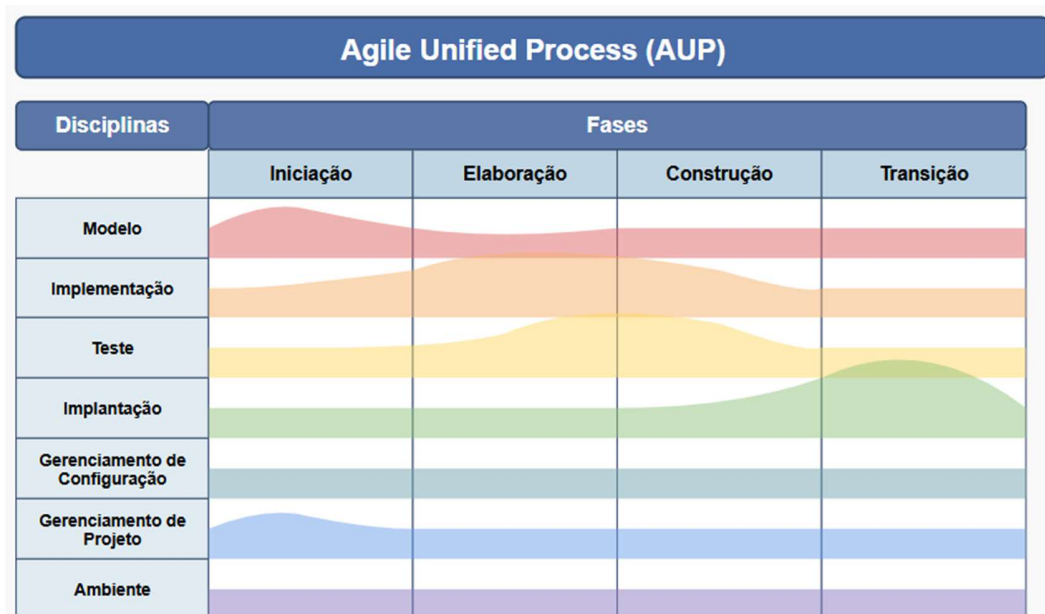


Figura 25 – Processo AUP.  
Fonte: AMBLER (2005), adaptado.

O processo é organizado em quatro fases sequenciais - concepção, elaboração, construção e transição - semelhantes às do RUP tradicional, mas com foco na entrega de software funcional, não em documentação extensiva (AMBLER, 2005). Dentro de cada fase, o trabalho é realizado de forma iterativa e incremental, com ênfase na colaboração entre os membros da equipe e os stakeholders.

Uma característica do AUP é sua organização em sete disciplinas que ocorrem em paralelo: modelo, implementação, teste, implantação, gerenciamento de configuração, gerenciamento de projeto e ambiente. Comparado ao RUP, o AUP consolida várias disciplinas e simplifica significativamente os artefatos produzidos, mantendo apenas os essenciais (AMBLER, 2016).

O AUP é especialmente adequado para organizações que desejam uma transição gradual de processos mais dirigidos por plano para abordagens ágeis, oferecendo um equilíbrio entre estrutura e flexibilidade (OTHMAN et al., 2014). Esta metodologia híbrida permite que equipes mantenham aspectos arquiteturais e disciplinas do RUP que consideram valiosos, enquanto incorporam práticas ágeis para aumentar a produtividade e responsividade (AMBLER; LINES, 2012).

### 3.3.2 Processos com Foco em Práticas Técnicas Específicas

#### 3.3.2.1 eXtreme Programming

O eXtreme Programming (XP) (Figura 26) é uma metodologia ágil de desenvolvimento de software criada por Kent Beck no final dos anos 1990, focada em melhorar a qualidade do software e a capacidade de resposta às mudanças nos requisitos do cliente (BECK; ANDRES, 2004). O XP leva práticas reconhecidas como testes, refatoração e simplicidade a níveis “extremos”, integrando-as em um processo coeso e sinérgico (WILLIAMS, 2010).

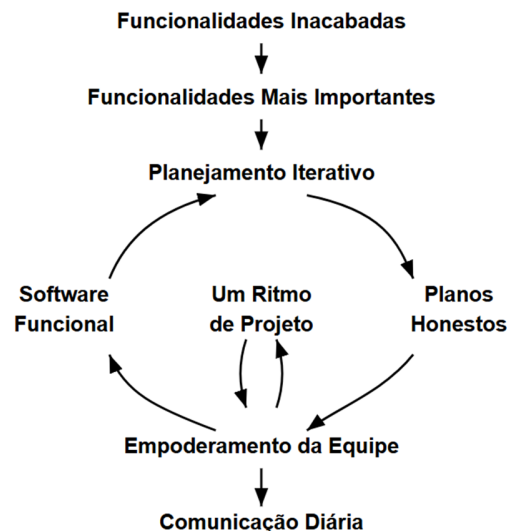


Figura 26 – Processo XP.  
Fonte: Wells (2013), adaptado.

O XP é estruturado em torno de cinco valores fundamentais: comunicação, simplicidade, feedback, coragem e respeito, e implementa doze práticas principais, incluindo programação em pares, integração contínua, propriedade coletiva do código, desenvolvimento orientado a testes (TDD), design simples e pequenas releases (BECK; ANDRES, 2004).

Uma característica do XP é a programação em pares, onde dois programadores trabalham juntos em um único computador, colaborando continuamente em todo o código desenvolvido. Isto promove revisão de código contínua, melhor design e menos defeitos (COCKBURN; WILLIAMS, 2001).

O XP é particularmente adequado para projetos com requisitos dinâmicos, onde o feedback rápido e a adaptabilidade são essenciais (DINGSØYR et al., 2012). Entretanto, sua implementação eficaz requer equipes pequenas e co-localizadas, clientes altamente envolvidos e uma cultura organizacional que valorize a colaboração e adaptabilidade (WILLIAMS et al., 2004). Apesar dessas restrições, muitas de suas práticas foram amplamente adotadas na indústria, mesmo por equipes que não seguem o XP na íntegra (LINDSTROM; JEFFRIES, 2004).

### 3.3.2.2 Feature-Driven Development (FDD)

O Feature-Driven Development (FDD) é um processo ágil desenvolvido por Jeff De Luca e Peter Coad no final dos anos 1990, originalmente para um grande projeto bancário em Singapura (CHEN, 2017). O FDD é centrado no conceito de “features” (características) como a unidade principal de trabalho e progresso (PALMER; FELSING, 2002).

No contexto do FDD, uma feature é uma pequena função valorizada pelo cliente que pode ser implementada em duas semanas ou menos. Estas pequenas unidades de trabalho permitem um planejamento mais preciso, monitoramento de progresso detalhado e entregas frequentes de valor (ZHANG; JARZABEK, 2018).

O processo FDD consiste em cinco processos sequenciais (KAWANO et al., 2016):

1. **Desenvolver um modelo abrangente:** Criar um framework de alto nível do domínio do problema.
2. **Construir uma lista de features:** Decompor o domínio em áreas de funcionalidades e depois em features específicas.
3. **Planejar por feature:** Estabelecer a sequência de desenvolvimento com base na prioridade e dependências.
4. **Projetar por feature:** Elaborar designs detalhados para cada feature.
5. **Construir por feature:** Implementar features individuais em ciclos curtos.

O FDD dá grande ênfase à qualidade ao longo do processo, utilizando inspeções de design e código, além de promover a propriedade individual de classes (componentes de código) enquanto organiza o trabalho em equipes de features. Esta abordagem equilibra a responsabilidade individual com a colaboração em equipe (ALMEIDA et al., 2019).

Uma característica do FDD é seu sistema de relatórios de progresso baseado em marcos para cada feature (design concluído, design inspecionado, código concluído, inspeção de

código, promoção para construção), oferecendo visibilidade granular do status do projeto (PALMER; FELSING, 2002).

O FDD é particularmente adequado para projetos de médio a grande porte onde a organização em torno de funcionalidades do produto faz sentido e onde uma combinação de responsabilidade individual e trabalho em equipe é valorizada (THAKUR; KAUR, 2019).

### 3.3.3 Processos Híbridos/Adaptativos

#### 3.3.3.1 Espiral

O Modelo Espiral (Figura 27), proposto por Barry Boehm em 1988, é um processo de desenvolvimento de software que combina elementos do desenvolvimento iterativo com aspectos sistemáticos do modelo cascata, incorporando uma forte ênfase na análise de riscos (BOEHM, 2000). Representado graficamente como uma espiral, onde cada volta representa uma iteração completa do processo, abrangendo planejamento, análise de riscos, engenharia e avaliação (MUNASSAR; GOVARDHAN, 2010).

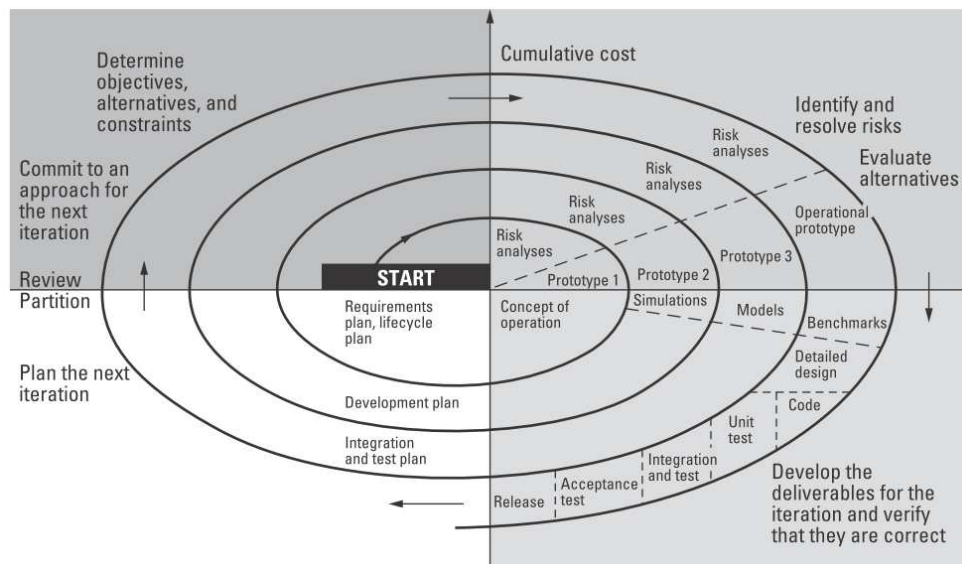


Figura 27 – Processos Espiral.  
Fonte: BOEHM, B. W. (1988).

Uma característica do modelo espiral é sua abordagem orientada a riscos, onde cada ciclo começa identificando objetivos, alternativas e restrições, seguido por uma avaliação das alternativas e identificação e resolução de riscos (PRESSMAN; MAXIM, 2021). Esta ênfase na mitigação precoce de riscos permite que projetos complexos sejam gerenciados de forma mais eficaz, abordando problemas potenciais antes que se tornem críticos.

Cada iteração no modelo espiral resulta em um produto mais refinado, com os requisitos sendo progressivamente detalhados e implementados. Esta abordagem iterativa permite feedback contínuo e adaptação, superando algumas das limitações do modelo em cascata (MATHARU et al., 2015).



O modelo espiral é particularmente adequado para projetos grandes, complexos e de alto risco, onde a incerteza inicial é significativa e a flexibilidade para acomodar mudanças é essencial (BOEHM; TURNER, 2003). No entanto, sua implementação requer expertise significativa em avaliação de riscos e pode ser mais complexa e dispendiosa para gerenciar em comparação com modelos mais simples (KHAN et al., 2014).

### 3.3.3.2 Dynamic Systems Development Method - DSDM

O DSDM (Figura 28) é uma abordagem ágil para o desenvolvimento de software que surgiu no início dos anos 1990, tornando-se um dos pioneiros no movimento ágil (AGILE BUSINESS CONSORTIUM, 2021). Diferentemente de outros métodos ágeis que emergiram de práticas de programação, o DSDM foi desenvolvido por um consórcio de organizações e profissionais com foco em processos de desenvolvimento completos (TURLEY, 2018).

O DSDM baseia-se na premissa de que projetos devem ser alinhados com objetivos estratégicos claramente definidos e entregar benefícios de negócio reais. Para alcançar isso, o método enfatiza a colaboração ativa entre negócios e equipes técnicas, com entregas frequentes de incrementos de produto (KHAN; QURASHI; KHAN, 2020).

Oito princípios fundamentais orientam o DSDM (AGILE BUSINESS CONSORTIUM, 2021):

1. Foco nas necessidades de negócio
2. Entrega dentro do prazo
3. Colaboração
4. Nunca comprometer a qualidade
5. Construir incrementalmente a partir de bases sólidas
6. Desenvolver iterativamente
7. Comunicar contínua e claramente
8. Demonstrar controle

Uma característica do DSDM é a priorização de requisitos usando a técnica MoSCoW (Must have, Should have, Could have, Won't have this time), que ajuda as equipes a entender quais requisitos são essenciais e quais podem ser adiados se necessário para cumprir os prazos (WELLS, 2020).

O DSDM é particularmente adequado para organizações que buscam uma abordagem ágil mais abrangente, que contenha gestão de projeto, governança e envolvimento do negócio, não apenas práticas de desenvolvimento (HNEIF; HOCK OW, 2019).

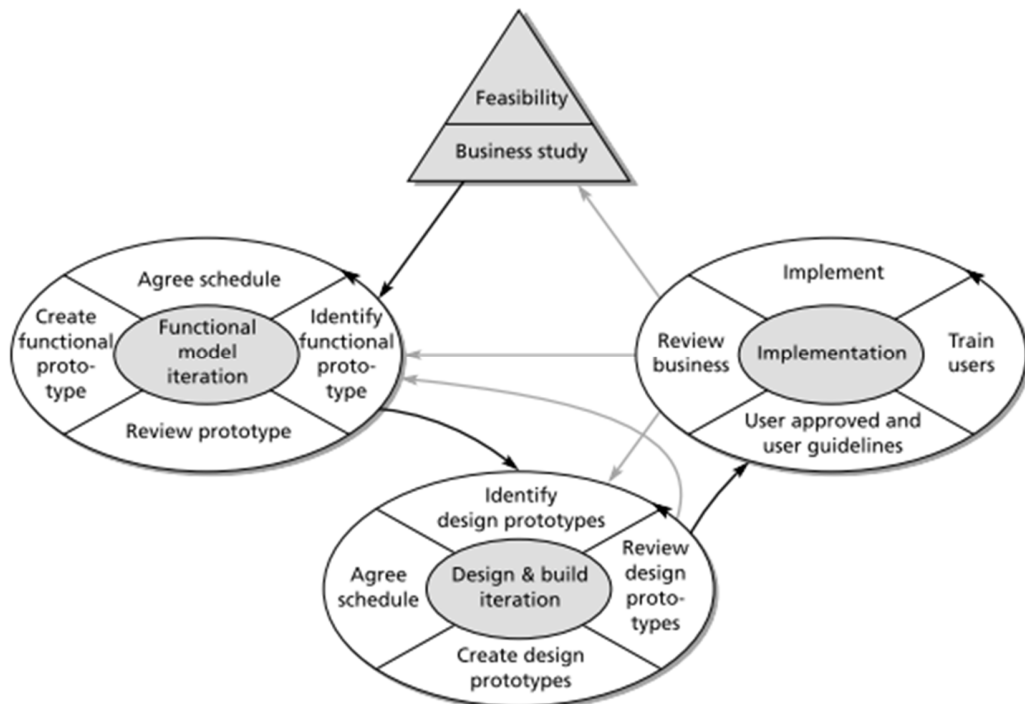


Figura 28 – DSDM.

Fonte: Agile Business Consortium (2021).

### 3.3.3.3 Rapid Application Development - RAD

O Rapid Application Development (RAD) (Figura 29) é uma metodologia de desenvolvimento que enfatiza prototipagem rápida e iterativa como alternativa ao planejamento extensivo, visando acelerar o desenvolvimento de aplicações (MARTIN, 1991). Formalizado por James Martin na década de 1980, o RAD surgiu como resposta às metodologias estruturadas anteriores que frequentemente resultavam em produtos que não atendiam às necessidades dos usuários (BEYNON-DAVIES et al., 1999).

O processo RAD é tipicamente dividido em quatro fases: planejamento de requisitos, design do usuário, construção e cutover (implementação final). A fase de design do usuário constitui o núcleo do processo, onde desenvolvedores e usuários colaboram intensamente, utilizando ferramentas CASE (Computer-Aided Software Engineering) e sistemas de prototipagem para desenvolver rapidamente modelos funcionais do sistema (MACKAY et al., 2000).

Uma característica do RAD é seu foco na participação ativa dos usuários durante todo o processo de desenvolvimento, permitindo feedback contínuo e ajustes (BERGER; BEYNON-DAVIES, 2009). Isto resulta em maior satisfação do usuário final e sistemas que melhor atendem aos requisitos reais dos stakeholders.

O RAD é particularmente eficaz para projetos com prazos curtos, requisitos bem definidos mas sujeitos a mudanças, e escopo que permite modularização (MANER, 1997). No

entanto, apresenta limitações para sistemas de grande escala, missão crítica, ou que requerem alta performance ou interoperabilidade significativa com sistemas legados (DALY, 2014).

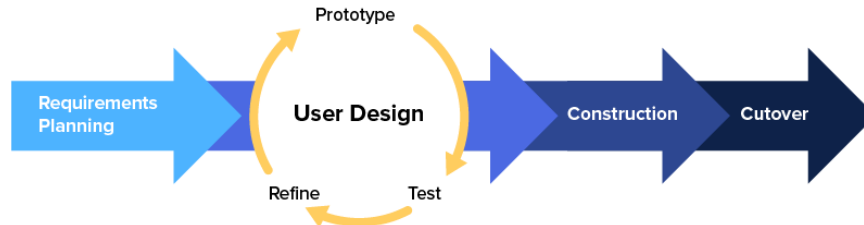


Figura 29 – Processo RAD.  
Fonte: Daly (2014).

### 3.3.4 Frameworks de Gerenciamento Ágil

#### 3.3.4.1 SCRUM

Scrum (Figura 30) é um framework ágil para gerenciamento de projetos complexos, inicialmente formalizado para desenvolvimento de software por Ken Schwaber e Jeff Sutherland na década de 1990 (SCHWABER; SUTHERLAND, 2020). Diferentemente de um processo ou técnica prescritiva, Scrum é um framework dentro do qual várias técnicas e processos podem ser empregados, caracterizado por suas regras, artefatos e papéis específicos que trabalham juntos para gerenciar o fluxo de trabalho (RUBIN, 2022).

O Scrum organiza o trabalho em ciclos de tempo fixo chamados Sprints, tipicamente de 1-4 semanas, nos quais uma versão incremental e potencialmente entregável do produto é desenvolvida (SCHWABER, 2004). Cada Sprint começa com uma reunião de planejamento e termina com uma revisão e retrospectiva, permitindo inspeção e adaptação contínuas (SCHWABER; SUTHERLAND, 2020).

Três papéis principais definem a equipe Scrum: o Product Owner, responsável por maximizar o valor do produto e gerenciar o Product Backlog; o Scrum Master, que facilita o processo e remove impedimentos; e o Time de Desenvolvimento, uma equipe auto-organizada que executa o trabalho (COHN, 2010).

Os artefatos do Scrum incluem o Product Backlog (lista ordenada de tudo que pode ser necessário no produto), o Sprint Backlog (conjunto de itens selecionados para o Sprint) e o Incremento (soma de todos os itens do Product Backlog completados durante um Sprint) (SCHWABER; SUTHERLAND, 2020).

O Scrum é particularmente eficaz em ambientes complexos e dinâmicos onde os requisitos evoluem rapidamente, permitindo adaptabilidade, transparência e inspeção contínua (RIGBY; SUTHERLAND; TAKEUCHI, 2016). Sua simplicidade relativa e foco na entrega de valor têm contribuído para sua ampla adoção, embora sua implementação eficaz requeira uma mudança significativa de mentalidade em organizações tradicionalmente hierárquicas (DIKERT; PAASIVAARA; LASSENIUS, 2016).

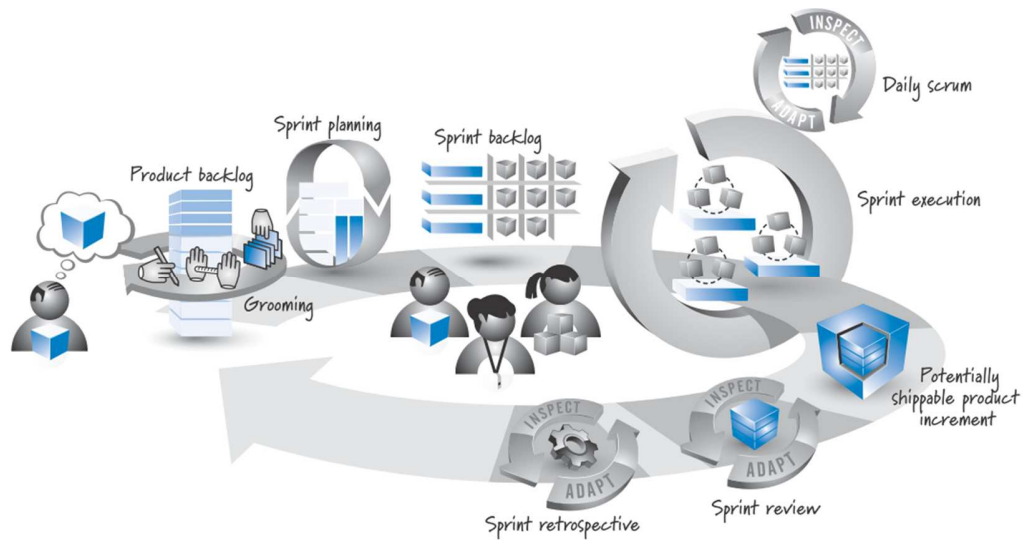


Figura 30 – Framework SCRUM.  
Fonte: Rubin (2022).

### 3.3.4.2 Kanban

O Kanban é um framework ágil para gerenciamento de fluxo de trabalho que se originou no sistema de produção da Toyota na década de 1940 e foi adaptado para o desenvolvimento de software por David J. Anderson em meados dos anos 2000 (ANDERSON, 2010). Diferentemente de outros frameworks ágeis, o Kanban não prescreve papéis específicos, cerimônias fixas ou iterações de duração determinada, focando em vez disso na visualização do fluxo de trabalho, limitação do trabalho em progresso (WIP) e otimização contínua do processo (AHMAD et al., 2018).

A abordagem Kanban é baseada em cinco práticas fundamentais (ANDERSON; CARMICHAEL, 2016):

1. **Visualizar o fluxo de trabalho:** Representação visual do trabalho em um quadro Kanban, onde cada item passa por diferentes estágios desde a concepção até a entrega.
2. **Limitar o trabalho em progresso (WIP):** Estabelecimento de limites para a quantidade de itens que podem estar em cada estágio simultaneamente, reduzindo multitarefa e aumentando a eficiência.
3. **Gerenciar o fluxo:** Monitoramento e análise do fluxo de trabalho para identificar e resolver gargalos, visando um fluxo mais suave e previsível.
4. **Tornar políticas de processo explícitas:** Documentação clara das regras e processos que governam o sistema.
5. **Implementar ciclos de feedback:** Estabelecimento de pontos de verificação regulares para revisão do processo e melhoria contínua.

Uma característica do Kanban é seu modelo de entrega contínua, onde itens de trabalho fluem individualmente pelo sistema assim que estão prontos, em contraste com modelos

iterativos como o Scrum, que organiza entregas em sprints de duração fixa (LEOPOLD; KALTENECKER, 2015). Isto permite maior flexibilidade na priorização e uma resposta mais ágil a mudanças de requisitos ou condições de mercado.

O Kanban também introduz métricas importantes para análise de desempenho e melhoria contínua, como lead time (tempo total desde a entrada de um item no sistema até sua conclusão), cycle time (tempo de processamento efetivo), e throughput (quantidade de itens concluídos por unidade de tempo) (BURROWS, 2014). Estas métricas ajudam equipes a identificar oportunidades de melhoria e a fazer previsões mais precisas sobre prazos de entrega.

O framework é particularmente adequado para equipes que lidam com trabalho variável e imprevisível, como suporte técnico, manutenção de software, ou ambientes onde as prioridades mudam frequentemente (ANDERSON, 2010). Sua natureza não-disruptiva, permitindo implementação gradual sobre processos existentes, torna o Kanban uma escolha interessante para organizações que buscam melhorar seus processos sem grandes reestruturações (AHMAD et al., 2018).

### 3.3.5 Frameworks de Escalabilidade

#### 3.3.5.1 Scaled Agile Framework (SAFe)

O Scaled Agile Framework (SAFe) (Figura 31) é um framework abrangente para escalabilidade ágil desenvolvido por Dean Leffingwell, que fornece orientações estruturadas para implementar práticas ágeis em grandes organizações (LEFFINGWELL, 2020). O SAFe combina elementos do Lean, Agile e DevOps, oferecendo uma abordagem prescritiva para aplicar princípios ágeis em múltiplos níveis organizacionais, desde equipes até portfólios completos (SCALED AGILE, 2023).

O framework SAFe está organizado em quatro níveis de configuração, cada um adaptável às necessidades específicas da organização (LEFFINGWELL et al., 2017):

1. **Essential SAFe:** Configuração básica que foca na coordenação de múltiplas equipes ágeis por meio do Agile Release Train (ART), uma estrutura de longo prazo que sincroniza o trabalho de 5-12 equipes.
2. **Large Solution SAFe:** Expande o Essential SAFe para lidar com soluções complexas que exigem múltiplos ARTs e coordenação adicional.
3. **Portfolio SAFe:** Adiciona a gestão de portfólio ágil, alinhando iniciativas estratégicas com a execução ágil através de epics e value streams.
4. **Full SAFe:** A configuração mais abrangente que integra todos os níveis anteriores para organizações muito grandes com portfólios complexos e diversificados.

Uma característica fundamental do SAFe é o Program Increment (PI), um intervalo de planejamento de 8-12 semanas no qual múltiplas equipes ágeis trabalham juntas para entregar valor incremental. O PI começa com um evento de planejamento (PI Planning) que alinha todas as equipes em objetivos comuns e termina com uma demonstração integrada do incremento desenvolvido (LEFFINGWELL, 2020).

O SAFe implementa um modelo de requisitos em múltiplos níveis, desde a estratégia até a implementação (SCALED AGILE, 2023):

- **Epics:** Iniciativas estratégicas de grande escala que cruzam múltiplos value streams e ARTs.
- **Capabilities:** Funcionalidades de nível superior que exigem múltiplas equipes.
- **Features:** Serviços específicos que atendem a necessidades dos stakeholders.
- **Stories:** Requisitos de baixo nível implementados por equipes.

Esta hierarquia de requisitos permite o desdobramento da estratégia em componentes implementáveis, mantendo a rastreabilidade desde os objetivos de negócio até o código (TUNSTADT; STOJANOV; TRIENEKENS, 2017).

O SAFe é particularmente adequado para grandes organizações que buscam escalabilidade ágil enquanto mantêm alinhamento estratégico e governança consistente. No entanto, sua natureza prescritiva e abrangente pode ser considerada excessiva para organizações menores ou que precisam de maior flexibilidade de implementação (PAASIVAARA, 2017).

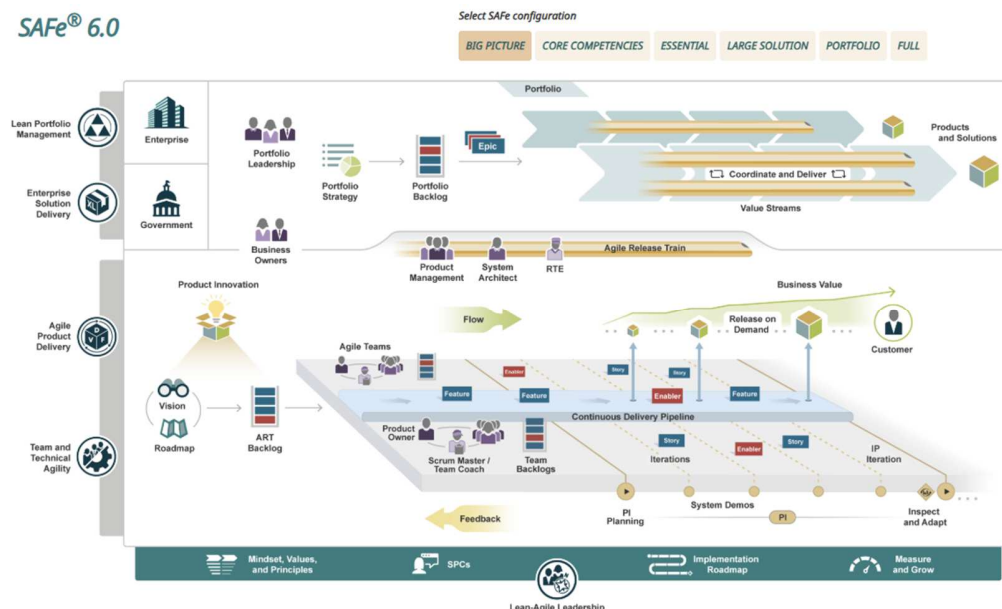


Figura 31 – Framework SAFe.  
Fonte: Scaled Agile (2023).

### 3.3.5.2 Large-Scale Scrum (LeSS)

O Large-Scale Scrum (LeSS) (Figura 32) é um framework de escalabilidade ágil desenvolvido por Craig Larman e Bas Vodde, baseado em mais de uma década de experiência escalando o Scrum em grandes organizações (LARMAN; VODDE, 2016). Diferentemente de outros frameworks de escalabilidade, o LeSS busca manter-se fiel aos princípios e regras do Scrum, expandindo-os para coordenar múltiplas equipes com mínima adição de estruturas ou processos (DIKERT; PAASIVAARA; LASSENIUS, 2016).

O LeSS se destaca por sua abordagem minimalista à escalabilidade, seguindo o princípio “mais com LeSS” (more with LeSS), que enfatiza a remoção de complexidade organizacional desnecessária ao invés de adicionar novas camadas (LARMAN; VODDE, 2016). Isto reflete uma filosofia sistêmica que vê as organizações como sistemas adaptativos complexos onde a simplificação é preferível à adição de novos processos.

O framework LeSS oferece duas configurações principais (LARMAN; VODDE, 2016):

1. **LeSS Básico:** Para 2-8 equipes (aproximadamente 50 pessoas), mantendo um único Product Owner, Product Backlog e Sprint sincronizado para todas as equipes.
2. **LeSS Huge:** Para situações com mais de 8 equipes, introduzindo o conceito de “Áreas de Requisitos”, cada uma com seu próprio Product Owner de Área, mantendo ainda um Product Owner e Product Backlog geral para todo o produto.

Uma característica fundamental do LeSS é a ênfase na orientação ao produto em vez de orientação ao projeto. Todas as equipes trabalham no mesmo produto, compartilhando objetivos comuns e definição de pronto, com foco em entregar valor ao cliente de forma integrada (PAASIVAARA; LASSENIUS, 2019).

O LeSS também promove práticas específicas para melhorar a colaboração entre equipes, como reuniões de planejamento multiequipe, refinamento de backlog conjunto e demonstrações integradas do produto (SANTOS, J. et al., 2020). Estas práticas visam manter a coerência da abordagem enquanto escalando para múltiplas equipes.

No tratamento de requisitos, o LeSS mantém a abordagem do Scrum baseada em Product Backlog e histórias de usuário, mas adiciona técnicas para lidar com a complexidade em larga escala, como “Áreas de Requisitos” para organizar funcionalmente requisitos relacionados, permitindo que equipes se especializem em determinadas áreas do produto enquanto mantêm uma visão integrada (LARMAN; VODDE, 2016).

O framework é particularmente adequado para organizações que valorizam a agilidade técnica e desejam manter a simplicidade enquanto escalam, evitando burocracias e estruturas de coordenação complexas. Sua ênfase em mudanças estruturais organizacionais, incluindo redesenho de papéis tradicionais e redução de camadas gerenciais, pode representar um desafio para organizações mais tradicionais ou hierárquicas (PAASIVAARA; BEHM; LASSENIUS, 2018).

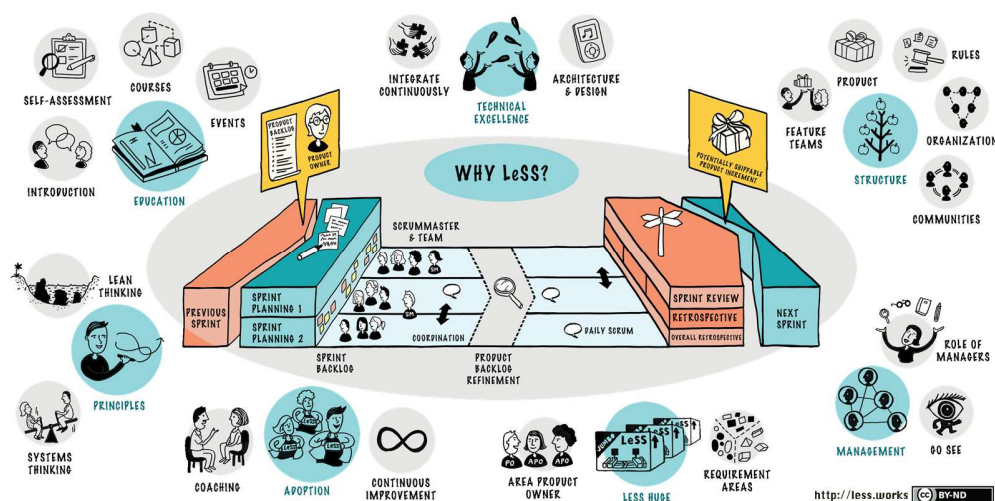


Figura 32 – Framework LeSS.  
Fonte: LeSS Works (2024).

### 3.4 Relação entre Abordagens, Ciclos de Vida, Processos e Frameworks

O Quadro 3, a seguir mapeia a relação entre as diferentes camadas metodológicas do desenvolvimento de software, mostrando como abordagens filosóficas se manifestam em ciclos de vida específicos, que por sua vez são implementados através de processos e frameworks concretos.

| Abordagem                 | Ciclos de Vida                                                                                                 | Processos e Frameworks                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dirigida por Plano</b> | <ul style="list-style-type: none"> <li>• Preditivo</li> <li>• Iterativo e incremental (estruturado)</li> </ul> | <b>Processos Abrangentes:</b> <ul style="list-style-type: none"> <li>• Cascata</li> <li>• Modelo V</li> <li>• Unified Process (UP)</li> </ul>                                                                                                                                                                                                                                                                              |
| <b>Ágil</b>               | <ul style="list-style-type: none"> <li>• Iterativo e incremental</li> <li>• Ágil</li> </ul>                    | <b>Processos com Foco em Práticas Técnicas:</b> <ul style="list-style-type: none"> <li>• eXtreme Programming (XP)</li> <li>• Feature-Driven Development (FDD)</li> </ul> <b>Frameworks de Gerenciamento Ágil:</b> <ul style="list-style-type: none"> <li>• Scrum</li> <li>• Kanban</li> </ul>                                                                                                                              |
| <b>Híbrida</b>            | <ul style="list-style-type: none"> <li>• Adaptativo</li> <li>• Iterativo e incremental (adaptável)</li> </ul>  | <b>Processos Híbridos/Adaptativos:</b> <ul style="list-style-type: none"> <li>• Espiral</li> <li>• Dynamic Systems Development Method (DSDM)</li> <li>• Rapid Application Development (RAD)</li> <li>• Agile Unified Process (AUP)</li> <li>• OpenUP</li> </ul> <b>Frameworks de Escalabilidade:</b> <ul style="list-style-type: none"> <li>• Scaled Agile Framework (SAFe)</li> <li>• Large-Scale Scrum (LeSS)</li> </ul> |

Quadro 3 – Mapeamento entre as diferentes camadas metodológicas do desenvolvimento de software.



## CAPÍTULO 3: PONTOS-CHAVE

- O capítulo apresenta a **estrutura conceitual que conecta abordagens, ciclos de vida, processos e frameworks de desenvolvimento de software**, explicando como esses elementos se complementam na prática e sustentam a adaptabilidade metodológica nas organizações.
- As **abordagens de desenvolvimento de software** definem a filosofia geral que orienta valores, princípios e modos de trabalho. Elas moldam a cultura da equipe e influenciam diretamente os processos e frameworks adotados.
- As **abordagens dirigidas por plano** priorizam o planejamento detalhado e a previsibilidade. São adequadas para contextos com **requisitos estáveis e alta necessidade de controle**, sendo exemplificadas por modelos como **Cascata** e **Modelo V**.
- As **abordagens ágeis** valorizam **adaptação, colaboração e entrega contínua de valor**, respondendo rapidamente a mudanças de contexto e necessidade do cliente. Práticas como **Scrum, Kanban, XP e FDD** representam esse paradigma.
- O texto introduz as **abordagens híbridas**, que combinam práticas de previsibilidade e flexibilidade. Elas reconhecem que diferentes partes de um projeto podem exigir níveis distintos de formalidade, permitindo equilibrar planejamento e adaptação.
- São descritos os **ciclos de vida de software** (sequencial, iterativo, incremental, espiral e adaptativo) como estruturas temporais que organizam as atividades de engenharia, determinando quando e como práticas técnicas e gerenciais são aplicadas.
- Os **processos de desenvolvimento de software** concretizam as abordagens e ciclos de vida, definindo atividades, papéis e artefatos técnicos. O livro os classifica em:
  - **Processos abrangentes**: estruturam todas as disciplinas da engenharia de software (ex.: *Cascata, Modelo V, Unified Process, OpenUP, AUP*);
  - **Processos com foco em práticas técnicas específicas**: enfatizam práticas de engenharia, como *XP* e *FDD*;
  - **Processos híbridos/adaptativos**: combinam características estruturadas e ágeis, como *Espiral, DSDM, RAD* e *AUP*.
- Os **frameworks de gerenciamento ágil**, como *Scrum* e *Kanban*, oferecem estruturas organizacionais para coordenar equipes e atividades, definindo papéis, cerimônias e artefatos sem prescrever práticas técnicas específicas.
- Já os **frameworks de escalabilidade**, como *SAFe* e *LeSS*, ampliam os princípios ágeis para contextos corporativos de múltiplas equipes, promovendo alinhamento estratégico e integração entre níveis organizacionais.
- O capítulo conclui mostrando a **relação entre abordagens, ciclos de vida, processos e frameworks**, evidenciando como cada camada metodológica se apoia na anterior — das filosofias e princípios às práticas concretas de desenvolvimento.
- A **adaptabilidade contextual** é destacada como um princípio transversal: organizações eficazes selecionam e combinam elementos de diferentes abordagens e processos

conforme o domínio de aplicação, cultura organizacional, criticidade do sistema e maturidade da equipe.

## Capítulo 4

A engenharia de requisitos constitui o coração de qualquer desenvolvimento de software, transformando necessidades humanas em declarações técnicas e materializando ideias inovadoras. No universo em constante transformação tecnológica, a forma como os requisitos são compreendidos e gerenciados pode determinar o sucesso ou fracasso de um projeto.

Cada processo metodológico – seja dirigido por plano, ágil ou híbrido – carrega uma filosofia única para capturar e implementar requisitos. Este capítulo mergulha nessa diversidade metodológica, examinando como diferentes processos e frameworks de desenvolvimento de software tratam os requisitos, revelando as estratégias desenvolvidas pelos profissionais para transformar necessidades de negócio em soluções tecnológicas efetivas.

Ao explorar cada processo e framework, revelaremos seus princípios, forças, limitações e contextos ideais de aplicação. A engenharia de requisitos não é mais vista como uma etapa inicial e estática, mas como um processo dinâmico, colaborativo e em constante evolução. Este capítulo convida o leitor a uma compreensão mais rica, onde requisitos são pontes vivas entre visão estratégica, necessidades humanas e soluções tecnológicas.

### 4 Como Processos de Desenvolvimento de Software e Frameworks Lidam com Requisitos de Software?

A maneira como os requisitos de software são tratados constitui um dos pilares fundamentais que diferencia os diversos processos de desenvolvimento e frameworks na engenharia de software. Desde a elicitação e descoberta até a declaração, validação, organização e gestão de mudanças, cada abordagem metodológica estabelece práticas e fluxos de trabalho distintos que refletem sua filosofia subjacente e valores prioritários.

Os requisitos de software representam a tradução das necessidades dos stakeholders em declarações que orientam todo o ciclo de desenvolvimento. A forma como são tratados influencia diretamente aspectos críticos como qualidade do produto final, capacidade de adaptação a mudanças, velocidade de entrega e satisfação do cliente. Em um cenário onde as organizações enfrentam pressões simultâneas por previsibilidade e agilidade, compreender as nuances de como diferentes processos e frameworks abordam os requisitos torna-se essencial para a tomada de decisões sobre qual deles adotar em um contexto específico.

Esta seção examina como cada processo de desenvolvimento (Cascata, Modelo V, Unified Process (UP), OpenUP, AUP, XP, FDD, Espiral, DSDM e RAD) e framework (Scrum, Kanban, SAFe, LeSS) apresentados neste capítulo, abordam o ciclo de vida dos requisitos. Para cada um deles, serão descritas suas atividades, estrutura de organização dos requisitos, forças e limitações, cenários adequados para sua aplicação e tendências.

Ao compreender as diferentes perspectivas sobre o tratamento de requisitos, profissionais e organizações podem selecionar e adaptar abordagens que melhor se alinhem às características específicas de seus projetos, domínios de aplicação e culturas organizacionais, implementando a adaptabilidade contextual que é fundamental para o sucesso no desenvolvimento de software contemporâneo.

## 4.1 Cascata

O modelo Cascata, como um dos processos mais antigos e estruturados de desenvolvimento de software, adota uma abordagem sistemática e sequencial para o tratamento de requisitos, caracterizada por fases bem definidas com clara documentação e aprovação formal (SOMMERVILLE, 2019).

### Processo de Requisitos no Modelo Cascata

A engenharia de requisitos ocorre como a primeira fase formal do ciclo de desenvolvimento, precedendo todas as outras atividades técnicas. Esta fase é considerada crucial e deve ser concluída integralmente antes que o design do sistema possa começar (BALAJI; MURUGAIYAN, 2012). O processo típico inclui:

1. **Elicitação e Descoberta de Requisitos:** É caracterizada por um processo extensivo e detalhado no início do projeto, buscando capturar todos os requisitos possíveis antes de avançar para as fases subsequentes (PRESSMAN; MAXIM, 2021). As técnicas comumente empregadas incluem, entrevistas estruturadas, questionários, análise de documentos existentes e workshops de requisitos. Uma importante característica é a presunção de que é possível e desejável capturar todos os requisitos significativos durante esta fase inicial (KHAN; KHAN, 2019).
2. **Análise e Negociação:** Após a elicitación, os requisitos passam por um processo formal de análise para identificar e resolver ambiguidades, inconsistências e incompletudes, resolver conflitos entre requisitos concorrentes e determinar limites e restrições do sistema.
3. **Documentação e Especificação:** O Cascata coloca grande ênfase na documentação abrangente e detalhada dos requisitos, resultando em artefatos como, Documento de Requisitos de Software (SRS)<sup>8</sup>, modelos conceituais<sup>9</sup> e matrizes de rastreabilidade<sup>10</sup>. Esta documentação serve como um “contrato” entre clientes e desenvolvedores, estabelecendo uma base formal para todas as fases subsequentes (PRESSMAN; MAXIM, 2021).
4. **Validação e Verificação:** A validação dos requisitos ocorre formalmente no final da fase de requisitos, por meio de revisões técnicas formais e aprovação formal do cliente. Esta validação visa garantir que os requisitos documentados sejam corretos, completos, consistentes, verificáveis e testáveis (DAGNINO,

<sup>8</sup> Uma especificação formal e exaustiva de todos os requisitos funcionais e não funcionais, seguindo frequentemente padrões como IEEE 830 ou ISO/IEC/IEEE 29148:2018 (IEEE, 2018).

<sup>9</sup> Diagramas UML, modelos de dados e outros artefatos que representam visualmente aspectos do sistema.

<sup>10</sup> Documentos que mapeiam requisitos a outros elementos do desenvolvimento e entre si.

2014). Uma vez que os requisitos são validados e aprovados, eles são “congelados” e mudanças subsequentes são desestimuladas ou submetidas a rigoroso controle de mudanças.

5. **Gerenciamento de Mudanças:** Implementa um processo formal de controle de mudanças para requisitos, que tipicamente inclui um Comitê de Controle de Mudanças (CCB), Formulários de Solicitação de Mudança (CR) e análise de impacto. As mudanças nos requisitos são tratadas como exceções que devem ser formalmente aprovadas, documentadas e comunicadas (PETERSEN; WOHLIN; BACA, 2009).

### Estruturas de Organização de Requisitos

O Cascata organiza os requisitos em uma estrutura hierárquica e formal (SOMMERVILLE, 2019; PRESSMAN; MAXIM, 2021):

- **Requisitos de Negócio:** Objetivos e necessidades de alto nível.
- **Requisitos de Usuário:** Capacidades que o sistema deve fornecer aos usuários.
- **Requisitos Funcionais:** Funções específicas que o sistema deve realizar.
- **Requisitos Não-Funcionais:** Qualidades do sistema (desempenho, segurança, usabilidade).
- **Requisitos de Interface:** Especificações de como o sistema interage com entidades externas.
- **Restrições:** Limitações de design, implementação ou ambiente.

### Forças e Limitações

A abordagem do Cascata para requisitos apresenta vantagens específicas em contextos onde estabilidade e previsibilidade são essenciais:

#### Forças:

- Documentação abrangente que facilita transferência de conhecimento e manutenção de longo prazo.
- Processo bem definido e gerenciável, com marcos claros e entregáveis específicos.
- Adequado para sistemas críticos onde especificações completas são necessárias antes da implementação.

#### Limitações:

- Dificuldade em acomodar mudanças nos requisitos após a fase inicial.
- Pressupõe que requisitos podem ser completamente entendidos e documentados antecipadamente.
- Feedback tardio sobre os requisitos, potencialmente levando a implementações que não atendem às necessidades reais dos usuários (KHAN; KHAN, 2019).

### Cenários de Aplicação Adequados

O Cascata continua sendo relevante para o gerenciamento de requisitos em contextos específicos, incluindo:

- Sistemas regulamentados onde conformidade e documentação extensiva são obrigatórias (ex: software médico, aeroespacial).
- Sistemas de missão crítica onde verificação formal dos requisitos é essencial antes do desenvolvimento.
- Projetos com requisitos bem definidos e estáveis, baseados em especificações técnicas precisas ou em versões anteriores do sistema (SOMMERVILLE, 2019).

### Tendências

Embora o Cascata puro seja menos comum atualmente, elementos de sua abordagem para requisitos continuam relevantes e são frequentemente incorporados em processos híbridos:

- Utilização de especificações formais para componentes críticos dentro de projetos ágeis.
- Adoção de práticas de validação formal e rastreabilidade do Cascata em contextos regulamentados, mesmo quando o desenvolvimento segue abordagens mais iterativas (IQBAL et al., 2022).
- Manutenção da documentação abrangente para sistemas de longa duração, enquanto permitindo desenvolvimento mais iterativo (MAIER; MOSER, 2021).

## 4.2 Modelo V

O Modelo V, evolução do Cascata, estabelece uma estrutura que vincula diretamente as fases de desenvolvimento às correspondentes fases de teste, formando uma representação em “V” que enfatiza verificação e validação (MATHUR; MALIK, 2016).

### Processo de Requisitos no Modelo V

A engenharia de requisitos no Modelo V não apenas precede o desenvolvimento, mas estabelece a base para todas as atividades de teste subsequentes. Os requisitos são tratados nas seguintes etapas:

1. **Elicitação e Análise:** Similar ao Cascata, porém com foco adicional na verificabilidade. Durante esta fase, os requisitos são elicitados e simultaneamente associados a critérios de teste específicos. O modelo divide os requisitos em níveis hierárquicos (usuário, sistema, componente) que correspondem a diferentes níveis de teste (BLACK et al., 2019).
2. **Especificação e Documentação:** Produz artefatos formais que incluem não apenas as funcionalidades requeridas, mas também como cada requisito será verificado. A documentação de requisitos deve ser precisa, mensurável e testável, formando a base para o desenvolvimento de planos de teste (KASURINEN et al., 2016).

3. **Validação Inicial:** Antes de prosseguir para o design, os requisitos são validados quanto à completude, consistência e testabilidade. Esta validação antecipada visa reduzir defeitos nas fases posteriores (WALID; AMRI, 2020).
4. **Verificação Contínua:** Ao contrário do Cascata, o Modelo V enfatiza a verificação contínua, estabelecendo uma correspondência direta entre cada nível de requisito e seu respectivo nível de teste – requisitos de usuário são verificados por meio de testes de aceitação, requisitos de sistema por meio de testes de sistema, e assim por diante.
5. **Gerenciamento de Mudanças:** Implementa controle formal, porém com atenção especial à propagação de mudanças nos requisitos para os planos de teste correspondentes, mantendo a integridade da estrutura em V.

### Estrutura de Organização de Requisitos

O Modelo V organiza os requisitos em níveis que correspondem diretamente aos níveis de teste (BLACK et al., 2019; MATHUR; MALIK, 2016):

- **Requisitos de Negócio:** Validados por testes de aceitação de usuário.
- **Requisitos de Sistema:** Validados por testes de sistema.
- **Requisitos de Arquitetura:** Validados por testes de integração.
- **Requisitos de Componente:** Validados por testes de unidade.

### Forças e Limitações

#### Forças:

- Detecção precoce de problemas nos requisitos através da criação paralela de casos de teste.
- Rastreabilidade clara entre requisitos e testes, facilitando verificação e validação.
- Redução de retrabalho e custos de correção através da antecipação de testes.

#### Limitações:

- Dificuldade em acomodar mudanças nos requisitos após a fase inicial.
- Pressupõe que requisitos podem ser completamente especificados antecipadamente.
- Pode resultar em verificação excessivamente rígida, dificultando inovação.

### Cenários de Aplicação Adequados

O Modelo V é particularmente adequado para:

- Sistemas críticos em segurança onde verificação rigorosa é mandatória.
- Domínios altamente regulamentados como médico, automotivo e aeroespacial.
- Sistemas embarcados com requisitos de hardware e software fortemente acoplados.

- Projetos com requisitos estáveis e escopo bem definido (MATHUR; MALIK, 2016).

### Tendências

Embora o Modelo V tradicional seja considerado menos ágil, adaptações atuais incluem:

- Incorporação de princípios ágeis em ciclos menores dentro da estrutura V.
- Automação de testes para acelerar a verificação enquanto mantém o rigor.
- Combinação com abordagens de desenvolvimento contínuo, preservando a rastreabilidade requisito-teste.
- Aplicação seletiva para componentes críticos dentro de projetos híbridos.

## 4.3 Unified Process (UP)

O Processo Unificado (UP) é um framework de desenvolvimento iterativo e incremental que organiza o trabalho em quatro fases sequenciais (Concepção, Elaboração, Construção e Transição), cada uma contendo múltiplas iterações que abordam requisitos de forma progressiva (KRUCHTEN, 2004).

### Processo de Requisitos no Unified Process

A engenharia de requisitos no UP é distribuída ao longo do ciclo de vida, com maior concentração nas fases iniciais:

1. **Elicitação e Descoberta:** Concentra-se principalmente na fase de Concepção e início da Elaboração. Utiliza casos de uso como técnica central, complementados por requisitos não-funcionais e regras de negócio. O UP enfatiza a compreensão do contexto do negócio antes da especificação detalhada (JACOBSON et al., 1999).
2. **Análise e Refinamento:** Os requisitos são refinados progressivamente através de iterações. Durante a fase de Elaboração, requisitos arquiteturalmente significativos são priorizados e detalhados, enquanto outros podem ser elaborados em iterações posteriores (KROLL; KRUCHTEN, 2003).
3. **Documentação:** Organiza requisitos em uma estrutura hierárquica: Documento de Visão (escopo e objetivos gerais), Modelo de Casos de Uso (requisitos funcionais), e Especificações Suplementares (requisitos não-funcionais). A modelagem UML é extensivamente utilizada para visualizar e especificar requisitos.
4. **Validação:** Realizada iterativamente, com validação progressiva dos artefatos de requisitos. Prototipagem e revisões com stakeholders são práticas comuns, especialmente na transição entre iterações (JACOBSON et al., 1999).
5. **Gerenciamento:** Utiliza um processo formal de gerenciamento de mudanças, com requisitos rastreados até outros artefatos de desenvolvimento. A priorização contínua direciona quais requisitos serão implementados em cada iteração (KRUCHTEN, 2004).

### Estrutura de Organização dos Requisitos

O UP emprega uma estrutura baseada em casos de uso (JACOBSON et al., 1999; KRUCHTEN, 2004):

- **Visão:** Escopo e objetivos gerais.
- **Modelo de Casos de Uso:** Requisitos funcionais.
- **Casos de Uso:** Cenários de interação entre atores e sistema.
- **Atores:** Usuários ou sistemas externos que interagem com o sistema.
- **Requisitos Suplementares:** Requisitos não-funcionais e restrições.
- **Glossário:** Terminologia do domínio.
- **Regras de Negócio:** Políticas e condições operacionais.

### Forças e Limitações

#### Forças:

- Equilíbrio entre documentação formal e adaptabilidade às mudanças.
- Foco em arquitetura baseada em riscos e requisitos significativos.
- Rastreabilidade entre requisitos e outros artefatos de desenvolvimento.
- Validação progressiva, reduzindo riscos de interpretação incorreta.

#### Limitações:

- Pode resultar em documentação excessiva se não for adequadamente adaptado.
- Complexidade do processo com múltiplos artefatos e atividades.
- Curva de aprendizado acentuada para equipes inexperientes.

### Cenários de Aplicação Adequados

O UP é particularmente adequado para:

- Sistemas de médio a grande porte com complexidade significativa.
- Organizações que necessitam de conformidade regulatória, mas desejam benefícios iterativos.
- Projetos onde riscos técnicos precisam ser mitigados precocemente.
- Equipes geograficamente distribuídas que necessitam de documentação mais formal.

### Tendências

O UP evoluiu para variantes mais leves como:

- Integração com práticas ágeis mantendo a orientação arquitetural.
- Redução de documentação prescritiva, focando em artefatos essenciais.
- Ênfase em modelagem visual apenas quando necessária.
- Adoção seletiva de práticas UP em contextos ágeis para projetos complexos (AMBLER, 2016).

## 4.4 OpenUP



O OpenUP representa uma versão simplificada e leve do Unified Process, mantendo seus princípios fundamentais enquanto reduz a complexidade e a documentação, adequando-se a equipes menores e projetos menos complexos (BALDUINO, 2007).

### Processo de Requisitos no OpenUP

A abordagem do OpenUP para requisitos mantém a natureza iterativa e incremental do UP, mas com foco na simplicidade:

1. **Elicitação e Análise:** Realizada principalmente nas fases de Concepção e Elaboração inicial, utilizando casos de uso leves ou histórias de usuário para capturar requisitos essenciais. Enfatiza a colaboração direta com stakeholders em vez de documentação extensiva (BALDUINO, 2007).
2. **Documentação:** Adota um conjunto mínimo de artefatos - Visão (objetivos e escopo), Lista de Requisitos (funcionalidades chave), e Modelo de Casos de Uso simplificado. A documentação é criada apenas quando agrega valor tangível para o projeto (KROLL; MACISAAC, 2006).
3. **Refinamento:** Os requisitos são progressivamente detalhados conforme necessário para implementação, evitando especificação excessiva antecipada. Requisitos de alto risco ou alta prioridade são detalhados primeiro (AMBLER, 2016).
4. **Validação:** Ocorre continuamente por meio de revisões com stakeholders, demonstrações e testes. O feedback é incorporado nas iterações subsequentes, permitindo ajustes constantes.
5. **Gerenciamento:** Utiliza um processo mais leve de controle de mudanças, apropriado para equipes pequenas e co-localizadas. Priorização baseada em valor para o cliente determina o sequenciamento da implementação (SHUJA; KREBS, 2008).

### Estrutura de Organização dos Requisitos

O OpenUP simplifica a estrutura do UP (BALDUINO, 2007; KROLL; MACISAAC, 2006):

- **Visão:** Objetivos e escopo do projeto.
- **Casos de Uso ou Histórias de Usuário:** Requisitos funcionais.
- **Requisitos Técnicos:** Restrições e requisitos de implementação.
- **Requisitos Não-Funcionais:** Qualidades do sistema.

### Forças e Limitações

#### Forças:

- Documentação enxuta e voltada para o essencial.
- Processo escalável que pode ser adaptado conforme necessidade.
- Equilíbrio entre disciplina e agilidade.
- Ciclos de feedback rápidos permitindo ajustes frequentes nos requisitos.

#### Limitações:

- Pode ser insuficiente para sistemas de grande escala ou críticos.
- Demanda colaboração próxima e regular com stakeholders.
- Menos estruturado para equipes inexperientes que necessitam de mais orientação.

### Cenários de Aplicação Adequados

O OpenUP é particularmente adequado para:

- Equipes pequenas e co-localizadas (tipicamente 3-10 pessoas).
- Projetos onde a comunicação direta é viável e preferível à documentação extensa.
- Organizações que desejam migrar do tradicional para o ágil gradualmente.
- Sistemas não-críticos com requisitos de conformidade moderados (SHUJA; KREBS, 2008).

### Tendências

O OpenUP evoluiu para se alinhar com as tendências atuais:

- Integração com práticas DevOps para entrega contínua.
- Aumento da ênfase em design thinking e técnicas de elicitação centradas no usuário.
- Ferramentas de colaboração digital para apoiar equipes distribuídas.
- Adoção de práticas específicas do OpenUP em métodos híbridos personalizados (AMBLER, 2016).

## 4.5 Agile Unified Process (AUP)

O Agile Unified Process (AUP) representa uma versão simplificada do Rational Unified Process (RUP) que incorpora técnicas ágeis, oferecendo uma abordagem mais pragmática ao desenvolvimento de software (AMBLER, 2005).

### Processo de Requisitos no AUP

O AUP mantém a estrutura das quatro fases do RUP (Concepção, Elaboração, Construção e Transição), mas simplifica consideravelmente o tratamento dos requisitos:

1. **Elicitação e Modelagem:** Combina técnicas formais e ágeis, utilizando user stories como principal veículo para requisitos em vez de casos de uso detalhados. Emprega modelagem ágil, onde diagramas e modelos são criados apenas quando necessários para tornar mais claros requisitos complexos (AMBLER, 2005).
2. **Documentação:** Adota o princípio de “documentação suficiente” – apenas o necessário para atender às necessidades do projeto. Utiliza um documento de visão simplificado, user stories e, quando apropriado, modelagem visual simplificada (AMBLER, 2005).
3. **Priorização:** Emprega o conceito de “requisitos emergentes”, reconhecendo que não todos os requisitos podem ser identificados antecipadamente. Prioriza

requisitos com base no valor de negócio e risco, incorporando mudanças durante todo o ciclo de vida (QUMER; HENDERSON-SELLERS, 2008).

4. **Validação:** Enfatiza a validação contínua através de demonstrações frequentes, feedback do usuário e testes. Incorpora práticas como desenvolvimento orientado a testes (TDD) para validar requisitos através de testes automatizados.
5. **Gerenciamento:** Implementa controle de mudanças leve e apropriado ao contexto do projeto. Utiliza rastreabilidade seletiva, aplicando-a principalmente aos requisitos mais críticos (AMBLER; LINES, 2012).

### Estrutura de Organização de Requisitos

Utiliza uma combinação de artefatos UP e ágeis (AMBLER, 2005; QUMER; HENDERSON-SELLERS, 2008):

- **Visão de Produto:** Escopo e objetivos gerais.
- **Histórias de Usuário:** Requisitos funcionais em formato ágil.
- **Modelos Leves:** Diagramas e modelos simplificados quando necessários.
- **Requisitos Não-Funcionais:** Documentados em formato conciso.

### Forças e Limitações

#### Forças:

- Combina a estrutura do UP com a agilidade de métodos como XP e Scrum.
- Oferece um caminho de transição para organizações migrando de processos tradicionais.
- Mantém disciplinas essenciais sem burocracia excessiva.
- Adapta-se a diferentes contextos de projeto através de práticas opcionais.

#### Limitações:

- Menos adequado para equipes inexperientes que precisam de mais orientação.
- Pode não fornecer documentação suficiente para certos ambientes regulamentados.
- Requer equilíbrio cuidadoso entre formalidade e agilidade.

### Cenários de Aplicação Adequados

O AUP é particularmente adequado para:

- Organizações em transição de metodologias tradicionais para ágeis.
- Projetos que requerem alguma formalidade, mas se beneficiam de adaptabilidade.
- Ambientes onde a conformidade e documentação são importantes, mas não devem dominar o processo.
- Equipes de tamanho médio com experiência mista em processos (OTHMAN et al., 2014).

## Tendências

O AUP evoluiu consideravelmente:

- Foi incorporado ao framework mais abrangente Disciplined Agile Delivery (DAD)<sup>11</sup>.
- Integração com práticas DevOps e entrega contínua.
- Personalização para diferentes domínios de aplicação e contextos organizacionais.
- Adoção de abordagens “objetivo-baseadas” em vez de prescrições específicas (AMBLER; LINES, 2012).

## 6. eXtreme Programming (XP)

O eXtreme Programming (XP) é uma metodologia ágil que leva práticas de engenharia de software reconhecidas a níveis “extremos”, integrando-as em um processo coeso com foco em comunicação, simplicidade, feedback e coragem (BECK; ANDRES, 2004).

### Processo de Requisitos no XP

O XP aborda os requisitos de software de forma significativamente diferente dos métodos tradicionais:

1. **Elicitação e Descoberta:** Utiliza histórias de usuário (user stories) como principal técnica para capturar requisitos. Estas são escritas pelo cliente em linguagem não-técnica, descrevendo funcionalidades valiosas em pequenos incrementos implementáveis. A elicitación é contínua, com o cliente idealmente presente junto à equipe (BECK; ANDRES, 2004).
2. **Estimativa e Priorização:** As histórias são estimadas pela equipe técnica e priorizadas pelo cliente com base no valor de negócio. Este processo acontece durante a “Jogo do Planejamento” (Planning Game), onde se determina quais histórias serão implementadas no próximo ciclo de desenvolvimento (JEFFRIES; ANDERSON; HENDRICKSON, 2001).
3. **Refinamento:** As histórias são deliberadamente mantidas simples e breves; os detalhes são elaborados através de conversas diretas entre desenvolvedores e clientes no momento da implementação, em vez de documentação extensa antecipada (WILLIAMS, 2010).
4. **Validação:** Implementa validação contínua através de testes de aceitação automatizados, escritos pelo cliente com suporte técnico. Estes testes definem formalmente quando uma história está completa, substituindo especificações detalhadas (CRISPIN; GREGORY, 2009).
5. **Gerenciamento de Mudanças:** Acolhe mudanças nos requisitos a qualquer momento do processo. O planejamento é adaptativo, com novos releases a cada

---

<sup>11</sup> Disciplined Agile Delivery (DAD) é um framework híbrido de processo de desenvolvimento de software criado por Scott Ambler e Mark Lines em 2012. O DAD evoluiu a partir do Agile Unified Process (AUP) e representa uma abordagem orientada a objetivos que integra elementos de várias metodologias, incluindo Scrum, XP, Kanban, Lean, e práticas tradicionais de engenharia de software.

1-3 meses e iterações curtas de 1-2 semanas, permitindo reavaliação e redefinição frequentes de prioridades (BECK; ANDRES, 2004).

### Estrutura de Organização de Requisitos

Adota uma estrutura minimalista baseada em (BECK; ANDRES, 2004; CRISPIN; GREGORY, 2009):

- **Histórias de Usuário:** Principal unidade de requisitos, escrita em cartões físicos ou digitais.
- **CrITÉrios de Aceitação:** Condições para considerar uma história como completa.
- **Testes de Aceitação:** Especificações executáveis que definem o comportamento esperado.

### Forças e Limitações

#### Forças:

- Adaptabilidade excepcional a mudanças nos requisitos.
- Feedback rápido e contínuo sobre implementações.
- Validação precoce através de testes automatizados.
- Redução de desperdício com documentação mínima.

#### Limitações:

- Dependência de disponibilidade e envolvimento ativo do cliente.
- Desafios em sistemas que exigem documentação extensa para conformidade.
- Pode ser difícil escalar para equipes grandes ou distribuídas.
- Menos adequado para requisitos altamente complexos ou que exigem análise de domínio profunda.

### Cenários de Aplicação Adequados

O XP é particularmente adequado para:

- Projetos com requisitos voláteis ou mal compreendidos inicialmente.
- Ambientes onde o feedback rápido e a adaptabilidade são prioritários.
- Sistemas que podem evoluir incrementalmente com valor entregue desde cedo.
- Equipes pequenas e co-localizadas, incluindo representação do cliente (WILLIAMS et al., 2004).

### Tendências

O XP evoluiu significativamente:

- Integração de práticas XP em outros frameworks ágeis, particularmente Scrum.
- Adoção generalizada de práticas técnicas do XP (TDD, programação em pares, integração contínua).

- Adaptações para contextos distribuídos e equipes maiores.
- Evolução para incorporar conceitos de design thinking e lean startup (LINDSTROM; JEFFRIES, 2004).

## 4.6 Feature-Driven Development (FDD)

O Feature-Driven Development (FDD) é um processo ágil centrado no conceito de “features” (características) como unidade principal de trabalho e progresso, originalmente desenvolvido para um grande projeto bancário em Singapura (PALMER; FELSING, 2002).

### Processo de Requisitos no FDD

O FDD estabelece uma abordagem distinta para capturar e gerenciar requisitos:

1. **Elicitação e Modelagem:** Inicia com o desenvolvimento de um modelo abrangente do domínio, criado colaborativamente por especialistas de domínio e desenvolvedores. Este modelo fornece uma visão compartilhada do sistema e forma a base para a identificação de features (PALMER; FELSING, 2002).
2. **Decomposição e Detalhamento:** O FDD decompõe o sistema em áreas de funcionalidade e depois em features específicas. Uma feature é definida como “uma pequena função valorizada pelo cliente”, que pode ser implementada em duas semanas ou menos. Features são descritas em linguagem orientada ao cliente: “<ação> <resultado> <objeto>” (ZHANG; JARZABEK, 2018).
3. **Planejamento e Priorização:** As features são organizadas em conjuntos relacionados e priorizadas com base em dependências técnicas, valor para o cliente e complexidade. O planejamento é realizado por feature, não por iteração de tempo fixo (CHEN, 2017).
4. **Documentação:** Produz documentação concisa com foco em orientação prática. Features são documentadas em uma linguagem não-técnica, complementada por especificações detalhadas de design para cada feature antes de sua implementação (KAWANO et al., 2016).
5. **Verificação e Rastreabilidade:** Utiliza um sistema de relatórios de progresso baseado em marcos para cada feature (design concluído, design inspecionado, código concluído, inspeção de código, promoção para construção), oferecendo visibilidade granular e rastreabilidade (PALMER; FELSING, 2002).

### Estrutura de Organização de Requisitos

Organiza requisitos em uma hierarquia centrada em features (PALMER; FELSING, 2002; CHEN, 2017):

- **Áreas de Funcionalidade:** Agrupamentos de alto nível de características relacionadas.
- **Conjuntos de Features:** Agrupamentos de features relacionadas.
- **Features:** Pequenas funções valiosas para o cliente descritas no formato “<ação> <resultado> <objeto>”.
- **Tarefas de Implementação:** Decomposição técnica de features.

## Forças e Limitações

### Forças:

- Clareza organizacional através de decomposição em features gerenciáveis.
- Equilíbrio entre requisitos centrados no cliente e design técnico rigoroso.
- Sistema de relatórios de progresso detalhado e objetivo.
- Incorporação de revisões e inspeções formais, promovendo qualidade.

### Limitações:

- Foco menor em requisitos não-funcionais que alguns outros métodos.
- Pressupõe que um modelo de domínio estável pode ser estabelecido no início.
- Menos adaptativo a mudanças fundamentais em requisitos de negócio.
- Potencialmente mais orientado a documento que outras metodologias ágeis.

## Cenários de Aplicação Adequados

O FDD é particularmente adequado para:

- Sistemas de médio a grande porte com domínios de negócio complexos.
- Equipes com experiência em orientação a objetos e modelagem de domínio.
- Projetos que requerem visibilidade detalhada do progresso.
- Ambientes onde a propriedade individual de código e responsabilidade são valorizadas (THAKUR; KAUR, 2019).

## Tendências

O FDD evoluiu para se adaptar ao desenvolvimento contemporâneo:

- Integração com outras práticas ágeis como Scrum para gerenciamento de projeto.
- Adaptação para desenvolvimento incremental e contínuo.
- Automação de relatórios de progresso e ferramentas de rastreabilidade.
- Simplificação do processo para projetos menores mantendo o foco em features (ALMEIDA et al., 2019).

## 4.7 Espiral

O Modelo Espiral, proposto por Barry Boehm em 1988, combina elementos de desenvolvimento iterativo com aspectos sistemáticos do modelo cascata, incorporando uma forte ênfase na análise de riscos (BOEHM, 1988).

### Processo de Requisitos no Modelo Espiral

O tratamento de requisitos no Espiral é distintamente orientado a riscos e evolutivo:

1. **Elicitação Progressiva:** Os requisitos são elicitados e refinados através de ciclos sucessivos da espiral. Cada ciclo começa identificando objetivos, alternativas e restrições. A elicitação inicial pode ser de alto nível, tornando-se progressivamente mais detalhada em ciclos subsequentes (BOEHM, 2000).

2. **Análise de Riscos:** Uma característica fundamental é a análise de riscos associados aos requisitos. Incertezas, ambiguidades e potenciais problemas nos requisitos são identificados explicitamente, com estratégias de mitigação desenvolvidas antes de prosseguir (MUNASSAR; GOVARDHAN, 2010).
3. **Prototipagem e Validação:** Utiliza protótipos (desde rascunhos em papel até software funcional) para explorar e validar requisitos de alta incerteza ou risco. O feedback dos stakeholders sobre estes protótipos informa os ciclos subsequentes (MATHARU et al., 2015).
4. **Documentação Evolutiva:** A documentação de requisitos evolui ao longo dos ciclos, começando com descrições de alto nível e refinando-se progressivamente. O nível de formalidade da documentação pode ser adaptado conforme os riscos identificados (PRESSMAN; MAXIM, 2021).
5. **Gestão de Mudanças:** Incorpora mudanças naturalmente através da estrutura cíclica. Cada ciclo reavalia objetivos e requisitos, permitindo adaptação baseada em novas informações ou mudanças no ambiente (BOEHM; TURNER, 2003).

### Estrutura de Organização de Requisitos

Utiliza uma estrutura evolutiva (BOEHM, 1988; MUNASSAR; GOVARDHAN, 2010):

- **Objetivos:** Metas gerais para o incremento atual.
- **Alternativas:** Opções de implementação e abordagens.
- **Restrições:** Limitações e requisitos mandatórios.
- **Requisitos Incrementais:** Especificações que evoluem a cada ciclo.
- **Requisitos Baseados em Riscos:** Organizados conforme análise de riscos.

### Forças e Limitações

#### Forças:

- Identificação e mitigação precoce de riscos nos requisitos.
- Flexibilidade para acomodar mudanças e incertezas.
- Validação iterativa através de protótipos e feedback.
- Escalabilidade para sistemas complexos e de grande porte.

#### Limitações:

- Necessita expertise significativa em avaliação de riscos.
- Pode ser complexo gerenciar e documentar a evolução dos requisitos.
- Potencialmente mais custoso e demorado que outros modelos.
- Requer envolvimento contínuo dos stakeholders para avaliação.

### Cenários de Aplicação Adequados

O Modelo Espiral é particularmente adequado para:

- Projetos grandes, complexos e inovadores com alto grau de incerteza.
- Sistemas críticos onde a análise de riscos é essencial.



- Situações onde requisitos iniciais são pouco claros ou propensos a mudanças significativas.
- Desenvolvimento de novas linhas de produtos ou sistemas sem precedentes (BOEHM; TURNER, 2003).

### Tendências

O Espiral moldou significativamente a evolução de metodologias:

- Incorporação de princípios do Espiral em frameworks ágeis escalados.
- Aplicação em contextos de DevSecOps com ênfase em análise de riscos de segurança.
- Adaptação para domínios emergentes como IA e sistemas ciber-físicos.
- Integração com abordagens de design thinking e lean para exploração de requisitos (KHAN et al., 2014).

## 4.8 Dynamic Systems Development Method (DSDM)

O DSDM é uma abordagem ágil abrangente que surgiu nos anos 1990, diferenciando-se por sua orientação a negócios e foco em entrega dentro de restrições de tempo e orçamento (AGILE BUSINESS CONSORTIUM, 2021).

### Processo de Requisitos no DSDM

O DSDM oferece uma estrutura robusta para gerenciamento de requisitos em ambientes dinâmicos:

1. **Modelagem de Negócios e Viabilidade:** Inicia com um estudo de viabilidade e modelagem de processos de negócio para estabelecer uma compreensão sólida do contexto do projeto antes da elicitação detalhada de requisitos (AGILE BUSINESS CONSORTIUM, 2021).
2. **Priorização MoSCoW:** Adota uma técnica distinção de priorização de requisitos chamada MoSCoW (Must have, Should have, Could have, Won't have this time). Esta abordagem permite que as equipes entendam quais requisitos são essenciais e quais podem ser adiados se necessário para cumprir prazos (WELLS, 2020).
3. **Timeboxing:** Organiza o desenvolvimento em períodos fixos (geralmente 2-6 semanas), com conjuntos específicos de requisitos alocados a cada timebox. Ao contrário de outras metodologias ágeis, o DSDM mantém prazo e custo fixos, ajustando o escopo através da priorização MoSCoW (AGILE BUSINESS CONSORTIUM, 2021).
4. **Documentação Apropriada:** Equilibra a necessidade de documentação formal com agilidade, utilizando o princípio de “quanto basta”. Produz artefatos como business case, definição de arquitetura e protótipos evolutivos para capturar e comunicar requisitos (HNEIF; HOCK OW, 2019).
5. **Envolvimento de Usuários:** Implementa o conceito de “workshops facilitados” para elicitação e refinamento de requisitos, e estabelece papéis

específicos como “Embaixador do Negócio” e “Visionário do Negócio” para garantir que a perspectiva do usuário seja consistentemente representada (TURLEY, 2018).

### Estrutura de Organização de Requisitos

Emprega uma estrutura hierárquica com priorização MoSCoW (AGILE BUSINESS CONSORTIUM, 2021; WELLS, 2020):

- **Business Case:** Justificativa e objetivos de negócio.
- **Requisitos de Negócio:** Necessidades de alto nível.
- **Requisitos da Solução:** Detalhes das funcionalidades requeridas.
- **Classificação dos Requisitos:** Must Have: Essencial para o sucesso, Should Have: Importante, mas o sistema é viável sem ele, Could Have: Desejável se recursos permitirem, Won't Have (this time): Fora do escopo atual.

### Forças e Limitações

#### Forças:

- Forte foco em valor de negócio e viabilidade desde o início.
- Framework completo de governança adaptado a contextos empresariais.
- Técnica MoSCoW torna mais claras as prioridades e gerencia expectativas.
- Integração de perspectivas técnicas e de negócios na elicitação de requisitos.

#### Limitações:

- Mais complexo e prescritivo que algumas metodologias ágeis.
- Requer compromisso organizacional significativo para implementação efetiva.
- Pode parecer burocrático para equipes acostumadas com abordagens mais leves.
- Transição do nível estratégico para requisitos técnicos pode ser desafiadora.

### Cenários de Aplicação Adequados

O DSDM é particularmente adequado para:

- Organizações com fortes restrições de prazo e orçamento, mas requisitos flexíveis.
- Ambientes corporativos tradicionais transitando para agilidade.
- Projetos que requerem conformidade regulatória junto com adaptabilidade.
- Iniciativas de transformação de negócios com alto nível de incerteza (KHAN; QURASHI; KHAN, 2020).

### Tendências

O DSDM evoluiu significativamente:

- Integração com outros frameworks ágeis, particularmente Scrum e SAFe.
- Adaptação para transformação digital e iniciativas estratégicas.

- Evolução para o Agile Project Framework (APF)<sup>12</sup>, expandindo além do desenvolvimento de software.
- Incorporação de práticas de agilidade organizacional e design thinking (AGILE BUSINESS CONSORTIUM, 2021).

## 4.9 Rapid Application Development (RAD)

O Rapid Application Development (RAD) é uma metodologia que enfatiza prototipagem rápida e iterativa como alternativa ao planejamento extensivo, visando acelerar o desenvolvimento de aplicações e obter feedback precoce (MARTIN, 1991).

### Processo de Requisitos no RAD

O RAD aborda requisitos de forma distinta, enfatizando velocidade e colaboração:

1. **Planejamento de Requisitos:** A fase inicial do RAD elicitação e descoberta de requisitos de alto nível, focando em objetivos de negócio, escopo e restrições, sem buscar especificação exaustiva. O objetivo é estabelecer uma direção clara, mas deixando espaço para detalhamento durante a prototipagem (MARTIN, 1991).
2. **Workshop de Design do Usuário:** Caracteriza-se por uma fase intensa de colaboração onde usuários e desenvolvedores trabalham juntos, frequentemente em workshops estruturados, para refinar requisitos através de prototipagem rápida. Os requisitos emergem e evoluem durante este processo iterativo (MACKAY et al., 2000).
3. **Prototipagem Evolutiva:** Em vez de documentação formal extensiva, o RAD captura requisitos através de protótipos funcionais que evoluem progressivamente para o sistema final. Os protótipos servem simultaneamente como especificação e validação de requisitos (BEYNON-DAVIES et al., 1999).
4. **Feedback Contínuo:** Implementa ciclos curtos de prototipagem-revisão-refinamento, onde os stakeholders avaliam versões funcionais e fornecem feedback imediato. Este feedback direciona a evolução tanto dos requisitos quanto da implementação (BERGER; BEYNON-DAVIES, 2009).
5. **Documentação Mínima:** Produz documentação leve, focada principalmente em design de interface, fluxos de dados e mapeamentos de banco de dados. A documentação é criada apenas quando necessária para deixar mais claro ou suporte futuro (MANER, 1997).

### Estrutura de Organização dos Requisitos

Organiza os requisitos por meio de (MARTIN, 1991; BEYNON-DAVIES et al., 1999):

- **Requisitos de Negócio:** Objetivos gerais capturados no planejamento inicial.
- **Modelos de Interface:** Protótipos que evoluem para o sistema final.

---

<sup>12</sup> O Agile Project Framework (APF) é uma evolução do Dynamic Systems Development Method (DSDM), desenvolvido pelo Agile Business Consortium. Representa uma abordagem abrangente para gerenciamento de projetos ágeis que vai além do desenvolvimento de software, podendo ser aplicada a diversos tipos de projetos e iniciativas de negócio.

- **Modelos de Dados:** Estruturas de armazenamento requeridas.
- **Processos:** Fluxos de trabalho e lógica de negócio.

### Forças e Limitações

#### Forças:

- Redução significativa no tempo de elicitação e validação de requisitos.
- Visualização precoce do sistema proposto através de protótipos funcionais.
- Alta satisfação do usuário através de envolvimento ativo e feedback contínuo.
- Detecção precoce de problemas de usabilidade e gaps funcionais.

#### Limitações:

- Menor adequação para sistemas complexos ou de missão crítica.
- Risco de foco excessivo em interfaces e funcionalidades visíveis, negligenciando aspectos não-funcionais.
- Dependência de ferramentas CASE e frameworks de desenvolvimento rápido.
- Necessidade de comprometimento intenso de usuários finais durante todo o processo.

### Cenários de Aplicação Adequados

O RAD é particularmente adequado para:

- Sistemas de informação corporativos com ênfase em interface do usuário.
- Projetos com prazos muito curtos onde a velocidade é prioritária.
- Aplicações onde requisitos são difíceis de articular, mas fáceis de visualizar.
- Situações que exigem prototipagem rápida para validação de conceito ou viabilidade (DALY, 2014).

### Tendências

O RAD evoluiu para se alinhar com práticas atuais:

- Integração com metodologias DevOps para entrega contínua.
- Incorporação de ferramentas low-code/no-code para prototipagem ainda mais rápida.
- Adaptação para desenvolvimento mobile-first e experiências multicanal.
- Evolução para metodologias como Desenvolvimento Orientado ao Cliente (COD)<sup>13</sup> em ambientes digitais (BEYNON-DAVIES et al., 1999).

## 4.10 Scrum

---

<sup>13</sup> O Desenvolvimento Orientado ao Cliente (Customer-Oriented Development ou COD) é uma abordagem metodológica que coloca o cliente e suas necessidades no centro de todo o processo de desenvolvimento de software. Evoluindo a partir de conceitos do RAD (Rapid Application Development) e metodologias centradas no usuário, o COD intensifica o envolvimento do cliente para além da simples validação de requisitos (BEYNON-DAVIES et al., 1999).

O Scrum é um framework ágil de gerenciamento de projetos complexos, formalmente definido por Ken Schwaber e Jeff Sutherland, que organiza o trabalho em ciclos chamados Sprints (SCHWABER; SUTHERLAND, 2020).

### Processo de Requisitos no Scrum

O Scrum aborda os requisitos de software através de uma abordagem empírica e adaptativa:

1. **Captura e Representação:** Utiliza “Product Backlog” como repositório central de requisitos, representados principalmente como “User Stories” – declarações curtas de funcionalidades na perspectiva do usuário. Também podem incluir bugs, melhorias técnicas e requisitos não-funcionais (COHN, 2010).
2. **Priorização e Refinamento:** Implementa “Product Backlog Refinement” como atividade contínua onde itens são detalhados, estimados e priorizados. A priorização é responsabilidade do Product Owner, baseada em valor de negócio, riscos, dependências e custo (RUBIN, 2022).
3. **Planejamento e Compromisso:** Durante o “Sprint Planning”, a equipe seleciona um subconjunto do Product Backlog para implementação na próxima Sprint (tipicamente 1-4 semanas), baseando-se na capacidade da equipe e nas prioridades estabelecidas (SCHWABER; SUTHERLAND, 2020).
4. **Validação e Feedback:** Ao final de cada Sprint, realiza “Sprint Review” onde o incremento do produto é demonstrado a stakeholders para obter feedback. Este feedback é incorporado ao Product Backlog, permitindo adaptação contínua dos requisitos (PRIES; QUIGLEY, 2010).
5. **Adaptação:** Através da “Sprint Retrospective”, a equipe reflete sobre o processo, incluindo como os requisitos foram gerenciados, possibilitando ajustes na abordagem a cada ciclo (SCHWABER, 2004).

### Estrutura de Organização de Requisitos

Utiliza uma estrutura simples (SCHWABER; SUTHERLAND, 2020; COHN, 2010):

- **Visão do Produto:** Direção e objetivos de longo prazo.
- **Product Backlog:** Lista priorizada de todos os itens desejados.
  - **Épicos:** Grandes blocos de funcionalidade.
  - **Histórias de Usuário:** Requisitos de granularidade implementável.
  - **Tarefas:** Atividades técnicas específicas.
- **Definition of Done:** Critérios que definem quando um item está completo.

### Forças e Limitações

#### Forças:

- Alta adaptabilidade a mudanças nos requisitos através de ciclos curtos.
- Transparência contínua sobre o progresso e prioridades.
- Validação frequente com stakeholders minimiza riscos de interpretação incorreta.

- Refinamento progressivo evita especificação excessiva antecipada.

#### **Limitações:**

- Desafios com requisitos de longo prazo ou planejamento estratégico.
- Documentação potencialmente insuficiente para sistemas regulamentados.
- Dependência da disponibilidade e eficácia do Product Owner.
- Pode ser difícil lidar com dependências entre equipes em sistemas complexos.

#### **Cenários de Aplicação Adequados**

O Scrum é particularmente adequado para:

- Ambientes onde requisitos são emergentes ou sujeitos a mudanças frequentes.
- Projetos que beneficiam de feedback rápido e entrega incremental.
- Sistemas que podem ser desenvolvidos modularmente, com valor entregue progressivamente.
- Equipes multifuncionais dedicadas que podem atuar com autonomia (RIGBY et al., 2016).

#### **Tendências**

O Scrum continua evoluindo:

- Integração com práticas de UX e design thinking para melhor descoberta de requisitos.
- Adoção de técnicas de mapeamento de impacto e story mapping para visão estratégica.
- Combinação com Kanban (Scrumban) para fluxo mais contínuo de requisitos.
- Frameworks escalados para coordenar múltiplas equipes Scrum em sistemas complexos (DIKERT et al., 2016).

## **4.11 Kanban**

O Kanban é um framework ágil para gerenciamento de fluxo de trabalho que se originou no sistema de produção Toyota e foi adaptado para desenvolvimento de software por David J. Anderson, focando na visualização do trabalho, limitação do trabalho em progresso e otimização contínua (ANDERSON, 2010).

#### **Processo de Requisitos no Kanban**

O Kanban aborda requisitos com uma perspectiva baseada em fluxo:

1. **Captura e Visualização:** Utiliza um quadro Kanban para visualizar requisitos como itens de trabalho em diferentes estágios do processo (ex: Solicitado, Em Análise, Pronto para Desenvolvimento, Em Desenvolvimento, Em Teste,

Concluído). Cada item representa um requisito que flui através do sistema (ANDERSON; CARMICHAEL, 2016).

2. **Priorização Contínua:** Ao contrário de métodos baseados em iterações fixas, o Kanban permite priorização contínua do backlog. Novos requisitos podem ser inseridos a qualquer momento com base em políticas explícitas de classes de serviço (Standard, Expedite, Fixed Date, Intangible) (BURROWS, 2014).
3. **Refinamento Progressivo:** Os requisitos são refinados just-in-time, conforme se aproximam do desenvolvimento. Isto minimiza o trabalho desperdiçado em itens que podem nunca ser implementados ou que mudam significativamente antes da implementação (LEOPOLD; KALTENECKER, 2015).
4. **Limitação do Trabalho em Progresso (WIP):** Estabelece limites explícitos para a quantidade de requisitos em cada estágio do processo, evitando sobrecarga e garantindo foco. Esta limitação ajuda a identificar gargalos no processamento de requisitos (ANDERSON, 2010).
5. **Métricas e Análise:** Utiliza métricas como lead time (tempo total do pedido até entrega) e cycle time (tempo de processamento efetivo) para analisar e otimizar o processo de tratamento de requisitos, permitindo previsibilidade baseada em dados históricos (HAMMARBERG; SUNDEN, 2014).

### Estrutura de Organização de Requisitos

Não prescreve uma estrutura específica, mas tipicamente inclui (ANDERSON, 2010; HAMMARBERG; SUNDEN, 2014):

- **Itens de Trabalho:** Requisitos em vários níveis de granularidade.
- **Classes de Serviço:** Categorias de itens com políticas distintas (ex: Standard, Expedite).
- **Políticas Explícitas:** Regras que governam como os requisitos fluem pelo sistema.

### Forças e Limitações

#### Forças:

- Flexibilidade para responder a mudanças de prioridade em tempo real.
- Visualização clara de gargalos e problemas no fluxo de requisitos.
- Adaptação não-disruptiva a processos existentes.
- Capacidade de gerenciar diferentes tipos de demanda com políticas específicas.

#### Limitações:

- Menos estruturado para planejamento de longo prazo e visão estratégica.
- Potencial para diluição de foco sem disciplina na priorização.
- Requer maturidade organizacional para autogerenciamento eficaz.
- Pode ser difícil estabelecer previsibilidade inicial sem dados históricos.

### Cenários de Aplicação Adequados

O Kanban é particularmente adequado para:

- Equipes de suporte e manutenção com demandas variáveis e imprevisíveis.
- Ambientes onde interrupções e mudanças de prioridade são comuns.
- Organizações que buscam melhorar processos atuais sem grandes reestruturações.
- Sistemas que requerem entrega contínua em vez de releases em lotes (AHMAD et al., 2018).

### Tendências

O Kanban tem evoluído para abraçar novos contextos:

- Integração com frameworks de escalabilidade para gerenciar requisitos em múltiplos níveis.
- Adoção de práticas “upstream” para gerenciamento do funil de ideias e requisitos.
- Ferramentas de simulação e análise para previsão mais sofisticada.
- Expansão além do desenvolvimento para gestão de portfólio e estratégia de produto (ANDERSON, 2010).

## 4.12 Scaled Agile Framework (SAFe)

O Scaled Agile Framework (SAFe) é um framework abrangente para escalabilidade ágil desenvolvido por Dean Leffingwell, fornecendo orientações para implementar práticas ágeis em grandes organizações e alinhar múltiplas equipes a objetivos comuns (LEFFINGWELL, 2020).

### Processo de Requisitos no SAFe

O SAFe implementa uma abordagem multinível para requisitos, permitindo coordenação desde a estratégia até a implementação:

1. **Hierarquia de Requisitos:** Organiza requisitos em quatro níveis hierárquicos:
  - **Epics:** Iniciativas estratégicas de grande escala.
  - **Capabilities:** Funcionalidades de alto nível que exigem múltiplas equipes.
  - **Features:** Serviços específicos que atendem necessidades dos stakeholders.
  - **Stories:** Requisitos de baixo nível implementados por equipes individuais (SCALED AGILE, 2023).
2. **Planejamento Cadenciado:** Implementa “Program Increment (PI) Planning”, um evento de planejamento de 2 dias onde múltiplas equipes sincronizam suas atividades para os próximos 8-12 semanas. Durante este evento, features são detalhadas, decompostas em stories e dependências entre equipes são identificadas (LEFFINGWELL et al., 2017).
3. **Gerenciamento de Fluxo:** Utiliza “Kanban em Múltiplos Níveis” para visualizar e gerenciar o fluxo de trabalho em cada camada (portfólio, solução,



programa, equipe). Cada nível tem seu próprio quadro Kanban com diferentes tipos de itens de trabalho e políticas (LEFFINGWELL, 2020).

4. **Validação e Demonstração:** Ao final de cada PI, realiza “System Demo” onde o incremento integrado é apresentado a stakeholders para feedback. Este feedback alimenta o planejamento do próximo PI, permitindo ajustes táticos dentro de uma direção estratégica (SCALED AGILE, 2023).
5. **Gerenciamento Centralizado:** Implementa “Backlog Multinível”, onde requisitos são gerenciados de forma coordenada, mantendo rastreabilidade desde epics até stories. A priorização ocorre em cada nível, com maior ênfase estratégica nos níveis superiores (LEFFINGWELL, 2020).

### Estrutura de Organização de Requisitos

Implementa uma hierarquia de requisitos em quatro níveis (LEFFINGWELL, 2020; SCALED AGILE, 2023):

- **Epics:** Iniciativas estratégicas de grande escala.
- **Capabilities:** Funcionalidades de alto nível que requerem múltiplas equipes.
- **Features:** Serviços específicos que atendem necessidades dos stakeholders.
- **Stories:** Requisitos detalhados que podem ser implementados por uma equipe em uma iteração.

### Forças e Limitações

#### Forças:

- Alinhamento claro entre estratégia de negócio e execução técnica.
- Coordenação eficaz de dependências entre múltiplas equipes.
- Combina estabilidade de planejamento com flexibilidade de execução.
- Governança e visibilidade estruturadas em grande escala.

#### Limitações:

- Complexidade significativa na implementação completa.
- Potencial para burocratização e overhead se não for adequadamente adaptado.
- Riscos de sacrificar princípios ágeis fundamentais em favor da escalabilidade.
- Exige investimento substancial em treinamento e mudança organizacional.

### Cenários de Aplicação Adequados

O SAFe é particularmente adequado para:

- Grandes organizações com múltiplas equipes trabalhando no mesmo produto ou portfólio.
- Ambientes regulamentados que necessitam tanto de agilidade quanto governança.

- Projetos complexos com interdependências significativas entre componentes.
- Organizações transitando de modelos tradicionais para ágeis em escala (TURRETKEN et al., 2017).

### Tendências

O SAFe continua evoluindo rapidamente:

- Maior ênfase em Business Agility e agilidade organizacional além do desenvolvimento.
- Integração de princípios Lean-Agile com práticas DevOps e entrega contínua.
- Adaptações para diferentes contextos organizacionais e domínios específicos.
- Simplificação através de implementação incremental e configurações “sob medida” (PAASIVAARA, 2017).

## 4.13 Large-Scale Scrum (LeSS)

O Large-Scale Scrum (LeSS) é um framework de escalabilidade ágil desenvolvido por Craig Larman e Bas Vodde, baseado na premissa de “mais com menos”, buscando escalar o Scrum com mínima adição de processos, papéis ou artefatos (LARMAN; VODDE, 2016).

### Processo de Requisitos no LeSS

O LeSS aborda requisitos mantendo-se fiel aos princípios do Scrum enquanto expande para múltiplas equipes:

1. **Visão Compartilhada:** Todas as equipes trabalham com um único Product Owner e um único Product Backlog, mantendo uma visão unificada do produto. Isso evita silos de requisitos e promove otimização global em vez de local (LARMAN; VODDE, 2016).
2. **Refinamento Descentralizado:** Implementa “Multi-team Product Backlog Refinement”, onde representantes de todas as equipes participam da dissecação e compreensão inicial dos itens, seguido por refinamento específico por equipe para detalhamento técnico (LARMAN; VODDE, 2016).
3. **Priorização Centralizada:** Mantém a priorização como responsabilidade exclusiva do Product Owner, garantindo foco consistente no valor de negócio mais alto. O Product Owner trabalha estreitamente com stakeholders e Product Owners de Área (em LeSS Huge) para definir prioridades (PAASIVAARA; LASSENIUS, 2019).
4. **User Stories e Aceitação:** Utiliza user stories como principal veículo para requisitos, com critérios de aceitação explícitos. LeSS enfatiza que stories devem representar valor para o usuário final, não tarefas técnicas intermediárias (LARMAN; VODDE, 2016).

5. **Áreas de Requisitos:** Em sua configuração para mais de 8 equipes (LeSS Huge), introduz o conceito de “Áreas de Requisitos” para organizar o trabalho. Cada área possui um Product Owner de Área gerenciando um subconjunto do Product Backlog, mantendo alinhamento com o Product Owner geral (SANTOS, J. et al., 2020).

### **Estrutura de Organização de Requisitos**

Mantém a estrutura Scrum com adaptações para escala (LARMAN; VODDE, 2016; SANTOS, J. et al., 2020):

- **Product Backlog:** Lista única para todas as equipes.
- **Áreas de Requisitos** (em LeSS Huge): Agrupamentos de requisitos relacionados.
- **Histórias de Usuário:** Mesma estrutura do Scrum tradicional.
- **Definition of Done:** Compartilhado entre todas as equipes.

### **Forças e Limitações**

#### **Forças:**

- Simplicidade estrutural, minimizando overhead de coordenação.
- Forte foco no produto como um todo, evitando otimizações locais.
- Preservação da experiência Scrum autêntica mesmo em escala.
- Ênfase em adaptação organizacional em vez de apenas adoção de processos.

#### **Limitações:**

- Desafios práticos com um único Product Owner para muitas equipes.
- Exige transformação organizacional significativa e mudanças estruturais.
- Menos prescritivo, requerendo maior maturidade nas práticas ágeis.
- Pode ser desafiador em organizações com estruturas funcionais rígidas.

### **Cenários de Aplicação Adequados**

O LeSS é particularmente adequado para:

- Organizações dispostas a realizar mudanças estruturais profundas.
- Produtos que beneficiam de uma visão holística e integrada.
- Ambientes que valorizam autonomia técnica e auto-organização.
- Equipes co-localizadas ou com capacidade para colaboração remota eficaz (PAASIVAARA et al., 2018).

### **Tendências**

O LeSS continua evoluindo:

- Adaptações para equipes distribuídas geograficamente.
- Integração com práticas de engenharia técnica como Continuous Delivery.
- Foco crescente em design organizacional para suportar desenvolvimento ágil.

- Expansão da comunidade de praticantes compartilhando experiências de implementação em diversos contextos (PAASIVAARA; BEHM; LASSENIUS, 2018).

#### **4.14 Quadro Resumo: Como os Processos de Desenvolvimento de Software e Frameworks Lidam com Requisitos?**

O Quadro 4 a seguir apresenta uma análise comparativa de como diferentes processos de desenvolvimento de software e frameworks ágeis abordam a engenharia de requisitos. Esta síntese destaca as características essenciais de cada abordagem, desde processos dirigidos a plano como o Cascata até frameworks de escalabilidade como SAFe e LeSS.

Para cada processo ou framework, são apresentados aspectos fundamentais como a estrutura de organização dos requisitos, as forças e limitações da abordagem, os cenários mais adequados para sua aplicação e as tendências de evolução observadas na prática atual. Esta visão consolidada permite compreender rapidamente as diferenças significativas entre os processos e frameworks, facilitando a seleção da abordagem mais adequada para contextos específicos de desenvolvimento.

Esta comparação evidencia como a engenharia de requisitos evoluiu de modelos lineares e documentação extensiva para abordagens mais adaptativas, colaborativas e centradas no valor de negócio, refletindo a transformação da engenharia de requisitos nas últimas décadas.

| Processo / Framework        | Estrutura de Organização de Requisitos                                                                                              | Forças                                                                                                                                                                                        | Limitações                                                                                                                                                                             | Cenários Adequados para Uso                                                                                                                                                                                                                   | Tendências                                                                                                                                                                                                                            |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cascata</b>              | Requisitos de Negócio, Requisitos de Usuário, Requisitos Funcionais, Requisitos Não-Funcionais, Requisitos de Interface, Restrições | <ul style="list-style-type: none"> <li>• Documentação abrangente</li> <li>• Processo bem definido</li> <li>• Adequado para sistemas críticos</li> </ul>                                       | <ul style="list-style-type: none"> <li>• Dificuldade com mudanças</li> <li>• Pressupõe requisitos completos inicialmente</li> <li>• Feedback tardio</li> </ul>                         | <ul style="list-style-type: none"> <li>• Sistemas regulamentados</li> <li>• Sistemas de missão crítica</li> <li>• Projetos com requisitos estáveis</li> </ul>                                                                                 | <ul style="list-style-type: none"> <li>• Especificações formais para componentes críticos em projetos ágeis</li> <li>• Validação formal em contextos regulamentados</li> <li>• Documentação para sistemas de longa duração</li> </ul> |
| <b>Modelo V</b>             | Requisitos de Negócio, Requisitos de Sistema, Requisitos de Arquitetura, Requisitos de Componente                                   | <ul style="list-style-type: none"> <li>• Detecção precoce de problemas</li> <li>• Rastreabilidade clara entre requisitos e testes</li> <li>• Redução de retrabalho</li> </ul>                 | <ul style="list-style-type: none"> <li>• Dificuldade com mudanças</li> <li>• Pressupõe requisitos completos</li> <li>• Verificação potencialmente rígida</li> </ul>                    | <ul style="list-style-type: none"> <li>• Sistemas críticos em segurança</li> <li>• Domínios regulamentados</li> <li>• Sistemas embarcados</li> <li>• Projetos com escopo bem definido</li> </ul>                                              | <ul style="list-style-type: none"> <li>• Incorporação de princípios ágeis</li> <li>• Automação de testes</li> <li>• Combinação com desenvolvimento contínuo</li> <li>• Aplicação seletiva para componentes críticos</li> </ul>        |
| <b>UP (Unified Process)</b> | Visão, Casos de Uso, Atores, Requisitos Suplementares, Glossário, Regras de Negócio                                                 | <ul style="list-style-type: none"> <li>• Equilíbrio entre documentação e adaptabilidade</li> <li>• Foco em arquitetura</li> <li>• Rastreabilidade</li> <li>• Validação progressiva</li> </ul> | <ul style="list-style-type: none"> <li>• Potencial documentação excessiva</li> <li>• Complexidade do processo</li> <li>• Curva de aprendizado acentuada</li> </ul>                     | <ul style="list-style-type: none"> <li>• Sistemas médios a grande porte</li> <li>• Organizações que necessitam de conformidade regulatória</li> <li>• Projetos com riscos técnicos</li> <li>• Equipes geograficamente distribuídas</li> </ul> | <ul style="list-style-type: none"> <li>• Integração com práticas ágeis</li> <li>• Redução de documentação prescritiva</li> <li>• Modelagem visual quando necessária</li> <li>• Adoção seletiva em contextos ágeis</li> </ul>          |
| <b>OpenUP</b>               | Visão, Casos de Uso/Histórias, Requisitos Técnicos, Requisitos Não-Funcionais                                                       | <ul style="list-style-type: none"> <li>• Documentação enxuta</li> <li>• Processo escalável</li> <li>• Equilíbrio entre disciplina e agilidade</li> <li>• Feedback rápido</li> </ul>           | <ul style="list-style-type: none"> <li>• Insuficiente para sistemas críticos</li> <li>• Demanda colaboração próxima</li> <li>• Menos estruturado para equipes inexperientes</li> </ul> | <ul style="list-style-type: none"> <li>• Equipes pequenas e co-localizadas</li> <li>• Projetos com comunicação direta</li> <li>• Transição gradual para ágil</li> <li>• Sistemas não-críticos</li> </ul>                                      | <ul style="list-style-type: none"> <li>• Integração com DevOps</li> <li>• Técnicas centradas no usuário</li> <li>• Ferramentas de colaboração digital</li> <li>• Práticas em métodos híbridos</li> </ul>                              |

| Processo / Framework | Estrutura de Organização de Requisitos                                                      | Forças                                                                                                                                                                                                                    | Limitações                                                                                                                                                                                                                                       | Cenários Adequados para Uso                                                                                                                                                                                                                             | Tendências                                                                                                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AUP</b>           | Visão de Produto, Histórias de Usuário, Modelos Leves, Requisitos Não-Funcionais            | <ul style="list-style-type: none"> <li>• Combina estrutura UP com agilidade</li> <li>• Caminho de transição metodológica</li> <li>• Disciplinas essenciais sem burocracia</li> <li>• Adaptabilidade contextual</li> </ul> | <ul style="list-style-type: none"> <li>• Menos adequado para equipes inexperientes</li> <li>• Documentação potencialmente insuficiente</li> <li>• Requer equilíbrio cuidadoso</li> </ul>                                                         | <ul style="list-style-type: none"> <li>• Organizações em transição metodológica</li> <li>• Projetos que requerem formalidade e adaptabilidade</li> <li>• Ambientes com necessidades de conformidade</li> <li>• Equipes com experiência mista</li> </ul> | <ul style="list-style-type: none"> <li>• Incorporação ao DAD</li> <li>• Integração com DevOps</li> <li>• Personalização para domínios específicos</li> <li>• Abordagens “objetivo-baseadas”</li> </ul>                                         |
| <b>XP</b>            | Histórias de Usuário, Critérios de Aceitação, Testes de Aceitação                           | <ul style="list-style-type: none"> <li>• Adaptabilidade excepcional</li> <li>• Feedback rápido e contínuo</li> <li>• Validação precoce</li> <li>• Redução de documentação</li> </ul>                                      | <ul style="list-style-type: none"> <li>• Dependência de envolvimento do cliente</li> <li>• Desafios com documentação para conformidade</li> <li>• Escalabilidade limitada</li> <li>• Menos adequado para requisitos complexos</li> </ul>         | <ul style="list-style-type: none"> <li>• Projetos com requisitos voláteis</li> <li>• Ambientes que priorizam feedback rápido</li> <li>• Sistemas que evoluem incrementalmente</li> <li>• Equipes pequenas com cliente presente</li> </ul>               | <ul style="list-style-type: none"> <li>• Integração em outros frameworks</li> <li>• Adoção de práticas técnicas específicas</li> <li>• Adaptações para equipes distribuídas</li> <li>• Evolução para design thinking e lean startup</li> </ul> |
| <b>FDD</b>           | Áreas, Conjuntos, Features, Tarefas                                                         | <ul style="list-style-type: none"> <li>• Clareza organizacional</li> <li>• Equilíbrio cliente/técnico</li> <li>• Sistema de relatórios detalhado</li> <li>• Revisões formais para qualidade</li> </ul>                    | <ul style="list-style-type: none"> <li>• Menos foco em requisitos não-funcionais</li> <li>• Pressupõe modelo de domínio estável</li> <li>• Menor adaptabilidade a mudanças fundamentais</li> <li>• Potencialmente documento-orientado</li> </ul> | <ul style="list-style-type: none"> <li>• Sistemas médio/grande porte</li> <li>• Equipes experientes em OO</li> <li>• Projetos que requerem visibilidade detalhada</li> <li>• Ambientes que valorizam responsabilidade individual</li> </ul>             | <ul style="list-style-type: none"> <li>• Integração com outras práticas ágeis</li> <li>• Adaptação para desenvolvimento contínuo</li> <li>• Automação de relatórios</li> <li>• Simplificação para projetos menores</li> </ul>                  |
| <b>Espiral</b>       | Objetivos, Alternativas, Restrições, Requisitos Incrementais, Requisitos baseados em riscos | <ul style="list-style-type: none"> <li>• Identificação precoce de riscos</li> <li>• Flexibilidade para mudanças</li> <li>• Validação iterativa</li> <li>• Escalabilidade para sistemas complexos</li> </ul>               | <ul style="list-style-type: none"> <li>• Requer expertise em avaliação de riscos</li> <li>• Complexo de gerenciar</li> <li>• Potencialmente custoso</li> <li>• Requer envolvimento contínuo</li> </ul>                                           | <ul style="list-style-type: none"> <li>• Projetos grandes e inovadores</li> <li>• Sistemas críticos</li> <li>• Requisitos iniciais pouco claros</li> <li>• Desenvolvimento sem precedentes</li> </ul>                                                   | <ul style="list-style-type: none"> <li>• Incorporação em frameworks ágeis</li> <li>• Aplicação em DevSecOps</li> <li>• Adaptação para IA e sistemas ciber-físicos</li> <li>• Integração com design thinking e lean</li> </ul>                  |

| Processo / Framework | Estrutura de Organização de Requisitos                                   | Forças                                                                                                                                                                                                                      | Limitações                                                                                                                                                                                                                                | Cenários Adequados para Uso                                                                                                                                                                                                             | Tendências                                                                                                                                                                                                            |
|----------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DSDM</b>          | Business Case, Requisitos de Negócio, Requisitos de Solução              | <ul style="list-style-type: none"> <li>• Foco em valor de negócio</li> <li>• Framework de governança abrangente</li> <li>• Clareza de prioridades</li> <li>• Integração técnica/negócios</li> </ul>                         | <ul style="list-style-type: none"> <li>• Complexo e prescritivo</li> <li>• Requer compromisso organizacional</li> <li>• Potencialmente burocrático</li> <li>• Desafios na transição estratégico-técnica</li> </ul>                        | <ul style="list-style-type: none"> <li>• Organizações com restrições de prazo/custo</li> <li>• Ambientes corporativos tradicionais</li> <li>• Projetos com necessidades de conformidade</li> <li>• Transformação de negócios</li> </ul> | <ul style="list-style-type: none"> <li>• Integração com outros frameworks</li> <li>• Adaptação para transformação digital</li> <li>• Evolução para APF</li> <li>• Incorporação de agilidade organizacional</li> </ul> |
| <b>RAD</b>           | Requisitos de Negócio, Modelos de Interface, Modelos de Dados, Processos | <ul style="list-style-type: none"> <li>• Redução no tempo de elicitação</li> <li>• Visualização precoce</li> <li>• Alta satisfação do usuário</li> <li>• Detecção precoce de problemas</li> </ul>                           | <ul style="list-style-type: none"> <li>• Menos adequado para sistemas complexos</li> <li>• Risco de foco em interfaces</li> <li>• Dependência de ferramentas CASE</li> <li>• Necessidade de comprometimento intenso</li> </ul>            | <ul style="list-style-type: none"> <li>• Sistemas de informação corporativos</li> <li>• Projetos com prazos curtos</li> <li>• Requisitos difíceis de articular</li> <li>• Validação de conceito</li> </ul>                              | <ul style="list-style-type: none"> <li>• Integração com DevOps</li> <li>• Ferramentas low-code/no-code</li> <li>• Adaptação para mobile-first</li> <li>• Evolução para COD</li> </ul>                                 |
| <b>Scrum</b>         | Visão, Product Backlog (Épicos, Histórias, Tarefas), Definition of Done  | <ul style="list-style-type: none"> <li>• Alta adaptabilidade</li> <li>• Transparência contínua</li> <li>• Validação frequente</li> <li>• Refinamento progressivo</li> </ul>                                                 | <ul style="list-style-type: none"> <li>• Desafios com requisitos de longo prazo</li> <li>• Documentação potencialmente insuficiente</li> <li>• Dependência do Product Owner</li> <li>• Desafios com dependências entre equipes</li> </ul> | <ul style="list-style-type: none"> <li>• Ambientes com requisitos emergentes</li> <li>• Projetos que se beneficiam de feedback rápido</li> <li>• Sistemas modulares</li> <li>• Equipes multifuncionais autônomas</li> </ul>             | <ul style="list-style-type: none"> <li>• Integração com design thinking/UX</li> <li>• Técnicas de mapeamento de impacto</li> <li>• Combinação com Kanban</li> <li>• Frameworks escalados</li> </ul>                   |
| <b>Kanban</b>        | Itens de Trabalho, Classes de Serviço, Políticas Explícitas              | <ul style="list-style-type: none"> <li>• Flexibilidade para mudanças em tempo real</li> <li>• Visualização clara de gargalos</li> <li>• Adaptação não-disruptiva</li> <li>• Gerenciamento de diferentes demandas</li> </ul> | <ul style="list-style-type: none"> <li>• Menos estruturado para planejamento</li> <li>• Potencial diluição de foco</li> <li>• Requer maturidade organizacional</li> <li>• Desafios de previsibilidade inicial</li> </ul>                  | <ul style="list-style-type: none"> <li>• Equipes de suporte/manutenção</li> <li>• Ambientes com mudanças frequentes</li> <li>• Melhoria de processos gradual</li> <li>• Sistemas que requerem entrega contínua</li> </ul>               | <ul style="list-style-type: none"> <li>• Integração com frameworks de escala</li> <li>• Práticas “upstream”</li> <li>• Ferramentas de simulação/análise</li> <li>• Expansão para gestão de portfólio</li> </ul>       |
| <b>SAFe</b>          | Epics, Capabilities, Features, Stories                                   | <ul style="list-style-type: none"> <li>• Alinhamento estratégia-execução</li> </ul>                                                                                                                                         | <ul style="list-style-type: none"> <li>• Complexidade significativa</li> <li>• Potencial burocratização</li> </ul>                                                                                                                        | <ul style="list-style-type: none"> <li>• Grandes organizações multi-equipe</li> </ul>                                                                                                                                                   | <ul style="list-style-type: none"> <li>• Ênfase em Business Agility</li> <li>• Integração com DevOps</li> </ul>                                                                                                       |

| Processo / Framework | Estrutura de Organização de Requisitos                                                  | Forças                                                                                                                                                                                              | Limitações                                                                                                                                                                                                                    | Cenários Adequados para Uso                                                                                                                                                                                                           | Tendências                                                                                                                                                                                                                       |
|----------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      |                                                                                         | <ul style="list-style-type: none"> <li>• Coordenação de dependências</li> <li>• Combina estabilidade e flexibilidade</li> <li>• Governança estruturada</li> </ul>                                   | <ul style="list-style-type: none"> <li>• Riscos de sacrificar princípios ágeis</li> <li>• Exige investimento substancial</li> </ul>                                                                                           | <ul style="list-style-type: none"> <li>• Ambientes regulamentados</li> <li>• Projetos complexos com interdependências</li> <li>• Organizações em transição para ágil</li> </ul>                                                       | <ul style="list-style-type: none"> <li>• Adaptações para contextos específicos</li> <li>• Implementação incremental</li> </ul>                                                                                                   |
| <b>LeSS</b>          | Product Backlog único, Áreas de Requisitos, Histórias, Definition of Done compartilhado | <ul style="list-style-type: none"> <li>• Simplicidade estrutural</li> <li>• Foco no produto como um todo</li> <li>• Preservação da experiência Scrum</li> <li>• Adaptação organizacional</li> </ul> | <ul style="list-style-type: none"> <li>• Desafios com Product Owner único</li> <li>• Exige transformação organizacional</li> <li>• Requer maturidade em práticas ágeis</li> <li>• Desafiador em estruturas rígidas</li> </ul> | <ul style="list-style-type: none"> <li>• Organizações abertas a mudanças estruturais</li> <li>• Produtos com visão holística</li> <li>• Ambientes que valorizam autonomia</li> <li>• Equipes com capacidade de colaboração</li> </ul> | <ul style="list-style-type: none"> <li>• Adaptações para equipes distribuídas</li> <li>• Integração com Continuous Delivery</li> <li>• Foco em design organizacional</li> <li>• Expansão da comunidade de praticantes</li> </ul> |

Quadro 4 – Como os Processos de Desenvolvimento de Software e Frameworks Lidam com Requisitos?

Fonte: elaborado pelo autor (2025).



## CAPÍTULO 4: PONTOS-CHAVE

- O capítulo analisa **como diferentes processos de desenvolvimento de software e frameworks tratam a engenharia de requisitos**, destacando que a forma de lidar com requisitos é um dos principais fatores que distinguem abordagens metodológicas na engenharia de software.
- Cada processo ou framework carrega uma **filosofia própria de captura, documentação, validação e gestão de requisitos**, refletindo seus valores centrais — como previsibilidade, controle, adaptabilidade ou colaboração contínua.
- O **modelo Cascata** aborda os requisitos como uma fase inicial e estática, buscando completude e estabilidade antes do início do design e da implementação. Sua ênfase está na **documentação formal** e na aprovação sequencial, sendo adequado para contextos com **requisitos fixos e ambientes regulamentados**.
- O **Modelo V** mantém a linearidade do Cascata, mas introduz o **paralelismo entre fases de desenvolvimento e testes**, reforçando a verificação e validação dos requisitos desde o início. É apropriado para sistemas de **alta criticidade e exigência de conformidade**.
- O **Unified Process (UP)** estabelece um **processo iterativo e incremental**, guiado por **casos de uso e centrado na arquitetura**, permitindo que requisitos evoluam de forma controlada a cada iteração. Favorece rastreabilidade e adaptação em projetos de médio e grande porte.
- O **OpenUP** representa uma versão leve do UP, mantendo princípios de iteração, priorização por valor e riscos e **colaboração entre stakeholders**, mas com **redução de formalismo e documentação**. Adequa-se a equipes pequenas e contextos ágeis.
- O **Agile Unified Process (AUP)** combina práticas do UP com princípios ágeis, promovendo **curtos ciclos de entrega, validação contínua e integração frequente de requisitos**, conciliando estrutura e flexibilidade.
- O **Feature-Driven Development (FDD)** organiza o processo em torno de **funcionalidades (“features”) pequenas e bem definidas**, com modelagem prévia do domínio e **planejamento incremental baseado em valor de negócio**.
- O **Modelo Espiral** introduz a **análise sistemática de riscos como eixo central** da engenharia de requisitos, permitindo revisões sucessivas e evolução progressiva do produto. É especialmente útil em **projetos complexos e de alta incerteza**.
- O **Dynamic Systems Development Method (DSDM)** enfatiza **colaboração, entrega incremental e priorização de valor de negócio**, estruturando os requisitos em níveis (essenciais, condicionais e desejáveis) e reforçando a **participação ativa dos usuários**.
- O **Rapid Application Development (RAD)** foca na **velocidade de entrega e iteração com usuários**, utilizando prototipação e feedback constante para refinar requisitos, priorizando **tempo de resposta e flexibilidade**.
- O **Scrum** organiza o trabalho em **sprints e backlog priorizado**, tratando requisitos como **itens de produto (user stories)** que evoluem com o aprendizado do time e o feedback do

cliente. A **responsabilidade coletiva e a inspeção adaptativa** guiam a evolução dos requisitos.

- O **Kanban** gerencia requisitos de forma contínua e visual, através do **fluxo de trabalho limitado em progresso (WIP)**, permitindo priorização dinâmica e foco em eficiência e estabilidade do fluxo.
- O **Scaled Agile Framework (SAFe)** amplia os princípios ágeis para múltiplos níveis organizacionais, articulando requisitos em diferentes camadas (épicas, features e histórias) e **alinhando estratégia, produto e execução**.
- O **Large-Scale Scrum (LeSS)** estende o Scrum para múltiplas equipes, mantendo simplicidade e princípios ágeis fundamentais, com **coordenação descentralizada e foco na visão integrada do produto**.
- A **tabela comparativa final do capítulo** sintetiza as diferenças entre todos os processos e frameworks quanto à estrutura de requisitos, forças, limitações, contextos de aplicação e tendências, revelando uma **evolução contínua da engenharia de requisitos**: de práticas lineares e documentais para abordagens **iterativas, colaborativas e centradas em valor**.
- O capítulo conclui que **não há uma forma única de tratar requisitos**, mas múltiplas estratégias moldadas por valores, contextos e objetivos. A escolha adequada depende da **natureza do produto, da cultura organizacional e da maturidade da equipe**, reforçando a importância da **adaptabilidade metodológica** na engenharia contemporânea de requisitos.

# Capítulo 5

Após examinarmos a evolução histórica da Engenharia de Software e como diferentes processos e frameworks lidam com requisitos, este capítulo aprofunda-se nos fundamentos e conceitos essenciais da Engenharia de Requisitos.

## 5 A Engenharia de Requisitos

A Engenharia de Requisitos (ER) é uma disciplina dinâmica e interdisciplinar que evolui continuamente em resposta às mudanças das necessidades técnicas, sociais, humanas e organizacionais. Ao longo dos anos, conceitos centrais têm se mantido constantes, enquanto novas perspectivas e ideias emergem, enriquecendo o entendimento da área.

Inicialmente, conforme descrito por Zave (1997), o foco principal da ER recaía sobre objetivos técnicos, funções e restrições dos sistemas. Entretanto, progressivamente, autores como Nuseibeh e Easterbrook (2000) ampliaram essa visão, incorporando elementos das ciências sociais e cognitivas, reforçando a importância de entender como as pessoas interagem com a tecnologia e como contextos sociais influenciam essas interações.

Carvalho (2003), introduziu a perspectiva de um processo iterativo, incremental e criativo, que considera diversos segmentos organizacionais, enfatizando ainda mais a complexidade social e cognitiva envolvida. Easterbrook (2004), por sua vez, reforçou a dimensão comunicativa da Engenharia de Requisitos, destacando a necessidade de estabelecer claramente o contexto de uso dos sistemas, percebendo-a como uma ponte vital entre necessidades humanas e possibilidades tecnológicas.

Van Lamsweerde (2009) e Pohl (2010) aprofundaram essa visão multidimensional, destacando explicitamente a importância dos requisitos não-funcionais mais relacionados a qualidade do produto e do contexto organizacional e técnico no qual os sistemas operam. Além disso, a perspectiva normativa representada pela ISO/IEC/IEEE 29148:2018 aborda a ER como uma área interdisciplinar que visa realizar a mediação (ponte) entre os domínios do adquirente e do fornecedor ou desenvolvedor, para estabelecer e manter os requisitos a serem atendidos pelo sistema, produto de software ou serviço de interesse.

Recentemente, Berry (2019) e Laplante e Kassab (2022) trouxeram à tona aspectos ainda mais complexos e contemporâneos, enfatizando fortemente o papel da comunicação, negociação e gestão de interesses diversos dos stakeholders. Eles destacaram desafios éticos e de sustentabilidade como preocupações emergentes, refletindo uma conscientização maior da complexidade sociotécnica dos sistemas.

Em síntese, ao longo do tempo, a Engenharia de Requisitos tem mantido princípios fundamentais como o processo sistemático (KOTONYA; SOMMERVILLE, 1998; WIEGERS e BEATTY, 2013) e a centralidade dos stakeholders (NUSEIBEH; EASTERBROOK, 2000), enquanto expande continuamente suas fronteiras conceituais para incluir perspectivas sociais, comunicativas e éticas mais profundas (EASTERBROOK, 2004; BERRY, 2019; LAPLANTE; KASSAB,

2022). Essa combinação assegura sua relevância constante diante dos desafios atuais e futuros no desenvolvimento de produtos de software. A Figura 33 ilustra essa evolução conceitual.

Este livro reconhece a validade e relevância de todas as conceituações e perspectivas apresentadas pelos autores citados, as quais contribuíram significativamente para o amadurecimento da Engenharia de Requisitos ao longo das décadas. Nossa proposta não é formular um novo conceito ou complemento teórico, mas sim evidenciar a natureza multifacetada desta disciplina, que integra dimensões técnicas, humanas, sociais e organizacionais. Compreendemos a Engenharia de Requisitos como a mais humana das disciplinas da Engenharia de Software, na qual o profissional realiza um trabalho complexo de mediação entre distintas realidades, necessidades e expectativas, construindo pontes essenciais entre pessoas e tecnologias para viabilizar soluções que efetivamente atendam às necessidades dos stakeholders.

## Linha do Tempo Conceitual da Engenharia de Requisitos

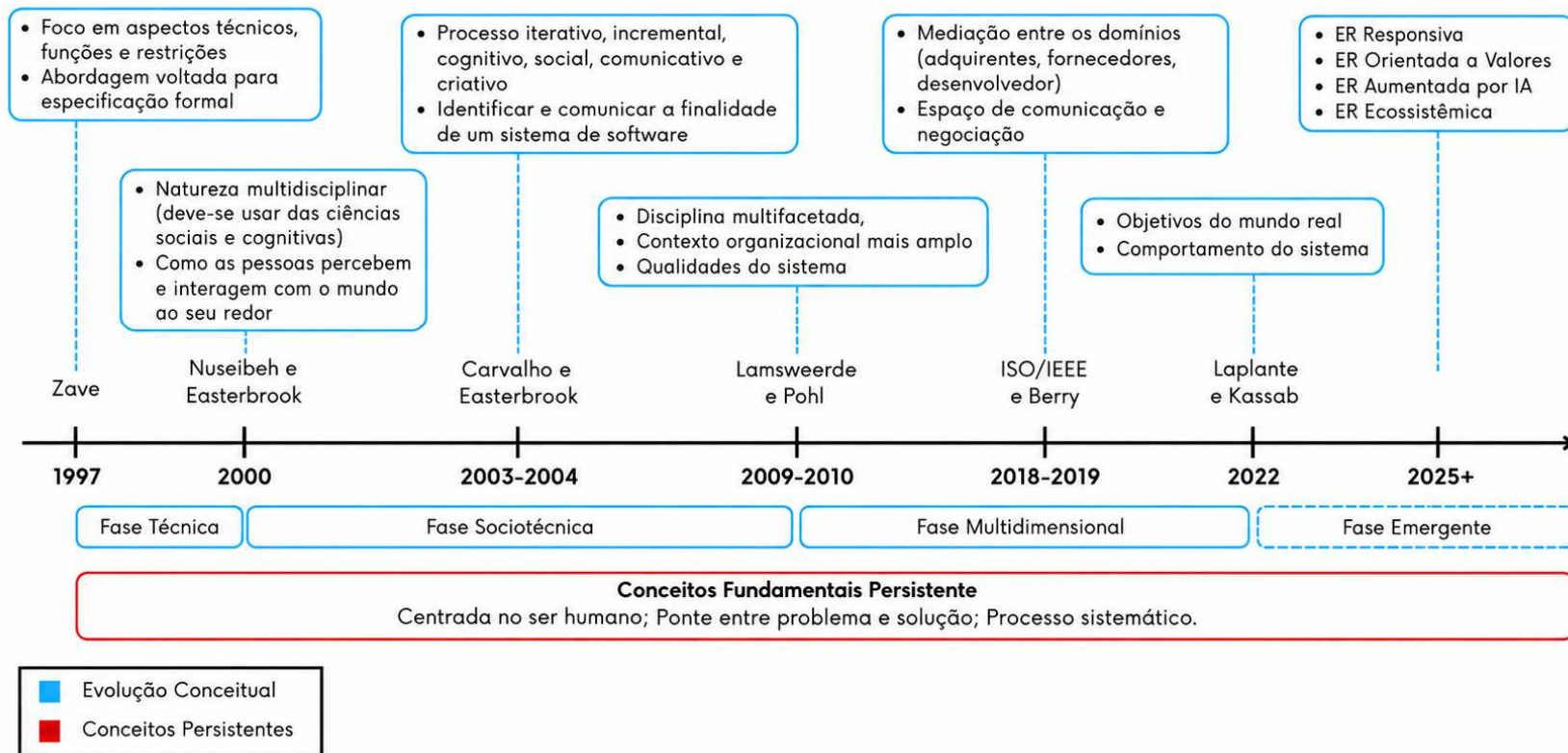


Figura 33 – Evolução conceitual da Engenharia de Requisitos.  
Fonte: elaborado pelo autor (2025).

## 5.1 Considerações Iniciais sobre as Atividades da ER

Para compreender como a Engenharia de Requisitos (ER) funciona na prática, é essencial examinar suas atividades fundamentais, as quais ocorrem tanto em processos dirigidos por plano quanto em abordagens ágeis ou híbridas, embora a filosofia de cada paradigma influencie significativamente sua execução.

A Engenharia de Requisitos, como disciplina da Engenharia de Software, é essencialmente centrada no ser humano. Embora demande competências técnicas sólidas, seu êxito depende, sobretudo, da qualidade das interações entre pessoas: stakeholders e engenheiros de software. Requisitos não são apenas informações técnicas; eles são construções comunicativas que refletem necessidades, expectativas, restrições e objetivos, em constante negociação e alinhamento.

Nesse sentido, optamos por enfatizar a **comunicação** como princípio estruturante da disciplina. Desde a elicitação e descoberta, passando pela organização e atualização, até chegar à validação, todo o processo de requisitos é permeado pela forma como informações circulam entre as fontes de conhecimento e os responsáveis pela construção do produto.

As seis atividades da Engenharia de Requisitos apresentadas neste livro não seguem um único formato estabelecido (sequencial, iterativo etc.). Na prática, elas estão profundamente entrelaçadas, com feedback contínuo entre si, de modo que elicitação e descoberta, análise e consenso, declaração, representação, verificação e validação e, organização e atualização podem ocorrer em paralelo e em diferentes níveis de abstração. Sua eficácia depende tanto de competências técnicas quanto de habilidades interpessoais, refletindo a natureza dual da ER como disciplina ao mesmo tempo técnica e social.

O propósito integrado dessas atividades é construir e sustentar, ao longo da evolução do produto, um entendimento suficientemente claro, compartilhado e útil sobre seus requisitos. Cada atividade assume uma responsabilidade predominante nessa construção, sem deixar de se relacionar com as demais. Falhas nesse trabalho podem resultar em consequências graves, como software excelente para o problema errado, desperdício de recursos, frustração pessoal, insatisfação dos usuários, fracasso do projeto e potenciais consequências ambientais negativas.

A Figura 34 ilustra as atividades operacionais da ER, sem a intenção de indicar sequencialidade em sua realização. Na prática, várias dessas atividades ocorrem paralelamente ou facilitando umas às outras. As “pequenas bolinhas vermelhas”, na figura, representam os elementos abordados pela ER – problemas, necessidades, características, requisitos funcionais e não funcionais – ilustrando que podem passar por uma ou várias atividades, conforme necessário. A seguir, cada atividade será brevemente descrita para compreensão inicial, sendo posteriormente detalhada com suas técnicas, exemplos e cenários de aplicação.

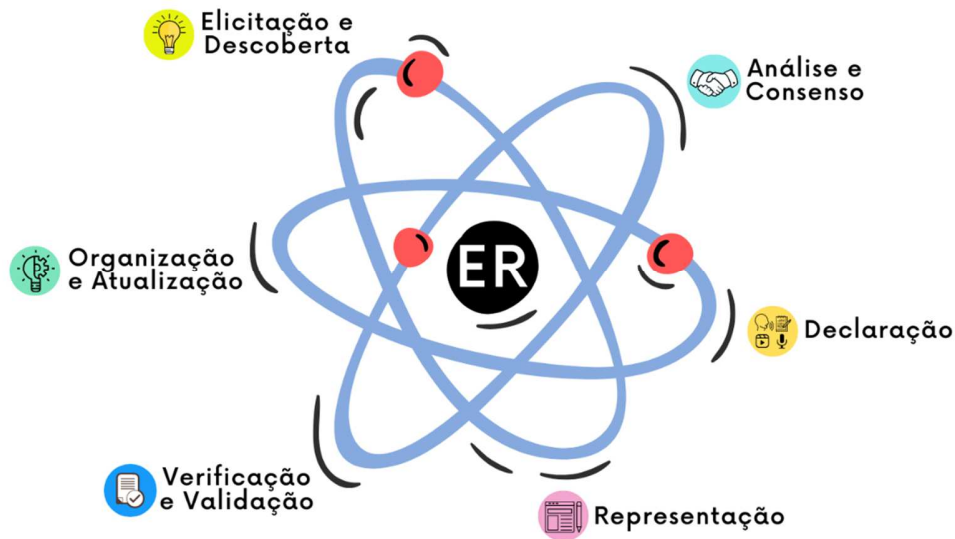


Figura 34 – Atividades de Engenharia de Requisitos.  
Fonte: elaborado pelo autor (2025).

## 5.2 Escolhas Terminológicas

Deste ponto em diante, neste livro, algumas decisões terminológicas foram tomadas para refletir a natureza comunicativa e adaptativa da Engenharia de Requisitos.

Inicialmente, é possível observar que, na literatura, a atividade inicial da ER aparece sob diferentes denominações, tais como “**levantamento de requisitos**” (PRESSMAN; MAXIM, 2021), “**aquisição de requisitos**” (MAIDEN; RUGG, 1996), “**identificação de requisitos**” (POHL, 2010), entre outros. Embora essas expressões descrevam essencialmente o processo de obter informações de stakeholders e do contexto, neste livro optamos por utilizar o termo **elicitação e descoberta** de requisitos por ele ressaltar tanto a dimensão de elicitar informações declaradas quanto a de descobrir requisitos latentes, muitas vezes não verbalizados.

Também optamos por não utilizar “**documentação**” ou “**especificação**” como **nomes de atividades** de Engenharia de Requisitos. Neste livro, esses termos são tratados como **formas/técnicas de declaração e representação** (isto é, meios e formatos pelos quais os requisitos podem ser expressos). O mais importante é assegurar a comunicação eficaz do requisito, independentemente do formato adotado, seja textual, áudio ou vídeo.

Da mesma forma, não utilizamos o termo “**modelagem de requisitos**”, pois entendemos que **representação de requisitos** é uma noção mais abrangente, capaz de englobar diferentes meios de comunicação — como prototipação, diagramas ou notações formais — sem restringir-se a uma técnica específica.

Também evitamos o uso do termo “**gerenciamento de requisitos**”, geralmente associado a práticas formais e rigorosas (SWEBOK, 2024). Em seu lugar, utilizamos **organização e atualização de requisitos**, mais simples, flexível e adaptável à evolução natural dos produtos de software.

Essas escolhas não se limitam a ajustes de vocabulário, mas refletem uma visão conceitual que compreende a Engenharia de Requisitos como uma prática essencialmente comunicativa e adaptativa. Essa perspectiva orienta toda a abordagem adotada neste livro.

### 5.3 Atividades da Engenharia de Requisitos

As considerações apresentadas nas seções anteriores — tanto sobre a centralidade da comunicação quanto sobre as escolhas terminológicas adotadas — servem de base para a compreensão das atividades que compõem a Engenharia de Requisitos.

Conforme ilustrado na Figura 34, essas atividades não devem ser entendidas como etapas lineares, mas como um conjunto de práticas iterativas e entrelaçadas, que se apoiam mutuamente. Em diferentes momentos de um projeto, elas podem ocorrer em paralelo, retroalimentar-se ou mesmo ser revisitadas, de acordo com a evolução do entendimento dos requisitos e do próprio produto de software.

Ao longo das próximas subseções, cada atividade será brevemente descrita, destacando seu propósito. Essa descrição inicial tem caráter introdutório e será aprofundada posteriormente em capítulos específicos.

Para fins deste livro, a distinção entre as atividades será feita principalmente pela responsabilidade que cada uma assume, e não pelo artefato, método ou técnica utilizada. Uma mesma técnica pode apoiar diferentes atividades, e atividades distintas podem ocorrer simultaneamente. O enquadramento depende do propósito com que a técnica está sendo empregada e do resultado que se pretende produzir.

#### 5.3.1 Elicitação e Descoberta

A elicitación e descoberta é a atividade voltada à exploração contínua de informações relevantes para compreender o problema, o contexto, as necessidades e os requisitos relacionados ao produto. Embora costume ganhar maior intensidade nos momentos iniciais de uma iniciativa, não se restringe a eles: novas dúvidas, mudanças de contexto, feedbacks, evidências de uso e aprendizados podem exigir sua retomada ao longo de toda a evolução do produto.

Suas fontes incluem stakeholders, documentos, sistemas existentes, observação do contexto de uso, dados, ferramentas e outros meios capazes de ampliar o conhecimento disponível. Seu foco está em descobrir e aprofundar informações, necessidades, objetivos, regras, restrições e possíveis requisitos. Quando uma descoberta exige uma decisão sobre significado, viabilidade, prioridade ou escopo, o trabalho prossegue em Análise e Consenso; quando o conteúdo precisa ser expresso linguisticamente, prossegue em Declaração.

#### 5.3.2 Análise e Consenso

A análise e consenso é a atividade responsável por construir entendimento acordado sobre o significado, a viabilidade, a prioridade e o escopo dos requisitos. A análise examina as informações disponíveis para esclarecer significados e implicações, compreender alternativas, dependências, riscos e restrições e avaliar a viabilidade do que está sendo considerado.



O consenso trata das situações em que diferentes interpretações, interesses ou possibilidades precisam ser confrontados e negociados para que decisões possam ser estabelecidas. É também nessa atividade que ocorre a priorização propriamente dita: critérios são definidos ou acordados, as avaliações necessárias são obtidas, os critérios são aplicados e os resultados são discutidos até que prioridades propostas ou acordadas possam ser estabelecidas.

As decisões resultantes orientam o conteúdo posteriormente declarado ou representado e são incorporadas ao escopo mantido pela atividade de Organização e Atualização.

### 5.3.3 Declaração

A declaração é a atividade responsável por expressar linguisticamente os requisitos de modo adequado ao propósito comunicacional, ao público e ao nível de abstração necessário. Essa expressão pode ocorrer por meios escritos, orais ou multimodais, com diferentes graus de estruturação.

A declaração trabalha sobre o entendimento produzido pelas demais atividades: ela não decide o que o requisito deve significar, mas escolhe como esse conteúdo será enunciado para que possa ser comunicado e utilizado. Assim, requisitos funcionais e não funcionais, regras de negócio, critérios e outras informações podem ser formulados por narrativas, declarações estruturadas, histórias de usuário, especificações textuais de casos de uso, tabelas ou outros formatos adequados ao contexto.

Quando o entendimento muda ou amadurece, a declaração é revista para expressar adequadamente o conteúdo vigente.

### 5.3.4 Representação

A representação é a atividade responsável por tornar visíveis aspectos do entendimento dos requisitos que se beneficiam de formas complementares à declaração textual. Diagramas, modelos, protótipos, mockups, storyboards e representações formais podem explicitar contexto, comportamento, estados, fluxos, dados, responsabilidades, interações, relações ou propriedades.

Laplante (2022) categoriza as representações de requisitos em três níveis:

- **Informal:** representações informais, ainda que possam seguir algum padrão visual;
- **Semiformal:** representações que seguem uma notação ou linguagem específicas;
- **Formal:** representações baseadas em uma sintaxe e semântica matemática.

A escolha da representação e de seu nível de formalidade depende do aspecto que se pretende tornar perceptível, do público envolvido e do propósito de sua utilização. Representação pode ocorrer em paralelo com outras atividades: um protótipo, por exemplo, é produzido como representação e pode simultaneamente ser utilizado para apoiar descoberta, análise ou validação. Nesses casos, a produção e a revisão do artefato permanecem responsabilidade da Representação, enquanto sua utilização atende ao propósito da atividade que ele está apoiando.

### 5.3.5 Verificação e Validação

A verificação e validação é a atividade responsável por avaliar a qualidade dos requisitos e sua adequação ao entendimento e ao contexto. A verificação (qualidade interna) examina propriedades de qualidade dos requisitos como, consistência, completude, testabilidade e outras características intrínsecas dos requisitos declarados, assim como a coerência entre os elementos relacionados; a validação (qualidade externa) confronta os requisitos com necessidades, objetivos, expectativas, valor e contexto dos stakeholders.

Sua questão central é: *os requisitos possuem qualidade suficiente e correspondem ao entendimento correto do que precisa ser atendido?*

Tal questão pode ser subdividida em:

- Verificação: *“Estou realizando de maneira correta o requisito?”*
- Validação: *“Estou realizando o requisito correto?”*

O resultado da atividade são evidências, achados, dúvidas e divergências que permitem avaliar a adequação dos requisitos e indicar a necessidade de retorno a outras atividades. V&V não assume a responsabilidade de corrigir o conteúdo, reformular declarações, alterar representações ou redefinir prioridades; ela evidencia o que precisa ser revisto e fornece elementos para que a atividade responsável realize o ajuste.

### 5.3.6 Organização e Atualização

A organização e atualização é a atividade responsável por manter o escopo de requisitos do produto estruturado e coerente com o entendimento vigente.

A Organização diz respeito à forma como os requisitos e os elementos relacionados ao escopo são dispostos em estruturas adequadas ao contexto do produto, como backlogs, catálogos, mapas, roadmaps, baselines ou outros arranjos. É também na organização que são estabelecidas as relações de rastreabilidade necessárias para conectar requisitos a suas fontes, justificativas, critérios, representações, decisões e demais elementos relacionados.

A Atualização mantém essas estruturas e relações aderentes ao entendimento atual, incorporando mudanças, decisões e resultados produzidos pelas demais atividades e preservando versões e histórico no nível necessário ao contexto.

Prioridades, decisões sobre escopo, viabilidade ou significado são produzidas em Análise e Consenso; declarações são produzidas em Declaração; representações são produzidas em Representação; e achados de qualidade são produzidos em V&V. Organização e Atualização incorpora esses resultados ao escopo estruturado e mantém suas relações coerentes ao longo do tempo.

### 5.3.7 Síntese das atividades

Embora possuam responsabilidades próprias, as atividades da Engenharia de Requisitos não funcionam de forma isolada. Elas se relacionam, podem ocorrer em paralelo e se retroalimentam à medida que novas informações, decisões e aprendizados surgem. Para tornar mais explícito o papel predominante de cada uma, o Quadro 5 sintetiza suas responsabilidades centrais, as questões que orientam sua realização e seus principais resultados. Essa distinção não estabelece uma

sequência de execução, mas oferece referências para compreender o que cada atividade busca realizar no conjunto da Engenharia de Requisitos.

| Atividade                        | Responsabilidade central                                                                                                                        | Questão central da atividade                                                                                                    | Principais resultados                                                                                                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Elicitação e Descoberta</b>   | Explorar continuamente informações relevantes para compreender o problema, o contexto, as necessidades e os requisitos relacionados ao produto. | <i>O que precisamos compreender ou descobrir, neste momento, sobre o problema, o contexto, as necessidades e os requisitos?</i> | Informações sobre contexto e domínio; necessidades, objetivos, restrições, regras e requisitos candidatos ou preliminares identificados.                                                        |
| <b>Análise e Consenso</b>        | Transformar os insumos disponíveis em um entendimento analisado e acordado sobre o que precisa ser atendido.                                    | <i>O que precisa ser compreendido e acordado quanto ao conteúdo, viabilidade, prioridade e escopo?</i>                          | Entendimento refinado e acordado; conflitos tratados; prioridades e escopo acordados; decisões, critérios e compromissos estabelecidos.                                                         |
| <b>Declaração</b>                | Expressar linguisticamente os requisitos de forma adequada ao propósito, ao público e ao nível de abstração.                                    | <i>Como cada requisito deve ser expresso para comunicar adequadamente seu conteúdo?</i>                                         | Requisitos declarados em formas linguísticas adequadas, compreensíveis e compatíveis com o nível de abstração necessário.                                                                       |
| <b>Representação</b>             | Representar aspectos do entendimento dos requisitos por meios que favoreçam compreensão, comunicação, análise ou decisão.                       | <i>Que aspectos do entendimento precisam ser representados para apoiar sua compreensão, comunicação, análise ou decisão?</i>    | Diagramas, modelos, protótipos, mapas, visualizações e outras representações do entendimento.                                                                                                   |
| <b>Verificação e Validação</b>   | Avaliar a qualidade dos requisitos e sua adequação ao entendimento e ao contexto.                                                               | <i>Os requisitos possuem qualidade suficiente e correspondem ao entendimento correto do que precisa ser atendido?</i>           | Achados e evidências sobre qualidade e adequação; defeitos, lacunas, inconsistências, dúvidas e necessidades de revisão identificados.                                                          |
| <b>Organização e Atualização</b> | Manter o escopo de requisitos do produto estruturado e coerente com o entendimento vigente sobre o produto.                                     | <i>Como o escopo de requisitos deve ser estruturado e mantido aderente ao entendimento vigente?</i>                             | Escopo de requisitos estruturado e atualizado; estruturas de organização mantidas; vínculos de rastreabilidade estabelecidos e atualizados; versões e histórico preservados quando necessários. |

Quadro 5 – Responsabilidade, questão central e principais resultados das atividades de ER.

Fonte: elaborado pelo autor (2026).

O Quadro 5 evidencia diferenças de responsabilidade sem estabelecer fronteiras rígidas de execução. Uma técnica, uma representação ou um artefato pode apoiar mais de uma atividade, e uma atividade pode produzir resultados que demandem a retomada de outra. O que distingue

cada atividade é sua responsabilidade predominante diante do entendimento que está sendo construído. A Figura 35 ilustra essas informações.

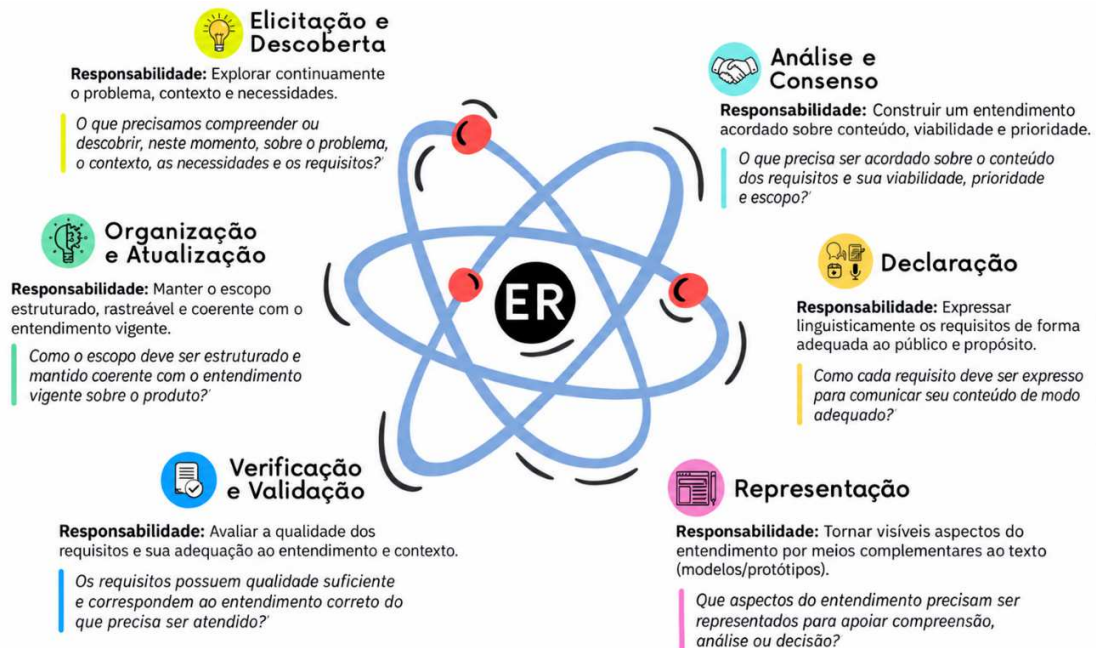


Figura 35 – Responsabilidades e questões das atividades de ER.  
Fonte: elaborado pelo autor (2026).

Nos capítulos 7 a 12, todas as atividades da ER serão detalhadas. No entanto, para que sejam realizadas com eficácia, é necessário um alicerce sólido de princípios e valores que orientem como os profissionais devem abordar e executar essas tarefas (Figura 36). A seção a seguir explora esses fundamentos essenciais.

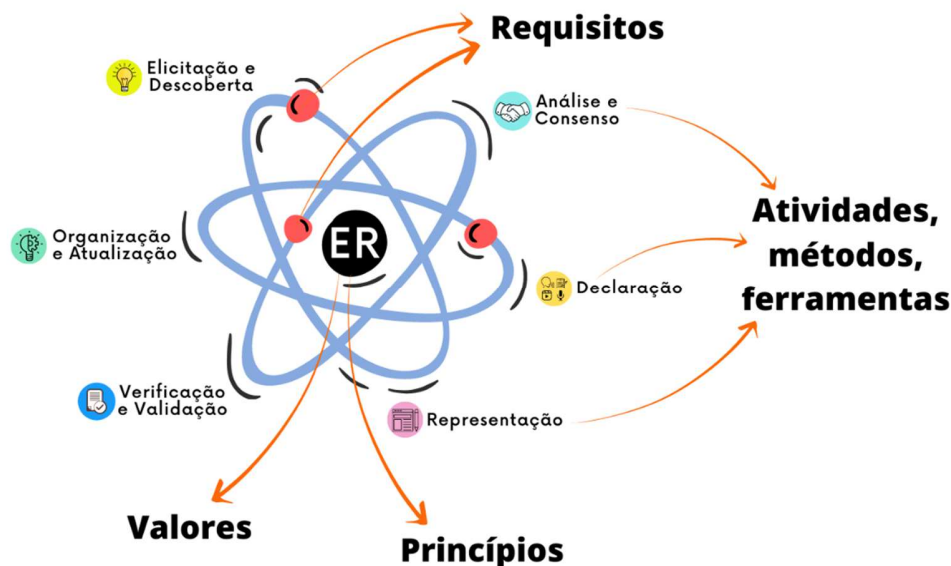


Figura 36 – Engenharia de Requisitos e seus Componentes.  
Fonte: elaborado pelo autor (2025).

## 5.4 Valores e Princípios da Engenharia de Requisitos

A Engenharia de Requisitos, assim como outras disciplinas da Engenharia de Software, beneficia-se significativamente quando guiada por um conjunto claro de valores e princípios. Estes funcionam como uma bússola que orienta decisões e comportamentos, especialmente em situações ambíguas ou desafiadoras.

Os valores e princípios apresentados, a seguir, foram propostos tendo como base o Handbook for the CPRE Foundation Level, version 1.1.0, September 2022 (IREB, 2022), e a obra “Mastering the Requirements Process” de Suzanne Robertson e James Robertson (2014), além de incorporar elementos adaptados dos valores e princípios do Manifesto Ágil (BECK et al., 2001) e do eXtreme Programming (BECK e ANDRES, 2004).

Enquanto as **atividades** descritas anteriormente fornecem o “*como fazer*” da Engenharia de Requisitos, os **valores e princípios** oferecem o “*por que fazer*” e “*com qual mentalidade fazer*”. Esta fundamentação ética e filosófica é particularmente importante para uma disciplina que se situa na interseção entre necessidades humanas e soluções tecnológicas.

Vale ressaltar que estes não são os únicos valores e princípios possíveis, mas representam um conjunto fundamental que pode guiar efetivamente a prática da Engenharia de Requisitos. O mais importante é que a equipe alinhe seu comportamento a valores compartilhados, criando uma base comum para colaboração e tomada de decisões.

Nas seções seguintes, exploraremos sete valores essenciais e onze princípios fundamentais que podem transformar significativamente a eficácia da Engenharia de Requisitos em qualquer organização.

### 5.4.1 Os 7 Valores da ER

Os valores propostos para a Engenharia de Requisitos (Figura 37) são derivados e adaptados principalmente do eXtreme Programming (XP) e do Scrum, acrescidos da confiança. Importante ressaltar que estes valores não direcionam a ER para uma abordagem específica, seja ágil ou dirigida a plano, mas sim promovem uma prática adaptável ao contexto de aplicação. Embora não sejam os únicos valores possíveis, funcionam como elementos de condução da ER, sendo fundamental que o comportamento da equipe esteja alinhado com estes valores.

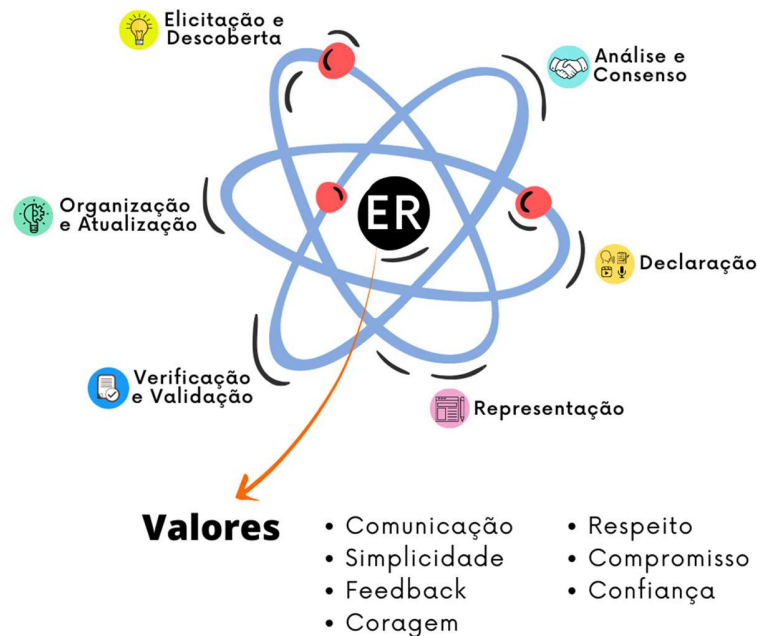


Figura 37 – Valores da Engenharia de Requisitos.  
Fonte: elaborado pelo autor (2025).

**Comunicação** é essencial para transmitir requisitos e compartilhar informações e conhecimentos entre todos os envolvidos. Este valor incentiva o feedback e contribui para criar um senso de equipe e cooperação entre as partes interessadas. Toda comunicação, seja face a face, escrita, em áudio, vídeo ou por meio de representações, deve ser adequada ao seu contexto de aplicação e às pessoas que a utilizarão.

**Feedback** representa o reconhecimento de que a compreensão das necessidades dos usuários e do negócio é um aprendizado constante. As equipes de software devem se esforçar para gerar o máximo de feedback possível sobre os requisitos, o mais rapidamente possível. Quanto mais cedo problemas ou novas necessidades forem identificados, mais cedo será possível adaptar-se. O feedback manifesta-se de várias formas, como opiniões sobre ideias, adequação da organização do backlog, clareza dos requisitos ou adequação do nível de granularidade da declaração dos requisitos. Este valor é parte crítica da comunicação e contribui significativamente para a simplicidade.

**Simplicidade** é considerado o valor mais intensamente intelectual, exigindo esforço para tornar os requisitos simples o suficiente para resolver apenas o problema atual, evitando a “Síndrome de Nostradamus” - tendência de desenvolver soluções que também resolvam problemas futuros ainda não manifestados. A simplicidade só faz sentido dentro de um contexto específico (produto, equipe, cliente) e pode ser alcançada através da comunicação, que ajuda a eliminar requisitos desnecessários ou adiáveis.

**Coragem** é definida como ação efetiva diante do medo. Isoladamente é perigosa, mas em conjunto com os outros valores torna-se poderosa. A coragem de falar verdades, agradáveis ou desagradáveis, fomenta a comunicação e a confiança. Este valor incentiva o descartar

de soluções falhas e a busca por novas alternativas, promovendo a simplicidade. A equipe de engenharia de software deve ter coragem para ser protagonista nas conversas com o cliente, responsabilizar-se pelos requisitos resultantes da interação, buscar conhecimentos sobre o domínio do negócio, adaptar-se às características do cliente e contexto, e construir relações de confiança. A coragem torna-se ainda mais necessária nos momentos de crise, ajudando a lidar com os medos tanto do cliente quanto do engenheiro de software.

**Respeito** reconhece que toda pessoa envolvida no desenvolvimento de software tem o mesmo valor como ser humano, sem hierarquias intrínsecas de importância. Para que o desenvolvimento de software melhore simultaneamente em humanidade e produtividade, as contribuições de cada pessoa da equipe precisam ser respeitadas, partindo do princípio fundamental de que *“eu sou importante e você também”*.

**Compromisso** resulta de uma conversa que visa alcançar compreensão mútua sobre quem fará o quê e quando. É estabelecido a partir de um pedido ou oferta seguido de uma declaração de aceitação. Sem o compromisso dos membros da equipe entre si e com o que estão fazendo, a Engenharia de Requisitos não funcionará adequadamente. Da mesma forma, sem compromisso com o projeto, nada poderá salvá-lo, e sem compromisso com os clientes, nenhuma interação resolverá os problemas.

**Confiança** é considerada a pedra fundamental das relações que promovemos e sustentamos na ER. Embora por vezes seja vista como algo que simplesmente acontece de forma quase mágica, é necessário aprender a vê-la, falar dela, fundamentá-la, construí-la e reconstruí-la. Não aprender a partir da confiança pode fazer com que as relações se quebrem, gerando ressentimento, descrença e insatisfação. Em relações maduras, a confiança deve se tornar algo mais “palpável” e menos mágico, permitindo identificar objetivamente os motivos de confiar ou não em uma pessoa. A confiança pode ser analisada sob quatro aspectos: domínio, sinceridade, responsabilidade e competência.

## 5.4.2 Os 11 Princípios da ER

Os princípios da Engenharia de Requisitos (Figura 38) complementam seus valores, oferecendo diretrizes práticas para orientar as atividades e decisões dos profissionais. Estes onze princípios fundamentais estabelecem as bases para uma prática eficaz e adaptável da ER.



Figura 38 – Princípios da Engenharia de Requisitos.  
Fonte: elaborado pelo autor (2025).

**Princípio 1: Orientação a valor** reconhece que requisitos são meios para um fim, não um fim em si mesmos. O ato de declarar ou representar os requisitos não constitui um objetivo por si só, para serem úteis, os requisitos precisam agregar valor. Os benefícios dos requisitos referem-se a quanto eles contribuem para a redução de riscos e retrabalho durante o desenvolvimento da solução, e para a construção de soluções bem-sucedidas que satisfaçam as necessidades dos stakeholders. Estes benefícios estão diretamente relacionados à qualidade dos requisitos – quanto maior a qualidade, menores os riscos e o retrabalho associados, e maiores as chances de construir soluções de sucesso. O valor da ER pode ser considerado cumulativo ao longo do tempo e, principalmente, direto, uma vez que estabelece o que a solução deve fazer, devendo estar presente na solução em si.

**Princípio 2: Comunicação dos requisitos** enfatiza que a ER não é sobre documentação, mas sobre comunicação. Os requisitos precisam ser comunicados adequadamente para conhecimento de todos os envolvidos, incluindo clientes, usuários e equipe técnica. Para realizar uma comunicação adequada, a equipe de software precisa estar atenta às características dos clientes, do produto, da própria equipe e do contexto de negócio. A comunicação pode variar quanto ao meio (face a face, áudio, vídeo, escrita, desenho), quanto à temporalidade (diária, a cada três dias, semanal, etc.) e quanto à forma (formal e informal).

**Princípio 3: Satisfazer as partes interessadas** define que o principal objetivo da ER é entender os desejos e necessidades das partes interessadas e minimizar o risco de entregar uma solução que não os atenda. As partes interessadas e seus papéis devem ser considerados para atingir esse objetivo, reconhecendo que cada stakeholder tem um papel específico no contexto da solução (usuário, cliente, operador, regulador, etc.) e, portanto, diferentes pontos de vista sobre os requisitos. É importante classificar



as partes interessadas quanto ao grau de sua influência (crítica, maior, menor) para o sucesso da solução, o que ajuda na avaliação da criticidade dos requisitos e na gestão de conflitos negociais. Para o sucesso da ER, é vital envolver as pessoas certas com papéis relevantes e identificar e resolver inconsistências e conflitos entre os requisitos de diferentes partes interessadas.

**Princípio 4: Entendimento compartilhado** reconhece que um entendimento comum é a base para a construção de soluções. Um processo de ER geralmente envolve múltiplas pessoas e fontes de informação, sendo responsabilidade da ER buscar estabelecer um entendimento compartilhado entre essas pessoas quanto ao problema/oportunidade, necessidades e requisitos. Este entendimento pode ser explícito (frequentemente utilizado em processos orientados a planos, com requisitos detalhadamente declarados e escritos) ou implícito (comum em processos ágeis, com requisitos amplamente declarados em conversas face a face). Para estabelecer um entendimento compartilhado, podem-se usar práticas como construção de glossários e protótipos, utilização de sistemas existentes como referência, fornecimento de exemplos de resultados esperados e realização de ciclos curtos de feedback.

**Princípio 5: Contexto da solução** estabelece que os requisitos não surgem por acaso, sem contexto. As soluções de software estão sempre inseridas em um contexto maior, e sem entender esse contexto, é quase impossível estabelecer corretamente os requisitos de uma solução. O contexto é definido como a parte do ambiente de um sistema (domínio da aplicação) relevante para a compreensão da solução e seus requisitos. Compreender o contexto da solução é importante para identificar interfaces relevantes com sistemas existentes, regulamentos e partes interessadas, além de ajudar a fixar tanto o limite do contexto quanto o limite da solução. A ER vai além de considerar requisitos no limite da solução, lidando também com aspectos que vêm do contexto do sistema.

**Princípio 6: Problema, requisitos e solução** enfatiza que estes três elementos devem estar sempre entrelaçados. Qualquer situação em que as pessoas estejam insatisfeitas com processos e resultados pode ser considerada um problema. Para eliminar o problema, um sistema sociotécnico pode entrar em ação, necessitando de requisitos para tornar-se uma solução eficaz. O projeto de um produto inovador pode ser conduzido por uma solução que acaba resolvendo um problema conhecido. A compreensão do problema e da solução leva à elaboração de requisitos que satisfaçam as necessidades do usuário e sejam implementados em uma solução concreta.

**Princípio 7: Validação de requisitos** afirma que requisitos não validados são inúteis. A validação é uma atividade central da ER e descreve o processo de confirmação de que os requisitos declarados são corretos e correspondem às necessidades das partes interessadas. Para a validação, é crucial que os conflitos entre as partes interessadas tenham sido resolvidos, as prioridades definidas (consenso), e que as necessidades das partes interessadas estejam cobertas pelos requisitos.

**Princípio 8: Mudança nos requisitos** reconhece que a mudança nos requisitos deve ser simples e natural. Necessidades, negócios e capacidades estão sempre expostos à evolução e, conseqüentemente, os requisitos para soluções que os atendam também mudarão. Se os requisitos não acompanharem a evolução do sistema, eventualmente perderão valor e tornar-se-ão inúteis. As razões para mudanças são vastas e podem incluir processos de negócios alterados, detecção de erros, mudanças tecnológicas ou novos produtos concorrentes no mercado. Os engenheiros de software devem permitir que os requisitos mudem, pois ignorar sua necessidade de mudança seria um erro.

**Princípio 9: Inovação da ER** estabelece que mais do mesmo não é suficiente. Embora a principal preocupação da ER seja satisfazer as necessidades das partes interessadas, é crucial não se limitar a registrar passivamente o que os stakeholders dizem, pois isso significaria perder a oportunidade de fazer coisas novas e melhores, além de abdicar da responsabilidade pela solução. Os engenheiros de software devem equilibrar-se entre ser inovadores sem presumir que sabem mais que as partes interessadas. Em pequena escala, a ER molda sistemas inovadores buscando novos recursos e facilidade de uso, mas também deve considerar o quadro geral, explorando com as partes interessadas possíveis abordagens disruptivas que possam levar à inovação em larga escala.

**Princípio 10: Processo de ER** afirma que o processo deve ser inovador e adequado. A ER é considerada a mais humana das disciplinas da Engenharia de Software, envolvendo fortemente competências técnicas e humanas. Seu processo não deve ser visto apenas tecnicamente, mas também como uma abordagem para lidar com pessoas de diferentes características e perspectivas. Tecnicamente, o processo deve ser sistemático e disciplinado; humanamente, a abordagem com as pessoas deve ser flexível e adaptável. Assim como na Engenharia de Software, não há um “modelo único” para um processo de ER aplicável a todos os contextos. Os engenheiros de software precisam conceituar processos adequados para cada problema específico, selecionando práticas e produtos de trabalho que melhor se adequem ao problema, contexto e ambiente em questão.

**Princípio 11: Impacto da ER** reconhece que a ER influencia as pessoas e o ambiente. O processo de ER auxilia a promover, entre os envolvidos, uma reflexão e amadurecimento sobre o problema, necessidades e possíveis impactos que a solução de software poderá gerar em seu ambiente. A ER pode mudar a maneira como os envolvidos pensam sobre seu problema, tanto no presente quanto no futuro. Além disso, ao estabelecer o que a solução de software fará, suas restrições e potencialidades, a ER define elementos que irão impactar significativamente no ambiente social no qual o software estará inserido.

## 5.5 A dimensão humana, social e sociotécnica da Engenharia de Requisitos

Os valores e princípios discutidos na seção anterior não constituem apenas um conjunto de orientações para a condução da Engenharia de Requisitos. Eles revelam, de modo mais profundo,

a natureza da própria disciplina. Requisitos não surgem em um vazio técnico, nem podem ser adequadamente compreendidos como simples descrições de funcionalidades, restrições e critérios de qualidade. Eles emergem de contextos concretos de ação humana, de uso e de vida social, nos quais pessoas, tecnologias, linguagens, normas, expectativas, valores, práticas e limitações materiais se influenciam mutuamente. Em consequência, a Engenharia de Requisitos deve ser compreendida como uma prática inseparavelmente técnica, humana, social e sociotécnica, orientada por comunicação, interpretação, negociação e construção progressiva de sentido.

Essa compreensão é especialmente importante porque o universo do software é muito mais amplo do que o ambiente organizacional, ainda que este seja um de seus campos mais recorrentes. Sistemas e produtos de software atravessam hoje domínios tão distintos quanto educação, saúde, entretenimento, mobilidade, governo digital, redes sociais, aviação, comércio, segurança, comunicação e inteligência artificial. Há softwares que apoiam o cuidado clínico, softwares que mediam a aprendizagem, softwares que organizam interações em comunidades digitais, softwares que sustentam jogos e experiências imersivas, softwares embarcados em contextos críticos e softwares que influenciam decisões por meio de modelos algorítmicos. Em todos esses casos, a Engenharia de Requisitos participa da definição de como pessoas irão agir, decidir, perceber, interagir e atribuir valor às soluções construídas.

Como vêm mostrando Nuseibeh e Easterbrook (2000), Easterbrook (2004), Van Lamsweerde (2009), Pohl (2010), Berry (2019) e Laplante e Kassab (2022), a evolução da área ampliou progressivamente a compreensão da ER para além de uma atividade exclusivamente técnica. Reconhecer essa ampliação não reduz o rigor da disciplina. Ao contrário, torna mais precisa sua compreensão, pois evidencia que a qualidade dos requisitos depende não apenas de sua correção formal, mas também da qualidade do entendimento construído entre os atores, da adequação ao contexto de uso e da consciência acerca dos impactos produzidos pela solução.

### 5.5.1 ER como prática sociotécnica

A Engenharia de Requisitos não pode ser reduzida à definição técnica do que um sistema deve fazer. Embora a explicitação de funcionalidades, restrições, interfaces e qualidades seja parte essencial do trabalho, esses elementos só adquirem sentido quando situados em um contexto concreto de uso. Um requisito não é apenas uma descrição abstrata do comportamento esperado de um software; ele é também uma interpretação sobre um problema, uma escolha sobre o que deve ser valorizado e uma aposta sobre como determinada solução poderá responder a necessidades humanas em um ambiente real. Nesse sentido, a ER é sociotécnica porque opera precisamente no encontro entre artefatos tecnológicos e práticas humanas situadas.

Essa condição sociotécnica se manifesta em múltiplos domínios. Em um sistema educacional, requisitos não decorrem apenas de funções de cadastro, avaliação ou acompanhamento; eles se relacionam a concepções pedagógicas, ritmos de aprendizagem, inclusão, atenção, autonomia do estudante e formas de acompanhamento docente. Em sistemas de saúde, requisitos se articulam com segurança do paciente, continuidade do cuidado, carga cognitiva dos profissionais, tempo clínico, comunicação entre equipes e qualidade do registro assistencial. Em jogos digitais, os requisitos se conectam a experiência, desafio, recompensa, cooperação, competição, acessibilidade e permanência do jogador. Em sistemas embarcados na aviação, relacionam-se a segurança

operacional, temporalidade das decisões, coordenação entre atores, percepção situacional e distribuição de responsabilidades. Em plataformas sociais ou sistemas baseados em IA, entram em cena questões como visibilidade, recomendação, moderação, privacidade, explicabilidade, confiança e impacto sobre a experiência dos usuários. Em todos esses cenários, o requisito emerge da interação entre técnica e vida social, e não apenas da lógica interna do sistema.

Essa perspectiva também ajuda a compreender por que problemas de requisitos raramente decorrem apenas de falhas de documentação. Em muitos casos, as dificuldades surgem porque os problemas enfrentados são ambíguos, dinâmicos, interpretativos e dependentes de contexto. Diferentes atores observam a mesma situação a partir de posições distintas, com vocabulários próprios, expectativas diferentes e critérios diversos de sucesso. Aquilo que para um professor representa flexibilidade pedagógica pode, para um gestor educacional, parecer falta de padronização. O que para um médico é essencial para a segurança clínica pode, para a equipe de desenvolvimento, parecer um custo de implementação elevado. O que para o jogador é parte da experiência pode, para o projetista, ser apenas uma regra de progressão. A ER lida justamente com essa pluralidade de leituras, buscando transformar diversidade de percepções em entendimento suficientemente consistente para orientar decisões de projeto.

Compreender a ER como prática sociotécnica implica, portanto, reconhecer que requisitos emergem da interação entre tecnologia, linguagem, trabalho, aprendizagem, cuidado, lazer, cultura, normas, interesses, valores e contexto. Isso vale tanto para softwares utilizados em organizações quanto para aqueles que permeiam a vida cotidiana, a esfera pública e as relações sociais em sentido mais amplo. Decisões de requisitos nunca permanecem confinadas ao artefato técnico. Elas repercutem sobre experiências, rotinas, relações, oportunidades de ação e formas de participação. É por isso que a sociotécnica não deve ser tratada como um adjetivo periférico da ER, mas como uma chave de compreensão de sua própria natureza.

### **5.5.2 Comunicação, mediação e construção de entendimento**

Se a ER é uma prática sociotécnica, então a comunicação é o seu princípio estruturante. Requisitos não são objetos prontos, ocultos na mente dos stakeholders, à espera de serem simplesmente extraídos e registrados. Eles são formulados, reinterpretados, refinados, negociados e validados em interações nas quais diferentes atores expressam necessidades, receios, expectativas, restrições, hipóteses e critérios de valor. Nesse sentido, requisitos não são apenas “coletados”; eles são socialmente construídos ao longo de um processo comunicativo e interpretativo, como destacam Easterbrook (2004), Pohl (2010) e Robertson e Robertson (2014).

Essa construção comunicativa exige mais do que troca de informações. Exige escuta qualificada, formulação de perguntas relevantes, capacidade de tradução entre linguagens distintas e sensibilidade para perceber ambiguidades, silêncios, pressupostos e assimetrias de entendimento. O estudante descreve suas dificuldades de uso de um ambiente virtual de aprendizagem de forma diferente de um professor ou de um gestor acadêmico. Um paciente percebe um sistema de agendamento de forma distinta de um profissional de saúde ou de um administrador hospitalar. Um controlador de tráfego aéreo, um piloto e um engenheiro de software não descrevem uma mesma necessidade com o mesmo repertório conceitual. Um jogador, um designer de jogo e um analista

de dados também não. O trabalho da ER consiste, em larga medida, em criar pontes entre esses mundos de sentido sem reduzir a complexidade de nenhum deles.

É nesse ponto que a mediação se torna central. O profissional de requisitos atua como mediador entre pessoas, perspectivas, interesses, linguagens e níveis de abstração distintos. Ele não é apenas um registrador de falas, mas um articulador de entendimento. Sua tarefa consiste em ajudar os envolvidos a tornarem explícitas suas expectativas, confrontarem interpretações divergentes, identificarem conflitos e construir um núcleo compartilhado de significado que permita avançar com legitimidade. Como observam Dick, Hull e Jackson (2017), bem como Young (2004), a qualidade do trabalho com requisitos depende fortemente da capacidade de representar problemas e soluções de modo compreensível para públicos diferentes, sem perder precisão e rastreabilidade.

Por isso, a comunicação em ER não se limita à linguagem textual ou verbal. Ela pode ser apoiada por narrativas, cenários, exemplos, protótipos, esboços, storyboards, mapas de stakeholders, diagramas de contexto, demonstrações, vídeos, áudios e outras formas de representação capazes de tornar o entendimento mais tangível e discutível. O ponto central não está no formato em si, mas em sua função comunicativa. Um artefato de requisitos é valioso quando favorece compreensão compartilhada, e não apenas quando parece tecnicamente correto. Em alguns casos, uma representação simples e visual comunica melhor do que uma descrição detalhada; em outros, uma especificação mais estruturada será necessária. O critério decisivo é sempre o quanto aquela forma de declaração sustenta entendimento entre os envolvidos.

Essa visão também ajuda a evitar um equívoco frequente: o de tratar divergências de interpretação como meras falhas que precisam ser eliminadas o mais rápido possível. Em muitos casos, alguma divergência é inevitável, pois cada stakeholder observa o problema a partir de sua experiência e de seu horizonte de ação. O desafio da ER não é apagar a diferença, mas trabalhá-la produtivamente. Entendimento compartilhado não significa uniformidade absoluta de pensamento, mas a construção gradual de um sentido comum suficiente para orientar decisões, validações e compromissos. É precisamente por isso que a comunicação não é apenas um recurso auxiliar da ER. Ela é uma de suas condições de possibilidade.

### **5.5.3 Stakeholders, contextos de uso, poder e negociação**

Uma consequência direta dessa perspectiva é reconhecer que stakeholders não formam um grupo homogêneo. Eles possuem interesses, experiências, repertórios, expectativas, responsabilidades e graus de influência distintos. Também não observam o problema nem a solução sob o mesmo ponto de vista. Usuários finais, profissionais especializados, gestores, mantenedores, reguladores, cidadãos, pacientes, professores, estudantes, operadores, desenvolvedores, cientistas de dados, designers, moderadores de plataforma, equipes de suporte e grupos afetados indiretamente podem todos ter algo relevante a dizer sobre o sistema, ainda que por razões diferentes. A Engenharia de Requisitos se realiza, assim, em um espaço de pluralidade.

Essa pluralidade não se restringe ao ambiente organizacional. Em muitos casos, o contexto relevante é institucional, comunitário, doméstico, regulatório, cultural ou público. Um software educacional se insere em práticas de ensino e aprendizagem que não se explicam apenas por

estruturas administrativas. Uma rede social opera em ecossistemas de interação, reputação, exposição e moderação que excedem os limites de qualquer organização específica. Um jogo digital mobiliza comunidades, culturas de participação, economias de atenção e experiências subjetivas. Um sistema de apoio à decisão em saúde precisa considerar não apenas a instituição hospitalar, mas também o cuidado, a vulnerabilidade do paciente, normas clínicas e relações entre diferentes serviços. Em todos esses casos, a ER precisa olhar para os contextos de uso e de vida social nos quais o software fará sentido ou produzirá fricções.

É justamente nesses contextos que relações de poder se manifestam. Poder, aqui, não deve ser entendido apenas em sentido hierárquico formal. Ele pode decorrer de autoridade institucional, expertise técnica, domínio de linguagem, acesso privilegiado à informação, controle sobre recursos, capacidade de definir prioridades, influência econômica, legitimidade regulatória ou poder de moldar a visibilidade e a participação em plataformas digitais. Em um projeto, certas vozes podem ser mais facilmente escutadas do que outras. Alguns grupos conseguem transformar suas preferências em requisito; outros têm suas necessidades tratadas como periféricas. Quando essas assimetrias permanecem invisíveis, o risco é produzir requisitos aparentemente adequados do ponto de vista técnico, mas frágeis em termos de legitimidade, adoção e impacto.

Por essa razão, a negociação ocupa papel central na ER. Negociar não significa apenas resolver incompatibilidades entre requisitos, mas construir compromissos viáveis entre valores que nem sempre podem ser atendidos simultaneamente. Em um sistema educacional, pode haver tensão entre flexibilidade pedagógica e padronização institucional. Em saúde, entre segurança, rapidez e usabilidade. Em redes sociais, entre liberdade de interação, moderação, privacidade e bem-estar. Em jogos, entre desafio, acessibilidade, equilíbrio competitivo e engajamento. Em sistemas críticos, entre automação, controle humano, rastreabilidade e eficiência operacional. Toda decisão de requisitos envolve algum tipo de trade-off, isto é, uma escolha que privilegia certos objetivos em detrimento de outros.

Diante disso, o consenso não deve ser compreendido como mera unanimidade aparente, nem como eliminação das diferenças. Em muitos casos, ele é melhor entendido como um acordo social legítimo, suficientemente sólido para orientar o avanço do projeto, ainda que preserve tensões e revisões futuras. Como ressaltam Pohl (2010) e Robertson e Robertson (2014), a qualidade desse consenso depende da explicitação dos critérios de decisão, dos conflitos envolvidos e das consequências das escolhas realizadas. A boa prática de ER não é a que evita o conflito a qualquer custo, mas a que cria condições para que divergências possam ser tratadas de forma produtiva, transparente e respeitosa.

#### **5.5.4 Software como intervenção social: impactos pretendidos e efeitos emergentes**

Todo software é uma intervenção social. Ao ser introduzido em uma escola, hospital, aeroporto, serviço público, comunidade digital, ambiente doméstico, empresa, plataforma de comunicação ou qualquer outro contexto de uso, ele tende a alterar práticas, comportamentos, relações, fluxos de informação, formas de coordenação, critérios de decisão, possibilidades de ação e distribuição de autonomia. Mesmo quando concebido com objetivos aparentemente específicos e delimitados, o software reconfigura o ambiente em que passa a operar. Ele facilita certos modos de agir, dificulta outros, torna determinadas informações mais visíveis, invisibiliza outras, amplia

algumas possibilidades e restringe outras. Em consequência, falar de requisitos é também falar de transformações na vida social mediada por tecnologia.

Essa afirmação é decisiva porque ajuda a superar uma visão limitada segundo a qual o software apenas automatiza atividades previamente definidas. Na prática, ele não apenas automatiza; ele reorganiza. Uma plataforma educacional não apenas registra notas ou disponibiliza conteúdo: ela pode redefinir ritmos de estudo, modos de acompanhamento docente, critérios de participação e formas de avaliação. Um sistema clínico não apenas armazena informações: ele pode alterar a sequência do atendimento, a relação entre registro e cuidado, a distribuição de atenção entre paciente e interface e a coordenação entre equipes. Um jogo digital não apenas executa regras computacionais: ele molda padrões de cooperação, competição, recompensa, frustração, engajamento e sociabilidade. Uma rede social não apenas hospeda interações: ela estrutura visibilidade, circulação de conteúdos, mecanismos de recomendação, regimes de atenção e modos de participação pública. Um sistema na aviação não apenas exibe dados: ele intervém sobre percepção situacional, carga cognitiva, coordenação entre agentes e responsabilização em ambientes críticos. Um sistema baseado em IA não apenas processa dados: ele influencia classificações, recomendações, priorizações e decisões que podem afetar oportunidades concretas para pessoas e grupos.

Por isso, requisitos não são neutros. Eles incorporam escolhas sobre o que será visível, exigido, permitido, recomendado, bloqueado, monitorado, flexibilizado ou automatizado. Um requisito relativo à forma de apresentar feedback ao estudante pode influenciar motivação, ansiedade e autonomia. Um requisito de alerta em contexto clínico pode melhorar segurança, mas também gerar saturação cognitiva. Um requisito de progressão em jogo pode favorecer senso de conquista, mas também induzir padrões específicos de permanência. Um requisito de recomendação em uma plataforma social pode ampliar relevância percebida, mas também reorganizar exposição, atenção e interação. Um requisito de explicabilidade em sistemas de IA pode ampliar confiança e auditabilidade, enquanto sua ausência pode tornar opaco o modo como decisões são apoiadas. Em todos esses casos, a definição do requisito participa da modelagem das condições concretas de experiência e ação.

É por isso que a Engenharia de Requisitos precisa formular uma pergunta adicional, que não substitui as perguntas tradicionais, mas as amplia: não apenas “*o que o sistema deve fazer?*”, mas também “*que tipo de intervenção social, humana, institucional ou cultural este sistema tende a produzir?*”. Essa questão é central porque nem todos os impactos são pretendidos. Alguns efeitos são deliberadamente buscados, como melhorar acesso a serviços, apoiar a aprendizagem, ampliar segurança, facilitar coordenação, reduzir erros, promover inclusão ou aumentar transparência. Outros, porém, emergem do encontro entre o sistema e a prática real de uso. Podem surgir efeitos não antecipados sobre autonomia, visibilidade, controle, colaboração, exclusão, dependência, confiança, bem-estar, privacidade, responsabilização, experiência subjetiva ou qualidade da interação humana. Em certos casos, uma solução pensada para eficiência pode gerar sobrecarga; em outros, uma funcionalidade desenhada para personalização pode produzir opacidade; em outros ainda, uma forma de automação pode reduzir a capacidade de julgamento situado dos usuários.

Distinguir impactos pretendidos de efeitos emergentes não significa supor que a ER possa prever integralmente o futuro. Tampouco significa transformar a disciplina em sociologia aplicada. Significa, antes, reconhecer que ela precisa incorporar consciência sociotécnica, responsabilidade reflexiva e sensibilidade para impactos. O papel do engenheiro de software não é controlar completamente as consequências do sistema, mas explicitar hipóteses, considerar grupos afetados, tornar visíveis trade-offs, antecipar riscos plausíveis e favorecer decisões mais conscientes. Como vêm enfatizando Berry (2019) e Laplante e Kassab (2022), questões éticas, sociais, de sustentabilidade, privacidade e impacto ampliam hoje o escopo de atenção da Engenharia de Requisitos. Esse alargamento não descaracteriza a disciplina; ao contrário, responde à complexidade efetiva dos sistemas contemporâneos.

Reconhecer o software como intervenção social reposiciona, assim, a própria responsabilidade da ER. Se os requisitos ajudam a moldar como pessoas aprendem, cuidam, se comunicam, se deslocam, jogam, trabalham, decidem, colaboram e convivem, então a disciplina não pode se limitar à captura de funcionalidades. Ela precisa refletir sobre que mudança está ajudando a conceber, para quem essa mudança cria valor, quem pode ser negativamente afetado e que efeitos merecem ser acompanhados ao longo do ciclo de vida do sistema. Nesse sentido, a ER não é apenas uma etapa do desenvolvimento: ela é também um momento de escolha sobre os modos de vida, interação e ação que o software tenderá a reforçar ou transformar.

### 5.5.5 Implicações para a prática da Engenharia de Requisitos

Compreender a ER como prática humana, social e sociotécnica produz implicações diretas para sua realização profissional. Na elicitación e descoberta, por exemplo, não basta reunir preferências declaradas ou registrar listas de funcionalidades desejadas. É necessário compreender o contexto de uso, a prática concreta, os objetivos efetivos dos atores, as limitações do ambiente, as necessidades latentes e os grupos que poderão ser afetados, inclusive aqueles que nem sempre participam diretamente das discussões iniciais. Em um sistema educacional, isso pode significar olhar para a experiência de aprendizagem, e não apenas para o controle acadêmico. Em saúde, para o cuidado real e não apenas para o registro formal. Em jogos, para a experiência do jogador e da comunidade. Em plataformas sociais, para as dinâmicas de interação e moderação. Em sistemas críticos, para as condições de operação sob pressão, tempo e risco.

Na análise e consenso, essa mesma perspectiva exige que conflitos, ambiguidades e lacunas sejam tratados como parte constitutiva do processo, e não como anomalias periféricas. Priorizar requisitos significa tomar decisões sobre valor, risco, impacto, viabilidade e consequências. Por isso, a análise não deve se limitar ao refinamento lógico ou textual dos requisitos, mas também explicitar quem é afetado por determinada escolha, quais pressupostos estão sendo assumidos e que valores estão sendo privilegiados. Dependendo do domínio, esses valores podem incluir aprendizagem, segurança, acessibilidade, bem-estar, inclusão, confiança, privacidade, justiça, engajamento, autonomia ou continuidade operacional. A boa priorização é aquela que reconhece a pluralidade desses critérios sem perder a coerência do produto. Já o consenso não é o mais rápido, nem o que apenas encerra a reunião; é aquele que produz compromisso coletivo suficientemente claro, justificável e revisável diante do contexto sociotécnico em que o sistema será implantado.



Na declaração e na representação, as implicações também são significativas. Se a função principal dos artefatos de requisitos é comunicar e alinhar entendimento, então a escolha de formatos, níveis de abstração e meios expressivos deve ser guiada por adequação comunicacional, não por apego a modelos padronizados como se fossem fins em si mesmos. Isso reforça a ideia de que declarar é mediar linguagens e tornar explícito um entendimento partilhado. Uma boa declaração é aquela que permite que diferentes atores compreendam o requisito, discutam-no, validem-no e o utilizem como referência para decisão e desenvolvimento. Quando necessário, isso pode envolver combinar texto, imagem, narrativa, protótipo, áudio ou vídeo, desde que se preserve clareza, rastreabilidade e coerência.

Na verificação e validação, essa ampliação exige compreender que o acordo obtido não é apenas técnico, mas também social. Um requisito pode estar sintaticamente correto e, ainda assim, ser inadequado para a experiência real dos usuários ou produzir efeitos indesejáveis no contexto de uso. Portanto, não é somente verificar se o requisito foi bem escrito; é examinar se ele faz sentido para os envolvidos, se responde ao problema real, se preserva valores relevantes do domínio e se suas consequências foram minimamente consideradas.

Na organização e atualização dos requisitos, se a Engenharia de Requisitos lida com contextos dinâmicos, com entendimento progressivo e com negociações sucessivas entre múltiplos stakeholders, então o conjunto de requisitos não pode ser tratado como um repositório estático de decisões já encerradas. Ele precisa permanecer adequadamente estruturado, inteligível e atual ao longo do ciclo de vida do produto. Isso envolve organizar os requisitos de modo coerente com o produto e com o projeto, estabelecer relações entre eles, favorecer rastreabilidade, apoiar refinamentos e prioridades sucessivas e garantir que mudanças, aprendizados, consensos revisados e evidências produzidas nas demais atividades sejam efetivamente refletidos nesse conjunto. Organizar e atualizar requisitos, portanto, não é apenas uma tarefa administrativa: é uma atividade de sustentação do entendimento compartilhado ao longo do tempo.

Essa atividade possui, inclusive, uma dimensão claramente humana e social. Mudanças nos requisitos não decorrem apenas de novas funcionalidades desejadas, mas também de transformações no contexto, de novas interpretações do problema, de conflitos resolvidos, de necessidades antes invisíveis, de feedback de uso e de efeitos emergentes produzidos pela própria solução. Manter o conjunto de requisitos atualizado significa, assim, reconhecer que o entendimento sobre o sistema amadurece, se desloca e, por vezes, precisa ser renegociado. Sob essa perspectiva, organização e atualização funcionam como memória estruturada da evolução do entendimento coletivo, preservando coerência sem congelar artificialmente aquilo que precisa continuar evoluindo.

Essa perspectiva também amplia o conjunto de competências esperado do profissional que atua com requisitos. Além de domínio técnico, tornam-se particularmente relevantes capacidades como escuta ativa, comunicação clara, formulação de perguntas, facilitação de grupos, mediação de conflitos, interpretação de contexto, sensibilidade para diversidade de perspectivas, pensamento crítico e reflexão sobre impactos. Isso não relativiza o rigor da disciplina. Ao contrário, revela que o rigor da ER precisa ser exercido sobre problemas que são sempre, em alguma medida, humanos e situados. Como já sugerem Robertson e Robertson (2014), Young (2004), Dick,

Hull e Jackson (2017), Berry (2019) e Laplante e Kassab (2022), uma parte importante dos fracassos em requisitos decorre justamente da subestimação dessas dimensões.

Por fim, compreender a dimensão humana, social e sociotécnica da Engenharia de Requisitos reforça uma ideia central deste livro: a ER é, inseparavelmente, uma atividade técnica e uma prática de mediação entre pessoas, sentidos, contextos e tecnologias. Seu propósito não é apenas produzir requisitos formalmente corretos, mas contribuir para a concepção de soluções que façam sentido no mundo em que serão usadas. Isso vale tanto para softwares organizacionais quanto para sistemas educacionais, de saúde, entretenimento, interação social, mobilidade, governo, aviação ou inteligência artificial, entre outros. Em última análise, fazer Engenharia de Requisitos é participar da construção de futuros possíveis por meio do software. E, se o software transforma modos de aprender, cuidar, jogar, circular, decidir, trabalhar, interagir e conviver, então a ER precisa assumir conscientemente sua responsabilidade na qualidade técnica e na qualidade humana dessas transformações.

## CAPÍTULO 5: PONTOS-CHAVE

- O capítulo organiza a ER em **seis atividades fundamentais: elicitação e descoberta, análise e consenso, declaração, representação, verificação e validação e organização e atualização** — deixando claro que elas **se entrelaçam** e não são estritamente sequenciais.
- **Elicitação e descoberta:** ponto de partida para compreender necessidades explícitas e latentes; envolve múltiplas fontes (stakeholders, documentos, ferramentas/IA, sistemas em operação) e já antecipa ciclos de **análise e consenso, declaração e representação**.
- **Análise e consenso:** transforma requisitos “brutos” em entendimento compartilhado, **removendo ambiguidades, resolvendo contradições e definindo prioridades**, por meio de negociação e facilitação entre perspectivas distintas.
- **Declaração:** **comunica** requisitos em linguagem natural (escrita/oral) com diferentes granularidades, escolhendo nível de formalidade conforme contexto, público e criticidade.
- **Representação:** **complementa a declaração** com formas visuais/estruturadas (modelos, diagramas, expressões, visualizações); adota níveis **informal, semiformal e formal**, escolhidos segundo complexidade, público e rigor requerido.
- **Verificação e validação:** tratam da **qualidade dos requisitos** em dimensões complementares — validação verifica “**o requisito certo**” (alinhamento com partes interessadas), verificação confere “**o requisito feito corretamente**” (consistência, completude, testabilidade).
- **Organização e atualização:** mantém o conjunto **estruturado e atual** ao longo do ciclo de vida; explicita **como estruturar, rastrear, refinar e priorizar**, e como **refletir mudanças e consensos** emergentes.
- O capítulo apresenta ainda **7 valores e 11 princípios** para a ER:
- Sete valores da Engenharia de Requisitos:
  - **Comunicação** — base de todo o processo; a compreensão mútua é mais importante que a documentação isolada.

- **Simplicidade** — buscar o essencial, evitando complexidade desnecessária nas representações e decisões.
- **Feedback** — promover ciclos curtos de retorno e aprendizado contínuo entre stakeholders e equipe técnica.
- **Coragem** — questionar decisões, revisar requisitos e corrigir rumos sem apego a soluções anteriores.
- **Respeito** — reconhecer diferentes perspectivas, linguagens e ritmos de aprendizado entre os envolvidos.
- **Compromisso** — agir com responsabilidade coletiva diante dos objetivos do projeto e dos resultados esperados.
- **Confiança** — sustentar relações transparentes e éticas que permitam colaboração genuína e segurança no diálogo.
- Onze princípios orientadores:
  1. **Orientação a valor** – todo requisito deve contribuir de forma mensurável para o valor do produto e para os objetivos da organização.
  2. **Comunicação dos requisitos** – o processo de requisitos é essencialmente comunicativo; compreender precede documentar.
  3. **Satisfazer as partes interessadas** – a ER deve equilibrar perspectivas e necessidades, conciliando interesses de negócio, usuário e tecnologia.
  4. **Entendimento compartilhado** – o sucesso da ER depende da construção coletiva do significado dos requisitos.
  5. **Contexto da solução** – requisitos devem ser interpretados à luz do ambiente, das restrições e dos propósitos da solução.
  6. **Problema, requisitos e solução** – a ER deve manter clara a distinção e a relação entre esses três domínios.
  7. **Validação de requisitos** – a verificação e validação são contínuas e garantem a qualidade técnica e o alinhamento com as expectativas.
  8. **Mudança nos requisitos** – a ER deve tratar a mudança como parte natural do aprendizado e da evolução do produto.
  9. **Inovação da ER** – a disciplina deve incorporar novos métodos, tecnologias e práticas de forma crítica e contextualizada.
  10. **Processo de ER** – a ER deve ser conduzida como um processo sistemático, iterativo e integrado ao ciclo de vida do software.
  11. **Impacto da ER** – as decisões de requisitos influenciam todo o sistema; compreender seu impacto é essencial para a qualidade global do produto.
- Esses valores e princípios formam o eixo ético e prático da disciplina, sustentando as atividades e moldando a postura dos profissionais envolvidos.
- A **dimensão humana, social e sociotécnica da Engenharia de Requisitos**, reforça que a disciplina não se limita à definição técnica de funcionalidades e restrições, mas se realiza na interseção entre pessoas, contextos, linguagens, interesses, normas e tecnologias.

- A ER é compreendida como uma **prática sociotécnica**, na qual os requisitos emergem de contextos reais de uso e ação, em domínios diversos como educação, saúde, jogos, redes sociais, sistemas críticos, serviços públicos, aviação e inteligência artificial.
- Nessa perspectiva, os requisitos não podem ser entendidos apenas a partir da lógica interna do software, pois resultam da **interação entre necessidades humanas, características do domínio, possibilidades tecnológicas e condições concretas de uso**.
- **A comunicação é tratada como princípio estruturante da ER**, o que significa reconhecer que requisitos não são simplesmente coletados, mas construídos, interpretados, negociados, refinados e compartilhados ao longo das interações entre os envolvidos.
- O capítulo enfatiza também o **papel da mediação**, mostrando que o engenheiro de software atua como ponte entre diferentes perspectivas, vocabulários, níveis de abstração e interesses, contribuindo para a construção de entendimento compartilhado.
- **Stakeholders, contextos de uso, poder e negociação** são apresentados como elementos constitutivos da prática de requisitos, e não como fatores externos ao processo.
- **Conflitos, trade-offs, prioridades e assimetrias de influência** fazem parte do trabalho com requisitos e precisam ser tratados com clareza, legitimidade e responsabilidade.
- Um argumento central é que **todo software é uma intervenção social**, pois, ao ser introduzido em um contexto de uso, tende a alterar práticas, comportamentos, relações, fluxos de informação, formas de decisão, visibilidade, controle e autonomia.
- Por essa razão, **requisitos não são neutros**: eles ajudam a moldar a forma como o software intervirá no mundo e influenciará pessoas, grupos, instituições e práticas sociais.
- A ER, portanto, deve considerar não apenas o que o sistema deve fazer, mas também **que tipo de intervenção social, humana, institucional ou cultural** está ajudando a projetar, distinguindo impactos pretendidos de efeitos emergentes ou não antecipados.
- Essa perspectiva **amplia o alcance de todas as atividades da ER**, exigindo que elicitação e descoberta, análise e consenso, declaração, representação, verificação e validação, organização e atualização considerem contexto, grupos afetados, critérios de valor, evolução do entendimento e efeitos da solução ao longo do tempo.
- O capítulo conclui que a **Engenharia de Requisitos é um processo sociotécnico, colaborativo e reflexivo**, em que o conhecimento é construído e refinado coletivamente, e onde técnica e linguagem se unem para gerar valor, sentido e aprendizado contínuo.

# Capítulo 6

## 6 Introdução aos Requisitos de Software

No contexto da engenharia de software, os requisitos são fundamentais para garantir que os produtos desenvolvidos atendam às necessidades dos stakeholders. Eles funcionam como a base sobre a qual se estruturam todas as fases subsequentes do projeto de software (SOMMERVILLE, 2019).

### 6.1 Conceituação de Requisitos

De acordo com a norma ISO/IEC/IEEE 15288:2023 (IEEE, 2023), um requisito é definido como uma *declaração mensurável* que descreve características, atributos, funcionalidades ou capacidades que um sistema deve possuir para atender às necessidades dos stakeholders. Os requisitos são derivados de uma *visão orientada pelo usuário* e devem ser expressos de maneira técnica, buscando sempre, quando possível, evitar a imposição de uma solução específica. Esses requisitos são fundamentais para guiar o projeto e o desenvolvimento de sistemas, assegurando que o produto final satisfaça as expectativas operacionais estabelecidas (IEEE, 2023). A Figura 39 ilustra esse conceito.



Figura 39 – Representação do conceito de requisito de produto de software.  
Fonte: baseado em IEEE (2023).

Visando deixar ainda mais claro os conceitos relacionados aos requisitos, de acordo com a norma ISO/IEC/IEEE 15288:2023, tem-se:

- **Declaração mensurável**

Uma declaração é considerada mensurável caso seja passível de **verificação, teste e mensuração** de forma objetiva com critérios específicos. Um requisito precisa ser expresso de maneira que permita determinar claramente se foi atendido ou não. Isso geralmente envolve especificações quantitativas (números, intervalos, limites) ou critérios qualitativos bem definidos com condições verificáveis de sucesso ou falha.

**Exemplos:** O sistema deve processar 100 transações por segundo sob carga normal de operação; O tempo de resposta do sistema para consultas de usuário não deve exceder 2 segundos em 95% dos casos; A interface do usuário deve ser considerada 'intuitiva' quando 85% dos novos usuários conseguirem completar as tarefas básicas sem auxílio em sua primeira tentativa.

- **Visão orientada pelo usuário**

Uma abordagem que prioriza a perspectiva de quem utilizará o sistema, concentrando-se em como ele será aplicado no contexto real. Requisitos orientados pelo usuário descrevem **o que o sistema deve fazer em termos de valor e utilidade para seus usuários**, em vez de focar em como será construído. Esta visão considera necessidades, objetivos, limitações e contextos de uso dos diversos stakeholders.

**Exemplos:** Os usuários devem conseguir realizar uma compra completa em no máximo 5 passos; Os médicos devem conseguir acessar o histórico completo do paciente em uma única tela, sem necessidade de navegar entre múltiplas páginas; Os operadores de suporte devem ser capazes de identificar a causa de falhas comuns em menos de 3 minutos.

- **Evitar a imposição de uma solução específica**

Os requisitos devem especificar o que o sistema precisa fazer (resultado desejado), sem prescrever como deve ser feito (implementação). Isso permite que os engenheiros de software tenham liberdade para identificar e implementar as melhores soluções técnicas, incentivando a inovação e permitindo que o design do sistema evolua com avanços tecnológicos. Ao especificar o **“o quê” em vez do “como”**, os requisitos permanecem relevantes mesmo quando a tecnologia muda.

**Exemplos:** O sistema deve verificar a identidade do usuário (em vez de “O sistema deve usar reconhecimento facial para verificar o usuário”); O relatório financeiro deve ser gerado com precisão de duas casas decimais e agrupar despesas por categoria (em vez de “Use o algoritmo X para calcular e a biblioteca Y para formatar”); O sistema deve proteger dados pessoais contra acesso não autorizado (em vez de “Use criptografia AES-256 com chaves de 2048 bits”).

- **Sobre satisfazer as expectativas operacionais estabelecidas**

O sistema deve atender às expectativas definidas para seu desempenho e comportamento no ambiente de operação real. Isso significa que, quando implementado e colocado em uso, o sistema funciona conforme antecipado pelos stakeholders em suas condições de uso previstas. As expectativas operacionais abrangem não apenas as funcionalidades básicas, mas também aspectos como desempenho, confiabilidade, segurança e experiência

do usuário conforme definido pelos stakeholders durante a fase de levantamento de requisitos.

**Exemplos:** O sistema de diagnóstico médico deve identificar corretamente pelo menos 95% dos casos de diabetes tipo 2 quando comparado com diagnósticos realizados por especialistas; O sistema de atendimento ao cliente deve resolver 80% das solicitações no primeiro contato, sem necessidade de encaminhamento; O software de controle de tráfego aéreo deve manter rastreamento preciso de todas as aeronaves em um raio de 50 km mesmo durante condições climáticas adversas.

## 6.2 Tipos de Requisitos: Funcionais e Não Funcionais

### 6.2.1 Requisitos Funcionais (RFs)

Requisitos funcionais são declarações que especificam o **comportamento observável** que um sistema deve apresentar. Eles descrevem os serviços que o software deve fornecer, assim como as respostas a entradas específicas e os comportamentos esperados em diferentes condições operacionais. Segundo o SWEBOK v4.0 (2024), esses requisitos representam as capacidades funcionais do sistema, **expressas em termos de interações com os usuários**, com outros sistemas e com o ambiente externo, sem necessariamente explicitar os mecanismos internos de implementação.

Sommerville (2019) complementa essa visão ao afirmar que requisitos funcionais são **declarações do que o sistema deve fazer**, indicando como ele deve reagir a certas entradas e como deve se comportar em determinadas situações. Além disso, ele observa que, em alguns casos, esses requisitos podem explicitar ações que o sistema não deve realizar, estabelecendo limites funcionais.

De modo geral, os requisitos funcionais se concentram em **o que o sistema deve realizar para atender às necessidades dos stakeholders**, sendo fundamentais para a definição e validação das funcionalidades do software. Eles devem ser compreensíveis e verificáveis, podendo ser expressos em diferentes níveis de detalhe (abstração) — desde declarações amplas até especificações refinadas, conforme o público-alvo (usuários, gerentes ou desenvolvedores) e o estágio do processo de engenharia de requisitos.

Neste livro, ao abordarmos o nome dos requisitos funcionais será utilizado como padrão de escrita, verbo no infinitivo acrescido do objeto. A partir disso, tem-se os seguintes exemplos de nomes de requisitos funcionais:

- Cadastrar cliente;
- Gerar relatório contábil;
- Realizar compra de produto;
- Consultar notas fiscais;
- Incluir produto no pedido;
- Emitir relatório de movimentação de estoque;
- Emitir boleto bancário;
- Realizar login do usuário;

- Reservar item para pedido.

## 6.2.2 Requisitos Não Funcionais (RNFs)

Os requisitos não funcionais descrevem **propriedades globais, restrições e qualidades** que um sistema deve atender, não estando diretamente relacionados aos serviços funcionais prestados aos usuários. De acordo com Sommerville (2019), no geral, esses requisitos especificam características do sistema como um todo, como confiabilidade, tempo de resposta, uso de memória, segurança e interoperabilidade. Podem ainda impor restrições à implementação, como limitações de dispositivos, padrões de dados ou integração com outros sistemas.

Esses requisitos **tendem a ser mais críticos que os requisitos funcionais individuais**, pois sua violação pode comprometer completamente a viabilidade do sistema. Enquanto funcionalidades insatisfatórias podem ser contornadas pelos usuários, o não cumprimento de um requisito não funcional — como um requisito de confiabilidade em sistemas aeronáuticos — pode impedir sua certificação e uso (SOMMERVILLE, 2019).

São exemplos de requisitos não funcionais:

- Garantir tempo de resposta inferior a 2 segundos por requisição;
- Suportar até 10.000 acessos simultâneos;
- Garantir disponibilidade mínima de 99,9% durante horário comercial;
- Registrar falhas de execução em arquivo de log;
- Criptografar dados armazenados em banco de dados;
- Permitir navegação com teclado em todas as funcionalidades;
- Garantir compatibilidade com versões do Android a partir da 8.0 (Oreo);
- Garantir compatibilidade com versões do iOS a partir da 13.0;
- Documentar código-fonte com padrão interno da organização.

### 6.2.2.1 Classificação dos Requisitos Não Funcionais

Os requisitos não funcionais possuem diversas classificações, sendo importante destacar que não existe uma classificação única e universalmente aceita para requisitos não funcionais (RNFs). As diferentes propostas de categorização existentes na literatura (CHUNG et al., 2000; ISO/IEC 25010:2011, 2011; BASS, CLEMENTS e KAZMAN, 2012; PRESSMAN; MAXIM, 2021; WIEGERS, 2013; SOMMERVILLE, 2019; entre outros) refletem perspectivas complementares, formuladas a partir de contextos específicos de aplicação, objetivos de modelagem ou visões metodológicas distintas.

Neste livro apresentaremos duas das classificações mais referenciadas na literatura propostas por Grady e Caswell (1987) e por Sommerville (2019) e, que embora convergentes em vários aspectos, refletem abordagens e contextos distintos.

O modelo **URPS+**, criado na Hewlett-Packard, categoriza os RNFs com base em atributos de qualidade do produto (GRADY e CASWELL, 1987). Cada letra representa um grupo:



- **Usabilidade (Usability)** – Podem incluir subcategorias como: fatores humanos, estética, consistência na interface com o usuário, ajuda on-line e sensível ao contexto, assistentes e agentes, documentação do usuário e materiais de treinamento.

Exemplos: Interface responsiva compatível com acessibilidade WCAG 2.1; A interface deve ser compatível com leitores de tela para atender a usuários com deficiência visual; O sistema deve disponibilizar tutorial interativo para novos usuários no primeiro acesso.

- **Confiabilidade (Reliability)** – São considerados: frequência e gravidade de falha, possibilidade de recuperação, possibilidade de previsão, precisão e tempo médio entre falhas.

Exemplos: Recuperar automaticamente após falha de rede em até 60 segundos; O sistema deve apresentar disponibilidade mínima de 99,9% durante o horário comercial; Em caso de falha de energia, o sistema deve restaurar o último estado salvo automaticamente ao reiniciar.

- **Desempenho (Performance)** – Impõe condições aos requisitos funcionais, como, por exemplo, parâmetros de desempenho: velocidade, eficiência, disponibilidade, precisão, produtividade, tempo de resposta, tempo de recuperação e uso de recurso.

Exemplos: Tempo médio de carregamento da página inferior a 800 ms; O sistema deve processar até 100 requisições por segundo com uso de CPU abaixo de 70%; O tempo de resposta para geração de relatórios não deve exceder 5 segundos para até 10 mil registros.

- **Suportabilidade (Supportability)** – Podem incluir: possibilidade de teste, extensibilidade, adaptação, manutenção, compatibilidade, configuração, instalação e localização.

Exemplos: O sistema deve permitir ativar/desativar logs de depuração via painel administrativo; Todas as mensagens de erro devem ser localizáveis e configuráveis para múltiplos idiomas.

- **Restrições de Design** – Especifica ou restringe o design de um sistema, por exemplo, uma arquitetura, um conjunto de componentes, formas de comunicação entre componentes.

Exemplos: O sistema deve ser construído com arquitetura de microsserviços; Os componentes do sistema devem se comunicar exclusivamente via RESTful APIs.

- **Requisitos de Implementação** – Especifica ou restringe o código ou a construção de um sistema. Por exemplo: padrões obrigatórios, linguagens de implementação, políticas para integridade de banco de dados, limites de recursos e ambientes de operação.

Exemplos: O backend deve ser implementado em Java 17 com uso obrigatório de Spring Boot; O sistema deve utilizar banco de dados PostgreSQL e adotar versionamento de scripts via Liquibase.

- **Requisitos de Interface** – Especifica um item externo com o qual o sistema deve interagir, restrições de formatos, tempos ou outros fatores utilizados por tal interação.

Exemplos: O sistema deve consumir a API pública da Receita Federal no formato JSON via HTTPS; A exportação de dados para o sistema contábil deve seguir o layout XML padronizado pela organização.

- **Requisitos Físicos** – Especifica uma característica física que um sistema deve possuir, por exemplo: material, forma, tamanho e peso.

Exemplos: O terminal de autoatendimento deve pesar no máximo 25 kg para facilitar o transporte; O equipamento deve suportar operação em ambientes com temperatura entre -10 °C e 50 °C.

Para Sommerville (2019) os RNFs estão estruturados em três grandes categorias:

- Requisitos do produto;
- Requisitos organizacionais;
- Requisitos externos.

A seguir, apresentamos uma descrição e exemplos de cada dos RNFs abordados por Sommerville (2019):

- **Requisitos do Produto:** Definem propriedades técnicas que o sistema deve apresentar durante sua execução. São diretamente percebidos pelo usuário ou pelo ambiente de operação do sistema.

- **Requisitos de Usabilidade:** Tratam da facilidade de uso, aprendizagem e interação do usuário com o sistema. Incluem aspectos como ergonomia, consistência visual, acessibilidade e suporte à aprendizagem.

Exemplos: A interface deve seguir os princípios de acessibilidade da WCAG 2.1.; O sistema deve disponibilizar assistente de preenchimento nos formulários de cadastro.

- **Requisitos de Dependabilidade:** Abrangem atributos como confiabilidade, disponibilidade, robustez e tolerância a falhas. Indicadores comuns são tempo médio entre falhas e capacidade de recuperação.

Exemplos: O sistema deve manter disponibilidade mínima de 99,9% durante o horário comercial; Após uma falha, o sistema deve retomar a execução no último estado válido em até 30 segundos.

- **Requisitos de Segurança da Informação (Security):** Referem-se à proteção de dados e recursos contra acessos não autorizados, perda, vazamento ou modificação maliciosa.

Exemplos: Implementar autenticação por múltiplos fatores para perfis administrativos; Todos os dados devem ser criptografados com AES-256.

- **Requisitos de Eficiência:** Estão relacionados à otimização de recursos computacionais como tempo de CPU, uso de memória, largura de banda e espaço de armazenamento.
  - **Requisitos de Desempenho:** Focam em tempo de resposta, throughput<sup>14</sup> e escalabilidade sob diferentes condições de carga.  
  
Exemplos: O tempo médio de resposta para buscas não deve ultrapassar 800 ms; O sistema deve suportar 1.000 conexões simultâneas sem degradação perceptível de desempenho.
  - **Requisitos de Espaço:** Estabelecem limites para uso de armazenamento, memória ou outros recursos físicos.  
  
Exemplos: O consumo de memória RAM não deve ultrapassar 300 MB em uso típico; Os arquivos de log devem ser rotacionados automaticamente quando atingirem 50 MB.
- **Requisitos Organizacionais:** São impostos por políticas, práticas e diretrizes da organização do cliente ou da equipe de desenvolvimento.
  - **Requisitos Ambientais:** Definem as condições físicas ou operacionais do ambiente onde o sistema será executado, como temperatura, umidade, energia ou espaço físico.  
  
Exemplos: O servidor deve operar de forma estável entre 0 °C e 45 °C; O equipamento embarcado deve resistir a vibrações em transporte ferroviário.
  - **Requisitos Operacionais:** Relacionam-se à rotina de funcionamento do sistema, incluindo regras de ativação, cronogramas, dependência de outros sistemas ou processos de operação.  
  
Exemplos: O sistema deve iniciar automaticamente junto com o sistema operacional; A rotina de backup deve ser executada diariamente às 2h da manhã, sem intervenção manual.
  - **Requisitos de Desenvolvimento:** Orientam ou restringem tecnologias, metodologias e ferramentas utilizadas na construção do sistema.  
  
Exemplos: Todas as APIs devem ser desenvolvidas seguindo o padrão OpenAPI 3.0; O projeto deve utilizar controle de versão com Git e integração contínua via Jenkins.
- **Requisitos Externos:** Provenientes de legislações, normas e fatores externos ao projeto.
  - **Requisitos Regulatórios:** Impostos por normas técnicas, certificações ou contratos regulatórios.

---

<sup>14</sup> Throughput, no contexto de engenharia de software e desempenho de sistemas, é uma métrica que representa a quantidade de trabalho ou operações que um sistema consegue processar em um determinado intervalo de tempo.

Exemplos: A solução deve atender aos requisitos da norma ISO/IEC 27001; Sistemas de saúde devem estar em conformidade com a RDC 751/2022 da ANVISA.

- **Requisitos Éticos:** Estão ligados a princípios morais, privacidade, justiça e transparência na coleta e uso de informações.

Exemplos: O sistema não deve coletar dados de geolocalização sem o consentimento do usuário; Todos os relatórios devem permitir anonimização dos dados pessoais dos usuários.

- **Requisitos Legislativos:** Referem-se a leis e regulamentos de cumprimento obrigatório nos domínios em que o sistema será utilizado.

Exemplos: Todos os dados pessoais devem ser tratados de acordo com a Lei Geral de Proteção de Dados (LGPD); O sistema de faturamento deve gerar notas fiscais eletrônicas segundo o padrão NFe da Receita Federal.

- **Requisitos de Segurança (Safety) / Segurança da Informação (Security):**

Incluem tanto a segurança física (safety), relacionada à integridade de pessoas e equipamentos, quanto a segurança da informação (security).

Exemplos: O sistema de controle de tráfego deve prevenir colisões mesmo em caso de falha de sensores; Deve haver mecanismos de verificação de integridade de firmware em sistemas embarcados.

- **Requisitos Contábeis:** Relacionam-se à conformidade com práticas e regras contábeis da organização ou normas financeiras externas.

Exemplos: O sistema deve gerar relatórios de movimentação financeira com base no plano de contas vigente; Os lançamentos contábeis devem ser armazenados por no mínimo 5 anos, conforme legislação fiscal.

A classificação de requisitos não funcionais (RNFs) proposta por Sommerville (2019) e o modelo URPS+, desenvolvido por Grady e Caswell (1987), podem ser utilizadas de forma complementar ao longo do processo de engenharia de requisitos para oferecer uma cobertura abrangente, tanto em termos estruturais quanto contextuais (ver o Quadro 6).

| Critério         | Sommerville (2019)                                     | URPS+ (Grady e Caswell, 1987)                 |
|------------------|--------------------------------------------------------|-----------------------------------------------|
| Foco principal   | Origem e contexto dos requisitos.                      | Qualidade do produto (dimensões mensuráveis). |
| Organização      | Três grandes grupos: produto, organizacional, externo. | Cinco categorias técnicas + extensões (+).    |
| Aplicação típica | Identificação, rastreabilidade, conformidade.          | Especificação de atributos de qualidade.      |

Quadro 6 – Natureza das Classificações de RNFs: Visões Complementares.  
Fonte: elaborado pelo autor (2025).

## 6.3 Regras de Negócio (RNs)

No desenvolvimento de software, entender claramente as necessidades que devem ser atendidas pelo sistema é essencial para seu sucesso. Essas necessidades são definidas e organizadas em forma de requisitos, que descrevem o que o sistema deve realizar. Entretanto, igualmente importantes são as regras de negócio, que **orientam e estabelecem o comportamento esperado desses requisitos**, definindo restrições, condições ou políticas que regem o funcionamento do negócio e, consequentemente, o sistema de software.

Neste livro, regra de negócio é uma **declaração normativa do domínio** que expressa políticas, condições, restrições ou critérios de decisão que independem da solução tecnológica: ela diz o que deve valer (ou o que é permitido/proibido) para o negócio funcionar corretamente. Em geral, regras de negócio podem ser declaradas como “SE condição, ENTÃO consequência”, como fórmulas, limites, exceções, prioridades e condições de validade.

Por outro lado, **não é regra de negócio** aquilo que descreve como implementar (p.ex., “usar Kafka”, “persistir em PostgreSQL”, “aplicar lock pessimista”), nem aquilo que impõe um atributo de qualidade sem referência direta ao domínio (p.ex., “tempo de resposta < 2s”, “99,9% de disponibilidade”). Esses casos são, respectivamente, decisões de solução/arquitetura e requisitos não funcionais — ainda que possam ser motivados por riscos, custos ou conformidade. Assim, a regra de negócio pode gerar RNFs e decisões técnicas como decorrência, mas não se confunde com eles.

Por fim, regras de negócio **não são requisitos por si mesmas**: elas **condicionam** requisitos ao impor critérios e limites do domínio que precisam ser respeitados para que o sistema opere de forma correta e conforme o contexto organizacional (p.ex., legislação, regulamentos e políticas internas).

Por exemplo, considere o requisito funcional “Cadastrar Estudante”. As regras de negócio associadas poderiam incluir:

- Cada estudante só pode ter uma matrícula associada a ele;
- Cada número de matrícula só pode pertencer a um estudante;
- O estudante inicialmente possui status de ‘Pré-Matriculado’ até que seus documentos sejam validados;
- Não é permitido cadastrar estudantes com pendências financeiras anteriores.

Essas regras orientam claramente como o requisito deve se comportar, assegurando que o sistema funcione alinhado aos princípios e políticas organizacionais.

### 6.3.1 Complementaridade e Relacionamento entre Regras de Negócio, Requisitos Funcionais e Não Funcionais

A relação entre regras de negócio e requisitos é essencialmente complementar. As regras definem as condições e limitações contextuais, enquanto os requisitos descrevem as funcionalidades e capacidades esperadas do software. Além disso, as regras de negócio frequentemente estão associadas não apenas a requisitos funcionais (RFs), mas também a requisitos não funcionais (RNFs).

Ao observarmos as classificações de RNFs proposta por Sommerville (2019) e de RFs proposta por Grady e Caswell (1987) podemos identificar as seguintes relações entre RNFs, RFs e RNs. A Figura 40 que representa o modelo conceitual dessas relações.

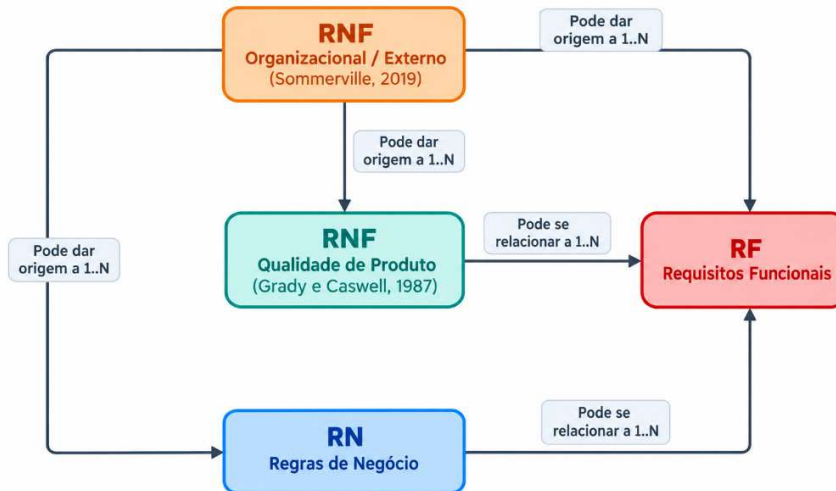


Figura 40 – Modelo Conceitual da Relação entre RNFs, RFs e RNs.  
Fonte: elaborado pelo autor (2025).

A seguir, apresentamos um exemplo visando tornar essas relações mais concretas. Para tanto, utilizamos como base uma norma da Agência Nacional de Vigilância Sanitária (ANVISA), do Governo Brasileiro, a qual estabelece os critérios e procedimentos para a classificação de risco, a regularização, a rotulagem e as instruções de uso de dispositivos médicos no Brasil.

- **RNF Externo (Regulatório):**
  - RDC nº 751/2022 da ANVISA
- **RNF de Produto (URPS+):**
  - Suportabilidade: O sistema deve garantir rastreabilidade dos registros de dispositivos médicos, incluindo identificação do autor da ação, data e hora da modificação, tipo de operação (criação, edição, exclusão) e vínculo com identificador único do dispositivo;
  - Suportabilidade: O sistema deve garantir auditabilidade por meio de geração automática de trilhas de auditoria invioláveis, acessíveis apenas a perfis autorizados.
- **Requisito Funcional:**
  - O sistema deve permitir o cadastro de dispositivos médicos.
- **Regra de Negócio:**
  - Todo dispositivo médico cadastrado deve conter as informações exigidas pela RDC nº 751/2022 antes do registro, sendo: identificação do produto, identificação do fabricante, documentação técnica, informações de rotulagem, instruções de uso, histórico e regularidade, dados de certificação e legislação aplicável.

Um sistema que implementa requisitos sem considerar regras de negócio corre o risco de entregar soluções inconsistentes com as políticas e expectativas da organização. Por outro lado,

considerar regras de negócio sem um claro mapeamento para requisitos pode levar a soluções rígidas e difíceis de evoluir.

Dessa forma, uma engenharia de requisitos eficaz incorpora explicitamente as regras de negócio em seu processo, garantindo uma solução tecnológica que seja ao mesmo tempo robusta, aderente e adaptável às necessidades organizacionais e operacionais da empresa.

Neste sentido, integrar efetivamente regras de negócio ao ciclo de vida dos requisitos de software é uma prática essencial na engenharia de requisitos. Esta integração proporciona clareza, reduz ambiguidades e assegura que o software desenvolvido atenda verdadeiramente às necessidades do negócio, reforçando a importância estratégica da engenharia de requisitos na criação de soluções de software eficazes e sustentáveis.

## CAPÍTULO 6: PONTOS-CHAVE

- Os requisitos são apresentados como **fundamentais na engenharia de software**, pois garantem que os produtos desenvolvidos atendam às necessidades dos stakeholders e sustentem as fases seguintes do projeto (SOMMERVILLE, 2019).
- De acordo com a ISO/IEC/IEEE 15288: 2023, um requisito é uma **declaração mensurável** que descreve características, atributos, funcionalidades ou capacidades que um sistema deve possuir para atender às necessidades das partes interessadas (IEEE, 2023).
- Os requisitos devem ser derivados de uma **visão orientada pelo usuário, expressos de forma técnica** e evitando a imposição de soluções específicas.
- O capítulo diferencia tipos de requisitos:
  - **Requisitos Funcionais (RFs)** — descrevem o que o sistema deve fazer, incluindo serviços, ações ou comportamentos esperados.
  - **Requisitos Não Funcionais (RNFs)** — descrevem como o sistema deve se comportar, envolvendo desempenho, segurança, usabilidade, disponibilidade e outros atributos de qualidade.
- São apresentadas características de um bom requisito, destacando-se **clareza, completude, consistência, verificabilidade e viabilidade**.
- O texto também introduz o conceito de **Regras de Negócio (RNs)**, que delimitam políticas, condições e restrições que influenciam o comportamento do sistema.
- Ao final do capítulo é apresentada a **complementaridade e o relacionamento entre RNs, RFs e RNFs**, enfatizando que as regras de negócio servem de base para a definição e a coerência dos requisitos funcionais e não funcionais, garantindo alinhamento entre o domínio organizacional e o sistema de software.

# Capítulo 7

## 7 Elicitação e Descoberta de Requisitos

### 7.1 Introdução

**Elicitação e descoberta de requisitos é uma das atividades fundamentais da Engenharia de Requisitos.** Se, no Capítulo 5, foram apresentadas as atividades constitutivas da ER, seus princípios e valores, este capítulo aprofunda o trabalho de aproximação com os stakeholders, com o domínio e com o contexto no qual a solução será desenvolvida.

A elicitación não deve ser confundida com simples coleta de informações. Trata-se de uma atividade essencialmente humana e investigativa, na qual expectativas, necessidades, desejos, frustrações, regras e restrições são explorados e interpretados para ampliar o entendimento sobre o problema e sobre aquilo que o produto precisa atender.

Embora costume ser particularmente intensa nos momentos iniciais, a elicitación e descoberta pode ser retomada em qualquer momento do ciclo de vida do produto. Novas informações, feedbacks, mudanças de contexto, dúvidas ou evidências de uso podem revelar aspectos ainda não compreendidos ou exigir maior aprofundamento de requisitos já conhecidos. Por isso, sua intensidade e seu foco variam ao longo do tempo.

Antes de aprofundar a elicitación e descoberta, é importante delimitar sua responsabilidade no conjunto da Engenharia de Requisitos. A Figura 41 sintetiza a questão que orienta essa atividade, o trabalho que nela é realizado, algumas técnicas e práticas que podem apoiá-la, seus principais resultados e suas relações com as demais atividades.

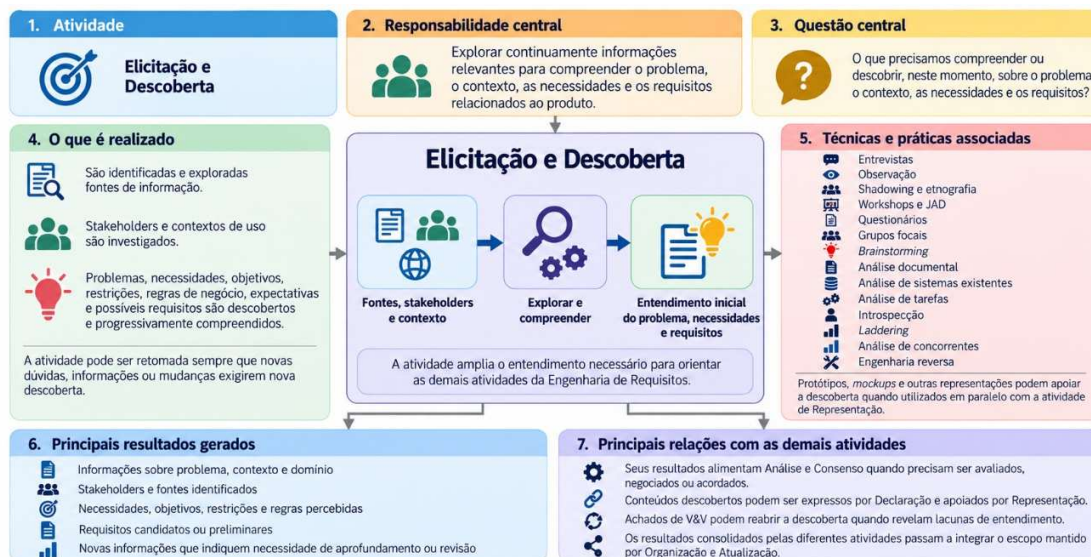


Figura 41 – Enquadramento da atividade de Elicitação e Descoberta.

Fonte: elaborado pelo autor (2026).



## 7.2 A importância da atividade de elicitación e descoberta

Diversos estudos apontam que a inadequada elicitación está entre as principais causas de fracasso em projetos de software e serviços digitais (SOMMERVILLE, 2019; WAGNER et al., 2019; STANDISH GROUP, 2020). Estatísticas recentes indicam que cerca de dois terços dos projetos apresentam problemas significativos relacionados a falhas de requisitos, seja pela incompletude, pela ambiguidade ou pela falta de alinhamento entre as partes envolvidas. A análise de Wagner et al. (2019), baseada em uma pesquisa global, confirma que problemas recorrentes em Engenharia de Requisitos não decorrem apenas de lacunas técnicas, mas de questões humanas como comunicação deficiente, divergência de expectativas e limitações culturais ou organizacionais.

A comunicação entre engenheiros de software e stakeholders ocorre por múltiplos meios — falas, gestos, anotações, imagens, demonstrações e registros de situações de uso — que ampliam as possibilidades de observação e interpretação. Embora muitas técnicas de elicitación utilizem registros escritos, recursos multimodais, como gravações de áudio, vídeo e registros visuais de oficinas, podem preservar nuances do contexto e da interação que dificilmente seriam recuperadas apenas por transcrições ou notas. Esses registros não substituem a interpretação humana, mas ampliam as fontes disponíveis para compreender manifestações, situações e perspectivas relevantes ao produto (SANTOS, A. C. M. et al., 2020).

A importância da elicitación e descoberta também se estende a iniciativas de transformação digital, inovação em serviços públicos ou privados e soluções baseadas em Inteligência Artificial. Nesses contextos, compreender o que stakeholders fazem, necessitam, esperam, valorizam ou percebem como problema exige atenção tanto ao que é explicitamente dito quanto às condições sociais, organizacionais, técnicas e normativas que moldam essas manifestações. A descoberta permite aproximar essas diferentes perspectivas e ampliar progressivamente o entendimento do problema e do produto.

Independentemente do ciclo de vida adotado, a qualidade da descoberta depende da capacidade de selecionar fontes relevantes, conduzir interações adequadas ao contexto, formular perguntas produtivas, observar práticas, reconhecer informações implícitas e registrar evidências que possam ser retomadas ao longo do trabalho. Essas competências combinam conhecimento técnico e de domínio com habilidades de comunicação, escuta, interpretação e investigação, especialmente porque stakeholders nem sempre expressam suas necessidades de maneira direta, completa ou livre de ambiguidades (SOMMERVILLE, 2019; WIEGERS; HOKANSON, 2023).

## 7.3 Fontes de informação

Um dos aspectos críticos da elicitación e descoberta é a definição das **fontes de informação** (Figura 42) que alimentam o processo. A literatura reconhece que a qualidade dos requisitos está diretamente associada à diversidade e confiabilidade dessas fontes (HULL et al., 2011; WIEGERS e HOKANSON, 2023). Na prática, não basta ouvir apenas os usuários finais ou consultar documentação existente: é necessário construir uma visão multifacetada que incorpore perspectivas de stakeholders, artefatos organizacionais, práticas cotidianas e dados de apoio.

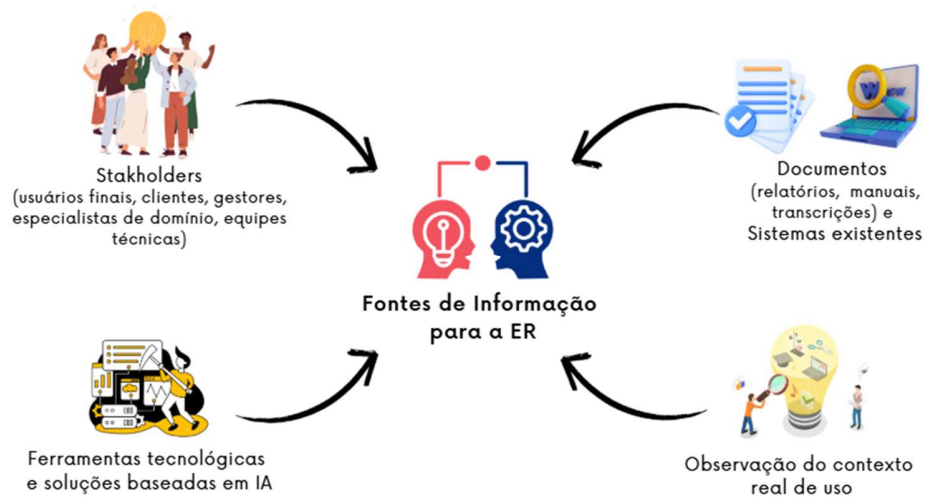


Figura 42 – Fontes de Informação da ER.  
Fonte: elaborado pelo autor (2025).

A primeira e mais evidente fonte de informação são os **stakeholders**. Esses atores incluem usuários finais, clientes, gestores, especialistas de domínio, equipes técnicas e, em alguns casos, órgãos reguladores. Entretanto, stakeholders não são homogêneos: possuem interesses divergentes, diferentes níveis de poder e vocabulários muitas vezes incompatíveis. Como observa Young (2004), a elicitação falha quando supõe que todos falam a mesma língua ou compartilham a mesma visão do problema. O papel do engenheiro de software, nesse caso, é reconhecer e lidar com os conflitos inerentes, transformando divergências em insumos úteis para a construção da solução. Nesse conjunto, merece atenção especial o **usuário final**, cuja experiência cotidiana de quem irá utilizar a solução traz necessidades práticas e restrições de uso que dificilmente podem ser plenamente representadas apenas por gestores ou especialistas. Preservar a voz do usuário ao longo de todo o processo de ER é fundamental para garantir que a solução final seja não apenas tecnicamente correta, mas também efetivamente útil no contexto real de aplicação.

#### Box 7.3-1 – Reconhecer e lidar com conflitos

Este ponto concretiza o **Princípio 3 (Satisfazer as partes interessadas)** ao reconhecer papéis e perspectivas distintas, e o **Valor Respeito** ao tratar diferenças de vocabulário e poder como dados do problema — não como ‘ruído’ a ser ignorado.

Outra fonte importante são os **documentos e sistemas existentes**. Relatórios, manuais, registros e sistemas legados podem oferecer informações valiosas sobre práticas estabelecidas e problemas recorrentes. Contudo, há riscos evidentes: esses materiais podem estar desatualizados ou refletem apenas a visão formal da organização, sem capturar o que realmente acontece no cotidiano. A literatura aponta para a necessidade de **triangulação**, ou seja, confrontar informações documentais com observações diretas e relatos de usuários (SOMMERVILLE, 2019).

### Box 7.3-2 – Triangulação de Técnicas

A triangulação reforça o **Princípio 4 (Entendimento compartilhado)**: documentos estabelecem memória, mas o entendimento só se consolida quando evidências documentais são confrontadas com observação e fala situada — em ciclos de **Feedback**.

Outra fonte de informação documental a ser considerada são as **transcrições** realizadas a partir de atividades conversacionais da elicitación e descoberta, como entrevistas, reuniões de alinhamento, workshops de requisitos e oficinas de co-criação – as quais, em contextos atuais, podem ocorrer de modo **remoto** e mediado por plataformas de videoconferência (Teams, Meet, Zoom, entre outros), o que torna viável registrar o encontro por gravação e produzir os textos das falas dos participantes, para uso posterior.

A **observação do contexto real de uso** constitui uma fonte frequentemente negligenciada, mas extremamente poderosa. Métodos como *shadowing*<sup>15</sup>, etnografia<sup>16</sup> e análise de tarefas<sup>17</sup> permitem revelar discrepâncias entre o “trabalho prescrito” (como deveria ser feito segundo normas e manuais) e o “trabalho real” (como de fato acontece). Essa distinção é fundamental, pois muitas frustrações de stakeholders só emergem quando se acompanha a execução das atividades em seu ambiente natural. Um exemplo típico ocorre em serviços de atendimento ao público, onde gestores podem afirmar que o sistema funciona bem, enquanto operadores revelam, em observações diretas, que precisam duplicar cadastros devido a limitações do software atual.

### Box 7.3-3 – ‘Trabalho prescrito’ versus ‘trabalho real’

Essa distinção explicita o **Princípio 5 (Contexto da solução)**: requisitos emergem do sistema sociotécnico em operação, não apenas de narrativas formais; por isso, observar o ‘real’ é uma prática de **Comunicação** com o contexto, não um detalhe metodológico.

Por fim, as **ferramentas tecnológicas e soluções baseadas em Inteligência Artificial (IA)** vêm se consolidando como fontes emergentes de apoio. Modelos de Processamento de Linguagem Natural podem analisar grandes volumes de documentos e identificar termos recorrentes; algoritmos de mineração de dados podem detectar padrões de uso e sugerir melhorias; e chatbots podem facilitar a coleta inicial de manifestações de usuários. Entretanto, tais ferramentas não estão isentas de limitações: podem gerar informações enviesadas, apresentar resultados incorretos ou expor riscos de confidencialidade (ZHAO; XU; YANG, 2022). Essas ferramentas podem ser utilizadas também, em conjunto com fontes de informação documental, por exemplo, transcrições, a partir das quais, ferramentas baseadas em IA podem auxiliar a identificar problemas,

<sup>15</sup> Técnica de observação na qual o engenheiro de software acompanha de perto um usuário durante a execução de suas atividades, registrando ações, dificuldades e interações em tempo real. Essa abordagem é destacada por Sommerville (2019) como essencial para revelar requisitos que dificilmente emergem em entrevistas formais.

<sup>16</sup> Método de imersão do analista no ambiente de trabalho dos stakeholders, utilizado para captar práticas, interações e restrições que não aparecem em documentos ou declarações formais. Pohl (2010) enfatiza que a etnografia contribui para identificar requisitos implícitos e compreender o “trabalho real” em contraste com o “trabalho prescrito”.

<sup>17</sup> Técnica estruturada que decompõe atividades em etapas menores, identificando objetivos, decisões e erros potenciais dos usuários. Sharp, Rogers e Preece (2019) apresentam a análise de tarefas como instrumento fundamental para compreender o comportamento dos usuários e apoiar a descoberta de requisitos de usabilidade e desempenho.

necessidades de usuários, requisitos funcionais e não funcionais, regras de negócios e fluxos de trabalho, por exemplo.

#### Box 7.3-4 – Princípios para o uso e IA

O uso de IA na elicitação e descoberta, portanto, deve ser pautado pelos princípios de responsabilidade, transparência e ética apresentados no Capítulo 6, funcionando sempre como apoio complementar à interação humana e não como substituto dela, em coerência com o **Princípio 2 (Comunicação dos requisitos)** e com os valores de **Coragem** (para questionar saídas plausíveis) e **Confiança** (sustentada pela explicitação de limitações e vieses).

Ao reunir essas diferentes fontes (Quadro 7), o engenheiro de software cria um mosaico interpretativo que aumenta a confiabilidade da elicitação e descoberta. Mais do que quantidade de informações, o que importa é a **capacidade crítica de interpretar, confrontar e negociar** esses dados. É nesse ponto que os princípios discutidos no Capítulo 6 — colaboração, adaptabilidade e transparência — tornam-se guias indispensáveis para conduzir um processo de descoberta eficaz e responsável.

| Fonte                                   | Exemplos                                                      | Desafios associados                                                           |
|-----------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Stakeholders</b>                     | Usuários finais, clientes, gestores, especialistas de domínio | Interesses divergentes, conflitos de poder, vocabulários distintos            |
| <b>Documentos e sistemas existentes</b> | Manuais, relatórios, registros históricos, sistemas legados   | Informação desatualizada, visão parcial da realidade                          |
| <b>Observação do contexto real</b>      | Shadowing, etnografia, análise de tarefas                     | Resistência de equipes, percepção seletiva do analista                        |
| <b>Ferramentas e IA</b>                 | Repositórios de requisitos, NLP, mineração de dados, chatbots | Viés algorítmico, risco de confidencialidade, necessidade de validação humana |

Quadro 7 – Principais fontes de informação para a elicitação e descoberta.  
Fonte: elaborado pelo autor (2025).

## 7.4 Elementos fundamentais da elicitação e descoberta

Para compreender a elicitação e descoberta na prática, é útil reconhecer alguns elementos que orientam a investigação do problema, do contexto e das manifestações dos stakeholders (Figura 43). Esses elementos não funcionam como etapas nem como condições que precisam estar completamente resolvidas antes que outras atividades ocorram. Eles constituem referências para observar o domínio, delimitar problemas, compreender objetivos, reconhecer valor e identificar pontos que ainda precisam ser explorados ao longo da construção do entendimento.



Figura 43 – Elementos para a eliciação e descoberta.  
Fonte: elaborado pelo autor (2025).

O primeiro elemento é a **compreensão do domínio do negócio**. Nenhum requisito pode ser corretamente entendido sem uma visão mínima dos processos, regras, terminologias e restrições do contexto em que a solução será inserida. Isso inclui desde áreas técnicas (como saúde, educação ou finanças) até aspectos culturais e regulatórios. Por exemplo, em um projeto de modernização de serviços educacionais, compreender o calendário acadêmico, os processos de matrícula e as regras de avaliação é tão essencial quanto conhecer as necessidades dos alunos e professores. Sem esse entendimento, corre-se o risco de propor requisitos inviáveis ou desconectados da realidade do domínio.

O segundo elemento é a **delimitação do problema a ser resolvido**. Muitas iniciativas fracassam não porque os engenheiros não coletaram informações suficientes, mas porque não houve clareza sobre qual problema realmente deveria ser tratado. A literatura reforça que requisitos mal definidos são frequentemente consequência de uma visão difusa sobre os objetivos (SOMMERVILLE, 2019; WIEGERS e HOKANSON, 2023). Em termos práticos, delimitar o problema significa responder a perguntas simples, mas críticas: *o que exatamente precisa mudar? para quem? com qual finalidade?*

Outro pilar é a **identificação de objetivos e valor esperado**. De acordo com os princípios apresentados no Capítulo 6, a Engenharia de Requisitos deve estar orientada a gerar valor, e não apenas a entregar funcionalidades. Isso implica reconhecer que um requisito só é válido se contribuir para objetivos claros — sejam eles aumentar eficiência, reduzir custos, melhorar a experiência do usuário ou assegurar conformidade regulatória. Um caso ilustrativo ajuda a compreender: em um hospital, gestores pediram uma “nova tela de relatórios” para agilizar decisões. Mas a análise revelou que a necessidade real era **garantir informações em tempo real para decisões clínicas**. O valor não estava no relatório em si, mas na agilidade de resposta em situações críticas.

#### Box 7.4-1 – Inovação na elicitação e descoberta

Este ponto concretiza o **Princípio 9 (Inovação da ER)**: a elicitação e descoberta não se limita a registrar o que é dito, mas busca ampliar a compreensão do problema por meio de perguntas, reframing (mudar o “modo de ver” o problema) e exploração de alternativas — sempre em diálogo com os stakeholders e com o contexto. A inovação, aqui, aparece como responsabilidade técnica: propor possibilidades, explicitar trade-offs e testar entendimentos.

Durante a descoberta, características como vagueza, contradição, ausência de condições importantes ou dificuldade de imaginar como uma expectativa poderia ser observada funcionam principalmente como sinais de que o entendimento ainda precisa ser aprofundado. Uma manifestação como “o sistema deve ser rápido”, por exemplo, não precisa ser descartada por ainda ser imprecisa; ela pode orientar novas perguntas sobre em que situações o desempenho importa, para quem, sob quais condições e qual resultado seria considerado adequado. Os critérios de qualidade dos requisitos serão examinados sistematicamente no Capítulo 11, mas já durante a elicitação podem ajudar a reconhecer questões que merecem investigação adicional (ISO/IEC/IEEE 29148:2018; WIEGERS; BEATTY, 2013).

Os elementos sintetizados no Quadro 8 ajudam, portanto, a orientar a investigação sem transformar a elicitação em um procedimento mecânico. Seu valor está em ampliar a capacidade de compreender o domínio, o problema, os objetivos e as manifestações dos stakeholders, preservando abertura suficiente para que novas informações e interpretações continuem emergindo à medida que o trabalho evolui.

| Elemento                                  | Papel no processo                                     | Caso ilustrativo                                                                                                   |
|-------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Compreensão do domínio</b>             | Captar processos, regras, restrições e terminologias. | Projeto educacional: sem conhecer o calendário acadêmico, requisitos de matrícula ficaram inconsistentes.          |
| <b>Delimitação do problema</b>            | Definir claramente o que deve ser resolvido.          | Projeto de logística: coletaram-se 50 requisitos, mas o problema real era reduzir atrasos de entrega em 20%.       |
| <b>Identificação de objetivos e valor</b> | Relacionar requisitos a benefícios concretos.         | Hospital: pedido de “relatórios” era, na verdade, necessidade de informações em tempo real para decisões clínicas. |
| <b>Crítérios de qualidade</b>             | Garantir clareza, viabilidade e testabilidade.        | “Sistema deve ser rápido” → reformulado para tempo de resposta de até 2 segundos em 95% dos casos.                 |

Quadro 8 – Elementos fundamentais da elicitação e descoberta.  
Fonte: elaborado pelo autor (2025).

## 7.5 Necessidades, desejos e frustrações

Stakeholders raramente expressam de forma direta o que realmente precisam. Muitas vezes manifestam **desejos superficiais**, que podem refletir preferências pessoais ou modismos tecnológicos; outras vezes apontam **necessidades críticas**, que expressam condições indispensáveis para o funcionamento do sistema ou serviço; em outros casos revelam **frustrações**, que emergem de situações reais experimentadas no uso de processos ou ferramentas existentes. Diferenciar esses três tipos de manifestação é uma das tarefas mais delicadas da elicitação e descoberta (Figura 44).



Figura 44 – Desejos, necessidades e frustrações.  
Fonte: elaborado pelo autor (2025).

Os **desejos** frequentemente aparecem como pedidos de funcionalidades específicas. Contudo, um desejo nem sempre corresponde a um requisito necessário. Por exemplo, em uma startup de e-commerce, alguns gestores pediram que o aplicativo tivesse integração com uma rede social emergente. Esse pedido refletia mais uma **tendência de mercado** do que uma real necessidade de negócio. Após análise, percebeu-se que o requisito não agregava valor mensurável e poderia desviar recursos de prioridades mais relevantes. O engenheiro de software deve, nesse caso, acolher o desejo como informação, mas questionar se ele se alinha aos objetivos do projeto.

As **necessidades** representam aquilo que é essencial para o sucesso da solução. Elas estão ligadas a objetivos de negócio, metas organizacionais ou requisitos normativos. Retomando o exemplo do Cap. 5, uma necessidade poderia ser “reduzir o tempo de atendimento em 30%” em um call center. Essa declaração traduz diretamente uma meta organizacional em um requisito de desempenho que deve orientar o projeto.

As **frustrações**, por sua vez, são manifestações negativas, muitas vezes não expressas como pedidos diretos. Elas emergem quando os stakeholders relatam experiências ruins ou limitações dos sistemas atuais. Por exemplo, em um serviço público, cidadãos relatavam que “ninguém conseguia concluir a inscrição online à noite”. Essa frustração revelou uma falha de disponibilidade no sistema, que só pôde ser identificada após observação do uso real. Frustrações são importantes porque frequentemente revelam requisitos ocultos — aspectos críticos que não seriam captados apenas em entrevistas ou documentos.

Em muitos casos, a análise dessas frustrações leva à identificação dos chamados **requisitos latentes**, isto é, necessidades que não são declaradas de forma explícita pelos stakeholders, mas que influenciam o sucesso da solução. Esses requisitos emergem quando se observa o contraste entre o “trabalho prescrito” e o “trabalho real”, ou quando usuários relatam dificuldades sem traduzi-las diretamente em pedidos de novas funcionalidades. Por exemplo, ao afirmar que “o sistema fica indisponível à noite”, o cidadão pode estar revelando um requisito latente de disponibilidade contínua do serviço, que não havia sido formulado claramente até então.

Distinguir entre desejos, necessidades, frustrações e requisitos latentes (Quadro 9) exige do engenheiro de software competências analíticas e sociais. A análise não deve ser feita de forma isolada, mas **dialogada com os stakeholders**, equilibrando perspectivas técnicas, organizacionais e humanas. Como destacam Bano e Zowghi (2015), o envolvimento dos usuários só gera impacto positivo quando há mecanismos adequados de interpretação e priorização. Assim, a elicitação se torna não apenas uma atividade de registro, mas uma prática de interpretação crítica e construção de sentidos.



| Manifestação             | Exemplo prático                                                                                                                                                                                      | Interpretação na elicitação                                                                                           |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Desejo</b>            | “Queremos um chatbot no aplicativo de vendas”.                                                                                                                                                       | Pode ser descartável ou substituído por outra solução, depende do valor agregado.                                     |
| <b>Necessidade</b>       | “Precisamos reduzir o tempo médio de atendimento em 30%”.                                                                                                                                            | Essencial, traduz objetivo organizacional em requisito mensurável.                                                    |
| <b>Frustração</b>        | “O sistema fica indisponível à noite”.                                                                                                                                                               | Aponta lacuna crítica, revela requisito oculto (latente) de disponibilidade.                                          |
| <b>Requisito latente</b> | Ao afirmar que “o sistema fica indisponível à noite”, o cidadão pode estar revelando um requisito latente de disponibilidade contínua do serviço, que não havia sido formulado claramente até então. | Necessidade não verbalizada, descoberta por observação ou análise crítica, mas fundamental para o sucesso da solução. |

Quadro 9 – Diferenciação entre desejos, necessidades, frustrações e requisitos latentes.

Fonte: elaborado pelo autor (2025).

O que se descobre pela elicitação passa a integrar o entendimento em construção sobre o produto e pode ser aprofundado, discutido, declarado, representado ou avaliado conforme a necessidade do contexto. Essas interações também podem produzir novas descobertas.

## 7.6 Descoberta de requisitos funcionais e não funcionais

Uma dimensão central da elicitação e descoberta é reconhecer, a partir das manifestações dos stakeholders e de outras fontes, requisitos que ainda não haviam sido explicitados ou aspectos que aprofundam requisitos já conhecidos. Desejos, necessidades, frustrações, regras, restrições e evidências do contexto podem revelar comportamentos esperados e condições de qualidade relevantes para o produto. Nesse processo, requisitos funcionais e não funcionais podem emergir em diferentes níveis de abstração e amadurecer à medida que o entendimento evolui.

Os **requisitos funcionais (RFs)** descrevem *o que a solução deve fazer*, ou seja, as funções, comportamentos ou serviços que o sistema, produto ou processo deve oferecer. Já os **requisitos não funcionais (RNFs)** especificam *como a solução deve se comportar*, incluindo critérios de desempenho, segurança, usabilidade, acessibilidade, conformidade legal e outros atributos de qualidade (ISO/IEC 25010, 2011). Ambos são igualmente importantes: enquanto os funcionais definem a utilidade básica da solução, os não funcionais determinam sua aceitação, satisfação e longevidade no contexto de uso.

Um exemplo ilustrativo ajuda a distinguir esses dois tipos. Em um projeto de desenvolvimento de um aplicativo bancário, um RF poderia ser “permitir a transferência de valores entre contas de forma online”. Esse requisito descreve uma função essencial do serviço. Já um RNF associado poderia ser “garantir que 95% das transações sejam concluídas em até três segundos, com disponibilidade mínima de 99,9%”. Neste caso, a função existe, mas o valor percebido pelo usuário depende das condições de qualidade em que ela é executada. Outros exemplos podem ser vistos no Quadro 10.



| Tipo                 | Exemplos práticos                                                              | Observações na elicitación                                                                       |
|----------------------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Funcional</b>     | “Permitir que o usuário solicite segunda via de boleto online”                 | Descreve <i>o que</i> a solução deve oferecer. Pode emergir de necessidade explícita do usuário. |
| <b>Funcional</b>     | “Registrar histórico de atendimentos no prontuário eletrônico”                 | Frequentemente associado a requisitos legais e de auditoria.                                     |
| <b>Não funcional</b> | “O sistema deve autenticar usuários em menos de 2 segundos em 95% dos acessos” | Atributo de desempenho; essencial para percepção de qualidade.                                   |
| <b>Não funcional</b> | “A plataforma deve estar em conformidade com a LGPD e regulamentos setoriais”  | Atributo legal e ético; traduz restrições do domínio em requisitos verificáveis.                 |
| <b>Não funcional</b> | “A interface deve ser acessível segundo diretrizes WCAG 2.1”                   | Atributo de usabilidade e inclusão; atende a princípios de valor e responsabilidade (Cap. 6).    |

Quadro 10 – Exemplos de requisitos funcionais e não funcionais.  
Fonte: elaborado pelo autor (2025).

A descoberta de requisitos não é automática: depende de mecanismos de interação e análise que funcionam como contextos de diálogo e negociação. Entre os mais comuns estão entrevistas, workshops, prototipação e mineração de dados, cada qual com potencialidades e desafios. Entrevistas permitem maior profundidade, mas podem refletir apenas a visão de stakeholders influentes. Workshops estimulam colaboração, mas tendem a favorecer perfis mais assertivos. Prototipação facilita a visualização de ideias e revela frustrações ocultas, embora possa gerar apego precoce a soluções imaturas. Já a análise de dados operacionais e de feedback digital amplia o olhar para além do discurso, identificando padrões de comportamento em escala (Ferrari et al., 2020; Kurtanović e Maalej, 2018).

Ao descobrir requisitos, é importante manter uma postura de interpretação crítica e diálogo contínuo. O objetivo não é produzir a maior quantidade possível de itens, mas ampliar progressivamente a compreensão sobre os comportamentos e condições de qualidade relevantes ao produto. A literatura alerta para o risco de conjuntos de requisitos excessivos, pouco relacionados ao valor ou construídos a partir de demandas aceitas sem questionamento suficiente (LAUESEN, 2002; PRESSMAN; MAXIM, 2021). Por isso, a descoberta permanece aberta a revisões e novas investigações conforme surgem evidências, dúvidas e mudanças de contexto.

Além disso, o contexto atual traz novas possibilidades e desafios. O uso de técnicas de Inteligência Artificial para apoiar a análise de requisitos não funcionais, por exemplo, permite identificar inconsistências linguísticas ou estimar impacto de desempenho em cenários simulados (ZHAO; XU; YANG, 2022). Da mesma forma, a análise de feedback em massa — seja em redes sociais, app stores ou sistemas de help desk — pode revelar frustrações coletivas que se transformam em requisitos relevantes, muitas vezes ignorados em processos tradicionais de entrevista.

Assim, descobrir requisitos funcionais e não funcionais não consiste apenas em classificá-los depois que aparecem. Trata-se de reconhecer, nas diferentes fontes e manifestações observadas, comportamentos, qualidades, restrições e condições relevantes que ainda não haviam sido explicitados ou suficientemente compreendidos. O resultado é um entendimento progressivamente

mais rico sobre aquilo que o produto precisa atender, sem pressupor que todos os requisitos já estejam estabilizados, detalhados ou definitivamente acordados nesse momento.

## 7.7 Papéis e competências na elicitação e descoberta

Se até aqui discutimos fontes, elementos, necessidades e mecanismos de descoberta, é indispensável reconhecer quem conduz esse processo e com quais capacidades. A atividade de elicitação e descoberta não é autossuficiente: ela depende da atuação de pessoas que desempenham papéis específicos e que, em conjunto, garantem a qualidade das descobertas realizadas. Entre esses papéis, destaca-se o do **engenheiro de software**, responsável por orquestrar a atividade, equilibrando aspectos técnicos, sociais e organizacionais.

O engenheiro de software atua como **mediador** da comunicação e da compreensão entre perspectivas que podem utilizar vocabulários, referências e critérios bastante diferentes. Gestores podem enfatizar custos e objetivos organizacionais; usuários podem concentrar-se nas condições concretas de uso; especialistas técnicos podem perceber limitações ou possibilidades que outros participantes desconhecem; e órgãos reguladores podem introduzir exigências que atravessam todas essas perspectivas. Trabalhar nesse ambiente exige conhecimento técnico e analítico, mas também competências interpessoais como escuta ativa, empatia, comunicação clara e capacidade de lidar construtivamente com divergências (WIEGERS; HOKANSON, 2023).

Essa mediação requer também interpretação crítica das manifestações recebidas. Aquilo que um stakeholder solicita pode expressar uma necessidade, uma preferência, uma hipótese de solução ou apenas uma maneira inicialmente disponível de descrever o problema. Em um projeto de governo eletrônico, por exemplo, usuários solicitaram que todos os formulários fossem convertidos em PDFs preenchíveis. A exploração da demanda mostrou que o valor buscado não estava especificamente no formato PDF, mas na necessidade mais ampla de mobilidade e acessibilidade multiplataforma. O aprofundamento da conversa permitiu compreender melhor a necessidade sem tomar a solução inicialmente proposta como requisito definitivo.

Outro aspecto fundamental é a **responsabilidade ética e legal**, o qual precisa estar atento a normas de acessibilidade, leis de proteção de dados (como a LGPD), regulamentos setoriais e princípios éticos do uso de IA e análise de dados. Ao elicitar requisitos, muitas vezes surgem dilemas: até que ponto explorar dados de usuários é aceitável? Como equilibrar inovação com respeito à privacidade? Essas questões exigem uma postura responsável, que conecta a prática da elicitação com os valores de **transparência, ética e responsabilidade** discutidos no Capítulo 6.

Entretanto, o engenheiro de software não atua sozinho. Outros papéis contribuem para o processo (Quadro 11), como os **stakeholders-chave** (que fornecem informações críticas), os **facilitadores** (responsáveis por dinâmicas colaborativas em workshops), e os **analistas de domínio** (que aprofundam aspectos regulatórios e técnicos). Em projetos de grande porte, pode haver ainda papéis especializados, como engenheiros de usabilidade ou especialistas em segurança, que auxiliam na descoberta de requisitos não funcionais. Essa diversidade de papéis amplia a riqueza de informações, mas também torna o processo mais complexo, exigindo coordenação.

| Papel                           | Principais responsabilidades                                      | Competências essenciais                            | Caso ilustrativo                                                                                                           |
|---------------------------------|-------------------------------------------------------------------|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Engenheiro de software</b>   | Conduzir a elicitação, estruturar requisitos, negociar conflitos  | Escuta ativa, empatia, análise crítica, negociação | Em projeto de governo eletrônico, reinterpretou pedido de PDFs em requisito de acessibilidade multiplataforma              |
| <b>Stakeholders-chave</b>       | Fornecer informações sobre necessidades, restrições e frustrações | Clareza, comprometimento, conhecimento de domínio  | Em hospital, médicos trouxeram frustração sobre atraso em dados clínicos, revelando requisito de atualização em tempo real |
| <b>Facilitador</b>              | Conduzir workshops e dinâmicas colaborativas                      | Comunicação, imparcialidade, mediação de grupos    | Em workshop de inovação, evitou que gestores dominassem discussões, permitindo voz a operadores                            |
| <b>Especialistas de domínio</b> | Interpretar restrições legais, técnicas e regulamentárias         | Expertise setorial, precisão analítica             | Em sistema financeiro, analista de compliance traduziu exigências do Banco Central em RNFs claros                          |

Quadro 11 – Papéis e competências na elicitação e descoberta.  
Fonte: elaborado pelo autor (2025).

A diversidade de participantes e fontes torna a elicitação e descoberta uma atividade coletiva que exige coordenação cuidadosa. O engenheiro de software precisa criar condições para que informações relevantes sejam trazidas à conversa, diferenças de linguagem não impeçam a compreensão e manifestações aparentemente simples possam ser investigadas em profundidade. Suas competências técnicas tornam-se mais efetivas quando combinadas com sensibilidade para o contexto humano, organizacional e social no qual os requisitos emergem.

Além das competências interpessoais, são importantes capacidades como identificar e selecionar fontes de informação, planejar e conduzir entrevistas e oficinas, realizar observações, formular perguntas produtivas, registrar evidências de maneira confiável e perceber lacunas ou contradições que indiquem necessidade de investigação adicional. Conhecimento de tecnologia, domínio, regulamentação e restrições do ambiente também ajuda a formular perguntas mais pertinentes e a reconhecer aspectos que poderiam passar despercebidos. O conjunto dessas competências reforça que elicitar e descobrir requisitos exige preparação, reflexão e responsabilidade, e não apenas habilidade para conduzir reuniões.

Além dos papéis apresentados no Quadro 11, é importante destacar o conjunto de **competências transversais** que sustentam a atividade de elicitação e descoberta. Essas competências não se restringem a um único papel, mas atravessam todos os envolvidos no processo, especialmente o engenheiro de software, que deve equilibrar habilidades sociais, interpessoais, organizacionais, técnicas e éticas. O Quadro 12 apresenta uma síntese dessas competências, funcionando como complemento ao Quadro 11 e reforçando a ideia de que a elicitação requer tanto sensibilidade humana quanto rigor técnico e responsabilidade.

| Dimensão                 | Competências exemplares                   | Relevância para a atividade                                          |
|--------------------------|-------------------------------------------|----------------------------------------------------------------------|
| <b>Humanas e sociais</b> | Escuta ativa, empatia, comunicação clara. | Favorecem a compreensão de diferentes perspectivas dos stakeholders. |

| Dimensão                              | Competências exemplares                                                                | Relevância para a atividade                                                          |
|---------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Interpessoais</b>                  | Negociação, mediação de conflitos, gestão de grupos.                                   | Viabilizam o equilíbrio entre interesses divergentes e reduzem resistências.         |
| <b>Organizacionais</b>                | Planejamento de sessões, definição de agenda, seleção de participantes.                | Garantem que a elicitação ocorra de forma estruturada e eficaz.                      |
| <b>Observacionais</b>                 | Shadowing, etnografia, análise de tarefas.                                             | Permitem revelar práticas reais e requisitos ocultos que não emergem em entrevistas. |
| <b>Interpretativas</b>                | Diferenciar desejos, necessidades e frustrações; identificar lacunas e contradições.   | Transformam manifestações dos stakeholders em insumos úteis para requisitos.         |
| <b>Técnicas de apoio à elicitação</b> | Conduzir entrevistas, organizar workshops, registrar informações de forma estruturada. | Asseguram consistência técnica e qualidade na coleta de informações.                 |
| <b>Tecnológicas</b>                   | Conhecimento de plataformas, restrições arquiteturais, custos e requisitos legais.     | Evitam propor soluções inviáveis ou desalinhadas ao contexto do projeto.             |
| <b>Ética e legal</b>                  | Conformidade com LGPD, acessibilidade, normas regulatórias.                            | Garante que a coleta e o uso de informações respeitem valores éticos e legais.       |

Quadro 12 – Exemplos de competências para a elicitação e descoberta de requisitos.  
Fonte: elaborado pelo autor (2025).

## 7.8 Dificuldades comuns na elicitação e descoberta

A atividade de elicitação e descoberta é marcada por uma série de dificuldades que não se limitam a aspectos técnicos. Grande parte dos desafios está enraizada no fator humano e nas interações sociais que permeiam o processo. Como discutido no Capítulo 6, princípios como colaboração, adaptabilidade e transparência são indispensáveis justamente porque a elicitação acontece em ambientes de incerteza, conflito de interesses e limitações de comunicação. Reconhecer essas dificuldades é o primeiro passo para mitigá-las.

### 7.8.1 Barreiras de comunicação, resistência à mudança e excesso ou falta de requisitos

Um dos problemas mais frequentes na elicitação é a barreira de comunicação. Stakeholders utilizam linguagens diferentes, recorrem a jargões de suas áreas ou interpretam termos de forma distinta. Essa diversidade de vocabulário pode gerar mal-entendidos que se propagam para o restante do projeto. Young (2004) destaca que a ausência de uma linguagem comum está entre as maiores fontes de requisitos ambíguos.

**Caso ilustrativo:** em um projeto de integração de dados de saúde, médicos usavam o termo “prontuário” para se referir a registros clínicos digitais, enquanto a equipe de TI entendia o mesmo termo como banco de dados estruturado. A falta de clareza levou a requisitos conflitantes até que a equipe elaborou um glossário compartilhado.

Outra dificuldade recorrente é a **resistência à mudança**. Stakeholders podem sentir-se ameaçados por novas soluções, temendo perda de autonomia ou de status. Como descreve Lauesen (2002), essa resistência pode levar à omissão de informações relevantes ou até à sabotagem velada.

**Caso ilustrativo:** durante a implantação de um novo sistema de gestão acadêmica, parte da equipe administrativa resistia a detalhar processos manuais, pois temia que a automação reduzisse sua importância. Somente após reuniões individuais e demonstrações de benefícios pessoais o grupo passou a colaborar.

Também é comum enfrentar problemas de **excesso ou falta de requisitos**. A sobrecarga ocorre quando stakeholders trazem listas extensas e desorganizadas de desejos, dificultando a priorização. A escassez aparece quando fornecem informações vagas ou incompletas, impedindo que o escopo seja bem definido. Pressman e Maxim (2021) alerta que ambos os extremos comprometem a clareza e a viabilidade do projeto.

**Caso ilustrativo:** em um projeto de comércio eletrônico, gestores listaram mais de cem “requisitos”, desde integrações complexas até preferências cosméticas de interface. Ao mesmo tempo, usuários finais não souberam definir requisitos mínimos de navegação. O resultado inicial foi um escopo irrealista, ajustado apenas após oficinas de priorização.

### 7.8.2 Distorções sociais e políticas no processo de elicitação

A elicitação não ocorre em um vácuo técnico, mas em ambientes organizacionais permeados por relações de poder, disputas de recursos e agendas ocultas. Goguen (1994) descreve os requisitos como construções sociais, frequentemente moldadas por pressões políticas. Assim, determinados grupos podem impor prioridades que não refletem as necessidades coletivas.

**Caso ilustrativo:** em um projeto de modernização de sistemas de transporte urbano, gestores municipais priorizaram requisitos ligados à visibilidade política — como relatórios para divulgação de resultados — em detrimento de funcionalidades críticas apontadas por operadores, como a integração entre rotas. O resultado foi um sistema alinhado à agenda política, mas insuficiente para resolver os problemas operacionais do transporte.

Essas distorções sociais e políticas são particularmente desafiadoras porque não podem ser resolvidas apenas por técnicas de modelagem. Exigem sensibilidade, ética e habilidades de negociação para equilibrar interesses legítimos e questionar imposições que comprometem o valor da solução.

### 7.8.3 Reflexões

As dificuldades discutidas nesta seção não são meros incidentes operacionais, mas expressam a natureza profundamente social, interpretativa e política da Engenharia de Requisitos. Pesquisas recentes confirmam que problemas clássicos como falhas de comunicação, omissão de stakeholders e priorização inadequada permanecem entre os maiores obstáculos (WAGNER et al., 2019).

Autores clássicos já apontavam nessa direção: Goguen (1994) destacou que requisitos não são verdades absolutas, mas construções sociais moldadas por interações e relações de poder. Esse diagnóstico continua válido à luz de estudos mais recentes, como os de Horkoff et al. (2019), que mostram que a ausência de objetivos explicitados e compartilhados leva a requisitos mal interpretados e inconsistentes.

Pressman e Maxim (2021) reforçam que a validação contínua é essencial para reduzir ambiguidades, enquanto Ferrari et al. (2020) demonstram empiricamente que mesmo com ferramentas baseadas em NLP, a qualidade dos requisitos ainda sofre com problemas de clareza e consistência. Lauesen (2002) já alertava para o excesso de requisitos não priorizados, e pesquisas atuais

em *crowdsourcing*<sup>18</sup> (HOSSEINI et al., 2022) confirmam que, mesmo em abordagens mais flexíveis, o ruído e a falta de critérios de priorização continuam comprometendo resultados.

Young (2004) enfatizou a comunicação como raiz de falhas, e estudos contemporâneos de Daneva et al. (2021) em ambientes ágeis mostram que, embora haja maior iteração, problemas de coordenação entre equipes e definição de prioridades permanecem centrais. Leffingwell (2011) reforça essa perspectiva ao destacar que a colaboração em larga escala exige práticas específicas de alinhamento entre múltiplas equipes. Essa necessidade de coordenação torna-se ainda mais evidente em abordagens atuais de desenvolvimento contínuo, nas quais a elicitación de requisitos precisa acompanhar ciclos curtos e interdependentes, evitando que falhas de comunicação ou de priorização se multipliquem em grande escala.

Além disso, a incorporação de Inteligência Artificial como apoio à elicitación abre novas oportunidades, mas também traz riscos. Zhao, Xu e Yang (2022) observam que vieses algorítmicos e interpretações equivocadas podem acentuar problemas já existentes, em vez de eliminá-los.

Em síntese, a literatura — clássica e recente — converge para um ponto central: a elicitación e descoberta é essencialmente uma atividade humana e social, sujeita a ambiguidades, interesses conflitantes e limitações comunicacionais. Além de dominar técnicas formais, o engenheiro de software precisa cultivar competências interpessoais, críticas e éticas para lidar com barreiras humanas e políticas.

Contudo, é igualmente necessário reconhecer seu caráter técnico: sem a capacidade de identificar fontes de informação, selecioná-las e utilizá-las adequadamente, formular perguntas produtivas, reconhecer informações implícitas e registrar evidências relevantes, até mesmo interações bem conduzidas podem resultar em um entendimento frágil ou insuficiente sobre o produto. Esses desafios não desaparecem em abordagens dirigidas a plano, ágeis ou híbridas: apenas assumem formas distintas em cada contexto. Reconhecer suas causas e refletir criticamente sobre elas é o primeiro passo para mitigá-las.

## CAPÍTULO 7: PONTOS-CHAVE

- Elicitación e descoberta é a **atividade de investigação** por meio da qual se amplia o entendimento sobre o problema, o domínio, os stakeholders e aquilo que o produto precisa atender.
- A atividade **não se resume à coleta de informações**. Necessidades, desejos, frustrações, regras, restrições e requisitos podem estar explícitos, implícitos ou distribuídos entre diferentes fontes e precisam ser investigados criticamente.
- Stakeholders, documentos, sistemas existentes, observação do contexto de uso, dados, registros de interações e recursos tecnológicos constituem **fontes complementares de informação**. A diversidade de fontes reduz a dependência de uma única perspectiva.

---

<sup>18</sup> Para Hosseini et al. (2022), *crowdsourcing* em ER não é apenas “ouvir muita gente”, mas um processo estruturado de envolver multidões via tecnologias digitais para enriquecer as atividades de requisitos, com ganhos de diversidade e escala, mas que exige mecanismos para lidar com o excesso e a qualidade variável das informações.

- Compreender o domínio, delimitar o problema e reconhecer objetivos e valor esperado orientam a descoberta, sem pressupor que todas essas questões estejam completamente estabilizadas desde o início.
- Desejos, necessidades e frustrações não são equivalentes e, **explorar suas diferenças** ajuda a evitar que preferências ou soluções inicialmente sugeridas sejam assumidas automaticamente como requisitos.
- Requisitos funcionais e não funcionais podem emergir em **diferentes níveis de abstração** e tornar-se mais compreendidos à medida que novas evidências e interações ampliam o conhecimento sobre o produto.
- Entrevistas, workshops, observação, prototipação, análise documental, mineração de dados e outras práticas podem **apoiar a descoberta**, devendo ser selecionadas conforme o contexto e as fontes disponíveis.
- A elicitação e descoberta é uma atividade profundamente **comunicacional e sociotécnica**. Sua qualidade depende tanto de competências técnicas quanto de escuta, interpretação, empatia, capacidade investigativa e atenção às diferentes vozes envolvidas.
- **Novas descobertas** podem ocorrer em qualquer momento do ciclo de vida do produto, a partir de feedback, mudanças de contexto, dúvidas e evidências de uso podendo revelar necessidades ou aspectos de requisitos ainda não suficientemente compreendidos.

# Capítulo 8

## 8 Análise e Consenso de Requisitos

### 8.1 Introdução

Os requisitos coletados apresentam-se, muitas vezes, como um conjunto heterogêneo de informações — algumas explícitas, outras implícitas; algumas consistentes, outras contraditórias. Nesse estágio inicial, podem ainda estar cheios de ambiguidades, lacunas e diferentes interpretações dos stakeholders. É nesse ponto que a atividade de **análise e consenso** assume um papel importante na Engenharia de Requisitos: transformar esse material bruto em uma base estruturada, compreensível e acordada entre as partes envolvidas.

A **análise** busca refinar tecnicamente os requisitos, eliminando inconsistências, identificando dependências e garantindo que estejam claros, completos e viáveis dentro das restrições do projeto. Já o **consenso** enfatiza a dimensão social do processo, envolvendo negociação, priorização e alinhamento de expectativas entre stakeholders que frequentemente possuem objetivos, interesses e visões divergentes.

Neste livro, o termo **consenso** não deve ser entendido necessariamente, como unanimidade, mas como a construção de um acordo viável, rastreável e aceito pelos diferentes grupos de stakeholders. A ideia de ‘acordo viável’ concretiza o **Valor Respeito** (nem todos terão o mesmo objetivo) e o **Valor Compromisso** (decidir e sustentar o decidido), reduzindo o risco de consensos performáticos. O papel do engenheiro de software é facilitar esse processo, equilibrando vozes e registrando compromissos de modo transparente.

Questões de análise e consenso podem surgir sempre que o entendimento disponível exige comparar interpretações, examinar alternativas, esclarecer implicações ou construir acordos. Isso ocorre tanto diante de informações recém-descobertas quanto quando uma decisão, mudança ou evidência coloca em questão entendimentos anteriormente aceitos. A atividade assume, portanto, intensidade variável ao longo do desenvolvimento, acompanhando os pontos em que escolhas precisam ser examinadas ou negociadas.

A análise e consenso envolve decisões que afetam diretamente o entendimento compartilhado sobre os requisitos, suas prioridades e o escopo do produto. Para delimitar essa responsabilidade e distingui-la das demais atividades da Engenharia de Requisitos, a Figura 45 apresenta seu enquadramento.





Figura 45 – Enquadramento da atividade de Análise e Consenso.  
Fonte: elaborado pelo autor (2026).

### Box 8.1-1 – Mecanismos de construção de significado

Este é o ponto em que a ER torna explícito o **Princípio 4** (Entendimento compartilhado): análise e consenso não são ‘revisão textual’, mas mecanismos de construção de significado comum — sustentados por comunicação e por ciclos de feedback.

## 8.2 Natureza dos Requisitos Brutos

Os insumos iniciais da Engenharia de Requisitos raramente se apresentam em forma clara e estruturada. Ao contrário, surgem como enunciados brutos provenientes de conversas, entrevistas, observações, protótipos, documentos de negócio ou mesmo inferências realizadas pela equipe. Esses enunciados refletem problemas, necessidades e frustrações percebidas pelos stakeholders, mas ainda carecem de precisão, completude e alinhamento (SOMMERVILLE, 2019; WIEGERS; BEATTY, 2013).

É fundamental compreender que a noção de “completude” ou “incompletude” de um requisito depende do nível de abstração exigido pelo momento do projeto. Em fases iniciais, uma simples lista de requisitos com nomes ou títulos pode ser suficiente e, nesse contexto, considerada completa. Em fases posteriores, entretanto, essa mesma lista pode ser insuficiente, por não conter condições, restrições ou critérios de aceitação necessários ao design, construção ou testes (POHL, 2010; PRESSMAN; MAXIM, 2021).

Entre as características mais comuns dos requisitos brutos estão (em complemento, ver Quadro 13):

- **Incompletude:** falta de informações necessárias a um requisito em relação ao momento em que o projeto se encontra, podendo ser, condições, restrições, critérios de aceitação.

*Exemplo:*

- “O sistema deve permitir que o usuário realize pagamento online.”
- Se o projeto estiver em tempo de desenvolvimento, pode ser considerado incompleto porque não especifica quais métodos de pagamento (cartão de crédito, débito, Pix, boleto), nem trata de segurança ou confirmação da transação.

- **Ambiguidade:** são formulados em linguagem natural vaga ou vários sentidos, dependendo do contexto, permitindo múltiplas interpretações (LAUESEN, 2002; SOMMERVILLE, 2019).

*Exemplo:*

- “O sistema deve ser rápido.”
- “Rápido” pode significar tempo de resposta inferior a 1 segundo processamento de até 1.000 requisições/minuto ou até mesmo velocidade de navegação. Cada stakeholder pode interpretar de forma diferente.

- **Contradições:** diferentes fontes de informação podem dar origem a expectativas e necessidades opostas, seja em termos de prioridade, de funcionalidades ou de restrições (POHL, 2010; LAPLANTE; KASSAB, 2022).

*Exemplo:*

- Requisito A: “O sistema deve permitir acesso de qualquer usuário sem autenticação para facilitar o uso.”
- Requisito B: “O sistema deve exigir autenticação multifator para todo acesso.”
- Contradição entre usabilidade e segurança, exigindo negociação e esclarecimento.

- **Redundância:** a mesma necessidade pode ser relatada de formas distintas, dificultando a consolidação (WIEGERS; BEATTY, 2013).

*Exemplo:*

- Requisito A: “O sistema deve permitir que o usuário recupere a senha via e-mail.”
- Requisito B: “O sistema deve oferecer funcionalidade de redefinição de senha por meio de mensagem enviada ao e-mail do usuário.”
- Ambos descrevem a mesma funcionalidade, mas de forma redundante, o que pode gerar duplicidade no backlog ou confusão durante testes.

- **Implícitos:** certos requisitos não são verbalizados diretamente, mas emergem da observação de práticas, de dados ou de regras não ditas da organização (POHL, 2010; SOMMERVILLE, 2019).

*Exemplo:*

- “O sistema deve permitir a emissão de notas fiscais.”
- Implícito que o sistema deve estar integrado à legislação vigente (ex: regras da Receita Federal, validade do XML, assinatura digital). Esse aspecto não está declarado, mas é obrigatório para que o requisito faça sentido.
- Em um sistema de apoio a enfermeiros, nenhum requisito inicial mencionava a necessidade de registrar informações em plantões noturnos com baixa

iluminação. Essa condição só emergiu durante a observação direta do trabalho, evidenciando um requisito implícito de usabilidade (interface com alto contraste).

Essas características não devem ser vistas como falhas da elicitação, mas como traços naturais do processo de descoberta. Elas revelam a complexidade das interações humanas e organizacionais, bem como a natureza multifacetada dos problemas de negócio e sociais (LAUESEN, 2002; WAGNER et al., 2019). O papel da análise e do consenso, portanto, é lidar com esse material heterogêneo, buscando clareza técnica e alinhamento entre as partes.

Além disso, é preciso reconhecer que a condição de “bruto” não se limita ao início do projeto. Mesmo em fases mais avançadas, novos requisitos ou mudanças de contexto podem reintroduzir ambiguidades, contradições e lacunas. Isso ocorre em ambientes variados — organizacionais, sociais, governamentais, comunitários — nos quais a Engenharia de Requisitos é aplicada. Por isso, análise e consenso não apenas refinam requisitos iniciais, mas acompanham continuamente sua evolução, ajustando o grau de detalhe necessário para cada momento (SOMMERVILLE, 2019; PRESSMAN; MAXIM, 2021).

O Quadro 13 serve como ‘ponte’ entre princípios e prática: cada estratégia busca maximizar **Entendimento compartilhado (Princípio 4)** com **simplicidade** (resolver o suficiente para o próximo passo, sem formalismo excessivo).

| Característica dos requisitos brutos        | Descrição                                                                                                                                                                | Possíveis encaminhamentos diante da característica                                                                                                                                  |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Incompletude</b>                         | Um requisito pode ser suficiente em uma fase (ex: lista inicial de necessidades), mas insuficiente em fases posteriores que demandam condições, restrições ou critérios. | Evolução progressiva do detalhamento; uso de cenários e casos de uso para complementar; iteração constante com stakeholders.                                                        |
| <b>Ambiguidade e múltiplos significados</b> | Linguagem natural vaga ou com múltiplos sentidos, levando a interpretações distintas entre pessoas ou grupos.                                                            | Reformulação em linguagem controlada; construção de protótipos ou modelos visuais; validação colaborativa em oficinas.                                                              |
| <b>Contradições</b>                         | Expectativas divergentes entre stakeholders — em organizações (ex: áreas de negócio vs. TI) ou em comunidades (ex: grupos sociais com interesses distintos).             | Técnicas de negociação e mediação; priorização colaborativa (MoSCoW, AHP); análise de dependências e trade-offs.                                                                    |
| <b>Redundância</b>                          | Necessidades semelhantes são expressas de formas diferentes, dificultando a consolidação.                                                                                | Comparação e consolidação de itens semelhantes; padronização de termos; revisão colaborativa. Relações de rastreabilidade existentes podem apoiar a identificação de sobreposições. |

| Característica dos requisitos brutos | Descrição                                                                                                                                        | Possíveis encaminhamentos diante da característica                                                                                                                                                                                             |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Implícitos</b>                    | Necessidades não ditas explicitamente, mas perceptíveis em práticas, dados ou normas culturais - tanto em organizações quanto em grupos sociais. | Retomar a investigação quando necessário, utilizando observação, análise de dados e registros, <i>shadowing</i> ou <i>storytelling</i> para ampliar o entendimento; discutir posteriormente os efeitos dessas descobertas sobre os requisitos. |

Quadro 13 –Características dos requisitos brutos e estratégias de análise e consenso em diferentes contextos  
Fonte: elaborado pelo autor (2025).

### 8.3 Objetivos da Análise e Consenso

A Engenharia de Requisitos (ER) serve como alicerce para o desenvolvimento de software, respondendo a questões fundamentais como “O que estamos construindo?”, “Por que isso é valioso?” e “Como saberemos que cumprimos nossos objetivos?”. Nesse contexto, a análise e consenso contribui para aprofundar e refinar o entendimento dos requisitos, examinando seu conteúdo, relações e implicações em diferentes níveis de abstração (DICK; HULL; JACKSON, 2017). Esse trabalho ajuda a tornar explícitas questões ainda insuficientemente compreendidas e a sustentar decisões e acordos sobre aquilo que o produto precisa atender. Quando realizado de forma inadequada, problemas de requisitos podem permanecer ocultos e contribuir significativamente para dificuldades posteriores no desenvolvimento (CROWDER; HOFF, 2022).

#### Box 8.3-1 – Problema, requisitos e solução

Este ponto concretiza o **Princípio 6 (Problema, requisitos e solução)**: durante a análise e o consenso, cada refinamento precisa preservar (ou tornar explícito) o vínculo entre o **problema** que motiva o requisito, o **requisito** como compromisso verificável e a **solução** como hipótese de atendimento. Quando esse encadeamento se rompe, surgem requisitos “corretos no texto”, mas desconectados do propósito, ou soluções implementadas sem aderência às necessidades reais.

Os principais objetivos da Análise e Consenso de Requisitos são:

#### 8.3.1 Remover Ambiguidades e Preencher Lacunas

Requisitos ainda em análise, especialmente quando formulados em linguagem natural, podem conter termos vagos ou admitir diferentes interpretações conforme o contexto (LAUESEN, 2002; SOMMERVILLE, 2019). Também podem apresentar lacunas relacionadas a condições, restrições, exceções, regras de negócio ou critérios ainda não suficientemente compreendidos (POHL, 2010; PRESSMAN; MAXIM, 2021). O trabalho analítico consiste em tornar essas questões explícitas, investigar o significado pretendido e esclarecer o que ainda precisa ser compreendido ou decidido. Exemplos, cenários, reformulações, modelos ou protótipos podem apoiar essa discussão quando ajudam a examinar o conteúdo e suas implicações.

*Exemplo:* Em um sistema de gestão hospitalar, um requisito foi registrado como: “O sistema deve oferecer relatórios rápidos para os médicos”. Na análise, o termo “*rápidos*” gerou

interpretações divergentes: para alguns médicos significava em menos de 5 segundos, para outros no mesmo dia. Após discussão, definiu-se o critério mensurável: “O sistema deve gerar relatórios clínicos em até 5 segundos após a solicitação”. Com a discussão, os envolvidos explicitaram o significado pretendido para “rápidos” e acordaram o limite de cinco segundos, eliminando a divergência de interpretação.

### 8.3.2 Resolver Conflitos e Inconsistências

Uma característica comum dos requisitos ainda em análise é a presença de contradições e inconsistências. Diferentes fontes de informação ou stakeholders podem expressar expectativas, necessidades ou restrições incompatíveis, inclusive em relação a funcionalidades e prioridades (POHL, 2010; LAPLANTE; KASSAB, 2022). Esses conflitos podem ocorrer entre áreas de uma mesma organização, entre equipes técnicas e de negócio ou entre diferentes grupos de stakeholders. Resolver essas divergências exige explicitar as perspectivas envolvidas, examinar as razões que as sustentam e construir acordos que permitam orientar o desenvolvimento do produto. Técnicas de negociação e mediação, priorização colaborativa, análise de dependências e discussão de *trade-offs* podem apoiar a construção desses acordos (ALEXANDER; BEUS-DUKIC, 2009; COHN, 2006).

*Exemplo:* No projeto de um aplicativo bancário, o setor de marketing exigia “login simplificado por PIN de 4 dígitos” para aumentar a usabilidade, enquanto o setor de segurança exigia “autenticação multifator em todas as sessões”. O conflito foi tratado em reunião de análise. O consenso estabelecido foi: login simplificado permitido para operações de baixo risco (consulta de saldo) e autenticação multifator obrigatória para operações de alto risco (transferências e pagamentos). Assim, foi produzido um equilíbrio entre segurança e usabilidade.

*Exemplo:* Em um sistema de e-commerce, o time de UX queria permitir “compra com um clique” para acelerar a experiência. O time jurídico exigia “confirmação de termos de uso em cada compra”. O consenso foi implementar “Compra com um clique” apenas após a aceitação dos termos de uso na primeira compra, armazenando essa informação para futuras transações. Com isso, foi resolvido um problema de conflito entre questões legais e estratégias de uso, entre as áreas. Pode ser necessário que esse tipo de decisão deva ser registrada explicitamente, vinculando a stakeholders envolvidos e às hipóteses de risco assumidas, de modo a preservar a rastreabilidade da escolha.

### 8.3.3 Estabelecer Entendimento Compartilhado

A Engenharia de Requisitos envolve múltiplas pessoas e fontes de informação, tornando essencial a busca por um entendimento comum e compartilhado sobre o problema, as necessidades e os requisitos. Este entendimento pode ser explícito, através de requisitos detalhadamente declarados e escritos (comum em processos orientados a planos), ou implícito, alcançado por meio de conversas *face a face* (comum em processos ágeis). Promover essa visão comum é crucial para alinhar as expectativas de todos os envolvidos, incluindo clientes, usuários e a equipe técnica, e para evitar mal-entendidos que possam surgir de vocabulários incompatíveis ou da ausência de objetivos explicitados. Para auxiliar na criação desse entendimento, são utilizadas práticas como a construção de glossários e dicionários de termos (ALEXANDER; BEUS-DUKIC, 2009), o uso

de protótipos, a referência a sistemas existentes e a realização de ciclos curtos de *feedback*. Além disso, workshops de requisitos são considerados uma ferramenta eficaz para obter decisões e acordos por parte de grupos de *stakeholders* (GOTTESDIENER, 2002).

*Exemplo:* No desenvolvimento de um sistema de logística, o requisito “otimizar rotas de entrega” foi entendido de formas diferentes: para a equipe de operações, significava reduzir o tempo médio de entrega e, para a diretoria financeira, significava reduzir o custo do combustível. Na análise, decidiu-se estabelecer duas métricas como requisitos complementares: redução do tempo médio de entrega em 15% e redução do custo médio de combustível por entrega em 10%. Com isso, foi garantido um entendimento comum entre stakeholders com interesses distintos.

### 8.3.4 Definir Prioridades e Escopo Acordado

A definição de prioridades e de um escopo acordado é parte essencial da análise, pois permite alinhar expectativas e dar transparência às escolhas. O processo de priorização deve ser conduzido de forma explícita e participativa, tornando claro para todos os stakeholders quais requisitos são essenciais, quais podem ser postergados e quais serão descartados. As técnicas específicas que apoiam esse processo serão detalhadas na seção 8.4.

Priorizar não significa apenas ordenar uma lista pronta, envolve explicitar ou acordar critérios de decisão, reunir as avaliações necessárias, aplicar esses critérios aos requisitos considerados e discutir os resultados com os participantes pertinentes. Valor, urgência, risco, dependências, custo ou esforço podem compor essa avaliação, conforme o contexto (WIEGERS; BEATTY, 2013; LEFFINGWELL, 2011). O resultado deve ser compreensível e justificável, evitando que a prioridade represente apenas uma preferência momentânea.

*Exemplo:* No backlog de um aplicativo de transporte urbano, havia 30 requisitos. O cliente queria dar prioridade à “integração com cartão de crédito internacional”. O time técnico alertou que havia dependências não resolvidas. Usando a técnica MoSCoW, decidiu-se priorizar da seguinte maneira: (Must-have) cadastro de usuário, geolocalização e pagamento nacional, (Should-have) integração internacional (a ser entregue na 2ª versão).

### 8.3.5 Analisar Viabilidade, Dependências, Riscos e Implicações

A análise de requisitos precisa considerar se o entendimento construído é compatível com as restrições e condições do contexto, incluindo aspectos de viabilidade técnica, operacional, econômica ou normativa quando pertinentes, bem como dependências, riscos, premissas e efeitos que determinados requisitos ou alternativas podem produzir sobre o restante do produto (POHL, 2010; WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

Esse exame não pressupõe que as consequências sejam conhecidas antecipadamente, mas busca tornar visíveis fatores que podem limitar escolhas, exigir compromissos ou demandar investigação adicional. Uma dependência pode alterar uma decisão de prioridade; um risco pode justificar maior aprofundamento; uma restrição normativa pode eliminar alternativas; e uma incerteza técnica pode exigir experimentação antes de um acordo mais firme.

A profundidade dessa análise deve ser proporcional ao contexto. Produtos críticos, regulados ou com alto custo de falha podem exigir avaliações mais sistemáticas de risco, dependências e

viabilidade, enquanto produtos exploratórios ou de menor criticidade podem trabalhar com análises mais leves e progressivas. Essa calibragem retoma o princípio de **adaptabilidade contextual** discutido nos capítulos iniciais: práticas e níveis de rigor devem ser ajustados às características do produto, do domínio, do projeto e da organização, evitando tanto análises insuficientes quanto formalização desnecessária (BOEHM; TURNER, 2003; KUHRMANN et al., 2017; DINGSØYR et al., 2018).

Viabilidade, dependências e riscos, portanto, não devem ser tratados como propriedades isoladas de uma frase de requisito, pois emergem das relações entre requisitos, contexto, tecnologia, organização e decisões já assumidas. Considerá-los em conjunto ajuda a evitar acordos que pareçam adequados localmente, mas que se mostrem incompatíveis com o produto como um todo.

## 8.4 Técnicas de Apoio à Análise e ao Consenso

A análise e o consenso não se resumem à interpretação individual dos requisitos, mas dependem de práticas que facilitem a colaboração, a tomada de decisão e a construção de entendimentos compartilhados. Diversas técnicas podem apoiar essa atividade, oferecendo mecanismos estruturados para lidar com ambiguidades, conflitos e prioridades.

A seguir, destacam-se algumas delas, acompanhadas de exemplos que ilustram sua aplicação prática:

### 8.4.1 Workshops de Requisitos

Workshops de requisitos são encontros colaborativos que reúnem stakeholders em sessões facilitadas, com a finalidade de identificar necessidades, resolver ambiguidades e alinhar entendimentos de forma intensiva. Quando bem conduzidos, favorecem a cocriação, reduzem o tempo de ciclos longos de entrevistas individuais e expõem divergências de maneira transparente. A literatura destaca sua efetividade na construção de consenso, pois permitem integrar perspectivas de negócio, técnicas e de uso real em um ambiente de colaboração controlada (DICK; HULL; JACKSON, 2017; FERNANDES; MACHADO, 2015).

*Exemplo:* Em um projeto de sistema de transporte urbano por meio de um encontro presencial de quatro horas, que reuniu gestores de frota, motoristas e representantes dos usuários. Cada grupo respondeu individualmente, em cartões, à pergunta “o que significa para você rotas otimizadas?”. As respostas foram coladas em um quadro e agrupadas por afinidade: os gestores destacaram redução de custos de combustível e diminuição do desgaste da frota, enquanto os usuários enfatizaram menor tempo de deslocamento e maior previsibilidade dos horários. A discussão foi conduzida por um facilitador, que ajudou a traduzir essas expectativas em métricas objetivas, como “redução de custos de combustível em pelo menos 10%” e “tempo médio de viagem 15% menor em relação ao atual”. O resultado final do workshop foi um conjunto de requisitos complementares registrados no backlog, com rastreabilidade explícita para os grupos que os originaram.

#### 8.4.2 Técnicas de Priorização (MoSCoW e Matriz Avaliação Técnica × Valor de Negócio)

A priorização busca tornar explícitos os critérios e os *trade-offs* envolvidos nas escolhas, permitindo compreender por que determinados requisitos recebem maior precedência que outros. As técnicas funcionam como apoio a esse processo ao estruturar avaliações e tornar seus resultados discutíveis entre os participantes. Entre as técnicas conhecidas estão:

- **MoSCoW**: classifica requisitos em quatro grupos — *Must have* (obrigatórios), *Should have* (importantes, mas não críticos), *Could have* (desejáveis se houver tempo/recursos) e *Won't have for now* (fora do escopo atual). Essa categorização facilita a comunicação entre áreas de negócio e equipe técnica, mostrando de forma clara o que é essencial e o que pode ser negociado (LEFFINGWELL, 2011; FERNANDES; MACHADO, 2015).
- **Matriz de Avaliação Técnica × Valor de Negócio**: posiciona requisitos em uma grade que cruza valor percebido pelo cliente e complexidade/esforço técnico estimado pela equipe. Ao combinar essas duas perspectivas, fornece uma visão mais realista das escolhas, permitindo identificar entregas de alto valor e baixa complexidade (candidatos a entregas rápidas), além de itens estratégicos de alto valor e alta complexidade, que podem ser planejados para ciclos posteriores (LEFFINGWELL, 2011; LAPLANTE, 2017).

*Exemplo:* Em um e-commerce em fase inicial, a equipe de produto aplicou o método MoSCoW em uma reunião com stakeholders. A discussão começou com uma lista de 20 requisitos. Usando cartões visuais, cada requisito foi classificado coletivamente: o “pagamento nacional integrado” foi definido como Must-have (obrigatório para lançamento), enquanto a “integração com carteiras internacionais” ficou como Should-have (para versões futuras). O uso da técnica garantiu foco no essencial e transparência na decisão de escopo.

*Exemplo:* Em um projeto de aplicativo de serviços públicos digitais, a equipe plotou os requisitos em uma matriz 2×2, considerando esforço técnico estimado (baixo ou alto) e valor percebido para o cidadão (baixo ou alto). Essa visualização permitiu identificar funcionalidades de alto valor e baixa complexidade como candidatas ideais para a primeira entrega — por exemplo, consulta rápida de status de processos. Já itens de alto valor e alta complexidade, como integração com sistemas legados, foram mantidos no backlog, planejados para ciclos posteriores. A técnica forneceu uma visão ampla e mais realista do que era necessário e possível fazer, equilibrando expectativa de impacto com viabilidade de entrega.

#### 8.4.3 Negociação

A negociação é um processo estruturado de mediação de interesses, necessário quando requisitos apresentam restrições conflitantes ou prioridades divergentes. Não busca eliminar diferenças, mas transformá-las em soluções de compromisso aceitáveis para todos os envolvidos. A negociação é reconhecida como prática essencial na Engenharia de Requisitos, pois garante equilíbrio entre atores com diferentes níveis de poder e promove decisões rastreáveis que sustentam a evolução do produto (LAPLANTE, 2017; DICK; HULL; JACKSON, 2017).

*Exemplo:* No desenvolvimento de um aplicativo bancário, houve conflito entre marketing (que defendia login simplificado por PIN de 4 dígitos) e segurança (que exigia autenticação



multifator). A equipe de requisitos conduziu a negociação em três etapas: (i) levantamento dos riscos de cada proposta, (ii) apresentação de alternativas híbridas e (iii) votação entre os representantes. O acordo estabelecido foi login simplificado apenas para consultas de saldo e autenticação multifator obrigatória para transações financeiras, equilibrando conveniência e segurança.

#### 8.4.4 Resolução de Conflitos

Quando divergências não são superadas apenas pelo diálogo, recorrem-se a técnicas de resolução de conflitos, como mediação, votação ou critérios objetivos predefinidos. O objetivo é assegurar que todas as vozes sejam ouvidas e que a decisão seja legítima, documentada e sustentável. Em contextos regulados ou críticos, recomenda-se o uso de critérios de precedência, em que requisitos legais ou de segurança prevalecem sobre conveniências de negócio, reduzindo impasses e facilitando a tomada de decisão (CROWDER; HOFF, 2022; DICK; HULL; JACKSON, 2017).

*Exemplo:* Em um projeto de ERP, gestores de finanças e produção discordavam sobre qual módulo deveria ser priorizado. Utilizando a técnica *round robin*<sup>19</sup>, cada stakeholder expôs sua demanda por tempo igual. Em seguida, foi aplicada uma matriz de critérios objetivos (impacto legal, valor de negócio e custo). O resultado mostrou que os relatórios financeiros eram críticos para cumprir obrigações legais, e a equipe decidiu priorizá-los. Os relatórios de produção foram planejados como segunda entrega, com registro formal do acordo.

#### 8.4.5 Prompt IA

O uso de inteligência artificial no apoio à análise de requisitos é uma prática emergente. Modelos de linguagem podem auxiliar na identificação de ambiguidades, sugerir alternativas mais claras ou propor métricas verificáveis a partir de enunciados vagos. Essa prática tem sido explorada como forma de acelerar o refinamento e apoiar a criatividade no processo, mas sempre com supervisão humana para validar resultados e evitar vieses ou riscos de confidencialidade. Na literatura recente, a integração de IA em atividades de requisitos aparece como tendência promissora, ainda em consolidação, que deve ser tratada como apoio complementar às técnicas tradicionais (CROWDER; HOFF, 2022; CPRE Handbook, 2022).

*Exemplo:* Em um sistema educacional, surgiu o requisito: “O sistema deve ser intuitivo”. Ao aplicar um prompt em uma ferramenta de IA (prompt: *revise o requisito a seguir, propondo versões alternativas que sejam mensuráveis e testáveis. Requisito: “O sistema deve ser intuitivo”.*), foram geradas alternativas de refinamento, como: “O usuário deve conseguir concluir o cadastro em até 3 minutos sem consultar ajuda” e “A taxa de erros em formulários deve ser inferior a 5% em testes de usabilidade”. Essas propostas foram discutidas com stakeholders e serviram de base para definir métricas objetivas, que substituíram o requisito vago original. Cabe destacar que, o uso de IA deve ser feito com cautela, sendo necessário considerar riscos de vies, confidencialidade das informações e a necessidade de validação humana. A IA pode apoiar a análise, mas não substitui a decisão coletiva nem a responsabilidade do engenheiro de software.

---

<sup>19</sup> É uma técnica de facilitação usada em reuniões colaborativas para garantir que todos os participantes tenham voz de forma equilibrada, onde é atribuído a cada participante o mesmo tempo de fala.

O Quadro 14 sintetiza as cinco técnicas apresentadas, organizando-as de acordo com seus objetivos, vantagens, limitações e situações em que podem ser aplicadas. Mais do que uma simples lista, trata-se de um conjunto de instrumentos que, quando escolhidos conforme o contexto do projeto, fortalecem a qualidade da análise e aumentam as chances de alcançar consensos sustentáveis entre os stakeholders.

| Técnica                                                                               | Objetivo                                                                 | Vantagens                                                               | Limitações                                                                  | Quando Usar                                                                 |
|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Workshops de Requisitos</b>                                                        | Promover discussão colaborativa entre múltiplos stakeholders.            | Gera entendimento compartilhado; acelera identificação de conflitos.    | Exige facilitador experiente; risco de dominância de vozes fortes.          | Projetos com muitos stakeholders e diversidade de visões.                   |
| <b>Técnicas de Priorização (MoSCoW e Matriz Avaliação Técnica × Valor de Negócio)</b> | Definir ordem de implementação com base em valor, risco e dependências.  | Apoia decisões objetivas; visualiza trade-offs; destaca ganhos rápidos. | Pode simplificar demais questões técnicas; exige boa estimativa de esforço. | Quando é necessário alinhar expectativas de negócio com capacidade técnica. |
| <b>Negociação</b>                                                                     | Mediar interesses divergentes entre stakeholders.                        | Permite construir compromissos viáveis; fortalece a colaboração.        | Pode ser demorada; exige habilidade de mediação.                            | Quando requisitos entram em choque ou são incompatíveis.                    |
| <b>Resolução de Conflitos</b>                                                         | Aplicar estratégias formais para superar impasses.                       | Dá transparência ao processo; evita decisões unilaterais.               | Pode gerar frustração se mal conduzida; risco de acordos frágeis.           | Quando há bloqueio de decisões em grupos heterogêneos.                      |
| <b>Prompt IA</b>                                                                      | Apoiar a análise de requisitos com sugestões automáticas de refinamento. | Identifica ambiguidades rapidamente; gera alternativas de redação.      | Dependência de ferramentas externas; risco de vieses nas respostas.         | Projetos com grande volume de requisitos ou linguagem ambígua.              |

Quadro 14 – Exemplos de Técnicas de análise e consenso.  
Fonte: elaborado pelo autor (2025).

## 8.5 Colaboração e Dinâmica Humana no Consenso

Embora técnicas estruturadas apoiem a análise, é a dinâmica humana que define a qualidade do consenso. Confiança, influência, clareza de papéis e abertura para participação equilibrada moldam o processo tanto quanto os métodos aplicados.

Como ressaltam Dick, Hull e Jackson (2017), fatores humanos são determinantes para a qualidade dos requisitos, uma vez que confiança, clareza de papéis e abertura para participação equilibrada moldam diretamente os resultados da análise.

Em projetos de software, é comum observar situações em que a decisão final não decorre apenas da análise técnica, mas da capacidade de determinados atores influenciarem a discussão. Representantes de áreas estratégicas, por exemplo, podem ter mais espaço de fala que usuários operacionais, ainda que estes últimos tragam informações cruciais sobre o uso cotidiano do produto. Essa **assimetria de poder** mostra que o consenso não deve ser entendido como unanimidade espontânea, mas como um processo negociado, no qual a mediação e a facilitação são indispensáveis para equilibrar vozes.

### Box 8.5-1 – Impacto da ER nas decisões de consenso

Este ponto concretiza o **Princípio 11 (Impacto da ER)**: decisões de consenso não apenas refinam requisitos — elas moldam relações de trabalho, distribuem poder decisório, afetam rotinas e influenciam diretamente a experiência de quem usa e opera o sistema. Por isso, a análise deve explicitar consequências, riscos e grupos afetados, evitando que acordos “eficientes para fechar a reunião” gerem impactos negativos no ambiente sociotécnico após a implantação.

Entre essas vozes, a do **usuário final** merece atenção especial. Diferente de outros stakeholders, que muitas vezes falam a partir de metas de negócio ou de visões institucionais, o usuário final expressa a experiência concreta de interação com a solução digital. Sua ausência ou participação marginal pode gerar soluções que cumprem objetivos organizacionais, mas falham em atender às demandas reais de uso. Garantir que esses usuários sejam escutados de forma contínua e estruturada é condição essencial para que o consenso represente não apenas interesses estratégicos, mas também a viabilidade prática e a utilidade cotidiana do produto.

Outro aspecto central é a **construção de confiança**. O compartilhamento de requisitos sensíveis — como problemas de desempenho, falhas operacionais ou restrições de processo — só ocorre em ambientes em que os stakeholders se sentem ouvidos e respeitados. Sem esse clima de abertura, há risco de ocultação de informações, o que pode comprometer a completude da análise.

Além disso, uma boa colaboração depende da **clareza de papéis e responsabilidades**. Quando não está claro quem tem autoridade para decidir, quem pode propor e quem é responsável por validar, o processo tende à frustração e retrabalho. Estabelecer esses papéis de forma explícita, mesmo que em acordos simples, contribui para que o consenso seja alcançado de maneira transparente e legítima.

*Exemplo:* Em um projeto de sistema hospitalar, um workshop inicial foi dominado pela equipe administrativa, que insistia em priorizar relatórios financeiros. Como médicos e enfermeiros não se sentiram à vontade para expor suas necessidades, requisitos clínicos importantes — como a rapidez no acesso a prontuários e a facilidade de registrar informações em plantões — ficaram de fora da primeira versão do backlog. O problema só foi percebido em testes-piloto, quando ficou evidente que a solução não atendia adequadamente ao trabalho diário da equipe de saúde. A situação levou a uma revisão do processo de análise, introduzindo regras de facilitação mais claras, como a técnica *round robin* e o registro estruturado de vozes minoritárias, garantindo que nas reuniões seguintes todos os grupos, especialmente os usuários finais, fossem igualmente representados.

Esse exemplo mostra que o resultado da análise não é apenas uma lista de requisitos refinados, mas também um acordo social que sustenta o compromisso coletivo com o produto a ser desenvolvido, preservando tanto as metas organizacionais quanto as necessidades concretas de quem usará o sistema no dia a dia.

Casos como esse reforçam a importância de mecanismos explícitos de facilitação para equilibrar vozes, prática já destacada em estudos sobre dinâmicas colaborativas em requisitos (FERNANDES; MACHADO, 2015).

O Quadro 15 sintetiza esses fatores humanos, descrevendo seus efeitos sobre a análise e o consenso, bem como práticas que podem ser utilizadas para mitigar riscos e fortalecer a participação equilibrada de todos os stakeholders.

| Fator Humano                | Descrição                                                                    | Implicações na Análise e Consenso                                                                    | Práticas de Mitigação                                                                         |
|-----------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Influência e Poder</b>   | Diferença de peso entre stakeholders mais ou menos influentes.               | Pode gerar requisitos enviesados, refletindo apenas interesses dominantes.                           | Técnicas de facilitação (ex.: <i>round robin</i> ); registro explícito de vozes minoritárias. |
| <b>Usuário Final</b>        | Atores que interagem diretamente com a solução no dia a dia.                 | Se pouco ouvidos, requisitos podem atender metas organizacionais mas falhar no uso prático.          | Inclusão ativa em workshops e revisões; uso de cenários, personas e observação direta.        |
| <b>Confiança</b>            | Clima de abertura para compartilhar informações reais e problemas sensíveis. | Sem confiança, stakeholders ocultam dados críticos, levando a requisitos incompletos ou distorcidos. | Estimular transparência; criar ambiente seguro; mediação imparcial e escuta ativa.            |
| <b>Clima Colaborativo</b>   | Grau de cooperação e respeito nas interações.                                | Ambientes hostis reduzem participação e dificultam a construção de consenso.                         | Dinâmicas colaborativas; regras claras de interação; feedback contínuo.                       |
| <b>Clareza de Papéis</b>    | Definição de quem decide, quem sugere e quem valida requisitos.              | Ambiguidade gera frustração e atrasos, prejudicando a legitimidade das decisões.                     | Definir papéis e responsabilidades de forma explícita antes das reuniões.                     |
| <b>Compromisso Coletivo</b> | Envolvimento ativo dos stakeholders e aceitação das decisões tomadas.        | Sem compromisso, decisões formais não se traduzem em adesão prática, gerando resistência posterior.  | Inclusão de todos os grupos; registro formal de acordos; validação periódica iterativa.       |

Quadro 15 – Fatores humanos e seus efeitos sobre a análise e consenso.

Fonte: elaborado pelo autor (2025).

Em síntese, a colaboração em torno dos requisitos não é apenas uma questão de técnica, mas de equilíbrio entre fatores humanos. Reconhecer e tratar dimensões como confiança, poder, clareza de papéis e, sobretudo, a voz do usuário final, é o que garante que o consenso seja legítimo e represente tanto os interesses institucionais quanto as necessidades práticas de uso.

Entretanto, mesmo em ambientes colaborativos bem estruturados, o processo de análise e consenso está sujeito a falhas recorrentes. Reconhecer essas armadilhas é parte essencial da prática, permitindo agir preventivamente. A seção seguinte descreve os principais riscos identificados na literatura e na prática.

## 8.6 Riscos e Armadilhas na Análise e Consenso

Apesar de sua importância, a atividade de análise e consenso pode ser comprometida por riscos e armadilhas recorrentes. Esses problemas decorrem tanto de limitações técnicas quanto de dinâmicas humanas mal conduzidas, resultando em requisitos frágeis ou enviesados. Identificá-los é fundamental para preveni-los.

Um dos riscos mais comuns é o **consenso superficial**, em que o grupo aparenta concordar apenas para encerrar a discussão, sem resolver efetivamente ambiguidades ou conflitos. Essa prática gera requisitos que parecem estáveis, mas que se mostram contraditórios ou incompletos em fases posteriores do projeto (DICK; HULL; JACKSON, 2017).

Outra armadilha é a **exclusão ou sub-representação de usuários finais**. Quando gestores e especialistas dominam as discussões, necessidades práticas de quem interage diretamente com o sistema ficam ausentes, levando a produtos que cumprem metas institucionais, mas falham no uso cotidiano (LAPLANTE, 2017).

Também é frequente o **foco precoce em soluções**, em que os participantes passam a discutir como implementar algo antes de compreender plenamente o que deve ser alcançado. Essa inversão pode cristalizar escolhas de design sem que os requisitos tenham sido devidamente analisados (FERNANDES; MACHADO, 2015).

Por fim, há o risco de **paralisia decisória**, quando divergências não são mediadas de forma estruturada e a busca por unanimidade bloqueia o avanço da análise. Nessas situações, é papel do engenheiro de software facilitar a negociação e registrar compromissos viáveis, mesmo que não sejam perfeitos.

Reconhecer e mitigar esses riscos é parte essencial da prática. Evitar consensos frágeis, garantir a inclusão da voz do usuário final, separar análise de soluções e conduzir decisões de forma transparente são medidas que fortalecem a atividade e aumentam a probabilidade de sucesso do projeto.

O Quadro 16 sintetiza os principais riscos e armadilhas identificados na literatura e na prática, destacando suas consequências e estratégias de mitigação que podem ser adotadas pelo engenheiro de software e pelos facilitadores do processo.

| Risco / Armadilha                | Consequências                                                                                   | Estratégias de Mitigação                                                                              |
|----------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Consenso superficial</b>      | Requisitos aparentemente acordados, mas ambíguos ou contraditórios surgem em fases posteriores. | Incentivar debate aberto; registrar divergências; validar requisitos em diferentes perspectivas.      |
| <b>Exclusão do usuário final</b> | Produto atende a metas institucionais, mas falha no uso cotidiano.                              | Garantir participação efetiva de usuários finais; usar cenários, personas e observação direta.        |
| <b>Foco precoce em soluções</b>  | Discussão de “como implementar” antes de definir claramente “o que é necessário”.               | Diferenciar requisitos de design; usar facilitadores para manter o grupo no nível certo de abstração. |
| <b>Paralisia decisória</b>       | Bloqueio do avanço da análise pela busca de unanimidade ou por conflitos não resolvidos.        | Aplicar técnicas de negociação e resolução de conflitos; registrar compromissos viáveis e iterativos. |

Quadro 16 – Fatores humanos e seus efeitos sobre a análise e consenso.  
Fonte: elaborado pelo autor (2025).

O reconhecimento desses riscos evidencia que a análise e o consenso não são etapas meramente formais, mas práticas que exigem atenção contínua ao equilíbrio entre técnica e dinâmica social. Nesse sentido, as estratégias de mitigação sintetizadas no Quadro 16 concretizam o **Princípio 10 (Processo de ER)** ao exigir disciplina (registrar divergências, critérios e decisões) e flexibilidade (iterar e renegociar), e reforçam **Confiança** ao tornar compromissos rastreáveis. Mitigar armadilhas como consensos frágeis, exclusão do usuário final ou foco precoce em soluções é fundamental para que os requisitos avancem com qualidade.

## CAPÍTULO 8: PONTOS-CHAVE

- Análise e consenso **examina o entendimento disponível sobre os requisitos** para esclarecer significados, relações, alternativas e implicações e para construir acordos necessários à continuidade do produto.
- Informações inicialmente disponíveis podem apresentar ambiguidades, lacunas, contradições, redundâncias ou conteúdos implícitos. Essas características são naturais em um processo de construção progressiva de entendimento.
- A análise ajuda a **tornar explícitas questões ainda abertas**, permitindo investigar significados, condições, exceções, dependências, riscos e restrições relevantes.
- Conflitos entre stakeholders, áreas ou critérios não devem ser tratados como anomalias. **Negociação, mediação e discussão de trade-offs** ajudam a construir compromissos viáveis quando perspectivas legítimas são incompatíveis.
- Consenso não significa unanimidade, mas um **acordo suficientemente compreendido e sustentado** pelos participantes pertinentes para orientar decisões.
- **Priorizar** envolve definir ou acordar critérios, reunir avaliações, aplicá-las aos requisitos e discutir seus resultados. Valor, risco, urgência, esforço, custo e dependências podem assumir pesos diferentes conforme o contexto.
- A análise de viabilidade, dependências, riscos e implicações ajuda a **identificar fatores que limitam alternativas ou exigem investigação adicional** antes de decisões mais firmes.
- Workshops, técnicas de priorização, negociação e resolução de conflitos oferecem **estruturas para apoiar decisões coletivas** sem substituir julgamento, contexto ou diálogo.
- A **qualidade dos acordos** depende também das **condições sociais** em que são construídos. Assimetrias de poder, ausência de stakeholders relevantes, consenso superficial e foco prematuro em soluções podem comprometer o resultado da análise.

# Capítulo 9

## 9 Declaração de Requisitos

### 9.1 Introdução

A declaração de requisitos constitui uma das atividades centrais da Engenharia de Requisitos, pois torna explícito aquilo que foi identificado e discutido ao longo do processo de interação com stakeholders. Diferentemente de uma etapa isolada ou sequencial, a declaração deve ser entendida como **atividade transversal**, que ocorre de forma integrada com a elicitação e descoberta, a análise e consenso (POHL, 2010). Sempre que uma necessidade, uma restrição ou uma funcionalidade é identificada e explicitada, ela já está sendo declarada, seja em um quadro de backlog, em uma narrativa de caso de uso ou mesmo em um registro de áudio de uma reunião (FERNANDES; MACHADO, 2015).

Ao tratar de declaração, partimos do entendimento de **requisito** já estabelecido no Capítulo 6: a declaração não altera a natureza do requisito, mas **operacionaliza** sua comunicação e sua verificabilidade no contexto sociotécnico. Assim, as técnicas de declaração discutidas a seguir preservam o núcleo do que se entende por requisito (ver seção 6.1), variando principalmente na **forma de enunciação** e no grau de estruturação adotado.

O valor da declaração não reside apenas no registro, mas sobretudo na **capacidade de comunicar e compartilhar entendimento** entre diferentes atores do projeto. Declarar requisitos significa criar artefatos — textuais, orais e/ou vídeos — que permitam alinhar perspectivas de negócio, técnicas e de uso, favorecendo tanto a compreensão coletiva quanto a rastreabilidade futura (DICK; HULL; JACKSON, 2017). Quando forem necessárias estruturas visuais, estas são tratadas como representações (ver capítulo 10). Nesse sentido, a declaração não é sinônimo de documentação formal, mas sim de **explicitação** das expectativas e condições que orientam o desenvolvimento de um produto ou sistema (LAPLANTE, 2017).

A forma como os requisitos são declarados pode variar de acordo com o contexto e com os princípios da ER: em projetos dirigidos por plano, declarações mais detalhadas e estruturadas, como catálogos ou especificações de requisitos, tendem a ser predominantes; em contextos ágeis, artefatos leves como histórias de usuário ou épicos cumprem esse papel de forma incremental; em abordagens híbridas, ambos os tipos podem coexistir em diferentes graus de formalidade (IREB, 2022). Independentemente da abordagem, a declaração deve preservar os valores e princípios definidos no Capítulo 5 — com destaque para **comunicação, feedback, simplicidade e confiança**, que sustentam colaboração, transparência e adaptação ao longo do projeto.

Além disso, a declaração não precisa estar restrita a documentos textuais. Ela pode assumir formas multimodais, como **áudio** (ex.: gravações / transcrições de entrevistas ou reuniões), **vídeo** (ex.: gravações de workshops ou oficinas) ou combinações de diferentes mídias, sempre que tais recursos favoreçam o entendimento e a comunicação entre os envolvidos (CROWDER; HOFF, 2022). Essa flexibilidade reforça a natureza humana, social e técnica da atividade de requisitos. Em qualquer caso, recomenda-se que mídias multimodais sejam tratadas como **evidências de**



**entendimento** e vinculadas a **identificadores, versões e decisões**, para que permaneçam rastreáveis e auditáveis ao longo do ciclo de vida.

Portanto, a declaração de requisitos deve ser compreendida como uma atividade contínua e adaptável, cujo propósito é **expressar o entendimento vigente** em formas linguísticas claras, acessíveis e adequadas ao contexto, permitindo que esse entendimento seja comunicado, discutido e utilizado ao longo da evolução do produto (POHL, 2010; LAPLANTE, 2017).

A declaração atua sobre a forma linguística pela qual o entendimento dos requisitos é expresso e compartilhado. A Figura 46 delimita essa responsabilidade e evidencia como ela se relaciona com a construção do conteúdo, com as representações e com as demais atividades da Engenharia de Requisitos.



Figura 46 – Enquadramento da atividade de Declaração.  
Fonte: elaborado pelo autor (2026).

## 9.2 Objetivos da Declaração de um Requisito

A declaração de requisitos cumpre um papel essencial na Engenharia de Requisitos: tornar explícitas, compreensíveis e comunicáveis as necessidades identificadas em um projeto. Ao declarar um requisito, cria-se uma base que não é apenas técnica, mas sobretudo comunicacional, capaz de conectar diferentes perfis de stakeholders em torno de um entendimento comum (POHL, 2010; DICK; HULL; JACKSON, 2017).

Entre os principais objetivos da declaração destacam-se:

- **Compartilhar entendimento:** permitir que todos os envolvidos — desde usuários finais até desenvolvedores, analistas de negócio e gestores — reconheçam de forma clara o que se espera do produto ou sistema (FERNANDES; MACHADO, 2015).



- **Apoiar a negociação e o consenso:** requisitos declarados com clareza facilitam a discussão sobre prioridades, restrições e alternativas, favorecendo acordos transparentes (POHL, 2010).
- **Orientar o desenvolvimento:** a declaração fornece elementos para que equipes técnicas possam planejar, construir e testar funcionalidades de maneira alinhada ao valor esperado (LAPLANTE, 2017).
- **Favorecer a validação contínua:** requisitos declarados tornam-se referência para verificar se o produto atende às necessidades, permitindo feedback iterativo e incremental (IREB, 2022).
- **Favorecer a rastreabilidade:** requisitos identificáveis e suficientemente estáveis em sua forma de expressão podem ser relacionados às suas origens, decisões, representações, critérios, mudanças e entregas, contribuindo para que essas relações sejam estabelecidas e acompanhadas ao longo do ciclo de vida (LAPLANTE, 2017).

É importante reforçar que a declaração não deve ser vista como produto final ou estático. Seu objetivo não é cristalizar requisitos de forma imutável, mas sim fornecer visibilidade e transparência ao que está sendo construído. A cada interação com stakeholders, a declaração pode e deve ser revisitada, incorporando novos aprendizados ou mudanças de contexto (DICK; HULL; JACKSON, 2017).

Assim, declarar requisitos significa transformar expectativas em enunciados claros, que possam sustentar tanto a colaboração entre atores quanto a adaptabilidade do processo. Nesse sentido, os objetivos da declaração se alinham diretamente aos princípios da Engenharia de Requisitos: promover colaboração efetiva, garantir transparência, orientar pelo valor e adaptar-se à realidade do projeto (POHL, 2010; IREB, 2022).

### 9.3 Níveis de Abstração de um Requisito

A declaração de requisitos não ocorre em um único plano de detalhamento. Dependendo da perspectiva, os requisitos podem ser formulados de maneira ampla, refletindo os objetivos de uma organização, ou em termos granulares, descrevendo funcionalidades específicas a serem implementadas. Essa diversidade de camadas é reconhecida na literatura como **níveis de abstração** (SOMMERVILLE, 2019; POHL, 2010), úteis para estruturar a comunicação entre stakeholders e assegurar coerência entre expectativas estratégicas e soluções técnicas.

No contexto deste livro, distinguimos três níveis de requisitos declarados: **requisitos de negócio**, que descrevem o *porquê* de um sistema ou produto; **requisitos de usuário**, que traduzem esse direcionamento em expectativas de interação e capacidades do sistema (*o quê*); e **requisitos de produto**, que detalham de *como* a solução deve funcionar, referindo-se não à implementação, mas ao nível de detalhamento comportamental e de restrições verificáveis — preservando a separação entre requisitos e decisão de design. Neste sentido, cada nível cumpre uma função distinta, mas interdependente, compondo uma cadeia de rastreabilidade que conecta o valor organizacional à realização técnica (DICK; HULL; JACKSON, 2017; IEEE COMPUTER SOCIETY, 2024).

No nível mais elevado encontram-se os **requisitos de negócio**, que capturam os problemas a serem enfrentados, as metas estratégicas da organização e as restrições externas impostas por legislação, normas ou políticas internas.

Um segundo nível corresponde aos **requisitos de usuário**, que dão corpo ao *o quê* o sistema deve oferecer em termos de capacidades e serviços. Nesse ponto, o foco desloca-se das metas organizacionais para as necessidades e expectativas de uso. São declarações que expressam como diferentes perfis de usuários pretendem interagir com o sistema ou quais funcionalidades de alto nível esperam encontrar.

No nível mais detalhado situam-se os **requisitos de produto**, que explicitam o *como* o sistema ou produto deve operar. Trata-se de declarações precisas sobre funcionalidades, fluxos de interação, manipulação de dados e restrições operacionais, incluindo requisitos não funcionais em nível técnico.

Esses três níveis não devem ser vistos como etapas sequenciais a serem percorridas em ordem rígida. Na prática, eles coexistem e se influenciam mutuamente (ver Figura 47): um protótipo construído a partir de requisitos de produto pode revelar necessidades de usuário ainda não enunciadas, enquanto novas metas de negócio podem levar a revisões em funcionalidades já detalhadas. O reconhecimento desses níveis como **camadas complementares, e não como fases lineares**, é fundamental para compreender a natureza dinâmica da declaração de requisitos e para manter a rastreabilidade entre o propósito estratégico e a realização técnica (PRESSMAN; MAXIM, 2021; VAN LAMSWEERDE, 2009).



Figura 47 – Níveis de abstração e tipos de abstração.  
Fonte: elaborado pelo autor (2025).

É importante, contudo, distinguir esses três níveis de requisitos — que são declarações propriamente ditas — de **artefatos de apoio**. Personas, jornadas de usuário, mapas de histórias e técnicas de Product Backlog Building (PBB), por exemplo, não constituem requisitos, mas sim instrumentos que auxiliam sua identificação, organização ou validação. Enquanto os requisitos de negócio, usuário e produto integram o corpo central da declaração, esses artefatos complementares funcionam como recursos para enriquecer a compreensão e facilitar a colaboração entre os

diferentes atores envolvidos (ALEXANDER; BEUS-DUKIC, 2009; ROBERTSON; ROBERTSON, 2014).

### 9.3.1 Requisitos de Negócio

Os requisitos de negócio constituem o nível mais elevado de abstração da declaração de requisitos. Eles expressam o “*porquê*” de um sistema ou produto, descrevendo os problemas que precisam ser resolvidos, as metas estratégicas que justificam o investimento e as restrições externas impostas por legislações, normas ou políticas organizacionais. Esse nível estabelece o valor esperado e cria a base sobre a qual os demais requisitos serão formulados (POHL, 2010; DICK; HULL; JACKSON, 2017; IEEE COMPUTER SOCIETY, 2024).

A clareza na declaração de requisitos de negócio é essencial para que a solução seja não apenas tecnicamente viável, mas também legítima e relevante. Quando mal definidos, é comum que sistemas cumpram especificações de produto, mas falhem em entregar benefícios reais ou em atender exigências críticas. Em domínios fortemente regulados, como saúde, finanças ou governo, a presença de requisitos de negócio explícitos é ainda mais importante para assegurar conformidade (ROBERTSON; ROBERTSON, 2014; CROWDER; HOFF, 2022).

O Quadro 17, a seguir, ilustra um cenário educacional, mostrando como os requisitos de negócio podem ser declarados em termos de problemas a resolver, metas estratégicas e restrições externas, além de metas complementares ligadas ao valor público.

#### Cenário: Modernização da gestão escolar em rede pública

| Tipo                       | Declaração                                                                                                    | Foco                                                                     |
|----------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Meta estratégica           | “Reduzir em 25% a evasão escolar no ensino médio até o final do próximo biênio.”                              | Conectar tecnologia à melhoria de indicadores educacionais.              |
| Necessidade organizacional | “Oferecer acompanhamento centralizado da frequência e do desempenho dos alunos em toda a rede estadual.”      | Apoiar gestores escolares com dados consolidados para tomada de decisão. |
| Restrição externa          | “Atender às diretrizes da Lei Geral de Proteção de Dados (LGPD) no tratamento das informações de estudantes.” | Garantir conformidade legal e proteção da privacidade dos alunos.        |
| Diretriz de valor público  | “Assegurar maior transparência na divulgação dos resultados de desempenho escolar para a sociedade.”          | Alinhar tecnologia a princípios de prestação de contas e cidadania.      |

Quadro 17 – Exemplos de requisitos de negócio.  
Fonte: elaborado pelo autor (2025).

### 9.3.2 Requisitos de Usuário

Os requisitos de usuário ocupam um nível intermediário de abstração e descrevem o “*o quê*” o sistema deve oferecer em termos de serviços, funcionalidades de alto nível e experiências de uso. Eles traduzem metas estratégicas em expectativas práticas, representando aquilo que diferentes atores — cidadãos, gestores, operadores ou sistemas externos — esperam realizar com a solução (NUSEIBEH; EASTERBROOK, 2000; DICK; HULL; JACKSON, 2017).

Diferentemente dos requisitos de negócio, que têm foco organizacional, os requisitos de usuário privilegiam a perspectiva de quem interage diretamente com o sistema. Por isso, devem ser expressos em linguagem acessível, evitando tecnicismos desnecessários. Esse cuidado amplia a

participação de stakeholders não técnicos e favorece o consenso sobre o que realmente agrega valor (ALEXANDER; BEUS-DUKIC, 2009; IEEE COMPUTER SOCIETY, 2024).

O Quadro 18 apresenta um cenário educacional, ilustrando como os requisitos de usuário podem ser formulados para expressar expectativas práticas de interação a partir das metas de negócio.

#### Cenário: Modernização da gestão escolar em rede pública

| Tipo                      | Declaração                                                                                                                     | Foco                                                                |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| Acesso à informação       | “O aluno deve poder consultar seu histórico de frequência e notas em tempo real pelo portal ou aplicativo.”                    | Atender diretamente às necessidades dos estudantes.                 |
| Apoio ao trabalho docente | “O professor deve poder registrar presenças e notas em dispositivos móveis, mesmo offline, com sincronização posterior.”       | Facilitar a rotina pedagógica e reduzir burocracias.                |
| Gestão escolar            | “O diretor deve poder consolidar relatórios de desempenho da escola em formato gráfico, comparando turmas e períodos letivos.” | Apoiar a tomada de decisão em nível institucional.                  |
| Integração externa        | “Sistemas de órgãos governamentais devem poder consumir dados da escola via API padronizada.”                                  | Viabilizar interoperabilidade e alinhamento com políticas públicas. |

Quadro 18 – Exemplos de requisitos de usuário.  
Fonte: elaborado pelo autor (2025).

### 9.3.3 Requisitos de Produto

Os requisitos de produto representam o nível mais detalhado de abstração. Eles descrevem o *como* o sistema deve funcionar, especificando comportamentos, fluxos de interação, estruturas de dados e restrições operacionais em termos verificáveis. É nesse nível que a declaração de requisitos se conecta diretamente com atividades de projeto, implementação e testes (ISO/IEC/IEEE 29148:2018; IEEE COMPUTER SOCIETY, 2024).

Diferentemente dos requisitos de usuário, que permanecem próximos da experiência de uso, os requisitos de produto precisam ser formulados de maneira **precisa e mensurável**. São declarações que orientam a equipe técnica, mas que devem manter rastreabilidade com os níveis anteriores para garantir que o valor estratégico não se perca no detalhamento operacional (WIEGERS; BEATTY, 2013).

O Quadro 19, exemplifica requisitos de produto no cenário educacional, detalhando como funcionalidades e restrições técnicas podem ser declaradas de forma verificável.

#### Cenário: Modernização da gestão escolar em rede pública

| Tipo                     | Declaração                                                                                                                                                                   | Foco                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Funcionalidade detalhada | “O sistema deve autenticar o aluno com CPF e senha, e disponibilizar o histórico de frequência e notas em até 5 segundos após a solicitação.”                                | Garantir acesso rápido e seguro às informações pessoais. |
| Fluxo de interação       | “O professor deve poder registrar a frequência de cada aluno em uma tela única, com opção de marcação em lote (‘presente para toda a turma’) e edição individual posterior.” | Reduzir tempo de operação docente em sala de aula.       |

| Tipo                    | Declaração                                                                                                                                                       | Foco                                                              |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Especificação de dados  | “As notas devem ser armazenadas com precisão de uma casa decimal, vinculadas a cada disciplina e período letivo, seguindo os padrões da Secretaria de Educação.” | Assegurar consistência e interoperabilidade dos dados acadêmicos. |
| Requisito não funcional | “O sistema deve estar disponível 99,5% do tempo em dias letivos, com suporte a 10.000 acessos simultâneos.”                                                      | Garantir desempenho e confiabilidade em larga escala.             |

Quadro 19 – Exemplos de requisitos de produto.  
Fonte: elaborado pelo autor (2025).

### 9.3.4 Síntese dos Níveis de Abstração

A análise dos três níveis de abstração mostra que cada um deles cumpre um papel específico e complementar na declaração de requisitos.

- **Requisitos de negócio** estabelecem o “*porquê*”, ao enunciar problemas estratégicos, metas institucionais e restrições normativas que justificam o investimento.
- **Requisitos de usuário** traduzem essas diretrizes em expectativas práticas de interação, expressando o “*o quê*” precisa ser oferecido aos diferentes atores.
- **Requisitos de produto** descem ao nível do “*como*”, detalhando funcionalidades, fluxos, dados e restrições operacionais de maneira verificável.

No cenário educacional apresentado nas subseções 9.3.1 a 9.3.3, é possível observar como cada nível contribui: desde a meta de reduzir evasão escolar (negócio), passando pela necessidade de alunos, professores e gestores acessarem informações e serviços de forma ágil (usuário), até os requisitos técnicos de autenticação, registro de notas e disponibilidade do sistema (produto).

Assim, a principal contribuição dos níveis de abstração não é impor uma sequência rígida, mas oferecer uma estrutura conceitual que assegure alinhamento e rastreabilidade. Ao reconhecer que requisitos coexistem em diferentes camadas e que podem ser declarados em múltiplos formatos (textuais ou multimodais), a Engenharia de Requisitos fortalece sua função central: conectar necessidades humanas e organizacionais à construção de soluções de software sustentáveis e de valor.

## 9.4 Tipos de declarações

Os conteúdos apresentados na seção anterior (9.3) trataram do **que** é declarado — isto é, dos diferentes níveis de requisitos (de negócio, de usuário e de produto).

Esta seção aborda **como** essas informações podem ser declaradas, ou seja, as técnicas e formatos utilizados para comunicar requisitos de maneira compreensível e compartilhável entre diferentes públicos.

Distinguir o **conteúdo** da **técnica de declaração** é fundamental: enquanto o conteúdo se refere ao tipo de informação (meta, regra de negócio, requisito funcional, restrição etc.), a técnica indica a forma de comunicação adotada (texto livre, estrutura tabular, narrativa, checklist, modelo visual, entre outros). Essa distinção evita confusões comuns, como tratar “regras de negócio” ou “requisitos não funcionais” como técnicas, quando na verdade são tipos de informação que podem ser declarados de múltiplas formas.

A literatura sobre Engenharia de Requisitos identifica quatro grandes famílias de técnicas de declaração de requisitos (Figura 48), que se distribuem segundo o grau de estruturação e o propósito comunicativo (LAUESEN, 2002; WIEGERS; BEATTY, 2013; ROBERTSON; ROBERTSON, 2014; DICK; HULL; JACKSON, 2017; IREB, 2022):

- **Narrativas descritivas:** Baseadas em linguagem natural livre, utilizadas para contextualizar necessidades e experiências de uso.
- **Declarações estruturadas:** Empregam formato controlado, geralmente em listas ou tabelas, com sintaxe uniforme.
- **Orientadas a valor e experiência:** Organizadas em torno do propósito e benefício esperado por quem utiliza o sistema.
- **Catálogos e artefatos técnicos:** Consolidam requisitos em estruturas formais e rastreáveis, integradas ao processo de desenvolvimento.

### Tipos de Técnicas de Declaração de Requisitos



Figura 48 – Tipos de técnicas de declaração de requisitos.  
Fonte: elaborado pelo autor (2025).

As subseções a seguir detalham essas famílias, destacando finalidade, aplicação e limitações de cada uma.

#### 9.4.1 Narrativas Descritivas

As **narrativas descritivas** constituem uma forma inicial e natural de expressar requisitos, especialmente nas fases em que o entendimento do problema ainda está sendo construído. Elas utilizam a linguagem comum dos stakeholders, descrevendo **situações de uso, objetivos e intenções**, sem recorrer a estruturas rígidas ou notações específicas.

Essa forma de declaração privilegia a **compreensão compartilhada** sobre o comportamento esperado do sistema.

As narrativas descrevem *quem* interage, *o que* se pretende alcançar e *por que* a funcionalidade é importante, aproximando a visão técnica da visão de negócio.

**Exemplo:**

*“Durante o período de matrícula, alunos relatam filas longas na secretaria. A equipe sugere permitir upload prévio de documentos no portal para reduzir o atendimento presencial.”*

As narrativas descritivas são particularmente úteis em contextos onde há **múltiplas interpretações possíveis** sobre o domínio, pois permitem explicitar cenários, restrições e variações sem perder a naturalidade da comunicação. São eficazes para capturar percepções e expectativas sem exigir notações formais, favorecendo a empatia e a validação com stakeholders não técnicos. No entanto, tendem a apresentar **ambiguidade, redundância e falta de padronização**, exigindo etapas posteriores de consolidação e refinamento. São recomendadas quando se busca compreender o contexto de uso, **identificar oportunidades de melhoria** e **gerar entendimento compartilhado** antes da formalização dos requisitos.

Com o amadurecimento do entendimento, essas descrições tendem a evoluir para formas mais estruturadas — como casos de uso, histórias de usuário ou critérios de aceitação.

Em síntese, as narrativas descritivas representam a **primeira tradução das necessidades em linguagem compreensível**, servindo de base para que os requisitos sejam posteriormente organizados, refinados e priorizados.

#### 9.4.1.1 Narrativas em texto livre

As narrativas em texto livre constituem a forma mais simples e direta de declaração de requisitos, geralmente resultando de entrevistas, oficinas ou registros espontâneos feitos por stakeholders. A grande vantagem dessa técnica é a acessibilidade: qualquer participante pode produzir ou compreender o conteúdo, sem necessidade de treinamento em notações formais (SOMMERVILLE, 2019). Isso permite capturar percepções, expectativas e preocupações em linguagem natural, aproximando o processo de requisitos da experiência cotidiana dos usuários.

Em oficinas colaborativas, como JAD ou workshops de descoberta, é comum registrar interações preservando a riqueza do discurso. Robertson e Robertson (2014) destacam que esses registros servem como insumos brutos que precisam ser posteriormente organizados, pois carregam ambiguidades e redundâncias. Assim, seu principal ponto forte é a fidelidade à voz dos stakeholders, enquanto a limitação está na falta de padronização, que pode dificultar análise e rastreabilidade.

**Exemplo de texto livre:**

“Durante a matrícula, alunos relatam filas longas na secretaria. A equipe aponta que a conferência manual de documentos é o gargalo. Sugerem que o portal aceite upload prévio e checklist automático para reduzir atendimento presencial.”

#### 9.4.1.2 Casos de uso – breve descrição

Casos de uso em sua forma breve apresentam apenas a essência de uma interação típica entre usuário e sistema, descrita em poucas sentenças narrativas. Eles são úteis para mapear rapidamente o escopo funcional sem entrar em detalhes de fluxos alternativos ou condições excepcionais. Alexander e Beus-Dukic (2009) apontam que esse formato inicial ajuda a alinhar

expectativas entre diferentes públicos, funcionando como uma “porta de entrada” para discussões mais detalhadas.

O ponto forte dos casos de uso breves é a simplicidade: podem ser discutidos em alto nível antes de se investir esforço em modelagens mais sofisticadas. Contudo, a ausência de detalhamento pode gerar interpretações divergentes, especialmente em sistemas complexos. Por isso, sua principal aplicação é na fase inicial de levantamento, onde a ênfase está na comunicação e não ainda na validação formal dos requisitos.

#### **Exemplo de breve descrição de caso de uso:**

**Caso de uso:** Emitir comprovante de matrícula

**Descrição:** Este caso de uso permite ao aluno realizar a emissão de comprovante de matrícula, documento necessário para acessar benefícios estudantis. O processo ocorre de forma simples: após se autenticar no sistema, o aluno solicita o comprovante, que é validado quanto à existência de vínculos ativos e, em seguida, disponibilizado em formato PDF.

#### **9.4.1.3 Cenários narrativos**

Cenários narrativos descrevem histórias concretas de uso do sistema em situações específicas, explorando contexto, atores e consequências. Sutcliffe (2002) ressalta que cenários ampliam a empatia, pois permitem que stakeholders se coloquem no lugar do usuário, compreendendo necessidades de maneira situada. Essa característica os torna particularmente valiosos em sistemas interativos ou de domínio social, em que a experiência do usuário é central.

Na prática, cenários narrativos podem ser construídos em workshops, relatando sequências plausíveis de eventos que ocorrem no cotidiano da organização. Sua força está em revelar requisitos latentes — aspectos não explicitados em listas funcionais, mas evidentes quando se narra a experiência completa. A limitação, entretanto, é que cenários isolados não cobrem todas as variações possíveis, exigindo complementaridade com outras técnicas estruturadas (IEEE COMPUTER SOCIETY, 2024).

#### **Exemplo (cenário):**

**Cenário “Consulta de notas”** — Maria, aluna do 2º ano, abre o aplicativo no intervalo da aula para verificar a nota da prova de matemática. Ela encontra a disciplina pelo nome, toca em “Notas” e visualiza a lista por avaliação, com data e peso. Em seguida, compartilha o boletim com os responsáveis via link temporário.

#### **9.4.1.4 Texto narrativo em linguagem natural**

O uso de texto narrativo em linguagem natural permanece como a técnica mais difundida para declaração de requisitos, em virtude de sua universalidade. Wiegers e Beatty (2013) argumentam que, embora seja inevitavelmente ambígua, a linguagem natural tem papel insubstituível como meio de comunicação entre áreas técnicas e de negócio. Ela permite descrever requisitos em forma de declarações compreensíveis a todos, funcionando como um ponto comum de entendimento.



A aplicação prática inclui tanto descrições breves de requisitos funcionais (“o sistema deve permitir...”) quanto narrativas mais extensas para requisitos de contexto ou de qualidade. A força dessa técnica é a inclusão: qualquer stakeholder pode contribuir e revisar. Contudo, sua limitação reside na falta de precisão — diferentes interpretações podem levar a falhas de implementação. Lauesen (2002) recomenda, por isso, complementar textos narrativos com exemplos, tabelas ou critérios mensuráveis que reduzam ambiguidades.

#### **Exemplos (declarações curtas):**

- “O sistema deve disponibilizar histórico de frequência e notas em até 5 segundos após a solicitação.”
- “O portal deve permitir que responsáveis acessem boletins com autorização explícita do aluno.”
- “O sistema deve permitir a matrícula de alunos.”

#### **9.4.1.5 Storyboards descritivos**

Os storyboards descritivos combinam narrativa textual com uma sequência de representações visuais ou semi-visuais, organizadas como “quadros” de uma história. Muito utilizados em projetos centrados no usuário, permitem representar jornadas, interações e fluxos de interface de forma acessível, aproximando o processo de requisitos de técnicas de design de interação (SUTCLIFFE, 2002).

Na prática, são construídos a partir de protótipos de baixa fidelidade ou até esboços manuais, ou em auxílio a eles, acompanhados de textos explicativos que descrevem o que o usuário faz e o que o sistema responde em cada etapa. Seu ponto forte é a capacidade de engajar stakeholders não técnicos, tornando tangíveis situações que poderiam ser abstratas. A limitação é que podem gerar expectativas visuais precoces sobre a solução, induzindo stakeholders a confundir o storyboard com o design final do sistema (ALEXANDER e BEUS-DUKIC, 2009).

Essa técnica de declaração de requisitos transita / se conecta com as técnicas de representação livres (intuitivas) (ver seção 10.5.2).

#### **Exemplo (sequência textual):**

- **Quadro 1 — Tela inicial:** Aluna vê botão “Boletim”.
- **Quadro 2 — Seleção de disciplina:** Lista com busca por nome/professor.
- **Quadro 3 — Visualização:** Tabela de avaliações (prova, trabalho), data, peso e nota.
- **Quadro 4 — Ação de compartilhar:** Botão “Gerar link de leitura (48h)”.

#### **9.4.2 Declarações Estruturadas**

As **declarações estruturadas** correspondem a formas de comunicação de requisitos que utilizam **linguagem controlada e formato padronizado**, buscando maior precisão e uniformidade do que as narrativas descritivas. Elas reduzem ambiguidades, facilitam o rastreamento e permitem que diferentes engenheiros de software ou equipes interpretem o mesmo requisito de maneira consistente.

Nessas declarações, o conteúdo é expresso em frases curtas e diretas, frequentemente agrupadas em **tabelas, listas ou modelos textuais semi-estruturados**. São úteis quando o projeto já atingiu certo nível de entendimento sobre o domínio e é necessário **organizar o conhecimento coletado em estruturas comparáveis e auditáveis**.

O grau de formalidade pode variar conforme a técnica adotada. Em alguns casos, utiliza-se **linguagem natural controlada**, com vocabulário restrito e estrutura sintática uniforme — como nas listas de requisitos ou nas tabelas de ações e respostas. Em outros, adota-se **formato técnico padronizado**, projetado para integração ou rastreabilidade (por exemplo, templates, planilhas de requisitos). Essa variação permite equilibrar **clareza comunicativa** de acordo com a maturidade do processo e o público-alvo da declaração.

#### 9.4.2.1 Lista de Requisitos

As listas de requisitos constituem a forma mais simples e difundida de declaração estruturada. Elas apresentam os requisitos individualmente, em frases curtas e diretas, seguindo o padrão **“verbo no infinitivo + objeto”**, que explicita a ação esperada do sistema.

Cada item da lista expressa uma ação esperada do sistema, sem detalhar condições, fluxos ou interfaces. Trata-se, portanto, de uma declaração preliminar, situada em um nível de abstração mais alto, que servirá como ponto de partida para declarações e representações mais detalhadas. Essas declarações podem ser utilizadas como entrada comum para diferentes abordagens de desenvolvimento, sejam elas dirigidas por plano, ágeis ou híbridas.

##### Exemplos:

- Registrar solicitação de matrícula;
- Validar dados do estudante;
- Emitir comprovante de pagamento;
- Calcular média final das disciplinas;
- Enviar notificação de resultado.

O formato padronizado ajuda a evitar ambiguidades e facilita a manutenção de rastreabilidade entre requisitos, funcionalidades e objetivos de negócio. Além disso, torna-se útil em atividades de análise de escopo, planejamento de releases, organização inicial de backlog e alinhamento entre equipes técnicas e de negócio.

A simplicidade das declarações estruturadas não implica limitação: ela representa um estágio inicial de comunicação, a partir do qual cada requisito pode evoluir e ser refinado conforme a necessidade do projeto. A lista de requisitos representa um modo estruturado de pensar e comunicar o que o sistema deve fazer, sem antecipar soluções.

#### 9.4.2.2 Declarações textuais estruturadas (templates)

Os templates são estruturas padronizadas que orientam a escrita de requisitos em linguagem natural estruturada. A proposta é reduzir ambiguidades e omissões, promovendo consistência no registro. Robertson e Robertson (2014) defendem que um bom template ajuda a lembrar atributos

essenciais, como origem, prioridade e critério de aceitação. Além disso, favorece a rastreabilidade entre requisitos e elementos do projeto.

Na prática, equipes podem adotar estruturas como “O sistema deve <ação> <objeto> quando <condição>”. Essa abordagem diminui a variedade de estilos individuais, tornando os requisitos mais fáceis de revisar coletivamente. O ponto forte é a padronização, mas a limitação está na rigidez: um template inadequado pode forçar descrições inadequadas.

**Template:** *O sistema deve <ação> <objeto> [condição/opcional] [restrição/critério].*

**Exemplo:**

O sistema deve disponibilizar o histórico de notas <ação> do aluno <objeto> quando solicitado no portal acadêmico [condição/opcional], com tempo máximo de resposta de cinco segundos [restrição/critério].

#### 9.4.2.3 Texto estruturado/tabular

Tabelas permitem organizar requisitos em linhas e colunas, combinando clareza visual com concisão. Lauesen (2002) destaca que esse formato é especialmente útil para consolidar informações repetitivas ou variantes, como regras de cálculo ou condições de negócio. Ele facilita a leitura comparativa, já que o leitor enxerga padrões e exceções lado a lado.

No uso prático, planilhas ou quadros podem apresentar colunas para identificador, descrição, prioridade, critérios de aceitação e observações. Essa técnica é amplamente empregada em documentos de requisitos formais ou em catálogos de restrições técnicas. A limitação é que o formato tabular tende a ser menos narrativo, dificultando a compreensão por stakeholders não técnicos.

**Exemplo:**

| ID   | Requisito                                                                    | Prioridade           | Critério de Aceitação                                                      |
|------|------------------------------------------------------------------------------|----------------------|----------------------------------------------------------------------------|
| RF-X | <descrição do requisito>                                                     | <Alta, Média, Baixa> | <condição verificável>                                                     |
| RF-1 | O sistema deve validar pré-requisitos de disciplinas no momento da matrícula | Alta                 | A matrícula só será confirmada se todos os pré-requisitos forem cumpridos. |

#### 9.4.2.4 Listas estruturadas (Given/When/Then)

As listas estruturadas usam o formato “Dado/Quando/Então” para declarar condições, eventos e resultados esperados. Elas foram inspiradas em práticas de desenvolvimento dirigido por testes e são recomendadas pelo SWEBOK (2024) como técnica de linguagem natural estruturada. Sua força está na clareza: cada requisito é expresso como contexto, ação e efeito observável.

Na prática, equipes podem adotar essa estrutura tanto em requisitos funcionais quanto em critérios de aceitação, sem necessariamente vinculá-la a frameworks de automação. A vantagem é a simplicidade, mas a limitação está na cobertura: listas isoladas podem não capturar fluxos complexos ou exceções.

**Template:**

*Dado <condição inicial>*

*Quando <ação>*

*Então <resultado esperado>*

**Exemplo:**

Dado que o aluno esteja autenticado

Quando solicitar a emissão do comprovante de matrícula

Então o sistema deve gerar e disponibilizar o documento em PDF.

#### 9.4.2.5 Casos de uso detalhados (fluxos de interação etc.)

Diferentemente dos casos de uso breves, os detalhados descrevem atores, pré-condições, fluxo principal, fluxos alternativos e exceções. Alexander e Beus-Dukic (2009) enfatizam que esse formato auxilia na análise sistemática de interações, sendo adequado para sistemas críticos ou com alto número de cenários de exceção.

O ponto forte é a completude: fluxos alternativos e exceções evitam lacunas. A limitação está no esforço: casos de uso detalhados demandam tempo de escrita e revisão, podendo se tornar pesados em contextos ágeis.

**Template (estrutura resumida)**

*Título: <nome do caso>*

*Ator principal: <ator>*

*Pré-condição: <condição inicial>*

*Fluxo principal: <passos numerados>*

*Fluxos alternativos: <variações>*

*Exceções: <condições de erro>*

**Exemplo:**

Título: Emitir comprovante de matrícula

Ator principal: Aluno

Pré-condição: Estar autenticado no sistema

Fluxo principal:

1. O aluno solicita a emissão do comprovante.
2. O sistema valida os vínculos ativos.
3. O sistema gera o documento em PDF.
4. O sistema disponibiliza o arquivo para download.

Fluxo alternativo:

2a. Se houver pendência financeira, o sistema bloqueia a emissão e exibe mensagem de regularização.

#### 9.4.2.6 Critérios de aceitação

Critérios de aceitação complementam requisitos ou histórias, definindo condições objetivas que precisam ser atendidas para considerar o item concluído. Wiegers e Beatty (2013) ressaltam que esse formato conecta requisitos à validação, facilitando a comunicação entre equipe técnica e stakeholders.

Em nível alto, critérios podem ser simples (“o usuário deve conseguir concluir a matrícula”). Já em nível detalhado, eles são mensuráveis (“a matrícula deve ser concluída em até três passos e confirmada por mensagem no e-mail institucional”). O ponto forte é a verificabilidade, mas a limitação está no risco de se confundir critério de aceitação com design ou solução técnica.

##### **Estrutura:**

*[Condição a ser atendida de forma objetiva e verificável]*

##### **Exemplo:**

A matrícula só será considerada concluída quando o aluno receber e-mail de confirmação contendo número de protocolo e link para consulta.

#### 9.4.2.7 BDD (Behavior-Driven Development)

BDD é uma abordagem que combina especificação e testes automatizados. Diferente das listas Given/When/Then usadas de forma manual, o BDD conecta essas descrições a ferramentas (como Cucumber ou SpecFlow) que executam os cenários como testes. Leffingwell (2011) e o SWEBOK (2024) destacam que essa prática aproxima desenvolvedores, analistas e clientes, criando uma linguagem comum e verificável.

Seu ponto forte é a rastreabilidade direta entre requisito e teste, reduzindo ambiguidades e garantindo alinhamento contínuo entre o que foi declarado e o que foi implementado. A limitação é a exigência de infraestrutura técnica e disciplina de escrita, o que pode afastar stakeholders não técnicos se não houver mediação adequada.

##### **Estrutura:**

*Funcionalidade: <nome da funcionalidade>*

*Cenário: <descrição do cenário>*

*Dado <condição inicial>*

*Quando <ação>*

*Então <resultado esperado>*

##### **Exemplo:**

Funcionalidade: Emissão de comprovante de matrícula

Cenário: Aluno solicita emissão com vínculos ativos

Dado que o aluno esteja autenticado  
Quando solicitar o comprovante de matrícula  
Então o sistema deve gerar o documento em PDF  
E disponibilizar para download imediato

#### 9.4.2.8 Checklists estruturados

Os checklists estruturados organizam itens de verificação para garantir que aspectos importantes não sejam esquecidos. Robertson e Robertson (2014) sugerem seu uso especialmente para requisitos não funcionais e critérios de qualidade, pois ajudam a lembrar pontos como desempenho, segurança e usabilidade.

Na prática, cada checklist funciona como guia de revisão: os engenheiros verificam se cada requisito atende ou não a determinado aspecto. O ponto forte é a simplicidade e a abrangência, mas a limitação está na superficialidade – checklists não substituem análise aprofundada ou testes formais.

##### **Estrutura:**

[ ] *Item 1 de verificação*

[ ] *Item 2 de verificação*

[ ] *Item 3 de verificação*

##### **Exemplo:**

[ ] O sistema valida pré-requisitos da disciplina

[ ] O sistema calcula corretamente a carga horária total

[ ] O sistema bloqueia matrícula em disciplinas com conflito de horário

[ ] O sistema gera protocolo de matrícula após confirmação

#### 9.4.3 Declarações orientadas a valor

As especificações orientadas a valor e experiência buscam descrever os requisitos sob a perspectiva do usuário final e do valor gerado pela interação com o sistema. Diferem das anteriores por enfatizar propósito e benefício, mais do que a ação ou a estrutura técnica. Essas declarações assumem diferentes formas, entre as quais se destacam:

- Histórias de usuário
- Épicos e features
- Narrativas curtas ligadas ao impacto de negócio

O foco dessas declarações está em explicitar o valor entregue e fortalecer o entendimento empático sobre quem utiliza o sistema e em que contexto. Por isso, são amplamente aplicáveis em projetos orientados à experiência do usuário, co-design e desenvolvimento de produtos digitais. No entanto, sua utilidade não se restringe a abordagens ágeis — elas podem ser combinadas com

processos dirigidos por plano como forma de traduzir o raciocínio do negócio em linguagem compreensível e motivadora.

#### 9.4.3.1 Histórias de usuário

As histórias de usuário são uma das técnicas mais difundidas em contextos ágeis. Elas buscam expressar requisitos de forma simples, destacando quem se beneficia, o que se deseja e qual o valor esperado. O formato tradicional (“Como <usuário>, quero <objetivo>, para <benefício>”) aproxima o requisito da experiência prática, reforçando a empatia com os usuários (COHN, 2004; WIEGERS e BEATTY, 2013).

Na aplicação prática, histórias de usuário são frequentemente utilizadas em backlogs de produto, complementadas por critérios de aceitação que as tornam verificáveis. Seu ponto forte é a simplicidade e a capacidade de facilitar o diálogo entre equipe técnica e stakeholders. A limitação é a ambiguidade: histórias muito genéricas ou mal detalhadas podem perder valor analítico e se tornar apenas slogans, sem conexão clara com entregáveis ou testes.

**Template:**

*Como <usuário>, Quero <objetivo>, Para <benefício ou valor>.*

**Exemplo:**

Como aluno, Quero acessar meu boletim online, Para acompanhar meu desempenho sem precisar ir à secretaria.

#### 9.4.3.2 Épicos

Épicos são histórias de grande porte, que representam funcionalidades amplas ou necessidades estratégicas de negócio. Servem como contêineres para várias histórias menores, permitindo organizar o backlog em torno de objetivos mais abrangentes (LEFFINGWELL, 2011). Em programas de larga escala, épicos ajudam a alinhar prioridades de valor entre times, funcionando como elo entre visão estratégica e incrementos de produto.

O ponto forte dos épicos é fornecer uma narrativa unificada, que conecta funcionalidades dispersas a um propósito comum. Contudo, sua limitação está no nível de abstração: épicos sozinhos não são implementáveis. Para se tornarem acionáveis, precisam ser decompostos em histórias menores e acompanhados de critérios que possibilitem priorização e validação.

**Template:**

*Épico: <declaração ampla de uma necessidade de negócio ou de usuário>*

**Exemplo:**

Épico: Garantir que alunos e responsáveis tenham acesso digital completo a informações acadêmicas, incluindo matrícula, notas e frequência, em qualquer dispositivo.

#### 9.4.3.3 Narrativas curtas ligadas ao impacto de negócio

As narrativas curtas de impacto de negócio são declarações diretas que descrevem uma funcionalidade ou serviço associado a um resultado mensurável para a organização. Diferem das

histórias de usuário por focarem no **benefício institucional ou estratégico**, em vez da perspectiva individual do usuário. Robertson e Robertson (2014) sugerem esse tipo de narrativa como forma de traduzir requisitos em valor percebido pelo negócio.

Na prática, essas narrativas são úteis em comunicações com gestores e patrocinadores, pois sintetizam de forma simples como um requisito contribui para reduzir custos, aumentar eficiência ou melhorar a satisfação de clientes. Seu ponto forte é o foco no impacto organizacional, mas a limitação está na falta de detalhamento técnico, exigindo complementação por técnicas estruturadas ou critérios de aceitação.

**Template:**

*<Descrição breve da funcionalidade ou serviço>, com <impacto ou benefício esperado para o negócio ou usuário>.*

**Exemplo:**

Permitir que o aluno compartilhe o boletim via link temporário, com redução da demanda de atendimentos presenciais na secretaria e melhoria da comunicação com responsáveis.

#### 9.4.4 Catálogos e artefatos técnicos

Catálogos e artefatos técnicos podem oferecer uma forma estruturada de registrar declarações de requisitos e informações relacionadas. Em vez de apresentar apenas um enunciado isolado, esses artefatos podem reunir atributos que ajudam a identificar e contextualizar cada item, como tipo, origem, critérios associados e outras informações pertinentes ao uso pretendido (LAUESEN, 2002; ROBERTSON; ROBERTSON, 2014).

Do ponto de vista deste capítulo, o interesse está na forma como o conteúdo de cada requisito é declarado e compreendido dentro dessas estruturas. A definição da estrutura do catálogo, os vínculos entre seus elementos, o versionamento e sua manutenção ao longo do tempo serão aprofundados no Capítulo 12.

Diferentes catálogos podem coexistir em um mesmo produto, conforme a natureza das informações declaradas:

- Catálogo de requisitos funcionais (RFs);
- Catálogo de requisitos não funcionais ou de qualidade (RNFs);
- Catálogo técnico;
- Catálogo de regras de negócio (ver definição de RN na Seção 6.3);
- Glossário e catálogo de dados.

Assim, o catálogo pode servir como suporte persistente para diferentes tipos de declaração sem que sua existência, por si só, assegure qualidade, coerência ou rastreabilidade. Esses resultados dependem tanto da qualidade das declarações quanto da forma como o conjunto é organizado e mantido.



#### 9.4.4.1 Catálogos de metas

Catálogos de metas combinam declarações de objetivos de negócio e requisitos relacionados em uma estrutura que torna visível como determinados requisitos contribuem para resultados estratégicos (LAUESEN, 2002; ROBERTSON; ROBERTSON, 2014). Neste capítulo, interessa a clareza com que metas e requisitos são declarados; a organização e a manutenção dos vínculos entre esses elementos serão retomadas no Capítulo 12.

##### Exemplo:

| Meta                             | Descrição                                       | Requisitos Associados                                 | Indicadores                    |
|----------------------------------|-------------------------------------------------|-------------------------------------------------------|--------------------------------|
| Aumentar retenção de alunos      | Melhorar experiência do aluno na matrícula      | RF-01 (matrícula online), RF-03 (aviso de pendências) | Taxa de evasão                 |
| Reduzir custos administrativos   | Diminuir atendimentos presenciais na secretaria | RF-05 (upload de documentos digitais)                 | Nº de atendimentos presenciais |
| Melhorar transparência acadêmica | Ampliar acesso a dados acadêmicos               | RF-07 (consulta de notas e frequência)                | Nº de acessos ao portal        |

#### 9.4.4.2 Glossários de dados

Glossários definem termos, entidades e atributos usados nos requisitos. Segundo Sommerville (2019), são fundamentais para reduzir ambiguidades semânticas e garantir que todos compreendam os conceitos da mesma forma. Seu ponto forte é a consistência terminológica; a limitação é a manutenção contínua, pois mudanças no domínio impactam o glossário.

##### Exemplo:

| Termo      | Definição                                | Tipo/Domínio                | Observações                     |
|------------|------------------------------------------|-----------------------------|---------------------------------|
| CPF        | Número de Cadastro de Pessoa Física      | Numérico, 11 dígitos        | Validado por dígito verificador |
| Matrícula  | Identificador único de vínculo acadêmico | Alfanumérico, 12 caracteres | Gera protocolo de matrícula     |
| Disciplina | Unidade curricular de um curso           | Texto, até 100 caracteres   | Código único por instituição    |
| Frequência | Percentual de presença em aulas          | Numérico (%)                | Cálculo automático pelo sistema |

#### 9.4.4.3 Tabelas estruturadas

Tabelas estruturadas são empregadas para organizar regras, condições e cálculos de forma precisa. Elas são particularmente eficazes para requisitos de negócio que envolvem múltiplas combinações de entradas e saídas. A vantagem é a clareza visual e a redução de ambiguidades; a limitação está na dificuldade de leitura para stakeholders menos técnicos.

##### Exemplo:

| Condição                       | Ação Esperada       | Resultado                                   |
|--------------------------------|---------------------|---------------------------------------------|
| Aluno com pendência financeira | Solicitar matrícula | Sistema bloqueia matrícula e exibe mensagem |

| Condição                       | Ação Esperada                 | Resultado                                      |
|--------------------------------|-------------------------------|------------------------------------------------|
| Aluno sem pendência financeira | Solicitar matrícula           | Sistema confirma matrícula e gera protocolo    |
| Pré-requisito não atendido     | Solicitar disciplina avançada | Sistema impede matrícula e informa requisito   |
| Horário em conflito            | Selecionar disciplina         | Sistema bloqueia escolha e sugere alternativas |

#### 9.4.4.4 Catálogos de requisitos de qualidade

Os catálogos de qualidade reúnem requisitos não funcionais de maneira padronizada, voltados a como o produto deve se comportar do ponto de vista do usuário, da experiência ou da operação, por exemplo desempenho, segurança, usabilidade ou confiabilidade. A vantagem é permitir comparações e priorização entre atributos de qualidade, conforme sugerido por Wiegers e Beatty (2013). A limitação está em sua subjetividade: algumas qualidades só se tornam mensuráveis se traduzidas em critérios claros.

##### Exemplo:

| Categoria      | Requisito de Qualidade (RNF) | Medida/Métrica          | Critério de Aceitação                  |
|----------------|------------------------------|-------------------------|----------------------------------------|
| Desempenho     | Tempo de resposta            | Segundos por requisição | $\leq 2s$ em 95% das solicitações      |
| Usabilidade    | Facilidade de uso            | Nº de cliques/tarefa    | Concluir matrícula em até 3 cliques    |
| Segurança      | Confidencialidade de dados   | Tipo de criptografia    | Todas as senhas armazenadas em SHA-256 |
| Confiabilidade | Disponibilidade              | Percentual mensal       | $\geq 99,5\%$ de uptime                |

#### 9.4.4.5 Catálogos técnicos

Os catálogos técnicos apresentam requisitos não funcionais voltados a solução técnica, como o sistema deve ser construído e em que ambiente deve operar, consolidando restrições de plataforma, integrações, padrões e interfaces externas. São essenciais em contextos onde interoperabilidade e conformidade precisam ser asseguradas Dick, Hull e Jackson (2017). Seu ponto forte é a centralização de restrições; a limitação é que exigem conhecimento especializado para serem elaborados e interpretados.

##### Exemplo:

| Categoria          | Especificação Técnica            | Fonte                               | Observação            |
|--------------------|----------------------------------|-------------------------------------|-----------------------|
| Integração         | API RESTful para autenticação    | Padrão de arquitetura institucional | OAuth 2.0 obrigatório |
| Plataforma         | Banco de dados PostgreSQL v.14   | Diretriz do órgão central de TI     | Alta disponibilidade  |
| Segurança          | Criptografia TLS 1.3 obrigatória | Norma ISO/IEC 27001                 | Compatível com mobile |
| Interoperabilidade | Suporte a padrão LTI em AVA      | Regulamento MEC                     | Versão 1.3            |

O catálogo de requisitos de qualidade e o catálogo técnico abordam requisitos não funcionais (RNFs), podendo também ser apresentados em um único catálogo, classificados por exemplo, pelo URPS+, já que ambos referem-se a RNFs de produtos. O Quadro 20 apresenta uma comparação entre os dois tipos de catálogos.

| Aspecto                       | Catálogos de Requisitos de Qualidade            | Catálogos Técnicos                                        |
|-------------------------------|-------------------------------------------------|-----------------------------------------------------------|
| <b>Pergunta que respondem</b> | “Quão bom o sistema precisa ser?”               | “Com o que e sob quais regras o sistema será construído?” |
| <b>Foco</b>                   | Propriedades do produto e da experiência de uso | Infraestrutura, tecnologia e conformidade                 |
| <b>Perspectiva</b>            | Usuário, operação e manutenção                  | Arquitetura, engenharia e governança de TI                |
| <b>Verificabilidade</b>       | Por métricas (tempo, taxa, porcentagem)         | Por inspeção de aderência técnica                         |
| <b>Exemplo típico</b>         | “O sistema deve responder em até 2 segundos.”   | “O sistema deve usar banco PostgreSQL e API RESTful.”     |

Quadro 20 – Comparação entre catálogo de requisitos de qualidade e catálogo técnico.

Fonte: elaborado pelo autor (2025).

#### 9.4.4.6 Catálogos de requisitos funcionais

Catálogos de metas organizam objetivos de negócio e requisitos derivados em formato estruturado, facilitando a rastreabilidade. Essa técnica é útil para mostrar como cada requisito contribui para um resultado estratégico (LAUESEN, 2002; ROBERTSON e ROBERTSON, 2014). O ponto forte é o alinhamento com a visão organizacional, mas a limitação está na necessidade de revisão constante, já que metas podem mudar ao longo do projeto.

Os catálogos de requisitos funcionais reúnem, de forma organizada e rastreável, as funcionalidades que o sistema deve oferecer para atender aos objetivos do negócio e às necessidades dos usuários.

Diferem das narrativas e casos de uso por apresentarem as informações em formato mais sintético e estruturado, permitindo rápida consulta, manutenção e vinculação a requisitos não funcionais e critérios de aceitação.

Geralmente são representados em tabelas ou planilhas, contendo identificadores únicos, descrições resumidas, prioridade, origem, versão e estado de validação.

Esse tipo de catálogo é útil em ambientes que demandam **controle formal de escopo**, auditoria e **reutilização de requisitos** entre projetos, e pode ser integrado a ferramentas de gestão de backlog ou sistemas de gerenciamento de requisitos.

##### Exemplo de elementos típicos:

- RF-01 – Registrar novo usuário (origem: história US-05; prioridade: alta; status: aprovado)
- RF-02 – Consultar histórico de solicitações (origem: caso de uso UC-03; prioridade: média; status: em análise)

#### 9.4.4.7 Catálogos de regras de negócio

Os catálogos de regras de negócio consolidam **políticas, restrições e condições operacionais** que orientam o comportamento do sistema e determinam como as funcionalidades devem ser executadas no contexto organizacional.

Enquanto os requisitos funcionais descrevem **o que o sistema faz**, as regras de negócio definem **como e sob quais condições** essas ações ocorrem, refletindo leis, normas internas, políticas de segurança ou práticas de operação.

Sua estrutura costuma incluir identificador, nome da regra, descrição, justificativa, fonte (norma, legislação, processo interno) e impacto esperado.

Esse tipo de catálogo apoia a consistência de decisões de projeto, facilita a automação de regras em sistemas corporativos e mantém a conformidade entre requisitos e diretrizes institucionais.

##### Exemplo de elementos típicos:

- RN-01 – Todo pedido de reembolso deve ser aprovado por um gerente antes do pagamento.
- RN-02 – Usuários com perfil “Convidado” não podem acessar relatórios financeiros.

### 9.5 Relação entre níveis de abstração de requisitos e tipos de técnicas de declaração de requisitos

A variedade de técnicas apresentadas nas subseções anteriores reflete a diversidade de formas possíveis de declarar requisitos. Entretanto, essas técnicas não se aplicam de modo isolado ou rígido: elas se distribuem ao longo dos diferentes níveis de abstração de requisitos, acompanhando o amadurecimento progressivo do entendimento — do propósito organizacional até a verificação do produto.

A Figura 49 sintetiza essa correspondência, mostrando como as famílias de técnicas de declaração de requisitos se posicionam em relação aos níveis de abstração dos requisitos (apresentados na Seção 9.3).

A Figura 49 mostra que cada tipo de técnica de declaração de requisitos tem um foco predominante, mas não exclusivo, dentro dos níveis de abstração:

- As **narrativas descritivas** predominam nos requisitos de negócio e de usuário, favorecendo a compreensão de metas, intenções e contextos.
- As **declarações orientadas a valor e experiência** conectam necessidades dos usuários a resultados percebidos, apoiando priorização e alinhamento de propósito.
- As **declarações estruturadas** ocupam uma faixa de transição entre os níveis de usuário e de produto, pois traduzem comportamentos e interações em requisitos verificáveis e rastreáveis.
- Os **catálogos e artefatos técnicos** consolidam requisitos no nível de produto, organizando-os de forma formal e auditável.

|                                                                                                                                                                                                                                                                                                                                                                                              | Requisitos de Negócio | Requisitos de Usuário                | Requisitos de Produto                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|--------------------------------------|--------------------------------------|
| <b>Narrativas Descritivas</b> <ul style="list-style-type: none"> <li>Narrativas em texto livre</li> <li>Casos de uso – breve descrição</li> <li>Cenários narrativos</li> <li>Texto narrativo em linguagem natural</li> <li>Storyboards descritivos</li> </ul>                                                                                                                                | ✓<br><br><br>✓        | ✓<br>✓<br>✓<br>✓<br>✓                |                                      |
| <b>Declarações Estruturadas</b> <ul style="list-style-type: none"> <li>Listas de Requisitos</li> <li>Declarações textuais estruturadas (templates)</li> <li>Texto estruturado/tabular</li> <li>Listas estruturadas (Given/When/Then)</li> <li>Casos de uso detalhados</li> <li>Critérios de aceitação</li> <li>BDD (Behavior Driven Development)</li> <li>Checklists estruturados</li> </ul> |                       | ✓<br>✓<br>✓<br>✓<br>✓<br>✓<br>✓<br>✓ | ✓<br>✓<br>✓<br>✓<br>✓<br>✓<br>✓<br>✓ |
| <b>Declarações Orientadas a Valor</b> <ul style="list-style-type: none"> <li>Histórias de usuário</li> <li>Épicos</li> <li>Narrativas curtas ligadas ao impacto de negócio</li> </ul>                                                                                                                                                                                                        |                       | ✓<br>✓<br>✓                          | ✓                                    |
| <b>Catálogos e Artefatos Técnicos</b> <ul style="list-style-type: none"> <li>Catálogo de metas</li> <li>Glossário de dados</li> <li>Tabelas estruturadas</li> <li>Catálogo de requisitos de qualidade</li> <li>Catálogo técnico</li> <li>Catálogo de requisitos funcionais</li> <li>Catálogo de regras de negócio</li> </ul>                                                                 | ✓                     | ✓<br>✓<br>✓<br>✓<br>✓<br>✓<br>✓      | ✓<br>✓<br>✓<br>✓<br>✓<br>✓<br>✓      |

Figura 49 –Relação entre níveis de abstração de requisitos e tipos de técnicas declarativas.  
Fonte: elaborado pelo autor (2025).

A relação é fluida e complementar: técnicas de uma família podem apoiar atividades de outro nível, desde que adequadamente adaptadas. Mais importante do que a posição exata é entender a progressão, que vai do discurso narrativo e empático à declaração técnica e rastreável. Essa visão integrada evidencia como os tipos de declaração se complementam ao longo do amadurecimento dos requisitos.

A diversidade de técnicas de declaração de requisitos reflete a natureza plural da Engenharia de Requisitos: uma disciplina que busca compreender e comunicar intenções humanas por meio de declarações estruturadas.

Cada técnica apresentada — seja uma narrativa descritiva, uma declaração estruturada, uma expressão orientada a valor ou um catálogo técnico — representa uma forma distinta de tornar explícito o que se espera de um sistema, equilibrando clareza, contexto e verificabilidade.

Mais do que instrumentos isolados, essas técnicas compõem um ecossistema de comunicação. O engenheiro de software, o usuário, e o gestor encontram nelas meios de expressão complementares, capazes de adaptar-se ao momento do projeto, ao público envolvido e ao grau de maturidade do produto.

Assim, o ato de declarar um requisito não se reduz à produção de documentos, mas se traduz em um processo contínuo de externalização, validação e refinamento do entendimento compartilhado.

A escolha da técnica mais adequada depende de fatores como a abordagem de desenvolvimento, o tipo de produto, o perfil dos stakeholders e o contexto organizacional — temas que serão explorados na seção seguinte.

O essencial é reconhecer que não existe uma forma única de declarar requisitos, e sim formas apropriadas a diferentes propósitos e níveis de abstração. Essa flexibilidade é o que permite que a Engenharia de Requisitos continue sendo uma prática viva, situada e capaz de acompanhar a evolução das organizações, das equipes e das tecnologias.

## 9.6 Fatores que influenciam a forma de declaração de requisitos

A forma de declarar um requisito é antes de tudo um **ato de comunicação**. Como lembra Mike Cohn (2004), “*requisitos de software são um problema de comunicação: quem quer o software deve se comunicar com quem vai construí-lo*”. Mais do que registrar necessidades, declarar requisitos significa estabelecer um canal de entendimento mútuo entre stakeholders e equipe técnica — seja pela palavra escrita, falada, filmada ou modelada. Assim, a escolha de como declarar depende de como se comunica, com quem se comunica e em que contexto o projeto se encontra (LAPLANTE, 2017; DICK; HULL; JACKSON, 2017; SOMMERVILLE, 2019).

A literatura (WIEGERS e BEATTY, 2013; ROBERTSON e ROBERTSON, 2014; SWEBOK v4, 2024) e a prática profissional convergem em um ponto: **a forma declarativa deve se adaptar ao contexto**. Entre os principais fatores que moldam essa adaptação estão: a abordagem de desenvolvimento e o framework utilizado, o tipo e a complexidade do produto, o perfil dos stakeholders, a maturidade da equipe, a fase do projeto e o nível de granularidade com que os requisitos são expressos.

### 9.6.1 Abordagem de Desenvolvimento

A abordagem de desenvolvimento expressa a filosofia com que o projeto conduz seu ciclo de vida — se orientado por planos estáveis, por iterações adaptativas ou por uma combinação de ambos. Essa filosofia afeta diretamente como os requisitos são declarados, pois define expectativas distintas de estabilidade, detalhamento e comunicação.

Nos projetos dirigidos por plano, a prioridade é previsibilidade: os requisitos precisam ser conhecidos e estáveis o suficiente para permitir planejamento e controle de custos, prazos e qualidade. Por isso, as declarações tendem a assumir formas estruturadas e formais, como documentos de especificação, catálogos funcionais e modelos de caso de uso (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).

Essa formalidade não implica rigidez, mas busca assegurar consistência e rastreabilidade ao longo das etapas de desenvolvimento. Mesmo nesses contextos, a declaração evolui por revisões sucessivas e pela interação contínua entre analistas, projetistas e clientes — um processo de refinamento que acompanha o amadurecimento do entendimento do sistema (WIEGERS e BEATTY, 2013).

Em ambientes ágeis, a preocupação central desloca-se para a entrega incremental de valor. A comunicação ocorre de forma mais direta e contextual, e as declarações assumem formatos enxutos e compreensíveis, como histórias de usuário, épicos ou critérios de aceitação.

Esses artefatos não substituem a formalização, mas a distribuem ao longo do tempo: os detalhes surgem à medida que o conhecimento se consolida. Como observa Sommerville (2019), a agilidade não elimina a necessidade de documentação, mas redefine o seu propósito — de registro final para instrumento de diálogo contínuo.

Entre esses polos, encontram-se as abordagens híbridas, hoje predominantes na prática. Processos como o Agile Unified Process (AUP) (AMBLER, 2005) e frameworks escalados, como o Scaled Agile Framework (SAFe) (SCALED AGILE, 2023), conciliam previsibilidade e adaptação: utilizam estruturas hierárquicas de requisitos — épicos, features, histórias — mas mantêm mecanismos de rastreabilidade e controle herdados de processos mais formais (DICK; HULL; JACKSON, 2017).

Nesses cenários, a forma declarativa deve permitir transitar entre níveis de abstração, conectando objetivos estratégicos a comportamentos verificáveis do sistema.

Em síntese, a abordagem de desenvolvimento não determina um único modo de declarar requisitos, mas orienta a ênfase:

- Projetos mais previsíveis exigem declarações estáveis e auditáveis;
- Projetos mais exploratórios valorizam declarações leves e evolutivas.

Em todos os casos, o essencial é preservar a comunicação efetiva e rastreável entre quem solicita e quem constrói o software.

### 9.6.2 Estrutura de organização em que o requisito será inserido

A forma de declarar um requisito também pode ser influenciada pela estrutura utilizada para organizá-lo no conjunto do escopo. Um backlog, um catálogo de requisitos ou um mapa de histórias, por exemplo, não modificam necessariamente o conteúdo essencial do requisito, mas criam diferentes necessidades de identificação, contextualização, nível de detalhe e forma de expressão. Assim, um mesmo entendimento pode assumir declarações distintas conforme a estrutura em que será organizado e a finalidade de comunicação associada a ela.

Essa influência pode ser observada acompanhando um mesmo requisito em diferentes estruturas. Considere, por exemplo, a necessidade de permitir que um cidadão registre um agendamento para um serviço, em data e horário disponíveis. O núcleo funcional permanece o mesmo — **Registrar agendamento** —, mas sua declaração pode assumir formas diferentes conforme o contexto em que será utilizada.

Em um **backlog** que utilize histórias de usuário, esse entendimento poderia ser declarado como *US01 - Registrar agendamento: “Como cidadão, quero agendar um atendimento, para garantir meu atendimento sem precisar comparecer presencialmente apenas para realizar o agendamento.”* A forma declarativa explicita o usuário, sua intenção e o valor associado ao requisito, mantendo o item suficientemente conciso para discussão e evolução ao longo do desenvolvimento (LEFFINGWELL, 2011).

Em um **catálogo de requisitos**, o mesmo entendimento poderia ser declarado como *RF-12 - Registrar agendamento: “O sistema deve permitir que o cidadão registre um agendamento para*

*um serviço disponível em data e horário selecionados.*” Nesse caso, a identificação explícita e a formulação centrada no comportamento esperado favorecem a referência ao requisito e sua utilização em um conjunto estruturado de informações. Catálogos podem combinar a declaração do requisito com atributos adicionais pertinentes ao contexto, sem que esses atributos modifiquem seu conteúdo essencial (LAUESEN, 2002; ROBERTSON; ROBERTSON, 2014).

Em um **mapa de histórias**, a mesma história pode ser posicionada em uma estrutura que evidencia seu lugar na experiência do usuário. Por exemplo, *US01 - Registrar agendamento* pode aparecer sob uma atividade mais ampla, como *Gerenciar agendamentos*, juntamente com histórias funcionalmente relacionadas, como *US02 - Remarcar agendamento* e *US03 - Cancelar agendamento*. Nesse caso, não é necessário decompor “Registrar agendamento” em passos como selecionar serviço ou selecionar horário. O mapa acrescenta contexto à declaração ao tornar visível sua relação com outras histórias significativas da jornada ou do fluxo de uso (PATTON, 2014).

O Quadro 21 sintetiza os exemplos, mostrando como um mesmo entendimento pode ser declarado e contextualizado de maneiras diferentes conforme a estrutura utilizada para organizá-lo. Os exemplos não estabelecem correspondências obrigatórias entre estrutura e forma declarativa, mas procuram evidenciar como diferentes estruturas criam necessidades distintas de comunicação.

| Estrutura de organização      | Exemplo de declaração ou posicionamento                                                                                                                                                    | Influência sobre a declaração                                                                                                                                                                                 |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Backlog</b>                | <b>US01 — Registrar agendamento:</b> Como cidadão, quero agendar um atendimento, para garantir meu atendimento sem precisar comparecer presencialmente apenas para realizar o agendamento. | Favorece uma declaração compacta que <b>explicita</b> usuário, intenção e valor, adequada à discussão e evolução do item.                                                                                     |
| <b>Catálogo de requisitos</b> | <b>RF-12 — Registrar agendamento:</b> O sistema deve permitir que o cidadão registre um agendamento para um serviço disponível em data e horário selecionados.                             | Favorece identificação explícita e uma formulação centrada no comportamento <b>esperado</b> , permitindo que o requisito seja referenciado dentro de um conjunto estruturado.                                 |
| <b>Mapa de histórias</b>      | <b>Gerenciar agendamentos</b> → US01 - Registrar agendamento, juntamente com US02 - Remarcar agendamento e US03 - Cancelar agendamento.                                                    | Acrescenta contexto à declaração ao posicioná-la em relação à experiência do usuário e a outras histórias funcionalmente relacionadas, sem exigir a decomposição do requisito em simples passos de interação. |

Quadro 21 – Influência da estrutura de organização sobre a declaração de um requisito  
Fonte: elaborado pelo autor (2026).

A relação entre estrutura e declaração não deve ser interpretada como uma associação rígida entre determinado formato e uma abordagem de desenvolvimento. Um backlog não precisa conter exclusivamente histórias de usuário, assim como um catálogo pode utilizar diferentes formas de declaração. Da mesma maneira, histórias de usuário podem aparecer em outras estruturas



quando forem adequadas ao propósito comunicacional. A escolha depende do público, do nível de abstração, da precisão necessária e da maneira como os requisitos serão utilizados.

Em ambientes que trabalham com diferentes níveis de abstração, itens mais amplos e mais detalhados também podem coexistir em uma mesma estrutura. É possível, por exemplo, organizar épicos, *features* e histórias de usuário ou relacionar requisitos de negócio, de usuário e de produto, preservando as relações entre esses níveis sem tratá-los como etapas obrigatoriamente sequenciais (LEFFINGWELL, 2011; DICK; HULL; JACKSON, 2017).

Desse modo, a estrutura de organização atua como um dos fatores que influenciam a forma declarativa, mas não determina o significado do requisito. Neste capítulo, interessa compreender como essa estrutura pode afetar sua identificação, contextualização, nível de detalhe e forma de expressão. As diferentes estruturas utilizadas para organizar e materializar o escopo, bem como os mecanismos empregados para manter relações, versões e rastreabilidade, serão aprofundados no Capítulo 12.

### 9.6.3 Tipo de produto a ser construído

O tipo de produto também influencia como os requisitos são declarados, uma vez que diferentes domínios impõem distintos graus de formalização, precisão e adaptabilidade. Embora todo requisito deva ser completo, consistente e verificável, conforme definido nas normas e boas práticas da Engenharia de Requisitos (DICK; HULL; JACKSON, 2017; ISO/IEC/IEEE 29148:2018), o nível de detalhamento técnico e rigor na declaração varia conforme o contexto do produto e seus riscos associados.

Nos sistemas críticos — como os empregados em controle aéreo, aplicações médicas, sistemas financeiros ou de defesa —, há uma tendência de maior necessidade de formalização, rastreabilidade e auditabilidade. Nesses casos, não se trata de exigir requisitos “mais completos” ou “mais verificáveis”, mas de garantir maior precisão técnica e capacidade de comprovação de conformidade, com uso de linguagem estruturada e declarações mensuráveis. Modelos formais, catálogos de requisitos não funcionais e especificações suplementares são amplamente empregados para reduzir ambiguidade e assegurar reprodutibilidade (LAUESEN, 2002; LAPLANTE, 2017).

Nos sistemas corporativos e de gestão, onde a complexidade é organizacional e envolve múltiplos atores e regras de negócio, prevalecem declarações estruturadas e híbridas, combinando narrativas de uso, catálogos funcionais e modelos de dados (ROBERTSON e ROBERTSON, 2014; DICK; HULL; JACKSON, 2017). Esse tipo de produto requer clareza para o entendimento humano, sem perder a rastreabilidade entre processos de negócio, interações de usuário e comportamentos esperados do sistema.

Nos produtos digitais voltados ao mercado de consumo, como aplicativos móveis, plataformas e serviços web, a prioridade está em velocidade, experiência e valor percebido pelo usuário. As declarações são mais enxutas e iterativas, representando hipóteses e comportamentos: histórias de usuário, jornadas, protótipos e critérios de aceitação são artefatos típicos (CROWDER e HOFF, 2022; SOMMERVILLE, 2019). O foco desloca-se da completude inicial para a descoberta progressiva de requisitos, ajustando o produto conforme feedback real.

Por fim, produtos sob encomenda ou de caráter experimental, como soluções customizadas ou protótipos de pesquisa, demandam flexibilidade e abertura à reformulação conceitual. Nesses casos, as declarações de requisitos funcionam mais como hipóteses de valor e restrições operacionais, devendo ser continuamente revisadas à medida que o entendimento evolui (WIEGERS e BEATTY, 2013). A ênfase recai sobre a clareza da intenção e a rastreabilidade das decisões, mais do que na formalidade documental.

Em síntese, o tipo de produto define o equilíbrio entre rigor técnico e adaptabilidade comunicacional. Sistemas críticos demandam declarações formais e mensuráveis; produtos corporativos, estruturas intermediárias e rastreáveis; e produtos digitais inovadores, declarações leves e evolutivas. Como observa Lauesen (2002), *“o modo como o requisito é expresso determina a precisão com que o produto pode ser compreendido, validado e mantido”*.

#### 9.6.4 Foco do requisito

O foco do requisito define a perspectiva a partir da qual o comportamento desejado do sistema é expresso. Cada declaração de requisito reflete uma posição cognitiva e comunicacional — isto é, de quem observa, para quem comunica e o que considera essencial representar. Dessa escolha decorrem diferenças no nível de abstração, na linguagem empregada e na forma de validação dos requisitos.

Segundo Pohl (2010), o foco adotado na declaração influencia diretamente a natureza da informação registrada e sua utilidade para diferentes públicos. Três focos predominam na prática: centrado no usuário, centrado na solução e centrado na funcionalidade.

- **Foco no usuário:**
  - ✓ A declaração parte da perspectiva de quem utiliza o sistema, descrevendo intenções, objetivos e experiências esperadas.
  - ✓ As formas mais comuns incluem histórias de usuário, cenários narrativos e critérios de aceitação, que privilegiam a compreensão e o alinhamento de propósito (ROBERTSON e ROBERTSON, 2014; ALEXANDER e BEUS-DUKIC, 2009).
  - ✓ O vocabulário é natural e contextual, facilitando o diálogo entre usuários e equipe técnica, ainda que exija complementação posterior para atingir o nível de verificabilidade necessário.
- **Foco na solução:**
  - ✓ O requisito é declarado a partir do comportamento do sistema, descrevendo o que ele deve fazer para atender às necessidades identificadas.
  - ✓ Esse tipo de declaração é frequente em casos de uso, modelos de interação e catálogos funcionais, nos quais o sistema é o sujeito da ação (DICK; HULL; JACKSON, 2017).
  - ✓ O foco na solução favorece clareza técnica e rastreabilidade com o projeto de software, mas pode ocultar o propósito original do requisito se não for vinculado à necessidade que o originou.
- **Foco na funcionalidade:**
  - ✓ A declaração enfatiza funções específicas e condições de operação, traduzindo comportamentos e restrições em linguagem verificável.

- ✓ Esse tipo de foco é comum em catálogos técnicos e requisitos não funcionais, nos quais predominam parâmetros quantitativos e critérios de desempenho (DICK; HULL; JACKSON, 2017).
- ✓ Embora essencial para o detalhamento e teste, um foco excessivamente funcional pode reduzir a compreensão do valor e do contexto de uso.

Esses três focos não são excludentes — ao contrário, formam uma cadeia de complementação entre níveis de abstração. Um mesmo requisito pode ser inicialmente declarado do ponto de vista do usuário, depois traduzido em termos de solução e finalmente descrito como funcionalidade técnica.

O engenheiro de software deve compreender e manter essas relações, assegurando que a evolução da forma declarativa não rompa a rastreabilidade entre intenção, comportamento e implementação (WIEGERS e BEATTY, 2013; POHL, 2010).

Em suma, o foco do requisito orienta a natureza e a função da declaração. Enquanto o foco no usuário comunica valor e propósito, o foco na solução expressa comportamento e interação, e o foco na funcionalidade define execução e verificação. Declarar requisitos de modo eficaz exige transitar conscientemente entre esses níveis, mantendo coerência semântica e alinhamento de propósito em todo o ciclo de vida do produto.

#### **Box 9.6.4-1 – Foco do requisito versus foco no usuário**

Esse trânsito explícito é uma aplicação direta do **Princípio 1 (Orientação a valor)**: a declaração não é fim; ela preserva a rastreabilidade entre propósito (valor), comportamento (solução) e verificabilidade (funcionalidade) com o menor custo comunicacional possível (**Simplicidade**).

### **9.6.5 Proximidade com o cliente**

A proximidade com o cliente é outro fato que influencia como os requisitos são declarados e mantidos ao longo do projeto. O modo de comunicação, o nível de formalização dos artefatos e a frequência das revisões variam conforme o grau de interação entre quem define o problema e quem constrói a solução. Essa relação determina, em grande medida, a fluidez da elicitação e descoberta, e a estabilidade das declarações.

Quando há **alta proximidade** — como em projetos co-localizados, com comunicação frequente e acesso direto aos usuários —, as declarações podem ser mais leves e iterativas, pois o entendimento é sustentado por interações contínuas (WIEGERS e BEATTY, 2013; ROBERTSON e ROBERTSON, 2014). Nesse contexto, histórias de usuário, cenários narrativos e critérios de aceitação são preferíveis, pois funcionam como *prompts* para a conversação e o aprendizado conjunto. A informalidade relativa desses registros é compensada pela velocidade de validação e pela presença constante do cliente no processo decisório.

#### Box 9.6.5-1 – Alta proximidade versus baixa proximidade

Nesses cenários, a declaração também realiza o **Princípio 7 (Validação de requisitos)** ao viabilizar auditoria, verificação posterior e gestão de mudança com base rastreável — reforçando **Compromisso** entre as partes.

Nos contextos de **baixa proximidade**, em que a comunicação ocorre de forma mediada — por distância geográfica, múltiplos níveis hierárquicos ou restrições contratuais —, a declaração tende a ser mais estruturada, padronizada e mantida ao longo do ciclo de vida do projeto. São comuns catálogos de requisitos, casos de uso e documentos de especificação baseados em padrões reconhecidos, como a ISO/IEC/IEEE 29148:2018, que garantem clareza e rastreabilidade mesmo na ausência de diálogo frequente (SOMMERVILLE, 2019; DICK; HULL; JACKSON, 2017).

Nesses casos, a declaração pode também assumir um papel contratual e de controle, servindo como base para verificações, auditorias e gestão de mudanças.

#### Box 9.6.5-2 – Alta proximidade versus baixa proximidade

Este gradiente concretiza o **Princípio 2 (Comunicação dos requisitos)**: o formato não é dogma, é escolha comunicacional orientada por contexto, risco e frequência de interação — com **Feedback** como contrapeso à informalidade e com **clareza/rastreabilidade** como contrapeso à distância.

**Situações intermediárias** — típicas de equipes distribuídas ou de produtos desenvolvidos por múltiplos fornecedores — requerem uma combinação de ambos os estilos: comunicação digital contínua (reuniões, quadros virtuais, protótipos navegáveis) associada a declarações formais de referência. Segundo Young (2004), essa dualidade é inevitável em ambientes complexos: a efetividade depende menos do formato e mais da consistência entre os meios de comunicação e o nível de confiança entre as partes.

O grau de proximidade, portanto, não define apenas *quanto* se declara, mas *como* se declara. Quanto maior a distância entre cliente e equipe, maior tende a ser a necessidade de formalização, clareza e manutenção das declarações. Quanto mais próxima a colaboração, maior tende a ser a ênfase em compreensão compartilhada, iteração e adaptação rápida (POHL, 2010; ROBERTSON e ROBERTSON, 2014).

Desta forma, a proximidade com o cliente orienta o equilíbrio entre flexibilidade comunicacional e rigor da declaração. Projetos de alta interação privilegiam artefatos leves e conversacionais; projetos de baixa interação requerem declarações explícitas e rastreáveis. O papel do engenheiro de software é reconhecer esse gradiente e ajustar o formato declarativo de modo a manter o entendimento claro e verificável, independentemente da distância relacional.

### 9.6.6 Perfil do cliente

O perfil do cliente molda a forma como os requisitos são declarados, pois define o modo de compreender, validar e priorizar as informações. Aspectos como formação técnica, experiência em projetos de software, linguagem organizacional, estilo de comunicação e preferências

cognitivas influenciam a escolha das técnicas e o grau de formalização necessários para assegurar entendimento mútuo.

Clientes com **perfil técnico** — engenheiros, analistas de sistemas ou especialistas de domínio — tendem a compreender melhor declarações estruturadas, catálogos e especificações mais detalhadas. Para esses perfis, a clareza deriva da precisão e do controle terminológico: a comunicação flui por meio de artefatos padronizados, como tabelas de requisitos, modelos de dados e descrições condicionais (WIEGERS e BEATTY, 2013; POHL, 2010). A linguagem usada é objetiva e operacional, favorecendo a identificação de lacunas e inconsistências.

Clientes com perfil **não técnico** — gestores, usuários finais ou representantes de negócio — necessitam de declarações mais narrativas e contextuais. Formas como histórias de usuário, storyboards, cenários de uso podem ser mais adequadas, pois permitem compreender o produto e seu propósito sem recorrer a jargões técnicos (ROBERTSON e ROBERTSON, 2014; ALEXANDER e BEUS-DUKIC, 2009). Nesses casos, o valor da declaração está em promover compreensão compartilhada e reduzir ambiguidades.

Além dessas diferenças, há situações em que o cliente apresenta necessidades específicas de **acessibilidade** que impactam diretamente o formato das declarações. Pessoas com deficiência visual, por exemplo, demandam artefatos compatíveis com leitores de tela (áudio) e linguagens verbais claras. Clientes com deficiência auditiva podem preferir declarações escritas e visuais, acompanhadas de materiais complementares em formato textual ou legendado. Aqueles com dificuldades de leitura, dislexia ou deficiência intelectual leve se beneficiam de declarações em linguagem simples, estruturadas em blocos curtos e apoiadas por ícones ou fluxos visuais.

Essas condições não alteram o conteúdo dos requisitos, mas influenciam o meio e a forma como eles são declarados e comunicados, reforçando a importância de práticas inclusivas. Segundo Young (2004) e Sommerville (2019), a Engenharia de Requisitos é tanto técnica quanto humana: deve garantir que todos os envolvidos possam compreender, participar e validar o produto, independentemente de limitações sensoriais, cognitivas ou culturais.

Além disso, fatores organizacionais e culturais também afetam o estilo de declaração. Ambientes hierárquicos podem priorizar registros escritos e aprovações formais, enquanto equipes colaborativas preferir comunicação visual e iterações curtas.

O nível de confiança e de familiaridade do cliente com o processo de desenvolvimento influencia o grau de detalhamento necessário: quanto menor a familiaridade, maior a importância de exemplos concretos e linguagem acessível.

Adaptar o formato das declarações ao perfil e às condições do cliente é, portanto, uma competência essencial da Engenharia de Requisitos (DICK; HULL; JACKSON, 2017). Mais do que traduzir termos técnicos, trata-se de adequar a linguagem, o meio e o nível de abstração à capacidade interpretativa de cada interlocutor, garantindo inclusão, clareza e compreensão comum.

Clientes técnicos tendem a requerer maior estrutura e rastreabilidade, enquanto clientes não técnicos ou com necessidades específicas demandam acessibilidade e representações

compreensíveis. Cabe ao engenheiro de software equilibrar essas dimensões para garantir precisão, clareza e participação efetiva de todos os envolvidos

### 9.6.7 Características da equipe técnica

As características da equipe técnica influenciam a forma de declarar requisitos, pois determinam o grau de experiência, domínio das práticas e coerência coletiva alcançado no processo. Competências técnicas, estabilidade do time e cultura organizacional tendem a afetar a qualidade das declarações e a capacidade de manter entendimento comum ao longo do ciclo de desenvolvimento.

Equipes mais experientes em Engenharia de Requisitos, ou seja, com maior domínio das práticas, costumam empregar estruturas declarativas bem definidas, como templates, glossários e checklists de avaliação da qualidade. Esses instrumentos ajudam a reduzir ambiguidades, manter uniformidade na terminologia e facilitar a revisão colaborativa (WIEGERS e BEATTY, 2013; POHL, 2010). Além disso, tais equipes compreendem melhor a diferença entre declarar, analisar e representar requisitos, aplicando cada tipo de artefato no momento adequado do processo.

Equipes menos experientes tendem a produzir declarações que não comunicam claramente o que se pretende construir ou por que aquilo é importante. Quando faltam estrutura e uniformidade na declaração, cada pessoa pode interpretar o requisito de modo diferente — e as discussões tornam-se dispersas, repetitivas e pouco produtivas. A adoção de padrões mínimos pode ajudar a garantir que todos compreendam o mesmo significado e saibam a que necessidade o requisito se refere. Mais do que padronizar, esses elementos facilitam o diálogo entre áreas técnicas e de negócio, reduzindo dúvidas e criando uma base comum para revisar e aprimorar as declarações ao longo do projeto.

A qualidade da declaração está, portanto, na sua clareza, utilidade e capacidade de gerar entendimento compartilhado, e não apenas na forma como é documentada (DICK; HULL; JACKSON, 2017; WIEGERS e BEATTY, 2013).

A composição e diversidade da equipe também influenciam o modo de declarar. Times multidisciplinares — que reúnem desenvolvedores, designers, analistas e profissionais de negócio — exigem declarações acessíveis e baseadas em linguagem comum. Nesses ambientes, elementos visuais, protótipos e quadros colaborativos podem apoiar a compreensão das declarações, mas não as substituem: são formas de representação utilizadas para validar interpretações e alinhar expectativas (CROWDER e HOFF, 2022; SOMMERVILLE, 2019). Já equipes compostas predominantemente por profissionais técnicos tendem a preferir declarações mais estruturadas e rastreáveis, próximas da modelagem funcional.

A cultura organizacional também exerce influência sobre a forma de declarar requisitos, embora essa distinção tenha se tornado mais fluida ao longo do tempo. Tradicionalmente, instituições com forte base regulatória — como bancos, empresas de energia e órgãos públicos — valorizavam documentos formais e rastreáveis, adequados a auditorias e processos de conformidade.

Contudo, nas últimas décadas, essas mesmas organizações vêm incorporando práticas ágeis e colaborativas, buscando reduzir distâncias entre áreas de negócio e equipes técnicas.

Hoje, é comum encontrar estruturas híbridas, nas quais rigor e adaptabilidade convivem: declarações formais seguem existindo, mas são complementadas por artefatos mais leves, como histórias de usuário, protótipos e backlogs evolutivos, que permitem maior transparência e aprendizado contínuo (YOUNG, 2004; SOMMERVILLE, 2019; DICK; HULL; JACKSON, 2017).

As características da equipe técnica expressam, em última instância, sua capacidade coletiva de transformar conhecimento disperso em entendimento comum. Cabe ao engenheiro de software reconhecer o perfil e a experiência da equipe, selecionando formas declarativas que promovam clareza, disciplina e colaboração, mantendo a integridade do processo sem comprometer sua flexibilidade.

### 9.6.8 Momento do produto e necessidade de uso

A forma de declarar um requisito também pode variar conforme o entendimento disponível sobre ele e, principalmente, conforme a finalidade para a qual sua declaração será utilizada. Em situações mais exploratórias, nas quais ainda se busca compreender necessidades, comportamentos ou possibilidades, formulações mais amplas podem ser suficientes para sustentar a conversa. Quando o mesmo requisito passa a orientar decisões mais específicas, desenvolvimento ou avaliação, podem ser necessárias informações adicionais, maior estruturação ou condições mais precisas. Essa variação não significa que todo requisito deva percorrer uma sequência predeterminada de formas declarativas, mas que sua expressão deve ser adequada ao uso que precisa sustentar em determinado momento (ROBERTSON; ROBERTSON, 2014; WIEGERS; BEATTY, 2013; POHL, 2010).

Considere, por exemplo, o requisito **Registrar agendamento**, utilizado anteriormente neste capítulo. Em uma situação ainda exploratória, na qual stakeholders e equipe procuram compreender a necessidade e o comportamento esperado, pode ser suficiente registrar:

**Registrar agendamento:** permitir que o cidadão agende um atendimento para um serviço disponível.

Essa formulação torna explícito o comportamento central sem antecipar detalhes que ainda não são necessários ou suficientemente compreendidos. Ela permite discutir quem realiza o agendamento, para quais serviços ele se aplica, como a disponibilidade será percebida e que resultado se espera dessa interação.

Quando esse mesmo requisito precisa orientar mais diretamente o desenvolvimento, sua declaração pode incorporar informações já compreendidas como relevantes:

**RF-12 - Registrar agendamento:** o sistema deve permitir que o cidadão registre um agendamento para um serviço disponível, selecionando uma data e um horário dentre as opções oferecidas.

O requisito continua expressando o mesmo núcleo de sentido — o cidadão registra um agendamento —, mas a declaração oferece informações adicionais que ajudam a orientar o comportamento a ser implementado. Isso não significa que a formulação anterior estivesse incorreta; significa que a necessidade de uso da declaração mudou e que o entendimento disponível permite torná-la mais precisa.

Em situações nas quais esse requisito precise sustentar avaliação, conformidade ou decisões para as quais determinadas condições sejam particularmente relevantes, sua declaração pode exigir maior precisão. Por exemplo:

**RF-12 - Registrar agendamento:** o sistema deve permitir que o cidadão registre um agendamento para um serviço disponível, selecionando uma data e um horário dentre aqueles que possuam vagas. Após a confirmação, o sistema deve registrar o agendamento e apresentar ao cidadão sua identificação, data, horário e unidade de atendimento.

Nesse caso, condições e resultados que poderiam permanecer abertos durante uma exploração inicial tornam-se necessários para o uso pretendido da declaração. A diferença não está simplesmente em produzir um texto mais longo, mas em explicitar aspectos cujo conhecimento passou a ser relevante para orientar decisões ou permitir que o comportamento esperado seja examinado com maior precisão.

O exemplo evidencia que **momento do produto e necessidade de uso não devem ser interpretados como fases rígidas de amadurecimento de todos os requisitos**. Em um mesmo produto, pode haver requisitos ainda expressos de maneira exploratória, outros suficientemente detalhados para orientar desenvolvimento e outros que exigem maior precisão por sua criticidade, risco ou finalidade de avaliação. O grau de estruturação deve acompanhar o propósito comunicacional, o nível de entendimento disponível e a precisão necessária em cada situação (DICK; HULL; JACKSON, 2017; SOMMERVILLE, 2019).

### 9.6.9 Resumo dos fatores de influência na declaração de requisitos

Ao longo desta seção foram apresentados os principais fatores que condicionam a forma de declarar requisitos. Cada um deles expressa dimensões complementares — técnicas, humanas e contextuais — que definem como a informação sobre o produto é tornada explícita, compreensível e compartilhável. A escolha adequada do tipo e do formato da declaração depende, portanto, de reconhecer essas variáveis e ajustar o nível de formalidade, linguagem e meio de comunicação às características do projeto.

O Quadro 22 apresenta as principais características de cada fator e seus efeitos sobre a escolha do tipo de declaração a ser realizada.

| Fator                                                            | Aspecto de influência                                                                                     | Efeitos sobre a escolha do tipo de declaração                                                                                                                                                 |
|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Abordagem de desenvolvimento</b>                              | Reflete a filosofia de trabalho (dirigida por plano, ágil ou híbrida).                                    | Abordagens dirigidas por plano favorecem declarações mais formais e completas; abordagens ágeis priorizam declarações curtas e evolutivas; abordagens híbridas combinam ambos os estilos.     |
| <b>Estrutura de organização em que o requisito será inserido</b> | Considera como o requisito será identificado, contextualizado e relacionado a outros elementos do escopo. | Backlogs, catálogos e mapas de histórias, entre outros, podem demandar diferentes formas de identificação, contextualização e nível de detalhe, sem determinar uma forma única de declaração. |



| Fator                                          | Aspecto de influência                                                               | Efeitos sobre a escolha do tipo de declaração                                                                                                                                                                                   |
|------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tipo de produto a ser construído</b>        | Determina o nível de rigor e formalização exigido.                                  | Produtos críticos demandam declarações precisas e auditáveis; produtos corporativos equilibram estrutura e clareza; produtos digitais e inovadores utilizam declarações mais leves e centradas na experiência.                  |
| <b>Foco do requisito</b>                       | Define a perspectiva predominante da declaração (usuário, solução, funcionalidade). | Foco no usuário privilegia narrativas e histórias; foco na solução usa casos de uso e catálogos funcionais; foco na funcionalidade adota linguagem estruturada e mensurável.                                                    |
| <b>Proximidade com o cliente</b>               | Indica a frequência e a qualidade da interação com stakeholders.                    | Alta proximidade permite declarações breves e iterativas; baixa proximidade requer registros mais estruturados, padronizados e de referência estável.                                                                           |
| <b>Perfil do cliente</b>                       | Considera o nível técnico, cognitivo e as condições de acessibilidade.              | Clientes técnicos preferem linguagem precisa e estruturada; clientes não técnicos e com necessidades específicas demandam linguagem simples, visual e acessível, apoiada por exemplos e descrições contextuais.                 |
| <b>Características da equipe técnica</b>       | Relaciona-se à experiência, composição e cultura da equipe.                         | Equipes mais experientes utilizam estruturas claras e terminologia uniforme; equipes em formação dependem de guias e padrões mínimos; equipes multidisciplinares requerem linguagem comum e apoio visual para alinhamento.      |
| <b>Momento do produto e necessidade de uso</b> | Considera o propósito comunicacional e o nível de entendimento disponível.          | Situações exploratórias admitem declarações mais amplas; usos que sustentam decisões, implementação ou avaliação podem exigir maior precisão e estruturação. Diferentes graus de detalhamento podem coexistir no mesmo produto. |

Quadro 22 – Fatores que influenciam a declaração de requisitos.  
Fonte: elaborado pelo autor (2025).

O conjunto desses fatores mostra que a declaração de requisitos não é uma atividade isolada, mas um processo situado, moldado pelas condições do projeto e pelas pessoas envolvidas. Compreender essas interdependências é essencial para garantir que os requisitos sejam expressos de modo claro, rastreável e ajustado à realidade de cada contexto — princípios que sustentam a qualidade e a evolução contínua das declarações, tema abordado na próxima seção.

## 9.7 Escolher é equilibrar: adequação e qualidade das declarações

Depois de compreender os fatores que moldam a forma de declarar requisitos, surge uma pergunta inevitável: *“como escolher o tipo de declaração mais adequada ao meu projeto?”*

Responder a essa pergunta exige mais do que conhecer técnicas. A escolha da forma declarativa não é um ato prescritivo, mas um exercício de **interpretação e equilíbrio**. Ela depende da leitura que o engenheiro de software faz do contexto, das pessoas envolvidas e do propósito comunicacional que se deseja alcançar. Não existe uma correspondência direta entre tipo de projeto e tipo

de declaração; o que existe são **relações de adequação situadas**, nas quais o formato certo é aquele que cumpre sua função de comunicar, alinhar e orientar decisões.

### 9.7.1 A escolha é um ato de mediação

Escolher a forma de declarar é, antes de tudo, **mediar linguagens**. Cada grupo de stakeholders se expressa de modo diferente: usuários falam sobre necessidades e frustrações, analistas sobre comportamento, desenvolvedores sobre implementação.

A declaração funciona como uma ponte entre esses mundos, permitindo que todos conversem sobre o mesmo sistema, ainda que partindo de perspectivas distintas (DICK; HULL; JACKSON, 2017; ROBERTSON e ROBERTSON, 2014).

Nesse sentido, a decisão sobre “*como declarar*” não é técnica no sentido restrito, mas **comunicacional e relacional**. Um modelo de caso de uso, uma história de usuário ou um catálogo funcional são, antes de tudo, **formas de diálogo**.

A boa escolha é aquela que traduz intenções em enunciados que possam ser compreendidos e validados por todos os envolvidos. Como observa Young (2004), o papel do engenheiro de software é criar um espaço em que diferentes vozes possam ser ouvidas e articuladas sem perda de sentido.

### 9.7.2 Adequação depende de propósito, não de formato

A qualidade de uma declaração está menos no formato escolhido e mais na adequação entre forma e propósito. Cada momento do projeto requer um tipo de comunicação distinto:

- Quando o objetivo é **explorar o problema**, declarações narrativas, abertas e descritivas favorecem a descoberta de sentido;
- Quando o objetivo é **especificar comportamentos**, declarações estruturadas permitem detalhar interações e condições;
- Quando o objetivo é **verificar e testar**, declarações mensuráveis e rastreáveis asseguram controle e validação (WIEGERS e BEATTY, 2013; POHL, 2010).

Portanto, o engenheiro de software deve começar pelo propósito da comunicação e não pelo artefato disponível.

A pergunta correta não é “*qual técnica usar?*”, mas “*qual tipo de compreensão essa declaração precisa gerar?*”. Essa mudança de foco transforma a escrita de requisitos em uma atividade reflexiva, ajustada ao aprendizado coletivo.

### 9.7.3 Combinar é mais realista do que escolher

Na prática, projetos raramente adotam uma única forma declarativa. O que se observa é uma **combinação de formatos**, cada um ocupando uma função dentro da hierarquia de abstração: narrativas para alinhar propósitos, casos de uso para detalhar comportamentos, catálogos para consolidar requisitos técnicos. Essa coexistência é inevitável e desejável, desde que exista coerência entre as formas.

O desafio está em **articular essas declarações como partes de um mesmo discurso**. Quando a história de usuário remete ao caso de uso, que por sua vez se conecta a um requisito funcional no catálogo, cria-se uma **trilha de sentido** entre a intenção do usuário e a implementação no produto (ROBERTSON e ROBERTSON, 2014; DICK; HULL; JACKSON, 2017). O papel do engenheiro de software é manter essa continuidade, garantindo que a diversidade de artefatos não produza ruído, mas **riqueza interpretativa**.

#### 9.7.4 A qualidade está na função comunicacional

Avaliar a qualidade de uma declaração não é medir o quanto ela é formal, extensa ou detalhada, mas o quanto ela cumpre sua função de comunicação. Uma boa declaração é aquela que:

- Pode ser compreendida igualmente por diferentes públicos;
- Expressa com precisão a intenção e o comportamento esperados;
- Mantém o mesmo significado mesmo após revisões e discussões.

Em outras palavras, a qualidade está na capacidade da declaração de sustentar entendimento compartilhado.

Não basta ser correta: é preciso ser **compreensível, discutível e útil**. Como sintetiza Pohl (2010), o requisito só se torna realmente “engenheirado” quando deixa de ser apenas informação registrada e passa a ser entendimento consolidado entre as partes.

#### 9.7.5 O papel reflexivo do engenheiro de software

A escolha da forma de declarar um requisito exige do engenheiro de software uma postura **reflexiva e adaptativa**. Ele atua menos como autor de documentos e mais como **curador do entendimento**: seleciona o meio e o formato que melhor conectam **intenção, comportamento e solução**, conforme o **estágio do projeto** e as **pessoas envolvidas**.

Essa prática requer **escuta ativa**, domínio de **múltiplas linguagens** e **sensibilidade** para perceber quando a declaração deixa de cumprir sua função comunicativa.

Como ressaltam Dick, Hull e Jackson (2017), a Engenharia de Requisitos não busca eliminar a variabilidade, mas **aprender a trabalhar com ela**.

Ser engenheiro de software, nesse contexto, é gerir a ambiguidade de forma produtiva — transformando diversidade de percepções em coerência de propósito.

Neste sentido, escolher a forma de declarar não é um ponto de chegada, mas um processo de ajuste contínuo. Cada nova interação, revisão ou decisão de projeto redefine o que significa “clareza”, “suficiência” e “adequação”.

Reconhecer esse movimento é o que diferencia a **engenharia de requisitos mecânica** (técnica), baseada em formatos fixos, da **engenharia de requisitos reflexiva**, orientada à aprendizagem e à construção colaborativa do entendimento.

Nesse equilíbrio entre técnica e interpretação, a declaração deixa de ser apenas registro e passa a ser **ato de mediação** — o ponto em que o conhecimento sobre o produto se torna, enfim, compreensível e compartilhável.

## 9.8 Integração e evolução das declarações

Se nas seções anteriores foi mostrado que a forma de declarar requisitos depende do contexto e das pessoas envolvidas, aqui o foco passa a ser **como essas formas se conectam e se transformam ao longo do tempo**.

As declarações não são peças isoladas, mas partes de um mesmo diálogo que se desenvolve conforme o projeto amadurece. Cada técnica — narrativa, estruturada ou técnica — contribui com uma visão complementar sobre o produto, e o papel do engenheiro de software é garantir que essas múltiplas vozes formem um discurso coerente.

A integração das declarações ocorre, portanto, tanto no plano **conceitual** (onde o significado é compartilhado e negociado) quanto no plano **informacional** (onde artefatos são vinculados, atualizados e rastreados).

Essa dinâmica faz da Engenharia de Requisitos não apenas um processo de registro, mas um processo de tradução contínua entre formas, linguagens e intenções (DICK; HULL; JACKSON, 2017; DICK; HULL; JACKSON, 2017).

### 9.8.1 Complementaridade entre técnicas

As diversas técnicas de declaração de requisitos — narrativas, estruturadas ou formais — não devem ser vistas como alternativas excludentes, mas como **partes complementares de um mesmo ecossistema comunicacional**. Cada técnica se apoia em um modo distinto de expressar entendimento: umas privilegiam o contexto e o valor, outras enfatizam comportamento e rastreabilidade, e algumas consolidam o conhecimento em registros auditáveis. Juntas, formam uma trama contínua de representação do sentido: da descoberta de intenções até a especificação verificável do sistema.

Nos estágios iniciais do projeto, **técnicas narrativas** — como storyboards descritivos, relatos de oficinas, cenários de uso e narrativas em linguagem natural — desempenham papel essencial na **exploração e negociação de significados**. Elas capturam nuances humanas e contextuais que dificilmente emergem em formatos rígidos, promovendo empatia, imaginação e alinhamento de propósito (ALEXANDER e BEUS-DUKIC, 2009; ROBERTSON e ROBERTSON, 2014). Essas formas de declaração são eficazes para conectar objetivos organizacionais e experiências de usuário, atuando como ponto de partida para a formalização posterior.

À medida que o entendimento amadurece, **técnicas estruturadas** entram em cena para **traduzir intenções em comportamentos observáveis**. Modelos como listas Given/When/Then, templates de requisito, critérios de aceitação e casos de uso fornecem consistência semântica e facilitam a verificação (WIEGERS e BEATTY, 2013; DICK; HULL; JACKSON, 2017). Essas técnicas operam como uma ponte entre linguagem natural e linguagem técnica, mantendo a legibilidade humana enquanto introduzem estrutura lógica e controle de rastreabilidade.

Nas etapas seguintes, **catálogos e artefatos técnicos** consolidam o conhecimento acumulado. Catálogos de requisitos de qualidade, glossários de dados e tabelas estruturadas oferecem uma base formal para rastreabilidade e auditoria (DICK; HULL; JACKSON, 2017; SWEBOK, 2024). Eles funcionam como o ponto de estabilização da declaração — um repositório onde o que foi

discutido, compreendido e validado se torna referência verificável para o desenvolvimento e a manutenção.

Essas famílias de técnicas se **alimentam mutuamente**: o que nasce como narrativa de valor pode ser refinado em uma declaração estruturada e, mais tarde, consolidado em um catálogo técnico. Um exemplo dessa continuidade é o encadeamento entre histórias de usuário, casos de uso e catálogos de requisitos, que representam sucessivas camadas de formalização sem ruptura de significado (COHN, 2004; PATTON, 2014; JACOBSON et al., 2024). Cada forma retém parte do sentido da anterior e adiciona precisão ou verificabilidade à medida que o produto se desenvolve.

Como observa Lauesen (2002), a força da Engenharia de Requisitos está em manter coerência entre diferentes estilos de expressão — o que ele chama de *thread of understanding*. Essa continuidade não é apenas sintática, mas cognitiva: o requisito mantém sua identidade mesmo quando muda de formato.

Sommerville (2019) e o SWEBOK v4 (2024) reforçam que a combinação de técnicas é não apenas inevitável, mas necessária para lidar com a diversidade de domínios, produtos e pessoas. O valor não está em escolher uma técnica “melhor”, mas em **saber combiná-las de modo que o conjunto expresse o entendimento coletivo de forma clara e rastreável**.

Assim, a complementaridade entre técnicas é menos sobre substituição e mais sobre **sinergia entre modos de pensar**. Cada técnica traz consigo uma epistemologia: a narrativa favorece empatia e descoberta; a estruturação promove precisão e verificabilidade; a catalogação assegura estabilidade e reuso.

O engenheiro de software atua como **intérprete entre esses modos**, garantindo que a passagem de uma forma para outra preserve o significado e o valor das intenções originais. É nesse equilíbrio entre expressão humana e formalização técnica que se revela a maturidade da prática declarativa.

### 9.8.2 Evolução temporal e revisões sucessivas das declarações

As **declarações de requisitos** não são entidades fixas, mas expressões situadas de um entendimento que evolui conforme novas perspectivas e aprendizados emergem. Cada reformulação reflete o grau de consenso momentâneo e a maturidade interpretativa alcançada pelas pessoas envolvidas.

Com o avanço do projeto, a experiência prática, a experimentação com protótipos e a interação contínua com os stakeholders refinam o modo como as necessidades são percebidas e comunicadas. Essa evolução temporal é parte intrínseca da natureza social e cognitiva da Engenharia de Requisitos (DICK; HULL; JACKSON, 2017; DICK; HULL; JACKSON, 2017).

As revisões sucessivas das declarações não indicam erro, mas **aprendizado coletivo e amadurecimento de compreensão**. Cada iteração — seja uma reformulação escrita, uma nova explicação verbal ou uma versão revisada de um artefato audiovisual — representa um passo na consolidação do entendimento compartilhado.

Como observam Wiegers e Beatty (2013), revisões bem conduzidas reduzem ambiguidades e aumentam a precisão sem eliminar a diversidade de interpretações que sustenta a colaboração. A clareza não nasce da forma, mas da *reconciliação progressiva de significados* entre diferentes perspectivas.

Essa evolução assume várias formas. Declarações inicialmente narrativas podem se tornar estruturadas; listas verbais podem ser reorganizadas em formatos tabulares; e gravações de discussões podem originar declarações mais formais.

Em outros casos, declarações anteriores são revisitadas quando **o contexto de compreensão se renova** — por exemplo, diante de revisões regulatórias, reinterpretações de objetivos ou ajustes técnicos. Cada reformulação é, em essência, **um novo ato de interpretação**, que reconstrói o entendimento à luz de um contexto em transformação (ROBERTSON e ROBERTSON, 2014; SOMMERVILLE, 2019).

Tratar as declarações como **artefatos vivos** significa compreender que elas são continuamente lidas, ouvidas, vistas e reinterpretadas. Fernandes e Machado (2015) enfatizam que o valor de uma declaração não está apenas no que ela expressa, mas na **trajetória de aprendizado que carrega**.

Ao revisitar registros anteriores — sejam atas de oficinas, gravações de reuniões ou versões intermediárias de requisitos — a equipe não apenas compara conteúdos, mas reflete sobre *como o entendimento se formou* ao longo do tempo. Esse processo gera uma **memória interpretativa**, fundamental para compreender por que determinadas decisões foram tomadas e como certas necessidades se consolidaram.

A evolução das declarações é também um fenômeno **relacional**. Cada revisão resulta de novas conversas, interpretações e negociações entre diferentes papéis de engenharia. Crowder e Hoff (2022) descrevem esse processo como *sensemaking coletivo*: um ciclo de reinterpretação social em que o conhecimento se estabiliza apenas temporariamente, até que novas interações o reconfigurem. Nesse contexto, o engenheiro de software atua como **facilitador do entendimento**, ajudando a preservar o vínculo com o propósito original, sem impedir a evolução natural da interpretação.

Conduzir essa evolução requer equilíbrio entre **liberdade interpretativa e consistência conceitual**. Como argumenta Laplante (2017), revisões e rastreabilidade são importantes para manter coerência, mas o controle excessivo pode engessar a dinâmica de aprendizado. O desafio está em **preservar o histórico e as razões das reformulações** sem bloquear a experimentação e o diálogo. Quando bem conduzida, a evolução das declarações se torna um ciclo virtuoso de comunicação, no qual cada revisão aprofunda o entendimento e reforça o compromisso coletivo com o valor que o sistema deve entregar.

#### 9.8.2.1 A base de entendimento: o núcleo de sentido

A distinção entre amadurecimento e mudança depende da **preservação ou ruptura do núcleo de sentido** de um requisito — a intenção fundamental que define seu propósito no sistema. Esse núcleo pode ser identificado pelo **nome do requisito**, geralmente formulado como *verbo no infinitivo + objeto*, como em *realizar pagamento*, *consultar relatório* ou *enviar notificação*.

Enquanto essa ação essencial se mantém, o requisito é o mesmo; quando o verbo ou o objeto se alteram, há mudança de requisito.

Assim, o amadurecimento ocorre quando o entendimento se aprofunda **dentro do mesmo núcleo de sentido** — refinando regras, condições e critérios de aceitação. Já a mudança ocorre quando há **ruptura** desse núcleo, transformando a ação principal ou seu propósito. Essa noção de núcleo de sentido funciona como **referência semântica** para avaliar a estabilidade e a coerência das declarações (ver Quadro 23).

| Tipo de evolução                      | Continuidade do núcleo de sentido | Tipo de transformação               | Exemplo                                                           |
|---------------------------------------|-----------------------------------|-------------------------------------|-------------------------------------------------------------------|
| <b>Amadurecimento de entendimento</b> | Preservada                        | Aprofundamento e esclarecimento     | Realizar pagamento → Realizar pagamento via PIX, boleto ou cartão |
| <b>Mudança de requisito</b>           | Rompida                           | Alteração de propósito ou escopo    | Realizar pagamento → Consultar pagamento                          |
| <b>Substituição/exclusão</b>          | Extinta                           | Remoção ou troca por novo propósito | Realizar pagamento → removido por decisão de negócio              |

Quadro 23 – Tipo de evolução e núcleo de entendimento.

Fonte: elaborado pelo autor (2025).

### 9.8.2.2 O ponto de partida: de onde se parte para avaliar a evolução

Para distinguir se uma revisão é amadurecimento ou mudança, é necessário definir a **base de entendimento** — o ponto de referência a partir do qual se analisa a transformação. Essa avaliação pode ser feita em três **camadas de referência**, conforme apresenta o Quadro 24.

| Camada                      | Foco de análise                                  | Função na avaliação                                            |
|-----------------------------|--------------------------------------------------|----------------------------------------------------------------|
| <b>Contextual (por que)</b> | Objetivos de negócio e justificativas originais. | Identifica se o propósito geral do requisito permanece válido. |
| <b>Funcional (o que)</b>    | A ação ou resultado esperado do sistema.         | Avalia se o comportamento ou efeito pretendido se manteve.     |
| <b>Descritiva (como)</b>    | Condições, regras, interfaces e formatos.        | Indica se as mudanças estão apenas no detalhamento.            |

Quadro 24 – Camadas da base de entendimento.

Fonte: elaborado pelo autor (2025).

Quando as alterações se restringem ao detalhamento de aspectos descritivos — sem afetar a ação ou o propósito do requisito —, trata-se de amadurecimento; se alcançam a ação ou o propósito, há mudança. Essa abordagem permite avaliar o tipo de transformação **com base na profundidade semântica**, e não apenas na aparência textual.

### 9.8.2.3 Limites e nuances

Na prática, a fronteira entre amadurecimento e mudança nem sempre é nítida. Pequenos acréscimos ou ajustes de regras podem alterar, indiretamente, o escopo técnico ou o custo de implementação.

Do mesmo modo, decisões de priorização ou restrição de funcionalidades podem parecer mudanças, mas refletem apenas nova interpretação de contexto.

Essas zonas cinzentas exigem julgamento de engenharia, apoiado pela análise do núcleo de sentido e das camadas de referência.

O amadurecimento pode levar à mudança; e a mudança, por sua vez, pode demandar novo amadurecimento. Compreender essa interação é o que garante coerência e rastreabilidade de sentido no processo declarativo.

#### 9.8.2.4 Síntese conceitual

A maturidade do entendimento e a estabilidade dos requisitos estão diretamente ligadas à **preservação do núcleo de sentido**. Enquanto o amadurecimento aprofunda o entendimento de um mesmo requisito, a mudança redefine o que o sistema deve fazer. Reconhecer essa diferença é fundamental para distinguir revisões interpretativas de transformações substantivas, preservando a continuidade cognitiva entre versões e artefatos (ver Quadro 25).

| Dimensão                         | Amadurecimento do Entendimento                                                             | Mudança de Requisito                                                                   |
|----------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>Natureza</b>                  | Cognitiva e interpretativa. Processo de refinamento do significado e da comunicação.       | Substantiva e estratégica. Altera o conteúdo, propósito ou escopo do requisito.        |
| <b>Referência semântica</b>      | O mesmo núcleo de sentido é preservado — expresso pelo nome do requisito (verbo + objeto). | O núcleo de sentido é alterado — muda o verbo e/ou o objeto, transformando a intenção. |
| <b>Camada afetada</b>            | Descritiva — detalhamento de condições, formatos, regras ou critérios.                     | Funcional e/ou contextual — alteração de ação, propósito ou papel do requisito.        |
| <b>Efeito</b>                    | Aprofundamento do entendimento; maior clareza e rastreabilidade.                           | Redefinição do escopo; necessidade de reavaliação de impactos e prioridades.           |
| <b>Exemplo</b>                   | Realizar pagamento → Realizar pagamento via PIX, boleto ou cartão.                         | Realizar pagamento → Consultar pagamento.                                              |
| <b>Indicador de estabilidade</b> | O requisito tende a se tornar estável à medida que o entendimento se consolida.            | A instabilidade decorre da mudança de intenção ou propósito.                           |

Quadro 25 – Núcleo de sentido, amadurecimento e mudança.  
Fonte: elaborado pelo autor (2025).

#### 9.8.3 Mudanças e continuidade de sentido nas declarações

As mudanças nos requisitos são parte natural da evolução de produtos de software e podem decorrer de novas decisões estratégicas, alterações normativas, descobertas técnicas, mudanças de contexto ou revisões na percepção de valor dos stakeholders (WIEGERS; BEATTY, 2013; SOMMERVILLE, 2019). Como discutido na seção anterior, nem toda alteração em uma declaração significa, contudo, que o requisito mudou. Em alguns casos, novas informações apenas tornam mais explícito aquilo que já se esperava do produto; em outros, a própria expectativa sobre seu comportamento é modificada.

Considere, por exemplo, um sistema de comércio eletrônico no qual foi identificado o seguinte requisito:



**RF-18 - Solicitar reembolso:** O sistema deve permitir que o cliente solicite o reembolso de uma compra cancelada.

A declaração expressa o comportamento central esperado, mas ainda deixa várias questões em aberto. Durante o aprofundamento do entendimento sobre o processo de reembolso, descobre-se que a compra precisa ter sido efetivamente paga, que o cliente deve informar o motivo da solicitação e que precisa acompanhar seu processamento. Também se torna necessário preservar a relação do reembolso com o valor pago e com a forma de pagamento utilizada. A declaração pode então evoluir para:

**RF-18 - Solicitar reembolso:** O sistema deve permitir que o cliente solicite o reembolso de uma compra que tenha sido paga e posteriormente cancelada. Ao realizar a solicitação, o cliente deve informar o motivo do reembolso. O sistema deve associar a solicitação à compra correspondente, ao valor efetivamente pago e à forma de pagamento utilizada, registrar a data e a hora da solicitação, atribuir um identificador ao pedido de reembolso e disponibilizar seu status para acompanhamento até a conclusão do processo.

Embora a segunda declaração seja mais detalhada, o núcleo de sentido foi preservado. O comportamento principal continua sendo iniciado pelo cliente: diante de uma compra cancelada, o cliente solicita seu reembolso. As novas informações esclarecem condições, dados, relações e resultados associados ao comportamento que já era esperado. Houve **amadurecimento do entendimento** e, conseqüentemente, maior precisão da declaração, mas não uma alteração naquilo que o produto deve fazer.

Uma situação diferente ocorre se a organização alterar sua política comercial e decidir que compras canceladas antes da expedição não devem mais depender de uma solicitação de reembolso. Nessa situação, o cancelamento passa a desencadear automaticamente o processo, sem qualquer ação adicional do cliente. A declaração poderia passar a ser:

**RF-18 - Processar reembolso automático:** O sistema deve iniciar automaticamente o reembolso integral de uma compra que tenha sido paga e cancelada antes de sua expedição, sem exigir solicitação do cliente. O reembolso deve ser associado à compra cancelada, considerar o valor efetivamente pago e a forma de pagamento utilizada, registrar a data e a hora em que o processamento foi iniciado e disponibilizar seu status para acompanhamento até a conclusão do processo.

Nesse caso, vários elementos permanecem: existe uma compra paga e cancelada, um valor a ser devolvido, uma forma de pagamento relacionada e um processo cujo andamento pode ser acompanhado. Entretanto, modificou-se um aspecto essencial do comportamento esperado. Antes, o cliente iniciava o processo ao solicitar o reembolso; agora, o próprio sistema inicia o reembolso quando identifica o cancelamento em determinada condição. A **mudança** não está simplesmente na quantidade de informações da declaração, mas **na ação central**, em quem a inicia e no evento que desencadeia o comportamento.

O Quadro 26 evidencia essas diferenças.

| Aspecto                                | Amadurecimento — Solicitar reembolso                                                                                          | Mudança — Processar reembolso automático                                                                                      |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Comportamento central                  | Permitir que o cliente solicite o reembolso.                                                                                  | Iniciar automaticamente o reembolso.                                                                                          |
| Quem inicia o comportamento            | Cliente.                                                                                                                      | Sistema.                                                                                                                      |
| Evento que desencadeia o comportamento | O cliente decide solicitar o reembolso de uma compra já cancelada.                                                            | Uma compra paga é cancelada antes de sua expedição.                                                                           |
| Participação do cliente                | O cliente realiza a solicitação e informa o motivo.                                                                           | O cliente não precisa solicitar o reembolso.                                                                                  |
| Informações associadas                 | Compra, valor efetivamente pago, forma de pagamento, motivo, data e hora da solicitação e status.                             | Compra, valor efetivamente pago, forma de pagamento, data e hora de início do processamento e status.                         |
| Elementos preservados                  | Reembolso associado a uma compra paga e cancelada, ao valor pago e à forma de pagamento, com possibilidade de acompanhamento. | Reembolso associado a uma compra paga e cancelada, ao valor pago e à forma de pagamento, com possibilidade de acompanhamento. |
| Núcleo de sentido                      | Cliente solicita reembolso.                                                                                                   | Sistema processa reembolso automaticamente.                                                                                   |
| Interpretação                          | Novas informações aprofundam e tornam mais precisa a compreensão do mesmo comportamento.                                      | O comportamento esperado do produto é modificado de forma substantiva.                                                        |

Quadro 26 – Exemplo de amadurecimento e mudança de um requisito.  
Fonte: elaborado pelo autor (2026).

O exemplo mostra que **amadurecimento** e **mudança** não podem ser distinguidos simplesmente comparando a quantidade de palavras acrescentadas, retiradas ou modificadas. Uma declaração pode receber diversas condições, regras ou informações adicionais e continuar expressando o mesmo requisito. Em contrapartida, poucas palavras podem alterar profundamente seu significado quando modificam a ação esperada, o objeto, o participante responsável, o evento disparador ou o propósito que sustenta o requisito. A avaliação precisa considerar aquilo que a declaração significa, e não apenas aquilo que mudou textualmente.

Essa distinção também ajuda a evitar que revisões sucessivas ocultem mudanças substantivas. Se uma formulação é apenas amadurecida (detalhada), deve ser possível reconhecer a continuidade do mesmo requisito entre suas diferentes declarações. Se o núcleo de sentido se modifica, a nova declaração precisa evidenciar que o compromisso anterior também foi alterado. No exemplo, a passagem de “o cliente solicita o reembolso” para “o sistema inicia automaticamente o reembolso” precisa permanecer perceptível, pois essa mudança pode afetar outros requisitos, regras de negócio, interfaces, decisões técnicas e critérios de avaliação.

A maneira de registrar essa evolução — incluindo a decisão de preservar ou alterar identificadores, manter versões anteriores, registrar justificativas e relacionar requisitos afetados — depende dos mecanismos de organização e atualização adotados no contexto do produto. Esses mecanismos serão aprofundados no Capítulo 12. No âmbito da declaração, o aspecto fundamental é assegurar que as sucessivas formulações continuem comunicando com clareza o que permaneceu, o que foi aprofundado e o que efetivamente mudou.

Assim, revisar uma declaração não significa necessariamente mudar um requisito, da mesma forma que uma mudança de requisito não deve ser tratada como simples alteração de redação. A declaração precisa tornar perceptível a evolução do entendimento para que diferentes participantes possam reconhecer quando estão discutindo uma formulação mais madura do mesmo compromisso e quando passaram a esperar do produto um comportamento diferente.

#### 9.8.4 Meios Linguísticos e Multimodais de Declaração

A declaração de requisitos é uma prática comunicativa que torna explícito o entendimento por meios linguísticos e multimodais capazes de ser compartilhados, retomados e discutidos. O texto escrito continua sendo um meio particularmente útil porque combina permanência, possibilidade de revisão e referência precisa, favorecendo a explicitação de requisitos em diferentes graus de estruturação (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

A comunicação humana, contudo, não se limita ao texto. Gestos, entonações, imagens, demonstrações e registros audiovisuais também participam da construção do significado e podem preservar elementos de contexto que se perdem em uma transcrição exclusivamente textual (WINOGRAD; FLORES, 1986; CLARK; BRENNAN, 1991).

Gravações de entrevistas, reuniões, oficinas ou demonstrações podem, portanto, apoiar a declaração quando preservam contexto relevante ou tornam explícito um comportamento esperado. Para que permaneçam utilizáveis, esses registros precisam estar associados a informações que permitam identificar a que requisito, decisão ou situação se referem. O nível de formalidade necessário para essa vinculação depende do contexto do produto e do uso posterior do material.

Estudos recentes evidenciam o potencial desses recursos na melhoria da comunicação e na redução de ambiguidades. O mapeamento conduzido por Santos, A. C. M et al. (2020) identificou mais de quarenta abordagens que integram áudio e vídeo às atividades de requisitos, com aplicações que vão da elicitación à declaração e rastreabilidade. Entre elas, destacam-se o TRECE Framework, que organiza gravações e metadados em camadas navegáveis; o Sysiphus Database, que armazena clips multimídia vinculados a requisitos extraídos; o ReqVidA, que conecta atas textuais a trechos de vídeo de reuniões; e o Wiki System Multimedia, que associa entrevistas gravadas a descrições de requisitos em wikis colaborativas. Outras técnicas, como VisionCatcher e The Software Cinema Technique, empregam vídeos curtos ou narrativas visuais para tornar visível o comportamento esperado de sistemas complexos. Os resultados apontam benefícios concretos: maior compreensão compartilhada, redução de ambiguidades, familiarização rápida com cenários e melhoria da rastreabilidade entre discurso e artefato.

A função dos meios multimodais varia conforme a etapa e o propósito comunicativo.

- **Durante a elicitación**, gravações e vídeos ampliam o campo de observação e compreensão, funcionando como instrumentos de descoberta.
- **Durante a declaração**, esses mesmos recursos operam como registros de decisão e reforço semântico, associados a artefatos textuais que expressam o requisito aceito. Essa complementaridade reflete uma evolução na prática da engenharia: o requisito não é mais visto apenas como um texto estático, mas como uma unidade comunicacional composta, cujo significado emerge da articulação entre linguagem escrita, oral e visual.

A multimodalidade, quando empregada com propósito, **não substitui o texto — complementa-o**. Ela amplia a fidelidade comunicativa e a memória organizacional, oferecendo à equipe não apenas o “que” foi decidido, mas também “como” e “por que” a decisão foi tomada. O papel do engenheiro de software, nesse cenário, é articular essas diferentes formas de expressão em uma linha coerente de sentido, transformando a pluralidade de registros em declarações consistentes, rastreáveis e semanticamente claras.

A ampliação dos meios de declaração — incorporando textos, vozes e registros audiovisuais — evidencia que declarar um requisito é mais do que documentar decisões: é **tornar explícito um entendimento partilhado**, sustentado por múltiplas formas de expressão. A prática da Engenharia de Requisitos, nesse sentido, evolui não apenas em suas técnicas, mas em sua consciência linguística. O desafio contemporâneo é equilibrar **expressividade e precisão, contexto e verificabilidade**, reconhecendo que cada forma de declaração carrega uma intenção comunicativa e um grau de estabilidade distinto.

A Figura 50 sintetiza as relações entre os conceitos discutidos neste capítulo.



Figura 50 –Relação entre os conceitos apresentados no capítulo 9.  
Fonte: elaborado pelo autor (2025).

## CAPÍTULO 9: PONTOS-CHAVE

- A declaração **torna explícito o entendimento sobre os requisitos** por meio de formas linguísticas que possam ser comunicadas, discutidas e retomadas ao longo do desenvolvimento.
- Declarar **não significa apenas produzir documentação formal**. Requisitos podem ser expressos por linguagem natural, estruturas textuais, narrativas orientadas a valor e, quando adequado, meios multimodais.
- A declaração favorece entendimento compartilhado, negociação, desenvolvimento, V&V e rastreabilidade ao **oferecer formas identificáveis de expressão dos requisitos**.
- Requisitos podem ser declarados em **diferentes níveis de abstração** — negócio, usuário e produto — que coexistem e se relacionam sem constituir etapas obrigatoriamente sequenciais.
- Narrativas, casos de uso, histórias de usuário, declarações estruturadas e catálogos oferecem diferentes possibilidades de expressão. **Nenhuma forma é universalmente superior**; sua adequação depende do propósito comunicacional.
- **A forma de declaração é influenciada** pela abordagem de desenvolvimento, pelo tipo de produto, pelo perfil dos stakeholders e da equipe, pela estrutura em que o requisito será utilizado e pelo momento e necessidade de uso.
- **Diferentes formas declarativas podem coexistir no mesmo produto**. Combinar técnicas é frequentemente mais adequado do que tentar representar todo o entendimento por um único formato.
- **As declarações amadurecem à medida que o entendimento evolui**. É importante distinguir o aprofundamento de um mesmo núcleo de sentido de mudanças substantivas no requisito.
- **Textos, áudio, vídeo e outros registros** podem atuar de forma complementar, desde que contribuam para preservar e comunicar o significado relevante.
- A organização das estruturas que reúnem as declarações, a manutenção dos vínculos entre seus elementos, o versionamento e o histórico de mudanças são aprofundados no Capítulo 12.

# Capítulo 10

## 10 Representação de Requisitos

### 10.1 Introdução

A representação de requisitos é a atividade responsável por **tornar visível o entendimento sobre o produto**. Ela materializa aspectos desse entendimento em formas perceptíveis e estruturadas — diagramas, modelos, fluxos, protótipos ou outras visualizações — que permitem compreender, discutir e evoluir o conhecimento sobre o problema e a solução. Relações, comportamentos, estados, dados, restrições e contextos que podem ser difíceis de perceber apenas por meio textual tornam-se mais facilmente observáveis quando representados.

Enquanto a **declaração** se concentra principalmente em *dizer o que se sabe* — expressando requisitos por meios linguísticos, como linguagem natural ou textual estruturada —, a **representação** procura *mostrar aspectos desse conhecimento*, utilizando formas visuais, gráficas, simbólicas ou matemáticas. Um fluxo, por exemplo, pode evidenciar a sequência e as alternativas de um comportamento; um diagrama pode tornar visíveis relações entre elementos; e um protótipo pode concretizar aspectos de uma interação que seriam mais difíceis de compreender apenas por uma descrição textual.

A distinção entre declaração e representação é, portanto, de natureza, e não de importância ou de ordem. A declaração explicita predominantemente o **conteúdo por meio da linguagem**, enquanto a representação torna determinados aspectos desse conteúdo **visíveis e observáveis por outras formas de expressão**. Uma não substitui a outra. Ao contrário, podem coexistir e se complementar conforme aquilo que precisa ser compreendido e comunicado.

A representação também não redefine o que é um requisito, mas, torna explícitas relações, dependências, estruturas e outros aspectos associados ao entendimento construído, ajudando a analisar e comunicar aquilo que conceitualmente já caracteriza um requisito (ver Seção 6.1). Em outras palavras, mantém-se a mesma referência conceitual, variando o **modo de organizar e tornar observável o entendimento** para apoiar comunicação, análise, decisão, verificação e validação.

Essa representação pode ser utilizada sempre que tornar algo visível contribuir para o trabalho com requisitos. Ideias ainda pouco compreendidas podem ser exploradas por meio de um esboço ou protótipo; relações entre requisitos podem ser discutidas com auxílio de diagramas; comportamentos podem ser examinados por meio de fluxos ou modelos; e propriedades que exigem maior rigor podem ser expressas por representações mais estruturadas ou formais. Esses usos não constituem etapas obrigatórias e podem ocorrer em diferentes combinações, ser retomados ou coexistir ao longo da evolução do produto.

Essa flexibilidade evidencia o caráter **transversal e iterativo** da representação, uma vez que a visualização pode provocar novas perguntas, revelar relações antes não percebidas, expor inconsistências ou simplesmente oferecer outra perspectiva sobre um entendimento já compartilhado.

Por isso, cada representação deve ser tratada como uma construção revisável, que pode ser ampliada, ajustada ou substituída conforme o entendimento evolui. Seu nível de detalhe e formalidade deve ser adequado ao propósito, ao público, à criticidade do produto e ao tipo de conhecimento que se pretende tornar explícito.

Importante notar que **representar não significa apenas desenhar telas ou interfaces**. A representação não se limita ao plano visual da interação humana; ela descreve **qualquer estrutura, comportamento ou relação relevante para compreender o sistema**.

Mesmo produtos sem interface gráfica — como softwares de linha de comando, sistemas embarcados, controladores industriais ou agentes autônomos — utilizam de representações: fluxos de dados, diagramas de estado, mapas de dependência, expressões lógicas ou estruturas de controle.

Em essência, **a Engenharia de Requisitos é representacional por natureza**.

Projetos que não utilizam diagramas ou modelos formais ainda assim recorrem a esboços visuais, storyboards, mapas conceituais, fluxos de interação ou representações de processo — artefatos informais, mas que cumprem o papel de tornar o entendimento compartilhável e discutível. Representar é, portanto, inevitável e fundamental: é o meio pelo qual o conhecimento ganha forma e pode ser aprimorado coletivamente.

A representação cumpre, assim, dupla função:

1. **Cognitiva**, ao permitir que indivíduos pensem sobre o sistema de modo mais estruturado e visual;
2. **Comunicacional**, ao possibilitar que diferentes perfis — técnicos ou não técnicos — compreendam e questionem o mesmo fenômeno.

Representar é transformar a compreensão individual em significado coletivo. É o ponto em que o pensamento ganha forma, o diálogo se torna mais concreto e o conhecimento pode ser aprimorado.

Na Engenharia de Requisitos, **representar é compreender melhor**.

#### Box 10.1-1 – Representação e simplicidade

Representar é aplicar o valor de **Simplicidade** (modelar apenas o que reduz ambiguidade relevante) e os princípios de **Entendimento compartilhado (Princípio 4)**, escolhendo a forma que melhor **comunica** a intenção ao público certo (**Princípio 2**).

Ao longo deste capítulo, as representações serão classificadas em **informais, semiformais e formais** (ver Capítulo 5), conforme o grau de estrutura e precisão exigidos.

Em síntese, representar requisitos é dar forma ao entendimento obtido durante a interação com stakeholders. Essa forma pode variar em grau de formalidade e detalhamento, mas sempre cumpre o papel de tornar visível o que deve ser construído e o porquê. Assim como as demais atividades descritas no Capítulo 5, a representação é escolhida de forma situada: seu formato e nível de rigor devem refletir propósito (explorar, alinhar, analisar, validar), público e criticidade,



preservando os valores e princípios da ER — em especial comunicação, simplicidade e entendimento compartilhado.

Representar requisitos significa escolher formas adequadas para tornar determinados aspectos do entendimento perceptíveis e discutíveis. Como essas representações frequentemente também são utilizadas durante descoberta, análise ou V&V, é importante distinguir a produção da representação do uso que outras atividades fazem dela. A Figura 51 sintetiza esse enquadramento.



Figura 51 – Enquadramento da atividade de Representação.  
Fonte: elaborado pelo autor (2026).

## 10.2 Por que representar?

A utilidade da representação vai além de registrar o que foi compreendido: ela apoia o raciocínio coletivo, a comunicação entre papéis e a validação progressiva das ideias. Por isso, compreender *por que* representar é tão importante quanto saber *como* representar.

Ao converter o entendimento em formas visuais, estruturadas ou simbólicas, a equipe amplia sua capacidade de analisar, discutir e validar requisitos de modo coletivo. A representação não é apenas um registro; é **um instrumento de raciocínio** que revela o que ainda não está claro e conecta perspectivas distintas.

O principal motivo para representar é **reduzir a distância entre o que se entende e o que se comunica**.

Em equipes multidisciplinares, as diferenças de linguagem entre negócio, design e engenharia de software tornam fácil o surgimento de ambiguidades. Modelos, fluxos e diagramas criam uma **base comum de entendimento**, permitindo que todos observem o mesmo fenômeno e o interpretem sob ângulos complementares. Além de comunicar, representar **ajuda a descobrir**.

Ao tentar estruturar graficamente um comportamento ou processo, lacunas conceituais tornam-se visíveis. O ato de representar obriga a equipe a **organizar o raciocínio**, explicitar suposições e identificar inconsistências.

Por isso, a representação é também um **mecanismo de aprendizado** — um meio de compreender melhor o problema à medida que ele é modelado.

O Quadro 27 apresenta os principais benefícios práticos da representação:

| Benefício                 | Descrição                                                                      |
|---------------------------|--------------------------------------------------------------------------------|
| <b>Clareza</b>            | Torna o raciocínio acessível e verificável por outros, reduzindo ambiguidades. |
| <b>Alinhamento</b>        | Integra diferentes áreas em torno de uma visão comum e compartilhável.         |
| <b>Consistência</b>       | Evidencia contradições, redundâncias e lacunas cedo no processo.               |
| <b>Validação</b>          | Permite testar hipóteses e confirmar entendimentos antes da implementação.     |
| <b>Economia cognitiva</b> | Reduz a carga mental necessária para compreender o sistema e suas relações.    |

Quadro 27 – Benefícios da representação.  
Fonte: elaborado pelo autor (2025).

Antes de detalhar os focos e as técnicas, explicitamos as fronteiras entre ER, Design e Arquitetura (Seção 10.3), para que os artefatos aqui descritos não sejam confundidos com decisões de solução.

#### Box 10.2-1 – Inovação na ER

Este ponto concretiza o **Princípio 9 (Inovação da ER)**: representar não é apenas aplicar um repertório fixo de artefatos, mas ampliar, de modo responsável, as formas de tornar o entendimento compartilhável e analisável. A inovação em representação surge quando a equipe experimenta novas visualizações, combina mídias e adapta notações ao contexto — sem perder coerência semântica, rastreabilidade e capacidade de verificação.

### 10.3 Fronteiras da representação de requisitos segundo o SWEBOK v4.0

A representação de requisitos situa-se em uma fronteira natural entre a **compreensão do problema** e a **concepção da solução**.

Por essa razão, modelos, fluxos e diagramas são frequentemente utilizados tanto por pessoas responsáveis pelos requisitos quanto por pessoas responsáveis por projetar o software. Compreender onde termina a Engenharia de Requisitos e onde começam o Design e a Arquitetura é essencial para evitar sobreposições e preservar a coerência do processo de desenvolvimento.

De acordo com o *Guide to the Software Engineering Body of Knowledge* (SWEBOK v4.0, 2024), a disciplina de **Software Requirements** compreende as atividades voltadas a definir o que o sistema deve fazer, sob quais restrições e com que qualidade, sem prescrever como ele será estruturado ou implementado. Seu foco está na compreensão das necessidades e na comunicação dos comportamentos esperados, de forma verificável e rastreável pelos stakeholders.

A área de **Software Architecture**, por sua vez, tem como objetivo definir a organização fundamental do sistema e as relações entre seus principais componentes. Ela estabelece decisões estruturais de alto nível, identifica elementos críticos e suas interfaces e orienta como o sistema será dividido, integrado e mantido ao longo do tempo. Em essência, a arquitetura transforma requisitos significativos — especialmente os de qualidade, desempenho, segurança e escalabilidade — em estruturas técnicas que moldam a solução.

Já a área de **Software Design** tem como propósito detalhar como cada parte do sistema atenderá aos requisitos definidos e às restrições impostas pela arquitetura. O Design traduz as intenções arquiteturais em especificações de componentes, fluxos internos, algoritmos e interações. Também define comportamentos, dados e interfaces, tornando a implementação possível e coerente com os objetivos estabelecidos. Enquanto a Arquitetura oferece uma visão estrutural e estratégica, o Design fornece a visão tática e operacional, especificando como cada unidade concretiza o que foi concebido.

No que se refere às **interfaces e protótipos**, o SWEBOK v4 diferencia claramente as representações de requisitos dos artefatos de design. Os protótipos de baixa fidelidade e storyboards pertencem à Engenharia de Requisitos quando usados para elicitare, comunicar ou validar entendimentos com os stakeholders, funcionando como instrumentos de descoberta e aprendizado. Por outro lado, os wireframes detalhados, protótipos de alta fidelidade e guias de estilo visuais integram o Software Design, pois materializam decisões sobre aparência, interação e experiência de uso.

Em resumo, na ER, protótipos servem para perguntar “*é isso que você precisa?*”; no Design, para responder “*é assim que será construído*”.

Em termos práticos, uma representação pertence à Engenharia de Requisitos **quando seu propósito é comunicar, esclarecer ou validar comportamentos esperados, necessidades ou restrições observáveis**. Quando o modelo passa a **detalhar elementos internos da solução** — fluxos de controle, algoritmos, estruturas de dados, decisões de interface ou layouts de tela —, ele deixa de ser um artefato de requisitos e passa a integrar a esfera do Design ou da Arquitetura.

A seguir, o Quadro 28 sintetiza as fronteiras da representação de requisitos conforme o SWEBOK v4.0.

| Tipo de representação                                                           | Finalidade dentro da ER                                                                                        | Situação segundo o SWEBOK v4.0           | Quando se torna Design                                                                                                                      | Quando se torna Arquitetura                                                                     |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Sketches, mapas mentais e conceituais, mockups e protótipos de baixa fidelidade | Apoiar a descoberta, comunicação e validação de requisitos por meio de representações simples e exploratórias. | Dentro da ER.                            | Quando evoluem para wireframes detalhados, protótipos navegáveis ou padrões visuais de interface – domínio de <i>User-Centered Design</i> . | —                                                                                               |
| Diagramas de contexto, mapas de stakeholders e modelos de fronteira             | Delimitar escopo e representar interações externas do sistema.                                                 | Dentro da ER.                            | —                                                                                                                                           | Quando descrevem topologias, alocações internas ou mecanismos de comunicação entre subsistemas. |
| Casos de uso (diagramas e narrativas)                                           | Descrever comportamentos observáveis e fluxos externos de interação entre atores e sistema.                    | Dentro da ER.                            | Quando detalham mensagens, lógica de controle ou exceções internas – <i>Behavioral Design</i> .                                             | Quando usados como visões arquiteturais para verificar consistência global.                     |
| Fluxos de navegação (conceituais)                                               | Representar caminhos e transições entre telas ou funcionalidades em nível conceitual, validando fluxos de uso. | Dentro da ER.                            | Quando descrevem interações detalhadas, microcomportamentos ou navegação real de interface – <i>Interaction Design / UI Design</i> .        | —                                                                                               |
| Fluxos de estados (conceituais)                                                 | Representar o ciclo de vida de entidades do domínio (ex.: solicitação criada → analisada → concluída).         | Dentro da ER.                            | Quando modelam máquinas de estados internas, triggers ou eventos técnicos.                                                                  | —                                                                                               |
| Fluxogramas e fluxos de atividades (de alto nível)                              | Descrever sequências de ações observáveis e papéis envolvidos.                                                 | Dentro da ER (comportamento do domínio). | Quando especificam controle interno de processos ou condições lógicas complexas – <i>Behavioral Design</i> .                                | —                                                                                               |
| Wireframes (conceituais)                                                        | Ilustrar disposição e fluxo entre telas de forma simplificada, para apoiar validação com stakeholders.         | Dentro da ER.                            | Quando passam a detalhar layouts, componentes gráficos, estilos e padrões de interação – <i>User Interface Design</i> .                     | —                                                                                               |
| Storyboards                                                                     | Representar jornadas de uso e validar entendimento com stakeholders.                                           | Dentro da ER.                            | Quando evoluem para narrativas visuais de interface ou fluxos navegáveis detalhados – <i>User-Centered Design</i> .                         | —                                                                                               |

| Tipo de representação                                     | Finalidade dentro da ER                                   | Situação segundo o SWEBOK v4.0                        | Quando se torna Design                                                                                       | Quando se torna Arquitetura                                     |
|-----------------------------------------------------------|-----------------------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Modelos entidade–relação e mapas conceituais estruturados | Explicitar conceitos e relações semânticas do domínio.    | Parcial – dentro da ER se usados em nível conceitual. | Quando incluem atributos técnicos, chaves, normalização ou persistência de dados – parte do Design de Dados. | Quando se tornam viewpoints informacionais que cruzam sistemas. |
| Diagramas de sequência                                    | Mostrar trocas de mensagens entre objetos ou componentes. | Fora da ER.                                           | Design → <i>Behavioral Design Descriptions</i> .                                                             | —                                                               |
| Diagramas de componentes, classes, camadas e deployment   | Descrever estrutura interna e interdependências técnicas. | Fora da ER.                                           | —                                                                                                            | Arquitetura → <i>Architecture Views and Viewpoints</i> .        |

Quadro 28 – Fronteiras da Representação de Requisitos conforme o SWEBOK v4.0.  
Fonte: elaborado pelo autor (2025).

## 10.4 Focos da representação

A representação de requisitos busca **mostrar o que o sistema deve fazer e como se comporta**, sem antecipar decisões de projeto ou implementação. Seu papel é estruturar a compreensão do problema, dos objetivos e das interações relevantes, mantendo o foco no **domínio do requisito**, não na solução técnica.

Os focos de representação variam conforme o tipo de sistema e o aspecto do comportamento ou da necessidade que se deseja explicitar.

O Quadro 29, a seguir, resume exemplos de representação dentro do escopo da Engenharia de Requisitos.

| Foco da representação | Descrição                                                                                                       | Exemplos típicos de artefatos de representação                                                                   | Tipo de Representação predominante |
|-----------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------|
| <b>Contextual</b>     | Define o escopo do sistema e suas interações com o ambiente, destacando fronteiras e atores externos.           | Diagramas de contexto, mapas de stakeholders, modelos de fronteira.                                              | Informal → Semiformal              |
| <b>Comportamental</b> | Descreve ações, eventos e respostas observáveis do sistema a partir da perspectiva dos atores.                  | Casos de uso (diagramas e narrativas), fluxos de atividades, fluxos de estados conceituais, fluxos de navegação. | Semiformal                         |
| <b>Informacional</b>  | Representa conceitos, entidades e relações que estruturam o domínio de informação e significado.                | Mapas conceituais, modelos entidade–relação conceituais, diagramas de classes conceituais, glossários de termos. | Semiformal                         |
| <b>Experiencial</b>   | Ilustra a jornada do usuário e as interações percebidas, apoiando a comunicação e validação de requisitos.      | Sketches, storyboards, wireframes conceituais, mockups e protótipos de baixa fidelidade.                         | Informal                           |
| <b>Formal</b>         | Expressa requisitos de forma matemática ou lógica, permitindo análise e verificação automática de consistência. | Lógica proposicional e temporal, autômatos, Z notation, Alloy, VDM, especificações formais baseadas em modelos.  | Formal                             |

Quadro 29 – Exemplos de representações na ER.

Fonte: elaborado pelo autor (2025).

Os diferentes focos de representação — contextual, comportamental, informacional, experiencial e formal — evidenciam que não há uma única maneira de representar requisitos, mas sim diferentes perspectivas e graus de precisão.

Cada foco privilegia um aspecto do entendimento: o contexto, o comportamento, a informação, a experiência ou a forma lógica subjacente ao sistema.

Como relatado no Capítulo 5, as representações podem assumir três tipos básicos:

- **Informais**, quando livres ou parcialmente estruturadas, usadas para explorar ideias e comunicar entendimentos iniciais.
- **Semiformais**, quando seguem uma notação ou convenção reconhecida, facilitando a consistência e a rastreabilidade.
- **Formais**, quando baseadas em linguagens matemáticas ou lógicas, permitindo verificação rigorosa e análise automatizada.

Cada foco tende a se expressar por um desses tipos, dependendo do propósito, do público e do grau de maturidade do projeto.

As representações contextuais e experienciais predominam em abordagens informais; as comportamentais e informacionais, em notações semiformais; e as formais ocupam o extremo do espectro de rigor e precisão.

A Figura 52 sintetiza esses focos e seus tipos predominantes como um mapa iterativo: conforme o risco, o público e as dúvidas abertas, a equipe alterna e retorna entre representações informais, semiformais e formais para refinar entendimento, consolidar estrutura e explicitar propriedades críticas. A formalização seletiva é uma decisão orientada ao **Princípio 1** (Orientação a valor): elevar rigor apenas quando o benefício (redução de risco, retrabalho, ambiguidade crítica) supera o custo. Isso preserva **Simplicidade** sem abrir mão de verificabilidade quando necessária. A próxima seção aprofunda essas diferenças, começando pelas representações informais, que constituem o primeiro nível de materialização dos requisitos e são amplamente utilizadas para descoberta, validação e comunicação com stakeholders.

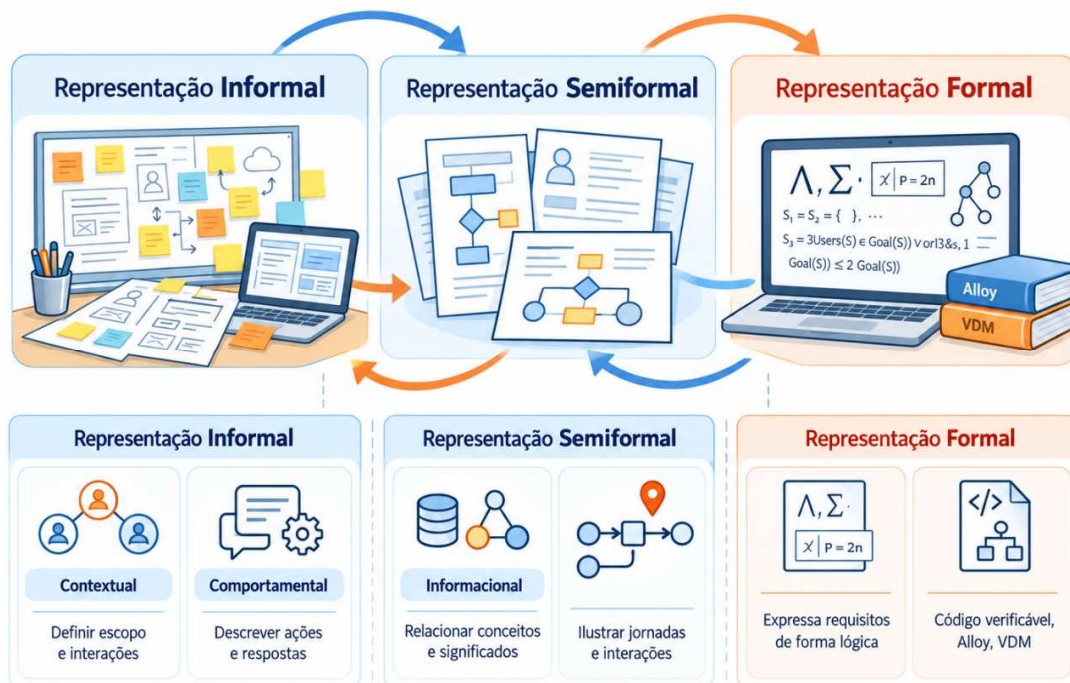


Figura 52 – Mapa iterativo entre os tipos de representação de requisitos.  
Fonte: elaborado pelo autor (2025).



## 10.5 Representações informais

As representações informais constituem o primeiro nível de externalização visual dos requisitos. Diferentemente das representações semiformais ou formais, elas não se apoiam em notações padronizadas nem buscam precisão sintática. Seu propósito é **tornar visíveis ideias, necessidades, contextos e relações ainda em formação**, favorecendo o entendimento compartilhado entre stakeholders.

Essas técnicas funcionam como **meios de pensamento visual colaborativo**, permitindo que concepções, hipóteses e interpretações sejam expressas e discutidas de forma acessível. Mais do que etapas lineares, as representações informais operam como **camadas cognitivas complementares**, aplicáveis em momentos distintos ou paralelos do processo de Engenharia de Requisitos.

De modo geral, essas representações cumprem quatro papéis principais:

1. **Compreender o contexto e as fronteiras do sistema** (*sistêmicas*);
2. **Explorar ideias e experiências de uso** (*livres*);
3. **Estruturar visualmente possibilidades, relações ou fluxos para discussão** (*semiestruturadas*);
4. **Integrar e consolidar conhecimento coletivo** (*cognitivas*).

Esses papéis não devem ser vistos como etapas fixas, mas como **modos interligados de raciocínio e comunicação visual**, que podem ocorrer simultaneamente ou se retroalimentar ao longo do projeto.

A seguir, a Figura 53 ilustra a relação conceitual entre os quatro grupos de representações informais e seus propósitos cognitivos.



Figura 53 – Lógica cognitiva das representações informais na Engenharia de Requisitos.  
Fonte: elaborado pelo autor (2025).



Cada grupo favorece um tipo de raciocínio — **compreender, criar, estruturar e integrar** —, e todos podem ser usados de forma iterativa e complementar. As sobreposições indicam que as técnicas associadas (como Rich Pictures, Sketches, Mockups e Mapas Conceituais) podem ser combinadas em diferentes momentos, conforme a necessidade de entendimento e comunicação entre stakeholders.

O Quadro 30 resume esses grupos, destacando seus propósitos, modos de raciocínio e técnicas mais recorrentes na prática da Engenharia de Requisitos.

| Grupo de Representações                                         | Tipo de raciocínio predominante | Técnicas associadas                                                              | Propósito principal                                                              |
|-----------------------------------------------------------------|---------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>Sistêmicas (contextuais)</b><br>(ver 10.5.1)                 | Compreender                     | Rich pictures, mapa de stakeholders, modelos de fronteira, diagramas de contexto | Analisar o ambiente, os atores e as fronteiras do sistema.                       |
| <b>Livres (intuitivas)</b><br>(ver 10.5.2)                      | Criar                           | Sketches, storyboards                                                            | Explorar ideias e visualizar experiências de uso.                                |
| <b>Semiestruturadas (organizacionais)</b><br>(ver 10.5.3)       | Estruturar                      | Mockups, protótipos de baixa fidelidade                                          | Organizar e validar visualmente soluções e fluxos.                               |
| <b>Cognitivas (conceituais e colaborativas)</b><br>(ver 10.5.4) | Integrar                        | Mapas de afinidades, mapas mentais e conceituais, mapas de stakeholders          | Sintetizar e consolidar conhecimento coletivo, conectando percepções e decisões. |

Quadro 30 –Quadro-guia inicial: Propósito e Aplicação das Representações Informais.  
Fonte: elaborado pelo autor (2025).

As subseções a seguir detalham cada grupo de representações, apresentando seus conceitos, finalidades, recomendações e exemplos ilustrativos, de modo a evidenciar como diferentes formas de representação se complementam no processo de compreensão e validação colaborativa dos requisitos.

### 10.5.1 Representações Sistêmicas (contextuais)

As representações sistêmicas — também chamadas de **representações contextuais** — têm como propósito situar o produto, serviço ou sistema dentro de seu **ambiente operacional e organizacional**, descrevendo as interações que o conectam a pessoas, estruturas, normas e outros sistemas.

Elas não tratam do que o sistema “faz” internamente, mas **de onde ele se insere e com quem interage**, formando o primeiro quadro cognitivo compartilhado sobre seu escopo, fronteiras e interdependências (POHL, 2010; SOMMERVILLE, 2019).

No processo de Engenharia de Requisitos, essas representações antecedem e fundamentam as demais, pois um requisito só é completo quando o contexto que o sustenta é compreendido (DICK; HULL; JACKSON, 2017). Sua função é alinhar percepções, reduzir ambiguidades e construir uma compreensão coletiva que guiará outras etapas de elicitação e descoberta, e representação.

Em termos práticos, essas representações podem ser criadas em diferentes momentos, mas há uma ordem lógica na forma como o entendimento costuma emergir:

1. **Rich Pictures**, de caráter exploratório e interpretativo;
2. **Mapas de Stakeholders**, que organizam as relações humanas e institucionais;
3. **Diagramas de Contexto**, que consolidam o escopo técnico e os fluxos principais; e
4. **Modelos de Fronteira**, que formalizam responsabilidades e camadas de interação.

Essa sequência não é rígida — em abordagens iterativas ou ágeis, pode ser ajustada conforme a maturidade das informações —, mas reflete uma lógica de progressão do mais abstrato ao estruturado, do interpretativo ao técnico, e do coletivo ao específico. O importante é que o conjunto dessas representações funcione como uma base convergente de entendimento sistêmico, sobre a qual as representações cognitivas e estruturais se apoiarão.

#### 10.5.1.1 Rich Pictures

São representações gráficas livres que **integram elementos humanos, organizacionais e tecnológicos** em um mesmo diagrama. Originários da abordagem *Soft Systems Methodology* (CHECKLAND, 1981), ela combina ícones, metáforas e anotações curtas para retratar o sistema e seu entorno — incluindo problemas, oportunidades, emoções e relações informais.

Seu objetivo é compreender o contexto do problema antes de formalizar modelos de solução, privilegiando diversidade de perspectivas e comunicação entre diferentes públicos.

#### 10.5.1.2 Mapa De Stakeholders

O mapa organiza de forma analítica quem são os atores e como se relacionam com o sistema e entre si. Ele traduz a riqueza narrativa das *Rich Pictures* em um modelo relacional de influência e interesse, classificando as partes interessadas conforme seu papel (usuário, mantenedor, integrador, regulador etc.), grau de poder decisório e relevância informacional (POHL, 2010; HRUSCHKA et al., 2022).

Esse mapa é essencial para planejar estratégias de envolvimento e comunicação ao longo do ciclo de vida do projeto.

#### 10.5.1.3 Diagrama de Contexto

O diagrama de contexto, consolida os achados anteriores em uma visão técnica de alto nível. Nele, o sistema é representado como uma entidade central (“caixa preta”) cercada por atores e sistemas externos, conectados por fluxos de informação, eventos ou mensagens. Sua notação propositalmente simples facilita o diálogo entre áreas de negócio e equipes de tecnologia, ao mesmo tempo em que estabelece **fronteiras explícitas de escopo e responsabilidade** (SOMMERVILLE, 2019).

Esse diagrama torna evidente o que pertence ao sistema e o que é externo a ele — informação crucial para evitar redundâncias ou expectativas irreais sobre seu comportamento.

#### 10.5.1.4 Modelo de Fronteira

O modelo de fronteira aprofunda o entendimento das interações abordadas nas representações anteriores, apresentando **camadas de contexto** (organizacional, de aplicação, de infraestrutura e física). Com ele, é possível definir claramente **quais atores e componentes pertencem a cada camada**, suas interfaces e restrições associadas (DICK; HULL; JACKSON, 2017).

Esse modelo é especialmente útil em ambientes distribuídos, com múltiplas organizações ou tecnologias, pois demonstra como responsabilidades e dependências se distribuem no ecossistema.

Assim, cada representação alimenta a próxima: a **rich picture** revela o ambiente humano e simbólico; o **mapa de stakeholders** estrutura as relações de poder e comunicação; o **diagrama de contexto** formaliza os fluxos e fronteiras; e o **modelo de fronteira** consolida a arquitetura de interfaces. Juntas, essas quatro visões constroem uma base cognitiva e comunicacional para o trabalho da ER.

As representações sistêmicas combinam práticas gerais (ver Quadro 31), aplicáveis a todo o grupo, com orientações específicas para cada técnica (ver Quadro 32).

| Prática geral                                  | Descrição / Aplicação                                                                                                       |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Começar pelo ambiente, não pela solução</b> | Mapear propósitos, atores e restrições antes de discutir funcionalidades ou requisitos específicos.                         |
| <b>Promover coautoria</b>                      | Envolver diferentes perfis de stakeholders (usuários, analistas, gestores, técnicos) na construção das representações.      |
| <b>Registrar fontes e hipóteses</b>            | Indicar de onde derivam as informações e manter rastreabilidade das decisões de modelagem.                                  |
| <b>Manter a clareza visual</b>                 | Priorizar legibilidade, evitando excesso de elementos, ícones ou formalismo prematuro.                                      |
| <b>Revisar iterativamente</b>                  | Atualizar representações conforme surgem novas informações, mantendo alinhamento com o entendimento coletivo.               |
| <b>Conectar representações</b>                 | Garantir consistência entre as visões de contexto (10.5.1), os fluxos cognitivos (10.5.2) e as estruturas formais (10.5.3). |

Quadro 31 – Práticas gerais recomendadas para as representações sistêmicas na ER.  
Fonte: elaborado pelo autor (2025).

| Técnica                      | Boas práticas específicas                                                                                           | Finalidade principal                                                                                            |
|------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Rich Pictures</b>         | Usar metáforas e símbolos reconhecíveis; estimular livre expressão; priorizar síntese visual sobre precisão formal. | Facilitar a interpretação e o diálogo entre stakeholders, capturando aspectos sociais e emocionais do contexto. |
| <b>Mapas de Stakeholders</b> | Posicionar atores segundo poder e interesse; evidenciar vínculos, dependências e conflitos potenciais.              | Planejar estratégias de comunicação, engajamento e priorização de requisitos.                                   |
| <b>Diagramas de Contexto</b> | Representar fluxos principais de dados e eventos; explicitar fronteiras de escopo e interação do sistema.           | Delimitar responsabilidades e identificar integrações com sistemas externos.                                    |

| Técnica                     | Boas práticas específicas                                                                             | Finalidade principal                                                                          |
|-----------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Modelos de Fronteira</b> | Identificar camadas de responsabilidade e interfaces entre organizações, aplicações e infraestrutura. | Relacionar o ambiente técnico e institucional, apoiando decisões de arquitetura e governança. |

Quadro 32 – Práticas específicas recomendadas para as representações sistêmicas na ER.

Fonte: elaborado pelo autor (2025).

## Exemplo

Durante a fase inicial do Sistema de Agendamento de Consultas Públicas, a equipe de requisitos promoveu uma série de oficinas para compreender o ecossistema de funcionamento do serviço.

O processo começou com a criação de uma **Rich Picture** (Figura 54) colaborativa, na qual os participantes desenharam o percurso do cidadão ao tentar agendar uma consulta. O diagrama revelou frustrações com cancelamentos, dificuldades de comunicação e a necessidade de integração entre diferentes canais de atendimento.



Figura 54 – Rich Picture - Sistema de Agendamento de Consultas Públicas.

Fonte: elaborado pelo autor (2025).

A partir dessa representação, construiu-se um **mapa de stakeholders** (Figura 55) destacando cidadãos, atendentes, gestores regionais, equipes técnicas e órgãos de auditoria. A análise evidenciou que alguns atores, embora pouco visíveis, exerciam forte influência sobre decisões operacionais.



Figura 55 – Mapa de Stakeholders - Sistema de Agendamento de Consultas Públicas.  
Fonte: elaborado pelo autor (2025).

Com base nesses achados, elaborou-se o **diagrama de contexto** (Figura 56), posicionando o sistema central em relação a quatro sistemas externos — Cadastro Nacional do Cidadão, Plataforma Gov.br, Módulo de Notificações e Base de Profissionais de Saúde — e identificando os principais fluxos de dados entre eles.

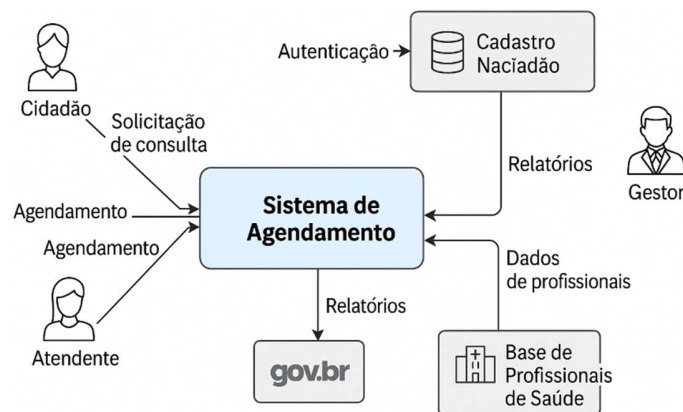


Figura 56 – Diagrama de Contexto - Sistema de Agendamento de Consultas Públicas.  
Fonte: elaborado pelo autor (2025).

Por fim, foi construído um **modelo de fronteira** (Figura 57), que distinguiu as camadas **organizacional** (Ministério e Secretarias), de **aplicação** (módulos de agendamento, fila e confirmação) e de **infraestrutura** (API Gateway e datacenter). Essa visão consolidada orientou decisões posteriores sobre responsabilidades, integrações e requisitos de segurança.

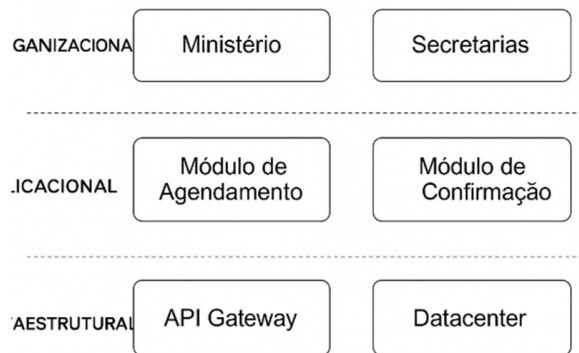


Figura 57 – Modelo de Fronteira - Sistema de Agendamento de Consultas Públicas.  
Fonte: elaborado pelo autor (2025).

### Síntese do uso e integração

Essas representações formam o núcleo de entendimento contextual do sistema, estabelecendo a base sobre a qual as representações livres aprofundam a compreensão de ideias, fluxos e raciocínios coletivos.

O Quadro 33 compara as quatro representações contextuais apresentadas, destacando seus propósitos, focos e formas de complementaridade.

| Aspecto                      | Rich Picture                                                                                  | Mapa de Stakeholders                                                      | Diagrama de Contexto                                                                         | Modelo de Fronteira                                                                                        |
|------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Propósito principal</b>   | Explorar o ambiente de forma interpretativa, revelando percepções, conflitos e oportunidades. | Organizar e analisar os atores e suas relações de influência e interesse. | Delimitar o escopo funcional e as trocas de informação entre o sistema e o ambiente externo. | Estruturar as camadas de interação e responsabilidade entre o sistema, as organizações e a infraestrutura. |
| <b>Foco da representação</b> | O ambiente social e simbólico do sistema.                                                     | As relações humanas e institucionais que moldam o sistema.                | As interfaces e fluxos entre o sistema e elementos externos.                                 | As camadas técnicas e organizacionais que sustentam o funcionamento.                                       |
| <b>Natureza visual</b>       | Livre e expressiva, com uso de ícones, metáforas e anotações.                                 | Estruturada, relacional e analítica.                                      | Formal e técnica, com diagramas simples e setas de fluxo.                                    | Hierárquica, organizada em níveis e fronteiras delimitadas.                                                |
| <b>Uso típico</b>            | Oficinas iniciais de entendimento e descoberta de contexto.                                   | Planejamento de envolvimento de partes interessadas e comunicação.        | Definição de escopo, integrações e fronteiras do sistema.                                    | Especificação de responsabilidades e arquitetura de interação entre camadas.                               |

| Aspecto                                | Rich Picture                                                                         | Mapa de Stakeholders                                                                    | Diagrama de Contexto                                                                 | Modelo de Fronteira                                                                                              |
|----------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Pergunta que ajuda a distinguir</b> | <i>“Como as pessoas e instituições percebem e vivem o sistema em seu contexto?”</i>  | <i>“Quem são os envolvidos e como suas relações influenciam o projeto?”</i>             | <i>“O que está dentro e fora do sistema, e como ele se comunica com o ambiente?”</i> | <i>“Como as camadas organizacionais, de aplicação e de infraestrutura se articulam para suportar o sistema?”</i> |
| <b>Complementaridade</b>               | Alimenta a construção do Mapa de Stakeholders e ajuda a revelar relações invisíveis. | Serve de insumo para a criação do Diagrama de Contexto, definindo papéis e prioridades. | Fornece base para o Modelo de Fronteira, conectando o ambiente aos níveis técnicos.  | Consolida as informações anteriores em uma visão estruturada e verificável, usada para decisões arquitetais.     |

Quadro 33 – Síntese do uso e integração das representações contextuais.  
Fonte: elaborado pelo autor (2025).

## 10.5.2 Representações Livres (intuitivas)

As representações livres correspondem ao nível mais espontâneo e criativo da expressão visual de requisitos. Elas são utilizadas **para explorar ideias, visualizar cenários de uso e promover entendimento comum** entre os participantes de uma oficina ou reunião inicial. Seu caráter é essencialmente **não prescritivo**, permitindo liberdade total na forma, no meio e no grau de detalhamento.

A principal finalidade dessas representações é **materializar o pensamento** de forma simples e rápida, tornando tangíveis conceitos abstratos e hipóteses ainda em formulação. Por isso, são particularmente úteis nas fases iniciais da **elicitação e descoberta de requisitos**, quando o diálogo é mais exploratório e aberto.

Nesse grupo, destacam-se duas técnicas amplamente difundidas e complementares: sketches e storyboards.

### 10.5.2.1 Sketches

Os *sketches* são **esboços visuais feitos à mão** — em papel, quadro branco ou tela digital — que representam uma ideia, uma interface, um fluxo de interação ou um conceito de produto. Sua principal característica é a **simplicidade**, permitindo iterar e descartar facilmente alternativas.

De acordo com Lauesen (2002), o *sketch* serve para “tornar o produto real o suficiente para provocar reação”, estimulando stakeholders a verbalizarem expectativas e críticas que, de outra forma, permaneceriam implícitas. Alexander e Beus-Dukic (2009) reforçam esse papel provocativo: quanto mais rústico o desenho, **mais seguro o participante se sente para sugerir mudanças**, já que o artefato não parece finalizado.

O Quadro 34 reúne práticas para usar sketches na exploração e discussão de requisitos.

| Aspectos recomendados para uso de <i>sketches</i> na ER                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ✓ Usar poucos elementos visuais, com formas simples e rótulos curtos, mantendo o foco no conceito e evitando distrações estéticas.                                      |
| ✓ Evidenciar as interações principais, representando apenas o essencial do fluxo de uso e evitando detalhes gráficos supérfluos.                                        |
| ✓ Capturar rapidamente as ideias surgidas em conversas, registrando o conteúdo por meio de fotos ou digitalizações para posterior análise.                              |
| ✓ Trabalhar em duplas ou trios, favorecendo a troca de perspectivas e a diversidade de soluções visuais.                                                                |
| ✓ Registrar múltiplas versões de um mesmo conceito, permitindo comparar alternativas e analisar a evolução das ideias ao longo das discussões.                          |
| ✓ Adicionar anotações curtas que expressem intenções, hipóteses ou dúvidas, enriquecendo a interpretação coletiva sem comprometer a simplicidade do esboço.             |
| ✓ Revisar os <i>sketches</i> ao final de cada oficina, identificando padrões de entendimento, convergências e lacunas que indiquem novos requisitos ou inconsistências. |

Quadro 34 – Boas práticas para uso de *sketches* na ER.  
Fonte: elaborado pelo autor (2025).

### 10.5.2.2 Storyboards

Os *storyboards* evoluem o conceito de *sketch* para uma **narrativa visual estruturada**, composta por uma sequência de quadros que descrevem a experiência do usuário ao interagir com o sistema. Cada quadro representa um **momento da jornada** — antes, durante e depois do uso —, contextualizando a motivação, a ação e o resultado percebido.

Segundo Laplante (2017), os *storyboards* são eficazes para **alinhar expectativas entre usuários e desenvolvedores**, pois combinam elementos visuais com narrativa textual, criando empatia e compreensão sobre o contexto de uso. Pichler (2010) também os recomenda como ferramenta de **visão de produto**, ao conectar objetivos de negócio a situações reais.

O Quadro 35 reúne práticas para construir e discutir storyboards com os stakeholders.

| Aspectos recomendados para uso de <i>storyboards</i> na ER                                                                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ✓ Representar personas e cenários de forma simples, utilizando anotações curtas que facilitem a compreensão da narrativa.                                              |
| ✓ Ilustrar situações de valor para o usuário, priorizando experiências significativas em vez de apenas funcionalidades técnicas.                                       |
| ✓ Usar sequências curtas (de 4 a 8 quadros) para preservar a clareza narrativa e evitar sobrecarga visual.                                                             |
| ✓ Incluir notas sobre emoções, frustrações e resultados esperados, reforçando empatia e foco na experiência humana.                                                    |
| ✓ Criar coletivamente com stakeholders e designers, promovendo empatia e coautoria sobre a jornada representada.                                                       |
| ✓ Validar com usuários representativos após a oficina, assegurando que a narrativa reflète comportamentos e necessidades reais.                                        |
| ✓ Utilizar <i>storyboards</i> como insumo para histórias de usuário, casos de uso e critérios de aceitação, garantindo rastreabilidade entre requisitos e experiência. |



#### Aspectos recomendados para uso de *storyboards* na ER

- ✓ Atualizar o *storyboard* sempre que o fluxo de interação mudar, mantendo coerência entre visão de uso, requisitos e produto.

Quadro 35 – Boas práticas para uso de storyboards na ER.

Fonte: elaborado pelo autor (2025).

### Exemplo

Exploração e representação da experiência de uso em um Sistema de Agendamento de Consultas Públicas

Durante uma oficina de descoberta de requisitos conduzida com profissionais de saúde e engenheiros de software, os participantes foram convidados a representar como imaginavam o funcionamento e o uso de um sistema público de agendamento de consultas médicas.

A atividade foi dividida em dois momentos complementares:

#### 1. Momento de concepção visual – *Sketch de interface*

Os participantes iniciaram com *sketches* rápidos (Figura 58), esboçando **as telas principais do sistema**: tela inicial com botão Agendar Consulta, seleção de data e horário, e tela de confirmação.

O objetivo era **tornar visíveis as ideias de navegação e estrutura básica da solução**, permitindo discutir alternativas de fluxo e identificar requisitos funcionais iniciais, como autenticação, feedback de confirmação e gerenciamento de horários disponíveis. Essa representação serviu como base para alinhar o entendimento sobre *o que o sistema deve permitir fazer*.

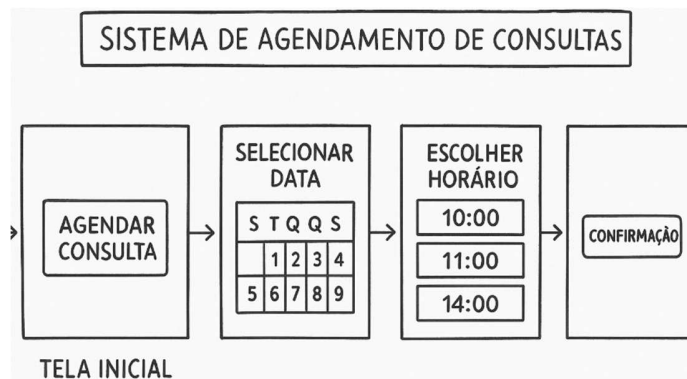


Figura 58 –Sketch de interface – Sistema de Agendamento de Consultas Públicas (exemplo).

Fonte: elaborado pelo autor (2025).

#### 2. Momento de contextualização – *Storyboard da jornada do cidadão*

Em seguida, a equipe criou um *storyboard* (Figura 59) para **representar o uso real do sistema no cotidiano do cidadão**.

A sequência de quadros ilustra a jornada desde o momento em que o usuário percebe a necessidade de marcar uma consulta até o comparecimento ao posto de saúde. As ilustrações trazem expressões e anotações que evidenciam **emoções, intenções e**

**resultados esperados**, permitindo refletir sobre aspectos de usabilidade e requisitos não funcionais (como simplicidade, clareza e satisfação).

O foco, aqui, desloca-se do sistema para **a experiência humana de uso**, complementando o entendimento obtido com o *sketch*.

A integração entre os dois artefatos — *sketch* e *storyboard* — possibilitou uma visão mais completa do problema e da solução: o primeiro estruturou a interface e as interações, enquanto o segundo revelou como essas interações se inserem na vida real do usuário. Ambos contribuíram para refinar requisitos, antecipar melhorias de design e criar uma base sólida para os próximos níveis de representação (protótipos e modelos semiformais).

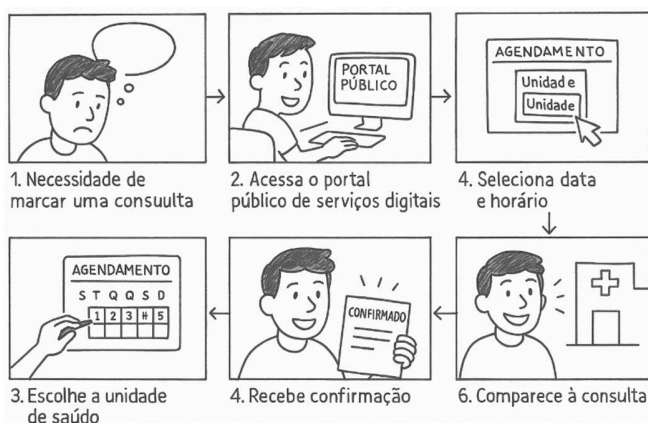


Figura 59 –Storyboard – Sistema de Agendamento de Consultas Públicas (exemplo).

Fonte: elaborado pelo autor (2025).

## Síntese do uso e integração

O Quadro 36 compara sketches e storyboards quanto ao propósito, ao foco da representação e à contribuição de cada um para o entendimento dos requisitos.

| Aspecto                                | Sketch                                                                   | Storyboard                                                                      |
|----------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Propósito principal</b>             | Explorar ideias e testar hipóteses de design ou fluxo funcional.         | Comunicar o contexto de uso e a experiência do usuário ao longo do tempo.       |
| <b>Foco da representação</b>           | O sistema: telas, componentes e interações.                              | O usuário: jornada, emoções, situações e objetivos.                             |
| <b>Natureza visual</b>                 | Estática — mostra estados do sistema.                                    | Sequencial — mostra ações e transições.                                         |
| <b>Uso típico</b>                      | Oficinas de ideação e descoberta de requisitos; discussões de interface. | Validação de cenários de uso; alinhamento com stakeholders sobre a experiência. |
| <b>Pergunta que ajuda a distinguir</b> | “Como o sistema pode funcionar?”                                         | “Como o usuário vive essa situação de uso?”                                     |

| Aspecto           | Sketch                                                               | Storyboard                                                                                     |
|-------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Complementaridade | Serve de insumo para detalhar funcionalidades e desenhar protótipos. | Serve de insumo para identificar requisitos não funcionais, jornadas e critérios de aceitação. |

Quadro 36 – Síntese do uso e integração de sketch e storyboard.  
Fonte: elaborado pelo autor (2025).

Esses artefatos podem servir de insumo direto para as representações do próximo nível — **semi-estruturadas**, apresentadas na subseção 10.5.3 —, nas quais as ideias passam a ser organizadas em estruturas visuais mais consistentes e utilizáveis como base para prototipagem e validação incremental.

### 10.5.3 Representações Semiestruturadas (organizacionais)

As representações semiestruturadas marcam a transição entre a exploração livre de ideias e a organização visual da solução. Diferentemente das representações intuitivas, que são predominantemente expressivas e abertas, as semiestruturadas já impõem algum grau de estrutura e coerência visual, buscando representar como o sistema pode se comportar ou se apresentar ao usuário.

Seu propósito é **refinar a comunicação entre a equipe técnica e os stakeholders**, traduzindo hipóteses e fluxos em artefatos que se aproximam da aparência e do funcionamento desejados, ainda sem o compromisso de implementação.

Nesse grupo, destacam-se duas técnicas complementares: **mockups** e **protótipos de baixa fidelidade**. Ambas são amplamente utilizadas na Engenharia de Requisitos para **visualizar soluções potenciais**, facilitar validação e promover consenso em torno de decisões de design.

#### 10.5.3.1 Mockups

Os *mockups* são **representações visuais estáticas** da interface de um sistema, geralmente elaboradas com maior nitidez e acabamento que os *sketches*, porém **sem interatividade**. Seu objetivo é **mostrar a aparência e o layout da aplicação**, servindo para discutir estética, hierarquia de informação e usabilidade básica.

Laplante (2017) define o *mockup* como um artefato que “permite visualizar o produto final de forma tangível e iterativa, sem a necessidade de código”. Já Sommerville (2019) o descreve como um instrumento de **validação de entendimento**, útil para verificar se os stakeholders reconhecem suas expectativas na estrutura apresentada.

O Quadro 37 reúne práticas para utilizar mockups na discussão da organização visual e das informações apresentadas ao usuário.

| Aspectos recomendados para uso de mockups na ER |                                                                                                                                        |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| ✓                                               | Manter o foco no layout e na organização da informação, evitando detalhar comportamentos dinâmicos que pertencem a etapas posteriores. |
| ✓                                               | Utilizar contrastes visuais simples para destacar áreas de conteúdo e interação, priorizando clareza e legibilidade.                   |
| ✓                                               | Apresentar versões alternativas de tela para estimular a comparação entre soluções e favorecer decisões colaborativas.                 |

| Aspectos recomendados para uso de mockups na ER |                                                                                                                                      |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| ✓                                               | Incluir rótulos e anotações curtas, contextualizando os elementos visuais sem comprometer a simplicidade do artefato.                |
| ✓                                               | Elaborar em conjunto com stakeholders, garantindo que a disposição visual reflita prioridades e expectativas de uso.                 |
| ✓                                               | Usar ferramentas acessíveis, analógicas ou digitais, para viabilizar iterações rápidas e coletivas.                                  |
| ✓                                               | Revisar os <i>mockups</i> após feedback dos usuários, ajustando nomenclaturas, agrupamentos e hierarquia visual.                     |
| ✓                                               | Empregar os <i>mockups</i> como base para protótipos de baixa fidelidade, mantendo consistência entre conteúdo e fluxo de interação. |

Quadro 37 – Boas práticas para uso de mockups na ER.  
Fonte: elaborado pelo autor (2025).

### 10.5.3.2 Protótipos de Baixa Fidelidade

Os protótipos de baixa fidelidade (*Low-Fidelity Prototypes*) são representações simples, muitas vezes interativas, que simulam **a navegação e o comportamento básico do sistema**. Ao contrário dos *mockups*, que são estáticos, esses protótipos permitem **explorar fluxos de interação** e testar compreensões sobre a dinâmica do produto antes de qualquer codificação.

Segundo Lauesen (2002), a principal vantagem dos protótipos de baixa fidelidade é **tornar o produto “real o suficiente” para provocar reação dos usuários**, estimulando insights sobre requisitos que poderiam ser negligenciados. Sommerville (2019) os classifica como *throwaway prototypes* — artefatos descartáveis cujo valor está em **facilitar a comunicação e a aprendizagem coletiva**, não em servir de base técnica para o desenvolvimento.

O Quadro 38 reúne práticas para explorar fluxos de interação com protótipos de baixa fidelidade e discutir o feedback obtido.

| Aspectos recomendados para uso de protótipos de baixa fidelidade na ER |                                                                                                              |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| ✓                                                                      | Focar na sequência de interação, priorizando a navegação entre telas e estados em vez da aparência estética. |
| ✓                                                                      | Representar transições com setas, cliques simulados ou trocas de papel para tornar o fluxo compreensível.    |
| ✓                                                                      | Permitir a interação do usuário, mesmo simbólica, usando cartões, post-its ou slides.                        |
| ✓                                                                      | Conduzir sessões de avaliação com usuários reais, registrando reações e comentários durante os testes.       |
| ✓                                                                      | Prototipar de forma incremental, adicionando elementos conforme o entendimento evolui.                       |
| ✓                                                                      | Discutir o feedback coletivamente, ajustando inconsistências entre expectativa e uso percebido.              |
| ✓                                                                      | Descartar o protótipo após o aprendizado, evitando confusão com entregáveis finais.                          |

Quadro 38 – Boas práticas para uso de protótipos de baixa fidelidade na ER.  
Fonte: elaborado pelo autor (2025).

### Exemplo

Validação de interface e fluxo em um Sistema de Agendamento de Consultas Públicas

Durante a sequência seguinte à oficina, a equipe evolui os *sketches* (Figura 58) e *storyboards* (Figura 59) iniciais para artefatos semiestruturados, com o objetivo de **validar o fluxo de interação e a organização da interface** do mesmo sistema de agendamento de consultas.

### 1. Momento de visualização estática – *Mockups*

Foram criados *mockups* digitais representando as telas principais (Figura 60): Tela Inicial, Escolha de Unidade de Saúde, Seleção de Data e Horário e Confirmação. A ênfase estava na **organização visual** e na clareza de rótulos e botões, permitindo discutir a coerência entre as informações e as expectativas dos usuários.

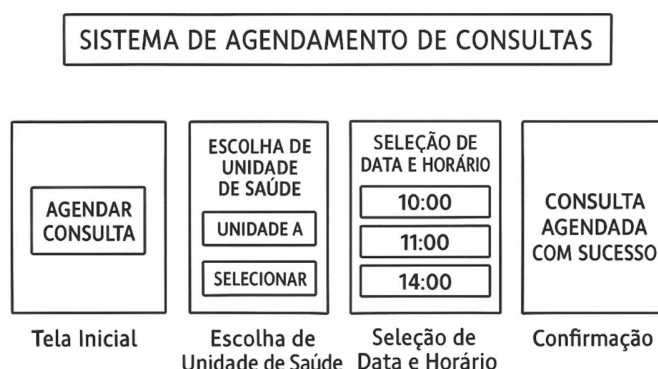


Figura 60 – Mockups de interface – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

### 2. Momento de simulação de uso – *Protótipos de baixa fidelidade*

Com base nos *mockups*, a equipe produziu um protótipo de baixa fidelidade (Figura 61) em slides interativos, simulando a navegação entre as telas e permitindo **testes de usabilidade informal** com cidadãos e profissionais de saúde. As observações coletadas foram utilizadas para ajustar a nomenclatura dos botões e simplificar etapas redundantes no fluxo de agendamento.

Figura 61 – Protótipos de baixa fidelidade – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

## Síntese do uso e integração

O Quadro 39 compara mockups e protótipos de baixa fidelidade, evidenciando o que cada representação permite discutir e como podem ser utilizadas em conjunto.

| Aspecto                         | Mockup                                                    | Protótipo de baixa fidelidade                          |
|---------------------------------|-----------------------------------------------------------|--------------------------------------------------------|
| Propósito principal             | Mostrar a aparência e o layout do sistema.                | Simular o fluxo e o comportamento básico de navegação. |
| Foco da representação           | Estrutura visual e hierarquia de informação.              | Sequência de interação e dinâmica entre telas.         |
| Natureza visual                 | Estática — mostra estados fixos.                          | Interativa — mostra transições e respostas do sistema. |
| Uso típico                      | Discussão de design, estética e organização da interface. | Validação de fluxo e compreensão de usabilidade.       |
| Pergunta que ajuda a distinguir | <i>“Como o sistema se apresenta?”</i>                     | <i>“Como o usuário interage com o sistema?”</i>        |
| Complementaridade               | Serve de base para a criação do protótipo.                | Permite testar o que foi definido no mockup.           |

Quadro 39 – Síntese do uso e integração de mockups e protótipos de baixa fidelidade.  
Fonte: elaborado pelo autor (2025).

O uso combinado de *mockups* e protótipos de baixa fidelidade permitiu refinar o entendimento das interações e validar a arquitetura da informação antes da prototipagem funcional, reduzindo retrabalho e fortalecendo o alinhamento entre áreas de negócio e desenvolvimento.

Esses resultados consolidam as representações semiestruturadas como etapa intermediária fundamental, preparando o terreno para as representações sistêmicas, apresentadas a seguir, que ampliam a análise para o contexto e os limites de atuação do sistema.

### 10.5.4 Representações Cognitivas (conceituais e colaborativas)

As representações cognitivas têm como foco **estruturar o pensamento coletivo e promover a construção compartilhada de significados**. Diferem das representações livres (10.5.2), que estimulam a criatividade e a ideação, e das semiestruturadas (10.5.3), que ilustram visualmente a solução.

Essas representações ocupam um espaço intermediário no processo de representação: são suficientemente livres para favorecer a reflexão e a colaboração, mas também organizadas conceitualmente, permitindo convergência e síntese entre ideias (POHL, 2010; LAUESEN, 2002).

Sua principal função é **revelar a lógica interna de entendimento do domínio e das necessidades**, integrando percepções dispersas em estruturas cognitivas de relação — seja por agrupamento semântico, por hierarquia de conceitos ou por associação de ideias. Essas representações são especialmente úteis em contextos de co-design, oficinas de priorização ou sessões de análise colaborativa de requisitos.

As técnicas mais comuns desse grupo são:

- Mapas de Afinidades
- Mapas Mentais

- Mapas Conceituais

#### 10.5.4.1 Mapas de Afinidades

São uma técnica de organização colaborativa de ideias, utilizada para **consolidar resultados de oficinas, entrevistas ou brainstormings**. A partir de post-its, anotações ou registros digitais, os participantes agrupam conceitos ou observações por similaridade temática, revelando padrões emergentes e categorias espontâneas (LAUESEN, 2002).

Essa técnica é particularmente valiosa em etapas de análise exploratória, pois permite perceber conexões entre requisitos latentes e identificar dimensões do problema ainda não estruturadas.

#### 10.5.4.2 Mapas Mentais

São representações **radiais** em torno de um conceito central. Utilizam palavras-chave, ramificações e ícones para expressar **relações de associação e decomposição hierárquica** entre ideias.

Na Engenharia de Requisitos, auxiliam na **organização de temas, funcionalidades e requisitos não funcionais**, sendo úteis em reuniões de síntese e priorização, quando é preciso alinhar entendimento entre equipes (POHL, 2010).

#### 10.5.4.3 Mapas Conceituais

Estruturam o conhecimento de maneira mais formal, utilizando **ligações rotuladas** (verbo + substantivo) para representar proposições (“Agendamento envolve Usuário”, “Notificação utiliza Canal”).

São especialmente adequados para **sistematizar o domínio do problema**, apoiar o registro e preparar insumos para modelagem ontológica, glossários ou catálogos de requisitos (SOMMERVILLE, 2019).

Embora distintas, essas técnicas formam uma cadeia cognitiva integrada: o **mapa de afinidades** organiza percepções brutas, o **mapa mental** as sintetiza em torno de temas, e o **mapa conceitual** as formaliza como conhecimento estruturado.

As representações cognitivas combinam práticas gerais (ver Quadro 40), aplicáveis a todo o grupo, com orientações específicas para cada técnica (ver Quadro 41).

| Prática geral                               | Descrição / Aplicação                                                                                                     |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Promover a coautoria ativa</b>           | Envolver múltiplos stakeholders na construção dos mapas, favorecendo trocas de perspectivas.                              |
| <b>Evoluir do divergente ao convergente</b> | Iniciar com ideias livres e progredir para estruturas hierárquicas ou semânticas consolidadas.                            |
| <b>Registrar contexto e data de criação</b> | Indicar quando e em que etapa do projeto o mapa foi produzido, para contextualizar decisões.                              |
| <b>Usar rótulos curtos e significativos</b> | Palavras-chave devem ser breves e claras, evitando ambiguidades e jargões técnicos.                                       |
| <b>Integrar resultados</b>                  | Relacionar os mapas cognitivos aos artefatos declarativos e estruturados subsequentes (histórias, requisitos, catálogos). |

Quadro 40 – Práticas gerais recomendadas para as representações cognitivas na ER.

Fonte: elaborado pelo autor (2025).

| Técnica                   | Boas práticas específicas                                                                                               | Finalidade principal                                                                                     |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Mapa de Afinidades</b> | Utilizar frases curtas; permitir reorganização livre; identificar categorias consensuais e nomeá-las colaborativamente. | Sintetizar ideias dispersas e identificar padrões emergentes após oficinas e entrevistas.                |
| <b>Mapa Mental</b>        | Usar ramificações coloridas e hierarquias visuais; manter um único conceito central; revisar o mapa coletivamente.      | Organizar e priorizar temas, requisitos ou funcionalidades de forma visual e intuitiva.                  |
| <b>Mapa Conceitual</b>    | Expressar relações com conectores verbais (“é”, “usa”, “depende de”); validar proposições com especialistas de domínio. | Formalizar o conhecimento do domínio e estabelecer uma base semântica para a documentação de requisitos. |

Quadro 41 – Práticas específicas recomendadas para as representações cognitivas na ER.

Fonte: elaborado pelo autor (2025).

## Exemplo

Na continuação do projeto do Sistema de Agendamento de Consultas Públicas, após as etapas de contextualização e ideação, a equipe utilizou representações cognitivas para **organizar e consolidar os resultados das oficinas**.

Primeiro, foi construído um **mapa de afinidades** (Figura 62) com as observações registradas nas entrevistas com cidadãos, atendentes e gestores. As ideias foram agrupadas em temas como Acesso à Informação, Tempo de Espera, Integração de Sistemas e Comunicação com o Usuário. Esse agrupamento revelou padrões recorrentes de insatisfação e oportunidades de melhoria.

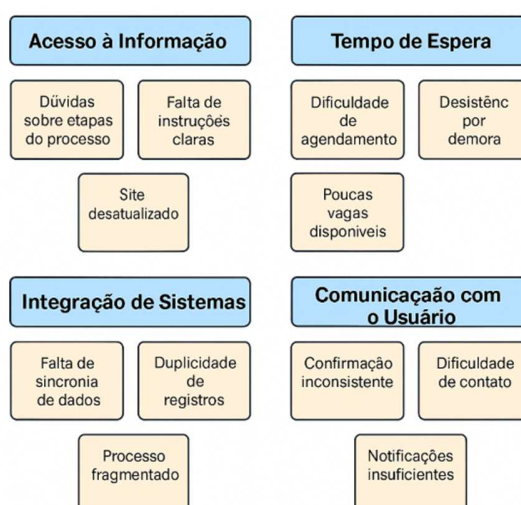


Figura 62 – Mapa de afinidades – Sistema de Agendamento de Consultas Públicas (exemplo).

Fonte: elaborado pelo autor (2025).

Em seguida, elaborou-se um **mapa mental** (Figura 63) a partir desses grupos, colocando o termo “Agendamento Público” no centro e ramificações para Fluxo de Atendimento, Notificações, Usuários e Regras de Negócio. Essa representação permitiu visualizar a abrangência do domínio e hierarquizar prioridades.



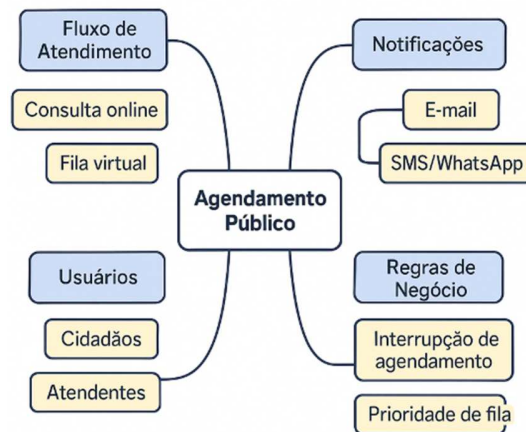


Figura 63 – Mapa de mental – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Por fim, a equipe criou um **mapa conceitual** (Figura 64) que formalizou os relacionamentos principais: “Cidadão solicita Consulta”, “Sistema envia Notificação”, “Módulo de Confirmação atualiza Agenda”, “Gestor monitora Relatórios”. Esse artefato consolidou a base semântica utilizada para a documentação de requisitos funcionais e não funcionais, além de apoiar a definição de glossário e critérios de aceitação.

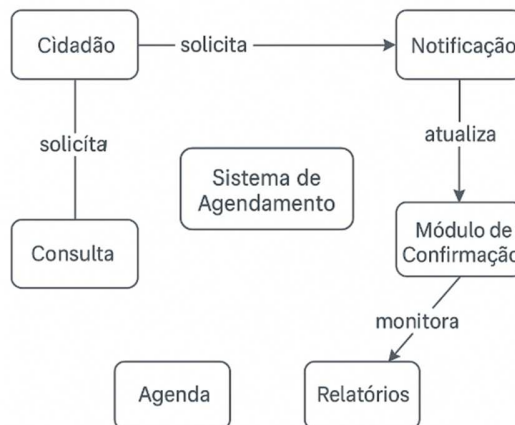


Figura 64 – Mapa de conceitual – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

O conjunto das três representações cognitivas permitiu à equipe transformar informações dispersas em **estrutura conceitual coerente**, facilitando a comunicação entre os participantes e a transição para representações mais formais.

### Síntese do uso e integração

O Quadro 42 compara mapas de afinidades, mapas mentais e mapas conceituais, mostrando seus diferentes focos e como uma representação pode fornecer insumos para outra.

| Aspecto                                | Mapa de Afinidades                                                               | Mapa Mental                                                             | Mapa Conceitual                                                                   |
|----------------------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Propósito principal</b>             | Agrupar ideias e percepções brutas para revelar padrões e categorias emergentes. | Organizar e priorizar temas ou requisitos de modo visual e intuitivo.   | Estruturar e formalizar o conhecimento do domínio e das relações entre conceitos. |
| <b>Foco da representação</b>           | Ideias e observações obtidas nas oficinas e entrevistas.                         | Tópicos e funcionalidades relacionados a um conceito central.           | Conceitos e proposições semânticas que expressam o domínio do problema.           |
| <b>Natureza visual</b>                 | Agrupamento livre e colaborativo.                                                | Estrutura radial com ramificações hierárquicas.                         | Rede de nós e conexões rotuladas.                                                 |
| <b>Uso típico</b>                      | Análise de dados qualitativos; síntese de brainstorming e feedback de usuários.  | Sessões de priorização e alinhamento conceitual entre equipes.          | Documentação de domínio, elaboração de glossários e validação semântica.          |
| <b>Pergunta que ajuda a distinguir</b> | <i>“Quais ideias se relacionam e formam um mesmo tema?”</i>                      | <i>“Como esses temas se organizam em torno de um conceito central?”</i> | <i>“Quais conceitos se relacionam e como se conectam logicamente?”</i>            |
| <b>Complementaridade</b>               | Fornecer insumos para o Mapa Mental.                                             | Alimenta o Mapa Conceitual.                                             | Consolida e formaliza o conhecimento para documentação e validação de requisitos. |

Quadro 42 – Síntese e integração das representações cognitivas.  
Fonte: elaborado pelo autor (2025).

Essas representações completam o ciclo das técnicas de representação de requisitos apresentadas na seção 10.5, fechando o percurso que vai do entendimento do contexto (sistêmico) à estruturação conceitual do conhecimento, preparando o terreno para as atividades de verificação, validação e evolução apresentadas nos capítulos seguintes.

## 10.6 Representações Semiformais

### 10.6.1 Introdução e natureza das representações semiformais

As representações semiformais ocupam uma posição intermediária entre as linguagens informais e as formais, combinando elementos gráficos e textuais que reforçam a precisão da comunicação sem exigir o rigor matemático dos modelos formais (SOMMERVILLE, 2019; POHL, 2010). O termo “semiformais” designa técnicas baseadas em notações estruturadas, com sintaxe parcialmente definida e semântica suficientemente clara para reduzir ambiguidades — mas ainda compreensível a públicos não técnicos. Assim, elas constituem **um ponto de convergência entre entendimento humano e precisão técnica**, promovendo a transição gradual entre as representações conceituais e as especificações verificáveis.

De acordo com o *SWEBOK v4.0* (IEEE, 2024), tais representações pertencem ao domínio da **modelagem de requisitos**, cuja função é “fornecer meios estruturados e verificáveis para a descrição de comportamento e estrutura do sistema sem ainda caracterizar sua implementação”. Essa fronteira, explicitada na seção 10.3 deste livro, delimita a diferença entre representar o *que* o sistema deve fazer (requisitos) e projetar o *como* ele o fará (design). Desse modo, diagramas

semiformais não são artefatos de projeto, mas **meios de representação estruturada do conhecimento de requisitos**, úteis para apoiar validação, rastreabilidade e análise de consistência.

A literatura (DICK; HULL; JACKSON, 2017; LAUESEN, 2002; LEFFINGWELL, 2011) identifica três papéis centrais dessas representações:

1. **Cognitivo:** apoiar a compreensão compartilhada entre stakeholders e engenheiros de requisitos.
2. **Comunicacional:** estabelecer uma linguagem visual comum entre domínios técnico e de negócio.
3. **Analítico:** permitir verificação de coerência, identificação de dependências e rastreabilidade entre requisitos.

Em contextos ágeis, as representações semiformais são aplicadas de modo incremental e evolutivo — “*just enough, just in time*” (HRUSCHKA et al., 2022) — complementando narrativas como histórias de usuário e critérios de aceitação. Quando utilizadas colaborativamente, tornam-se instrumentos de aprendizado coletivo sobre o comportamento do sistema e suas restrições operacionais.

### 10.6.2 Modelos e notações

As representações semiformais baseiam-se em **notações gráficas padronizadas**, isto é, linguagens visuais com sintaxe e semântica parcialmente definidas, que permitem construir diagramas e modelos de requisitos de forma estruturada.

Essas notações não têm caráter de implementação, mas de **expressão conceitual e comunicacional**, situando-se entre a linguagem natural e as linguagens formais (POHL, 2010; SWEBOK v4.0, 2024).

Entre as notações utilizadas na Engenharia de Requisitos, destacam-se:

- **UML (Unified Modeling Language):** notação amplamente usada para representar comportamento e estrutura do sistema. Dentro da Engenharia de Requisitos, utiliza-se apenas um subconjunto restrito de seus diagramas, como Casos de Uso, Atividades e Diagramas de Contexto, evitando diagramas estruturais ou de implementação (como classes, componentes, sequência e deployment), que pertencem ao domínio do design.
- **SysML (Systems Modeling Language):** linguagem de modelagem voltada a sistemas complexos e multidisciplinares. Na ER, o diagrama de requisitos pode representar requisitos e relações como derivação, refinamento, composição e verificação, favorecendo a visualização de dependências e vínculos entre diferentes níveis (DICK; HULL; JACKSON, 2017; IEEE, 2024).
- **BPMN (Business Process Model and Notation):** notação específica para modelagem de processos de negócio, aplicada na Engenharia de Requisitos para compreender e validar fluxos organizacionais e interações entre papéis.
- **DFD (Data Flow Diagram Notation):** notação tradicional para representar transformações e movimentos de dados no sistema, associada a abordagens estruturadas.

A partir dessas notações, constroem-se diagramas e modelos que traduzem requisitos de forma estruturada, como casos de uso, atividades, fluxos de dados, processos de negócio e modelos de requisitos.

Cada diagrama é um artefato resultante da aplicação de uma notação a um contexto específico de análise, podendo coexistir com representações textuais e narrativas.

Em síntese:

- A **notação** define o vocabulário visual e as regras;
- O **modelo** representa um conjunto organizado de conceitos sob essa notação; e
- O **artefato** é o produto concreto, documentado e validado durante o processo de ER.

A seguir, são apresentadas as principais técnicas e tipos de diagramas construídos com essas notações, dentro dos limites estabelecidos pelo *SWEBOK v4.0* para a atividade de representação de requisitos.

#### 10.6.2.1 Diagramas de Casos de Uso

Os diagramas de casos de uso, construídos com base na notação UML, representam as interações entre atores externos e o sistema, permitindo visualizar o propósito e as fronteiras de suas funcionalidades (JACOBSON et al., 2024).

Essa representação oferece uma forma simples e compartilhável de responder à pergunta “**quem faz o quê com o sistema**”, estabelecendo uma ponte direta entre os objetivos dos usuários e os comportamentos esperados.

Ao reduzir a complexidade técnica em favor de uma linguagem visual acessível, o diagrama de casos de uso facilita a validação colaborativa e o alinhamento de expectativas (DICK; HULL; JACKSON, 2017; LAUESEN, 2002).

Em contextos ágeis, ele é frequentemente utilizado em conjunto com narrativas textuais e critérios de aceitação, servindo como um mapa funcional do produto. Sua utilidade é especialmente evidente nas fases iniciais da Engenharia de Requisitos, quando o foco está na definição do escopo e das fronteiras do sistema, não em sua estrutura interna. Diagramas mais simples e genéricos podem ser empregados para comunicar a visão geral, enquanto versões expandidas descrevem fluxos principais e alternativos de interação (LEFFINGWELL, 2011; HRUSCHKA et al., 2022).

O Quadro 43 reúne recomendações para manter os diagramas de casos de uso legíveis e coerentes com as descrições complementares dos requisitos.

| Aspecto           | Recomendações práticas                                                                                    |
|-------------------|-----------------------------------------------------------------------------------------------------------|
| Clareza de escopo | Identifique atores reais e nomeie casos com verbo + objeto (“Emitir relatório”, “Cadastrar solicitação”). |
| Coerência         | Mantenha coerência semântica entre nomes, relações e descrições complementares.                           |
| Consenso          | Revise coletivamente o diagrama em sessões de validação.                                                  |

| Aspecto                   | Recomendações práticas                                                  |
|---------------------------|-------------------------------------------------------------------------|
| Documentação complementar | Associe casos de uso a narrativas textuais e critérios Given–When–Then. |
| Legibilidade              | Evite sobrecarga visual e preserve hierarquia entre atores e casos.     |

Quadro 43 – Recomendações práticas para diagramas de casos de uso.  
Fonte: elaborado pelo autor (2025).

### 10.6.2.2 Diagramas de Atividades

Os **diagramas de atividades** expressam graficamente o fluxo de ações e decisões que ocorrem em um processo ou requisito funcional, descrevendo a sequência de eventos e as condições que determinam caminhos alternativos (SOMMERVILLE, 2019; LEFFINGWELL, 2011).

Essa representação amplia a compreensão sobre **como o sistema ou os usuários se comportam ao longo de um processo**, permitindo que regras de negócio, exceções e dependências sejam discutidas visualmente com todos os participantes.

A clareza visual desses diagramas estimula a validação coletiva, funcionando como um artefato de consenso sobre o comportamento esperado.

Quando construídos de forma incremental, esses modelos permitem capturar desde fluxos simples e lineares até execuções paralelas ou condicionais, mantendo-se sempre no domínio conceitual — sem descrever algoritmos ou estruturas técnicas (DICK; HULL; JACKSON, 2017; LAUESEN, 2002).

Essa característica faz do diagrama de atividades um elo natural entre o pensamento de processo do negócio e a análise de requisitos do produto.

O Quadro 44 reúne recomendações para representar ações, responsabilidades, decisões e exceções em diagramas de atividades.

| Aspecto             | Recomendações práticas                                               |
|---------------------|----------------------------------------------------------------------|
| Ações observáveis   | Modele apenas atividades perceptíveis ao usuário ou negócio.         |
| Responsabilidades   | Utilize <i>swimlanes</i> para distinguir papéis e responsabilidades. |
| Decisões e exceções | Explicite condições de transição em linguagem natural.               |
| Iteração            | Valide o fluxo com stakeholders durante oficinas de revisão.         |
| Consistência visual | Mantenha coerência entre eventos de início e término.                |

Quadro 44 – Recomendações práticas para diagramas de atividades.  
Fonte: elaborado pelo autor (2025).

### 10.6.2.3 Diagramas de Contexto e de Fluxo de Dados (DFD)

Os **diagramas de contexto** e os **diagramas de fluxo de dados (DFD)** utilizam uma notação própria da Engenharia Estruturada e permanecem amplamente reconhecidos pela clareza com que representam **as trocas de informações entre o sistema e o ambiente externo** (LAUESEN, 2002; POHL, 2010).

O diagrama de contexto apresenta uma visão global das fronteiras do sistema, enquanto os DFDs detalham progressivamente as transformações internas de dados, permitindo compreender como a informação entra, é processada e sai do sistema.

Essa abordagem é particularmente adequada em sistemas centrados em informação, bases de dados corporativas e integrações entre módulos, nos quais é essencial mapear fluxos e dependências de forma sintética.

Ao manter o foco no conteúdo e não na execução, o DFD evita o risco de antecipar decisões de design, permanecendo dentro do escopo da Engenharia de Requisitos.

Em projetos mais complexos, é comum o uso de diferentes níveis de detalhamento (0, 1, 2...), que aprofundam a decomposição conforme a complexidade funcional (DICK; HULL; JACKSON, 2017).

O Quadro 45 reúne recomendações para construir diagramas de contexto e de fluxo de dados com clareza e coerência entre níveis de detalhamento.

| Aspecto                | Recomendações práticas                                                             |
|------------------------|------------------------------------------------------------------------------------|
| Níveis de abstração    | Inicie pelo diagrama de contexto e decompõe gradualmente.                          |
| Clareza de rotulação   | Nomeie processos com verbos de transformação (“Validar Dados”, “Gerar Relatório”). |
| Interface clara        | Destaque entidades externas e trocas de informação.                                |
| Coerência entre níveis | Garanta correspondência entre entradas e saídas.                                   |
| Simplicidade           | Evite sobreposição de fluxos e priorize legibilidade.                              |

Quadro 45 – Recomendações práticas para diagramas contexto e fluxo de dados.

Fonte: elaborado pelo autor (2025).

#### 10.6.2.4 Modelos de Processos de Negócio (BPMN)

Os **modelos de processos de negócio**, baseados na notação BPMN (Business Process Model and Notation), são amplamente utilizados para descrever **como atividades humanas e automatizadas se articulam** em fluxos de trabalho organizacionais (POHL, 2010; HRUSCHKA et al., 2022).

Esses modelos são particularmente eficazes para representar interações entre áreas, papéis e sistemas, conectando requisitos de negócio e requisitos de sistema de forma coerente e auditável.

A modelagem em BPMN favorece a construção de uma visão compartilhada sobre o funcionamento real do processo, destacando eventos, decisões e colaborações entre atores. Sua aplicação na Engenharia de Requisitos é valiosa quando há necessidade de compreender **o contexto organizacional e as dependências entre atividades humanas e automatizadas**, funcionando como um ponto de convergência entre a análise de processos e a especificação de requisitos funcionais (SOMMERVILLE, 2019; LEFFINGWELL, 2011).

Para preservar seu caráter semiformal, a modelagem deve limitar-se à descrição conceitual dos fluxos, evitando a representação de componentes técnicos ou mecanismos de execução.

O Quadro 46 reúne recomendações para usar modelos de processos de negócio na compreensão das atividades, dos papéis e de suas relações com os requisitos.

| Aspecto       | Recomendações práticas                                                        |
|---------------|-------------------------------------------------------------------------------|
| Foco em valor | Cada processo deve representar um objetivo de negócio claro.                  |
| Uso racional  | Empregue elementos básicos (eventos, atividades, gateways, pools, mensagens). |

| Aspecto      | Recomendações práticas                             |
|--------------|----------------------------------------------------|
| Modularidade | Separe subprocessos e mantenha diagramas enxutos.  |
| Participação | Modele processos com representantes de cada papel. |
| Documentação | Relacione cada atividade a requisitos funcionais.  |

Quadro 46 – Recomendações práticas para modelos de processos de negócio.  
Fonte: elaborado pelo autor (2025).

#### 10.6.2.5 Modelos De Requisitos Em SysML

Na Engenharia de Requisitos, o diagrama de requisitos permite representar requisitos e relações como derivação, refinamento, composição e verificação, tornando visíveis dependências entre elementos e diferentes níveis de abstração (POHL, 2010; SOMMERVILLE, 2019).

Esse tipo de representação é particularmente útil em sistemas complexos, regulados ou críticos, nos quais relações entre requisitos precisam ser explicitadas de forma consistente. Ao organizar graficamente essas relações, o modelo pode apoiar análise de impacto, discussão de coerência e rastreabilidade, sem que essas finalidades dependam exclusivamente da notação.

O uso do SysML deve permanecer orientado pelo propósito da representação. Diagramas de requisitos podem ser suficientes quando o interesse está nas relações entre requisitos; outros diagramas da linguagem podem ser pertinentes quando ajudam a tornar comportamentos, estados ou contextos relevantes para os requisitos mais compreensíveis, sem deslocar o foco para decisões detalhadas de projeto (DICK; HULL; JACKSON, 2017; IEEE, 2024).

O critério para empregar outros diagramas da SysML não é simplesmente o tipo de diagrama, mas sua contribuição para tornar aspectos relevantes dos requisitos mais compreensíveis. Diagramas de estados, blocos ou outras representações podem ser utilizados quando ajudam a explicitar comportamento, contexto, relações ou restrições pertinentes ao entendimento dos requisitos, desde que o foco não se desloque para decisões detalhadas de projeto (DICK; HULL; JACKSON, 2017; IEEE, 2024).

O Quadro 47 reúne recomendações para explicitar relações entre requisitos e manter o modelo útil à análise, à comunicação e à rastreabilidade.

| Aspecto             | Recomendações práticas                                                 |
|---------------------|------------------------------------------------------------------------|
| Identificação       | Atribua identificadores únicos e descritivos a cada requisito.         |
| Relações explícitas | Utilize relações “derive”, “refine”, “satisfy” e “verify”.             |
| Integração          | Vincule requisitos a especificações textuais e critérios de aceitação. |
| Coerência           | Atualize periodicamente o modelo diante de mudanças.                   |
| Comunicação         | Simplifique visualizações para revisão com stakeholders.               |

Quadro 47 – Recomendações práticas para modelos de requisitos.  
Fonte: elaborado pelo autor (2025).

#### Exemplo

Dando continuidade ao exemplo utilizado nas seções anteriores, o Sistema de Agendamento de Consultas Públicas evoluiu da fase de representações informais — centradas em esboços, mapas conceituais e modelos de fronteira — para a construção de **representações semiformais**, que ampliaram a precisão e a rastreabilidade dos requisitos levantados.

O objetivo dessa etapa foi transformar o entendimento visual e narrativo, elaborado em oficinas colaborativas, em **modelos estruturados** capazes de expressar comportamentos, fluxos e relações entre atores de forma verificável.

A equipe iniciou pela criação de um **Diagrama de Casos de Uso (UML)** (Figura 65), delimitando o escopo funcional do sistema e as interações principais. Foram identificados três atores: *Cidadão*, *Servidor Público* e *Sistema de Notificações*. Cada um se relaciona com o sistema por meio de casos como *Agendar Atendimento*, *Confirmar Agendamento* e *Cancelar Solicitação*. Esse diagrama permitiu estabelecer as fronteiras do sistema e serviu como referência visual para as histórias de usuário e critérios de aceitação, tornando explícito o que estava dentro e fora do domínio de responsabilidade do produto (JACOBSON et al., 2024).

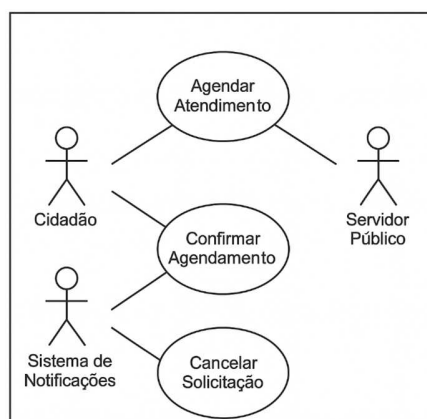


Figura 65 – Diagrama de caso de uso – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Com base nesse escopo, foi elaborado um **Diagrama de Atividades** (Figura 66), que representou o fluxo principal do caso *Agendar Atendimento*. O modelo iniciou com o evento “Solicitação de horário” e seguiu por etapas como *Selecionar Serviço*, *Verificar Disponibilidade*, *Confirmar Data/Hora* e *Gerar Notificação*. As decisões condicionais — como “Horário disponível?” e “Cidadão confirmou o agendamento?” — foram modeladas como *gateways*, evidenciando caminhos alternativos de execução. Esse diagrama permitiu validar as regras de negócio, revelar exceções operacionais e apoiar a construção de cenários de teste (SOMMERVILLE, 2019; LEFFINGWELL, 2011).





Figura 66 – Diagrama de atividades – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Em paralelo, a equipe construiu um **Diagrama de Contexto** (Figura 67), representando o sistema *Agendamento Público* como um único processo central conectado às entidades externas *Portal do Cidadão*, *Sistema de Notificações* e *Banco de Dados Institucional*. Esse modelo sintetizou as trocas de informação — requisições, confirmações e cancelamentos —, funcionando como mapa de interfaces e reforçando a coerência entre os requisitos funcionais identificados (LAUESEN, 2002).



Figura 67 – Diagrama de contexto – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Para alguns fluxos específicos, esse diagrama foi refinado em um **DFD de nível 1** (Figura 68), detalhando a transformação dos dados *Solicitação* → *Agendamento* → *Confirmação* e permitindo compreender como as informações circulavam entre módulos.

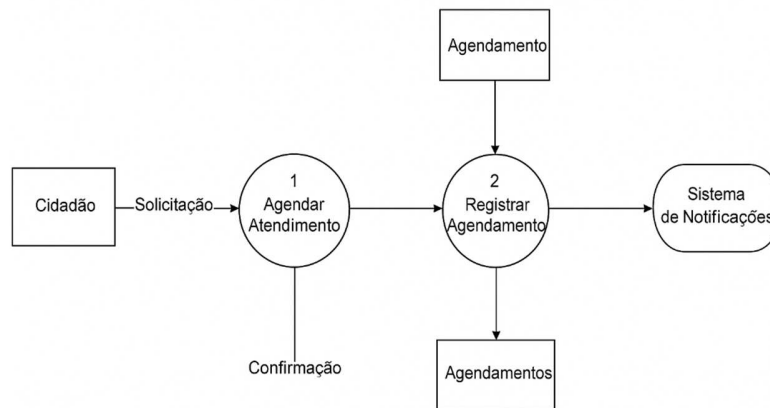


Figura 68 – Diagrama de fluxo de dados – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Posteriormente, o time utilizou a **notação BPMN** para representar o processo organizacional completo de atendimento, integrando ações humanas e automatizadas (Figura 69). O modelo mostrou o percurso do *Cidadão* (solicitação e confirmação), do *Servidor* (validação e reagendamento) e do *Sistema* (envio de notificações e registro de logs). Essa visão de processo favoreceu o alinhamento com as equipes administrativas e revelou pontos críticos, como a necessidade de sincronização de agendas entre unidades, permitindo que melhorias fossem sugeridas antes mesmo da implementação (HRUSCHKA et al., 2022; POHL, 2010).

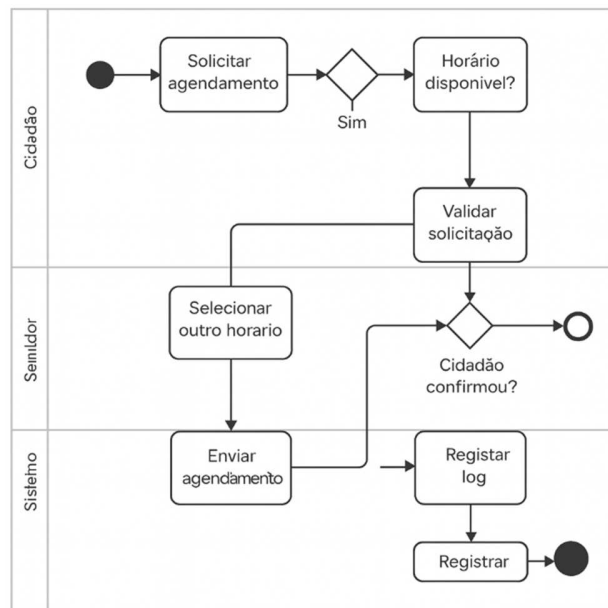


Figura 69 – Modelo BPMN – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Por fim, foi criado um modelo de requisitos em SysML (Figura 70) para tornar explícitas relações entre os requisitos identificados. Cada requisito funcional (*RF01 – Registrar Agendamento*, *RF02 – Enviar Notificação*, *RF03 – Consultar Histórico*) e não funcional (*RNF01 –*

*Disponibilidade mínima de 99%, RNF02 – Tempo máximo de resposta de 2s*) foi representado como elemento individual, associado às relações pertinentes entre os itens.

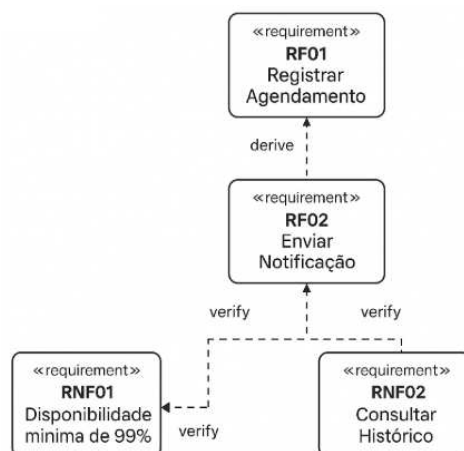


Figura 70 – Modelo de requisitos (SysML) – Sistema de Agendamento de Consultas Públicas (exemplo).  
Fonte: elaborado pelo autor (2025).

Esse modelo consolidou o vínculo entre as necessidades do cidadão e as propriedades verificáveis do sistema, servindo de base para futuras atividades de validação e auditoria (SOMMERVILLE, 2019; IEEE, 2024).

Esse conjunto de representações evidencia o papel das notações semiformais como instrumentos de convergência cognitiva: cada modelo foca um aspecto do domínio, mas todos se articulam em torno de um mesmo núcleo de sentido (ver Quadro 48).

O resultado é um ambiente de comunicação estruturada, no qual os requisitos deixam de ser apenas descrições textuais e passam a constituir uma rede visual e lógica de relações que apoia o raciocínio coletivo, a validação incremental e a evolução do produto.

| Notação      | Tipo de modelo                 | Aplicação no exemplo                            | Contribuição principal                                        |
|--------------|--------------------------------|-------------------------------------------------|---------------------------------------------------------------|
| <b>UML</b>   | Diagrama de Casos de Uso       | Delimitação de escopo e identificação de atores | Compreensão inicial do sistema e comunicação com stakeholders |
| <b>UML</b>   | Diagrama de Atividades         | Detalhamento do fluxo de agendamento            | Validação de regras e identificação de exceções               |
| <b>DFD</b>   | Diagrama de Contexto e nível 1 | Representação das trocas de informação          | Clareza sobre fronteiras e integrações externas               |
| <b>BPMN</b>  | Modelo de Processo de Negócio  | Descrição do fluxo organizacional               | Alinhamento entre negócio e sistema                           |
| <b>SysML</b> | Modelo de Requisitos           | Estruturação e rastreabilidade dos requisitos   | Coerência semântica e base para verificação                   |

Quadro 48 – Correlação entre as representações semiformais e seus objetivos no exemplo Sistema de Agendamento de Consultas Públicas.  
Fonte: elaborado pelo autor (2025).

A análise comparativa apresentada no Quadro 48 mostra que, embora cada notação possua uma finalidade específica, o verdadeiro valor das representações semiformais emerge da integração entre elas.

Enquanto os diagramas em UML privilegiam a compreensão das interações e dos comportamentos, o DFD enfatiza o fluxo de informações e as fronteiras do sistema, e a BPMN amplia a visão para o nível organizacional, conectando pessoas, sistemas e processos.

Por fim, o SysML complementa essa rede conceitual ao tornar explícitas relações entre requisitos funcionais e não funcionais, oferecendo uma visão que pode apoiar coerência, análise de impacto, rastreabilidade e V&V.

A seguir, o Quadro 49 sintetiza essas técnicas de forma comparativa, destacando suas diferenças de propósito, foco e natureza visual, além de evidenciar como se complementam na formação de uma visão integrada do sistema.

### 10.6.3 Síntese do uso e integração

| Aspecto                                | Casos de Uso (UML)                                                                            | Atividades (UML)                                                   | DFD / Contexto                                                                      | Processos de Negócio (BPMN)                                          | Requisitos (SysML)                                                           |
|----------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Propósito principal</b>             | Representar interações entre atores e o sistema, definindo fronteiras e objetivos funcionais. | Descrever o fluxo de ações e decisões de um processo ou requisito. | Mostrar o trânsito e a transformação de informações entre o sistema e seu ambiente. | Mapear atividades humanas e automatizadas em fluxos de negócio.      | Estruturar requisitos em hierarquias e relações de rastreabilidade.          |
| <b>Foco da representação</b>           | Relações de uso e valor percebido por cada ator.                                              | Comportamento e sequência de ações observáveis.                    | Dados e interfaces informacionais.                                                  | Processos organizacionais e interações entre papéis.                 | Dependências, derivação e verificação de requisitos.                         |
| <b>Natureza visual</b>                 | Diagrama de elipses (casos) e atores externos.                                                | Fluxograma orientado a decisões e paralelismos.                    | Modelo de blocos e setas de fluxo de dados.                                         | <i>Lanes</i> e <i>pools</i> com tarefas, eventos e <i>gateways</i> . | Blocos hierárquicos com ligações <i>derive</i> e <i>verify</i> .             |
| <b>Uso típico</b>                      | Delimitação de escopo e elicitação inicial de funcionalidades.                                | Validação de regras de negócio e identificação de exceções.        | Mapeamento de entradas, saídas e integrações de dados.                              | Compreensão e análise de processos de negócio.                       | Rastreabilidade e consistência entre requisitos funcionais e não funcionais. |
| <b>Pergunta que ajuda a distinguir</b> | <i>“Quem faz o quê com o sistema?”</i>                                                        | <i>“Como as ações se encaixam e se decidem?”</i>                   | <i>“Por onde as informações entram, circulam e saem?”</i>                           | <i>“Como o processo ocorre e quem participa dele?”</i>               | <i>“Como os requisitos se relacionam e se verificam mutuamente?”</i>         |
| <b>Complementaridade</b>               | Fornecer a visão inicial das interações e limites do sistema.                                 | Detalha os fluxos internos de execução e comportamento.            | Explicita as trocas informacionais que sustentam o processo.                        | Integra ações humanas e automatizadas no contexto organizacional.    | Consolida a rastreabilidade e a verificação de requisitos.                   |

Quadro 49 – Síntese e integração das representações semiformais.  
Fonte: elaborado pelo autor (2025).

A integração dessas notações reforça que nenhuma técnica isolada é capaz de representar a totalidade de um produto. Quando articuladas de modo coerente, diferentes representações ampliam a visibilidade sobre comportamentos, fluxos, dados, processos e relações entre requisitos, oferecendo perspectivas complementares para análise e comunicação.

As representações semiformais funcionam, assim, como um ponto de equilíbrio entre acessibilidade e estrutura, reduzindo parte da ambiguidade da linguagem natural sem exigir o rigor matemático das representações formais.

Esse continuum conduz às representações formais, nas quais propriedades de estado, comportamento, restrição ou tempo podem ser descritas com sintaxe e semântica matematicamente definidas. O ganho esperado não é uma garantia automática de correção, mas maior possibilidade de análise rigorosa, checagem de propriedades e produção de evidências em pontos nos quais a ambiguidade ou o custo do erro justificam esse rigor (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).

## 10.7 Representações Formais

### 10.7.1 Introdução e natureza das representações formais

As representações formais constituem o nível de maior rigor na atividade de representação de requisitos, pois adotam uma base matemática explícita para descrever propriedades de comportamento, estado, restrições e relações entre elementos do sistema. Enquanto as representações semiformais reduzem ambiguidades por meio de notações estruturadas (como UML, BPMN e SysML), as representações formais reduzem ambiguidades porque atribuem **semântica matemática** à especificação, permitindo análise rigorosa de consistência, completude parcial e correção de propriedades críticas (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).

Esse rigor é especialmente relevante quando a interpretação “plausível” não é suficiente. Em determinados contextos, não basta que stakeholders e equipe técnica “concordem” sobre o significado provável de um requisito; é necessário assegurar que o requisito não induza contradições lógicas, não permita comportamentos perigosos e não deixe espaço para variações que comprometam segurança, conformidade ou confiabilidade. É por essa razão que métodos formais aparecem historicamente associados a sistemas de alta criticidade (p.ex., controle aeronáutico, nuclear, armamentos e sistemas embarcados), nos quais a eliminação de classes de erro por verificação rigorosa é um objetivo explícito (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).

É importante distinguir dois sentidos de “formalidade” frequentemente confundidos. Em um primeiro sentido, formal pode significar **controle de processo**, governança e disciplina de mudanças (p.ex., baselines e change control). Em outro, formal significa **formalização da linguagem de representação**, com sintaxe e semântica matematicamente definidas. Um projeto pode ser altamente formal no processo e, ainda assim, representar requisitos apenas em linguagem natural; e pode, ao contrário, aplicar formalização matemática em partes críticas mesmo em um ambiente incremental, desde que haja clareza de objetivo e escopo (CROWDER; HOFF, 2022; LAPLANTE, 2017).

No continuum de representação adotado neste capítulo, as representações formais não substituem as demais, mas acrescentam rigor a recortes em que ambiguidades, inconsistências ou propriedades críticas justificam maior precisão. Aplicadas seletivamente, elas permitem explicitar condições e propriedades de modo analisável e, em determinados casos, submetê-las a prova ou checagem sistemática, fortalecendo a verificabilidade sem pressupor que todo o sistema precise ser formalizado (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).

### 10.7.2 Quando formalizar: critérios de decisão e delimitação de escopo

A principal decisão não é “usar ou não usar” formalização, mas **o que formalizar, com qual objetivo e até que profundidade**. A formalização integral do conjunto de requisitos raramente é justificável em projetos comuns, pois exige competência especializada, esforço adicional e tempo de análise. Por isso, a literatura recomenda a formalização **seletiva**, aplicada a requisitos que apresentem risco elevado, complexidade lógica significativa ou elevada probabilidade de interpretação divergente (LAPLANTE, 2017; WIEGERS; BEATTY, 2013).

Em termos práticos, os seguintes fatores funcionam como gatilhos para a formalização:

- **Criticidade:** requisitos de segurança (safety/security), confiabilidade e conformidade regulatória, nos quais falhas podem gerar danos relevantes.
- **Complexidade lógica:** regras com múltiplas condições, exceções e estados, cujo entendimento em linguagem natural tende a gerar leituras inconsistentes.
- **Interações e conflitos:** requisitos que podem se contradizer (ou entrar em conflito com restrições operacionais), exigindo análise sistemática de consistência.
- **Propriedades verificáveis:** requisitos que devem garantir invariantes (p.ex., “nunca ocorrerá X”) ou propriedades temporais (p.ex., “sempre que A ocorrer, B ocorrerá dentro de T”).
- **Necessidade de prova/checagem:** quando há exigência de evidência formal (auditoria, certificação, normas internas) ou quando testes isolados são insuficientes para cobrir o espaço de estados (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017; YOUNG, 2004).

Para preservar o caráter seletivo e orientado a evidência adotado neste capítulo, a decisão deve responder a três perguntas complementares: **o quê** formalizar (qual recorte), **por quê** formalizar (qual benefício e qual risco mitigado) e **até onde** formalizar (qual nível de formalidade é suficiente). O Quadro 50 organiza esses critérios como sinais observáveis em projetos e explicita um ponto de suficiência, evitando tanto o formalismo excessivo quanto a manutenção de requisitos criticamente ambíguos.

| Critério (sinal)        | O quê formalizar (recorte)                               | Por quê (benefício esperado)                               | Até onde (nível de formalidade)                         | Indicador prático de “suficiente”                                            |
|-------------------------|----------------------------------------------------------|------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------|
| Ambiguidade persistente | Termos críticos; regras de negócio; estados e transições | Eliminar interpretações divergentes; estabilizar semântica | Invariantes e contratos locais; glossário formal mínimo | Dois revisores independentes chegam à mesma leitura sem negociação adicional |

| Critério (sinal)                  | O quê formalizar (recorte)                                                                        | Por quê (benefício esperado)                                    | Até onde (nível de formalidade)                                            | Indicador prático de “suficiente”                                          |
|-----------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Risco alto de falha               | Requisitos que, se falharem, causam dano relevante (legal, reputacional, segurança, continuidade) | Aumentar confiança; reduzir defeitos tardios                    | Propriedades verificáveis + evidência planejada                            | Existe propriedade formal associada e estratégia explícita de verificação  |
| Concorrência/tempo                | Eventos quase simultâneos; timeouts; janelas; “sempre/eventualmente”                              | Capturar exceções e prioridade; reduzir lacunas comportamentais | Máquina de estados + propriedades temporais (LTL/CTL ou forma equivalente) | Há cenários de corrida resolvidos por regra explícita (prioridade/exceção) |
| Estados/consistência              | Regras de integridade: exclusão de estados, cardinalidade, integridade referencial                | Prevenir estados impossíveis; manter coerência do modelo        | Invariantes sobre domínio/estado; OCL/contratos                            | Invariantes cobrem “estados proibidos” e são rastreados ao requisito       |
| Verificabilidade insuficiente     | Requisito vago (“rápido”, “adequado”, “seguro”) ou sem métrica                                    | Tornar testável/checável; disciplinar verificação e validação   | Formalização como restrição mensurável (métrica/limite)                    | Há métrica e limiar; método de coleta/medição definido                     |
| Mudança frequente do requisito    | Itens voláteis no backlog; incerteza de escopo                                                    | Evitar custo de formalizar o que mudará                         | Formalização leve (vocabulário + invariantes estáveis)                     | Formaliza-se apenas o “núcleo de sentido” e o que é estrutural             |
| Custo/benefício desfavorável      | Baixo risco, alta estabilidade e boa testabilidade                                                | Evitar sobrecarga documental                                    | Manter semiformais + critérios de aceitação                                | Critérios de aceitação e testes já cobrem adequadamente o risco            |
| Dependência de interoperabilidade | Troca de dados, contratos entre sistemas, compliance                                              | Reduzir disputas semânticas; garantir compatibilidade           | Esquemas formais + constraints; validação automática                       | Existem regras de validação automática e exemplos de conformidade          |
| Exigência regulatória/contratual  | Auditoria, rastreabilidade obrigatória, conformidade                                              | Evidência e justificativa rastreável                            | Formalização + rastreabilidade + evidência registrada                      | Artefato de evidência pode ser auditado (o que, como, quando, por quem)    |

Quadro 50 – Critérios para decidir pela formalização (o quê, por quê e até onde).  
Fonte: elaborado pelo autor (2026).

Uma vez decidido formalizar, a delimitação do escopo evita um erro recorrente: transformar formalização em um fim em si mesmo. A formalização deve ser tratada como instrumento de representação orientado a resultados explícitos (p.ex., consistência, invariantes, ausência de conflito e propriedades temporais), com um critério claro de encerramento: ela “termina” quando as propriedades-alvo foram analisadas e seus resultados incorporados à base de requisitos e à rastreabilidade. Sinais práticos de desvio para decisões de solução (“virar design”) são sintetizados no Box 10.7-4 (LAPLANTE, 2017; ROBERTSON; ROBERTSON, 2014).

Para tornar explícito como a formalização se integra às representações semiformais já validadas, a Figura 71 ilustra a ideia de **camadas de formalização seletiva** aplicadas ao fluxo de agendamento: a base semiformal organiza vocabulário, estados e eventos, enquanto



invariantes/contratos e propriedades temporais são adicionados apenas sobre pontos de risco, produzindo evidências diretamente aproveitáveis em V&V.

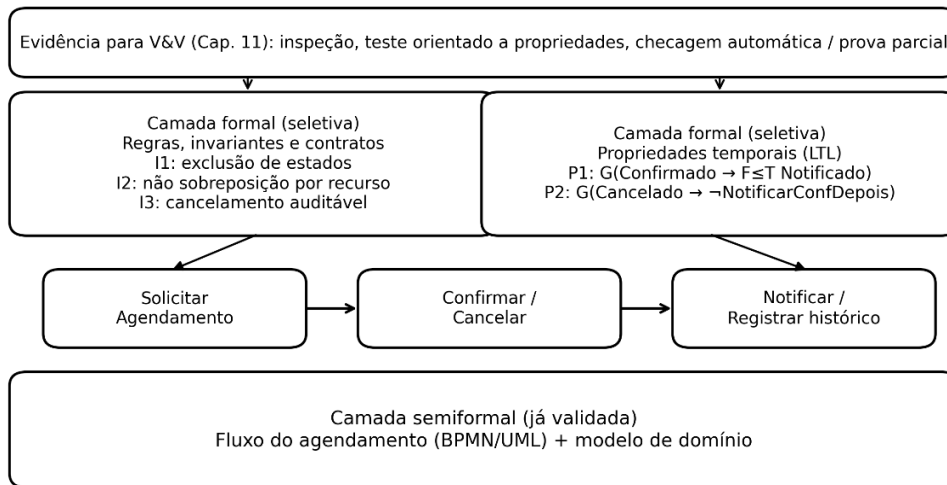


Figura 71 – Camadas de formalização seletiva integradas ao fluxo semiformal (exemplo).  
Fonte: elaborado pelo autor (2026).

### 10.7.3 Famílias de técnicas formais de representação

A literatura descreve diferentes famílias de técnicas formais. Elas variam conforme (i) o tipo de propriedade que tornam explícita (estado, comportamento, tempo, concorrência), (ii) o tipo de análise que permitem (prova, checagem automática, simulação), e (iii) o nível de especialização exigido. Em termos didáticos, quatro famílias são particularmente úteis para organizar o repertório de representação formal no contexto da Engenharia de Requisitos:

#### (a) Lógicas e restrições formais (regras, invariantes e contratos)

Uma forma pragmática de introduzir rigor é formalizar restrições sobre modelos já existentes, concentrando-se em invariantes e contratos aplicados às regras de negócio e à consistência do domínio. A leitura mínima dessas estruturas é sintetizada no Box 10.7-1 (MEYER, 1992; WARMER; KLEPPE, 2003).

Esse estilo é particularmente adequado quando o domínio já foi representado por modelos semi-formais (p.ex., modelo conceitual, requisitos em SysML), e o objetivo é eliminar interpretações ambíguas sobre consistência de dados e regras de negócio. A formalização pode ocorrer, por exemplo, com o uso de linguagens de restrição como OCL, que complementa modelos ao explicitar condições que não ficam claras apenas pelo diagrama (WARMER; KLEPPE, 2003).

#### **Box 10.7-1 – Como ler invariantes e contratos (sem “matematiquês”)**

Um **invariante** é uma regra que deve ser verdadeira **sempre**, em qualquer estado válido do sistema. Em termos práticos, ele define “estados proibidos” (p.ex., um agendamento não pode estar confirmado e cancelado ao mesmo tempo).

Um **contrato** descreve uma operação em duas partes: **pré-condição** (o que precisa ser verdadeiro antes da operação) e **pós-condição** (o que deve ser verdadeiro depois). Contratos não dizem como implementar; dizem quais condições precisam ser respeitadas para a operação ser considerada correta.

*Regra de bolso:* invariantes protegem a **consistência global**; contratos protegem a **correção local** das operações.

#### **(b) Métodos formais baseados em modelos (estado e operações)**

Métodos como Z, VDM e B se baseiam em lógica de predicados e teoria de conjuntos para descrever estados, invariantes e operações do sistema. Nessa família, o sistema é representado como um conjunto de estados possíveis e um conjunto de operações que transformam esses estados, com invariantes que precisam permanecer válidos. Essa abordagem é reconhecida por oferecer rigor para especificar funcionalidade e permitir provas de consistência e, em alguns casos, refinamento sistemático em direção à implementação (DICK; HULL; JACKSON, 2017; WOODCOCK; DAVIES, 1996; SCHNEIDER, 2001; WORDSWORTH, 1996).

Dick, Hull e Jackson (2017) destacam Z, VDM, LOTOS e B como métodos comuns para definição formal de funcionalidade, apontando também que alguns modelos (como B) podem ser executáveis e refináveis até código. Esse ponto é relevante porque reforça o caráter de “ponte” entre requisitos e verificação: a formalização não apenas descreve, mas permite analisar rigorosamente propriedades da especificação.

#### **(c) Modelos de comportamento, concorrência e tempo (máquinas de estados, redes e lógica temporal)**

Quando requisitos dependem de sequência de eventos, concorrência e sincronização, representações formais baseadas em comportamento tornam-se mais apropriadas. Máquinas de estados e statecharts (quando acompanhadas de semântica formal) permitem capturar transições, eventos e condições; redes de Petri modelam concorrência e fluxo; e lógicas temporais permitem expressar propriedades do tipo “sempre”, “eventualmente” e “até” (HAREL, 1987; LAPLANTE, 2017).

Esse tipo de formalização é especialmente útil para requisitos que envolvem: (i) eventos assíncronos, (ii) múltiplos atores/sistemas interagindo, (iii) condições de corrida, e (iv) garantias temporais. Também é valioso para revelar estados “não considerados” que frequentemente são a origem de defeitos sistêmicos.

#### **(d) Checagem automática e análise de propriedades (model checking e prova)**

A formalização cria base para análise automática quando a notação e a modelagem permitem exploração sistemática de estados ou prova de propriedades. Essa capacidade é decisiva em

situações onde testes por amostragem não conseguem cobrir o espaço de comportamento relevante. Em termos de Engenharia de Requisitos, isso se manifesta como verificação antecipada de consistência e propriedades críticas, reduzindo o risco de que contradições sejam detectadas apenas durante implementação ou validação tardia (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017; YOUNG, 2004).

Como se observa, as famílias de técnicas formais diferem menos pelo “grau de matemática” e mais pelo **tipo de propriedade** que tornam explícita: integridade e consistência (regras/invariantes/contratos), estados admissíveis e operações (métodos baseados em modelos), comportamento e concorrência (modelos comportamentais) e obrigações dependentes do tempo (lógica temporal), além da possibilidade de análise automatizada quando um modelo e propriedades são definidos com precisão. Para consolidar essa distinção e apoiar a seleção por objetivo (e não por preferência de notação), o Quadro 51 sintetiza as famílias discutidas e o tipo de propriedade que cada uma tende a explicitar com maior adequação.

| Família de técnica formal                                | O que ela representa melhor                            | Tipo de propriedade explicitada                                | Exemplos de notação (ilustrativo)                             | Limite típico (cuidado)                                                                        |
|----------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Lógicas e restrições (regras, invariantes, contratos)    | Regras estáveis do domínio e condições de operação     | Integridade, consistência, pré/pós-condições, impossibilidades | Lógica de predicados; Design by Contract; OCL                 | Pode “parecer matemática demais” se o vocabulário não for bem explicado                        |
| Métodos formais baseados em modelos (estado e operações) | Estados admissíveis e operações que transformam estado | Preservação de invariantes; consistência por construção        | Modelos de estado; Z/B/VDM (conceito); funções/transformações | Exige disciplina de abstração para não virar especificação de design                           |
| Modelos de comportamento e concorrência                  | Execuções possíveis (sequências de eventos)            | Segurança e vivacidade; prioridades; exclusão mútua            | Máquinas de estados; redes de Petri; autômatos                | Pode exigir decisão explícita de política de concorrência (o que é bom, mas custa esforço)     |
| Lógica temporal (tempo e reatividade)                    | Regras do tipo “sempre”, “eventualmente”, “até”        | Propriedades temporais; obrigações condicionais; timeouts      | LTL/CTL (ou forma equivalente); guardas temporais             | Notação deve ser acompanhada de “tradução” para evitar ruído                                   |
| Checagem e prova (análise automática)                    | Verificar se o modelo satisfaz propriedades            | Satisfação/violação; contraexemplos; argumentos de correção    | Model checking; prova/inspeção formal; SAT/SMT (conceito)     | Depende de um modelo formal suficientemente fiel; risco de “modelo bonito e sistema diferente” |
| Especificação de dados e interoperabilidade              | Estruturas, formatos e validações de dados             | Conformidade, consistência, validação                          | Esquema + constraints; regras de validação                    | Foco excessivo em dados pode ignorar comportamento; equilibrar com cenários                    |

Quadro 51 – Famílias de técnicas formais e o tipo de propriedade que explicitam.

Fonte: elaborado pelo autor (2026).

#### 10.7.4 Como integrar representações formais ao repertório do capítulo

Esta seção concretiza o **Princípio 10 (Processo de ER)**: escolhas de representação devem ser feitas por **adequação ao propósito e ao contexto**, e não por preferência de notação. Em particular, a formalização é tratada como recurso **seletivo e orientado a risco**, integrado ao sistema de requisitos para aumentar confiança e verificabilidade sem criar “um segundo documento”.

A tensão central das representações formais é conhecida: quanto maior o rigor, menor tende a ser a acessibilidade imediata para stakeholders não técnicos. Por isso, a integração deve seguir o mesmo princípio já adotado ao longo do capítulo: **representações distintas para propósitos distintos**, integradas por rastreabilidade e por um núcleo semântico consistente.

Esse arranjo preserva um aspecto essencial do livro: a representação não existe para “sofisticar a documentação”, mas para tornar o entendimento mais sólido e socialmente compartilhável. O rigor formal deve ser aplicado onde aumenta confiança e reduz risco, sem comprometer a possibilidade de comunicação e discussão com os stakeholders (ROBERTSON; ROBERTSON, 2014; WIEGERS; BEATTY, 2013).

Na prática, a integração depende de poucos mecanismos bem controlados: ancoragem em modelos semiformais, tradução legível e rastreabilidade bidirecional. O box a seguir sintetiza um checklist mínimo para garantir essa integração sem criar artefatos paralelos.

##### Box 10.7-3 – Checklist mínimo de integração (sem artefato paralelo)

Antes de considerar a formalização “pronta”, verifique:

1. Existe vínculo explícito **requisito ↔ propriedade formal** (ID e nome)?
2. A propriedade tem **tradução em linguagem natural** (1 frase)?
3. Está registrada a **justificativa (risco)** para formalizar?
4. Está definida a **forma de verificação** (inspeção, checagem de propriedades, simulação controlada (quando aplicável), prova parcial)?
5. Há critério de “suficiente”: a formalização cobre os estados/eventos críticos, sem expandir escopo para design?

O Quadro 52 consolida essas estratégias de integração e explicita como aplicá-las sem ruptura de comunicação com stakeholders

| Estratégia de integração               | Como aplicar (passos curtos)                                                | Onde ancorar no sistema de requisitos               | Benefício                             | Risco evitado                                |
|----------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------|---------------------------------------|----------------------------------------------|
| Formalizar “sobre” o modelo semiformal | Partir de casos de uso/BPMN/SysML e extrair invariantes/contratos           | Mesmo requisito (ID) + links para diagrama/fluxo    | Evita duplicação; mantém continuidade | Criação de “documento paralelo” desconectado |
| Formalização seletiva por risco        | Escolher poucos requisitos críticos (top N) e formalizar propriedades-chave | Backlog/SRS: marcar como “candidato à formalização” | Maximiza custo/benefício              | Overengineering; formalizar tudo             |

| Estratégia de integração              | Como aplicar (passos curtos)                                                     | Onde ancorar no sistema de requisitos                | Benefício                            | Risco evitado                            |
|---------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------|--------------------------------------|------------------------------------------|
| “Tradução” lado a lado                | Para cada fórmula, incluir comentário em linguagem natural                       | No próprio quadro/tabela de formalização             | Mantém acessibilidade                | Rejeição pelos stakeholders não técnicos |
| Rastreabilidade bidirecional          | Requisito → propriedade → técnica → evidência; e retorno propriedade → requisito | Matriz de rastreabilidade (quadro único)             | Disciplina de evidência              | Perda de justificativa e de controle     |
| Integração com critérios de aceitação | Usar propriedades formais como reforço dos critérios                             | User Stories/critério de aceitação + propriedade     | Une agilidade e rigor                | Critério vago que não cobre exceções     |
| Ciclo curto de validação              | Validar primeiro o significado (com stakeholders), depois a checagem             | Workshops/revisões                                   | Evita formalizar entendimento errado | “Provar” algo que não era o requisito    |
| Evidência como saída, não como meta   | Tratar contraexemplo como gatilho de refinamento                                 | Registro de decisão (ADR) + atualização de requisito | Aprendizado rápido                   | Ignorar o sinal e “forçar o modelo”      |
| Escalonamento progressivo             | Começar com vocabulário + invariantes; só depois temporal/model checking         | Roadmap de formalização                              | Reduz barreira de entrada            | Saltar direto para notações complexas    |

Quadro 52 – Estratégias de integração entre semiformais e formais sem ruptura de comunicação.

Fonte: elaborado pelo autor (2026).

### 10.7.5 Recomendações práticas para aplicação em projetos reais

A adoção de técnicas formais é viável quando tratada como um esforço focal, com governança simples e critérios claros de sucesso. As recomendações a seguir devem ser lidas como aplicação combinada de Comunicação (forma adequada ao público), Feedback (iterações curtas) e Compromisso (registro explícito de decisões), alinhadas ao Princípio 2 e ao Princípio 10. Para isso, apresenta-se a seguir uma abordagem realista e aplicável:

1. **Definir o objetivo da formalização** (consistência, propriedade temporal, ausência de conflito, invariantes de dados).
2. **Selecionar requisitos candidatos por risco** (criticidade, complexidade lógica, impacto de falha).
3. **Escolher a técnica pela propriedade-alvo**, e não pela preferência da equipe.
4. **Formalizar por incrementos pequenos**, validando cada incremento por revisão técnica e inspeção com especialistas do domínio.
5. **Converter resultados em evidência rastreável**, conectando propriedades formais aos requisitos e aos critérios de aceitação.
6. **Evitar transbordamento para design** (ver Box 10.7-4) (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017; IEEE, 2024).

#### Box 10.7-4 – Quando a formalização vira “design” (e como evitar)

Sinais de que a formalização está saindo de requisitos:

- (a) Começa a prescrever estrutura interna (classes, camadas, filas, tabelas) como se fosse a única solução;
- (b) Introduz decisões de tecnologia e mecanismo (p.ex., “usar Kafka”, “usar lock pessimista”) sem justificativa de requisito;
- (c) Perde rastreabilidade com o requisito original e passa a “otimizar o modelo”.

Como evitar: formalizar **propriedades observáveis** (consistência, temporalidade, limites de desempenho), manter a ligação ao requisito (ID) e registrar a justificativa (risco) na matriz de rastreabilidade.

Para facilitar aplicação em projetos reais, o Quadro 53 reagrupa essas recomendações como ações, armadilhas comuns e antídotos práticos.

| Recomendação                                         | Como executar em projeto (ação objetiva)                                  | Armadilha comum                         | Antídoto prático                                            |
|------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------|
| Formalize o “núcleo de sentido”, não a implementação | Formalizar regras e propriedades observáveis (estado, integridade, tempo) | Virar design técnico                    | Revisar: “isso é mecanismo ou propriedade?”                 |
| Comece pelo vocabulário                              | Definir termos, conjuntos, estados e predicados mínimos                   | Fórmulas sem significado compartilhado  | Quadro de vocabulário + exemplos concretos                  |
| Priorize riscos e exceções                           | Formalizar o que falha “pior” e o que gera disputa                        | Formalizar o trivial                    | Seleção por impacto/probabilidade e histórico de incidentes |
| Use tradução lado a lado                             | Cada expressão formal deve ter leitura em português claro                 | Stakeholder rejeita por não entender    | Comentários “Como ler” e exemplos                           |
| Evite formalização total                             | Escolher recortes críticos (top N) e manter o restante semiformal         | Custo alto e baixa manutenção           | Critérios de seleção e revisão periódica                    |
| Transforme RNF em métrica                            | Converter “rápido/estável” em limite mensurável e método de medição       | RNF vira “wish”                         | Definir métrica, limiar, e como medir                       |
| Use contraexemplo como ferramenta                    | Quando falhar, registrar decisão e ajustar requisito                      | “Consertar o modelo” e ignorar a lacuna | Registro de decisão + atualização de requisito              |
| Preserve rastreabilidade                             | Requisito ↔ propriedade ↔ evidência                                       | Formalização vira apêndice              | Matriz única (quadro) + referências cruzadas                |
| Valide cedo com stakeholders                         | Validar significado antes de checar propriedades                          | Provar coisa errada                     | Rodada de validação semântica antes da checagem             |
| Documente o “porquê”                                 | Registrar justificativa (risco) da formalização                           | Formalização sem motivação              | Coluna “risco/justificativa” em todos os quadros            |

Quadro 53 – Recomendações práticas para aplicação seletiva de representações formais.

Fonte: elaborado pelo autor (2026).

### 10.7.6 Limitações, riscos de uso inadequado e condições mínimas de sucesso

O uso de formalização pode falhar por motivos previsíveis. Os principais riscos são:

- **Overformalização:** formalizar áreas de baixo risco, gerando custo e atraso sem ganho proporcional.
- **Isolamento técnico:** formalização produzida sem integração com a base de requisitos, tornando-se um artefato “paralelo” que não orienta decisões.
- **Falta de competência:** notações formais exigem domínio conceitual; sem isso, o risco é produzir um formalismo apenas aparente, sem validação real.
- **Ruptura com stakeholders:** quando o rigor substitui a comunicação, a formalização perde sua função de engenharia e se torna barreira.

As condições mínimas de sucesso, portanto, não são “ter matemática no documento”, mas sim: (i) objetivo definido, (ii) escopo pequeno e crítico, (iii) revisão rigorosa, e (iv) rastreabilidade explícita dos resultados para o conjunto de requisitos e para verificação e validação (YOUNG, 2004; WIEGERS; BEATTY, 2013; LAPLANTE, 2017).

#### Exemplo

Dando continuidade ao exemplo do Sistema de Agendamento de Consultas Públicas, a equipe identificou que parte relevante dos riscos do produto estava concentrada em **regras de negócio** e em **propriedades temporais** associadas à confirmação, ao cancelamento e às notificações. As representações semiformais adotadas anteriormente (casos de uso, atividades, BPMN e SysML) foram suficientes para consolidar entendimento e delimitar o comportamento esperado, porém duas classes de problema permaneceram prováveis: (i) **interpretações divergentes** sobre consistência do estado do agendamento e integridade do histórico; e (ii) **exceções temporais e situações de concorrência** (p.ex., cancelamento quase simultâneo à confirmação). Para reduzir essas incertezas, a equipe selecionou três requisitos candidatos à formalização, aplicando formalização seletiva como reforço à verificabilidade e à evidência técnica.

- RF02 – Enviar Notificação;
- RF03 – Consultar Histórico; e
- RNF02 – Tempo máximo de resposta.

Para manter a formalização aderente ao escopo de requisitos, a equipe adotou um vocabulário mínimo (conjunto e predicados) consistente com a declaração e com o modelo de domínio já validado.

#### Sintaxe

**Ag:** conjunto de agendamentos

**Recurso:** conjunto de recursos agendáveis

**estado(a) ∈ {PENDENTE, CONFIRMADO, CANCELADO}**

#### Comentários / Tradução

“**Ag**” é o conjunto de **todos os agendamentos** que existem no sistema.

“**Recurso**” é o conjunto de coisas que podem ser reservadas (sala, unidade, servidor etc.).

**estado(a)** retorna o estado do agendamento “a”. O agendamento só pode estar em um desses estados: **PENDENTE** (criado/aguardando), **CONFIRMADO** (reservado) ou **CANCELADO** (revogado).

| Sintaxe                                                                           | Comentários / Tradução                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>inicio(a), fim(a)</b> : instantes de tempo, com <b>inicio(a) &lt; fim(a)</b>   | <b>inicio(a)</b> e <b>fim(a)</b> retornam o intervalo temporal do agendamento “a”. Pressuposto de sanidade: <b>inicio(a) &lt; fim(a)</b> . Todo agendamento tem início e fim; início precisa ser antes do fim.                                                                                                                             |
| <b>recurso(a) ∈ Recurso</b>                                                       | <b>recurso(a)</b> associa o agendamento “a” ao recurso agendado. Cada agendamento aponta para um recurso que está sendo reservado.                                                                                                                                                                                                         |
| <b>motivoCancel(a), tsCancel(a), tsConfirm(a)</b> : campos de registro            | Campos de registro: <b>motivo</b> e <b>horário</b> do cancelamento, e <b>horário</b> da confirmação. <b>motivoCancel(a)</b> : justificativa de cancelamento (auditabilidade). <b>tsCancel(a)</b> : timestamp do cancelamento (auditabilidade). <b>tsConfirm(a)</b> : timestamp da confirmação (rastreadabilidade operacional e histórico). |
| <b>sobrepos(a,b) ≡ inicio(a) &lt; fim(b) ∧ inicio(b) &lt; fim(a)</b>              | <b>sobrepos(a,b)</b> define formalmente o que significa “sobreposição”. Dois intervalos se sobrepõem quando um começa antes do outro terminar e <b>vice-versa</b> . Ex.: se “a” é 10:00–11:00 e “b” é 10:30–11:30, então <b>há sobreposição</b> .                                                                                          |
| <b>OpsCriticas</b> : {Confirmar, Cancelar, ConsultarHistorico}                    | “ <b>OpsCriticas</b> ” é o conjunto de operações consideradas críticas para o RNF02. No exemplo, inclui confirmar, cancelar e consultar histórico (o conjunto pode ser ajustado conforme o escopo do projeto).                                                                                                                             |
| <b>rt(op,a)</b> : tempo de resposta observado da operação op para o agendamento a | <b>rt(op,a)</b> é o tempo entre a requisição e a resposta da operação op aplicada ao agendamento a (métrica observável). No exemplo, RNF02 impõe $rt(op,a) \leq 2s$ para operações críticas.                                                                                                                                               |

Com esse “dicionário”, o time passa a escrever regras que ficam inequívocas. O próximo passo foi declarar **invariantes** (o que nunca pode acontecer) e **contratos** (o que precisa ser verdade antes/depois de uma operação), reduzindo ambiguidade sem entrar em projeto de implementação (MEYER, 1992; WARMER; KLEPPE, 2003).

#### (a) Lógicas e restrições formais (regras, invariantes e contratos)

| Sintaxe (Invariantes)                                                                                                                                                                                 | Comentários / Tradução                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (I1) $\forall a \in Ag : \neg(\text{estado}(a)=\text{CONFIRMADO} \wedge \text{estado}(a)=\text{CANCELADO})$                                                                                           | $\forall a \in Ag$ = “para todo agendamento a”; $\neg(\dots)$ = “não é permitido que (...) seja verdadeiro”; $\wedge$ = “e ao mesmo tempo”.<br>“Para qualquer agendamento, não é permitido que ele esteja <b>Confirmado</b> e <b>Cancelado</b> ao mesmo tempo.” |
| (I2) $\forall a,b \in Ag, a \neq b : (\text{estado}(a)=\text{CONFIRMADO} \wedge \text{estado}(b)=\text{CONFIRMADO} \wedge \text{recurso}(a)=\text{recurso}(b)) \Rightarrow \neg \text{sobrepos}(a,b)$ | $a \neq b$ = “agendamentos diferentes”; $\Rightarrow$ = “então”.<br>“Se dois agendamentos diferentes estiverem <b>Confirmados</b> e forem do <b>mesmo recurso</b> , então eles <b>não podem</b> ter sobreposição de horários.”                                  |
| (I3) $\forall a \in Ag : \text{estado}(a)=\text{CANCELADO} \Rightarrow (\text{motivoCancel}(a) \neq \emptyset \wedge \text{tsCancel}(a) \neq \emptyset)$                                              | $\neq \emptyset$ = “não vazio”.<br>“Se o agendamento está <b>Cancelado</b> , então deve existir <b>motivo</b> e <b>timestamp</b> do cancelamento. Se cancelou, tem que ficar rastreado no histórico.”                                                           |

Os invariantes atacam diretamente os riscos levantados: (i) evitam estados impossíveis (I1), (ii) impedem conflitos de agenda (I2) e (iii) asseguram auditabilidade mínima para o histórico (I3), reforçando a consistência associada ao **RF03 – Consultar Histórico**.



Além dos invariantes, operações críticas foram descritas por **contratos** (Design by Contract) (pré/pós-condições), de modo que a regra “não sobrepor” passe a ser condição de habilitação da confirmação, e não apenas uma frase no texto.

#### Sintaxe (Contratos)

##### Operação Confirmar(a)

**Pré:** estado(a)=PENDENTE  $\wedge$  inicio(a)<fim(a)  $\wedge \neg \exists b \in Ag : (\text{estado}(b)=CONFIRMADO \wedge \text{recurso}(b)=\text{recurso}(a) \wedge \text{sobrepor}(a,b))$

**Pós:** estado(a)=CONFIRMADO  $\wedge$  tsConfirm(a) $\neq \emptyset$

##### Operação Cancelar(a, motivo)

**Pré:** estado(a) $\neq$ CANCELADO  $\wedge$  motivo $\neq \emptyset$

**Pós:** estado(a)=CANCELADO  $\wedge$  motivoCancel(a)=motivo  $\wedge$  tsCancel(a) $\neq \emptyset$

#### Comentários / Tradução

Pré-condição = o que deve ser verdade antes de confirmar; Pós-condição = o que deve ser verdade depois.  $\exists b \in Ag$  = “existe um agendamento b”;  $\neg \exists$  = “não pode existir”.

**Pré:** “Só confirma se (1) está pendente, (2) o intervalo é válido e (3) não existe nenhum confirmado no mesmo recurso que conflite com o horário.”

**Pós:** “Depois, vira Confirmado e registra timestamp da confirmação.”

**Pré:** “Não cancela duas vezes e não cancela sem motivo.”

**Pós:** “Depois, fica Cancelado, grava o motivo e registra timestamp.” Isso reforça integridade do histórico (RF03) e a governança mínima sobre o evento de cancelamento.

Esse tipo de formalização é particularmente eficaz porque preserva a proximidade com o modelo semiformal validado e elimina interpretações ambíguas sobre regras invariantes e condições de exceção (MEYER, 1992; WARMER; KLEPPE, 2003; LAPLANTE, 2017).

### (b) Métodos formais baseados em modelos (estado e operações)

Para reforçar a regra de agenda sob uma perspectiva de “estado + operações”, o sistema foi modelado por um estado abstrato que, para cada recurso, associa o conjunto de intervalos confirmados.

#### Sintaxe

**confirmados : Recurso  $\rightarrow \mathbb{P}(\text{Intervalo})$**

**(M1)  $\forall r \in \text{Recurso}, \forall i, j \in \text{confirmados}(r), i \neq j : \neg \text{sobrepor}(i, j)$**

##### Operação Confirmar(r,i)

**Pré:**  $\neg \exists j \in \text{confirmados}(r) : \text{sobrepor}(i, j)$

**Pós:** confirmados'(r)=confirmados(r) $\cup \{i\}$

#### Comentários / Tradução

“Para cada recurso, existe um **conjunto** ( $\mathbb{P}$ ) de intervalos confirmados.” É como um mapa/dicionário: recurso  $\mapsto$  horários já reservados.

**(M1)** Invariante de não sobreposição por recurso.

“Dentro do mesmo recurso, quaisquer dois intervalos confirmados diferentes não podem se sobrepor.” Isso transforma a regra em **propriedade do próprio estado**: um estado com conflito simplesmente não é válido.

“Só adiciona o novo intervalo i se ele não conflitar com nenhum intervalo já confirmado do recurso r.” Depois, o estado é atualizado adicionando i ao conjunto de confirmados daquele recurso. Essa é a ideia de ‘preservar o invariante’: a operação só é permitida quando mantém (M1).

Essa sintaxe ilustra uma motivação comum para métodos formais baseados em modelos: quando a regra é crítica, vale representá-la como invariante de estado e garantir que as operações preservam a consistência. Isso prepara evidência de verificação e validação, por inspeção, prova parcial e/ou checagem automática (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).

### (c) Modelos de comportamento, concorrência e tempo (máquinas de estados, redes e lógica temporal)

Alguns requisitos do exemplo (especialmente RF02) são essencialmente comportamentais: dependem de sequência de eventos, estados e tempo.

Para facilitar a leitura do comportamento e reduzir a carga cognitiva da descrição textual, a Figura 72 apresenta a **máquina de estados do agendamento** com suas transições principais e **guardas temporais**. Essa visualização complementa a formalização ao tornar evidente quais sequências são permitidas, em que condições o cancelamento é válido, e como eventos temporais (p.ex., expiração) afetam o ciclo de vida do requisito.

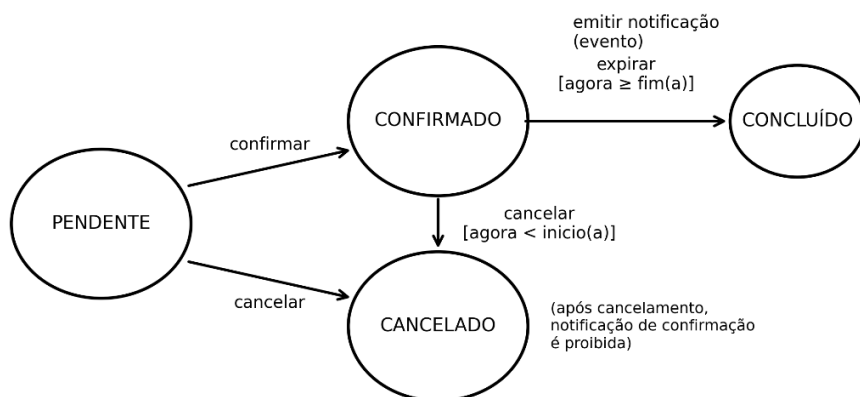


Figura 72 – Máquina de estados do agendamento com guardas temporais (exemplo).  
Fonte: elaborado pelo autor (2026).

## Sintaxe

Estados: PENDENTE, CONFIRMADO, CANCELADO

Eventos: confirmar, cancelar

Transições:

PENDENTE --confirmar--> CONFIRMADO

PENDENTE --cancelar--> CANCELADO

## Comentários / Tradução

**Estados do ciclo de vida** do agendamento no nível de requisitos (não de implementação). **PENDENTE**: solicitação registrada/ aguardando decisão. **CONFIRMADO**: reserva efetivada. **CANCELADO**: reserva revogada (por usuário/sistema), com registro em histórico.

**Eventos** são “ações observáveis” que provocam mudança de estado. **confirmar**: decisão/ação que efetiva a reserva. **cancelar**: decisão/ação que revoga a reserva. Esses eventos são coerentes com os fluxos semiformais já descritos (caso de uso/atividade), mas aqui aparecem de forma mais precisa.

Abaixo, cada linha define uma **mudança permitida** de estado. A representação “A —evento→ B” significa: “se o sistema está em A e ocorre o evento, então o novo estado pode ser B”. Ao explicitar as transições, a equipe elimina ambiguidades como “pode cancelar quando?” e “o que acontece se confirmar e cancelar quase juntos?”.

Um agendamento pendente pode ser confirmado. **Interpretação de requisito**: confirmações são permitidas apenas enquanto o agendamento ainda não foi cancelado. Essa transição deve respeitar os invariantes de agenda (por exemplo, não sobrepor confirmados no mesmo recurso).

Um agendamento pendente pode ser cancelado. **Interpretação de requisito**: o cancelamento pode ocorrer antes de qualquer confirmação, e deve produzir evidência mínima no histórico (motivo e timestamp).

#### Sintaxe

CONFIRMADO --cancelar [agora < inicio(a)]--> CANCELADO

#### Comentários / Tradução

Um agendamento confirmado pode ser cancelado **somente** se o horário atual (*agora*) ainda for antes do início do agendamento. **Por que a guarda existe:** frases como “cancelar antes do início” ficam ambíguas em linguagem natural; a guarda *agora* < *inicio(a)* torna a condição verificável e alinhável ao RNF/critério de aceitação. **Efeito prático:** essa regra também ajuda a tratar exceções temporais e reduzir conflitos de interpretação entre áreas.

A guarda ‘agora < inicio(a)’ elimina ambiguidade típica de frases como ‘pode cancelar antes do início’. Para tornar a obrigação de notificar formalmente observável, a equipe definiu uma propriedade temporal (forma LTL) coerente com **RF02**.

#### Sintaxe

G( ... )

F( ... )

(LTL1) G( Confirmado(a) → F Notificado(a) )

(LTL2) G( Cancelado(a) → ¬NotificadoConfirmacaoDepois(a) )

#### Comentários / Tradução

**G** significa “sempre”: em todos os momentos da execução do sistema.

**F** significa “eventualmente”: em algum momento no futuro.

“Sempre: se o agendamento a ficar Confirmado, então em algum momento depois disso ele será Notificado.” Isso formaliza a intenção do **RF02 – Enviar Notificação** como propriedade verificável.

“Sempre: se o agendamento a estiver Cancelado, não deve existir notificação de confirmação depois do cancelamento.” Essa propriedade evita comportamentos indevidos em cenários de exceção/concorrência.

A formalização temporal não substitui critérios de aceitação; ela os fortalece ao tornar explícito o que significa “deve notificar” em termos de comportamento ao longo do tempo. Ela também prepara a checagem automática e o tratamento de exceções (DICK; HULL; JACKSON, 2017; IEEE, 2024).

### (d) Checagem automática e análise de propriedades (model checking e prova)

#### Sintaxe

(P1) G ¬SobreposicaoMesmoRecurso

(P2) G( Confirmado(a) → F Notificado(a) )

(P3) G( Cancelado(a) → ¬NotificadoConfirmacaoDepois(a) )

**Traço/contraexemplo:** PENDENTE —confirmar→ CONFIRMADO —cancelar→ CANCELADO ; ¬Notificado(a)

#### Comentários / Tradução

“Sempre: nunca acontece sobreposição de confirmados no mesmo recurso.” É uma propriedade de **segurança** (algo ruim não pode acontecer).

Mesma ideia de (LTL1), agora como propriedade-alvo para checagem.

“Se confirmou, então em algum momento notifica.”

Mesma ideia de (LTL2), como propriedade-alvo de segurança.

“Confirmou, cancelou logo em seguida e a notificação não ocorreu.” O valor do contraexemplo é **revelar uma lacuna de requisito**: a equipe precisa decidir se (i) deve haver notificação mesmo assim (p.ex., notificação de cancelamento) ou (ii) o cancelamento bloqueia a notificação de confirmação, exigindo refinamento explícito da regra.

Esse resultado não é “erro do modelo” em si; ele força uma decisão de requisito: (i) o sistema deve notificar mesmo em caso de cancelamento subsequente (p.ex., notificação de cancelamento), ou (ii) a obrigação de notificar deve ser condicionada à ausência de cancelamento antes do envio. Em ambos os casos, a checagem automática induz a explicitação de prioridades e exceções que, em geral, permanecem implícitas em texto (DICK; HULL; JACKSON, 2017; YOUNG, 2004).

Para propriedades invariantes (como não sobreposição), a equipe também registrou evidência por preservação: se o invariante é verdadeiro no estado inicial e cada operação o preserva (por suas pré-condições), então o invariante é verdadeiro em todos os estados alcançáveis, reforçando o vínculo entre formalização e evidência de verificação e validação (DICK; HULL; JACKSON, 2017; YOUNG, 2004).

Ao final, a equipe produziu um quadro de rastreabilidade conectando: requisito → propriedade formal → justificativa (risco) → técnica aplicada → **forma de checagem** (inspeção, prova parcial, checagem automática e simulação/experimentação controlada) (ver Quadro 54). Esse arranjo tornou explícito que a formalização não é um “novo documento”, mas um reforço do próprio sistema de requisitos e de sua verificabilidade, preparando a transição para o capítulo de verificação e validação (YOUNG, 2004; IEEE, 2024).

| Requisito                                               | Propriedade/Regra formal                                         | Comentário (tradução)                                                                                               | Risco que motivou formalizar                                                          | Técnica aplicada                | Forma de verificação (evidência)                                                              |
|---------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------|-----------------------------------------------------------------------------------------------|
| <b>RF02 – Enviar Notificação</b>                        | <b>G(Confirmado(a) → F Notificado(a))</b>                        | “Se confirmou, então em algum momento notifica.”                                                                    | Notificação “perdida”; ambiguidade sobre obrigação de notificar; impacto operacional. | Lógica temporal (LTL)           | Checagem de propriedades; inspeção do modelo; testes orientados a propriedades.               |
| <b>RF02 – Enviar Notificação</b>                        | <b>G(Cancelado(a) → ¬NotificadoConfirmacaoDepois(a))</b>         | “Se cancelou, não pode notificar confirmação depois.”                                                               | Corrida confirmar/cancelar; notificação indevida; retrabalho e risco reputacional.    | Lógica temporal (LTL)           | Checagem + análise de contraexemplos; revisão explícita de exceções/prioridades.              |
| <b>RF03 – Consultar Histórico</b>                       | <b>estado(a)=CANCELADO ⇒ (motivoCancel(a)≠∅ ∧ tsCancel(a)≠∅)</b> | “Se cancelou, registra motivo e horário.”                                                                           | Histórico incompleto; falta de auditabilidade; inconsistência de registros.           | Invariante                      | Inspeção por checklist; testes de integridade do histórico derivados da regra.                |
| <b>RF03 – Consultar Histórico</b>                       | <b>Contrato Cancelar(a,motivo) (Pré/Pós)</b>                     | “Não cancela sem motivo; depois grava motivo e timestamp.”                                                          | Cancelamentos sem justificativa; rastreabilidade fraca; divergência de evidência.     | Contratos (DbC)                 | Inspeção do contrato; testes derivados de pré/pós; auditoria de registros.                    |
| <b>(Regra de agenda ligada a RF01)</b>                  | <b>(I2) e/ou (M1)</b>                                            | “Confirmados do mesmo recurso não sobrepõem.”                                                                       | Reserva dupla; conflito operacional; quebra de confiança.                             | Invariante + modelo de estado   | Inspeção; prova parcial por preservação; checagem (P1) quando houver modelo.                  |
| <b>RNF02 – Tempo máximo de resposta</b>                 | <b>∀ op ∈ OpsCriticas, ∀ a ∈ Ag : rt(op,a) ≤ 2s</b>              | “Para cada operação crítica e para cada agenda-mento, o tempo de resposta observado deve ser no máximo 2 segundos.” | Violação de tempo; não conformidade com RNF; degradação de percepção.                 | Propriedade temporal com janela | Testes de desempenho/latência; verificação de traços/eventos; mapeamento requisito → métrica. |
| <b>Consistência de estados (apoio ao ciclo de vida)</b> | <b>¬(estado(a)=CONFIRMADO ∧ estado(a)=CANCELADO)</b>             | “Não pode estar nos dois estados ao mesmo tempo.”                                                                   | Estados impossíveis; ambiguidade conceitual; erros de interpretação.                  | Invariante                      | Inspeção; revisão cruzada com máquina de estados e regras do domínio.                         |

Quadro 54 – Quadro de rastreabilidade: requisito → propriedade formal → comentário → justificativa (risco) → técnica aplicada → forma de verificação.

Fonte: elaborado pelo autor (2026).

### 10.7.7 Síntese do uso e integração

A discussão desta seção mostrou que a representação formal não é um “tipo especial de documento”, mas um modo de reforçar requisitos seletivamente, tornando explícitas propriedades que, em linguagem natural, permaneceriam sujeitas a interpretação ou de difícil verificação. Ao longo das subseções anteriores, foram delineados critérios de decisão, famílias de técnicas, tipos de resultados e estratégias de integração com representações semiformais, sempre preservando o princípio de comunicabilidade com stakeholders e a rastreabilidade com o sistema de requisitos.

Nesta etapa, o objetivo é consolidar esses elementos em uma visão operacional: **quando** cada família de técnica tende a ser indicada, **quais entradas mínimas** ela exige (vocabulário, modelo semiformal, estados/eventos, critérios de aceitação), **como** ela deve ser integrada ao repositório de requisitos sem criar artefatos paralelos, e **que tipo de evidência** ela produz para verificação e validação. O Quadro 55 sintetiza esse encadeamento como um guia de aplicação seletiva, permitindo que equipes repliquem a abordagem com custo controlado e foco explícito em verificabilidade e valor prático.

Por fim, este capítulo apresentou a atividade de **representação de requisitos** como um continuum de técnicas e artefatos, abrangendo desde representações textuais e estruturadas até modelos semiformais e, quando necessário, formalizações seletivas (ver Figura 73). Em todas as abordagens, buscou-se manter um eixo comum:

- Preservar o núcleo de sentido do requisito;
- Reduzir ambiguidades e tornar explícitas as condições de verificabilidade, sem romper o entendimento compartilhado com stakeholders.

Ao tratar das representações formais, o capítulo enfatizou que seu valor emerge quando orientadas por risco e integradas ao sistema de requisitos por rastreabilidade, produzindo propriedades e evidências que sustentam decisões técnicas e organizacionais.

Com isso, o capítulo também delimitou uma consequência direta: representar melhor não garante, por si só, que o requisito esteja correto, completo, consistente e aceitável para os envolvidos. A representação é condição necessária para reduzir incerteza e aumentar precisão, mas a confiança no conjunto de requisitos depende de atividades sistemáticas de verificação e validação. Por essa razão, o próximo capítulo aprofunda procedimentos, critérios e evidências usados para verificar a qualidade interna dos requisitos e validar, junto aos stakeholders, que os requisitos representam adequadamente necessidades, restrições e objetivos do contexto de solução, incluindo a articulação entre **inspeções e revisões técnicas**, checagem de propriedades (quando aplicável) e métodos de validação orientados a cenários.

| Elemento formal (família)                             | Quando usar (gatilho)                                                                                                     | Entradas mínimas (de onde vem)                                                                                                 | Como integrar no capítulo/sistema de requisitos                                                                                                                                  | Saídas produzidas (o que fica “pronto”)                                                                           | Evidência típica de verificação e validação                                                                                                          |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vocabulário mínimo (dicionário)</b>                | Termos ambíguos; dúvidas recorrentes; divergência semântica entre stakeholders.                                           | Glossário, modelo de domínio, lista de estados/eventos, regras de negócio já declaradas.                                       | Inserir como bloco de apoio ao exemplo ou apêndice local do requisito; referenciar nos requisitos críticos e nos critérios de aceitação.                                         | Conjunto de símbolos/definições (Ag, Recurso, estado(a), sobrepo(a,b) etc.) que padroniza leitura das expressões. | Inspeção por checklist de consistência terminológica; revisão cruzada com o modelo de domínio (IEEE, 2024).                                          |
| <b>Invariantes e restrições (lógicas/contratos)</b>   | Risco de “estado impossível”; integridade de registro; regras “sempre verdadeiras”.                                       | Requisitos (RF/RNF), regras de negócio, modelo de domínio e estados possíveis.                                                 | Associar cada invariante ao requisito correspondente (ex.: RF03 ↔ auditabilidade; integridade de agenda); manter rastreabilidade requisito → invariante → justificativa (risco). | Invariantes (I1–I3) e regras formais que eliminam leituras ambíguas e definem condições de integridade.           | Inspeção formal orientada a propriedades; testes de integridade derivados dos invariantes; revisão de consistência (MEYER, 1992; YOUNG, 2004).       |
| <b>Contratos de operação (pré/pós-condições)</b>      | Regras dependem de “antes/depois”; risco de interpretação em fluxos; exceções operacionais frequentes.                    | Casos de uso/fluxos, eventos, regras de negócio, cenários e critérios de aceitação existentes.                                 | Anexar o contrato à operação/ação do caso de uso; usar o contrato para derivar critérios de aceitação e casos de teste; mapear contrato ↔ requisito ↔ cenário.                   | Contratos como “Confirmar(a)” e “Cancelar(a,motivo)” com condições explícitas.                                    | Testes derivados de pré/pós-condições; inspeção de completude; análise de cenários-limite (MEYER, 1992; WARNER; KLEPPE, 2003).                       |
| <b>Modelos formais baseados em estado e operações</b> | Regra crítica afeta “estados atingíveis” (agenda, saldo, alocação); necessidade de demonstrar preservação de integridade. | Modelo de domínio + estrutura de dados conceitual (ex.: confirmados(r)); invariantes principais; operações que alteram estado. | Usar como “camada formal” sobre o mesmo recorte já descrito em BPMN/SysML; vincular cada operação formal a uma transição/ação do fluxo semiformal.                               | Definição do estado abstrato + invariante (M1) + operações que preservam invariantes.                             | Evidência por preservação de invariantes (prova parcial/argumento estruturado) e testes de propriedades (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017). |

| Elemento formal (família)                                   | Quando usar (gatilho)                                                                                        | Entradas mínimas (de onde vem)                                                                               | Como integrar no capítulo/sistema de requisitos                                                                                                                      | Saídas produzidas (o que fica “pronto”)                                                                       | Evidência típica de verificação e validação                                                                                                                            |
|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Modelos de comportamento e tempo (máquinas, LTL/CTL)</b> | Requisitos reativos; “sempre/eventualmente”; notificações; timeouts; concorrência e priorização de eventos.  | Máquina de estados/diagrama comportamental já discutido; eventos; gatilhos temporais; exceções.              | Conectar cada propriedade temporal ao requisito (RF02/RNF02); inserir como complemento ao critério de aceitação (“reforço formal”); explicitar exceções/prioridades. | Propriedades temporais (LTL1/LTL2) e regras com guardas temporais (“agora < início”).                         | Model checking de propriedades; análise de traços/contraexemplos; testes orientados a eventos e tempo (DICK; HULL; JACKSON, 2017; LAPLANTE, 2017).                     |
| <b>Checagem automática e análise de propriedades</b>        | Alto risco; custo alto de falha; necessidade de detectar lacunas cedo (concorrência, prioridades, exceções). | Modelo formal de estados/transições + propriedades declaradas + dicionário.                                  | Registrar resultados como evidência: propriedade verificada, propriedade violada e contraexemplo; gerar ação de refinamento de requisito; atualizar rastreabilidade. | Conjunto de propriedades (P1–P3), relatórios de verificação e contraexemplos interpretáveis para refinamento. | Evidência por verificação automática (pass/fail) + contraexemplo como “insumo de análise”; revisão de requisito e de critérios de aceitação (YOUNG, 2004; IEEE, 2024). |
| <b>Integração final (tabela de rastreabilidade)</b>         | Quando a formalização é seletiva e precisa ser justificada e gerenciável.                                    | Lista de requisitos críticos + propriedades/invariantes/contratos + risco + técnica + método de verificação. | Consolidar em quadro único (ex.: Quadro 53): requisito → expressão formal → risco → técnica → verificação; referenciar no Capítulo 11.                               | Artefato de integração que mostra que formalização é “reforço do requisito”, não “documento paralelo”.        | Auditoria de rastreabilidade; consistência entre requisitos, propriedades e plano de V&V (IEEE, 2024; YOUNG, 2004).                                                    |

Quadro 55 – Síntese e integração das representações formais.

Fonte: elaborado pelo autor (2026).



## 10.8 Síntese comparativa: entregas e riscos por tipo de representação

Até aqui, discutimos representações formais e as propriedades que elas podem tornar explícitas. Para manter o foco do capítulo na **escolha de representações por propósito e risco** — e não na ideia de uma “escada” obrigatória de rigor — é útil sintetizar, no Quadro 56, as contribuições típicas e os riscos mais comuns de cada tipo de representação (informal, semiformal e formal). Essa síntese deve ser lida como **repertório combinável**: em projetos reais, as equipes alternam e integram representações para sustentar entendimento, coordenação e evolução do sistema de requisitos.

| Tipo de representação                                                                 | Propósito predominante                                                                              | Entregas típicas (o que viabiliza)                                                                                                                                                                            | Quando tende a valer o custo                                                                                                                               | Riscos/armadilhas comuns                                                                                                                                                  |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Informal</b> (registros livres, linguagem natural, esboços)                        | <b>Explorar e alinhar entendimento</b> com linguagem acessível a todos os stakeholders              | Compreensão de contexto e fronteiras; hipóteses e decisões iniciais; vocabulário e exemplos; visibilidade de necessidades e tensões; consenso inicial e registro do racional                                  | Início do trabalho; alta incerteza; descoberta e negociação; necessidade de comunicação ampla e rápida                                                     | Ambiguidade persistente; decisões “orais” que se perdem; inconsistências não percebidas; baixa rastreabilidade; excesso de interpretação individual                       |
| <b>Semiformal</b> (modelos, diagramas, especificações estruturadas)                   | <b>Estruturar e coordenar</b> o sistema de requisitos com mais consistência, sem romper comunicação | Delimitação de escopo e interfaces; fluxos e exceções; responsabilidades e dependências; coerência entre partes do sistema; base mais estável para rastreabilidade e análise de impacto                       | Múltiplos times / áreas; integração com sistemas; complexidade moderada / alta; necessidade de reduzir conflitos antes de decisões se consolidarem         | “Modelo pelo modelo”; falsa sensação de completude; linguagem inacessível a decisores; esforço de manutenção; se tratado com rigidez pode impedir aprendizado incremental |
| <b>Formal</b> (uso seletivo; semântica matemática; checagens/provas quando aplicável) | <b>Explicitar propriedades críticas</b> com alto rigor, em recortes onde o custo do erro é alto     | Regras e invariantes sem ambiguidade relevante; consistência lógica em recortes críticos; propriedades comportamentais / temporais bem definidas; detecção precoce de contradições em especificações críticas | Alta criticidade, risco ou regulação; propriedades temporais / concorrência; integridade complexa; cenários onde interpretações divergentes têm alto custo | Custo / curva de aprendizagem; isolamento técnico; afastamento de stakeholders; formalizar o que é volátil; deslocamento precoce para decisões de solução (design)        |

Quadro 56 – Síntese comparativa dos tipos de representação de requisitos (informal, semiformal e formal).  
Fonte: elaborado pelo autor (2026).

É importante destacar que os tipos não são etapas obrigatórias nem “níveis de maturidade”. Em Engenharia de Requisitos, a prática mais comum é combinar representações: informais para descoberta e alinhamento, semiformais para estruturar e reduzir conflitos, e formais apenas quando o risco e a criticidade justificarem o investimento.

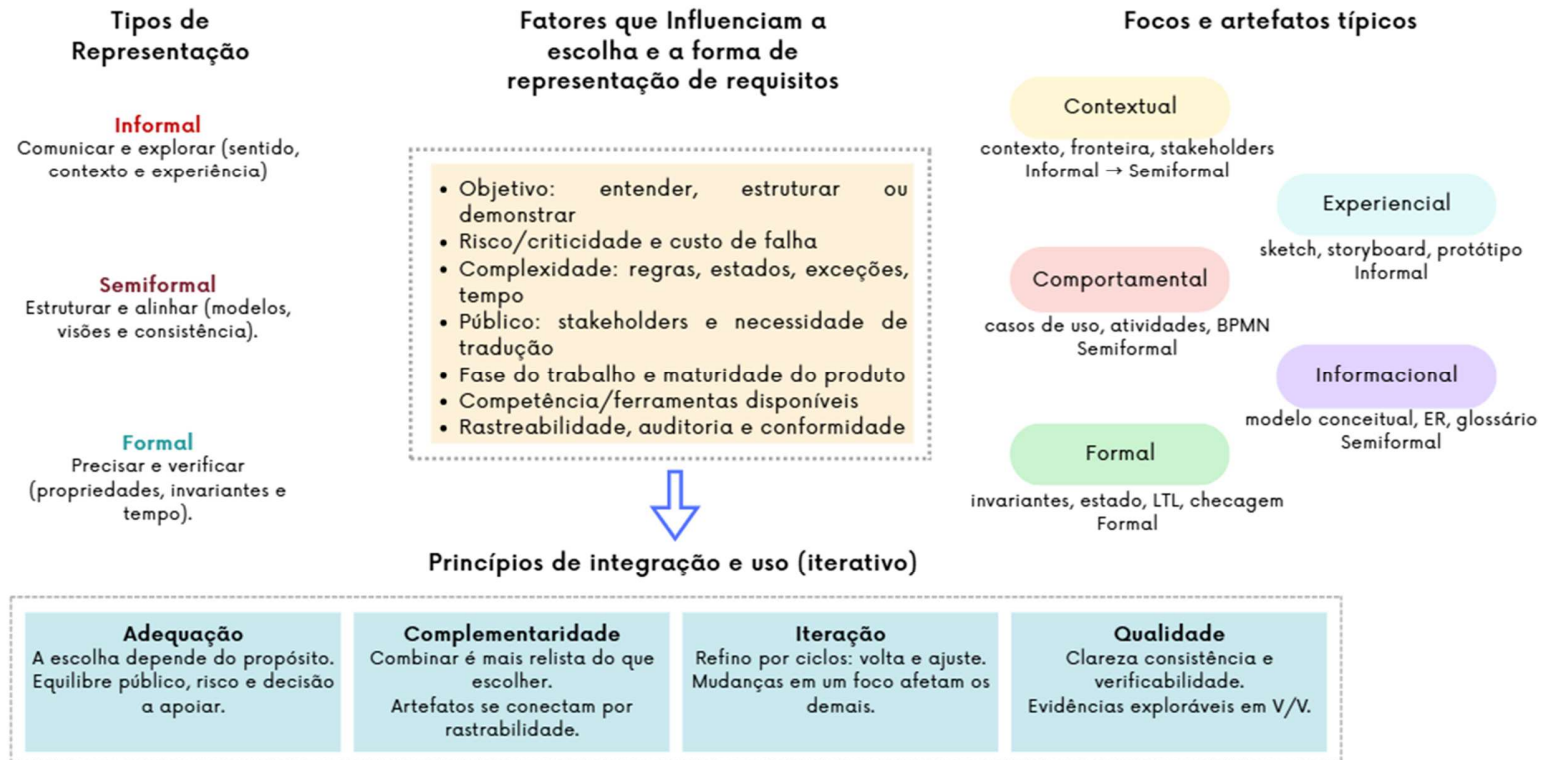


Figura 73 – Relação entre os conceitos apresentados no Capítulo 10.  
Fonte: elaborado pelo autor (2026).

## CAPÍTULO 10: PONTOS-CHAVE

- **Representar requisitos** significa tornar perceptíveis aspectos do entendimento por meio de formas visuais, gráficas, simbólicas ou matemáticas que favoreçam observação, discussão e análise.
- **Declaração e representação são formas complementares** de externalização do entendimento: uma privilegia a expressão linguística; a outra torna relações, estruturas, comportamentos ou propriedades visíveis por outros meios.
- **Representações informais** favorecem exploração, pensamento visual e comunicação acessível, sendo especialmente úteis quando ideias e relações ainda estão em construção.
- **Representações semiformais** utilizam notações parcialmente padronizadas, como UML, BPMN, DFD e SysML, para tornar comportamentos, processos, fluxos de dados e relações entre requisitos mais estruturados e examináveis.
- **Diferentes representações podem ser combinadas** quando oferecem perspectivas complementares sobre o mesmo sistema de requisitos; sua simples coexistência, contudo, não garante coerência.
- **Representações formais** utilizam bases matemáticas explícitas e podem apoiar análise rigorosa e checagem de propriedades em situações nas quais criticidade, risco ou necessidade de precisão justificam maior formalidade.
- **A escolha da representação** deve considerar propósito, público, risco, criticidade, custo e natureza do conhecimento que se pretende tornar visível, evitando tanto informalidade insuficiente quanto formalização desnecessária.
- **A integração entre representações** depende da preservação de um núcleo de significado comum e de relações que permitam navegar entre diferentes perspectivas sem criar artefatos desconectados.

# Capítulo 11

## 11 Verificação e Validação de Requisitos

### 11.1 Introdução

A verificação e a validação (V&V) constituem uma **prática transversal de qualidade no trabalho com requisitos**. Elas não devem ser compreendidas como um “momento final”, acionado apenas quando o conjunto de requisitos parece suficientemente estável, nem como mera conferência documental. Em Engenharia de Requisitos, V&V reúne ações formais ou informais pelas quais pessoas e equipes examinam o entendimento e confrontam declarações, representações, relações e decisões com critérios, evidências e expectativas, buscando tornar visíveis ambiguidades, lacunas, inconsistências e inadequações antes que se convertam em decisões equivocadas, retrabalho, defeitos de implementação ou frustração dos stakeholders (IREB, 2022; WIEGERS; BEATTY, 2013; ISO/IEC/IEEE 29148:2018).

Por ser transversal, V&V pode incidir sobre resultados produzidos em diferentes momentos da Engenharia de Requisitos e revelar novas necessidades de investigação, reconsideração de decisões, reformulações ou atualizações. Sua intensidade varia conforme risco, criticidade, volatilidade, custo do erro e maturidade do sistema de requisitos. Isso implica reconhecer, em termos didáticos, duas ideias centrais:

- V&V não é sinônimo de formalidade: ela pode assumir a forma de ciclos curtos de feedback e ainda assim produzir valor substantivo, desde que reduza incerteza, revele problemas relevantes e amplie o alinhamento entre os envolvidos.
- Tornar requisitos mais explícitos e melhor representados fortalece V&V, porque facilita discussão, revisão, comparação e produção de evidências, mas não constitui condição para que ela exista.

**A prática começa na própria comunicação**, isto é, no processo de construção de sentido compartilhado, e tende a se tornar mais estruturada à medida que o conjunto de requisitos amadurece.

Nessa perspectiva, verificar e validar não significa “encerrar” um requisito, mas **produzir evidências que permitam sustentar criticamente sua evolução**. Os achados podem revelar ambiguidades, lacunas, inconsistências, pressupostos implícitos ou desalinhamentos com necessidades e contexto, oferecendo elementos para que o entendimento seja revisto quando necessário. V&V funciona, assim, como mecanismo contínuo de aprendizagem e redução de incerteza, contribuindo para construir confiança qualificada sobre aquilo que o sistema deve fazer, em que condições, para quem, com quais restrições e com quais evidências de atendimento (IREB, 2022; WIEGERS; BEATTY, 2013; ISO/IEC/IEEE 29148:2018).

Como atividade transversal, a verificação e validação utiliza resultados produzidos pelas demais atividades e pode gerar achados que as reativam. A Figura 74 ilustra sua responsabilidade própria, distinguindo a avaliação da qualidade e adequação dos requisitos dos ajustes que eventualmente serão realizados a partir de seus achados.

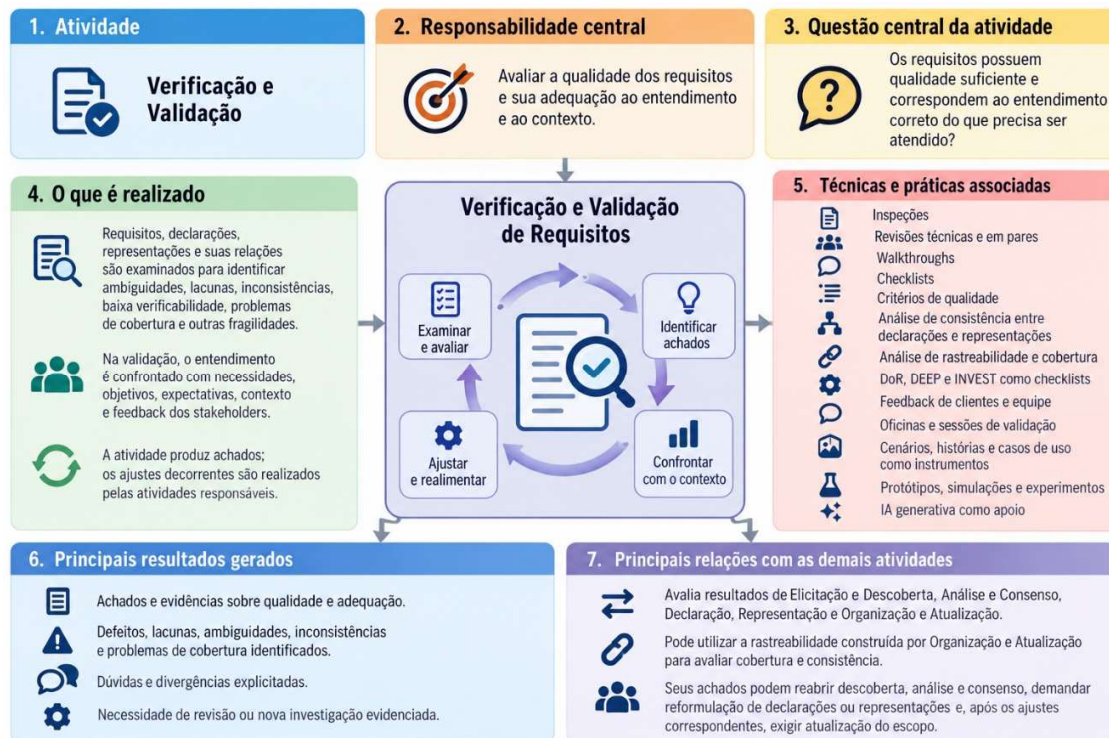


Figura 74 – Enquadramento da atividade de Verificação e Validação.  
Fonte: elaborado pelo autor (2026).

## 11.2 Conceitos e distinções essenciais

A distinção entre verificação e validação de requisitos (ver Figura 75) é frequentemente resumida pelas seguintes perguntas:

- **Verificação:** “O requisito foi feito da maneira correta?”
- **Validação:** “O requisito feito é o requisito certo?”

No contexto deste livro, “**correto**” não significa “verdade absoluta”, mas consistente, claro, completo o suficiente ao propósito, verificável e rastreável — isto é, adequado à função comunicacional e ao uso pretendido do requisito (ISO/IEC/IEEE 29148:2018; WIEGERS; BEATTY, 2013). Já “**certo**” significa alinhado ao valor e às necessidades dos stakeholders, considerando restrições do contexto e trade-offs.

A diferença também aparece na **natureza das evidências**:

- A **verificação** produz evidências predominantemente **técnicas e internas (qualidade interna)**: listas de defeitos de requisitos, inconsistências entre artefatos, violações de regras de escrita, conflitos de termos no glossário, lacunas em critérios de aceitação, violações de propriedades formais, falhas de rastreabilidade etc.
- A **validação** produz evidências predominantemente **sociotécnicas e externas** (mundo do uso e do negócio): aceite ou rejeição por stakeholders, adequação a cenários reais, evidências que podem confirmar ou questionar prioridades, identificação de expectativas

implícitas, alinhamento com objetivos de valor, adequação de critérios de aceite, compatibilidade com o fluxo de trabalho e viabilidade percebida no uso.

Essa distinção **não implica sequência**. Em Engenharia de Requisitos, verificação e validação frequentemente ocorrem na mesma interação. Um stakeholder pode rejeitar uma declaração (“*não foi isso que eu quis dizer*”), ao mesmo tempo em que a equipe de engenharia de software identifica ambiguidade ou lacuna de formulação. O valor da distinção está em tornar explícito qual tipo de incerteza está sendo reduzida: incerteza de **formulação** (verificação) ou de **propósito/valor** (validação) (IREB, 2022; WIEGERS; BEATTY, 2013).



Figura 75 – Conceito e distinções entre Verificação e Validação.  
Fonte: elaborado pelo autor (2026).

### Box 11.2-1 – Princípios em ação: V&V como materialização da comunicação e da validação

A V&V torna operacionais dois princípios do Capítulo 5:

- **Princípio 2 (Comunicação dos requisitos):** verificação e validação são instrumentos para reduzir ruído, tornar interpretações explícitas e preservar sentido compartilhado ao longo do tempo.
- **Princípio 7 (Validação de requisitos):** nenhum requisito deveria avançar como “verdade” apenas por ter sido declarado; ele precisa ser confrontado com cenários, objetivos e stakeholders, produzindo evidências de aceite (ou de ajuste).

## 11.3 Qualidade de requisitos como base de V&V

### 11.3.1 Confiança qualificada

A função central da verificação e da validação na Engenharia de Requisitos é **produzir confiança qualificada**. Não uma confiança vaga, baseada em impressão subjetiva ou em otimismo de equipe, mas uma confiança sustentada por critérios, por confronto entre artefatos, por diálogo



entre perspectivas e por evidências suficientes para apoiar decisões. Em projetos de software, a maior parte dos problemas graves não nasce de más intenções, mas de entendimento incompleto, de pressupostos não explicitados, de termos ambíguos, de conflitos silenciosos e de decisões tomadas cedo demais sem base adequada. V&V atua exatamente nesse espaço intermediário entre o que parece claro e o que de fato está sólido.

Por isso, a relação entre V&V e qualidade dos requisitos precisa ser tratada de **modo operacional**. Dizer que um requisito “deve ser claro” ou “deve ser completo” ainda é pouco, se a equipe não consegue transformar tais qualidades em perguntas verificáveis. Clareza, por exemplo, só se torna útil quando alguém pode perguntar se um leitor do domínio compreende o requisito sem pedir tradução adicional. Completude suficiente ao propósito só ganha valor quando se verifica se faltam ator, gatilho, dado, condição, exceção, restrição ou resultado observável. Consistência se torna concreta quando se pergunta se o requisito entra em choque com outro requisito, com uma regra de negócio, com um modelo ou com um critério já aceito. Verificabilidade só deixa de ser slogan quando se pergunta que evidência, medida, observação ou checagem poderia demonstrar atendimento.

Esse movimento é decisivo para posicionar V&V dentro da Engenharia de Requisitos. A atividade não existe para “policiar”, nem para fazer triagem cosmética de declaração ou representação. Seu lugar é mais profundo: transformar o conjunto de requisitos em base de trabalho confiável, sem perder de vista sua natureza comunicacional e sociotécnica. Requisitos não são apenas frases. São unidades de entendimento compartilhado, situadas em contextos organizacionais, técnicas e humanas. Verificá-los e validá-los implica, portanto, tratar tanto da sua qualidade interna quanto da sua adequação externa.

#### Box 11.3-1 – Princípios em ação: qualidade como comunicação e mediação

V&V não é “caça a defeitos” por formalismo; é um exercício de mediação e alinhamento, sustentado por:

- **Princípio 1 (Orientação a valor):** um requisito “bom” é aquele que sustenta decisões que entregam valor; verificar qualidade é garantir que o valor não se perca em ruído e retrabalho.
- **Princípio 4 (Entendimento compartilhado):** o critério prático de qualidade é a estabilidade do sentido entre diferentes atores; quando a equipe não consegue concordar sobre o que um requisito significa, V&V precisa atuar antes que o código “decida” por todos.
- **Valor Feedback:** a qualidade aumenta quando o requisito é confrontado repetidamente com evidências (cenários, protótipos, testes, modelos), não quando é “polido” em isolamento.

### 11.3.2 Critérios operacionais: de “qualidade desejável” a “perguntas verificáveis”

A literatura de Engenharia de Requisitos e padrões como a ISO/IEC/IEEE 29148 convergem em um ponto: um requisito só é útil se pode ser **entendido, discutido, verificado e mantido** (ISO/IEC/IEEE 29148:2018; WIEGERS; BEATTY, 2013; IREB, 2022). Porém, em prática,



dizer “o requisito deve ser claro” é insuficiente — é preciso transformá-lo em **perguntas e evidências**.

O Quadro 57 propõe um conjunto de critérios operacionais alinhados a este livro: V&V não avalia “documentos”, mas avalia a qualidade de **declarações e representações**, e a capacidade de essas unidades comunicacionais sustentarem decisões e evidências.

| Critério de qualidade                | Pergunta de verificação                                                                              | Evidências possíveis                                                                                     | Exemplo no caso condutor                                                                                                       |
|--------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Clareza / compreensão                | Um stakeholder do domínio entende o requisito sem precisar “traduzir” seu sentido?                   | Dúvidas registradas em revisão, necessidade de reformulação, divergência de interpretação entre leitores | “O sistema deve enviar notificação ao cidadão” exige esclarecer em que eventos, por quais canais e em que prazo                |
| Ausência de ambiguidade relevante    | Há termos, verbos ou condições que podem ser interpretados de mais de uma forma?                     | Marcação de expressões vagas, exemplos contraditórios, múltiplas leituras plausíveis                     | Termos como “rapidamente”, “quando necessário” ou “regularmente” precisam ser substituídos por condições observáveis           |
| Completeness suficiente ao propósito | Faltam ator, gatilho, entradas, regras, exceções, restrições ou resultado esperado?                  | Lacunas apontadas em walkthrough, cenários interrompidos, fluxos sem desfecho                            | No agendamento de consulta, a regra de tratamento para ausência de vagas precisa estar explícita                               |
| Consistência                         | O requisito conflita com outro requisito, regra de negócio, modelo, processo ou decisão já aceita?   | Incompatibilidades entre artefatos, contradições em tabelas, inconsistências entre texto e diagrama      | Um requisito que permita remarcar livremente consultas pode conflitar com política de limite de reagendamentos                 |
| Verificabilidade                     | É possível demonstrar, por observação, checagem, medição ou evidência, que o requisito foi atendido? | Critérios de aceite, exemplos de evidência, cenários verificáveis                                        | “Confirmar agendamento em até 2 minutos após a solicitação” admite verificação; “confirmar rapidamente” não                    |
| Rastreabilidade e cobertura          | É possível relacionar o requisito à sua origem, aos artefatos correlatos e às decisões que o afetam? | Links para objetivos, regras, cenários, modelos e itens do backlog                                       | O requisito de notificação deve poder ser rastreado ao objetivo de reduzir faltas e ao fluxo de confirmação                    |
| Pertinência / valor                  | O requisito responde a necessidade real, regra aplicável, risco relevante ou expectativa legitimada? | Justificativa explícita, vínculo com objetivo, priorização coerente                                      | Exigir impressão obrigatória de comprovante pode não gerar valor se o serviço for concebido para uso predominantemente digital |

Quadro 57 – Critérios operacionais de V&V.  
Fonte: elaborado pelo autor (2026).

O uso desses critérios ajuda a deslocar a discussão de juízos vagos para exame fundamentado. Em vez de afirmar genericamente que “o requisito está ruim”, a equipe consegue apontar onde está o problema: talvez ele esteja compreensível, mas incompleto; talvez esteja detalhado, porém inconsistente com uma regra do domínio; talvez tenha boa redação, mas não permita produzir evidência de atendimento. Essa distinção melhora a qualidade da conversa, evita críticas genéricas e favorece correções mais precisas.

Também é importante notar que esses critérios não devem ser aplicados de forma uniforme e mecânica a todo requisito, em qualquer contexto. A profundidade da checagem depende da criticidade da funcionalidade, do risco envolvido, da estabilidade do domínio, da maturidade dos artefatos e do custo do erro. Em um requisito sensível a conformidade legal, segurança, integridade de dados ou atendimento ao cidadão, a exigência de precisão e evidência tende a ser maior. Em requisitos exploratórios, iniciais ou sujeitos a aprendizagem incremental, pode ser mais adequado trabalhar com formulações provisórias, desde que isso seja conscientemente reconhecido e acompanhado.

Em outras palavras, operacionalizar qualidade não significa burocratizar V&V. Significa dar à equipe linguagem, foco e critérios para examinar o que realmente importa, com proporcionalidade e senso de contexto.

### 11.3.3 Defeitos típicos de requisitos e efeitos práticos

Defeitos de requisitos raramente são “falhas abstratas”; eles se manifestam como **efeitos concretos** em projetos. A seguir, alguns defeitos típicos (WIEGERS; BEATTY, 2013; YOUNG, 2004; ISO/IEC/IEEE 29148:2018):

- **Ambiguidade:** gera interpretações divergentes, disputa de aceite e testes frágeis.  
**Ex.:** “O sistema deve notificar o cidadão rapidamente após o agendamento.”  
Nesse enunciado, “notificar” pode significar e-mail, SMS ou mensagem no próprio sistema, e “rapidamente” pode ser interpretado como segundos, minutos ou até o mesmo dia. O efeito prático é que áreas diferentes passam a defender entendimentos distintos no momento do aceite.
- **Incompletude:** provoca decisões implícitas de implementação (“alguém vai decidir no código”), elevando risco.  
**Ex.:** “O sistema deve permitir cancelar agendamentos.”  
O requisito não informa quem pode cancelar, até quando o cancelamento é permitido, se há necessidade de justificativa, nem o que acontece com a vaga liberada. Na prática, essas decisões acabam sendo tomadas informalmente durante a implementação, sem validação adequada com stakeholders.
- **Inconsistência:** cria comportamento contraditório e retrabalho, especialmente quando modelos evidenciam conflitos.  
**Ex.:** um requisito textual afirma que “o cidadão pode cancelar o agendamento até 24 horas antes da consulta”, enquanto uma regra de negócio em outro artefato estabelece que “cancelamentos são permitidos até 12 horas antes”.  
O efeito prático é a implementação de comportamentos contraditórios, além de retrabalho quando o conflito é percebido tardiamente.
- **Não verificabilidade:** impede critérios de aceitação claros; o aceite vira opinião.  
**Ex.:** “O sistema deve ter uma interface amigável para o cidadão.”  
Embora a intenção seja compreensível, o requisito não explicita nenhuma condição observável ou critério verificável. Sem parâmetros mais concretos, como tempo para concluir tarefas, clareza de navegação ou taxa de erro, a aceitação dependerá de percepções subjetivas.

- **Solução embutida indevida:** restringe alternativas e pode gerar conflito com arquitetura/infraestrutura.  
**Ex.:** “O sistema deve enviar notificações exclusivamente por SMS.”  
Se a necessidade real é garantir que o cidadão receba aviso de confirmação ou cancelamento, fixar SMS desde o requisito pode restringir alternativas mais adequadas ou economicamente viáveis, como e-mail, aplicativo institucional ou múltiplos canais configuráveis.
- **Escopo confuso (não atômico):** dificulta priorização e evolução incremental (Cap. 9, Seção 9.8.2).  
**Ex.:** “O sistema deve permitir ao cidadão agendar, cancelar, remarcar consultas, consultar histórico, avaliar o atendimento e receber notificações.”  
O enunciado reúne várias capacidades distintas em um único requisito. Na prática, isso dificulta priorização, rastreamento, negociação e evolução incremental, pois não se sabe exatamente qual parte foi implementada, validada ou alterada. Essa declaração deveria ser dividida em vários requisitos específicos: (i) agendar, (ii) cancelar, (ii) remarcar consultas etc.

O Quadro 58 resume esses defeitos e seus efeitos mais usuais.

| Defeito típico            | Como aparece                                         | Efeito prático mais comum                                       | Exemplo resumido                                                     |
|---------------------------|------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------|
| Ambiguidade               | Termos vagos ou múltiplas leituras plausíveis        | Interpretações divergentes, disputa de aceite, testes frágeis   | “Notificar rapidamente”                                              |
| Incompletude              | Falta de ator, condição, exceção, dado ou efeito     | Decisões implícitas na implementação, aumento de risco          | “Permitir cancelar agendamentos”                                     |
| Inconsistência            | Contradição entre requisitos, regras ou modelos      | Comportamento contraditório, retrabalho, conflitos tardios      | Prazo de cancelamento de 24h declarado em um artefato e 12h em outro |
| Não verificabilidade      | Ausência de medida, observação ou critério de aceite | Aceite subjetivo, validação por opinião                         | “Interface amigável”                                                 |
| Solução embutida indevida | Tecnologia ou desenho fixado sem justificativa       | Restrição prematura de alternativas, conflito com arquitetura   | “Enviar exclusivamente por SMS”                                      |
| Escopo não atômico        | Múltiplas funcionalidades em um único enunciado      | Dificuldade de priorização, rastreamento e evolução incremental | “Agendar, cancelar, remarcar, avaliar...”                            |

Quadro 58 – Defeitos típicos e seus efeitos mais usuais.  
Fonte: elaborado pelo autor (2026).

#### 11.3.4 Técnicas de V&V e sua utilidade na identificação de defeitos

Nem toda técnica de verificação é igualmente eficaz para todo tipo de defeito. Inspeções, revisões técnicas, walkthroughs e checklists são particularmente úteis para revelar ambiguidades, lacunas, inconsistências e problemas de verificabilidade. A análise de consistência entre

declarações e representações tende a ser mais forte quando o defeito emerge da relação entre artefatos, como conflitos entre requisitos, regras de negócio, modelos, fluxos, estados ou critérios. A rastreabilidade e a análise de cobertura ajudam a identificar requisitos sem origem, sem justificativa, sem vínculo com objetivos ou sem evidências futuras de atendimento. Em contextos ágeis e híbridos, DoR, DEEP e INVEST funcionam como apoios leves para verificar a prontidão de itens de backlog, a qualidade do backlog como conjunto e a qualidade de histórias ou itens próximos da implementação. A IA generativa pode apoiar a revisão preliminar, sugerindo perguntas, reformulações e possíveis inconsistências, mas não substitui o julgamento humano nem o conhecimento de domínio.

A escolha da técnica, portanto, deve ser pragmática. Não se trata de acumular procedimentos, mas de combinar abordagens compatíveis com o tipo de defeito que se deseja encontrar e com o nível de risco do contexto (ver Quadro 59).

| Defeito típico                | Técnicas de verificação mais indicadas                                                                                | Técnicas complementares                                                           | Observação de uso                                                                                                                                                               |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ambiguidade                   | Inspeções; revisões técnicas; walkthroughs; checklists; critérios de qualidade de requisitos                          | Análise de consistência entre declarações e representações; IA generativa         | Úteis para identificar termos vagos, múltiplas leituras plausíveis, pressupostos ocultos e trechos que dependem de explicação oral para serem compreendidos.                    |
| Incompletude                  | Inspeções; walkthroughs; checklists; análise de rastreabilidade e cobertura; DoR                                      | Revisões técnicas; critérios de qualidade; IA generativa                          | Ajuda a revelar ausência de ator, condição, exceção, dado, regra, efeito, dependência ou evidência mínima para avançar.                                                         |
| Inconsistência                | Análise de consistência entre declarações e representações; revisões técnicas; inspeções; rastreabilidade e cobertura | Checklists; IA generativa                                                         | Indicada quando há contradição entre requisitos, regras de negócio, modelos, fluxos, estados, critérios, protótipos ou decisões registradas.                                    |
| Não verificabilidade          | Críticos de qualidade de requisitos; checklists; inspeções; revisões técnicas; DoR                                    | IA generativa para sugerir reformulações; análise de consistência entre artefatos | Útil para transformar formulações subjetivas, adjetivadas ou genéricas em condições observáveis, mensuráveis ou avaliáveis.                                                     |
| Solução embutida indevida     | Revisões técnicas; inspeções; checklists; critérios de qualidade de requisitos                                        | Rastreabilidade; walkthroughs; IA generativa                                      | Ajuda a identificar tecnologia, interface, arquitetura ou decisão de desenho fixada sem justificativa suficiente no problema, valor ou restrição real.                          |
| Escopo confuso ou não atômico | Checklists; revisões técnicas; walkthroughs; critérios de qualidade; INVEST                                           | DEEP; DoR; rastreabilidade e cobertura                                            | Útil para perceber requisitos que misturam múltiplas funcionalidades, regras, exceções ou objetivos, dificultando priorização, estimativa, rastreamento e evolução incremental. |

| Defeito típico                                                       | Técnicas de verificação mais indicadas                                                    | Técnicas complementares                                | Observação de uso                                                                                                                                                                |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Baixa rastreabilidade                                                | Análise de rastreabilidade e cobertura                                                    | Inspecções; revisões técnicas; checklists              | Indica requisitos sem origem clara, sem vínculo com objetivos, stakeholders, regras, decisões, artefatos de representação ou critérios de evidência.                             |
| Cobertura insuficiente                                               | Análise de rastreabilidade e cobertura; checklists; inspecções                            | Revisões técnicas; walkthroughs; IA generativa         | Ajuda a identificar objetivos não cobertos, regras sem requisitos associados, fluxos sem exceções tratadas ou requisitos sem critério de verificação futura.                     |
| Baixa prontidão para refinamento, planejamento ou implementação      | DoR; checklists; critérios de qualidade; INVEST                                           | Revisões técnicas; walkthroughs; DEEP                  | Indica itens priorizados, mas ainda imaturos: sem critérios mínimos, dependências visíveis, termos definidos, risco explicitado ou entendimento suficiente para avançar.         |
| Backlog desorganizado, obsoleto ou pouco acionável                   | DEEP; análise de rastreabilidade e cobertura; revisões técnicas                           | Checklists; IA generativa                              | Útil para verificar se o backlog está detalhado apropriadamente, emergente, estimado e priorizado, e se seus itens ainda representam o entendimento atual do produto.            |
| História ou item de backlog mal formulado                            | INVEST; checklists; critérios de qualidade; DoR                                           | Revisões em pares; IA generativa                       | Ajuda a verificar se a história ou item é independente, negociável, valioso, estimável, pequeno e testável, sem se confundir com tarefa técnica ou decisão prematura de solução. |
| Propriedades temporais, condicionais ou concorrenciais mal definidas | Análise de consistência entre declarações e representações; inspecções; revisões técnicas | Crítérios de qualidade; rastreabilidade; IA generativa | Útil quando o requisito envolve prazos, estados, transições, simultaneidade, exceções, dependências entre eventos ou condições de execução.                                      |

Quadro 59 – Defeitos típicos e técnicas usuais para tratá-los.  
Fonte: elaborado pelo autor (2026).

O Quadro 59 deve ser lido como apoio à escolha de técnicas de verificação, não como mecanismo automático de decisão. O julgamento profissional continua indispensável, sobretudo porque projetos diferem em criticidade, maturidade dos artefatos, disponibilidade de stakeholders, tempo e custo de revisão. Ainda assim, visualizar a relação entre defeitos e técnicas ajuda a equipe a evitar dois extremos igualmente problemáticos: aplicar sempre as mesmas técnicas por hábito, ou escolher técnicas de forma intuitiva e pouco justificada.

### 11.3.5 Síntese: qualidade operacional como fundamento da prática de V&V

Ao tratar qualidade de requisitos como base operacional de V&V (ver Figura 76), esta seção propõe uma mudança de postura. Em vez de considerar V&V como etapa de conferência documental, ela passa a ser compreendida como prática contínua de construção de confiança

qualificada. Essa confiança depende menos de formalismo em si e mais da capacidade de formular boas perguntas, produzir evidências adequadas, detectar defeitos relevantes e ajustar o entendimento antes que ele se converta em implementação, teste ou compromisso contratual.

Nesse sentido, a qualidade dos requisitos não deve ser vista como atributo abstrato ou ideal normativo distante da prática. Ela precisa ser continuamente traduzida em ações de revisão, confronto, experimentação, negociação e aprendizado. Quando isso ocorre, V&V deixa de ser percebida como custo adicional e passa a operar como mecanismo de prevenção de ruído, redução de retrabalho e fortalecimento da base decisória do projeto. É nessa condição que ela cumpre seu papel mais importante na Engenharia de Requisitos: tornar o entendimento compartilhado suficientemente confiável para sustentar a evolução do sistema e do próprio trabalho coletivo.

|                                                                                     |                                          |                                                                                                              |
|-------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------------|
|    | <b>Clareza</b>                           | <b>Stakeholders entendem sem tradução?</b><br>Ex: Evitar notificar o cidadão sem especificar prazo ou canal. |
|    | <b>Ausência de Ambiguidade</b>           | <b>Termos têm leitura única?</b><br>Ex: Substituir rapidamente por condições observáveis em minutos.         |
|    | <b>Completeness Suficiente</b>           | <b>Faltam atores, regras ou exceções?</b><br>Ex: Definir a regra se não houver vagas disponíveis.            |
|   | <b>Consistência</b>                      | <b>Conflita com regras ou diagramas aceitos?</b><br>Ex: Texto permite remarcar, mas política geral proíbe.   |
|  | <b>Verificabilidade</b>                  | <b>É possível demonstrar o atendimento?</b><br>Ex: Medir tempo exato vs. percepção abstrata de rápido.       |
|  | <b>Rastreabilidade &amp; Pertinência</b> | <b>Qual a origem e o valor de negócio atrelado?</b><br>Ex: Origem do pedido e métrica de sucesso.            |

Figura 76 – Qualidade de requisitos e suas perguntas.  
Fonte: elaborado pelo autor (2026).

## 11.4 Estratégia proporcional de V&V

A verificação e a validação de requisitos não devem ser aplicadas com a mesma intensidade a todos os requisitos, em todos os momentos e em todos os projetos. Uma prática madura de V&V exige proporcionalidade: requisitos mais críticos, mais arriscados, mais regulados, mais instáveis ou mais dependentes de interpretação compartilhada justificam maior rigor; requisitos exploratórios, iniciais ou de baixo impacto podem ser trabalhados com mecanismos mais leves, desde que suas incertezas permaneçam visíveis.

Essa proporcionalidade não significa reduzir a importância da V&V, mas calibrar sua aplicação. Em alguns casos, uma conversa estruturada, uma revisão em pares ou um checklist simples já são suficientes para reduzir ambiguidades relevantes. Em outros, pode ser necessário realizar inspeções formais, oficinas com stakeholders, análise de rastreabilidade, prototipação, simulações, definição rigorosa de critérios de aceite ou mesmo formalização seletiva de propriedades. A escolha não deve ser guiada por preferência metodológica, mas por risco, criticidade, custo do erro, maturidade do entendimento e necessidade de evidência.



Em termos práticos, a equipe de engenharia de software pode orientar sua estratégia de V&V por cinco perguntas (ver Figura 77):



Figura 77 – Estratégia proporcional de V&V.  
Fonte: elaborado pelo autor (2026).

### 1. Quando aplicar?

A V&V deve ocorrer sempre que um requisito passar por mudança relevante de entendimento, for usado para orientar decisão de projeto, entrar em refinamento para implementação, servir de base para contrato, envolver regra crítica de negócio ou afetar segurança, conformidade, privacidade, desempenho, acessibilidade ou experiência de uso.

### 2. O que verificar?

A verificação deve examinar a qualidade interna do requisito ou do conjunto de requisitos: clareza, ausência de ambiguidade relevante, completude suficiente, consistência, verificabilidade, rastreabilidade, cobertura e coerência entre declarações, representações, critérios e decisões.

### 3. O que validar?

A validação deve confrontar o requisito com necessidades, objetivos, expectativas e restrições reconhecidas pelos stakeholders. A pergunta central não é apenas se o requisito está bem formulado, mas se ele representa algo pertinente, valioso, necessário e adequado ao contexto de uso e decisão.

### 4. Quanto rigor aplicar?

O rigor deve crescer conforme aumentam a criticidade, o custo do erro, o grau de incerteza, a dependência entre requisitos, a quantidade de stakeholders afetados, a instabilidade do domínio e a necessidade de evidência auditável.

### 5. Quem envolver e que evidências produzir?

A verificação pode envolver engenheiros de requisitos, analistas, desenvolvedores, testadores, arquitetos e pares técnicos. A validação exige participação de stakeholders capazes de reconhecer valor, necessidade, adequação e consequências do requisito. As evidências podem variar de anotações em backlog e decisões registradas em ata até matrizes de rastreabilidade, critérios de aceite aprovados, protótipos avaliados, cenários validados e registros formais de aceite.

O Quadro 60 sintetiza essa estratégia de calibragem.

| Situação                                                      | Estratégia recomendada                                                | Técnicas adequadas                                                                                                   | Evidências esperadas                                                                            |
|---------------------------------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Requisito inicial, exploratório ou ainda pouco compreendido   | V&V leve e frequente, com foco em aprendizagem                        | Conversas de esclarecimento, revisão rápida, exemplos, perguntas-guia                                                | Dúvidas registradas, reformulações, hipóteses abertas, decisões provisórias                     |
| Requisito próximo de implementação                            | V&V moderada, com foco em prontidão e redução de ambiguidade          | Revisão em pares, checklist, critérios de aceite, análise de consistência, DEEP/INVEST quando aplicável              | Críticos de aceite, dependências visíveis, item refinado, inconsistências tratadas              |
| Requisito crítico, regulatório, contratual ou de alto risco   | V&V rigorosa, com foco em evidência e rastreabilidade                 | Inspeção, revisão técnica formal, análise de cobertura, rastreabilidade, simulação, protótipo, formalização seletiva | Registro de achados, decisões aprovadas, evidências de validação, vínculos rastreáveis          |
| Requisito associado a experiência de uso ou fluxo de trabalho | Validação colaborativa, com foco em reconhecimento pelos stakeholders | Oficinas, cenários, histórias, casos de uso, protótipos, simulações                                                  | Feedback de stakeholders, ajustes no fluxo, aceite ou rejeição fundamentada                     |
| Conjunto de requisitos em backlog ágil ou híbrido             | Verificação contínua de qualidade do backlog e dos itens              | Refinamento, DoR, DEEP, INVEST, critérios de aceite                                                                  | Backlog priorizado, itens suficientemente detalhados, histórias testáveis e pequenas o bastante |

Quadro 60 – Estratégia proporcional de V&V.  
Fonte: elaborado pelo autor (2026).

Essa estratégia também ajuda a evitar dois extremos. O primeiro é a informalidade ingênua, na qual requisitos avançam apenas porque “todos parecem ter entendido”. O segundo é a formalização excessiva, na qual a equipe produz muitos registros, mas pouca aprendizagem efetiva. Entre esses extremos, a V&V proporcional busca produzir evidência suficiente para sustentar boas decisões, sem transformar qualidade em burocracia. Por fim, cabe ressaltar que essa estratégia não deve ser lida como modelo fechado, mas como uma forma possível de orientar decisões proporcionais de V&V.

## 11.5 Técnicas de verificação

A verificação de requisitos incide sobre a qualidade do sistema de requisitos enquanto artefato de trabalho da Engenharia de Requisitos. Seu foco não está, em primeiro lugar, no comportamento do software executável, mas na clareza, consistência, completude suficiente, rastreabilidade e verificabilidade daquilo que foi declarado e representado. Verificar bem significa



produzir confiança antes da implementação: reduzir espaço para interpretações divergentes, explicitar lacunas, confrontar incoerências e tornar mais observável o que, mais adiante, precisará ser avaliado como atendido ou não atendido (WIEGERS; BEATTY, 2013; YOUNG, 2004; ISO/IEC/IEEE 29148:2018).

A seguir, é apresentado um conjunto (não limitante) de técnicas de verificação de requisitos comumente utilizadas na construção de produtos de software.

### 11.5.1 Inspeções

A inspeção é a **forma mais disciplinada de revisão** de requisitos. Em geral, ela é conduzida com algum grau de preparação prévia, papéis minimamente definidos, critérios explícitos e registro sistemático dos defeitos identificados. Seu valor não está apenas em “achar problemas”, mas em converter impressões vagas em achados objetivos: um termo ambíguo, uma exceção ausente, uma inconsistência com outro requisito, uma condição de aceitação não explicitada, um dado necessário que não aparece em nenhum artefato relacionado. Nesse sentido, a inspeção fortalece a verificabilidade porque exige que a crítica ao requisito seja fundamentada e passível de rastreamento (GILB; GRAHAM, 1993; WIEGERS; BEATTY, 2013).

Essa técnica é especialmente útil quando o projeto já possui um conjunto de requisitos suficientemente materializado e precisa elevar o grau de confiança antes de prosseguir. Ela também é valiosa em itens mais críticos, mais regulados ou mais propensos a efeitos em cascata, nos quais uma pequena ambiguidade pode contaminar fluxos, critérios de aceite e representações subsequentes.

A principal limitação da inspeção é o **risco de burocratização**: inspeções conduzidas apenas para “cumprir etapa” perdem densidade analítica e passam a produzir listas formais de não conformidades sem real impacto sobre a qualidade do entendimento. Por isso, inspeção não deve ser ritual vazio, mas atividade de leitura crítica orientada por propósito.

#### Box 11.5-1 – Princípios em ação: revisão exige coragem, respeito e compromisso

Revisões e inspeções só funcionam quando são sustentadas por valores do Capítulo 5:

- **Coragem:** apontar problemas de entendimento cedo, mesmo sob pressão de prazo.
- **Respeito:** criticar o *artefato* (declaração/representação), não a pessoa; manter ambiente seguro.
- **Compromisso:** registrar decisões e efetivamente atualizar requisitos, evitando “revisões teatrais”.

### 11.5.2 Revisões técnicas

As revisões técnicas possuem, em geral, **menor formalidade que as inspeções**, mas preservam um foco claro em consistência, impacto e adequação do requisito ao contexto do projeto. Em vez de se concentrarem apenas em defeitos, elas costumam mobilizar o olhar de pares ou de perfis complementares para discutir se o requisito está alinhado com **regras de negócio**, com **restrições do domínio**, com **outros requisitos** já declarados e com as **representações** produzidas ao

longo do trabalho de ER. São particularmente úteis quando o problema não é apenas a declaração do requisito, mas seu encaixe no sistema de requisitos como um todo (YOUNG, 2004; WIEGERS; BEATTY, 2013).

Seu valor está em ampliar o horizonte da checagem. Uma revisão técnica bem conduzida permite perguntar não apenas “*a declaração texto está clara?*”, mas também “*esse requisito faz sentido no contexto?*”, “*ele conflita com algo já acordado?*”, “*ele pressupõe uma decisão de solução ainda não justificada?*”, “*há efeito colateral sobre requisitos não funcionais ou regras de negócio?*”.

No contexto das revisões técnicas, cabem desde a **revisão individual**, útil para depuração preliminar, até revisões informais **em pares**, revisões em pares com perfis complementares e revisões **em pares progressivas**, distribuídas ao longo do amadurecimento do requisito. A revisão individual reduz ruído óbvio antes da conversa coletiva. A revisão informal em pares costuma ser valiosa quando o requisito ainda está se formando, pois permite correções rápidas e de baixo custo. A revisão em pares propriamente dita amplia o horizonte da checagem ao reunir olhares de negócio, domínio, UX, arquitetura, desenvolvimento, operação ou conformidade. Já a revisão em pares progressiva evita o problema comum da revisão tardia: em vez de concentrar toda a crítica no fim, acompanha o requisito em pequenos ciclos de leitura e ajuste.

O limite da revisão técnica se converte em **discussão aberta demais**, sem foco analítico ou sem critérios mínimos de observação. Nesses casos, ela tende a produzir comentários genéricos, preferências pessoais ou debates prematuros de solução. Para que cumpra seu papel, a revisão técnica precisa permanecer ancorada na qualidade do requisito e em seus vínculos com o restante do sistema de requisitos.

### 11.5.3 Walkthroughs

O walkthrough é uma **leitura guiada** do requisito ou do artefato correspondente, normalmente conduzida por quem o produziu, para que outros participantes façam perguntas, apontem dúvidas, identifiquem lacunas e explicitem interpretações divergentes. Em comparação com a inspeção, trata-se de uma **técnica mais leve e mais comunicacional**, frequentemente útil nas fases em que o requisito ainda está amadurecendo e precisa ser exposto ao olhar de outros antes de ganhar maior estabilidade formal (FREEDMAN; WEINBERG, 1990). A literatura e a prática profissional enfatizam esse caráter menos formal do walkthrough, destacando sua utilidade para requisitos e modelos, sobretudo quando o objetivo é comunicar o artefato e receber contribuições de melhoria.

O walkthrough não serve apenas para localizar defeitos; ele ajuda a **expor o requisito ao espaço da conversação**, no qual ambiguidades, pressupostos e omissões aparecem porque outras pessoas tentam compreender aquilo que o autor acreditava já estar claro. Nesse sentido, a técnica favorece a **construção de entendimento compartilhado**, ainda que seu foco imediato permaneça sendo a verificação.

Seu principal risco é confundir fluidez com profundidade. Um walkthrough muito rápido pode gerar a sensação de alinhamento sem de fato tensionar o requisito. Além disso, como a condução costuma permanecer com o autor, há o perigo de que as explicações orais “compensem”

fragilidades na declaração ou representação do requisito e ocultem a necessidade de ajustá-lo. Se um requisito só funciona quando o autor está presente para traduzi-lo, isso já é, em si, um sinal de baixa qualidade.

#### 11.5.4 Checklists

Checklists são instrumentos simples, mas particularmente eficazes na verificação de requisitos, porque transformam critérios gerais de qualidade em uma rotina prática de leitura crítica. Em vez de depender apenas da experiência individual de quem revisa, a equipe passa a contar com um **conjunto de itens que orienta a observação do requisito** e ajuda a tornar mais sistemática a identificação de ambiguidades, lacunas, inconsistências, conflitos e elementos insuficientemente explicitados. Seu valor, portanto, não está em formalizar uma conferência burocrática, mas em apoiar um **exame mais disciplinado do que foi declarado e representado**.

A força do checklist está em tornar repetível aquilo que muitas vezes permanece implícito. Em atividades de Engenharia de Requisitos, é comum que certos padrões de qualidade sejam reconhecidos intuitivamente por profissionais mais experientes, mas não estejam explicitados de forma compartilhável para o restante da equipe. O checklist ajuda justamente a converter esse repertório tácito em apoio concreto ao trabalho de verificação. Ao fazer isso, ele contribui para elevar a consistência das revisões e para reduzir a dependência de leituras puramente impressionistas (WIEGERS; BEATTY, 2013; YOUNG, 2004).

Um bom checklist tende a ser **curto, observável e contextualizado**. Curto, para que possa ser de fato utilizado; observável, para que seus itens não se reduzam a formulações genéricas; e contextualizado, porque requisitos funcionais, regras de negócio e requisitos não funcionais não exigem exatamente o mesmo tipo de checagem. Quando o checklist é excessivamente amplo, abstrato ou padronizado sem critério, ele perde poder analítico e se converte em simples marcação de conformidade.

Seu limite principal decorre exatamente daí: **checklist não substitui julgamento profissional**. Ele apoia o raciocínio, mas não pensa pela equipe. Quando tratado como formulário automático, produz a aparência de rigor sem necessariamente melhorar a qualidade do requisito. Por isso, seu uso mais produtivo ocorre quando ele serve como disparador de perguntas e de análise, e não como fim em si mesmo.

Dentre o conjunto de itens que podem ser utilizados para a construção de um bom checklist estão, **critérios de qualidade** (ver 11.3.2) e **critérios de aceitação** (ver 9.4.2.6), os quais podem ser apenas itens nominais ou serem transformados em perguntas.

No primeiro caso, os checklists traduzem qualidades abstratas em operações recorrentes de checagem utilizando perguntas como:

- O requisito está claro / compreensível?
- Não ambíguo (sem ambiguidade relevante)
- O requisito está completo o suficiente ao propósito
- O requisito é necessário (e orientado a valor)
- O requisito está viável?

Ao longo do tempo, equipes maduras internalizam parte dessas perguntas, elevando seu padrão de leitura crítica (WIEGERS; BEATTY, 2013; YOUNG, 2004).

Outra combinação possível é com critérios de aceitação. Nesse caso, o checklist ajuda a identificar o que precisa ser examinado, enquanto os critérios de aceitação ajudam a tornar explícito o que deverá ser observado para reconhecer que o requisito foi satisfatoriamente atendido. A relação entre ambos é importante porque a verificabilidade do requisito não depende apenas de sua boa declaração, mas também da possibilidade de explicitar evidências concretas de atendimento. Assim, o checklist funciona como mecanismo de triagem e análise, e os critérios de aceitação funcionam como mecanismo de explicitação da evidência.

**Exemplo:** RF01 - Registrar Agendamento

#### **Critérios de aceitação associados**

- O sistema registra o agendamento e apresenta o número de protocolo ao cidadão.
- O sistema informa quando o horário escolhido não está disponível.
- O sistema sugere horários alternativos quando não houver disponibilidade no horário selecionado.
- O sistema altera o status do agendamento para CONFIRMADO após a conclusão do registro.

#### **Checklist**

1. O fluxo principal do agendamento está coberto por critério de aceitação?
2. A indisponibilidade de horário está coberta por critério de aceitação?
3. Os critérios descrevem resultados observáveis no sistema?
4. Os critérios usam linguagem concreta e sem termos vagos?
5. Os critérios são coerentes com os demais requisitos relacionados?
6. Os critérios são suficientes para apoiar o aceite do requisito?

### **11.5.5 Uso de critérios de qualidade de requisitos como apoio à checagem**

Além da combinação com a técnica de checklist, os critérios de qualidade de requisitos também podem ser utilizados como base explícita para a leitura crítica. Essa abordagem ajuda a elevar o nível da checagem avaliando, por exemplo, se um requisito é:

- **Necessário:** obriga a explicitar a razão de negócio, o valor ou a restrição que o justifica.
- **Correto:** exige confrontá-lo com fontes autorizadas, regras do domínio e entendimento validado com stakeholders.
- **Não ambíguo:** força a equipe a olhar para termos vagos, expressões subjetivas e construções abertas a múltiplas interpretações.
- **Verificável:** exige imaginar que evidência futura poderia demonstrar seu atendimento.

Em todos esses casos, o critério de qualidade funciona como um mecanismo de depuração do requisito.

No entanto, também aqui há risco de superficialidade. Repetir atributos de qualidade sem traduzi-los em perguntas concretas pode não melhorar o requisito. Do mesmo modo, afirmar que

um requisito é “correto” ou “necessário” sem indicar a base dessa avaliação enfraquece a própria verificação. O ganho está na passagem do adjetivo para a evidência (ver Quadro 61). Em engenharia de requisitos pode-se dizer que: *“quanto mais adjetivos, pior é para um requisito”*, mais subjetivo ele tende a ficar, menos verificável.

| Requisito baseado em adjetivo                                    | Requisito baseado em evidência                                                                                                                                                                                                         |
|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| O sistema deve ser <b>rápido</b> .                               | O sistema deve apresentar ao cidadão as opções de horário disponíveis em <b>até 2 segundos</b> em <b>95% das consultas</b> realizadas em <b>horário de expediente</b> .                                                                |
| O sistema deve ter uma interface <b>amigável</b> para o cidadão. | O sistema deve permitir que o cidadão conclua o agendamento de uma consulta em <b>até 5 passos</b> , <b>sem necessidade de treinamento prévio</b> e com <b>mensagens</b> de erro compreensíveis <b>quando houver dados inválidos</b> . |

Quadro 61 – Requisito baseado em adjetivo para evidência.  
Fonte: elaborado pelo autor (2026).

### 11.5.6 Análise de consistência entre declarações e representações

Uma das contribuições mais importantes da representação para a verificação e a validação de requisitos é **tornar visíveis inconsistências** que podem permanecer ocultas quando a equipe trabalha apenas no nível da declaração. Ao representar um requisito, a equipe precisa explicitar elementos que, muitas vezes, ainda estavam apenas pressupostos: atores, estados, dados, transições, dependências, exceções e condições de execução. Esse movimento “força” escolhas e, ao fazê-lo, revela lacunas, contradições e ambiguidades que a formulação textual, sozinha, pode tolerar por mais tempo.

A análise de consistência entre declarações e representações consiste justamente em **confrontar aquilo que foi dito com aquilo que foi modelado, prototipado ou estruturado visualmente** em outros artefatos. Ela pode ocorrer, por exemplo, entre:

- A declaração e modelos semiformais, como UML, BPMN ou SysML;
- Requisitos e protótipos, quando estes já funcionam como expressão de comportamento esperado; ou ainda entre modelos distintos, como um fluxo de processo e um diagrama de estados.

Em todos esses casos, o foco da técnica não está em comparar artefatos apenas por semelhança formal, mas em verificar se eles sustentam o mesmo núcleo de sentido e se descrevem, de modo coerente, o comportamento, as restrições e as condições associadas ao requisito.

Essa técnica é particularmente útil porque desloca a verificação do plano exclusivamente textual para o plano da **articulação entre artefatos**. Muitas vezes, um requisito parece aceitável quando lido isoladamente, mas perde consistência quando comparado a um fluxo, a uma regra ou a um protótipo que explicita comportamentos não previstos. Nesse sentido, a análise de consistência contribui para fortalecer não apenas a qualidade da declaração, mas a coerência do sistema de requisitos como um todo.

Seu principal risco é supor que a simples existência de múltiplas representações já garanta qualidade. Modelos desatualizados, incompletos ou produzidos apenas para formalizar uma documentação podem gerar uma aparência de rigor sem efetivamente melhorar o entendimento. Há

também o perigo de tratar a consistência como mera coincidência sintática entre artefatos, sem examinar se eles realmente convergem quanto ao significado. A representação contribui para a verificação quando acrescenta precisão ao que foi declarado, explicita relações e torna visíveis lacunas ou inconsistências. Quando apenas repete o texto em outro formato, esse ganho tende a ser pequeno.

### 11.5.7 Análise de rastreabilidade e cobertura

A rastreabilidade pode ser compreendida como uma forma de preservar a memória de sentido dos requisitos. Ela permite que a equipe reconheça de onde um requisito veio, a que objetivos, cenários, regras de negócio e decisões ele se vincula, e por quais caminhos ele pode ser acompanhado ao longo de sua evolução. No trabalho de verificação e validação, essa memória não tem função apenas organizacional: ela se torna uma técnica de análise, porque ajuda a examinar se o requisito está suficientemente conectado às suas justificativas, aos artefatos que o representam e às evidências que deverão sustentar sua aceitação.

A análise de rastreabilidade e cobertura opera, assim, em pelo menos dois sentidos complementares:

- **Cobertura:** perguntar se todo requisito relevante possui origem identificável, justificativa explícita e relação com objetivos, cenários ou necessidades reconhecidas.
- **Preparação de evidência:** perguntar se todo requisito que se pretende verificável ou validável possui vínculos com critérios de aceitação, inspeções, revisões, testes ou outras formas de checagem que permitam observar seu atendimento.

Nesse sentido, rastrear não é apenas “ligar artefatos”, mas explicitar como um requisito se sustenta no sistema de requisitos e como poderá ser examinado ao longo do projeto.

A utilidade dessa técnica aumenta quando a equipe trabalha com diferentes níveis de abstração e com múltiplas representações.

Seu principal risco é transformar a rastreabilidade em exercício excessivamente burocrático, no qual se acumulam vínculos formais sem utilidade analítica real. Rastrear tudo indiscriminadamente pode gerar custo elevado e pouca aprendizagem, sobretudo quando as relações estabelecidas não ajudam a compreender impacto, cobertura ou evidência. Como técnica de V&V, a análise de rastreabilidade é útil quando os vínculos existentes permitem formular perguntas relevantes sobre origem, cobertura, coerência e evidências associadas aos requisitos.

### 11.5.8 Definição de Preparado (Definition of Ready - DoR)

Em contextos ágeis e híbridos, a Definition of Ready pode funcionar como apoio leve à verificação, desde que seja compreendida com a medida certa. Seu valor não está em criar um novo método nem em transformar a equipe em cumpridora de rituais, mas em explicitar uma pergunta prática e relevante para o fluxo do trabalho: este item já alcançou maturidade mínima para entrar em refinamento mais profundo, planejamento ou implementação sem arrastar consigo ambiguidades, lacunas e decisões implícitas demais? Vista dessa forma, a DoR cria condições para que a verificação aconteça de modo mais consistente e menos tardio.

Em muitos casos, essa maturidade mínima envolve elementos como critérios de aceite iniciais, dependências relevantes visíveis, termos críticos suficientemente definidos e risco principal explicitado. O efeito positivo dessa prática é simples, mas importante: ela ajuda a equipe a perceber que nem todo item “priorizado” está, de fato, preparado para sustentar estimativa, implementação ou aceite. Em vez de empurrar os requisitos (ainda frágeis) para frente, a equipe ganha um ponto de parada para perguntar se há entendimento suficiente para avançar com responsabilidade.

O principal equívoco, contudo, é tratar a DoR como selo automático de qualidade. Um item pode atender a uma lista local de “preparado” e, ainda assim, continuar ambíguo, inconsistente, mal delimitado ou desconectado de seus objetivos e critérios de evidência.

### 11.5.9 DEEP e INVEST como checklists de verificação em contextos ágeis e híbridos

Em contextos ágeis e híbridos, a qualidade dos requisitos frequentemente se manifesta na qualidade do backlog e dos itens que o compõem. Nesses ambientes, requisitos podem aparecer como épicos, funcionalidades, histórias de usuário, critérios de aceite, tarefas de refinamento, regras de negócio associadas ou itens técnicos necessários à evolução do produto. Por isso, além dos critérios gerais de qualidade de requisitos, algumas heurísticas originadas no contexto ágil podem apoiar a verificação.

Duas delas são particularmente úteis: DEEP, voltada à qualidade do backlog como conjunto, e INVEST, voltada principalmente à qualidade de histórias de usuário ou itens de backlog. Embora tenham surgido no contexto ágil, ambas podem ser usadas como checklists leves de verificação em ambientes nos quais requisitos sejam organizados de forma incremental, priorizada e evolutiva (PICHLER, 2010; LEFFINGWELL, 2011).

O acrônimo DEEP indica que um backlog de produto deve ser (PICHLER, 2010):

- Detalhado apropriadamente: itens mais próximos de implementação devem estar mais claros, refinados e acompanhados de informações suficientes; itens mais distantes podem permanecer em nível mais alto de abstração.
- Emergente: o backlog deve evoluir conforme novas necessidades, restrições, riscos e aprendizados surgem.
- Estimado: os itens devem permitir alguma avaliação de tamanho, esforço, complexidade ou incerteza, ainda que a estimativa seja progressivamente refinada.
- Priorizado: a ordem dos itens deve refletir valor, risco, urgência, dependências, oportunidades e restrições relevantes.

Como instrumento de verificação, DEEP não pergunta se cada item isolado está perfeito; pergunta se o backlog, como sistema de requisitos em evolução, está suficientemente organizado para orientar decisões. Um backlog pode conter boas histórias individuais e, ainda assim, ser ruim como conjunto: excesso de itens sem prioridade, baixa visibilidade de dependências, ausência de estimativas mínimas, épicos nunca refinados ou itens antigos que já não representam o entendimento atual.

O Quadro 62 apresenta perguntas de verificação baseadas em DEEP.

| Critério DEEP             | Pergunta de verificação                                                                                            | Evidência possível                                                                             | Sinal de alerta                                                                  |
|---------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Detalhado apropriadamente | Os itens mais próximos de desenvolvimento possuem informação suficiente para entendimento, estimativa e discussão? | Histórias refinadas, critérios de aceite, regras associadas, exemplos, dependências explícitas | Itens prioritários permanecem vagos, genéricos ou sem critérios mínimos          |
| Emergente                 | O backlog está sendo atualizado a partir de aprendizagem, feedback, riscos e mudanças de contexto?                 | Inclusão, remoção, divisão, reordenação ou reformulação de itens                               | Backlog tratado como lista fixa, acumulando itens obsoletos                      |
| Estimado                  | Os itens permitem avaliação razoável de esforço, complexidade ou incerteza?                                        | Estimativas, classificação de tamanho, marcação de incerteza ou necessidade de investigação    | Itens grandes ou obscuros avançam sem compreensão mínima de esforço              |
| Priorizado                | A ordem dos itens reflete valor, risco, dependência e urgência?                                                    | Priorização explícita, justificativa de ordenação, vínculo com objetivos                       | Itens aparecem ordenados por conveniência, pressão local ou sequência de chegada |

Quadro 62 – DEEP como checklist de verificação da qualidade do backlog.

Fonte: elaborado pelo autor (2026).

Já o acrônimo INVEST ajuda a verificar a qualidade de histórias de usuário ou itens de backlog em nível mais próximo da implementação. Uma boa história tende a ser (LEFFINGWELL, 2011):

- Independente: pode ser compreendida, planejada, desenvolvida e testada com dependências controladas.
- Negociável: não deve ser um contrato fechado de solução; deve preservar espaço para conversa e ajuste.
- Valiosa: deve expressar valor para usuário, cliente, organização ou outro stakeholder relevante.
- Estimável: deve permitir avaliação razoável de esforço, complexidade ou incerteza.
- Pequena: deve ter tamanho compatível com planejamento, implementação, validação e feedback em ciclo curto.
- Testável: deve permitir definição de critérios de aceite, exemplos, observações ou evidências de atendimento.

INVEST é especialmente útil quando histórias de usuário são usadas como técnica de declaração de requisitos. A história “Como cidadão, quero acompanhar meu agendamento, para ter mais controle sobre meu atendimento” pode estar correta em intenção, mas ainda ser fraca para orientar desenvolvimento se não explicitar quais informações serão acompanhadas, em que momento, por qual canal, com que restrições e com quais critérios de aceite. A verificação por INVEST ajuda a revelar esse tipo de lacuna antes que a equipe transforme uma narrativa promissora em trabalho mal compreendido.

O Quadro 63 apresenta perguntas de verificação baseadas em INVEST.



| Critério INVEST | Pergunta de verificação                                                       | Evidência possível                                                              | Sinal de alerta                                                                        |
|-----------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Independente    | A história pode ser entendida e tratada com dependências controladas?         | Dependências registradas, divisão de histórias, sequência de entrega clara      | História depende de muitas outras para produzir qualquer valor observável              |
| Negociável      | A história deixa espaço para conversa sobre solução, escopo e alternativas?   | Observações, hipóteses, critérios negociados com stakeholders                   | História escrita como especificação rígida de interface ou decisão técnica prematura   |
| Valiosa         | O benefício está explícito e reconhecível?                                    | Benefício declarado, vínculo com objetivo, justificativa de prioridade          | História descreve tarefa técnica sem conexão clara com valor ou necessidade            |
| Estimável       | A equipe consegue estimar ou identificar o que falta para estimar?            | Estimativa, spike, dúvida registrada, critérios mínimos                         | História ampla ou ambígua demais para qualquer avaliação razoável                      |
| Pequena         | A história cabe em um ciclo de trabalho e permite feedback útil?              | Divisão em histórias menores, fatiamento por fluxo, regra ou cenário            | História representa um épico disfarçado                                                |
| Testável        | É possível demonstrar atendimento por critério, cenário, observação ou teste? | Critérios de aceite, exemplos, cenários Dado-Quando-Então, evidências esperadas | História contém termos como “fácil”, “rápido” ou “intuitivo” sem referência observável |

Quadro 63 – INVEST como checklist de verificação da qualidade de histórias de usuário.  
Fonte: elaborado pelo autor (2026).

DEEP e INVEST não substituem validação com stakeholders. Um backlog pode ser DEEP e uma história pode atender a INVEST, mas ainda assim representar uma necessidade pouco relevante, uma prioridade equivocada ou uma interpretação inadequada do problema. Portanto, essas heurísticas devem ser compreendidas como instrumentos de verificação da qualidade dos itens e da estrutura de trabalho, não como prova de que o produto desejado é o produto certo.

No caso condutor do agendamento de consultas, por exemplo, DEEP ajuda a verificar se o backlog distingue adequadamente itens próximos, como “confirmar agendamento por mensagem”, de itens mais amplos, como “gerenciar jornada digital do paciente”. INVEST ajuda a verificar se uma história específica, como “Como paciente, quero receber confirmação do agendamento em até 2 minutos, para ter segurança de que minha consulta foi registrada”, possui valor explícito, tamanho adequado e critério de verificação observável. A validação, porém, ainda exigirá confrontar essa história com pacientes, atendentes, profissionais de saúde e regras institucionais, verificando se o comportamento proposto é realmente desejável, viável e suficiente.

### 11.5.10 IA generativa para a verificação

Modelos de IA generativa também podem apoiar a verificação de requisitos, mas seu uso precisa permanecer claramente subordinado à atividade humana de Engenharia de Requisitos. Empregados com senso crítico, esses modelos podem ajudar a acelerar etapas preliminares de revisão: sugerir reformulações mais observáveis ou mensuráveis, apontar ambiguidades lexicais, comparar versões de artefatos, levantar perguntas para revisão, identificar inconsistências superficiais e até apoiar a montagem inicial de checklists ou roteiros de walkthrough. Em equipes

pressionadas por volume de informação, esse tipo de apoio pode ser útil para ampliar a capacidade de leitura crítica e para antecipar problemas que, de outro modo, só seriam percebidos mais tarde.

Seu valor, no entanto, está muito mais em ampliar a preparação e a exploração inicial do que em “validar” ou “garantir” a qualidade do requisito. Modelos de IA tendem a não conhecer o domínio como os stakeholders o conhecem, não respondem pelas consequências das decisões tomadas e não substituem o trabalho de mediação entre perspectivas distintas. Podem oferecer sugestões plausíveis, mas também podem introduzir erros, perder nuances contextuais, simplificar indevidamente conflitos reais, reforçar vieses e produzir uma perigosa falsa sensação de completude. Em Engenharia de Requisitos, esse último risco é particularmente sério: uma declaração reescrita de forma elegante por IA pode parecer melhor do que realmente é, mascarando lacunas importantes sob aparência de clareza.

Por isso, o uso de IA generativa na verificação deve ser entendido como apoio auxiliar e criticamente supervisionado. Ela pode acelerar a crítica preliminar, enriquecer perguntas e tornar mais eficiente a preparação de sessões de revisão; não pode, porém, substituir análise humana, conhecimento de domínio, confronto entre artefatos, diálogo com stakeholders nem responsabilidade sobre a decisão final. Quando bem utilizada, ajuda a equipe a ver melhor; quando usada de modo ingênuo, pode fazer a equipe acreditar que já viu o suficiente.

## 11.6 Verificação e validação por tipo de requisito e forma de declaração

A atividade de V&V ganha precisão quando considera a natureza do requisito e a forma como ele foi declarado. Requisitos funcionais, requisitos não funcionais e regras de negócio não apresentam os mesmos riscos, nem exigem os mesmos tipos de evidência. Da mesma forma, narrativas descritivas, declarações estruturadas, declarações orientadas a valor e catálogos técnicos tornam visíveis aspectos diferentes do requisito. Por isso, verificar e validar bem exige ajustar perguntas, critérios e técnicas ao tipo de conteúdo em análise.

### 11.6.1 Verificação e validação de RFs, RNFs e RNs

Requisitos funcionais, requisitos não funcionais e regras de negócio devem ser verificados e validados de forma complementar. O Quadro 64 apresenta uma maneira de se fazer isso.

| Tipo de requisito             | O que verificar                                                                                                                | O que validar                                                                                                      | Evidências possíveis                                                                                   |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Requisito funcional (RF)      | Ator, gatilho, entradas, processamento esperado, saídas, exceções, regras aplicáveis, consistência com fluxos e representações | Se o comportamento descrito atende a uma necessidade real, apoia o trabalho do usuário e produz valor reconhecível | Cenários, casos de uso, histórias, critérios de aceite, walkthroughs, protótipos, feedback de usuários |
| Requisito não funcional (RNF) | Atributo de qualidade, métrica, condição de aplicação, limite, unidade de medida, forma de observação ou medição               | Se o nível exigido é necessário, suficiente, viável e adequado ao contexto de uso, risco e custo                   | Metas mensuráveis, simulações, benchmarks, protótipos, critérios de aceitação, análise de trade-offs   |

| Tipo de requisito     | O que verificar                                                                                       | O que validar                                                                                             | Evidências possíveis                                                                                  |
|-----------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Regra de negócio (RN) | Fonte, autoridade, escopo, exceções, conflitos com outras regras, atualidade e relação com RFs e RNFs | Se a regra está correta, vigente, legitimada e interpretada adequadamente pelos responsáveis pelo domínio | Confirmação com especialistas, referência normativa, glossário, matriz de regras, cenários de exceção |

Quadro 64 – V&V por tipo de requisito.  
Fonte: elaborado pelo autor (2026).

Em **requisitos funcionais**, a **verificação** tende a se concentrar na descrição do comportamento esperado. Perguntas como “quem inicia?”, “em que condição?”, “com quais dados?”, “qual resultado deve ocorrer?” e “que exceções precisam ser tratadas?” ajudam a revelar lacunas. A **validação**, por sua vez, pergunta se esse comportamento resolve efetivamente o problema, melhora o trabalho do usuário ou atende a uma necessidade reconhecida.

Em **requisitos não funcionais**, a **verificação** exige atenção a atributos de qualidade. Expressões como “rápido”, “seguro”, “intuitivo”, “escalável” ou “disponível” precisam ser transformadas em condições observáveis, mensuráveis ou avaliáveis. A **validação**, porém, não se limita a confirmar a métrica; ela pergunta se aquele nível de desempenho, segurança, usabilidade, disponibilidade ou confiabilidade é adequado ao contexto, ao risco e ao custo.

Em **regras de negócio**, a **verificação** examina fonte, consistência, exceções e impacto sobre outros requisitos. Uma regra pode estar bem escrita, mas desatualizada; pode ser válida para um setor, mas não para outro; pode parecer simples, mas gerar exceções críticas. A **validação** deve envolver pessoas com expertise no domínio, capazes de confirmar se a regra está vigente, se foi interpretada corretamente e se suas exceções foram consideradas.

### 11.6.2 Verificação e validação conforme a forma de declaração

Além do tipo de requisito, a forma de declaração também pode influenciar a V&V. Como discutido no Capítulo 9, requisitos podem ser declarados por narrativas, estruturas, orientações a valor ou catálogos técnicos. Cada forma favorece certos tipos de análise e oculta outros. O Quadro 65 relaciona essas formas aos focos de verificação e validação e aos riscos que merecem atenção em cada caso.

| Forma de declaração      | Foco da verificação                                                                                     | Foco da validação                                                                 | Risco típico                                                               |
|--------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Narrativas descritivas   | Clareza, termos ambíguos, sequência lógica, ator, objetivo e contexto                                   | Se a narrativa representa uma situação reconhecível e relevante para stakeholders | Texto compreensível, mas amplo demais ou interpretável de modos diferentes |
| Declarações estruturadas | Preenchimento dos campos, consistência entre objetivo, condição, resposta, exceção e critério de aceite | Se a estrutura representa adequadamente o trabalho real e o valor esperado        | Estrutura formalmente correta, mas artificial ou distante do uso           |

| Forma de declaração            | Foco da verificação                                                                        | Foco da validação                                                                       | Risco típico                                                     |
|--------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Declarações orientadas a valor | Vínculo entre requisito, benefício, objetivo e stakeholder afetado                         | Se o valor declarado é legítimo, prioritário e coerente com a estratégia ou necessidade | Benefício genérico, retórico ou difícil de demonstrar            |
| Catálogos e artefatos técnicos | Identificador, origem, tipo, prioridade, dependências, status, rastreabilidade e critérios | Se a organização dos itens permite decisão, comunicação e evolução adequada             | Catálogo completo, mas burocrático, desatualizado ou pouco usado |

Quadro 65 – V&V conforme a técnica de declaração do requisito.  
Fonte: elaborado pelo autor (2026).

Essa distinção evita uma avaliação superficial da qualidade. Uma narrativa pode ser excelente para iniciar entendimento, mas insuficiente para orientar implementação. Uma história de usuário pode comunicar valor, mas ainda carecer de critérios de aceite. Um catálogo pode ser rastreável, mas esconder requisitos ruins sob uma aparência de controle. Uma declaração estruturada pode preencher todos os campos e, ainda assim, representar uma necessidade equivocada.

Portanto, a equipe de engenharia de software deve evitar a pergunta genérica “o requisito está bom?”. A pergunta mais adequada é: “considerando o tipo de requisito, sua forma de declaração, seu momento de maturidade e seu uso pretendido, que verificação e que validação são necessárias agora?”.

## 11.7 Técnicas de validação

Se a verificação examina a qualidade interna do sistema de requisitos, a validação pergunta se esse sistema expressa de fato necessidades, expectativas, restrições e valores reconhecidos pelos stakeholders como pertinentes ao contexto. Validar requisitos, portanto, não é apenas revisar o que foi declarado e/ou representado; é confrontar o entendimento produzido com o mundo de uso, de trabalho, de decisão e de experiência no qual o software precisará fazer sentido. Isso torna a validação inseparável da dimensão humana da Engenharia de Requisitos: escuta, mediação, entendimento compartilhado e feedback deixam de ser acessórios e passam a compor o próprio núcleo da atividade (WIEGERS; BEATTY, 2013; POHL, 2010).

### 11.7.1 Feedback do cliente

O feedback do cliente é **uma das formas mais diretas de validação** porque recoloca o requisito diante da perspectiva de quem reconhece — ou não reconhece — valor naquilo que está sendo proposto. O material de apoio enfatiza que o cliente deve ser constantemente envolvido para oferecer retornos sobre o que está sendo construído ou formulado. No âmbito deste capítulo, esse princípio precisa ser compreendido menos como ritual de acompanhamento e mais como **mecanismo de confrontação do requisito com a necessidade real**: o requisito, tal como formulado, representa algo que o cliente considera necessário, adequado e útil?

Esse tipo de validação é especialmente importante quando há risco de a equipe técnica produzir um requisito coerente do ponto de vista interno, mas desalinhado do problema efetivo. O feedback do cliente ajuda a revelar expectativas não explicitadas, prioridades mal percebidas e interpretações que tecnicamente parecem aceitáveis, mas que falham do ponto de vista do uso ou do

valor. Mais do que um “sim” ou “não”, o que interessa aqui é a capacidade de o feedback **deslocar a equipe de suas pressuposições e obrigá-la a reconectar o requisito ao propósito**.

Seu limite está no fato de que o cliente nem sempre enxerga sozinho todas as implicações do requisito, tampouco fala a partir de uma posição neutra. Em certos contextos, ele valida a necessidade, mas não percebe conflitos operacionais; em outros, oferece preferências de solução como se fossem requisitos. Por isso, o feedback do cliente precisa ser acolhido criticamente, sem ser reduzido nem a autoridade absoluta nem a simples opinião entre outras.

### 11.7.2 Feedback da equipe

O feedback da equipe também é técnica de validação, embora por vezes seja subestimado por parecer “interno demais”. Na prática, a equipe de desenvolvimento, análise, qualidade ou operação frequentemente percebe incoerências entre o requisito formulado e a realidade de sua futura aplicação antes mesmo que isso se materialize no produto. Validar requisitos não é tarefa reservada apenas ao cliente final, mas atividade que pode — e deve — incorporar a experiência interpretativa e técnica da equipe (WIEGERS; BEATTY, 2013).

O valor dessa técnica está no fato de que a equipe vê o requisito a partir do esforço necessário para torná-lo realizável, coerente e sustentável. Isso não significa deslocar a validação para o terreno da solução, mas reconhecer que há inadequações que se tornam visíveis justamente quando alguém tenta imaginar como o requisito se comportará na prática. Em muitos casos, o feedback da equipe mostra que um requisito aparentemente desejável está incompleto, contraditório ou desancorado do contexto operacional.

Seu principal risco é converter limitações circunstanciais da equipe em suposta invalidação do requisito. Um requisito não deve ser considerado inválido apenas porque é incômodo, exige trabalho ou desafia hábitos estabelecidos. O feedback da equipe ganha valor de validação quando ilumina viabilidade contextual, coerência operacional e impacto sobre o sistema de requisitos, não quando opera apenas como resistência.

### 11.7.3 Uso de critérios de aceite

CrITÉRIOS de aceite constituem uma das pontes mais produtivas entre requisito e validação, porque **transformam formulações ainda abstratas em situações observáveis**. Quando stakeholders e equipe conseguem concordar sobre as condições sob as quais um requisito será reconhecido como atendido, estão, ao mesmo tempo, validando uma interpretação desse requisito. Por isso, critérios de aceite não devem ser vistos apenas como insumo para atividades posteriores de aceitação do produto, mas como recurso antecipado de validação do próprio requisito (WIEGERS; BEATTY, 2013).

Sua força está em obrigar a explicitação do que antes estava implícito. Quando se formula um critério de aceite, a equipe precisa dizer **em que contexto algo ocorre, qual ação é realizada e que resultado observável deve surgir**. Esse movimento é valioso porque, muitas vezes, a própria tentativa de construir o critério revela que o requisito ainda está vago, incompleto ou mal delimitado. Nesse sentido, critérios de aceite ajudam tanto a validar quanto a refinar.

Seu limite aparece quando são tratados como substitutos do requisito ou como roteiro exaustivo

do comportamento do sistema. Critérios de aceite iluminam o requisito, mas não esgotam sua semântica nem dispensam o trabalho de entendimento compartilhado. Além disso, um conjunto excessivo de critérios pode criar falsa sensação de completude, como se tudo o que não estivesse explicitamente escrito deixasse de importar.

#### **11.7.4 Validação colaborativa com stakeholders: oficinas e sessões de validação**

As oficinas e sessões colaborativas de validação são, por isso, instrumentos especialmente eficazes. Quando bem facilitadas, permitem comparar expectativas divergentes, negociar prioridades, refinar critérios de aceite e observar a linguagem efetivamente usada pelos participantes do domínio. Mais do que obter “opiniões”, a oficina de validação ajuda a construir reconhecimento compartilhado sobre o que o sistema deverá fazer, sob quais condições e com quais consequências. Seu maior risco está nas assimetrias de voz: stakeholders com maior poder hierárquico, maior repertório técnico ou maior desenvoltura comunicacional podem impor definições prematuras e produzir consensos apenas aparentes. Validar colaborativamente exige, portanto, facilitação legítima e atenção explícita às relações de poder que atravessam o processo (GOTTESDIENER; GORMAN, 2012; ROBERTSON; ROBERTSON, 2014).

#### **11.7.5 Cenários, histórias e casos de uso como instrumentos de validação**

Cenários, histórias e casos de uso são instrumentos privilegiados de validação exatamente porque deslocam o requisito do plano abstrato para situações reconhecíveis de uso, trabalho e decisão. Embora sejam, primariamente, técnicas de declaração, eles se tornam poderosos recursos de validação quando ajudam stakeholders a perguntar: é assim que isto deveria acontecer? O que ocorre quando o fluxo principal falha? Que exceções importam? Que resultado o usuário considera aceitável? Que informação precisa ficar visível? Que efeito colateral seria inadmissível? O valor do cenário está em tornar o requisito experienciável; o valor da história está em conectar comportamento e propósito; o valor do caso de uso está em organizar interações e responsabilidades. Nenhum deles substitui o requisito; todos ajudam a validar se o requisito faz sentido (WIEGERS; BEATTY, 2013; ROBERTSON; ROBERTSON, 2014).

#### **11.7.6 Protótipos, simulações e experimentos como validação**

Protótipos, simulações e experimentos, quando empregados na validação, cumprem função semelhante, mas com materialidade ainda maior. Eles tornam a conversa menos abstrata ao oferecer uma forma visível, navegável ou encenável de experimentar decisões de requisito. Isso é especialmente importante quando a dificuldade não está apenas em descrever a funcionalidade, mas em compreender fluxo de interação, compreensão do usuário, esforço operacional, ordenação de informações, efeitos temporais ou percepção de desempenho. Mesmo protótipos simples — sketches, mockups, fluxos clicáveis, simulações parciais — podem produzir validações fortes porque evidenciam rapidamente o que o texto ainda não estabilizou. Seu limite é igualmente conhecido: prototipar não pode significar validar apenas a aparência superficial da interface e esquecer regras de negócio, critérios de qualidade, exceções, auditoria, segurança ou impactos organizacionais (LAUESEN, 2002; WIEGERS; BEATTY, 2013).

### 11.7.7 IA Generativa para validação

A IA generativa também pode apoiar a validação, sobretudo na preparação de sessões: geração de perguntas de contraste, variações de cenário, versões alternativas de microtextos, síntese preliminar de feedback coletado e identificação de pontos potencialmente controversos. Mas ela não pode validar em lugar dos stakeholders, porque não experimenta o contexto, não assume responsabilidade sobre o valor e não responde pelas consequências sociotécnicas da solução. Sua contribuição é preparatória e exploratória; a validação continua sendo atividade humana, situada e socialmente negociada (ZHAO; XU; YANG, 2022; CROWDER; HOFF, 2022).

### 11.7.8 Síntese comparativa das técnicas de validação

As técnicas de validação discutidas nesta seção não competem entre si. Cada uma aproxima o requisito do mundo de uso e decisão por uma via diferente: feedback direto, colaboração, critérios de aceite, cenários, histórias, casos de uso, protótipos, simulações, experimentos ou apoio de IA generativa. A escolha deve considerar o tipo de requisito, o perfil dos stakeholders, o custo do erro, o grau de incerteza e o tipo de evidência necessário.

O Quadro 66 sintetiza possibilidades de uso, reforçando uma ideia central: validação não é um evento único de aprovação. Ela é um conjunto de práticas de confrontação do requisito com o mundo que ele pretende representar e transformar. Quanto mais complexo, crítico ou controverso for o requisito, maior deve ser a diversidade de perspectivas envolvidas e mais explícitas devem ser as evidências produzidas.

| Técnica de validação               | Uso adequado                                                                                   | Participantes principais                                     | Evidências produzidas                                      | Riscos                                                             | Exemplo no caso condutor                                                                                                        |
|------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Feedback do cliente                | Confirmar pertinência, prioridade e adequação de requisitos já formulados                      | Cliente, Product Owner, analista, representantes de usuários | Comentários, aceite, rejeição, ajustes solicitados         | Cliente validar superficialmente, por pressão ou autoridade        | Cliente confirma se a notificação de agendamento reduz dúvidas reais dos pacientes                                              |
| Feedback da equipe                 | Avaliar viabilidade, clareza, dependências e riscos percebidos pela equipe técnica             | Desenvolvedores, testadores, arquitetos, analistas           | Dúvidas técnicas, riscos, alternativas, dependências       | Equipe deslocar validação para preferências de solução             | Equipe identifica que confirmação por SMS depende de integração externa ainda não contratada                                    |
| CrITÉRIOS de aceite                | Tornar explícitas condições de satisfação e alinhamento mínimo antes da implementação          | PO, analista, equipe, stakeholders relevantes                | CrITÉRIOS aprovados, exemplos, cenários de aceite          | CrITÉRIOS virarem testes técnicos sem validação de valor           | “Dado que a consulta foi confirmada, quando o paciente concluir o agendamento, então deve receber confirmação em até 2 minutos” |
| Oficinas e sessões colaborativas   | Construir validação conjunta quando há múltiplas perspectivas ou conflitos                     | Stakeholders, equipe, facilitador, especialistas de domínio  | Decisões, conflitos resolvidos, prioridades, ajustes       | Consenso aparente, domínio de voz por stakeholders mais influentes | Oficina entre recepção, médicos e pacientes para validar regras de reagendamento                                                |
| Cenários, histórias e casos de uso | Validar comportamento em situações concretas de uso                                            | Usuários, analistas, equipe, testadores                      | Fluxos validados, exceções, lacunas, casos alternativos    | Confundir técnica de declaração com validação suficiente           | Caso de uso revela que paciente sem acesso à internet precisa alternativa de confirmação                                        |
| Protótipos                         | Validar entendimento, interação, fluxo, linguagem e experiência antes da construção definitiva | Usuários, UX, equipe, analistas, stakeholders                | Feedback sobre interface, fluxo, compreensão e usabilidade | Stakeholders confundirem protótipo com produto pronto              | Protótipo da tela de agendamento revela que usuários não entendem o status “pré-confirmado”                                     |



| Técnica de validação      | Uso adequado                                                                                         | Participantes principais                                            | Evidências produzidas                                              | Riscos                                          | Exemplo no caso condutor                                                              |
|---------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------------|
| Simulações e experimentos | Validar comportamento em condições controladas ou críticas                                           | Equipe, especialistas, usuários representativos, áreas operacionais | Medições, observações, resultados experimentais                    | Simulação simplificar demais o contexto real    | Simulação mostra gargalo quando muitos pacientes tentam reagendar no mesmo horário    |
| IA generativa como apoio  | Preparar perguntas, comparar interpretações, gerar cenários alternativos e apoiar revisão preliminar | Engenheiro de requisitos, equipe, eventualmente facilitador         | Lista de perguntas, hipóteses, cenários, inconsistências sugeridas | Alucinação, falsa completude, perda de contexto | IA sugere perguntas sobre exceções no agendamento, mas equipe valida com stakeholders |

Quadro 66 – Síntese comparativa de técnicas de validação.  
Fonte: elaborado pelo autor (2026)

## 11.8 Aplicação integrada das técnicas ao caso condutor do livro

O caso do **Sistema de Agendamento de Consultas Públicas** permite mostrar como a verificação e a validação se complementam quando tratadas como atividade contínua de qualidade da ER. Ao chegar ao Capítulo 11, a equipe já não parte do zero: o problema foi contextualizado por meio de representações sistêmicas, os stakeholders relevantes foram mapeados, o fluxo de uso começou a ser visualizado em sketches, storyboards, mockups e protótipos, e parte do comportamento já havia sido modelada em representações mais estruturadas. A questão, agora, não é apenas “capturar novos requisitos”, mas verificar e validar com mais rigor o que já foi construído como sistema de entendimento.

O primeiro movimento foi de **verificação**. Em uma revisão em pares progressiva, a equipe analisou os requisitos RF01 – Registrar Agendamento, RF02 – Enviar Notificação, RF03 – Consultar Histórico e RNF02 – Tempo máximo de resposta. A leitura crítica mostrou que RF02 ainda estava subespecificado: ele não deixava claro em quais eventos a notificação deveria ocorrer, se haveria distinção entre confirmação, cancelamento, remarcação e lembrete, nem qual seria a relação entre o requisito e a escolha de canal pelo cidadão. RF03, por sua vez, apresentava uma lacuna clássica de auditabilidade: dizia que o histórico deveria ser consultável, mas não explicitava quais dados mínimos precisariam ser registrados em caso de cancelamento. Já RNF02 parecia preciso à primeira vista, mas ainda não indicava contexto de medição suficiente para produzir evidência confiável.

Em seguida, a equipe realizou um **walkthrough** do fluxo de agendamento, apoiado pelo protótipo de baixa fidelidade e pela representação de estados do processo. Foi nesse momento que surgiu uma divergência importante de interpretação: o termo “pendente” vinha sendo entendido por parte da equipe como estado intermediário já praticamente concluído, enquanto alguns participantes o interpretavam como agendamento já efetivado. A verificação, aqui, não revelou apenas um problema lexical; revelou um risco semântico com impacto direto sobre comportamento esperado, comunicação ao cidadão e critérios futuros de aceite.

O passo seguinte foi comparar **declarações e representações**. Ao cruzar o texto dos requisitos com o fluxo de interação, o estado do agendamento e o protótipo de interface, a equipe percebeu que o comportamento previsto para cancelamento próximo ao horário da consulta não estava adequadamente estabilizado. Havia possibilidade de colisão entre confirmação, cancelamento e envio de notificação, além de insuficiente clareza sobre o histórico de eventos. O problema não estava apenas “no texto” nem apenas “no modelo”: estava na coerência do sistema de requisitos como um todo.

Com essa base, a equipe partiu para a **validação colaborativa**. Foi conduzida uma oficina com representante do cidadão, profissionais de atendimento, gestor público e equipe técnica. Em vez de discutir requisitos de forma isolada, a sessão foi organizada em torno de um cenário concreto: um cidadão agenda consulta, escolhe canal de contato, recebe confirmação, precisa cancelar perto da data e o gestor posteriormente precisa auditar o motivo e o volume de cancelamentos. A oficina produziu três validações decisivas. Primeiro, confirmou que a escolha de canal

preferencial de notificação era necessidade real do serviço. Segundo, evidenciou que o lembrete 24 horas antes da consulta tinha valor público direto, por reduzir faltas e melhorar aproveitamento da agenda. Terceiro, deixou claro que o histórico precisaria registrar motivo, horário e origem do cancelamento, pois sem isso a transparência operacional ficaria comprometida.

Na sequência, a equipe retomou o **protótipo** já ajustado e realizou sessões rápidas de validação com cidadãos e profissionais de saúde. O objetivo, nesse momento, não era apenas revisar aparência, mas testar compreensão e fluxo. O principal achado foi a inadequação do microtexto associado ao estado “pendente”. A alternativa “aguardando confirmação” foi melhor compreendida, especialmente quando acompanhada de indicação explícita de prazo e consequência temporal. Esse tipo de ajuste mostra como a validação pode atuar sobre elementos aparentemente pequenos, mas que afetam diretamente entendimento compartilhado, comportamento do usuário e qualidade do requisito declarado.

Com base nesses achados, a equipe reformulou os **critérios de aceite**. RF01 passou a incluir, entre outros pontos, a geração de número de protocolo, o tratamento explícito da indisponibilidade de horário e a sugestão de alternativas. RF02 foi refinado para contemplar confirmação, cancelamento e lembrete, associados ao canal preferencial escolhido pelo cidadão. RF03 passou a exigir dados auditáveis de histórico. RNF02 foi reescrito para explicitar tempo máximo de resposta sob condições determinadas de operação. Nesse momento, critérios de aceite atuaram como ponte entre validação antecipada e futura evidência de atendimento, sem deslocar o eixo do trabalho para a disciplina de testes.

Por fim, para um pequeno conjunto de regras mais críticas, a equipe recorreu à **formalização seletiva**. Não se tratou de formalizar o capítulo inteiro, nem o sistema inteiro, mas de reforçar verificabilidade em pontos de maior risco: impossibilidade de estados contraditórios, vedação de confirmação posterior ao cancelamento e integridade do histórico. Essas propriedades foram mantidas lado a lado com a linguagem natural e com os modelos já compreendidos pelos participantes, preservando rastreabilidade e comunicabilidade. O efeito dessa escolha foi o esperado: não “substituiu” a ER por notação formal, mas tornou mais sólido um núcleo específico do sistema de requisitos.

Ao final do processo, o ganho principal não foi apenas a correção de frases ou a produção de mais artefatos. O ganho foi a redução qualificada de incerteza. Declarações foram refinadas, representações foram ajustadas, o glossário amadureceu, critérios de aceite ganharam observabilidade e alguns vínculos rastreáveis relevantes foram preservados para a evolução futura. O caso mostra, assim, o que o restante deste capítulo procura sustentar conceitualmente: verificação e validação funcionam melhor quando são tratadas como atividade integrada de crítica, aprendizagem e mediação, e não como verificação tardia de conformidade textual.

## 11.9 Riscos, armadilhas e dificuldades comuns em V&V de requisitos

Uma primeira armadilha é reduzir V&V a **revisão textual** ou, no extremo oposto, a tratá-la apenas como preparação para testes. No primeiro caso, a equipe examina frases, mas não examina relações, contexto, valor e coerência entre artefatos. No segundo, a qualidade dos requisitos passa a ser julgada apenas pela futura possibilidade de teste, empobrecendo a dimensão

sociotécnica da ER. Em ambos os casos, perde-se o caráter próprio da atividade: reduzir incerteza sobre entendimento e pertinência do sistema de requisitos.

A segunda armadilha é a **burocratização**. Inspeções, checklists, rastreabilidade, registros de decisão e quadros de achados são úteis quando ajudam a pensar melhor e a preservar memória de sentido. Tornam-se problemáticos quando passam a existir por si mesmos, como ritos de governança desvinculados de aprendizagem real. A consequência costuma ser previsível: muito preenchimento, pouco entendimento, e uma falsa sensação de qualidade apoiada no volume de documentação, não na robustez das decisões.

Uma terceira armadilha está nas **assimetrias de voz e poder**. Requisitos podem parecer “validados” quando, na verdade, apenas foram aceitos por fadiga, deferência hierárquica ou baixa capacidade de contestação de certos grupos. Isso é particularmente crítico em contextos com múltiplos stakeholders, alta distância entre equipe técnica e usuários finais, ou forte peso institucional de áreas patrocinadoras. Validar bem exige reconhecer que o consenso precisa ser construído com legitimidade, mediação e clareza sobre quem decide o quê e com base em quais critérios.

Outra dificuldade recorrente é verificar e validar **tarde demais**. Quanto mais o requisito avança sem crítica suficiente, maior o custo de corrigir interpretação equivocada, exceção não prevista, dado ausente ou conflito com regra de negócio. Em ambientes iterativos, esse risco não desaparece por si só; ele apenas muda de forma. É perfeitamente possível trabalhar em ciclos curtos e, ainda assim, deixar requisitos passarem de um refinamento ao outro sem verificação suficiente, ou demonstrar incrementos sem validar adequadamente o significado do que foi entregue.

Também é comum o erro de **concentrar V&V apenas no texto** e negligenciar o restante do sistema de requisitos. Um requisito aparentemente correto pode permanecer problemático porque seu glossário é instável, seu diagrama de estados admite contradições, seu protótipo induz interpretação errada, sua regra de negócio não foi explicitada ou sua origem não está clara. Em livros, em projetos e em ferramentas, a unidade real de trabalho não é a frase isolada, mas o conjunto articulado de declarações, representações, decisões e evidências.

Há ainda a armadilha da **rastreabilidade indiscriminada**. Quando tudo precisa apontar para tudo, o sistema se torna caro de manter, difícil de usar e pouco analítico. O efeito é perverso: a equipe passa a “ter rastreabilidade”, mas deixa de conseguir pensar com ela. O mesmo vale para formalização excessiva. Quando se formaliza o que não precisa ser formalizado, o requisito pode até ganhar aparência de rigor, mas perde acessibilidade e afasta stakeholders do processo de entendimento.

Por fim, cresce hoje uma dificuldade adicional: o **excesso de confiança em ferramentas**, especialmente em modelos de IA generativa. Usados sem critério, esses modelos podem dar aparência convincente a reformulações equivocadas, introduzir restrições inexistentes, apagar contexto relevante ou sugerir completude onde ainda há lacunas importantes. A ferramenta pode apoiar a crítica; nunca deve substituir julgamento humano, conhecimento de domínio, negociação com stakeholders e responsabilidade sobre as decisões produzidas.

## 11.10 Tratamento de achados, aprendizagem e retroalimentação da ER

Potencialmente, toda atividade de V&V pode produzir achados. Alguns aparecem como defeitos claros, como ambiguidade, inconsistência, regra de negócio ausente ou critério de aceite insuficiente; outros surgem como dúvida legítima, conflito interpretativo, risco não tratado, necessidade emergente ou questionamento sobre uma decisão. Em todos os casos, identificar o achado é apenas parte do trabalho: **o conhecimento produzido precisa ser compreendido e reincorporado ao sistema de requisitos** de maneira compatível com sua natureza e seu impacto.

Esse tratamento não exige um ritual único nem um formulário universal, mas algum nível de registro pode ser necessário para que o achado não perca seu contexto. Em projetos pequenos, uma anotação no backlog, no artefato ou na ata da sessão pode ser suficiente. Em ambientes regulados ou críticos, podem ser necessários registros mais formais, vinculando evidência, impacto e decisão. **O nível de detalhe deve ser proporcional** ao risco e à importância de preservar o raciocínio que sustentou o tratamento.

Dependendo do que foi encontrado, podem ser necessárias mudanças em declarações, representações, critérios, prioridades, vínculos ou decisões, assim como novas interações com stakeholders ou investigações adicionais. Esses movimentos não formam uma sequência predefinida: podem ocorrer em diferentes combinações e, muitas vezes, em paralelo. O essencial é **impedir que o conhecimento produzido pela revisão permaneça isolado do entendimento vigente sobre o produto**.

Quando um stakeholder rejeita um requisito durante a validação, por exemplo, a resposta adequada nem sempre é simplesmente corrigir a declaração. A rejeição pode indicar que a necessidade, o contexto de uso, uma regra de negócio, uma expectativa de valor ou alguma restrição relevante ainda não foram suficientemente compreendidos. Nesse caso, a pergunta deixa de ser apenas *“como reescrever este requisito?”* e passa a incluir *“o que ainda não entendemos sobre esta necessidade?”*.

Essa distinção evita reduzir validação a edição superficial, ou seja, ajustar palavras pode resolver uma ambiguidade textual, mas não necessariamente um desalinhamento de entendimento. Quando a divergência revela uma questão mais profunda, o próprio achado se torna nova evidência para investigação e aprendizagem. A discordância do stakeholder, nesse sentido, não é apenas um problema a eliminar, mas informação relevante sobre a distância entre o entendimento vigente e o contexto que o requisito pretende representar.

Além disso, a própria **V&V é fonte de aprendizagem** para a equipe. Checklists amadurecem, perguntas recorrentes tornam-se repertório, padrões de ambiguidade do domínio tornam-se mais visíveis, tipos de stakeholders antes pouco ouvidos passam a ser considerados mais cedo, e o próprio engenheiro de software refina sua sensibilidade para distinguir ruído superficial de risco semântico importante. Nesse sentido, V&V não melhora apenas requisitos; melhora a prática de Engenharia de Requisitos. A Figura 78 representa esse ciclo.



Figura 78 – Ciclo de aprendizagem de V&V.  
Fonte: elaborado pelo autor (2026).

Essa discussão prepara diretamente o terreno para o próximo capítulo. Uma vez que achados precisam ser preservados, decisões precisam permanecer legíveis e mudanças precisam ser administradas com coerência, torna-se inevitável tratar da organização e da atualização do sistema de requisitos ao longo do tempo. V&V produz conhecimento crítico sobre o requisito; o Capítulo 12 tratará de como esse conhecimento permanece estruturado, atual e utilizável durante a evolução do produto.

## 11.11 Síntese do capítulo e transição

Este capítulo tratou a verificação e a validação como atividade central de qualidade da Engenharia de Requisitos. Em vez de apresentá-las como rito terminal ou como simples antessala de testes, procurou situá-las no lugar que lhes corresponde no livro: prática contínua, sociotécnica e comunicacional de redução de incerteza, construção de confiança qualificada e sustentação da evolução do sistema de requisitos. Verificar, neste contexto, significa examinar a qualidade interna das declarações, representações e vínculos que compõem o sistema de requisitos. Validar significa confrontar esse sistema com o contexto, o valor e a experiência dos stakeholders que lhe dão sentido.

Também vimos que a qualidade do requisito não é apenas textual. Ela é relacional, contextual e evolutiva. Por isso, as técnicas discutidas — inspeções, revisões, walkthroughs, checklists, análise de consistência, rastreabilidade e cobertura, oficinas, cenários, protótipos, critérios de aceite, formalização seletiva e apoio criterioso de IA — só produzem resultado quando combinadas com intencionalidade editorial e de engenharia. O ponto central não é usar mais técnicas, mas usar melhor as técnicas que tornam visíveis as incertezas mais relevantes.

Essa conclusão conduz ao Capítulo 12 porque os resultados produzidos ao longo da Engenharia de Requisitos precisam permanecer estruturados e atualizados à medida que novas descobertas, decisões, revisões e achados modificam o entendimento do produto. Organização e Atualização

oferece os mecanismos para preservar essa continuidade sem congelar os requisitos nem tratar a mudança como exceção. É esse trabalho que será aprofundado a seguir.

## CAPÍTULO 11: PONTOS-CHAVE DO CAPÍTULO

- Verificação e Validação constitui uma atividade central da Engenharia de Requisitos e deve ser tratada como **prática contínua de qualidade**, não como ritual terminal nem como introdução à disciplina de Teste de Software.
- A **verificação** examina a qualidade interna do sistema de requisitos; a **validação** confronta esse sistema com necessidades, contexto, valor e reconhecimento dos stakeholders.
- Em projetos, verificação e validação frequentemente **ocorrem na mesma interação**, porque um mesmo problema pode revelar, ao mesmo tempo, defeito de formulação e inadequação de propósito.
- A **qualidade do requisito** não é apenas textual: ela é também relacional, contextual e evolutiva, dependente da coerência entre declarações, representações, regras de negócio, critérios e decisões.
- A passagem de atributos abstratos de qualidade para perguntas **verificáveis** e **evidências observáveis** é uma das operações mais importantes de maturação do requisito.
- Inspeções, revisões técnicas, revisões em pares, walkthroughs e checklists são instrumentos de verificação valiosos quando ajudam a produzir crítica fundamentada, e não quando se tornam rituais burocráticos.
- A análise de consistência entre artefatos, assim como a análise de rastreabilidade e cobertura, fortalece a V&V porque desloca a atenção da frase isolada para o sistema de requisitos como conjunto coerente.
- Em contextos ágeis e híbridos, práticas como refinamento contínuo, critérios de aceite e até listas leves de maturidade mínima podem apoiar V&V, desde que não substituam julgamento de engenharia nem validação com stakeholders.
- Oficinas colaborativas, cenários, histórias, casos de uso, protótipos, simulações e experimentos são instrumentos de validação porque aproximam o requisito do mundo real de uso, trabalho e decisão.
- Critérios de aceite podem apoiar a validação antecipada, e testes de aceitação podem produzir evidências relevantes, mas o centro da atividade continua sendo a **qualidade do requisito** na Engenharia de Requisitos.
- Representações formais e checagem de propriedades devem ser usadas seletivamente, como reforço à verificabilidade em pontos de maior risco, e não como eixo metodológico dominante.
- **Modelos de IA** generativa podem apoiar revisão, comparação entre artefatos, geração de perguntas e preparação de sessões, mas não substituem análise humana, contexto, mediação e responsabilidade sobre a decisão.

- Achados de V&V precisam ser tratados de forma proporcional e reincorporados ao sistema de requisitos, **evitando perda de conhecimento** e permitindo que o **entendimento continue evoluindo de maneira coerente**.
- A estratégia de V&V deve ser proporcional ao contexto: requisitos mais críticos, arriscados, regulados, instáveis ou caros de corrigir exigem maior rigor, participação mais ampla e evidências mais explícitas.
- **RFs, RNFs e RNs** exigem perguntas distintas de verificação e validação: comportamentos precisam ser compreendidos em seus fluxos e exceções; atributos de qualidade precisam de condições observáveis; regras de negócio precisam de fonte, vigência, escopo e autoridade confirmados.
- Em contextos ágeis e híbridos, **DEEP** pode apoiar a verificação da qualidade do backlog como conjunto, enquanto **INVEST** pode apoiar a verificação da qualidade de histórias de usuário e itens de backlog.
- Atender a DEEP ou INVEST não substitui validação com stakeholders: um item pode estar bem estruturado e ainda assim representar uma necessidade pouco relevante, mal priorizada ou inadequadamente compreendida.
- Achados de validação podem revelar que a necessidade ou o contexto ainda não foram suficientemente compreendidos; nesses casos, não basta redeclarar o requisito, pois o entendimento que o sustenta também precisa ser aprofundado.
- V&V não “fecha” o requisito; ela sustenta sua **evolução com qualidade**.



# Capítulo 12

## 12 Organização e Atualização de Requisitos

### 12.1 Introdução

O presente capítulo desenvolve a sexta atividade da Engenharia de Requisitos: **Organização e Atualização**. Seu foco é mostrar como manter o escopo de requisitos do produto estruturado e coerente com o entendimento vigente sobre o produto, mesmo quando novas descobertas, decisões, validações, mudanças de contexto e aprendizados alteram esse entendimento. Isso envolve organizar requisitos e suas relações de modo que possam ser localizados, relacionados e utilizados pelas pessoas, bem como atualizar o escopo para que suas informações, prioridades, vínculos e registros continuem representando aquilo que passou a vigorar. Em termos simples, organizar e atualizar requisitos é cuidar para que o escopo continue útil, navegável e rastreável no nível necessário, sem perder aderência ao entendimento atual sobre o produto.

Os requisitos se relacionam com problemas e necessidades que lhes dão origem, objetivos que ajudam a justificar sua existência, regras de negócio e restrições que condicionam comportamentos, critérios que ajudam a avaliá-los, decisões que podem alterar seu conteúdo ou sua prioridade e evidências que modificam aquilo que se compreende sobre o produto. Por isso, cuidar dessas relações vai além de uma tarefa administrativa: quando elas se perdem, um requisito pode continuar registrado em um artefato ou ferramenta e, ainda assim, deixar de comunicar parte importante de seu significado. A Figura 79 ilustra como relações com problemas, objetivos, regras, critérios, decisões e evidências contribuem para preservar o significado de um requisito.



Figura 79 – Relações relevantes para preservar o significado de um requisito.

Fonte: elaborado pelo autor (2026).

Organizar requisitos significa **dar estrutura ao escopo que se conhece em determinado momento**, dispondo seus elementos em formas adequadas ao contexto, tornando relações relevantes visíveis e oferecendo caminhos para que diferentes participantes possam localizar, compreender

e relacionar aquilo de que precisam. Essa organização pode envolver, por exemplo, agrupamentos, identificadores, estados, prioridades já acordadas, relações entre requisitos e vínculos com informações relevantes para sua compreensão.

Atualizar requisitos significa **manter essa estrutura e seu conteúdo coerentes com a evolução do entendimento**. Novas descobertas podem introduzir requisitos ou relações antes desconhecidos; decisões podem modificar prioridades ou retirar itens do escopo; validações podem revelar inadequações; mudanças normativas podem alterar regras; e o uso do produto pode produzir evidências que exigem revisão. Nesses casos, não basta alterar uma frase isoladamente: é preciso avaliar quais partes do escopo também precisam refletir o novo entendimento.

Nessa perspectiva sociotécnica e comunicacional, o **escopo de requisitos** não é tratado apenas como um conjunto de registros técnicos. Ele constitui uma referência compartilhada sobre aquilo que se compreende e se espera do produto em determinado momento, permitindo que stakeholders, equipe técnica e organização retomem decisões, reconheçam relações e continuem a conversa sem depender exclusivamente da memória das pessoas.

A Organização e Atualização atua, portanto, sobre esse conjunto para mantê-lo estruturado e coerente com o entendimento vigente. A Figura 80 sintetiza a responsabilidade da atividade, suas principais práticas, resultados e relações com as demais atividades da Engenharia de Requisitos.

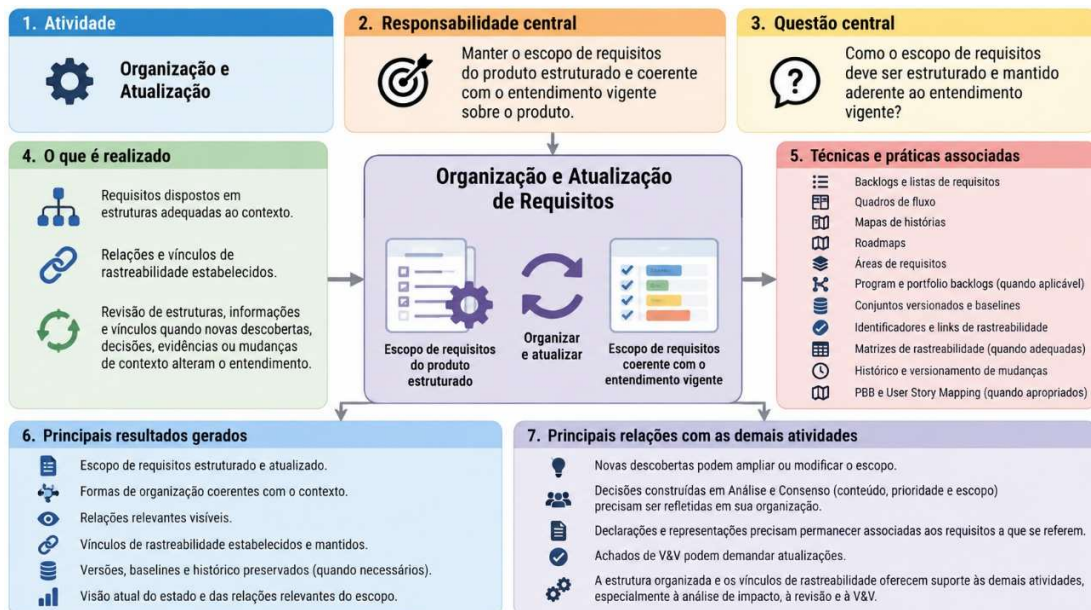


Figura 80 – Enquadramento da atividade de Organização e Atualização.

Fonte: elaborado pelo autor (2026).

### Box 12.1 — Valores e princípios em ação: organização e atualização como cuidado com o entendimento

Os valores da Engenharia de Requisitos tornam-se particularmente visíveis no cuidado com o escopo. A comunicação é favorecida quando esse escopo permanece legível para diferentes públicos; o feedback produz efeito quando validações, uso e aprendizagem retornam ao conjunto de requisitos; a **simplicidade** orienta a remoção de ruído e a preservação apenas da memória necessária; a **coragem** aparece quando requisitos obsoletos, duplicados ou sem valor claro são questionados; e **compromisso** e **confiança** são fortalecidos quando decisões, prioridades e mudanças permanecem compreensíveis para os envolvidos.

Os princípios também se tornam visíveis. A **orientação a valor** impede que a organização vire mera arrumação de itens. A **comunicação dos requisitos** e o **entendimento compartilhado** lembram que o escopo deve apoiar conversas e decisões. O **contexto da solução** e a **relação entre problema, requisitos e solução** ajudam a preservar o sentido dos requisitos. A **mudança nos requisitos** deixa de ser tratada como perturbação e passa a ser incorporada como parte do aprendizado. O **processo de Engenharia de Requisitos** e o **impacto da ER** reforçam que manter o escopo organizado e atual influencia diretamente a qualidade das decisões posteriores.

## 12.2 Organização e atualização na dinâmica da Engenharia de Requisitos

A Organização e Atualização se articula continuamente com as demais atividades da Engenharia de Requisitos. Novas descobertas podem introduzir necessidades, restrições ou requisitos ainda não considerados; a Análise e Consenso pode produzir decisões que alteram conteúdo, prioridade ou escopo; declarações e representações tornam diferentes aspectos do entendimento recuperáveis e relacionáveis; e a Verificação e Validação pode revelar achados que exigem revisão. A Organização e Atualização procura refletir esses efeitos no conjunto organizado de requisitos e manter visíveis as relações necessárias para que o entendimento vigente continue utilizável.

Quando o escopo está bem organizado e suas relações relevantes permanecem recuperáveis, torna-se mais fácil identificar lacunas, compreender impactos, localizar stakeholders e fontes de informação, comparar versões, revisar decisões e avaliar a coerência do conjunto. A Organização e Atualização, portanto, não atua apenas sobre resultados produzidos pelas demais atividades; ao manter o escopo estruturado e acessível, também cria condições para novas investigações, análises, declarações, representações e avaliações.

A Figura 81 sintetiza essa influência mútua ao mostrar que descobertas, decisões, declarações, representações, achados e evidências produzidos pelas demais atividades podem modificar o escopo, enquanto a Organização e Atualização preserva a estrutura e a coerência desse conjunto com o entendimento vigente. Com isso, o próprio escopo organizado passa a oferecer suporte para que as demais atividades continuem sendo realizadas à medida que o produto evolui.



Figura 81 – Organização e Atualização como atividade de sustentação e continuidade do escopo de requisitos.  
Fonte: elaborado pelo autor (2026).

Ao longo dessa evolução, essas relações não seguem uma sequência obrigatória. A organização pode começar quando ainda existem poucos requisitos e muitas incertezas, enquanto novas descobertas podem surgir quando o produto já está em uso. Uma validação pode exigir nova investigação, uma decisão pode modificar prioridades anteriormente registradas e uma mudança normativa pode demandar a revisão simultânea de requisitos, regras, critérios e representações. Em diferentes momentos e combinações, o cuidado necessário permanece o mesmo: fazer com que o escopo continue refletindo o entendimento que passou a vigorar sobre o produto.

Essa proximidade entre as atividades pode gerar dúvidas especialmente em relação à Verificação e Validação, pois ambas lidam com coerência, relações e mudanças no conjunto de requisitos. Seus focos, entretanto, são distintos. A Verificação e Validação examina requisitos e suas relações para produzir evidências sobre qualidade e adequação; a Organização e Atualização cuida para que o escopo reflita as mudanças, decisões e relações que passam a compor o entendimento vigente. O Quadro 67 sintetiza essa diferença de foco.

| Verificação e Validação                                                                          | Organização e Atualização                                                                                                 |
|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>Pergunta central:</b> os requisitos estão adequados?                                          | <b>Pergunta central:</b> o escopo continua estruturado e coerente com o entendimento vigente?                             |
| Examina qualidade, consistência, verificabilidade e adequação dos requisitos e de suas relações. | Mantém estruturas, relações, prioridades registradas e demais informações do escopo coerentes com o entendimento vigente. |
| Produz achados e evidências que podem indicar necessidade de revisão.                            | Reflete no escopo as mudanças decorrentes desses achados e mantém coerentes as relações afetadas.                         |

Quadro 67 – Diferença entre verificação e validação e organização e atualização.  
Fonte: elaborado pelo autor (2026).

A Organização e Atualização oferece, assim, continuidade ao sistema de requisitos sem pressupor estabilidade permanente. Seu papel não é impedir que requisitos mudem, mas evitar que o conjunto organizado deixe de representar aquilo que stakeholders e equipe passaram a compreender, decidir e considerar relevante para o produto.

## 12.3 Escopo de requisitos do produto

Para desenvolver a atividade de Organização e Atualização, este capítulo adota como objeto central de cuidado o escopo de requisitos do produto. Neste livro, essa expressão designa o conjunto organizado de requisitos que expressam aquilo que, em determinado momento, se entende que o produto precisa atender, considerados em seus diferentes níveis de abstração e associados aos elementos e relações necessários para compreender sua origem, significado, condições e relações com o restante do produto.

O núcleo desse escopo é formado pelos requisitos que expressam aquilo que o produto precisa atender. Regras de negócio, restrições e critérios podem estar diretamente associados a esses requisitos porque condicionam, delimitam ou ajudam a avaliar seu atendimento. Problemas, necessidades, objetivos, stakeholders, decisões, justificativas e evidências, por sua vez, não se transformam em requisitos apenas por estarem relacionados ao escopo. Quando relevantes, permanecem vinculados a ele para que seja possível compreender a origem, a intenção, as decisões e a evolução dos requisitos.

Essa distinção é importante para a rastreabilidade, por exemplo: um requisito funcional como **RF-01 — Registrar agendamento** pode estar relacionado a uma necessidade do cidadão, a um objetivo de redução de filas presenciais, a regras sobre disponibilidade de horários, a requisitos não funcionais de acessibilidade e desempenho, a critérios de aceite e a decisões posteriores sobre sua implementação. Nem todos esses elementos são o requisito, mas perder seus vínculos pode tornar difícil compreender por que ele existe, como deve ser interpretado ou que outros elementos serão afetados se mudar.

A expressão “**em determinado momento**” também é importante, pois o escopo de requisitos não representa uma descrição definitiva e imutável do produto. Ele expressa o entendimento vigente: alguns requisitos podem estar suficientemente compreendidos para determinado uso, enquanto outros permanecem exploratórios; alguns podem ter sido selecionados para uma entrega, enquanto outros continuam considerados para o produto, mas não para aquele momento. Organizar e atualizar requisitos exige tornar essas diferenças perceptíveis.

Para evitar confusões, convém distinguir o escopo de requisitos de outros três escopos com os quais ele se relaciona. O Quadro 68 apresenta essa diferença utilizando como exemplo um sistema de agendamento de consultas públicas.

| Conceito                               | O que delimita                                                                                                            | Exemplo no sistema de agendamento                                                                                                                                                                                                                        |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Escopo do produto</b>               | Define, em nível amplo, aquilo que o produto pretende oferecer e os limites do problema e do serviço que serão atendidos. | O produto deverá apoiar digitalmente o agendamento de consultas públicas, permitindo ao cidadão realizar e acompanhar operações relacionadas ao seu agendamento e oferecendo às unidades condições para administrar esse atendimento.                    |
| <b>Escopo de requisitos do produto</b> | Expressa, de forma organizada, aquilo que atualmente se entende que o produto precisa atender.                            | RFs como Registrar agendamento, Consultar agendamento e Cancelar agendamento; RNFs de acessibilidade, disponibilidade e desempenho; regras de negócio sobre horários e cancelamento; critérios e relações necessários para compreender esses requisitos. |



| Conceito                                        | O que delimita                                                                            | Exemplo no sistema de agendamento                                                                                                                                                                                                  |
|-------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Escopo da entrega, release ou incremento</b> | Seleciona uma parte do escopo de requisitos para determinado horizonte de entrega.        | Em uma primeira entrega podem ser selecionados Registrar agendamento, Consultar agendamento e os requisitos de acessibilidade essenciais, enquanto Cancelar agendamento permanece no escopo do produto para uma entrega posterior. |
| <b>Escopo do projeto</b>                        | Delimita o trabalho necessário para conduzir a iniciativa que cria ou modifica o produto. | Planejamento, oficinas com stakeholders, desenho da solução, desenvolvimento, preparação de ambientes, migração quando necessária, implantação, capacitação e demais atividades previstas para realizar o projeto.                 |

Quadro 68 – Relação entre diferentes escopos no sistema de agendamento.

Fonte: elaborado pelo autor (2026).

Esses quatro conceitos descrevem **o que está sendo delimitado** e não devem ser confundidos com as estruturas utilizadas para organizar ou tornar visíveis esses escopos e o trabalho relacionado a eles. Um Product Backlog, por exemplo, não constitui um quinto tipo de escopo; trata-se de uma estrutura cuja relação com os escopos apresentados depende da natureza dos itens que contém e da finalidade para a qual é utilizada. Essa distinção será aprofundada na próxima seção.

O exemplo evidencia que esses escopos se relacionam, mas respondem a perguntas diferentes. O **escopo do produto** estabelece a fronteira mais ampla daquilo que se pretende oferecer; o **escopo de requisitos** explicita os compromissos atualmente compreendidos para esse produto; o **escopo da entrega** seleciona quais desses compromissos serão tratados em determinado horizonte; e o **escopo do projeto** descreve o trabalho necessário para conduzir a iniciativa. A Figura 82 ilustra a relação entre esses conceitos.

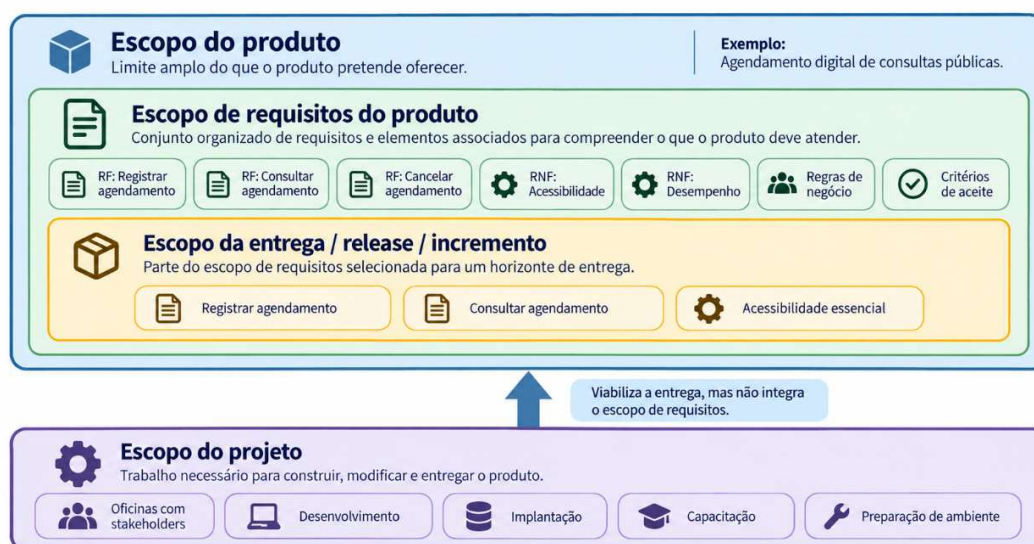


Figura 82 – Relação entre escopo do produto, escopo de requisitos, escopo da entrega e escopo do projeto.

Fonte: elaborado pelo autor (2026).

Selecionar apenas alguns requisitos para uma entrega não significa, portanto, retirar os demais do escopo do produto ou do escopo de requisitos. O requisito **Cancelar agendamento**, por

exemplo, pode continuar fazendo parte do entendimento vigente sobre o produto mesmo que não tenha sido selecionado para a primeira entrega. O escopo da entrega expressa um recorte temporal ou de compromisso sobre aquilo que será tratado naquele horizonte, e não uma redefinição automática de tudo o que o produto precisa atender.

Da mesma forma, trabalhos necessários para viabilizar o produto não se tornam requisitos apenas porque precisam ser realizados. Preparar ambientes, capacitar equipes, realizar migração de dados ou conduzir atividades de implantação pode fazer parte do escopo do projeto sem integrar o escopo de requisitos. Em outros casos, um trabalho técnico registrado em uma estrutura como o Product Backlog pode estar relacionado a um requisito sem ser, ele próprio, um requisito.

Essas distinções permitem separar os diferentes escopos das estruturas utilizadas para organizá-los, visualizá-los ou operacionalizar o trabalho relacionado ao produto. Backlogs, mapas de histórias, roadmaps, quadros de fluxo, áreas de requisitos e baselines serão discutidos na próxima seção justamente sob essa perspectiva.

## 12.4 Estruturas para organizar o escopo e o trabalho sobre o produto

Os escopos discutidos na seção anterior descrevem **aquilo que está sendo delimitado**: o produto, seus requisitos, uma determinada entrega ou o trabalho necessário para conduzir o projeto. Para que esses escopos possam apoiar comunicação, decisão e ação, entretanto, eles precisam ser tornados visíveis por alguma estrutura. Backlogs, mapas de histórias, roadmaps, quadros de fluxo, áreas de requisitos e baselines cumprem diferentes funções nesse sentido. Essas estruturas não constituem novos tipos de escopo; são formas de organizar, visualizar ou operacionalizar partes dos escopos e do trabalho relacionado ao produto.

Por isso, não existe uma correspondência obrigatória de uma estrutura para cada tipo de escopo. Uma mesma estrutura pode reunir elementos de naturezas diferentes, enquanto um mesmo escopo pode ser observado por mais de uma estrutura. Um Product Backlog, por exemplo, pode conter itens que expressam requisitos e também outros trabalhos necessários à evolução do produto. Um mapa de histórias pode oferecer uma visão de parte do escopo de requisitos a partir da experiência do usuário. Um roadmap pode situar intenções de evolução em diferentes horizontes. Uma baseline, por sua vez, pode preservar uma referência acordada do escopo de requisitos em determinado momento.

No Scrum, o **Product Backlog** é definido como uma lista ordenada e emergente do que é necessário para melhorar o produto e constitui a fonte de trabalho do Scrum Team (SCHWABER; SUTHERLAND, 2020). Essa definição ajuda a compreender por que Product Backlog e escopo de requisitos do produto não são conceitos equivalentes. Requisitos podem ser expressos ou detalhados por itens do backlog, mas o trabalho necessário para melhorar o produto não se limita necessariamente a requisitos. O backlog oferece, portanto, uma estrutura de ordenação do trabalho relacionado ao produto, dentro da qual parte importante do escopo de requisitos pode ser tornada operacional.

No Scrum, o **Sprint Backlog** reúne a Meta da Sprint, os itens do Product Backlog selecionados e um plano para produzir o Incremento (SCHWABER; SUTHERLAND, 2020). Sua relação é particularmente próxima do escopo de uma entrega ou de um horizonte imediato de trabalho,

mas ele não deve ser tratado como sinônimo desse escopo: além daquilo que se pretende produzir, contém elementos necessários para organizar a realização do trabalho.

Quando o interesse principal está no **fluxo do trabalho**, quadros e listas inspirados em Kanban oferecem outra perspectiva. Eles tornam visíveis estados, políticas de tratamento, trabalho em progresso e movimentação dos itens ao longo do processo (HAMMARBERG; SUNDEN, 2014; BURROWS, 2014; LEOPOLD; KALTENECKER, 2015). Requisitos podem aparecer nesses fluxos, assim como defeitos, investigações, melhorias ou outras demandas. O quadro ajuda a responder onde determinado trabalho se encontra e como ele se movimenta, mas não substitui as relações necessárias para compreender por que um requisito existe ou como se conecta a outros elementos do escopo.

Outras estruturas privilegiam a compreensão do próprio produto e de seus requisitos. **Mapas de histórias** organizam histórias e funcionalidades em relação à jornada, às atividades e aos passos realizados pelos usuários, favorecendo a compreensão de relações e possíveis recortes de entrega (PATTON, 2014). **Listas de features** oferecem uma visão predominantemente funcional, agrupando comportamentos ou funcionalidades relevantes para clientes ou para o negócio (PALMER; FELSING, 2002; LEFFINGWELL, 2011). Em produtos grandes, **áreas de requisitos** podem agrupar partes relacionadas do escopo para reduzir a complexidade de navegação e apoiar múltiplas equipes (LARMAN; VODDE, 2016). Nenhuma dessas estruturas, entretanto, precisa representar sozinha todos os aspectos do escopo.

Ao relacionar estratégia, valor e tempo em diferentes horizontes, os roadmaps ajudam a comunicar direções, objetivos e possíveis movimentos do produto em diferentes horizontes, relacionando estratégia, valor e tempo (PICHLER, 2010; LEFFINGWELL, 2011). Sua principal contribuição não é declarar requisitos nem definir detalhadamente uma entrega, mas permitir que partes do escopo sejam compreendidas em uma narrativa de evolução. Por isso, um roadmap deve ser interpretado como instrumento de orientação e comunicação, e não como promessa imutável de conteúdo e datas.

Em contextos com múltiplas equipes ou níveis de decisão, podem ser utilizadas estruturas adicionais, como **program backlogs, portfolio backlogs ou agrupamentos por áreas de requisitos**. Seu papel é oferecer recortes adequados à escala, permitindo visualizar dependências, responsabilidades e diferentes níveis de decisão sem perder a relação com o produto como um todo (LEFFINGWELL, 2011; LARMAN; VODDE, 2016). Quanto maior a quantidade de camadas introduzidas, porém, maior a necessidade de garantir que elas realmente ajudem a compreensão, em vez de apenas reproduzir a estrutura organizacional em novos artefatos.

Já em contextos contratuais, regulados, críticos ou que exigem maior controle de mudanças, **conjuntos versionados e baselines** permitem preservar um estado reconhecível do escopo de requisitos. Uma baseline estabelece uma referência acordada em determinado momento, a partir da qual alterações podem ser identificadas, justificadas e controladas. Ela não significa que os requisitos se tornaram imutáveis, mas que existe uma referência explícita para compreender o que mudou e em relação a qual estado anterior (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017; ISO/IEC/IEEE 29148:2018).



O Quadro 69 sintetiza essas diferenças e mostra que cada estrutura torna determinadas dimensões mais visíveis, sem que isso a transforme em um novo tipo de escopo.

| Estrutura                                         | O que torna mais visível                                                     | Relação predominante com os escopos                                                                 | Cuidado de interpretação                                                                                                                    |
|---------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Product Backlog</b>                            | Trabalho ordenado necessário à evolução do produto.                          | Pode materializar partes do escopo de requisitos e outros trabalhos relacionados ao produto.        | Nem todo item do backlog é necessariamente um requisito, nem todo o escopo de requisitos precisa estar explícito diretamente em seus itens. |
| <b>Sprint Backlog / seleção para uma entrega</b>  | Trabalho selecionado para um horizonte imediato e sua realização.            | Relaciona-se principalmente ao escopo da entrega e ao trabalho necessário para realizá-la.          | Não confundir a seleção de trabalho com todo o escopo de requisitos da entrega ou do produto.                                               |
| <b>Quadro ou lista de fluxo</b>                   | Estado, movimentação e tratamento dos itens de trabalho.                     | Pode apoiar escopo da entrega e trabalho do projeto relacionado ao produto.                         | Estado do trabalho não substitui as relações semânticas entre requisitos.                                                                   |
| <b>Mapa de histórias / lista de features</b>      | Jornada, funcionalidades, relações e possíveis recortes do produto.          | Oferece diferentes visões do escopo do produto e, principalmente, do escopo de requisitos.          | RNFs, regras e restrições transversais podem exigir formas complementares de organização.                                                   |
| <b>Roadmap</b>                                    | Direção e horizontes de evolução do produto.                                 | Relaciona escopo do produto a possíveis recortes futuros de entrega.                                | Não tratar intenção de evolução como compromisso imutável.                                                                                  |
| <b>Áreas de requisitos e estruturas de escala</b> | Agrupamentos, responsabilidades e dependências em ambientes de maior escala. | Organizam partes do escopo de requisitos e sua relação com diferentes equipes ou níveis de decisão. | Evitar fragmentação e excesso de camadas organizacionais.                                                                                   |
| <b>Baseline / conjunto versionado</b>             | Estado acordado dos requisitos em determinado momento.                       | Relaciona-se diretamente ao escopo de requisitos do produto.                                        | Uma versão estabilizada não significa que o entendimento deixou de evoluir.                                                                 |

Quadro 69 – Relação entre estruturas de organização e os escopos que apoiam.  
Fonte: elaborado pelo autor (2026).

A relação apresentada no Quadro 69 não deve ser interpretada como uma classificação rígida. Uma organização pode usar um Product Backlog juntamente com um roadmap; requisitos regulatórios podem ser mantidos em uma baseline e, ao mesmo tempo, relacionados a itens de backlog; e um quadro de fluxo pode mostrar o andamento de trabalhos associados a qualquer uma dessas estruturas. A escolha depende de quais informações precisam estar visíveis para quais pessoas e decisões.

O ponto central é que **estrutura e escopo não são a mesma coisa**. O escopo delimita aquilo que está sendo considerado; a estrutura oferece uma maneira de organizá-lo, visualizá-lo ou

transformá-lo em trabalho manejável. Essa distinção também permite que estruturas sejam combinadas sem que cada nova ferramenta, framework ou prática introduza um novo conceito de escopo.

Essa diferença torna-se mais clara quando as estruturas são aplicadas ao mesmo produto. A Figura 83 retoma o sistema de agendamento e mostra como um mesmo conjunto de requisitos e trabalhos relacionados pode ser observado de maneiras diferentes: o Product Backlog evidencia ordenação do trabalho; o mapa de histórias destaca relações com a experiência do usuário; o roadmap situa partes do escopo em diferentes horizontes; e a baseline preserva uma referência do conjunto de requisitos em determinado momento.

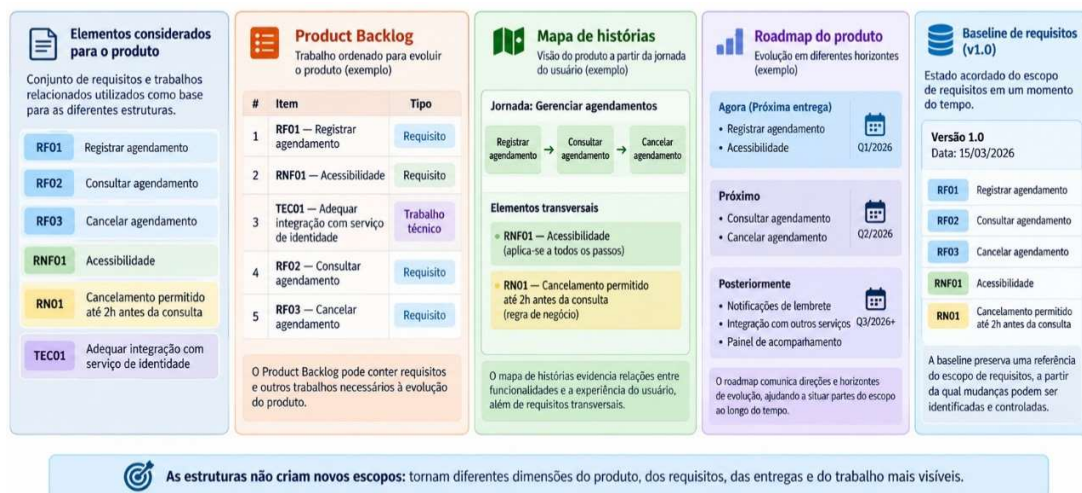


Figura 83 – Diferentes estruturas aplicadas ao mesmo escopo e ao trabalho sobre o produto.  
Fonte: elaborado pelo autor (2026).

Também é necessário distinguir essas estruturas das **formas utilizadas para declarar requisitos**. Histórias de usuário, casos de uso, cenários e outras formas linguísticas expressam requisitos e foram discutidas no Capítulo 9; um backlog, um mapa de histórias ou uma baseline oferecem estruturas nas quais essas declarações e outros elementos podem ser posicionados, relacionados ou utilizados. Da mesma forma, identificadores, links, histórico de versões e outros mecanismos podem apoiar essa organização sem constituírem, isoladamente, uma estrutura de escopo. Com essas distinções estabelecidas, a questão deixa de ser qual artefato “representa corretamente” o escopo e passa a ser como organizar seus elementos para que permaneçam compreensíveis, relacionáveis e úteis. Essa é a discussão desenvolvida na próxima seção.

## 12.5 Organizar requisitos: dar forma ao entendimento

Organizar requisitos é dar forma ao entendimento de maneira que o escopo possa ser consultado, relacionado e utilizado por diferentes pessoas. O desafio, portanto, não é apenas decidir onde armazenar cada requisito, mas permitir que stakeholders e equipe reconheçam quais requisitos existem, a que parte do produto se referem, quais elementos os condicionam e que relações precisam ser consideradas quando uma decisão é tomada.

Nesse sentido, um escopo é **navegável** quando uma pessoa consegue localizar e relacionar as informações de que necessita sem precisar percorrer todo o conjunto de registros ou depender da memória de quem participou das discussões anteriores. Em um sistema de agendamento, por exemplo, deve ser possível identificar quais requisitos tratam do cancelamento de consultas, qual regra estabelece o prazo permitido, quais requisitos não funcionais também se aplicam a essa funcionalidade, que critérios ajudam a avaliá-la e em que entrega ela está considerada. Quando essas respostas exigem procurar manualmente em vários documentos, interpretar nomes pouco claros ou perguntar a alguém que “lembra como foi decidido”, o escopo pode estar registrado, mas ainda está pouco organizado.

Essa navegabilidade também depende de quem utiliza o escopo e da finalidade desse uso. Diferentes pessoas procuram informações distintas e estabelecem relações diferentes entre os mesmos requisitos, conforme suas responsabilidades, decisões e necessidades de compreensão. A Figura 84 ilustra como um mesmo escopo organizado pode apoiar diferentes perspectivas de uso sem exigir conjuntos independentes de requisitos para cada público.

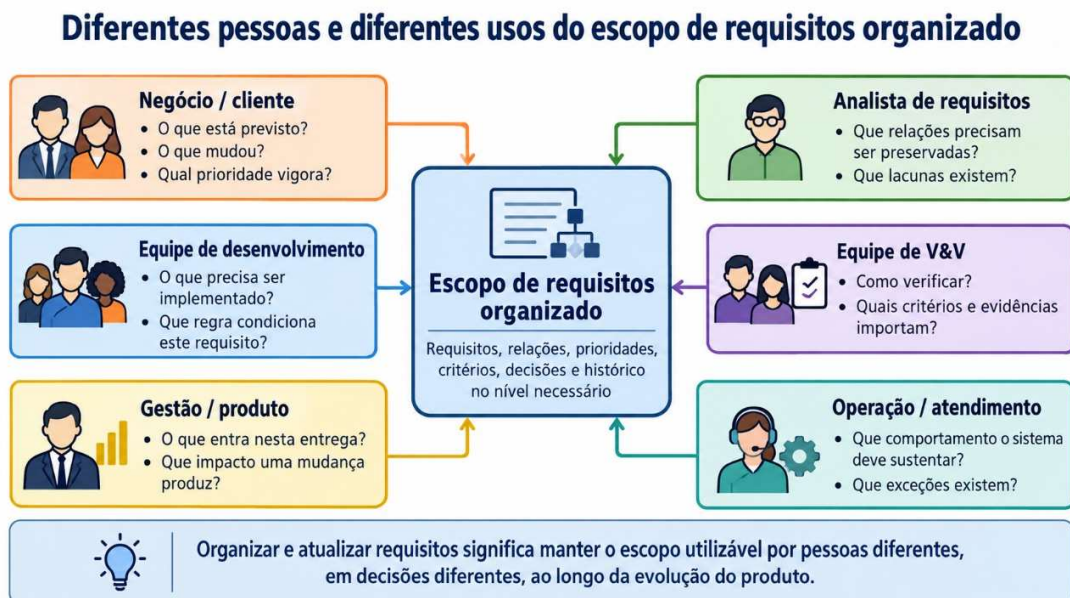


Figura 84 – Diferentes pessoas e diferentes usos do escopo de requisitos organizado.  
Fonte: elaborado pelo autor (2026).

As perspectivas apresentadas na Figura 84 não representam visões isoladas do produto, ou seja, elas partem de um mesmo escopo e destacam informações diferentes conforme a questão que precisa ser respondida. Enquanto representantes de negócio podem precisar identificar o que está previsto e o que mudou, equipes técnicas podem buscar dependências e regras aplicáveis, responsáveis pelo produto podem observar entregas e impactos, e pessoas envolvidas em V&V podem procurar critérios e evidências. Organizar o escopo significa permitir essas diferentes leituras sem perder a coerência entre elas.

A organização começa pela possibilidade de **distinguir, localizar e relacionar os elementos que compõem o entendimento**. Identificadores estáveis ajudam a referenciar requisitos sem

depende de sua posição em um documento; atributos permitem registrar informações como tipo, estado, origem ou entrega associada; agrupamentos oferecem diferentes formas de observar o conjunto; e vínculos explícitos permitem recuperar relações entre requisitos e outros elementos relevantes. A literatura sobre a ER destaca justamente o uso de atributos, estados, diferentes visões e ligações entre objetos de informação para tornar grandes conjuntos de requisitos administráveis e recuperáveis (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

Os agrupamentos utilizados dependem do produto e das perguntas que precisam ser respondidas. Requisitos podem ser organizados por área funcional, jornada do usuário, processo de negócio, parte do produto, requisito de nível mais abstrato, entrega ou outro critério que faça sentido naquele contexto. Essas formas não precisam ser exclusivas. O mesmo requisito pode aparecer em diferentes **visões** sem ser duplicado: uma visão pode reunir os requisitos relativos à jornada de agendamento, outra pode mostrar apenas os itens selecionados para determinada release e outra pode destacar requisitos afetados por uma regra regulatória. Ferramentas de organização de requisitos exploram justamente essa possibilidade de manter um conjunto de informações e gerar diferentes visões conforme atributos e relações relevantes para cada uso (WIEGERS; BEATTY, 2013).

No sistema de agendamento utilizado como exemplo neste capítulo, os requisitos **RF01 — Registrar agendamento**, **RF02 — Consultar agendamento** e **RF03 — Cancelar agendamento** podem ser agrupados sob uma área relacionada à gestão de agendamentos. A regra **RN01 — Cancelamento permitido até duas horas antes da consulta** deve permanecer relacionada ao RF03, enquanto o **RNF01 — Acessibilidade** pode estar associado aos diferentes requisitos cuja interação com o usuário precisa atender a essa qualidade. Critérios de aceite, representações ou decisões relevantes podem permanecer vinculados aos requisitos correspondentes sem que seu conteúdo precise ser repetido em cada lugar. A Figura 85 ilustra a diferença entre manter esses elementos apenas como uma lista e organizá-los de forma que suas relações possam ser percebidas.

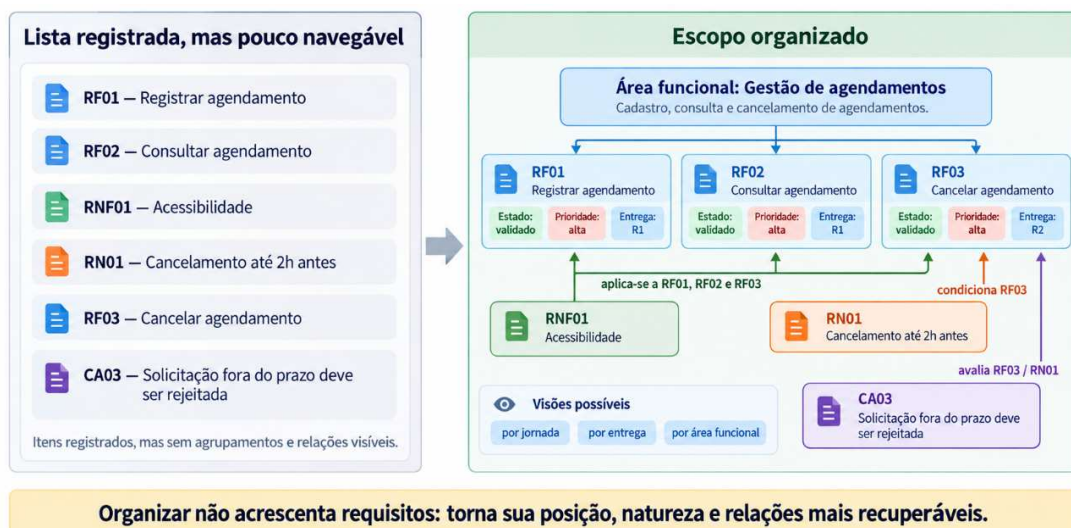


Figura 85 – De uma lista de itens a um escopo de requisitos organizado e navegável.  
Fonte: elaborado pelo autor (2026).

Na Figura 85, regras e critérios aparecem como elementos separados apenas para tornar suas relações visualmente explícitas. Em uma estrutura real, essas informações também podem estar incorporadas ao requisito ou ao artefato que o declara, desde que sua natureza e sua relação com o requisito continuem suficientemente claras. Organizar, portanto, não significa produzir um único arranjo considerado “correto”, mas oferecer formas adequadas para que as informações e relações relevantes possam ser recuperadas e compreendidas. Dependendo do contexto, isso pode ser realizado com uma planilha, um backlog, um repositório especializado, um conjunto de páginas vinculadas ou outra combinação de mecanismos. O valor da organização não está na sofisticação da ferramenta, mas na capacidade de ajudar as pessoas a compreender e utilizar o escopo.

Na prática, algumas ações simples ajudam a transformar essa preocupação em formas observáveis de organização. O Quadro 70 reúne exemplos dessas práticas.

| Prática                                                             | O que procura tornar possível                                                                                                                   | Exemplo no sistema de agendamento                                                                                                                                                                                |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Identificar os requisitos de forma estável</b>                   | Permitir referência inequívoca mesmo quando posição, agrupamento ou texto mudam.                                                                | RF01, RF02, RF03, RNF01.                                                                                                                                                                                         |
| <b>Distinguir a natureza dos elementos</b>                          | Permitir reconhecer a natureza e o papel das diferentes informações, estejam elas registradas separadamente ou reunidas em uma mesma estrutura. | Em uma história de usuário, distinguir o requisito expresso pela história dos critérios de aceitação associados; em um caso de uso, distinguir o comportamento esperado das regras de negócio que o condicionam. |
| <b>Agrupar segundo critérios úteis ao produto</b>                   | Permitir que partes relacionadas do escopo sejam localizadas em conjunto.                                                                       | Agrupar Registrar, Consultar e Cancelar em “Gestão de agendamentos”.                                                                                                                                             |
| <b>Registrar atributos necessários ao uso do escopo</b>             | Permitir filtros e visões sem alterar o conteúdo do requisito.                                                                                  | Estado, prioridade já acordada, releas considerada, responsável ou origem quando necessários.                                                                                                                    |
| <b>Tornar relações relevantes explícitas</b>                        | Permitir compreender dependências e associações sem depender apenas de memória.                                                                 | RN01 condiciona RF03; RNF01 aplica-se a RF01, RF02 e RF03.                                                                                                                                                       |
| <b>Criar diferentes visões sem duplicar o conteúdo</b>              | Atender às necessidades de diferentes públicos e decisões a partir do mesmo conjunto.                                                           | Visão por jornada, por entrega ou por área funcional.                                                                                                                                                            |
| <b>Relacionar requisitos de diferentes níveis quando necessário</b> | Preservar a compreensão de como requisitos mais específicos se associam a requisitos mais amplos.                                               | Um requisito mais amplo de “Gerenciar agendamentos” relacionado aos requisitos Registrar, Consultar e Cancelar.                                                                                                  |

Quadro 70 – Práticas para organizar o escopo de requisitos.

Fonte: elaborado pelo autor (2026), com base em Wiegers e Beatty (2013) e Dick, Hull e Jackson (2017).

Essas práticas não constituem uma lista obrigatória de campos ou controles. O nível de organização deve acompanhar a complexidade do produto, o número de requisitos, a quantidade de pessoas envolvidas e a necessidade de recuperar relações posteriormente. Em um produto



pequeno, identificadores, agrupamentos simples e alguns vínculos podem ser suficientes; em ambientes distribuídos, regulados ou de longa duração, diferentes atributos, visões e relações podem ser necessários. O princípio é **preservar informação suficiente para uso**, evitando tanto a perda de contexto quanto a criação de estruturas que ninguém efetivamente utiliza.

Dois limites ajudam a preservar a fronteira desta atividade com as demais. Quando requisitos em diferentes níveis de abstração coexistem, a Organização e Atualização pode registrar sua relação — por exemplo, indicando que requisitos mais específicos detalham um requisito mais amplo —, mas o trabalho de compreender, decompor ou reformular seu conteúdo decorre das atividades que aprofundam o entendimento e a declaração. De forma semelhante, a prioridade pode aparecer como atributo do requisito e servir para filtros, ordenações ou visões, mas sua avaliação e negociação são realizadas em Análise e Consenso; aqui importa manter visível a prioridade que passou a vigorar.

Organizar o escopo também não significa transformar cada informação relevante em um item independente. Requisitos, regras de negócio, critérios de aceitação, restrições, decisões e outras informações podem aparecer reunidos em uma mesma estrutura quando isso favorece sua compreensão. Critérios de aceitação, por exemplo, podem integrar uma história de usuário; regras de negócio podem ser registradas na especificação de um caso de uso; e restrições específicas podem permanecer junto ao requisito que condicionam. O que precisa ser preservado é a distinção de significado entre essas informações e a possibilidade de compreender como se relacionam, e não necessariamente sua separação física em diferentes registros.

A necessidade de registros independentes e vínculos explícitos aumenta quando uma informação é compartilhada por vários requisitos, reutilizada em diferentes contextos, possui relevância regulatória, precisa ser mantida por responsáveis distintos ou pode sofrer alterações com impactos em diferentes partes do produto. Nesses casos, separar o elemento e estabelecer relações pode reduzir duplicação e facilitar a análise de impacto. Em situações mais simples e locais, manter informações complementares juntas pode produzir uma estrutura mais compreensível e menos burocrática.

Parte dessas relações pode constituir rastreabilidade, mas sua preservação não exige necessariamente matrizes ou mecanismos complexos. Um vínculo explícito pode ser necessário em alguns contextos; em outros, a própria estrutura em que as informações estão reunidas já preserva suficientemente a relação. A Seção 12.8 aprofundará quais vínculos merecem ser mantidos, em que direções e com que nível de esforço. Por ora, importa compreender que organizar requisitos significa tornar recuperáveis as informações e relações necessárias para entender e utilizar o escopo, independentemente de estarem distribuídas em vários registros ou reunidas em uma mesma estrutura.

## 12.6 Atualizar requisitos: manter vivo o entendimento

Um escopo bem organizado representa o entendimento disponível em determinado momento, mas esse entendimento não permanece necessariamente estável. Novas descobertas podem revelar necessidades antes desconhecidas; decisões de negócio podem alterar aquilo que será atendido; atividades de Verificação e Validação podem apontar inadequações; regras e normas

podem mudar; restrições técnicas podem tornar alternativas inviáveis; e a experiência de uso pode produzir evidências que modificam expectativas sobre o produto. A mudança faz parte do desenvolvimento e da evolução de software, e seu tratamento exige que requisitos e informações relacionadas permaneçam coerentes com aquilo que passou a ser compreendido e decidido (WIEGERS; BEATTY, 2013; SOMMERVILLE, 2019).

Atualizar requisitos, nesse contexto, significa **incorporar ao escopo organizado os efeitos dessas mudanças de entendimento e decisões**. Isso pode envolver modificar informações existentes, incluir novos requisitos, retirar itens que deixaram de fazer parte do escopo, alterar relações, substituir referências, atualizar atributos, registrar uma nova versão ou adequar as estruturas utilizadas para tornar o escopo visível. O objetivo não é preservar indefinidamente a forma anterior do escopo, mas evitar que aquilo que está registrado permaneça representando um entendimento que já deixou de vigorar.

Como discutido na Seção 9.8.3, nem toda alteração em uma declaração significa que o requisito mudou. Uma declaração pode ganhar condições, exceções ou maior precisão sem que seu núcleo de sentido seja alterado; em outros casos, a própria expectativa sobre o comportamento do produto é modificada. Para a Organização e Atualização, a consequência dessa distinção é prática: qualquer alteração que passe a fazer parte do entendimento vigente precisa ser refletida nos pontos do escopo em que se tornou relevante, preservando as relações que continuam válidas e re- vendo aquelas que deixaram de ser.

Isso é particularmente importante porque uma mudança raramente permanece confinada ao registro em que foi inicialmente percebida. Considere que, no sistema de agendamento, uma decisão altere a regra de cancelamento de **até duas horas antes da consulta** para **até quatro horas antes**. Atualizar apenas o texto da RN01 pode deixar o conjunto inconsistente se o critério de aceitação continuar utilizando duas horas, uma mensagem de interface informar o prazo antigo ou uma representação do fluxo mantiver a condição anterior. Dependendo da forma como o escopo está organizado, outros requisitos ou entregas também podem precisar ser revistos. A análise de impacto e os vínculos entre informações ajudam justamente a localizar os elementos potencialmente afetados por uma mudança (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

A Figura 86 ilustra essa propagação onde, a mudança surge em um ponto específico do entendimento, mas sua incorporação exige verificar o local que aquela informação também está representada, referenciada ou utilizada.

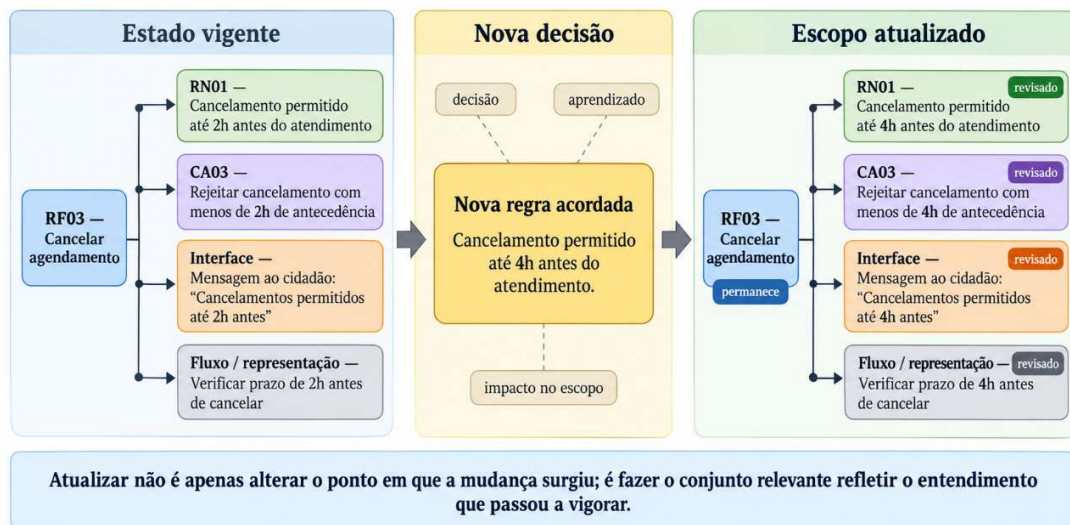


Figura 86 – Propagação de uma mudança pelo escopo organizado de requisitos.  
Fonte: elaborado pelo autor (2026).

A atualização também precisa alcançar a própria **estrutura de organização**. Se um requisito foi retirado do escopo, sua presença em uma visão de release precisa ser revista; se requisitos antes independentes passaram a ser tratados conjuntamente, agrupamentos ou relações podem precisar mudar; se uma prioridade foi renegociada, o valor registrado deve refletir a nova decisão; se uma regra compartilhada foi substituída, os vínculos para sua versão anterior não devem continuar aparentando validade. Em ambientes que utilizam versões ou baselines, a mudança também pode produzir uma nova referência do conjunto. A literatura da ER destaca a necessidade de manter atualizados os produtos de trabalho afetados e as informações de rastreabilidade depois que uma mudança é incorporada (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

Esse cuidado não pressupõe que toda alteração exija um processo formal de controle de mudanças. A intensidade da atualização depende do contexto, da criticidade, da quantidade de pessoas e equipes envolvidas e do impacto da alteração. Uma correção local pode exigir apenas a atualização de um registro e das informações que o acompanham; uma mudança regulatória pode afetar diversos requisitos, regras, critérios, representações e entregas. O princípio é proporcional: **quanto maior o risco de diferentes partes do escopo permanecerem contraditórias ou desatualizadas, maior a necessidade de tornar explícitos os efeitos da mudança.**

A atualização também evita que o escopo cresça apenas por acumulação, ou seja, quando uma decisão estabelece que um requisito não será mais atendido, que dois itens representam o mesmo compromisso ou que determinada informação perdeu validade, a estrutura precisa refletir essa decisão. Isso pode significar retirar, substituir, consolidar ou arquivar registros, mantendo histórico quando ele for necessário para compreender decisões anteriores. O importante é distinguir **preservar memória** de **preservar elementos obsoletos como se ainda estivessem vigentes**.

Dessa forma, manter vivo o entendimento não significa aceitar qualquer mudança nem decidir, nesta atividade, quais mudanças devem ser aprovadas. Significa garantir que, uma vez que novas descobertas, decisões ou evidências alterem aquilo que passa a valer para o produto, o escopo organizado acompanhe esse novo entendimento. **Um conjunto atualizado reduz a distância**



entre aquilo que as pessoas acreditam ter decidido e aquilo que os registros efetivamente comunicam. A Figura 87 ilustra essa diferença ao contrastar uma situação em que informações dispersas e inconsistentes sustentam entendimentos distintos com outra em que o escopo organizado e atualizado permite reconhecer o que está vigente e quais informações sustentam as decisões atuais.

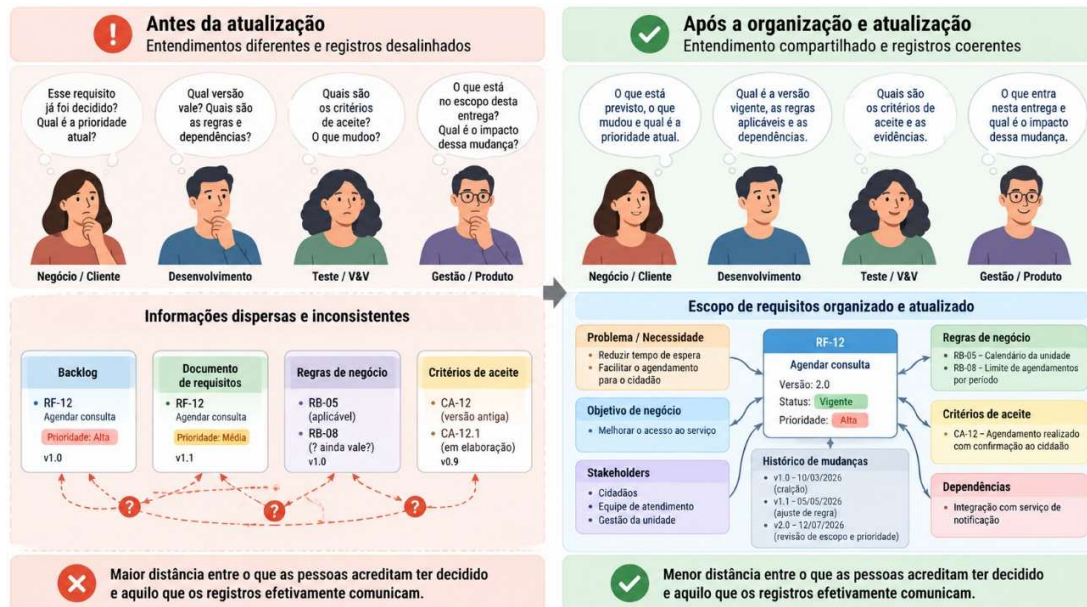


Figura 87 – Relação entre o entendimento das pessoas e as informações comunicadas pelo escopo de requisitos.  
Fonte: elaborado pelo autor (2026).

## 12.7 Movimentos recorrentes de cuidado com o escopo

As seções anteriores distinguiram dois aspectos complementares desta atividade: organizar o escopo para que seus elementos possam ser compreendidos e relacionados e atualizá-lo para que continue representando o entendimento vigente. Na prática, esse cuidado não acontece em um único momento nem segue uma sequência fixa. Ele reaparece sempre que novas informações, decisões, refinamentos ou mudanças exigem que a estrutura existente continue útil.

Esses movimentos também não significam que a Organização e Atualização passa a assumir decisões realizadas em outras atividades. Quando uma prioridade é acordada em Análise e Consenso, por exemplo, ela precisa permanecer visível no escopo; quando uma declaração é refinada, sua posição e suas relações podem precisar ser atualizadas; quando a Verificação e Validação produz um achado que leva a uma mudança acordada, seus efeitos precisam ser incorporados ao conjunto de requisitos. O cuidado com o escopo consiste, portanto, em fazer com que aquilo que passou a vigorar continue reconhecível nas estruturas utilizadas pelo produto.

O Quadro 71 sintetiza sete movimentos que ajudam a observar esse cuidado no trabalho cotidiano. Eles não formam etapas obrigatórias e podem ocorrer simultaneamente, com intensidades diferentes conforme o produto, a organização e o contexto.

| Movimento                                     | Cuidado com o escopo                                                                                                                                               | Exemplo no sistema de agendamento                                                                                               |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Dar visibilidade ao escopo</b>             | Tornar reconhecível o que está sendo considerado, o que permanece em discussão, o que foi selecionado para uma entrega e o que deixou de vigorar.                  | Cancelar agendamento permanece no escopo de requisitos do produto, mas está previsto para uma entrega posterior.                |
| <b>Organizar por relações úteis</b>           | Agrupar, relacionar e oferecer visões que ajudem diferentes pessoas a localizar e compreender informações relevantes.                                              | RF01, RF02 e RF03 aparecem em Gestão de agendamentos, enquanto RNF01 é mantido como requisito transversal relacionado aos três. |
| <b>Manter decisões de prioridade visíveis</b> | Registrar e refletir a prioridade que passou a vigorar, atualizando filtros, ordenações ou visões quando ela for renegociada.                                      | RF03 passa a aparecer com prioridade alta depois de uma nova decisão de negócio.                                                |
| <b>Refletir refinamentos na estrutura</b>     | Atualizar agrupamentos, relações e visões quando requisitos se tornam mais específicos, são desdobrados ou passam a ser compreendidos em outro nível de abstração. | Um requisito mais amplo de Gerenciar agendamentos permanece relacionado a RF01 — Registrar, RF02 — Consultar e RF03 — Cancelar. |
| <b>Incorporar aprendizagem e mudanças</b>     | Fazer com que novas decisões, evidências e mudanças de contexto sejam refletidas nos elementos afetados.                                                           | A alteração do prazo de cancelamento de duas para quatro horas atualiza RN01, CA03, mensagens e representação do fluxo.         |
| <b>Limpar o escopo</b>                        | Refletir decisões que tornaram registros obsoletos, duplicados ou desnecessários, retirando-os das visões vigentes e preservando histórico quando necessário.      | A versão anterior de RN01 deixa de aparecer como vigente, mas pode permanecer disponível no histórico.                          |
| <b>Preservar memória suficiente</b>           | Manter razões, decisões, relações e histórico na medida necessária para que o escopo continue compreensível e utilizável.                                          | O novo prazo de cancelamento permanece associado à decisão que o estabeleceu e aos elementos afetados.                          |

Quadro 71 – Movimentos recorrentes de cuidado com o escopo de requisitos.

Fonte: elaborado pelo autor (2026).

Os movimentos apresentados no Quadro 71 não constituem etapas sucessivas, mas diferentes formas de cuidar de um mesmo escopo ao longo de sua utilização e evolução, como sintetiza a Figura 88. Em determinado momento, alguns deles podem ser mais relevantes que outros, e vários podem ocorrer simultaneamente.

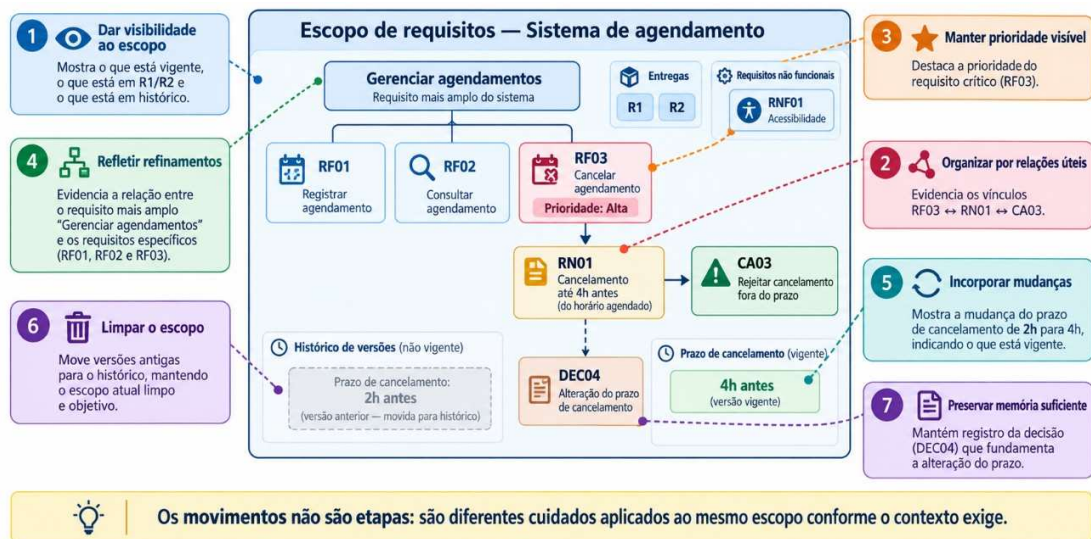


Figura 88 – Movimentos recorrentes de cuidado aplicados ao mesmo escopo de requisitos.  
Fonte: elaborado pelo autor (2026).

Os movimentos do Quadro 71 podem ser realizados com diferentes níveis de formalidade e, nesse sentido, em uma equipe pequena, atualizar uma prioridade, um vínculo e uma visão de entrega pode ser suficiente. Em um produto regulado ou distribuído entre várias equipes, a mesma mudança pode exigir atualização de versões, relações formais, justificativas, evidências e registros de decisão. O critério não é executar sempre o maior número possível de controles, mas impedir que informações relevantes deixem de representar aquilo que passou a valer.

O exemplo do prazo de cancelamento permite observar esses movimentos de forma integrada. Antes da mudança, RF03, RN01 e CA03 já ocupavam posições e mantinham relações no escopo. Quando uma nova decisão altera o prazo de duas para quatro horas, o trabalho não consiste em “executar sete passos”: a equipe atualiza a regra e as informações afetadas, preserva os vínculos ainda válidos, retira das visões correntes aquilo que ficou obsoleto, mantém a prioridade vigente e registra a razão da mudança quando essa memória for relevante. Os movimentos representam, portanto, diferentes perspectivas sobre um mesmo cuidado contínuo com o conjunto.

A memória necessária para realizar esse cuidado pode ser preservada por versões, atributos, justificativas, registros de decisão, relações explícitas ou pela própria estrutura em que informações complementares estão organizadas. Entre esses mecanismos, a rastreabilidade merece atenção específica porque permite seguir relações relevantes entre origem, requisito, decisão, evidência, mudança e impacto. A próxima seção aprofunda essa questão.

## 12.8 Rastreabilidade como memória suficiente

Preservar memória suficiente exige que algumas relações continuem recuperáveis mesmo quando o tempo passa, pessoas mudam de equipe ou o produto evolui. É nesse contexto que a rastreabilidade se torna particularmente útil: ela permite seguir relações entre requisitos e outros elementos relevantes para compreender sua origem, suas dependências, os elementos que os satisfazem ou avaliam e os efeitos produzidos por mudanças. A literatura da ER destaca justamente a possibilidade de seguir a vida de um requisito para trás e para frente, desde sua origem até

elementos posteriores do desenvolvimento, bem como de preservar relações entre requisitos, regras, testes e outros elementos do produto (WIEGERS; BEATTY, 2013).

A diferença entre preservar memória e simplesmente acumular registros pode ser melhor compreendida quando se observa o efeito dessas escolhas sobre as pessoas que dependem do escopo. A Figura 89 contrapõe duas situações: em uma, o histórico e as relações relevantes permanecem recuperáveis; em outra, registros obsoletos e vínculos desatualizados dificultam a compreensão do que está vigente.



Figura 89 – Preservar memória suficiente não é o mesmo que manter registros obsoletos como se ainda estivessem vigentes.  
Fonte: elaborado pelo autor (2026).

Essa compreensão vai além da simples existência de links, conforme destacam Dick, Hull e Jackson (2017) ao reportar que a rastreabilidade pode preservar também intenção e racionalidade, permitindo recuperar não apenas **o que está relacionado**, mas **por que determinada relação ou decisão existe**. Essa dimensão é especialmente importante quando o conhecimento que sustentou uma decisão poderia permanecer apenas na memória das pessoas e tornar-se difícil de reconstruir posteriormente.

No contexto deste capítulo, tratar rastreabilidade como **memória suficiente** significa preservar as relações necessárias para que perguntas relevantes possam continuar sendo respondidas. Diante de um requisito, pode ser necessário compreender de onde ele surgiu, que regra o condiciona, de quais outros requisitos depende, que critério ajuda a avaliá-lo, que decisão alterou seu conteúdo, em qual entrega está considerado ou que elementos precisam ser revistos caso ele mude. Nem todas essas relações precisam ser registradas em todos os produtos; sua utilidade depende das decisões e riscos existentes no contexto.

Essa memória também não exige que cada informação esteja armazenada separadamente. Como discutido na Seção 12.5, um critério de aceitação pode estar incorporado à própria história de usuário e uma regra pode fazer parte da especificação de um caso de uso. Nesses casos, parte da



relação já é preservada pela própria estrutura do artefato. Quando as informações estão separadas, são compartilhadas por vários requisitos ou precisam ser recuperadas independentemente, identificadores, referências, links ou outros mecanismos podem tornar a relação explícita. O que caracteriza a rastreabilidade não é a utilização de uma matriz específica, mas a possibilidade de seguir uma relação necessária sem depender exclusivamente da memória das pessoas.

No sistema de agendamento, por exemplo, o item **RF03 — Cancelar agendamento** pode manter registrada a relação com a **RN01 — Cancelamento permitido até quatro horas antes**, enquanto seus critérios de aceitação podem permanecer no próprio item ou na especificação que o descreve. A decisão **DEC04**, responsável pela alteração do prazo, pode ser referenciada quando for importante preservar a justificativa dessa mudança, e a indicação de que RF03 está previsto para determinada entrega pode aparecer como um atributo. Nesse arranjo, não é necessário construir uma grande matriz para responder a perguntas como “qual regra condiciona RF03?”, “por que o prazo é de quatro horas?” ou “em que entrega esse requisito está considerado?”.

A Figura 90 mostra esse exemplo e evidencia que uma rastreabilidade útil pode ser formada por poucos vínculos deliberadamente escolhidos.

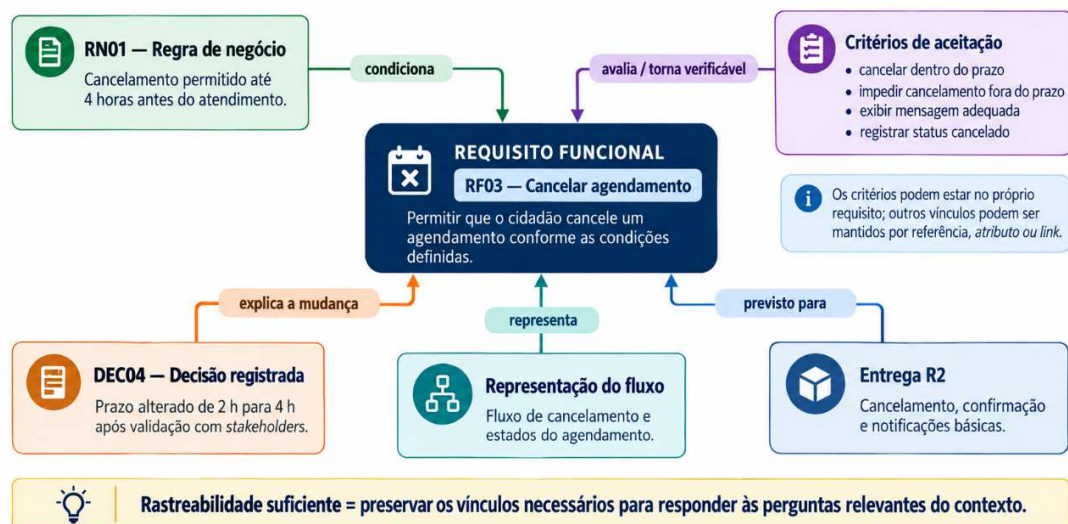


Figura 90 – Exemplo de rastreabilidade suficiente no sistema de agendamento.  
Fonte: elaborado pelo autor (2026).

O exemplo da Figura 90 também ajuda a perceber diferentes sentidos em que uma relação pode ser percorrida. A partir de RF03, pode-se retornar a informações que ajudam a explicar sua origem ou as decisões que o condicionam; seguir para critérios, testes, representações ou entregas que dependem dele; reconhecer relações com outros requisitos e regras; ou reconstruir alterações que explicam como o estado atual foi alcançado. Na literatura, a rastreabilidade para frente e para trás é utilizada justamente para compreender a origem dos requisitos e seus desdobramentos posteriores, além de apoiar análise de impacto e identificação de dependências (WIEGERS; BE-ATTY, 2013).

Em alguns contextos, a própria estrutura utilizada para declarar um requisito já conserva parte da memória necessária, enquanto, em outros, campos, identificadores, hiperlinks ou relações

entre itens oferecem apoio suficiente para recuperar as informações relevantes. Matrizes e repositórios especializados tornam-se mais úteis quando o volume de elementos, a quantidade de relações, a criticidade do produto ou as exigências de auditoria dificultam essa recuperação por mecanismos mais simples. O Quadro 72 apresenta essas possibilidades como formas progressivamente mais estruturadas de preservar rastreabilidade, sem pressupor que maior formalidade seja sempre a alternativa mais adequada.

| Forma                                          | Como preserva a relação                                                                                     | Exemplo                                                                                                           |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Informações reunidas no mesmo artefato</b>  | A própria estrutura mantém próximas informações que precisam ser interpretadas em conjunto.                 | Critérios de aceitação registrados junto à história de usuário; regra registrada na especificação do caso de uso. |
| <b>Referências e identificadores</b>           | Um elemento menciona explicitamente outro elemento identificável.                                           | RF03 informa que é condicionado por RN01 e que foi afetado pela DEC04.                                            |
| <b>Atributos e links entre itens</b>           | Campos ou vínculos permitem navegar entre informações mantidas separadamente.                               | RF03 relacionado à entrega R2 e a uma representação do fluxo de cancelamento.                                     |
| <b>Matrizes ou repositórios especializados</b> | Relações entre grandes conjuntos de elementos podem ser consultadas, filtradas e mantidas sistematicamente. | Requisitos regulatórios relacionados a regras, evidências, testes, versões e aprovações.                          |

Quadro 72 – Formas de preservar rastreabilidade conforme a necessidade.

Fonte: elaborado pelo autor (2026), com base em Wieggers e Beatty (2013) e Dick, Hull e Jackson (2017).

A escolha entre essas formas deve considerar o benefício esperado e o custo de manutenção. Wieggers e Beatty (2013) observam que não é necessário estabelecer todos os vínculos possíveis e recomendam selecionar aqueles que efetivamente contribuirão para o projeto e para a manutenção futura. Registrar relações que ninguém utiliza acrescenta esforço sem produzir memória útil; deixar de registrar relações cuja perda compromete análise de impacto, manutenção, conformidade ou entendimento transfere esse custo para o futuro.

Em produtos pequenos, com poucas pessoas envolvidas e baixo risco, referências simples entre requisitos, regras, critérios e decisões podem ser suficientes para preservar a memória necessária. À medida que aumentam a criticidade, o tempo de vida do produto, o número de equipes, as dependências ou as exigências regulatórias, pode ser necessário manter relações mais formais entre requisitos, regras, evidências, testes, versões e aprovações. Nesses contextos, a rastreabilidade pode apoiar tanto a análise de impacto e a manutenção quanto a demonstração de atendimento a requisitos regulatórios ou críticos (WIEGERS; BEATTY, 2013).

A utilidade dessa memória depende, por fim, de ela permanecer coerente com o escopo que pretende explicar. Links, referências ou matrizes que continuam formalmente registrados depois que requisitos, regras, versões ou decisões mudaram podem conduzir as pessoas a conclusões incorretas. Informações de rastreabilidade obsoletas não são neutras: além de perderem valor, podem reduzir a confiança da equipe no mecanismo utilizado. Por isso, quando uma atualização altera elementos relacionados, os vínculos relevantes também precisam ser revistos e, quando necessário, criados, modificados ou retirados (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

Encontrar esse equilíbrio significa evitar tanto a criação indiscriminada de vínculos quanto a dependência excessiva da memória individual. Rastrear tudo pode produzir ruído, esforço de manutenção e relações que ninguém consulta; rastrear pouco demais pode dificultar a compreensão de impactos, tornar decisões antigas irrecuperáveis e fazer com que o conhecimento sobre o produto fique concentrado em pessoas específicas. A memória suficiente situa-se entre esses extremos: preserva aquilo que precisa continuar recuperável para compreender, decidir e evoluir o produto sem transformar a rastreabilidade em burocracia.

## 12.9 Organização e atualização em diferentes contextos

A forma de organizar e atualizar requisitos precisa ser proporcional às características do produto, do projeto, da organização e do domínio. Criticidade, exigências regulatórias, duração esperada do produto, quantidade de equipes, distribuição dos participantes, estabilidade do contexto, compromissos contratuais e necessidade de auditoria influenciam quanto de estrutura, versionamento, histórico e rastreabilidade precisa ser preservado. A mesma preocupação — manter o escopo compreensível e coerente com o entendimento vigente — pode, portanto, ser atendida por mecanismos bastante diferentes.

Em produtos ou projetos que exigem maior previsibilidade, possuem compromissos contratuais relevantes ou dependem de aprovações formais, conjuntos versionados, baselines, estados de aprovação e mecanismos mais explícitos de controle de mudanças podem ser necessários. Nesses ambientes, preservar referências reconhecíveis do escopo ajuda a compreender o que foi acordado, identificar alterações posteriores e avaliar seus impactos. O cuidado necessário está em não transformar estabilidade de uma referência em pressuposto de imutabilidade: requisitos continuam sujeitos a revisão quando o contexto ou o entendimento muda (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

Em contextos nos quais o produto evolui de maneira incremental e novas informações são incorporadas com frequência, estruturas mais leves e facilmente atualizáveis podem favorecer o trabalho cotidiano. No Scrum, por exemplo, o Product Backlog é uma lista ordenada e emergente do que é necessário para melhorar o produto, e seus itens podem ser decompostos e definidos com maior precisão durante o refinamento (SCHWABER; SUTHERLAND, 2020). Como discutido na Seção 12.4, isso não transforma o Product Backlog em sinônimo do escopo de requisitos: ele organiza trabalho relacionado ao produto e pode materializar parte importante desse escopo. A leveza da estrutura também não elimina a necessidade de preservar relações, decisões ou requisitos transversais cuja perda comprometeria o entendimento.

Quando compromissos relativamente estáveis coexistem com partes do produto sujeitas a maior aprendizagem e mudança, diferentes formas de organização podem ser combinadas. Requisitos regulatórios ou contratuais podem permanecer associados a referências versionadas, enquanto funcionalidades e melhorias evoluem por meio de backlogs, entregas sucessivas ou outras estruturas mais dinâmicas. A coerência não depende de obrigar todo o escopo a utilizar o mesmo grau de formalidade, mas de permitir que essas diferentes estruturas continuem relacionadas e que as pessoas saibam qual informação está vigente e para que finalidade cada estrutura é utilizada.

Em produtos regulados, críticos ou de alta consequência, a necessidade de demonstrar atendimento, reconstruir decisões e avaliar impactos tende a exigir relações mais explícitas entre requisitos, regras, restrições, evidências, verificações, versões e aprovações. Baselines, histórico de mudanças e mecanismos formais de rastreabilidade podem ganhar importância não porque maior documentação seja desejável em si mesma, mas porque a perda de determinada informação pode comprometer segurança, conformidade, auditoria ou capacidade de evolução. Ainda nesses contextos, o grau de formalidade deve responder ao risco e à finalidade da informação, evitando produzir registros sem uso efetivo (WIEGERS; BEATTY, 2013; DICK; HULL; JACKSON, 2017).

À medida que o produto passa a envolver várias equipes, unidades organizacionais ou fornecedores, cresce também o risco de fragmentação do escopo. Agrupamentos por áreas de requisitos, estruturas em diferentes níveis, visões compartilhadas e mecanismos para tornar dependências visíveis podem apoiar a coordenação sem exigir que todas as pessoas trabalhem sobre a mesma visão detalhada. Abordagens de desenvolvimento em escala utilizam diferentes mecanismos para lidar com essa necessidade, desde estruturas hierárquicas de itens até áreas de requisitos e backlogs compartilhados (LEFFINGWELL, 2011; LARMAN; VODDE, 2016). O desafio está em introduzir apenas as camadas que ajudam a compreender e coordenar o produto, pois uma estrutura excessivamente fragmentada pode tornar o escopo administrável nos sistemas de gestão e, ao mesmo tempo, difícil de compreender por quem toma decisões.

Essas diferenças são sintetizadas no Quadro 73, que relaciona características do contexto aos cuidados que tendem a ganhar maior relevância. As relações apresentadas não constituem prescrições: um mesmo produto pode combinar várias dessas condições, e a forma de organização precisa ser ajustada conforme elas se alteram.

| Característica do contexto                                          | Cuidados que ganham relevância                                                                  | Possíveis formas de apoio                                                                                     |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Maior necessidade de previsibilidade ou compromissos formais</b> | Preservar referências acordadas, versões e mudanças reconhecíveis.                              | Baselines, conjuntos versionados, estados de aprovação, histórico de alterações.                              |
| <b>Maior frequência de aprendizagem e mudança</b>                   | Permitir atualização frequente sem perder decisões e relações importantes.                      | Backlogs, visões de entrega, links, atributos e registros leves de decisão.                                   |
| <b>Coexistência de partes estáveis e evolutivas</b>                 | Manter diferentes níveis de formalidade relacionados e distinguir qual informação está vigente. | Baselines combinadas com backlogs, versões e visões complementares.                                           |
| <b>Alta criticidade ou regulação</b>                                | Demonstrar origem, atendimento, evidências e impactos de mudanças.                              | Rastreabilidade explícita, versões, aprovações, evidências e registros de decisão.                            |
| <b>Múltiplas equipes, unidades ou fornecedores</b>                  | Evitar fragmentação e tornar dependências e responsabilidades recuperáveis.                     | Áreas de requisitos, visões compartilhadas, agrupamentos, estruturas em níveis e mecanismos de sincronização. |



| Característica do contexto                           | Cuidados que ganham relevância                                                                | Possíveis formas de apoio                                                   |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Produto pequeno, equipe próxima e baixo risco</b> | Preservar apenas a memória necessária para que decisões e relações importantes não se percam. | Identificadores simples, referências, atributos mínimos e registros locais. |

Quadro 73 – Calibragem da organização e atualização conforme o contexto.

Fonte: elaborado pelo autor (2026), com base em Wieggers e Beatty (2013), Dick, Hull e Jackson (2017), Schwaber e Sutherland (2020), Lef-fingwell (2011) e Larman e Vodde (2016).

Essa calibragem pode ser compreendida como um ajuste contínuo entre as características do contexto e a quantidade de estrutura necessária para preservar entendimento, como ilustra a Figura 91. O aumento de formalidade representa uma resposta possível quando a perda de informação, a dificuldade de coordenação ou o impacto de uma mudança tornam mecanismos adicionais justificáveis.

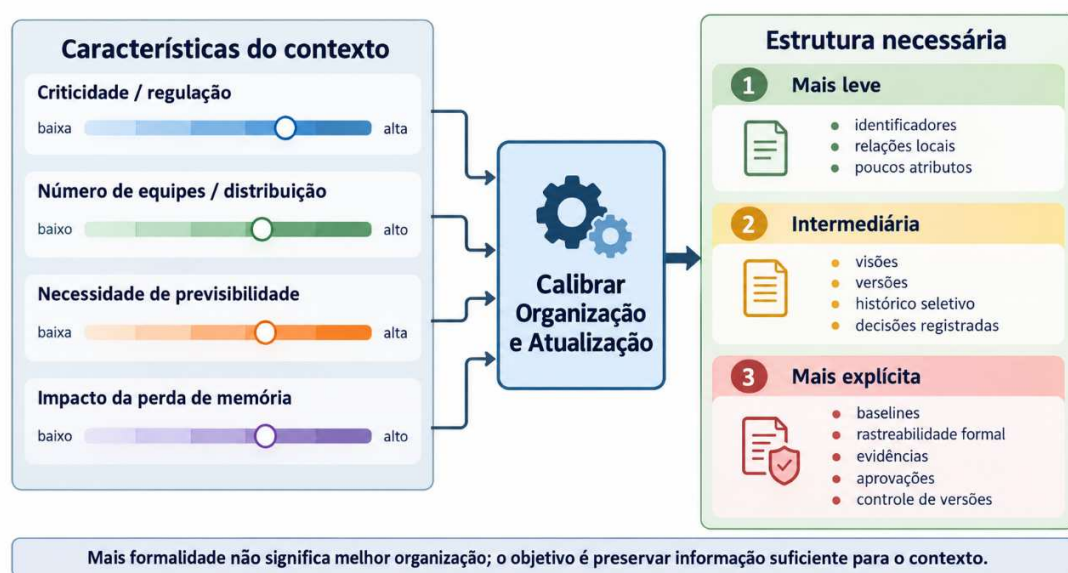


Figura 91 – Calibragem da Organização e Atualização conforme as características do contexto.

Fonte: elaborado pelo autor (2026).

Uma solução inicialmente pequena pode passar a atender múltiplas unidades; um produto experimental pode posteriormente ficar sujeito a regulação; uma iniciativa contratual pode adotar entregas incrementais; e uma equipe que antes dependia de comunicação informal pode precisar preservar mais memória à medida que pessoas e responsabilidades se distribuem. Como essas condições podem coexistir e se modificar ao longo da vida do produto, a calibragem da Organização e Atualização também precisa evoluir, ampliando ou reduzindo mecanismos de controle conforme mudam a complexidade, o risco e a quantidade de informação que precisa continuar recuperável.

## 12.10 Exemplo aplicado ao sistema de agendamento

No sistema de agendamento de consultas públicas utilizado ao longo do livro, considere agora uma situação que ainda não fazia parte do escopo de requisitos. O cidadão pode consultar

horários disponíveis e registrar um agendamento, mas, quando não encontra vaga para o serviço desejado, precisa retornar posteriormente ao sistema e realizar uma nova consulta. Depois de algum tempo de uso, dados das unidades de atendimento e relatos de usuários indicam que vagas liberadas posteriormente nem sempre são reaproveitadas, enquanto cidadãos interessados continuam sem encontrar horários disponíveis.

A análise dessa situação com representantes das unidades de atendimento e usuários leva a uma nova decisão sobre o produto: quando não houver horário disponível para determinado serviço, o cidadão poderá solicitar sua inclusão em uma **lista de espera**. Quando uma vaga compatível for liberada, o sistema deverá oferecê-la ao próximo cidadão elegível, que terá um período determinado para aceitá-la antes que a oportunidade seja encaminhada ao próximo interessado. A partir dessa decisão, o entendimento sobre aquilo que o produto precisa atender se amplia e o escopo de requisitos precisa incorporar essa nova realidade.

A atualização do escopo, nesse caso, não consiste apenas em acrescentar um novo item a um backlog ou catálogo. O novo comportamento precisa ser posicionado em relação ao que já existe, regras precisam estabelecer como a lista funcionará, critérios devem permitir avaliar os comportamentos esperados, notificações passam a participar de um novo contexto, estados adicionais podem precisar ser representados e os novos requisitos precisam ser associados à entrega em que serão considerados. Ao mesmo tempo, elementos já existentes que continuam válidos não precisam ser reescritos apenas porque passaram a participar de novas relações.

Para fins do exemplo, podem ser reconhecidos dois novos comportamentos funcionais: **Solicitar inclusão em lista de espera**, que permite ao cidadão manifestar interesse quando não houver vaga disponível, e **Ofertar vaga liberada**, que permite ao sistema oferecer uma vaga compatível ao próximo cidadão elegível. Esses comportamentos não aparecem isoladamente no escopo, pois dependem da consulta de disponibilidade já existente, utilizam mecanismos de notificação e precisam ser condicionados por regras que definam a ordenação da lista e o tempo durante o qual uma vaga oferecida permanece reservada.

Uma possível regra estabelece que os cidadãos sejam atendidos segundo a ordem de entrada na lista para o mesmo serviço e unidade, respeitadas condições específicas definidas pela organização. Outra regra pode determinar que uma vaga oferecida permaneça disponível por determinado período antes de ser encaminhada à próxima pessoa da lista. Essas regras podem ser registradas separadamente quando são reutilizadas ou precisam ser referenciadas por diferentes comportamentos, ou permanecer junto às especificações correspondentes quando essa forma favorecer a compreensão, conforme discutido nas seções anteriores.

Os critérios de aceitação associados aos novos comportamentos podem explicitar situações como permitir a inclusão na lista apenas quando não houver vaga disponível, impedir inscrições duplicadas para o mesmo serviço quando essa for a política adotada, ofertar uma vaga ao próximo cidadão elegível, registrar o prazo para resposta, considerar a vaga aceita quando o cidadão confirmar dentro desse período e oferecê-la ao próximo interessado quando o prazo expirar. Esses critérios podem permanecer integrados às histórias de usuário ou às especificações correspondentes, sem necessidade de se tornarem elementos independentes no repositório.

Ao incorporar a lista de espera, a organização do escopo também precisa tornar perceptível como os novos elementos se relacionam com o restante do produto. A consulta de disponibilidade passa a poder conduzir à solicitação de entrada na lista; uma vaga liberada pode iniciar o processo de oferta; o mecanismo de notificação passa a comunicar a existência e a expiração de ofertas; requisitos não funcionais de acessibilidade, privacidade, disponibilidade ou desempenho podem continuar aplicáveis; e as representações do fluxo de agendamento precisam incorporar estados como **aguardando vaga**, **vaga oferecida**, **oferta aceita** e **oferta expirada**, quando esses estados forem relevantes para a compreensão do comportamento.

O Quadro 74 sintetiza uma possível incorporação desse novo entendimento ao escopo organizado.

| Elemento                                     | Situação decorrente da nova decisão                                                                          | Tratamento no escopo                                                                                     |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Solicitar inclusão em lista de espera</b> | Novo comportamento esperado quando não houver horário disponível.                                            | Inserir como requisito funcional e relacioná-lo à consulta de disponibilidade.                           |
| <b>Ofertar vaga liberada</b>                 | Novo comportamento iniciado quando uma vaga compatível se torna disponível.                                  | Inserir como requisito funcional e relacioná-lo à liberação de vaga, à lista e à notificação.            |
| <b>Regra de ordenação da lista</b>           | Define quem deve receber primeiro uma vaga liberada.                                                         | Registrar junto à especificação ou separadamente quando for compartilhada por diferentes comportamentos. |
| <b>Regra de prazo para resposta</b>          | Define por quanto tempo a vaga ofertada permanece disponível.                                                | Relacionar à oferta, aos critérios e às informações apresentadas ao cidadão.                             |
| <b>Critérios de aceitação</b>                | Passam a contemplar inclusão, oferta, aceite, expiração e encaminhamento ao próximo interessado.             | Manter junto aos requisitos ou em estrutura relacionada, conforme a forma de declaração adotada.         |
| <b>Notificações</b>                          | Passam a comunicar oferta de vaga, prazo para resposta e eventual expiração.                                 | Relacionar o comportamento já existente ao novo contexto de uso.                                         |
| <b>Representações do fluxo</b>               | Precisam considerar espera, oferta, aceite e expiração.                                                      | Atualizar as representações afetadas sem alterar aquelas que permanecem válidas.                         |
| <b>Requisitos não funcionais aplicáveis</b>  | Acessibilidade, privacidade, desempenho ou disponibilidade podem também se aplicar aos novos comportamentos. | Estabelecer ou manter os vínculos necessários com os requisitos funcionais afetados.                     |
| <b>Escopo da entrega</b>                     | A lista de espera precisa ser posicionada em determinado horizonte de entrega.                               | Registrar a entrega que passou a vigorar após a decisão de planejamento correspondente.                  |
| <b>Decisão registrada</b>                    | Preserva a razão pela qual a lista de espera foi incorporada ao produto.                                     | Relacionar a decisão aos novos requisitos quando essa memória for relevante para o contexto.             |

Quadro 74 – Incorporação da lista de espera ao escopo de requisitos do sistema de agendamento.

Fonte: elaborado pelo autor (2026).

O Quadro 74 evidencia que uma ampliação do escopo não implica substituir ou reorganizar indiscriminadamente tudo aquilo que já estava registrado. A consulta de horários continua cumprindo sua finalidade, os requisitos de qualidade que permanecem aplicáveis podem apenas receber novas relações e o mecanismo de notificação pode continuar sendo o mesmo, embora passe a atender outro comportamento. Novos elementos são introduzidos apenas quando representam algo que passou efetivamente a fazer parte do entendimento sobre o produto.

A identificação dos impactos também depende da organização já existente. Quando requisitos, regras, representações, entregas e demais informações relevantes podem ser localizados e relacionados, torna-se mais fácil reconhecer onde o novo comportamento precisa ser incorporado. Os vínculos existentes não substituem a análise realizada pelas pessoas, mas ajudam a evitar que uma ampliação aparentemente localizada resulte em partes do produto ou do escopo que continuam expressando entendimentos incompatíveis.

A Figura 92 sintetiza esse exemplo mostrando como uma situação inicialmente tratada apenas como ausência de vagas produz uma nova decisão e, posteriormente, um escopo no qual requisitos, regras, critérios, notificações, representações e entrega passam a formar um novo conjunto de relações.

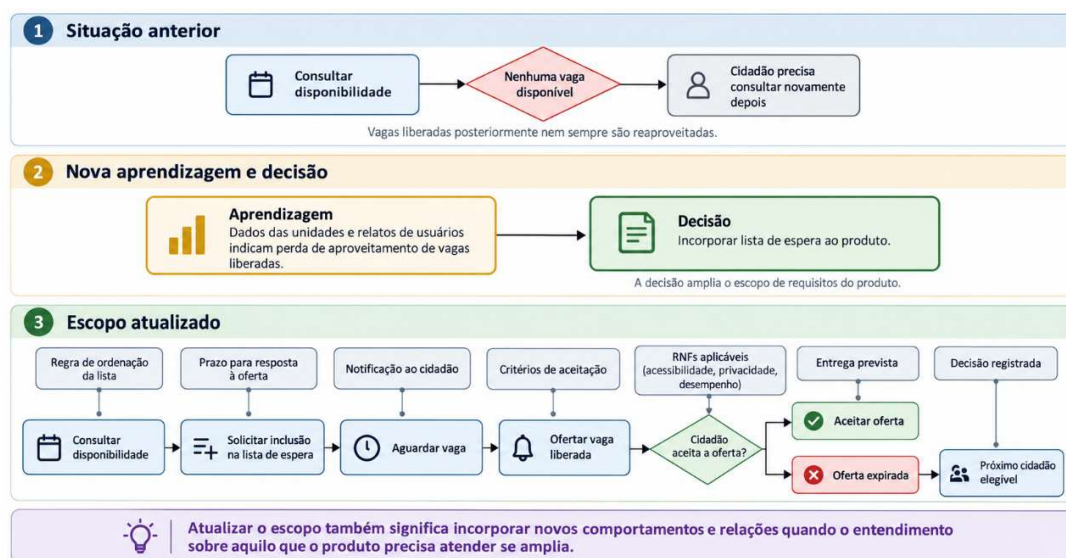


Figura 92 – Incorporação da lista de espera ao escopo de requisitos do sistema de agendamento.  
Fonte: elaborado pelo autor (2026).

Na situação representada na Figura 92, a mudança não decorre da simples reformulação de um requisito anterior, mas da incorporação de um entendimento que amplia aquilo que o produto deve atender. A lista de espera introduz novos comportamentos e regras, reutiliza elementos já existentes e modifica relações dentro do escopo, enquanto informações que não foram afetadas continuam válidas. A Organização e Atualização permite que essa ampliação deixe de existir apenas como uma decisão conhecida pelas pessoas envolvidas e passe a ser reconhecível nas estruturas utilizadas para compreender e conduzir o produto.

A memória dessa evolução pode ser preservada de forma proporcional ao contexto. Em um projeto pequeno, registrar os novos requisitos, suas regras, a entrega prevista e a justificativa principal pode ser suficiente; em um ambiente com múltiplas unidades, regras institucionais ou necessidade de auditoria, também podem ser necessários vínculos com evidências, versões, responsáveis e decisões de aprovação. Em ambos os casos, o objetivo permanece o mesmo: permitir que o escopo atualizado comunique aquilo que o produto passou a precisar atender e forneça caminhos suficientes para compreender as relações que sustentam esse novo entendimento.

O exemplo da lista de espera também evidencia a natureza articulada das atividades da Engenharia de Requisitos. A necessidade pode emergir de dados, observações ou feedback; seu significado e sua viabilidade precisam ser analisados e acordados; os novos requisitos precisam ser declarados e, quando útil, representados; critérios e outras evidências ajudam em sua Verificação e Validação; e a Organização e Atualização incorpora ao escopo aquilo que passou a vigorar, preservando relações e memória na medida necessária. Essas atividades podem ocorrer de forma iterativa e sobreposta, enquanto o escopo organizado funciona como uma referência compartilhada que acompanha a evolução do entendimento sobre o produto.

## 12.11 Riscos e armadilhas na organização e atualização

Quando o cuidado com o escopo passa a ser medido pela quantidade de campos preenchidos, documentos produzidos, estados controlados ou vínculos registrados, a Organização e Atualização pode se transformar em burocracia sem melhorar efetivamente a compreensão do produto. Ferramentas, backlogs, matrizes e repositórios são úteis quando ajudam pessoas a localizar informações, compreender relações, avaliar impactos ou recuperar decisões; quando seu uso se torna um fim em si mesmo, a equipe pode manter muitos registros e, ainda assim, continuar sem uma visão confiável do escopo.

O acúmulo progressivo de itens também pode comprometer essa visão quando novos requisitos, ideias, solicitações, versões antigas e registros abandonados permanecem misturados sem indicação clara de seu estado. Com o tempo, torna-se difícil distinguir aquilo que continua vigente daquilo que foi substituído, rejeitado, adiado ou incorporado por outro elemento. O mesmo problema ocorre quando diferentes equipes mantêm versões parcialmente divergentes do escopo em documentos, planilhas, backlogs ou ferramentas distintas, produzindo várias referências concorrentes sobre o que o produto precisa atender.

Alterações realizadas apenas nos registros, sem que as pessoas afetadas compreendam aquilo que mudou, podem preservar uma aparência de atualização e, ao mesmo tempo, enfraquecer o entendimento compartilhado. Nem toda modificação exige nova reunião ou negociação, mas mudanças que alteram compromissos, significado, relações importantes ou expectativas de stakeholders precisam ser comunicadas de forma compatível com seu impacto. Atualizar o escopo não substitui a conversa necessária para que as pessoas saibam qual entendimento passou a vigorar.

Vínculos, históricos e registros de decisão também podem produzir uma falsa sensação de segurança quando permanecem formalmente presentes, mas deixam de representar relações válidas. Uma referência para uma regra substituída, um link para uma versão antiga ou uma justificativa

que já não explica a decisão atual podem induzir interpretações equivocadas. Como discutido na Seção 12.8, informações de rastreabilidade precisam acompanhar as mudanças que pretendem explicar; quando se tornam obsoletas ou imprecisas, além de perder valor, podem reduzir a confiança das pessoas no próprio mecanismo de rastreabilidade (WIEGERS; BEATTY, 2013).

O Quadro 75 sintetiza essas armadilhas e os cuidados que ajudam a reconhecê-las no trabalho cotidiano.

| Risco ou armadilha                                   | Como se manifesta                                                                                                   | Consequência possível                                                                          | Cuidado necessário                                                                    |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Burocratização da organização</b>                 | Campos, estados, documentos ou vínculos são mantidos sem uso claro.                                                 | Aumento de esforço sem ganho proporcional de compreensão ou decisão.                           | Manter apenas informações e mecanismos que tenham finalidade reconhecível.            |
| <b>Fragmentação do escopo</b>                        | Diferentes ferramentas ou equipes mantêm versões parcialmente divergentes.                                          | Pessoas passam a trabalhar com referências distintas sobre o produto.                          | Definir como identificar o estado vigente e como as diferentes versões se relacionam. |
| <b>Acúmulo de elementos obsoletos</b>                | Itens substituídos, rejeitados ou incorporados permanecem misturados aos vigentes.                                  | Dificuldade de compreender o escopo atual e risco de decisões baseadas em informações antigas. | Distinguir claramente o que está vigente do que permanece apenas como histórico.      |
| <b>Atualização sem comunicação suficiente</b>        | Registros mudam sem que stakeholders ou equipes afetadas compreendam a mudança.                                     | Divergência entre o que está registrado e o que as pessoas acreditam estar acordado.           | Comunicar mudanças proporcionalmente ao impacto sobre compromissos e entendimento.    |
| <b>Rastreabilidade desatualizada</b>                 | Links, referências ou justificativas continuam apontando para informações que mudaram.                              | Análise de impacto equivocada e falsa confiança na estrutura existente.                        | Revisar os vínculos afetados sempre que mudanças relevantes forem incorporadas.       |
| <b>Controle excessivo ou insuficiente da mudança</b> | O escopo é tratado como praticamente imutável ou, no extremo oposto, muda sem avaliação suficiente de seus efeitos. | Desalinhamento com a realidade ou instabilidade e perda de coerência.                          | Calibrar mecanismos de atualização conforme impacto, risco e necessidade de memória.  |

Quadro 75 – Riscos e cuidados na organização e atualização do escopo.

Fonte: elaborado pelo autor (2026).

Os riscos do Quadro 75 não são resolvidos simplesmente com mais documentação, mais reuniões ou ferramentas mais sofisticadas. Em muitos casos, aumentar os mecanismos de controle pode inclusive ampliar o problema caso eles não estejam associados a uma necessidade concreta. A adequação está em preservar estrutura, memória e comunicação na medida necessária para que o escopo continue utilizável pelas pessoas que dependem dele.

Entre congelar artificialmente o escopo e permitir que qualquer alteração seja incorporada sem compreender suas consequências, existe uma faixa ampla de práticas que pode ser ajustada ao contexto. Um produto pode precisar de referências estáveis sem tratar requisitos como imutáveis, assim como pode acolher aprendizagem contínua sem transformar o escopo em um conjunto instável de registros. Manter esse equilíbrio exige que Organização e Atualização acompanhe a evolução do produto sem perder de vista sua finalidade: preservar um escopo que continue compreensível, coerente com o entendimento vigente e suficientemente conectado às decisões que explicam sua evolução.

## 12.12 Síntese do capítulo

Ao desenvolver a Organização e Atualização como a sexta atividade da Engenharia de Requisitos, este capítulo concentrou-se no cuidado necessário para que o escopo de requisitos do produto permaneça estruturado e coerente com o entendimento vigente. Esse cuidado envolve dar forma ao conjunto de requisitos, tornar recuperáveis as informações e relações necessárias à sua compreensão e incorporar ao escopo os efeitos de novas descobertas, decisões, evidências e mudanças de contexto. Organizar e atualizar, portanto, não significa apenas armazenar requisitos ou editar registros, mas preservar condições para que diferentes pessoas continuem compreendendo e utilizando o escopo ao longo da evolução do produto.

A distinção entre **escopo do produto**, **escopo de requisitos do produto**, **escopo da entrega** e **escopo do projeto** permitiu separar diferentes objetos que frequentemente aparecem misturados na prática. O escopo do produto delimita, em nível mais amplo, aquilo que o produto pretende oferecer; o escopo de requisitos expressa, de forma organizada, aquilo que se entende que o produto precisa atender; o escopo da entrega seleciona uma parte desse entendimento para determinado horizonte; e o escopo do projeto delimita o trabalho necessário para conduzir a iniciativa. Backlogs, mapas de histórias, roadmaps, quadros de fluxo, baselines e outras estruturas não constituem novos tipos de escopo, mas diferentes formas de organizar, visualizar ou operacionalizar partes desses escopos e do trabalho relacionado ao produto.

Na organização do escopo, identificadores, atributos, agrupamentos, visões e relações podem ser utilizados conforme as necessidades do contexto para facilitar localização, compreensão e uso das informações. Essa organização não exige que cada regra, critério, decisão ou restrição seja transformada em um registro independente: informações complementares podem permanecer reunidas em uma história de usuário, caso de uso ou outra estrutura quando isso favorece a compreensão. Quando precisam ser reutilizadas, compartilhadas, mantidas separadamente ou recuperadas de forma independente, vínculos explícitos podem tornar suas relações mais visíveis. O princípio é preservar distinção de significado e possibilidade de recuperação, e não impor uma única forma documental ao escopo.

Quando o entendimento sobre o produto muda, a atualização precisa fazer com que o conjunto organizado passe a refletir aquilo que passou a vigorar. Uma nova decisão pode introduzir requisitos, modificar regras, alterar critérios, afetar representações, mudar relações ou tornar registros anteriores obsoletos, enquanto outros elementos permanecem válidos. Os movimentos recorrentes discutidos no capítulo — dar visibilidade, organizar por relações úteis, manter

decisões de prioridade visíveis, refletir refinamentos, incorporar mudanças, limpar o escopo e preservar memória suficiente — representam diferentes formas desse cuidado e não etapas sucessivas de um processo.

Entre os mecanismos que sustentam essa continuidade, a rastreabilidade foi tratada como **memória suficiente**, isto é, como a preservação das relações necessárias para compreender origem, condicionantes, decisões, dependências, evidências, mudanças e impactos na medida exigida pelo contexto. Essa memória pode estar incorporada à própria estrutura dos artefatos, ser mantida por referências e identificadores, utilizar links entre itens ou, quando a escala, a criticidade ou a necessidade de auditoria justificarem, recorrer a matrizes e repositórios especializados. A utilidade da rastreabilidade depende menos da quantidade de vínculos registrados do que da capacidade de responder às perguntas relevantes sem transferir o entendimento para a memória exclusiva das pessoas.

Criticidade, regulação, quantidade de equipes, distribuição dos participantes, necessidade de previsibilidade e impacto da perda de memória modificam a quantidade de estrutura e formalidade necessária. Produtos pequenos e de baixo risco podem funcionar adequadamente com poucos atributos e relações locais, enquanto ambientes regulados, distribuídos ou de longa duração podem exigir versões, baselines, evidências, aprovações e rastreabilidade mais explícita. Essa calibragem não estabelece uma escala de maturidade em que maior formalidade seja necessariamente melhor; busca apenas compatibilizar o esforço de organização e atualização com a informação que precisa continuar compreensível e recuperável.

No exemplo do sistema de agendamento de consultas públicas, a decisão de incorporar uma **lista de espera** mostrou como uma aprendizagem pode ampliar o escopo de requisitos em vez de apenas modificar algo que já existia. Novos comportamentos passaram a se relacionar com consulta de disponibilidade, notificações, regras de ordenação e prazo, critérios de aceitação, requisitos não funcionais, representações e entregas, enquanto elementos anteriores que continuavam válidos foram preservados. O exemplo evidenciou que atualizar o escopo significa incorporar aquilo que passou a fazer parte do entendimento sobre o produto sem alterar indiscriminadamente todo o conjunto.

Quando esse cuidado é substituído por preenchimento de campos, acúmulo de registros, ferramentas tratadas como solução ou vínculos que já não representam relações válidas, a organização pode produzir aparência de controle sem preservar entendimento. O equilíbrio desenvolvido ao longo do capítulo consiste em manter estrutura suficiente para comunicar, decidir, recuperar memória e compreender impactos, sem transformar essa estrutura em burocracia. Nessa perspectiva, Organização e Atualização expressa de forma particularmente clara a natureza sociotécnica da Engenharia de Requisitos: o escopo não é apenas um conjunto técnico de registros, mas uma referência compartilhada que precisa acompanhar conversas, decisões, aprendizagem e evolução do produto.



## CAPÍTULO 12: PONTOS-CHAVE

- **Organização e Atualização** é a atividade responsável por manter o escopo de requisitos do produto estruturado e coerente com o entendimento vigente, incorporando os efeitos de novas descobertas, decisões, evidências e mudanças de contexto.
- **O escopo de requisitos do produto** corresponde ao conjunto organizado de requisitos que expressa aquilo que, em determinado momento, se entende que o produto precisa atender, associado às informações e relações necessárias para compreender sua origem, significado, condições e relações com o restante do produto.
- **Escopo do produto, escopo de requisitos do produto, escopo da entrega e escopo do projeto** são conceitos relacionados, mas distintos. O primeiro delimita aquilo que o produto pretende oferecer; o segundo explicita o que se entende que ele precisa atender; o terceiro seleciona parte desse entendimento para determinado horizonte; e o quarto delimita o trabalho necessário para conduzir a iniciativa.
- **Backlogs, mapas de histórias, roadmaps, quadros de fluxo, áreas de requisitos, baselines e outras estruturas não constituem novos tipos de escopo.** Eles oferecem diferentes formas de organizar, visualizar ou operacionalizar partes dos escopos e do trabalho relacionado ao produto.
- Organizar requisitos significa tornar o escopo **localizável, compreensível e navegável**, utilizando identificadores, atributos, agrupamentos, visões e relações conforme a necessidade do contexto. O objetivo não é produzir uma estrutura única ou universal, mas permitir que as pessoas encontrem e relacionem as informações necessárias às suas decisões.
- Informações complementares não precisam ser obrigatoriamente transformadas em registros independentes. Regras, critérios, restrições e outras informações podem permanecer reunidas com o requisito ou com o artefato que o declara quando isso favorece a compreensão; vínculos explícitos ganham importância quando essas informações precisam ser compartilhadas, reutilizadas ou recuperadas separadamente.
- Atualizar requisitos significa fazer com que o escopo organizado **acompanhe aquilo que passou a vigorar**. Isso pode envolver introduzir novos requisitos, modificar informações existentes, retirar elementos obsoletos, rever relações, atualizar atributos, incorporar novas versões e preservar histórico quando necessário.
- Decisões de **prioridade, refinamento, inclusão, remoção ou transformação de requisitos** podem surgir de outras atividades da Engenharia de Requisitos. Na Organização e Atualização, essas decisões precisam ser refletidas nas estruturas, relações e visões utilizadas para manter o escopo coerente com o entendimento vigente.
- Dar visibilidade, organizar por relações úteis, manter prioridades acordadas visíveis, refletir refinamentos, incorporar mudanças, limpar o escopo e preservar memória suficiente são **movimentos recorrentes de cuidado**, e não etapas sucessivas de um processo.
- A **rastreabilidade** pode ser compreendida como memória suficiente: preservar as relações necessárias para compreender origem, condicionantes, dependências, decisões, evidências, mudanças e impactos sem registrar indiscriminadamente todos os vínculos possíveis.

- Rastreabilidade não implica necessariamente matrizes ou ferramentas especializadas. Informações reunidas em um mesmo artefato, referências, identificadores, atributos e links podem ser suficientes em determinados contextos; relações mais formais tornam-se necessárias quando escala, criticidade, regulação ou necessidade de auditoria justificam esse esforço.
- Os mecanismos de organização e rastreabilidade precisam permanecer **atualizados juntamente com o escopo**. Um vínculo formalmente existente, mas apontando para uma regra, versão ou decisão que deixou de vigorar, pode produzir mais confusão do que a ausência do vínculo.
- A quantidade de estrutura necessária depende de fatores como **criticidade, regulação, número de equipes, distribuição dos participantes, necessidade de previsibilidade e impacto da perda de memória**. Maior formalidade não representa necessariamente maior maturidade; representa uma resposta possível às características do contexto.
- O exemplo da **lista de espera no sistema de agendamento de consultas públicas** mostrou que atualizar o escopo também pode significar ampliá-lo. Novos comportamentos podem introduzir regras, critérios, relações, representações e necessidades de entrega, enquanto elementos anteriores que continuam válidos são preservados.
- Organização e Atualização perde sua finalidade quando se transforma em **burocracia, acúmulo de registros, fragmentação entre ferramentas ou manutenção de vínculos sem significado atual**. A estrutura deve existir para apoiar compreensão, comunicação, decisão e evolução do produto.
- Sob uma perspectiva sociotécnica, organizar e atualizar requisitos significa preservar uma **referência compartilhada sobre o produto**, permitindo que decisões, aprendizagens e relações continuem compreensíveis mesmo quando o contexto muda, novas pessoas participam e o produto evolui.

## 13 BIBLIOGRAFIA

- AGILE BUSINESS CONSORTIUM. DSDM Agile Project Framework Handbook. London: Agile Business Consortium, 2021.
- AGUIAR, F.; CAROLI, P. Product Backlog Building - Um guia prático para criação e refinamento de backlog para produtos de sucesso. 1. ed. Rio de Janeiro: Editora Caroli, 2021.
- AHMAD, M. O. et al. Kanban in software engineering: A systematic mapping study. *Journal of Systems and Software*, v. 137, p. 96-113, 2018.
- ALEXANDER, I.; BEUS-DUKIC, L. *Discovering Requirements: How to Specify Products and Services*. Hoboken: Wiley, 2009.
- ALMEIDA, R. et al. Understanding the value of a Feature-Driven Development (FDD) in a company's software development process. In: *IEEE 19th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, p. 497-502, 2019.
- AMBLER, S. W. *The Agile Unified Process v1.1*. Ambysoft Inc, 2005.
- AMBLER, S. W. *The Elements of UML 2.0 Style*. Cambridge University Press, 2016.
- AMBLER, S. W.; LINES, M. *Disciplined Agile Delivery: A Practitioner's Guide to Agile Software Delivery in the Enterprise*. IBM Press, 2012.
- ANDERSON, D. J. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- ANDERSON, D. J.; CARMICHAEL, A. *Essential Kanban Condensed*. Lean Kanban University Press, 2016.
- ARORA, C. et al. Change-based requirements traceability: The case of natural language requirements and use case models. In: *IEEE 25th International Requirements Engineering Conference (RE)*, p. 246-251, 2017.
- BALAJI, S.; MURUGAIYAN, M. S. Waterfall vs. V-Model vs. Agile: A comparative study on SDLC. *International Journal of Information Technology and Business Management*, v. 2, n. 1, p. 26-30, 2012.
- BALDUINO, R. *Introduction to OpenUP (Open Unified Process)*. Eclipse Foundation, 2007.
- BASS, J. M.; WEBER, I.; ZHU, L. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional, 2015.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 3. ed. Addison-Wesley, 2012.
- BATRA, D.; XIA, W.; ZHANG, M. Collaboration in agile software development: Concept and dimensions. *Communications of the Association for Information Systems*, v. 41, n. 1, p. 429-449, 2017.

- BECK, K. et al. Manifesto for Agile Software Development. 2001. Disponível em: <http://agile-manifesto.org/>.
- BECK, K.; ANDRES, C. Extreme Programming Explained: Embrace Change. 2. ed. Addison-Wesley Professional, 2004.
- BERGER, H.; BEYNON-DAVIES, P. The utility of rapid application development in large-scale, complex projects. *Information Systems Journal*, v. 19, n. 6, p. 549-570, 2009.
- BERRY, D. M. The Philosophy of Software: Code and Mediation in the Digital Age. 2. ed. Palgrave Macmillan, 2019.
- BEYNON-DAVIES, P. et al. Rapid application development (RAD): an empirical review. *European Journal of Information Systems*, v. 8, n. 3, p. 211-223, 1999.
- BLACK, R.; VEENENDAAL, E. V.; GRAHAM, D. Foundations of Software Testing: ISTQB Certification. 4. ed. Cengage Learning, 2019.
- BOEHM, B. W. A Spiral Model of Software Development and Enhancement. *IEEE Computer*, v. 21, n. 5, p. 61-72, 1988.
- BOEHM, B. W. Spiral development: Experience, principles, and refinements. Special Report CMU/SEI-2000-SR-008, 2000.
- BOEHM, B. W.; TURNER, R. Balancing Agility and Discipline: A Guide for the Perplexed. Addison-Wesley Professional, 2003.
- BROWN, T. Change by Design: How Design Thinking Transforms Organizations and Inspires Innovation. Harper Business, 2009.
- BURROWS, M. Kanban from the Inside. Blue Hole Press, 2014.
- CAROLI, P. Lean Inception: Como Alinhar Pessoas e Construir o Produto Certo. São Paulo: Caroli.org, 2021.
- CARVALHO, A. E. S. Engenharia de Requisitos: Software Orientado ao Negócio. São Paulo: Brasport, 2003.
- CHECKLAND, P. B. Systems Thinking, Systems Practice. Chichester: John Wiley & Sons, 1981.
- CHEN, J. A study on the application of feature-driven development method in software process. In: IEEE 9th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA), p. 311-313, 2017.
- CHUNG, L. et al. Non-Functional Requirements in Software Engineering. Springer, 2000.
- CLARK, H. H.; BRENNAN, S. E. Grounding in communication. In: RESNICK, L. et al. Perspectives on Socially Shared Cognition. APA, 1991.
- COCKBURN, A. Writing Effective Use Cases. Addison-Wesley Professional, 2001.

- COCKBURN, A.; HIGHSMITH, J. Agile Software Development: The People Factor. IEEE Computer, v. 34, n. 11, p. 131-133, 2001.
- COCKBURN, A.; WILLIAMS, L. The costs and benefits of pair programming. In: Extreme Programming Examined, p. 223-243, 2001.
- COHN, M. Agile Estimating and Planning. Upper Saddle River, NJ: Prentice Hall, 2006.
- COHN, M. Succeeding with Agile: Software Development Using Scrum. Addison-Wesley Professional, 2010.
- COHN, M. User Stories Applied: For Agile Software Development. Addison-Wesley Professional, 2004.
- CONBOY, K.; FITZGERALD, B. Method and developer characteristics for effective agile method tailoring. ACM Transactions on Software Engineering and Methodology, v. 20, n. 1, p. 1-30, 2010.
- COOPER, A.; REIMANN, R.; CRONIN, D. About Face: The Essentials of Interaction Design. 4. ed. Wiley, 2014.
- CRISPIN, L.; GREGORY, J. Agile Testing: A Practical Guide for Testers and Agile Teams. Addison-Wesley Professional, 2009.
- CROWDER, J. A.; HOFF, C. W. Requirements Engineering: Laying a Firm Foundation. Springer Nature Switzerland AG, 2022.
- DAGNINO, A. Increasing the effectiveness of software development process improvement through software project telemetry. In: IEEE 39th Annual Computer Software and Applications Conference, v. 2, p. 765-770, 2014.
- DALY, J. K. Rapid Application Development Techniques. In: The Art of Software Modeling, p. 285-305. CRC Press, 2014.
- DANEVA, M. et al. Agile requirements engineering practices: an empirical study. Information and Software Technology, v. 137, p. 106603, 2021.
- DAVIS, A. M.; BERSOFF, E. H.; COMER, E. R. A strategy for comparing alternative software development life cycle models. IEEE Transactions on Software Engineering, v. 14, n. 10, p. 1453-1461, 1988. DOI: 10.1109/32.6190.
- DICK, J.; HULL, E.; JACKSON, K. Requirements Engineering. 4. ed. Cham: Springer, 2017.
- DIKERT, K.; PAASIVAARA, M.; LASSENIUS, C. Challenges and success factors for large-scale agile transformations. Journal of Systems and Software, v. 119, p. 87-108, 2016.
- DINGSØYR, T. et al. A decade of agile methodologies: Towards explaining agile software development. Journal of Systems and Software, v. 85, n. 6, p. 1213-1221, 2012.
- DINGSØYR, T. et al. Exploring software development at the very large-scale. Empirical Software Engineering, v. 23, n. 1, p. 490-520, 2018.

- DYBÅ, T.; DINGSØYR, T. Empirical studies of agile software development: A systematic review. *Information and Software Technology*, v. 50, n. 9-10, p. 833-859, 2008.
- EASTERBROOK, S. *Requirements Engineering*. London: Springer-Verlag, 2004.
- EVANS, E. *Domain-Driven Design*. Addison-Wesley Professional, 2003.
- FERNANDES, J. M.; MACHADO, R. J. *Requirements in Engineering Projects*. Cham: Springer, 2015.
- FERRARI, A.; ESULI, A.; GERVASI, V. NL-based requirements quality analysis: an empirical study. *Requirements Engineering*, v. 25, p. 379-397, 2020.
- FOWLER, M.; HIGHSMITH, J. The agile manifesto. *Software Development*, v. 9, n. 8, p. 28-35, 2001.
- FREEDMAN, D. P.; WEINBERG, G. M. *Handbook of Walkthroughs, Inspections, and Technical Reviews*. 3. ed. New York: Dorset House, 1990.
- GAROUSHI, V. et al. A survey of software testing practices in Canada. *Journal of Systems and Software*, v. 86, n. 5, p. 1354-1376, 2018.
- GARTNER. *Bimodal IT: How to Be Digitally Agile Without Making a Mess*. Gartner Research Report, 2014.
- GILB, T.; GRAHAM, D. *Software Inspection*. Reading: Addison-Wesley, 1993.
- GLASS, R. L. Agile versus traditional: Make love, not war! *Cutter IT Journal*, v. 14, n. 12, p. 12-18, 2001.
- GOGUEN, J. A. Requirements Engineering as a Social Process. In: *Proceedings of the IEEE International Symposium on Requirements Engineering*, 1994.
- GOTTESDIENER, E. *Requirements by Collaboration: Workshops for Defining Needs*. Boston: Addison-Wesley, 2002.
- GOTTESDIENER, E.; GORMAN, M. *Discover to Deliver: Agile Product Planning and Analysis*. EBG Consulting, 2012.
- GRADY, R. B.; CASWELL, D. L. *Software Metrics: Establishing a Company-Wide Program*. Englewood Cliffs, NJ: Prentice Hall, 1987.
- HAMMARBERG, M.; SUNDEN, J. *Kanban in Action*. Manning Publications, 2014.
- HAREL, D. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, v. 8, n. 3, p. 231-274, 1987.
- HIGHSMITH, J. *Adaptive Software Development*. Addison-Wesley Professional, 2013.
- HNEIF, M.; HOCK OW, S. Review of agile methodologies in software development. *International Journal of Research and Reviews in Applied Sciences*, v. 1, n. 1, p. 1-8, 2019.

- HORKOFF, J. Interactive goal modeling for requirements engineering. *Requirements Engineering*, v. 24, p. 81-102, 2019.
- HOSSEINI, M. et al. Crowdsourcing for requirements engineering: a systematic mapping study. *Requirements Engineering*, v. 27, p. 193-212, 2022.
- HRUSCHKA, P. et al. Handbook RE@Agile: Education and Training for IREB Certified Professional for Requirements Engineering. Version 2.0.0. IREB, 2022.
- HULL, E.; JACKSON, K.; DICK, J. *Requirements Engineering*. 3. ed. London: Springer, 2011.
- IEEE COMPUTER SOCIETY. Guide to the Software Engineering Body of Knowledge – SWE-BOK v4.0. IEEE, 2024.
- IEEE. ISO/IEC/IEEE 15288:2023 Systems and software engineering - Life cycle processes. IEEE Standards Association, 2023.
- IEEE. ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering. IEEE Standards Association, 2018.
- IIVARI, J.; IIVARI, N. The relationship between organizational culture and the deployment of agile methods. *Information and Software Technology*, v. 53, n. 5, p. 509-520, 2011.
- INAYAT, I. et al. A systematic literature review on agile requirements engineering practices and challenges. *Computers in Human Behavior*, v. 51, p. 915-929, 2015.
- IQBAL, T. et al. Hybrid software development methodologies for quality development and industry collaborations. In: *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, p. 437-446, 2022.
- IREB (International Requirements Engineering Board). Handbook for the CPRE Foundation Level, version 1.1.0, 2022.
- ISO/IEC. ISO/IEC 25010:2011 Systems and software engineering --- Quality Requirements and Evaluation (SQuaRE), 2011.
- JACKSON, M. *Problem Frames: Analyzing and Structuring Software Development Problems*. Addison-Wesley, 2000.
- JACOBSON, I. et al. *The Unified Software Development Process*. Reading, Mass.: Addison-Wesley, 1999.
- JACOBSON, I.; SPENCE, I.; DE MENDONCA, K. *Use-Case 3.0: The Definitive Guide – Refreshed*. Ivar Jacobson International, 2024.
- JEFFRIES, R.; ANDERSON, A.; HENDRICKSON, C. *Extreme Programming Installed*. Addison-Wesley Professional, 2001.
- JONES, C. *Estimating Software Costs*. 2. ed. McGraw-Hill Education, 2007.
- KASURINEN, J.; TAIPALE, O.; SMOLANDER, K. Software test automation in practice: empirical observations. *Advances in Software Engineering*, v. 2010, p. 1-18, 2016.

- KAWANO, K. et al. From requirements to workflow application development: a feature-driven development approach. In: IEEE 23rd Asia-Pacific Software Engineering Conference (APSEC), p. 57-64, 2016.
- KEOGH, M. BDD in Action. 2. ed. Manning Publications, 2019.
- KHAN, K.; KHAN, S. Software Requirements Engineering: A Review. International Journal of Computer Science and Network Security, v. 19, n. 11, p. 385-391, 2019.
- KHAN, M. E. et al. Addressing the challenges of application security through a spiral model. International Journal of Engineering Research e Technology, v. 3, n. 4, p. 990-996, 2014.
- KHAN, M. E.; KHAN, F. Empirical study of SDLC and its requirement engineering process models. International Journal of Advanced Research in Computer Science, v. 10, n. 3, p. 54-59, 2019.
- KHAN, S. A.; QURASHI, M. A.; KHAN, S. U. A systematic literature review on DSDM from its inception to present. International Journal of Information Engineering and Electronic Business, v. 12, n. 1, p. 24-45, 2020.
- KLEPPE, A.; WARMER, J.; BAST, W. MDA Explained: The Model Driven Architecture. Boston: Addison-Wesley, 2003.
- KNAPP, J.; ZERATSKY, J.; KOWITZ, B. Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days. Simon e Schuster, 2016.
- KOTONYA, G.; SOMMERVILLE, I. Requirements Engineering: Processes and Techniques. Chichester, Inglaterra: John Wiley e Sons, 1998.
- KROLL, P.; KRUCHTEN, P. The Rational Unified Process Made Easy. Addison-Wesley Professional, 2003.
- KROLL, P.; MACISAAC, B. Agility and Discipline Made Easy: Practices from OpenUP and RUP. Addison-Wesley Professional, 2006.
- KRUCHTEN, P. Adaptive Software Architecture. 2. ed. Addison-Wesley Professional, 2023.
- KRUCHTEN, P. Contextualizing agile software development. Journal of Software: Evolution and Process, v. 25, n. 4, p. 351-361, 2013.
- KRUCHTEN, P. The Rational Unified Process — An Introduction. 3. ed. Addison-Wesley Professional, 2004.
- KUHRMANN, M. et al. Hybrid software and system development in practice: Waterfall, Scrum, and beyond. In: INTERNATIONAL CONFERENCE ON SOFTWARE AND SYSTEM PROCESS (ICSSP), 2017. Proceedings [...]. 2017. DOI: 10.1145/3084100.3084104.
- KUHRMANN, M. et al. Hybrid software development approaches in practice: A European perspective. IEEE Software, v. 36, n. 4, p. 20-31, 2019.
- KURTANOVIĆ, Z.; MAALEJ, W. Mining user feedback from app stores. Empirical Software Engineering, v. 23, n. 3, p. 425-466, 2018.



- LAPLANTE, P. A. Requirements Engineering for Software and Systems. 3. ed. CRC Press, 2017.
- LAPLANTE, P. A. What Every Engineer Should Know about Software Engineering. CRC Press, 2007.
- LAPLANTE, P. A.; KASSAB, M. H. Requirements Engineering for Software and Systems. 4. ed. CRC Press, 2022.
- LARMAN, C. Agile and Iterative Development: A Manager's Guide. Addison-Wesley Professional, 2004.
- LARMAN, C.; VODDE, B. Large-Scale Scrum: More with LeSS. Addison-Wesley Professional, 2016.
- LAUESEN, S. Software Requirements: Styles and Techniques. Boston: Addison-Wesley, 2002.
- LEFFINGWELL, D. Agile Software Requirements. Addison-Wesley Professional, 2011.
- LEFFINGWELL, D. et al. SAgile® Reference Guide: Scaled Agile Framework® for Lean Enterprises. 4. ed. Scaled Agile, Inc., 2017.
- LEFFINGWELL, D. SAgile® 5.0 Distilled. Addison-Wesley Professional, 2020.
- LEFFINGWELL, D. Scaling Software Agility. Addison-Wesley Professional, 2021.
- LEMBERGER, P.; MOREL, M. Software Modeling and Testing in the V-Model. In: A Visual Approach to Software Design, p. 37-59. Springer, 2022.
- LEOPOLD, K.; KALTENECKER, S. Kanban Change Leadership. John Wiley e Sons, 2015.
- LESS WORKS. Less: Desenvolvimento de Software em Grande Escala. 2024. Disponível em: <https://less.works>.
- LINDSTROM, L.; JEFFRIES, R. Extreme programming and agile software development methodologies. Information Systems Management, v. 21, n. 3, p. 41-52, 2004.
- LINKER, Sheldon; BROWN, Michael Scott. *Requirements Engineering: A Modern Approach to the Requirements Engineering Body of Knowledge*. 1. ed., 2021.
- LYCETT, M. et al. Migrating agile methods to standardized development practice. Computer, v. 36, n. 6, p. 79-85, 2003.
- MAALEJ, W. et al. Machine learning for requirements engineering: A state-of-the-art review. In: IEEE 29th International Requirements Engineering Conference (RE), p. 54-65, 2021.
- MACKAY, H. et al. Reconfiguring the user: using rapid application development. Social Studies of Science, v. 30, n. 5, p. 737-757, 2000.
- MAIDEN, N.; RUGG, G. ACRE: Selecting Methods for Requirements Acquisition. Software Engineering Journal, v. 11, n. 3, p. 196-209, 1996.

- MAIER, A.; MOSER, T. Trade-offs in modern software development. *IEEE Software*, v. 38, n. 4, p. 85-91, 2021.
- MANER, W. Rapid application development using iterative prototyping. Retrieved March, v. 11, p. 2012, 1997.
- MARTIN, J. *Rapid Application Development*. Macmillan Publishing Co., 1991.
- MATHARU, G. S. et al. Empirical study of agile software development methodologies. *ACM SIGSOFT Software Engineering Notes*, v. 40, n. 1, p. 1-6, 2015.
- MATHEW, G. Enhancing requirements engineering with AI. In: *IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, p. 626-633, 2017.
- MATHEW, J. *Agile Requirements: Managing Requirements in Scrum Framework*. Edição Kindle, 2017.
- MATHUR, S.; MALIK, S. Advancements in the V-Model. *International Journal of Computer Applications*, v. 1, n. 12, p. 29-34, 2016.
- MEYER, B. Design by Contract. *IEEE Computer*, v. 25, n. 10, p. 40-51, 1992.
- MEYER, B. The Grand Challenge of Trusted Components. In: *Proc. ICSE 25*. IEEE Press, 2003.
- MUNASSAR, N. M. A.; GOVARDHAN, A. A comparison between five models of software engineering. *International Journal of Computer Science Issues (IJCSI)*, v. 7, n. 5, p. 94-101, 2010.
- NAUR, P.; RANDELL, B. (Eds.). *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee*. Garmisch, Germany, 1969.
- NERUR, S.; MAHAPATRA, R.; MANGALARAJ, G. Challenges of migrating to agile methodologies. *Communications of the ACM*, v. 48, n. 5, p. 72-78, 2005.
- NUSEIBEH, B.; EASTERBROOK, S. Requirements Engineering: A Roadmap. In: *Proceedings of the Conference on the Future of Software Engineering*. ACM Press, 2000. p. 35-46.
- OTHMAN, H. B. et al. A comparative study on software development methodologies. In: *World Congress on Computer Applications and Information Systems (WCCAIS)*, p. 1-6, 2014.
- PAASIVAARA, M. Adopting SAFe to scale agile in a globally distributed organization. In: *IEEE 12th International Conference on Global Software Engineering (ICGSE)*, p. 36-40, 2017.
- PAASIVAARA, M.; BEHM, B.; LASSENIUS, C. Large-scale agile transformation at Ericsson: a case study. *Empirical Software Engineering*, v. 23, n. 5, p. 2550-2596, 2018.
- PAASIVAARA, M.; LASSENIUS, C. Scaling scrum in a large globally distributed organization. In: *IEEE/ACM 41st International Conference on Software Engineering (ICSE-SEIP)*, p. 263-272, 2019.
- PALMER, S. R.; FELSING, J. M. *A Practical Guide to Feature-Driven Development*. Prentice Hall, 2002.

- PATTON, J. User Story Mapping. O'Reilly Media, 2014.
- PETERSEN, K.; WOHLIN, C.; BACA, D. The Waterfall Model in Large-Scale Development. In: 10th International Conference on Product-Focused Software Process Improvement, p. 386-400, 2009.
- PICHLER, R. Agile Product Management with Scrum. Addison-Wesley Professional, 2010.
- POHL, K. Requirements Engineering: Fundamentals, Principles, and Techniques. Cham: Springer, 2010.
- PRESSMAN, R. S.; MAXIM, B. R. Software Engineering: A Practitioner's Approach. 9. ed. McGraw-Hill Education, 2021.
- PRIES, K. H.; QUIGLEY, J. M. Scrum Project Management. CRC Press, 2010.
- PROJECT MANAGEMENT INSTITUTE. A Guide to the Project Management Body of Knowledge (PMBOK Guide). 6. ed. PMI, 2017.
- PROJECT MANAGEMENT INSTITUTE. Agile Practice Guide. Newtown Square, PA: Project Management Institute, 2017.
- QUMER, A.; HENDERSON-SELLERS, B. An evaluation of the degree of agility in six agile methods. Information and Software Technology, v. 50, n. 4, p. 280-295, 2008.
- RAMESH, B.; CAO, L.; BASKERVILLE, R. Agile requirements engineering practices and challenges. Information Systems Journal, v. 20, n. 5, p. 449-480, 2010.
- RIGBY, D. K.; SUTHERLAND, J.; TAKEUCHI, H. Embracing Agile. Harvard Business Review, v. 94, n. 5, p. 40-50, May 2016.
- ROBERTSON, J.; ROBERTSON, S. Mastering the Requirements Process. 3. ed. Boston: Addison-Wesley, 2014.
- ROYCE, W. W. Managing the Development of Large Software Systems. Proceedings of IEEE WESCON, v. 26, n. 8, p. 1-9, August 1970.
- RUBIN, K. S. Essential Scrum: A Practical Guide to the Most Popular Agile Process. 2. ed. Addison-Wesley Professional, 2022.
- RUPARELIA, N. B. Software development lifecycle models. ACM SIGSOFT Software Engineering Notes, v. 35, n. 3, p. 8-13, 2010.
- SANTOS, A. C. M.; COLAÇO JÚNIOR, M.; ANDRADE, E. C. Multimedia Resources as a Support for Requirements Engineering. Journal of Software: Evolution and Process, v. 36, e2327, 2020.
- SANTOS, J. et al. Expectations and perceptions of Scrum practitioners when scaling Scrum. In: Proceedings of the 35th Annual ACM Symposium on Applied Computing, Brno, p. 1280-1289, 2020.

- SCALED AGILE. SAFe 6.0 Framework. 2023. Disponível em: <https://www.scaledagileframework.com/>.
- SCHEIN, E. H. Organizational Culture and Leadership. 5. ed. Wiley, 2016.
- SCHNEIDER, S. The B Method. Houndmills, Reino Unido: Palgrave Macmillan, 2001.
- SCHÖN, E. M.; THOMASCHEWSKI, J.; ESCALONA, M. J. Agile requirements engineering: a systematic literature review. Computer Standards e Interfaces, v. 49, p. 79-91, 2017.
- SCHWABER, K. Agile Project Management with Scrum. Redmond, WA: Microsoft Press, 2004.
- SCHWABER, K.; SUTHERLAND, J. The Scrum Guide: The Definitive Guide to Scrum. 2020.
- SHUJA, A. K.; KREBS, J. IBM Rational Unified Process Reference and Certification Guide. IBM Press, 2008.
- SMART, J. F. BDD in Action. Manning Publications, 2014.
- SOMMERVILLE, I. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.
- STANDISH GROUP. CHAOS Report 2020: Beyond Infinity. Standish Group International, 2020.
- SUTCLIFFE, A. G. User-Centred Requirements Engineering. London: Springer, 2002.
- SUTHERLAND, J. Scrum: The Art of Doing Twice the Work in Half the Time. Crown Business, 2020.
- THAKUR, S.; KAUR, A. Role of Agile methodology in software development. International Journal of Computer Applications, v. 166, n. 10, p. 7-10, 2019.
- THEOCHARIS, G. et al. A survey on software development processes in small and medium-sized enterprises. In: IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C), p. 617-618, 2015.
- TURETKEN, O.; STOJANOV, I.; TRIENEKENS, J. J. M. Assessing the adoption level of scaled agile development. Journal of Software: Evolution and Process, v. 29, n. 6, e1796, 2017.
- TURLEY, F. The DSDM Agile Project Framework. In: Handbook of Project Management Methodologies. Routledge, 2018.
- VAN LAMSWEERDE, A. Requirements Engineering: From System Goals to UML Models to Software Specifications. Wiley, 2009.
- WAGNER, S. et al. Status quo in requirements engineering: A theory and a global family of surveys. ACM Transactions on Software Engineering and Methodology, v. 28, n. 2, p. 1-48, 2019.
- WALID, A.; AMRI, S. Modified V-model to improve embedded electronic designs. In: IEEE International Conference on Design e Test of Integrated Micro e Nano-Systems (DTS), p. 1-5, 2020.

- WARMER, J.; KLEPPE, A. The Object Constraint Language: Getting your models ready for MDA. Boston: Addison-Wesley, 2003.
- WELLS, C. The DSDM Agile Project Framework Handbook. Springer, 2020.
- WELLS, D. Extreme Programming: A gentle introduction. 2013. Disponível em: <http://www.extremeprogramming.org/>.
- WEST, D.; GRANT, T. Agile development: Mainstream adoption has changed agility. Forrester Research, v. 2, n. 1, p. 41-56, 2010.
- WIEGERS, K. E.; BEATTY, J. Software Requirements. 3. ed. Microsoft Press, 2013.
- WIEGERS, K.; HOKANSON, C. Software Requirements Essentials. Pearson/Addison-Wesley, 2023.
- WILLIAMS, L. Agile software development methodologies and practices. Advances in Computers, v. 80, p. 1-44, 2010.
- WILLIAMS, L. et al. Toward a framework for evaluating extreme programming. In: Proceedings of the 8th International Conference on Evaluation and Assessment in Software Engineering, p. 11-20, 2004.
- WINOGRAD, T.; FLORES, F. Understanding Computers and Cognition. Ablex, 1986.
- WOODCOCK, J.; DAVIES, J. Using Z: Specification, Refinement, and Proof. Hemel Hempstead, UK: Prentice Hall, 1996.
- WORDSWORTH, J. Software Engineering with B. Wokingham: Addison-Wesley, 1996.
- YOUNG, R. R. The Requirements Engineering Handbook. Norwood, MA: Artech House, 2004.
- ZAVE, P. Classification of Research Efforts in Requirements Engineering. ACM Computing Surveys, v. 29, n. 4, p. 315-321, 1997.
- ZHANG, Y.; JARZABEK, S. A case study on verification of function points. Information and Software Technology, v. 94, p. 1-13, 2018.
- ZHAO, L.; XU, L.; YANG, F. AI for requirements engineering: opportunities and challenges. IEEE Software, v. 39, n. 5, p. 34-42, 2022.